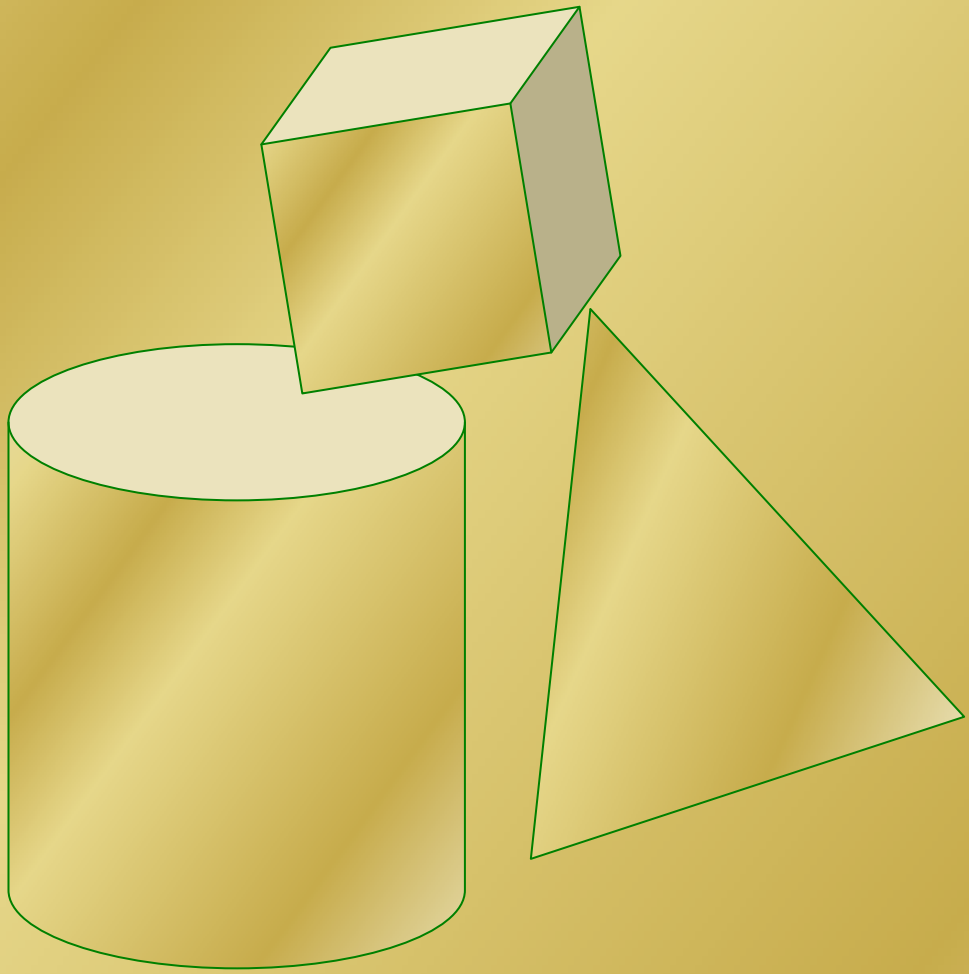


Experto Bucarell y

Hijo mío, no te olvides de mi ley, y tu corazón guarde mis mandamientos; Porque largura de días y años de vida y paz te aumentarán. Nunca se aparte de ti la misericordia y la verdad; Átalas a tu cuello, escríbelas en la tabla de tu corazón; Y hallarás gracia y buena opinión ante los ojos de Dios y de los hombres. **Proverbios 3:1-4.**



2da. Edición
IDCCLC

Visual BASIC 6.0
Orientado a bases de datos
Todos los derechos reservados
2005-2008

Orientado a bases de datos
Desarrollando Grupo Experto Bucarely

Segunda edición

DCLC
(Distribución del conocimiento libre de costo)

VISUAL BASIC 6.0

Orientado a Bases de Datos

Desarrollando Grupo Experto Bucarely

Segunda edición

CARLOS M. RODRIGUEZ BUCARELLY

Autor de este libro
Maestro actual del INFOTEP
Ing. en Sistemas

PABLO A. RODRIGUEZ BUCARELLY

Coautor de este libro
Encargado del departamento de monitoreo Banco Popular
Ing. en Sistemas

Diseño de páginas

CLARIBEL PIÑEYRO TORRES

Licda. En Mercadeo

Revisión técnica

CARLOS A. MOREL PICHARDO

Lic. en Informática

Impresor

EDGAR HILARIO SANCHEZ

Lic. en Informática



Agradecimientos



© 1998-2008

Mis más sincero agradecimiento a mi padre **DIOS**, ser supremo, creador del universo y todas las cosas dentro de él, cuya gloria sea toda para él por los siglos de los siglos, Amén.

Infinitas gracias a todos los lectores de la primera edición de este libro que gracias a ellos he obtenido la inspiración necesaria para poder escribir esta segunda edición.

Los siguientes revisores aportaron comentarios y sugerencias cuidadosas para el mejoramiento del manuscrito de esta edición: **Ing. Carlos A. Morel Pichardo, Adm. Nelson M. Rodríguez Bucarelly, Ing. Pablo A. Rodríguez Bucarelly, Ing. Starky H. Jáquez Medina y los lectores de la primera edición.**

Un agradecimiento muy especial a mi compañero de trabajo **Lic. Juan Francisco Mena Mañón**, ya que ha sido desde un principio un apoyo incondicional para las elaboraciones de mis proyectos, además de considerarlo como un padre para mí.



En el presente apartado se especifica la forma de distribuir este libro y los derechos propios del autor del libro.

Derechos del Autor

Este libro es exclusivamente una obra del **Ing. Carlos Manuel Rodríguez Bucarely** y no puede ser alterado ni modificado bajo ninguna condición sin antes haber sido consultado con el mismo.

Derechos del lector y distribuidor

- Este libro puede ser utilizado en cualquier institución educativa (colegios, universidades, institutos, politécnicos, internados, etc.).
- No se permite ventas de este libro para obtener beneficios económicos.
- Se debe considerar el nombre del autor en la bibliografía de cualquier manuscrito si se copia alguna porción de texto de este libro.
- El libro se distribuye de forma digital pero puede ser impreso en hojas de papel.
- Las distribuciones impresas en hojas de papel deben tener a totalidad el número de hojas obtenidas en el documento en su formato digital, incluyendo la portada del libro.

Responsabilidad del Autor

- El autor no se hace responsable del mal uso del conocimiento obtenido por este libro.
- El autor no se hace responsable de las sanciones aplicadas por la violación de los derechos del autor.

CONTENIDO REDUCIDO

Prólogo

X

Introducción a la programación
orientada a objetos

5-17

El EID (Entorno Integrado de
desarrollo) de Visual Basic 6.0

18-37

Introducción a los controles más
usuales de Visual Basic 6.0

38-218

Introducción al Lenguaje Basic

219-293

Los menús

294-304

La interfaz de usuario

305-359

Los Archivos

360-401

Introducción a las Bases de datos

402-436

Trucos de la red

437-442

Anexos y Bibliografía

443-445



2008

Visual Basic 6.0

**Prólogo**

CAPÍTULO I: INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS	5
1.1 Concepto de programación orientada a objetos (POO).....	6
1.1 Programas orientados a eventos.....	6
1.2 Creación de programas para el entorno de Windows.....	7
1.4 Elementos de una aplicación en el entorno de Windows.	7
- 1.4.1 Formularios y controles.....	7
- 1.4.2 Nombre de controles y objetos.....	8
- 1.4.3 Propiedades, clase y estado de objetos.....	8
- 1.4.4 Modo de diseño y Modo de ejecución.....	9
- 1.4.5 Procedimientos.....	10
- 1.4.6 Eventos.....	11
- 1.4.7 Métodos.....	12
- 1.4.8 Módulos.....	13
- 1.4.9 Encapsulación, herencia, polimorfismo y mensajes en objetos.....	14
1.5 Preguntas y ejercicios propuestos.....	15
CAPÍTULO II: EL EID (ENTORNO INTEGRADO DE DESARROLLO) DE VISUAL BASIC 6.0	18
2.1 ¿Qué es Visual Basic?.....	19
2.2 Historia y versiones de Visual Basic.....	19
2.3 El EID (Entorno Integrado de Desarrollo) de Visual Basic 6.0.....	20
- 2.3.1 La barra de menús.....	21
- 2.3.2 La barra de herramientas estándar.....	22
- 2.3.3 La caja de herramientas (Toolbox).....	22
- 2.3.4 Los formularios (Forms).....	23
- 2.3.5 El explorador de proyectos (Project Explorer).....	23
- 2.3.6 La ventana de propiedades (Properties Windows).....	24
- 2.3.7 La ventana de esquema de formularios (Form Layout Window).....	24
- 2.3.8 La ventana explorador de formulario (Form Explorer Window).....	25
2.4 La ayuda de MSDN Library.....	25
- 2.4.1 Instalación de MSDN Library.....	26
- 2.4.2 Comprender el visor de MSDN Library.....	26
- 2.4.3 Seleccionar la documentación adecuada.....	27
- 2.4.4 Buscar información en el visor de MSDN Library.....	27
- 2.4.4.1 Explorar mediante la Tabla de contenido.....	28
- 2.4.4.2 Encontrar información con el índice.....	28
- 2.4.4.3 Encontrar información con la búsqueda de texto completo.....	29
- 2.4.4.4 Crear una lista de temas favoritos.....	30
2.5 El Editor de Código (Code Editor).....	31
- 2.5.1 Características del Editor de Código (Code Editor).....	31
- 2.5.2 Personalizar el Editor de Código (Code Editor).....	32
2.6 El Depurador (Debugger).....	33
- 2.6.1 Depurar utilizando puntos de parada (Breakpoints).....	34
- 2.6.2 Depurar paso a paso por instrucciones (Step Into).....	34
- 2.6.3 Depurar paso a paso por procedimientos (Step Over).....	34
- 2.6.4 Depurar paso a paso para salir (Step Out).....	34
- 2.6.5 Ventana de Locales (Locals Window).....	35
- 2.6.6 Ventana Inmediato (Immediate Window).....	36
- 2.6.7 Ventana Inspección (Watch Window).....	36
- 2.6.8 Inspección rápida (Quick Watch).....	36
- 2.6.9 Pila de llamadas (Call Stack).....	37
2.7 Preguntas para contestar.....	37



CAPÍTULO III: INTRODUCCIÓN A LOS CONTROLES MÁS USUALES DE VISUAL BASIC 6.0	38
3.1 Los controles más usuales en Visual Basic 6.0	40
- 3.1.1 Los botones de comando (CommandButton)	40
- 3.1.1.1 Propiedades de los botones de comando	40
- 3.1.1.2 Uso de las propiedades en el Editor de Código	49
- 3.1.1.3 Eventos sobre los botones de comando	50
- 3.1.1.4 Métodos de los botones de comando	80
- 3.1.1.5 Ejercicios prácticos	84
- 3.2.1 Las etiquetas (Labels)	94
- 3.2.1.1 Propiedades de las etiquetas	94
- 3.2.1.2 Eventos sobre las etiquetas	104
- 3.2.1.3 Ejercicios propuestos	129
- 3.3.1 Las cajas de texto (TextBox)	130
- 3.3.1.1 Propiedades de las cajas de texto	130
- 3.3.1.2 Eventos sobre las cajas de texto	134
- 3.3.1.3 Métodos de las cajas de texto	161
- 3.3.1.4 Ejercicios prácticos	161
- 3.4.1 Los botones de opción (OptionButton)	166
- 3.4.1.1 Propiedades de los botones de opción	166
- 3.4.1.2 Eventos sobre los botones de opción	166
- 3.4.1.3 Métodos de los botones de opción	166
- 3.4.1.4 Ejercicios prácticos	166
- 3.5.1 Las cajas de verificación (CheckBox)	173
- 3.5.1.1 Propiedades de las cajas de verificación	173
- 3.5.1.2 Eventos sobre las cajas de verificación	173
- 3.5.1.3 Métodos de las cajas de verificación	173
- 3.5.1.4 Ejercicios prácticos	173
- 3.6.1 Las barras de desplazamiento (ScrollBar)	176
- 3.6.1.1 Propiedades de las barras de desplazamiento	176
- 3.6.1.2 Eventos sobre las barras de desplazamiento	177
- 3.6.1.3 Ejercicios prácticos	177
- 3.7.1 Las cajas de lista (ListBox)	182
- 3.7.1.1 Propiedades de las cajas de lista	182
- 3.7.1.2 Eventos sobre las cajas de lista	184
- 3.7.1.3 Métodos de las cajas de lista	184
- 3.7.1.4 Ejercicios prácticos	188
- 3.7.1.5 Ejercicios propuestos	191
- 3.8.1 Las cajas combinadas (ComboBox)	192
- 3.8.1.1 Ejercicios prácticos	192
- 3.9.1 El control tiempo (Timer)	193
- 3.9.1.1 Propiedades del control tiempo	193
- 3.9.1.2 Ejercicios prácticos	194
- 3.10.1 Controles relacionados con ficheros (FileList, DirList y DriveList)	196
- 3.10.1.1 Conectar los controles de ficheros	196
- 3.10.1.2 Ejercicios prácticos	197
3.2 El control de cuadros de diálogo (CommonDialog)	200
- 3.2.1 Los cuadros de diálogo Abrir y Guardar (Open/Save)	203
- 3.2.1.1 Ejercicios prácticos	206
- 3.2.2 El cuadro de diálogo Imprimir (Print)	207
- 3.2.2.1 Ejercicios prácticos	209
- 3.2.3 El cuadro de diálogo Fuente (Font)	211
- 3.2.3.1 Ejercicios prácticos	212
- 3.2.4 El cuadro de diálogo Color	214



- 3.2.4.1 Ejercicios prácticos.....	214
3.3 Arreglo en los controles.....	216
3.4 Figura con los controles más usuales de Visual Basic 6.0.....	218
CAPÍTULO IV: INTRODUCCIÓN AL LENGUAJE BASIC.....	219
4.1 El Lenguaje Basic.....	221
- 4.1.1 Antecedentes.....	221
- 4.1.2 Nacimiento y primeros años.....	221
- 4.1.3 Crecimiento Explosivo.....	222
- 4.1.4 Perfeccionamiento.....	222
4.2 Conceptos básicos.....	223
- 4.2.1 Identificadores.....	223
- 4.2.2 Palabras reservadas.....	224
- 4.2.3 Tipos de datos.....	225
- 4.2.3.1 Clasificación de los tipos de datos.....	225
- 4.2.3.1.1 Tipos enteros (Byte, Integer, Long).....	226
- 4.2.3.1.2 Tipos reales (Single, Double, Currency).....	226
- 4.2.3.1.3 Tipos cadena (String).....	226
- 4.2.3.1.4 Tipos lógicos (Boolean).....	227
- 4.2.3.1.5 Tipos variados (Variant).....	227
- 4.2.4 Constantes.....	227
- 4.2.4.1 Declaración de constantes.....	277
- 4.2.4.2 Declaración de constantes públicas.....	228
- 4.2.4.3 Declaración de constantes privadas.....	231
- 4.2.4.4 Declaración de constantes locales.....	231
- 4.2.5 Variables.....	232
- 4.2.5.1 Declaración de una variable.....	232
- 4.2.5.1.1 Declaración de una variable pública.....	233
- 4.2.5.1.2 Declaración de una variable privada.....	233
- 4.2.5.1.3 Declaración de una variable local.....	233
- 4.2.5.2 Nombre descriptivos de las variables.....	234
- 4.2.5.3 Almacenar y recuperar datos en variables.....	234
- 4.2.5.4 Operaciones aritméticas con variables.....	234
- 4.2.5.5 Ejercicios propuestos.....	237
- 4.2.6 Operaciones de entrada y salida.....	237
- 4.2.6.1 Función InputBox.....	237
- 4.2.6.2 Función MsgBox.....	239
4.3 Estructuras de control selectivas.....	244
- 4.3.1 Expresiones lógicas.....	244
- 4.3.1.1 Operadores aritméticos.....	245
- 4.3.1.2 Operadores de relación.....	245
- 4.3.1.3 Operadores lógicos.....	245
- 4.3.1.3.1 Operador lógico AND.....	246
- 4.3.1.3.2 Operador lógico OR.....	246
- 4.3.1.3.3 Operador lógico NOT.....	246
- 4.3.2 La sentencia If.....	246
- 4.3.3 La sentencia Case.....	250
4.4 Estructuras de control repetitivas.....	251
- 4.4.1 El Bucle For...Next.....	251
- 4.4.2 El Bucle Do...Loop.....	255
- 4.4.3 El Bucle While...Wend.....	256
- 4.4.4 El Bucle For Each...Next.....	257
- 4.4.5 Ejercicios propuestos.....	259



4.5 Las funciones en el lenguaje Basic.....	259
- 4.5.1 Funciones predefinidas.....	260
- 4.5.1.1 Funciones aritméticas.....	260
- 4.5.1.2 Funciones trigonométricas.....	263
- 4.5.1.3 Funciones matemáticas derivadas.....	264
- 4.5.1.4 Funciones de conversión de tipo de datos.....	265
- 4.5.1.5 Funciones de manipulación de cadenas de caracteres.....	266
- 4.5.1.5.1 Concatenación de cadenas.....	266
- 4.5.1.5.2 Obtener subcadenas.....	267
- 4.5.1.5.3 Funciones especiales.....	269
- 4.5.1.5.4 Longitud de una cadena.....	272
- 4.5.1.6 Ejercicios prácticos.....	273
- 4.5.2 Funciones definidas por el usuario.....	281
4.6 Introducción a las estructuras de datos.....	282
- 4.6.1 Los Arrays.....	282
- 4.6.1.1 Arrays unidimensionales: vectores.....	283
- 4.6.1.1.1 Declaración de un Array unidimensional.....	284
- 4.6.1.1.2 Almacenar y leer datos en un Array unidimensional.....	284
- 4.6.1.1.3 Operaciones con los elementos de un Array unidimensional.....	286
- 4.6.1.1.4 Ejercicios prácticos.....	287
- 4.6.1.2 Arrays multidimensionales: tablas y matrices.....	288
- 4.6.1.2.1 Recorrido por las filas y columnas de un Array multidimensional.....	288
- 4.6.1.2.2 Declaración de un Array multidimensional.....	289
- 4.6.1.2.3 Almacenando y leer datos en un Array multidimensional.....	290
- 4.6.1.2.4 Ejercicios propuestos.....	293
4.7 Preguntas para contestar.....	293
 CAPÍTULO V: LOS MENÚS	
5.1 ¿Qué son los menús?.....	294
5.2 Elementos de los menús.	295
5.3 El Editor de Menú (Menu Editor).....	296
- 5.3.1 Descripción de los elementos del Editor de Menús.....	296
- 5.3.2 Creación de menús en Visual Basic 6.0.....	297
- 5.3.3 Creación de submenús.....	203
5.4 Eventos sobre los menús.....	304
5.5 Ejercicios propuestos.....	304
 CAPÍTULO VI: LA INTERFAZ DEL USUARIO	
6.1 Los formularios.....	305
- 6.1.1 Concepto de formulario.....	306
- 6.1.2 Propiedades de los formularios.	306
- 6.1.3 Eventos de los formularios.	311
- 6.1.4 Trabajando con múltiples formularios.	315
- 6.1.4.1 Descargar un formulario.	317
- 6.1.4.2 Cargar un formulario.	317
- 6.1.4.3 Interactuar con controles de distintos formularios.	318
6.2 Controles comunes de ventanas.....	320
- 6.2.1 El control ImageList (Lista de Imágenes).	321
- 6.2.1.1 Configurando el control ImageList.	322
- 6.2.1.2 Extraer las imágenes de un ImageList.	324
- 6.2.1.3 Ejercicio práctico.	324



- 6.2.2 Control TreeView.	328
- 6.2.2.1 Configurar el control TreeView en tiempo de diseño.	328
- 6.2.2.2 Propiedades más importantes del control TreeView.	330
- 6.2.2.3 Cómo añadir objetos Node (Nodos).	330
- 6.2.2.4 Cómo leer el texto del Nodo seleccionado.	333
- 6.2.2.5 Eliminar uno o todos los elementos del control TreeView.	334
- 6.2.2.6 Ejercicios práctico.	334
- 6.2.3 Control ListView.	338
- 6.2.3.1 Configurar el control ListView en tiempo de diseño.	338
- 6.2.3.2 Cómo añadir encabezados en modo de diseño.	340
- 6.2.3.3 Cómo añadir encabezados en modo de ejecución.	341
- 6.2.3.4 Cómo añadir objetos ListViewItem (Filas).	341
- 6.2.3.5 Leer los datos de una o varias columnas.	344
- 6.2.3.6 Ordenar el contenido de las columnas.	345
- 6.2.3.7 Búsqueda de elementos.	345
- 6.2.3.8 Ejercicios práctico.	345
- 6.2.4 Control ToolBar.	348
- 6.2.4.1 Configurar el control ToolBar en tiempo de diseño.	348
- 6.2.4.2 Cómo reaccionar ante las acciones del usuario.	351
- 6.2.4.3 Crear menús dentro de los botones (ButtonMenu).	352
- 6.2.5 Control TabStrip.	353
- 6.2.5.1 Configurando el control TabStrip en tiempo de diseño.	353
- 6.2.5.2 Agregar objetos Tab.	354
- 6.2.5.3 Cómo determinar la ficha pulsada por el usuario.	355
- 6.2.5.4 Preparación de los contenedores.	356
- 6.2.6 Ejercicios propuestos.	358
CAPÍTULO VII: LOS ARCHIVOS	360
7.1 Concepto de Archivo	361
7.2 Característica de los Archivos	361
7.3 Creación de un archivo aleatorio (sin formato) desde Visual Basic 6.0	362
7.4 Almacenamiento de información un archivo de texto	363
7.5 Lectura de un archivo aleatorio (sin formato)	367
- 7.5.1 Lectura de un archivo utilizando un bucle.	369
- 7.5.2 Creación de un editor sencillo.	371
7.7 Archivos con estructura de base de datos	376
7.8 Crear un archivo con estructura de base de datos en Visual Basic 6.0	377
7.9 Ejercicios propuestos	401
CAPÍTULO VIII: INTRODUCCIÓN A LAS BASES DE DATOS	402
8.1 Introducción a los conceptos de base de datos	403
- 8.1.1 Concepto de base de datos.	403
- 8.1.2 Elementos de una base de datos.	403
- 8.1.3 Estructura de una base de datos.	404
- 8.1.4 Sistema de Gestión de Base de datos (SGBD).	404
- 8.1.5 Administrador de base de datos (ABD).	404
- 8.1.6 Diseñador de base de datos (DBD).	405
8.2 Creación de una base de datos	405
8.3 Conectar una base de datos con Visual Basic 6.0	406
- 8.3.1 El Modelo de datos ODBC (Open Database Connectivity).	406
- 8.3.2 El modelo de objetos DAO (Data Access Object).	406
- 8.3.3 El modelo de objetos RDO (Remote Data Object).	407
- 8.3.4 El modelo de objetos ODBCDirect.	407



- 8.3.5 El modelo de objetos OLE DB.	407
- 8.3.6 El modelo de objetos ADO (ActiveX Data Object).	407
8.4 Preparando la conexión a la base de datos.....	408
- 8.4.1 Objeto Recordset.	409
- 8.4.1.1 Propiedades del objeto Recordset.	409
- 8.4.1.2 Manejo de cursores.	411
- 8.4.1.3 Control de concurrencia.	413
- 8.4.1.4 Lectura de los campos de un Recordset.	413
- 8.4.1.5 Moverse por los registros de un Recordset.	419
- 8.4.1.6 Modificación de registros en un Recordset.	422
- 8.4.1.7 Eliminar el registro activo del Recordset.	423
- 8.4.1.8 Inserción de registros en el Recordset.	423
- 8.4.1.8 Establecer y leer la posición de un registro en el Recordset.	424
- 8.4.1.9 Ordenación de los registros de un Recordset.	425
- 8.4.1.10 Búsqueda de registros.	425
- 8.4.1.11 Verificar el estado del Recordset.	429
- 8.4.2 Eventos del objeto Recordset.....	430
- 8.4.2.1 Sucesos de recuperación de datos.	430
- 8.4.2.2 Sucesos de navegación.....	431
- 8.4.2.3 Sucesos de modificación de datos.	432
- 8.4.3 Generación de reportes.	433
8.5 Ejercicios propuestos.....	436
TRUCOS DE LA RED	437
9.1 Abrir la ventana de quitar o agregar programas de Windows.	438
9.2 Obtener el directorio desde donde estemos ejecutando nuestro programa.	438
9.3 Verificar si un archivo existe o no.	438
9.4 Capturar la pantalla entera o la ventana activa.	438
9.5 Desplegar la lista de un ComboBox automáticamente.	439
9.6 Cambiar el fondo de Windows.	439
9.7 Comprobar si el sistema posee una tarjeta de sonido.	439
9.8 Apagar el equipo, reiniciar Windows y reiniciar el sistema.	440
9.9 Situar un ScrollBar horizontal en una ListBox.	440
9.10 Centrar una ventana.	441
9.11 Obtener el directorio de Windows y el directorio de Sistema.	441
9.12 Crear un efecto Shade al estilo de los sistemas de instalación.	441
9.13 Hacer sonar un fichero WAV o una secuencia MIDI.	442
9.14 Ocultar y mostrar el cursor.	442
ANEXOS.....	443
BIBLIOGRAFIA.....	445



La mayoría de los usuarios de computadoras que se interesan por el área de la programación, en su inicialización han querido obtener una documentación capaz de introducirlos de forma adecuada en el manejo de la programación computarizada. Este libro garantiza la preparación del interesado mediante ejercicios, cuestionarios, análisis, notas y métodos bien definidos.

Esta edición se ha elaborado específicamente para el manejo de *bases de datos* debido al papel tan importante que juegan las *bases de datos* en una aplicación. La mayoría de las aplicaciones incluidas en este libro manejan *bases de datos* y están dirigidas específicamente al área comercial, por ejemplo: facturación, punto de venta (ptv), almacenes, mantenimiento, consultas, reportes, etc.

Se ha seleccionado como en la primera edición el lenguaje de programación **Visual Basic 6.0**, que es un lenguaje de programación orientado a objetos de fácil uso y uno de los más preferidos por los programadores de aplicaciones. Existen otros lenguajes de programación orientado a objetos, tales como: DELPHI, Visual FoxPro, Visual C++, entre otros, que también son muy potentes y muy utilizados por los programadores de aplicaciones.

Por su parte, los programas orientados a objetos (eventos) son los programas típicos de *Windows*, tales como *Word*, *Excel*, *PowerPoint*, etc. Cuando uno de estos programas ha arrancado, lo único que hace es quedarse a la espera de alguna *acción* del usuario, que en este caso a dicha acción en la programación orientada a objetos se le llama *evento*. Un *evento* es una acción que realiza el usuario hacia un objeto, por ejemplo, cuando el usuario hace clic en un botón de comando, esa acción de hacer clic en el botón se le llama *evento Click*. También cabe mencionar el evento **MouseMove** (*movimiento del ratón*) que ocurre cuando el usuario mueve el puntero del *mouse* (*ratón*) por cualquier objeto sobre una ventana.

Si usted ha programado con otros métodos de lenguajes de programación, ya sea de *Tipo secuencial o procedimental*, encontrará en la *programación orientada a objetos* el método más fácil y eficaz para la creación de sus aplicaciones.

A medida que avance cada capítulo de este libro, notará que ha adquirido los conocimientos necesarios para crear cualquier tipo de aplicación compatible con *Windows*, así como, la base necesaria para programar en otros lenguajes de programación orientada a objetos.



CONTENIDO

- 1.1 Concepto de programación orientada a objetos (POO).**
- 1.3 Programas orientados a eventos.**
- 1.4 Creación de programas para el entorno de Windows.**
- 1.4 Elementos de una aplicación en el entorno de Windows.**
 - 1.4.1 Formularios y controles.
 - 1.4.2 Nombre de controles y objetos.
 - 1.4.3 Propiedades, clase y estado de objetos.
 - 1.4.4 Modo de diseño y Modo de ejecución.
 - 1.4.5 Procedimientos.
 - 1.4.6 Eventos.
 - 1.4.7 Métodos.
 - 1.4.8 Módulos.
 - 1.4.9 Encapsulación, herencia, polimorfismo y mensajes en objetos.
- 1.5 Preguntas y ejercicios propuestos.**



1.5 Concepto de programación orientada a objetos (POO)

La *programación orientada a objetos* es aquella en la que trabajamos con *objetos* visibles, cada uno de los cuales posee sus propias *características*, *métodos* y *eventos*. La *programación orientada a objetos*, también llamada *lenguaje de quinta generación*, toma como entidad principal los *objetos* de la naturaleza misma que contienen sus propias *características* y alguna función de utilidad. En el mundo real un *objeto* es cualquier cosa material y determinada que tiene alguna función de utilidad y que posee *características* o *propiedades* iguales o distintas a la de otros *objetos*, por ejemplo, un carro, una computadora, un televisor, un celular, etc. En el mundo de las computadoras un *objeto* es todo lo que se ve en una aplicación típica de Windows, Linux, Beos, etc. Los *objetos* son, por ejemplo, un *botón de comando*, una *caja de texto*, una *imagen*, un *botón de opción*, una *lista desplegable*, una *barra de desplazamiento*, en general todo *objeto* visible que usted puede observar en la pantalla.

No podemos definir un *objeto* como algo que simplemente se ve en la pantalla, un *objeto* es algo más abstracto y de inmensa profusión de datos. Un *objeto* posee *características* o *propiedades* que definen su aspecto exterior, por ejemplo, el color, el tamaño, el estilo, etc. Un *objeto* también posee *métodos* y *eventos* que son elementos significativos del *objeto*. Estos dos últimos elementos los explicaremos más adelante.

En *Visual Basic 6.0* los *objetos* con que trabajamos poseen *propiedades*, *métodos* y *eventos*, aunque algunos *objetos* pueden no tener las mismas *propiedades*, *métodos* o *eventos* que otros *objetos* poseen, y también algunos *objetos* pueden tener las mismas *propiedades* de otros *objetos*. Un ejemplo de lo dicho anteriormente podría ser el siguiente: En Visual Basic existe un objeto o control llamado **TextBox** (caja de texto) y tiene una propiedad llamada **Text** (texto), que indica el texto que posee la caja y también existe un objeto o control llamado **CommandButton** (botón de comando) que obviamente usted se puede imaginar que no puede tener la propiedad **Text** que posee una caja de texto. Otro ejemplo sería: El evento **Change** (cambio) de una **TextBox** que ocurre cuando el usuario está escribiendo en la caja de texto, también está claro que en un **CommandButton** (botón de comando) no pueda ocurrir este evento.

Otros términos que apreciaremos luego y que están también relacionados con un *objeto* son: *clases*, *herencia*, *encapsulación*, *polimorfismo*, *estado de objetos* y *mensajes en objetos*. Todos estos vocablos son elementos de la *programación orientada a objetos* y son los que le dan verdadera potencia a este lenguaje de programación.

1.2 Programas orientados a eventos

Todas las aplicaciones creadas en un *lenguaje de programación orientado a objetos* serán por defecto un *programa orientado a evento*. Es un *programa orientado a eventos*, porque cuando este es cargado o ejecutado solo espera a que el usuario realice alguna acción sobre uno de los *objetos* que posee, por ejemplo, la calculadora de Windows espera a que el usuario haga *clic* (**Evento Click**) con el Mouse sobre uno de los *botones de comando* que contienen los números para luego ponerlo en la *caja de texto*, o bien, espera a que el usuario pulse un número (**Evento KeyPress**) desde el teclado para ponerlo en la *caja de texto*.

Los programas o paquetes de programas que utilizamos con frecuencia en Windows son *programas orientados a eventos*. Solo cuando el usuario realiza alguna acción sobre los *objetos* de un programa ocurre un determinado *tipo de evento*. Mediante ese *evento* el programa realiza una operación y luego notifica al usuario sobre los resultados de dicha operación. Puede probar esto cargando cualquier aplicación de Windows ya sea *Microsoft Word* o *Microsoft Excel*, cuando este es cargado se queda a la espera de que usted haga algo, por ejemplo, *Word* espera a que usted escriba alguna carta u otro tipo de documento, así como, *Excel* espera a que el usuario escriba, por ejemplo, los valores de alguna nomina de pago. Todo esto dicho anteriormente nos da a entender que cada programa es capaz de manejar un sin número de determinados *tipos de eventos* y que también son capaces de responder a cada uno de ellos.



1.3 Creación de programas para el entorno de Windows.

En *Visual Basic 6.0* es posible crear todo tipo de aplicación para Windows, pudiendo incorporar todas las características y elementos de un programa típico de Windows.

Visual Basic 6.0 posee una *barra de herramientas* donde es posible encontrar todos los elementos que apreciamos en una aplicación de Windows: *ventanas, botones, cajas de texto, cajas de diálogo, botones de opción, botones de selección, barras de desplazamiento, gráficos, menús*, en general, todo *objeto* visible en una aplicación de Windows.

Prácticamente, todos los elementos de interacción entre la aplicación y el usuario de los que dispone Windows pueden ser programados en Visual Basic 6.0 de un modo muy sencillo. Con asiduidad, los programadores crean aplicaciones para Windows con solo algunas líneas de códigos y pocas operaciones con el mouse y el teclado.

Algunos programas toman mucho más tiempo que otros para su realización, algunos toman meses, otros años, o simplemente días. El tiempo de duración para la realización de una aplicación, depende exclusivamente de la profundidad del programa y regularmente de la capacidad del programador.

1.4 Elementos de una aplicación en Windows

Las aplicaciones en Windows han evolucionado notablemente, incorporando un sin número de elementos gráficos que han permitido un mejor rendimiento por parte de los usuarios así como para el mismo sistema.

En un principio, crear aplicaciones para el entorno de Windows era bastante tedioso para los programadores, debido a que su principal preocupación era la determinación del entorno del programa y cómo responder a los posibles *eventos* del usuario, como *dónde se hacía clic, dónde se encontraba el usuario (en un menú o submenú) o si estaba haciendo doble clic, etc.* Todas estas cosas eran las principales preocupaciones del programador, y por ende, aumentaban el periodo de tiempo para la terminación del programa, además del vasto conocimiento que debía tener en ***lenguaje C***.

Con la aparición de *Visual Basic* este problema desapareció. Ahora es mucho más fácil crear aplicaciones para Windows sin la preocupación de tantas definiciones de *variables, constantes y punteros*. Los errores en Visual Basic no se generan tan frecuentemente y, en caso de que ocurra un error, es mucho más fácil de *depurar (corregir)*, debido a que cada *objeto* trabaja de forma independiente, es decir, posee su propio *código fuente* independientemente de otros *objetos*. Si un *objeto* genera un error, simplemente habría que, leer la codificación contenida en ese *objeto*.

Los elementos principales de una aplicación de Windows son: *formularios, controles, procedimientos, métodos, objetos y eventos*. Cada uno de estos elementos juega un papel muy importante en una aplicación de Windows.

- 1.4.1 Formularios o ventanas

En *Visual Basic*, un *formulario* es considerado como una ventana típica de Windows. En este se colocan todos los *controles* de los que dispone Visual Basic para crear una aplicación. Cada uno de los elementos gráficos que forman parte de una aplicación de Windows es un *tipo de control*: botones, cajas de texto, cajas de diálogo, barras de desplazamiento, menús, gráficos, cajas de verificación, y muchos otros elementos son *controles* para Visual Basic.

Un *formulario* también se puede considerar como una especie de contenedor para los *controles* de una aplicación. Una aplicación puede tener uno o varios *formularios*, pero un único *formulario*



puede ser suficiente para la creación de una potente aplicación. El número de *formularios* va a depender precisamente del tipo de aplicación y de la cantidad de módulos de trabajo que incluirá la aplicación.

- 1.4.2 Nombre de controles y objetos

Cada *control* u *objeto* en Visual Basic debe tener un *nombre*, por medio del cual se puede hacer referencia a dicho *objeto* en la aplicación. El *nombre (name)*, puede ser el que el programador desee, e incluso Visual Basic proporciona *nombres por defecto* para los diversos *controles*. Estos *nombres por defecto*, hacen referencia al tipo de *control* y van seguidos de un número, que se incrementa a medida que se van introduciendo más *controles* de ese mismo tipo en el *formulario*; por ejemplo, **Text1** para una caja de texto, **Text2** para otra caja de texto, **Command1** para un botón de comando, **Command2** para otro botón de comando, así sucesivamente.

Los *nombres por defecto* no son recomendables para los *controles* colocados en una aplicación de Visual Basic, debido a que solo hacen referencia al tipo de *control*, pero no al uso que de dicho *control* está haciendo el programador en la aplicación. Por ejemplo, si se agregan dos botones de comando en una aplicación (**Command1**, **Command2**), uno para salir de la aplicación y el otro para cancelar la salida de la aplicación, sería recomendable que el botón de **salir** lleve por *nombre* **cmdSalir** y que el botón de **cancelar** lleve por *nombre* **cmdCancelar** y no los *nombres por defecto* **Command1** y **Command2**, ya que en ocasiones no distinguiremos para que utilizamos uno y para que utilizamos el otro.

Para asignar los *nombres* a los *controles* de Visual Basic, existe una convención ampliamente aceptada por la gran mayoría de los programadores; se utilizan siempre tres letras en minúscula, que hacen referencia al tipo de *control*, seguido de otras letras (la primera en mayúscula), libremente escogida por el programador, que tienen que hacer referencia al uso que se va a dar a dicho *control* en la aplicación.

La **tabla 1.1** muestra las abreviaturas de los controles más usuales, junto con la nomenclatura inglesa de la que se derivan.

Abreviatura	Control	Abreviatura	Control
chk	CheckBox	cbo	Combo y Drop-ListBox
cmd	CommandButton	dir	DirListBox
drv	DriveListBox	fil	FileListBox
frm	Form	fra	Frame
hsb	HorizontalScrollBar	img	Image
lbl	Label	lin	Line
lst	List	mnu	Menu
opt	OptionButton	pct	PictureBox
shp	Shape	txt	TExTEditBox
tmr	Timer	vsb	VerticalScrollBar

Tabla 1.1. Abreviaturas para los controles más usuales.

- 1.4.3 Propiedades, clase y estado de objetos

Se ha dicho que los *objetos* que colocamos en un formulario de Visual Basic, también son llamados *controles*, cada uno de los cuales posee *propiedades*, *métodos* y *eventos*. Las *propiedades* son las características propias de un *objeto*, por ejemplo, imagínese *un carro de color rojo, del año 2005, de marca honda y con aros de magnesio*. El carro descrito posee características que definen o proporcionan la apariencia o el estado físico del *objeto*. A estas *características* se le denominan *propiedades*, y a los valores que poseen dichas *propiedades* se les denominan *estado del objeto*.



El *estado de un objeto* puede cambiar en Visual Basic, así como el carro mencionado anteriormente puede ser que cambie de color, en vez de rojo a negro. En este caso, se dice que el *estado* de la *propiedad Color* ha cambiado de valor, y por tal razón también ha cambiado el *estado del objeto, en este caso del carro*. Otro ejemplo sería, un botón de comando de color negro en una aplicación que se comporta de la siguiente manera: *cuando el usuario coloca el puntero del Mouse sobre el botón, entonces el botón cambia de color negro a blanco*.

Generalmente, los *nombres* de las *propiedades* de un *objeto* o *control* son campos que poseen valores lógicos (true o false), numéricos, alfabéticos ó alfanuméricos. Así pues, cada *clase, tipo de objeto o control* tienen su propio conjunto de *propiedades*. Se podría decir, que el carro mencionado anteriormente, posee las *propiedades Color, Año, Marca y Aros* con los valores *Rojo, 2005, Honda y Magnesio*. En representación de *Visual Basic* se haría de la siguiente manera:

```
Carro.Color = Rojo
Carro.Año = 2005
Carro.Marca = Honda
Carro.Aros = Magnesio
```

Casi todas las *propiedades* de los *controles* de *Visual Basic* pueden cambiarse en momento que la aplicación se esta diseñando (*modo de diseño*), y también casi siempre cuando la aplicación esta en ejecución (*modo de ejecución*). Para modificar u obtener el *estado de un objeto* se hace por medio del *nombre del objeto (Name)*, seguido de un punto (.) y el *nombre de la propiedad*. Por ejemplo, para cambiar el color de una *caja de texto* llamada Text1, se haría de la siguiente manera:

```
Text1.BackColor = vbRed
```

: Donde **Text1** es el *nombre del objeto*, **BackColor** el *nombre de la Propiedad* y **vbRed** es el color rojo por defecto de *Visual Basic*. En *Visual Basic* los colores se representan por *constantes* y *valores hexadecimales*.

Por otro lado, la **clase** representa la entidad genérica a la que pertenece un *objeto*, por ejemplo, en una aplicación, puede haber varios *botones de comando*, cada uno de los cuales es un *control* que pertenece a una *clase de objetos*, llamada **CommandButton** (*botones de comando*). La *clase* del *objeto* también determina las *propiedades* de los *objetos* de esa *clase*, es decir, cada *clase, tipo de objeto o control* tienen su conjunto de *propiedades*, y cada *objeto* o *control* tienen valores determinados para las *propiedades* de su *clase*.

- 1.4.4 Modo de diseño y Modo de ejecución

Anteriormente, se había comentado que en momento que se está diseñando la aplicación se le denomina *modo de diseño*, y cuando la aplicación se esta ejecutando se le denomina *modo de ejecución*. Las aplicaciones en Visual Basic pueden trabajar en dos modos distintos, que son: *modo de diseño* y *modo de ejecución*. En *modo de diseño*, el programador construye interactivamente la aplicación, colocando *controles* en el *formulario*, definiendo sus *propiedades* y codificando los *procedimientos* para gestionar los *eventos* de cada *control*.

La aplicación se prueba en *modo de ejecución*. En este caso, el programador actúa sobre la aplicación produciendo los *eventos* codificados y obteniendo los valores proporcionados por dichos *eventos*. Hay algunas *propiedades* de los *controles* que solo pueden establecerse en *modo de diseño*, pero muchas otras pueden cambiarse en *modo de ejecución*. Para que una aplicación pase del *modo de diseño* al *modo de ejecución* simplemente hay que pulsar la tecla **[F5]**, o bien, hacer clic en el botón **Iniciar (Start)** , de la *barra de herramientas estándar*. Otra forma no muy utilizada, es haciendo clic en la opción **Iniciar (Start)** del menú **Run (Ejecutar)**.



- 1.4.5 Procedimientos

Un *procedimiento* representa la definición o declaración de un *objeto*. Los *procedimientos* se encuentran dentro de los *módulos*, y contienen el conjunto de instrucciones que se ejecutan cuando el usuario realiza algún *evento* sobre el *objeto*, o cuando estos son llamados por su *nombre*. Un *procedimiento* viene representado por un *nombre*, los *argumentos* (*constantes, variables o expresiones*) y el código que compone el cuerpo del *procedimiento*. Algunos *procedimientos* sólo vienen representados con un *nombre*, es decir, sin *argumentos*, y otros vienen representados con el *nombre del procedimiento*, los *argumentos* y el *nombre del evento* que activa dicho *objeto*.

Para definir un *procedimiento*, se utiliza la palabra clave **Sub**, seguida del *nombre del procedimiento* y los *argumentos* (*si el procedimiento lo requiere*), así como el *alcance del procedimiento*. Cuando hablamos del *alcance del procedimiento*, nos estamos refiriendo a la posibilidad de llamar un *procedimiento* desde otros *módulos* de la misma aplicación, principalmente desde otros *formularios*.

El alcance de un *procedimiento* puede ser declarado de dos formas: *privado* ó *público*. *Privado* cuando el *procedimiento* puede ser utilizado en un mismo *módulo (formulario)*, es decir, sólo los *procedimientos* dentro del mismo módulo pueden invocar dicho *procedimiento*. Para declarar un *procedimiento* como *privado* se utiliza la palabra clave **Private**.

El alcance de un *procedimiento* declarado como *Público* es mucho más amplio. Un *procedimiento público* puede ser invocado desde otros *módulos* o *formularios* que se encuentren en la misma aplicación. Este segundo método de declaración es muy importante cuando un *procedimiento* es utilizado con mucha frecuencia en una aplicación. Para declarar un *procedimiento* como *privado* se utiliza la palabra clave **Public**.

En resumen, la representación o formato de un *procedimiento* sería el siguiente:

- 1) **Private Sub** ó **Public Sub** [*Nombre_del_procedimiento*] (*argumentos*)

End Sub

- 2) **Private Sub** ó **Public Sub** [*Nombre_del_procedimiento*][_*Evento*] (*argumentos*)

End Sub

- 3) **Private Sub** ó **Public Sub** [*Nombre_del_procedimiento*][_*Evento*]

End Sub

- 4) **Sub** [*Nombre_del_procedimiento*] ()

End Sub

En algunos *procedimientos* se omite su *alcance* como lo es en el *ejemplo 4*. Cuando se omite el *alcance* del *procedimiento*, entonces, *Visual Basic* lo interpreta como **Privado (Private)**.

Se ha dicho anteriormente que todo *objeto* o *tipo de control* viene representado por un *procedimiento*, y que cada *procedimiento* posee códigos que se ejecutan cuando el usuario realiza una acción (*evento*) sobre el *objeto*. Por ejemplo, el *evento Click* de un *botón de comando* vendría representado de la siguiente manera:



```
Private Sub Command1_Click ( )
```

```
End      'La sentencia End se utiliza para salir de la aplicación.
```

```
End Sub
```

Las especificaciones anteriores muestran un *procedimiento* que representa el *evento Click* de un *botón de comando*. Como explicamos anteriormente, la sentencia **Private Sub** indica que el *procedimiento* es *Privado*, y que sólo puede ser utilizado en un mismo *módulo*. **Command1** especifica el nombre del *objeto o control*, en este caso el *nombre por defecto* del *botón de comando*. El carácter **underscore** o **subrayado** (), se utiliza para separar el nombre del *objeto* y el nombre del *evento*. El término **Click** indica el nombre del *evento*, que especifica que la acción de ese *procedimiento* se ejecutará cuando el usuario haga click sobre el botón. Los paréntesis () se utilizan para separar los *argumentos* del *nombre del objeto* y el *nombre del evento*; en este caso, el *procedimiento* anterior no posee ningún *argumento*.

La sentencia **End** se utiliza para salir de la aplicación y en este caso es el cuerpo del *procedimiento*. La sentencia **End** cierra todos los formularios abiertos sin importar la acción que se este realizando sobre ellos. **End Sub** indica el fin del *procedimiento*, es decir, donde finaliza la codificación.

Otro ejemplo de un *procedimiento* sería el *evento KeyPress* de una *caja de texto (TextBox)*:

```
Private Sub Text1_KeyPress (KeyAscii As Integer)
```

```
End Sub
```

El término **KeyPress** es un *evento* que ocurre cuando el usuario presiona y suelta una tecla sobre la *caja de texto*. Este *evento* puede ser muy útil a la hora de programar una *caja de texto*, debido a que permite obtener el valor numérico de una tecla pulsada sobre la caja.

Se puede observar en este *procedimiento*, que existen algunas especificaciones dentro de los paréntesis, que en este caso son los *argumentos* del *evento KeyPress*. El término **KeyAscii** es una *variable* de tipo entero que almacena el valor **ANSI** de la tecla presionada. Los valores **ANSI** son Juegos de caracteres de 8 bits del *Instituto Americano de Normas Nacionales (ANSI)*, usado por Microsoft Windows, que permite representar hasta 256 caracteres (0–255) con el teclado. **As Integer** declara la variable **KeyAscii** como tipo entero, es decir, como un número sin punto decimal.

- 1.4.6 Eventos

Ya se ha dicho que las acciones que realiza el usuario sobre un *objeto* se llaman *eventos*. Son *eventos* típicos: hacer clic sobre un botón (**evento Click**), hacer doble clic sobre un fichero (**evento DblClick**), arrastrar un icono (**evento DragOver**), pulsar una tecla o una combinación de teclas (**evento KeyPress**), escribir en una caja de texto (**evento Change**), o simplemente desplazar el puntero del Mouse sobre un objeto (**evento MouseMove**).

Los *eventos* vienen especificados junto con el *procedimiento* del *objeto*, separado por el carácter **underscore** o **subrayado** (), como vimos anteriormente en las declaraciones de *procedimientos*.

En *Visual Basic 6.0*, existe un sin número de *eventos* que estudiaremos más adelante, en la definición de cada uno de los *controles* de *Visual Basic 6.0*. La siguiente tabla muestra los *eventos* más comunes de los *controles* de *Visual Basic 6.0*.



Evento	Descripción
Click	Ocurre cuando el usuario presiona y suelta un botón del mouse sobre un <i>objeto</i> .
DbClick	Ocurre cuando el usuario presiona y suelta dos veces un botón del mouse sobre un <i>objeto</i> .
DragDrop	Ocurre como resultado de arrastrar y soltar con el mouse un <i>control</i> sobre un determinado <i>tipo de objeto</i> .
DragOver	Ocurre cuando una operación de arrastrar y colocar está en curso. Puede usar este <i>evento</i> para controlar el puntero del mouse a medida que entra, sale o descansa directamente sobre un destino válido.
GotFocus	Ocurre cuando un <i>objeto</i> recibe el { CONTROL Internet.HHCtrl.1 } { HYPERLINK "JavaScript:alink_4.Click()" }, ya sea mediante una acción del usuario, como tabular o hacer clic en el objeto, o cambiando el enfoque en el código mediante el método SetFocus .
LostFocus	A diferencia del evento anterior, este <i>evento</i> ocurre cuando el <i>objeto</i> pierde el enfoque, ya sea mediante tabulaciones o hacer clic sobre otro <i>objeto</i> .
KeyDown	Ocurre cuando el usuario mantiene presionada una tecla.
KeyUp	Ocurre cuando el usuario termina la operación de pulsar una tecla. Se podría decir, que este evento ocurre precisamente al terminar el evento KeyDown .
KeyPress	Ocurre como resultado de presionar y soltar una tecla.
MouseDown	Ocurre cuando el usuario presiona un botón del mouse, pero a diferencia del <i>evento MouseDown</i> , permite identificar cuales de los tres botones del mouse fue presionado y las combinaciones de tecla ALT , MAYÚS y CTRL .
MouseUp	El evento MouseUp se produce cuando el usuario suelta el botón del mouse. MouseUp es un compañero útil a los eventos MouseDown y MouseMove .
MouseMove	Este evento ocurre mientras el usuario mueve o desplaza el puntero del mouse sobre un <i>objeto</i> .

Tabla 1.2 Eventos más comunes de los controles de Visual Basic 6.0.

Más adelante, veremos cómo utilizar cada uno de estos *eventos* y cómo se comportan ante algunas situaciones.

- 1.4.7 Métodos

Los *métodos* son funciones que también son llamadas desde el programa, pero a diferencia de los *procedimientos* no son codificados por el programador. Los *métodos*, son llamados desde código en Visual Basic de la misma manera como se hace referencia a una propiedad del *control*. Cada tipo de *objeto* o *control* poseen sus propios *métodos*, aunque algunos *tipos objetos* tienen *métodos* comunes.

Los *formularios* en Visual Basic poseen un *método* llamado **Hide**, que permite ocultar el *formulario* y otro *método* llamado **Show**, que permite mostrar el *formulario* después de haber sido ocultado. Un ejemplo de cómo hacer referencia a uno de estos *métodos* sería el siguiente:

- Para ocultar el formulario: **Form1.Hide**



Donde **Form1** indica el *nombre del control*, que en este caso es un formulario, y **Hide** especifica el *nombre del procedimiento*. Para mostrar el formulario se haría de la misma forma, pero en vez de utilizar el *método Hide* se utiliza el *método Show*.

A continuación, se muestran los métodos más comunes de los *controles* de *Visual Basic 6.0*:

Método	Descripción
Drag	Inicia, termina o cancela una operación de arrastre de cualquier <i>control</i> , excepto los <i>controles</i> Line , Menu , Shape , Timer o CommonDialog .
Move	Se utiliza para mover un <i>control</i> o <i>formulario</i> , especificando sus coordenadas (Top , Left) y su tamaño (Width , Height).
Refresh	Se utiliza para dibujar o actualizar gráficamente un <i>control</i> o un <i>formulario</i> . Se utiliza principalmente con los <i>controles</i> FileListBox y Data .
SetFocus	Este <i>método</i> se utiliza para hacer que un <i>objeto</i> reciba el enfoque. Este <i>método</i> es uno de los más usados para los <i>controles</i> de <i>Visual Basic 6.0</i> .
ShowWhatsThis	Permite mostrar un tema seleccionado de un archivo de Ayuda utilizando el menú emergente ¿Qué es esto? que ofrece la ayuda de Windows. Este <i>método</i> es muy útil para proporcionar ayuda interactiva en un menú contextual acerca de un <i>objeto</i> en una aplicación. Este <i>método</i> muestra el tema indicado por la propiedad WhatsThisHelpID del <i>objeto</i> especificado en la sintaxis.
Zorder	Se utiliza para que un <i>control</i> o un <i>objeto formulario</i> se coloque por encima o por debajo de otros <i>objetos</i> .

Tabla 1.3 Métodos más comunes de los controles de Visual Basic 6.0.

Al igual que con los *eventos*, todos estos *métodos* serán ejemplificados y utilizados más adelante en los programas resueltos y propuestos en los próximos capítulos.

- 1.4.8 Módulos

Un *módulo* se puede definir, como el lugar donde se almacena el *código fuente* de una aplicación en Visual Basic. Los *módulos* se clasifican en tres tipos que son: *formulario*, *estándar* y *de clase*.

- Módulos de formulario

Los *módulos de formulario* se almacenan en un archivo con extensión **.frm**, y son la base de la mayoría de las aplicaciones de *Visual Basic*. Pueden contener *procedimientos* que controlen *eventos*, *procedimientos generales* y declaraciones a nivel de *formulario* de variables, constantes, tipos y *procedimientos externos*. Si examina un *módulo de formulario* con un editor de textos, podrá ver las descripciones del formulario y sus controles, así como los valores de sus *propiedades*. El código que se escribe en un *módulo de formulario*, es específico de la aplicación a la que pertenece el *formulario* y puede hacer referencia a *otros formularios* u *objetos* de la aplicación.

- Módulos estándar

Los *módulos estándar* trabajan independientemente de la aplicación y se almacenan en un archivo con extensión **.bas**. En ocasiones, el programador necesitará declarar *módulos* generales, que contengan códigos que puedan ser utilizados desde varios *formularios*, para así evitar tener que volver a repetir el código. Pueden contener declaraciones disponibles para toda la aplicación o a nivel de *módulo de variables*, *constantes*, *tipos*, *procedimientos externos* y *procedimientos globales*. Es posible reutilizar un *módulo estándar* en distintas aplicaciones, si tiene cuidado de no hacer referencia a *controles* o *formularios* por su nombre.

Ing. Carlos Manuel Rodríguez Bucarely



- Módulos de clase

Los *módulos de clase*, son la base de la *programación orientada a objetos* en Visual Basic y se almacenan en archivos con extensión **.cls**. Puede escribir código en *módulos de clase* para crear nuevos *objetos*. Estos *objetos* nuevos pueden incluir *propiedades* y *métodos personalizados*. En realidad, los *formularios* sólo son *módulos de clase* que pueden tener *controles* y que pueden mostrar ventanas de *formulario*.

- 1.4.9 Encapsulación, herencia, polimorfismo y mensajes en objetos.

Los *objetos* o *controles* poseen códigos y estructuras internas ocultas para otras entidades. La capacidad de ocultar y aislar el código de un *objeto* o *control* en una aplicación se denomina *Encapsulación*. Por ejemplo, un componente *cliente* que realiza una consulta acerca de ingresos netos a un objeto empresarial no tiene que conocer el origen de los datos.

Existen *objetos* que adquieren o heredan *propiedades* y *métodos* de otros *objetos* de mayor jerarquía, ya sea de clases superiores o una *superclase*. Esta capacidad de compartir la estructura de otros *objetos* de *clases superiores* se denomina *Herencia*. Por ejemplo, podemos tener una *clase* de equipos de música llamada ALTOSONIDO, imagínese que esta marca lance al mercado la línea de equipo de música ALTO SN-X y de esta línea surgen los modelos: ALTO SN-X M1, ALTO SN-X M2 y ALTO SN-X M3. En este caso la *clase* de mayor jerarquía es ALTOSONIDO y las demás son *subclases* de esta *clase*. La *subclase* ALTO SN-X es considerada *superclase* de las *subclases* ALTO SN-X M1, ALTO SN-X M2 y ALTO SN-X M3, ya que estas heredan las *características* de la *superclase* ALTO SN-X. El ejemplo anterior se podría representar gráficamente de la siguiente manera:

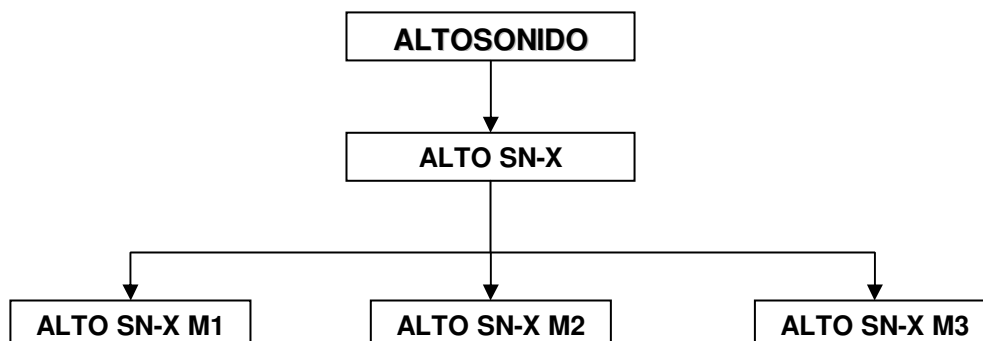


Figura 1.1 Ejemplo de jerarquías entre clases y subclases.

En caso de que la *subclase* ALTO SN-X M1 hubiese lanzando el modelo ALTO SN-X M1-001, entonces, la *subclase* ALTO SN-X M1-001 sería una *subclase* de la *clase* ALTO SN-X M1. De igual manera, si la *subclase* ALTO SN-X M2 lanza el modelo ALTO SN-X M2-001

Otro término muy importante en la *programación orientada a objetos* es el **Polimorfismo**, que significa que muchas *clases* pueden proporcionar la misma *propiedad* o el mismo *método*, y que el *objeto* que llama no tiene por qué saber la *clase* a la que pertenece el *objeto* que recibe la solicitud antes de ser invocada una *propiedad* o un *método* de dicho objeto.

Por ejemplo, una clase Carros y una clase Motocicletas podrían tener las dos un *método* llamado *Encender*. El *polimorfismo* significa que puede invocar *Encender* sin saber si el *objeto* es un Carro o una Motocicleta. Este ejemplo se podría representar gráficamente de la siguiente manera:

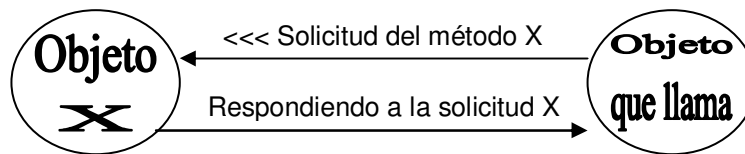


Figura 1.2 Llamada a un método de un objeto cualquiera.

Por otro lado, los *mensajes en objetos* no son más que las llamadas a los métodos de dicho *objeto*. Por ejemplo, cuando le decimos a un *objeto* Radio que se ponga en funcionamiento, estamos pasándole el mensaje “ponte en funcionamiento”.

Para mandar mensajes a los *objetos* utilizamos el *operador punto* (*.*), seguido del *método* que deseamos invocar:

ObjetoRadio.PonteEnFuncionamiento()

En este ejemplo pasamos el *mensaje* **PonteEnFuncionamiento()**. Los paréntesis se especifican en caso de que existan *argumentos*, de lo contrario no se especifican.

1.5 Preguntas y ejercicios propuestos

I.- Conteste:

- 1.) ¿Qué es la programación orientada a objetos?
- 2.) ¿Qué es un objeto?
- 3.) ¿Cuáles son los elementos principales de un objeto?
- 4.) Defina: eventos, métodos y propiedades.
- 5.) Diga la diferencia entre un evento y un método.
- 6.) Mencione algunos eventos de los controles de Visual Basic.
- 7.) ¿Qué es un programa orientado a eventos?
- 8.) Mencione algunos programas de Windows orientado a eventos.
- 9.) ¿Cuáles son los elementos principales de una aplicación de Windows?
- 10.) ¿Cuál era la principal preocupación de los programadores de aplicaciones para Windows, antes de la existencia de la programación orientada a objetos?
- 11.) ¿Qué es un formulario?
- 12.) ¿De qué depende el número de formularios que se deben incluir en una aplicación en Visual Basic?
- 13.) ¿Qué es un nombre de objeto?
- 14.) ¿Qué son nombres por defecto?
- 15.) ¿Cuál es la desventaja de los nombres por defecto?
- 16.) ¿Cómo se deben especificar los nombres a los controles de Visual Basic?
- 17.) Diga las abreviaturas para los controles: CheckBox, ListBox, Label, TextBox, CommandButton, Shape, Image y PictureBox.
- 18.) Defina: *clase y estado de objetos*.
- 19.) Diga que hace la diferencia entre dos tipos de controles.
- 20.) ¿Qué tipos de valores es posible asignar a las propiedades de un control de Visual Basic?
- 21.) Diga cómo se accede a una *propiedad* de un control de Visual Basic.
- 22.) Hable del modo de diseño y el modo de ejecución.
- 23.) ¿Qué son procedimientos?
- 24.) Especifique la forma de definir un procedimiento.
- 25.) Diga los modos de declarar un procedimiento y explique cada uno de ellos.
- 26.) ¿Qué son códigos ANSI?
- 27.) Diga que separa un evento del nombre de un procedimiento.



- 28.) Diga cuando ocurren los siguientes eventos: Click, DblClick, KeyDown, KeyUp, KeyPress, LostFocus, GotFocus, MouseDown, MouseUp y MouseMove.
- 29.) Diga la función de los métodos: Drag, Refresh, Move y SetFocus.
- 30.) ¿Qué son módulos?
- 31.) Diga como se clasifican los módulos y defina cada uno de ellos.
- 32.) Defina: Encapsulación, herencia, polimorfismo y mensajes en objetos.

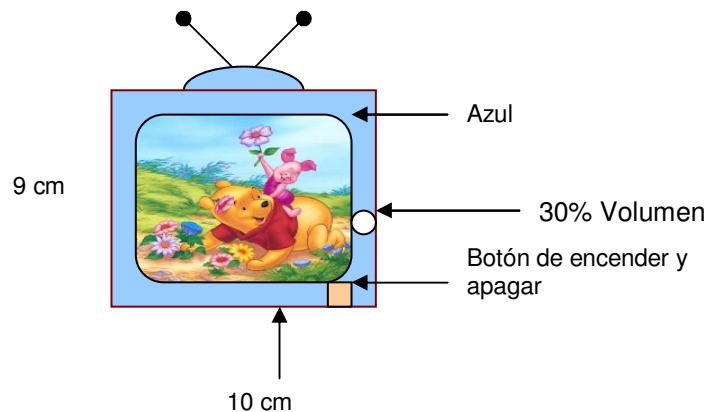
II.- Conteste F o V según crea:

- 1.) La programación orientada a objetos es un lenguaje de cuarta generación ____
- 2.) La programación orientada a objetos trata de imitar los objetos de la realidad ____
- 3.) Un objeto es simplemente algo que se ve en una ventana típica de Windows ____
- 4.) Todos los objetos poseen propiedades iguales ____
- 5.) Algunos objetos poseen propiedades que otros objetos no poseen ____
- 6.) Algunos objetos poseen propiedades que otros objetos no poseen ____
- 7.) Para que un programa sea orientado a eventos debe ser creado en un lenguaje de programación orientado a objetos ____
- 8.) En Visual Basic no es posible crear cualquier aplicación para Windows ____
- 9.) Todas las aplicaciones en Visual Basic toman el mismo tiempo para su realización ____
- 10.) El tiempo para la realización de una aplicación exclusivamente del conocimiento del programador ____
- 11.) Un formulario de Visual Basic también recibe el nombre de ventana ____
- 12.) Una aplicación con varios módulos de trabajo solo debe tener un único formulario ____
- 13.) Es posible hacer referencia a un *objeto* o *control* si este no posee un nombre ____
- 14.) Los nombres por defecto hacen referencia a la utilidad del *objeto* en la aplicación ____
- 15.) La abreviatura inglesa para el *control VerticalScrollBar* es bsv ____
- 16.) Las propiedades definen las características de un objeto ____
- 17.) En una línea de comando el punto se utiliza para hacer referencia únicamente a una propiedad del objeto ____
- 18.) El estado de un objeto puede cambiar en modo de ejecución ____
- 19.) Un evento puede modificar el estado de un objeto ____
- 20.) Cuando un objeto modifica su estado, es porque el valor de unas de sus propiedades ha cambiado ____
- 21.) El signo de igual se utiliza para asignar un valor a una propiedad, a una variable o a una constante ____
- 22.) Todos los objetos en Visual Basic pertenecen a una misma clase ____
- 23.) La clase define las propiedades, métodos y eventos de un objeto ____
- 24.) En el modo de diseño es posible ver los valores generados por los procesos de la aplicación ____
- 25.) Todos los objetos poseen procedimientos ____
- 27.) Todos los procedimientos incluyen eventos ____
- 28.) Todos los procedimientos deben tener un nombre ____
- 29.) Todos los procedimientos poseen variables, constates y expresiones ____
- 30.) La palabra clave Sub se utiliza para declarar un procedimiento ____
- 31.) Un procedimiento declarado como privado tiene el mismo alcance que un procedimiento declarado Publico ____
- 32.) El carácter underscore o subrayado (_) separa el evento de los argumentos ____
- 33.) La especificación End Sub indica el final de un procedimiento ____
- 34.) Los eventos son programados por el usuario mediante los procedimientos ____
- 35.) Los métodos son programados por los creadores del objeto ____
- 36.) Un objeto puede hacer una solicitud a un método de otro objeto ____
- 37.) Los códigos fuente de una aplicación se almacenan en los procedimientos ____
- 38.) Los procedimientos se almacenan dentro de los módulos ____
- 39.) Un formulario es un modulo con extensión .frm ____
- 40.) Los módulos estándar pueden ser utilizados en diversas aplicaciones ____
- 41.) Los módulos estándar depende exclusivamente del programa ____



- 42.) La encapsulación oculta el código fuente de un objeto ____
 43.) Todos los objetos heredan las mismas propiedades de otros objetos de su misma clase ____
 44.) Polimorfismo significa la capacidad de compartir recursos con otros objetos ____
 45) Un método es una acción que realiza un objeto sobre si mismo sin la intervención del usuario ____

III.- Complete la tabla con las posibles propiedades, estados, eventos y métodos del siguiente objeto:



Propiedades	Métodos	Eventos
Ejemplo: Televisor.Color = Azul	Ejemplo: Televisor.MostrarImagen	Ejemplo: Televisor_SubirVolumen
Televisor.Encendido = Si		

IV.- Dibuje 3 objetos y coloque sus propiedades, eventos y métodos.



CONTENIDO

2.1 ¿Qué es Visual Basic?

2.2 Historia y versiones de Visual Basic.

2.3 El EID (Entorno Integrado de Desarrollo) de Visual Basic 6.0.

- 2.3.1 La barra de menús.
- 2.3.2 La barra de herramientas estándar.
- 2.3.3 La caja de herramientas (Toolbox).
- 2.3.4 Los formularios (Forms).
- 2.3.5 El explorador de proyectos (Project Explorer).
- 2.3.6 La ventana de propiedades (Properties Windows).
- 2.3.7 La ventana de esquema de formularios (Form Layout Window).
- 2.3.8 La ventana explorador de formulario (Form Explorer Window).

2.4 La ayuda de MSDN Library.

- 2.4.1 Instalación de MSDN Library.
- 2.4.2 Comprender el visor de MSDN Library.
- 2.4.3 Seleccionar la documentación adecuada.
- 2.4.4 Buscar información en el visor de MSDN Library.
 - 2.4.4.1 Explorar mediante la Tabla de contenido.
 - 2.4.4.2 Encontrar información con el índice.
 - 2.4.4.3 Encontrar información con la búsqueda de texto completo.
 - 2.4.4.4 Crear una lista de temas favoritos.

2.5 El Editor de Código (Code Editor).

- 2.5.1 Características del Editor de Código (Code Editor).
- 2.5.2 Personalizar el Editor de Código (Code Editor).

2.6 El Depurador (Debugger).

- 2.6.1 Depurar utilizando puntos de parada (Breakpoints).
- 2.6.2 Depurar paso a paso por instrucciones (Step Into).
- 2.6.3 Depurar paso a paso por procedimientos (Step Over).
- 2.6.4 Depurar paso a paso para salir (Step Out).
- 2.6.5 Ventana de Locales (Locals Window).
- 2.6.6 Ventana Inmediato (Immediate Window).
- 2.6.7 Ventana Inspección (Watch Window).
- 2.6.8 Inspección rápida (Quick Watch).
- 2.6.9 Pila de llamadas (Call Stack).

2.7 Preguntas para contestar.



2.1 ¿Qué es Visual Basic?

Visual Basic es un lenguaje de *programación orientado a objetos* creado por la Microsoft. Este lenguaje incorpora todas las herramientas necesarias para la creación de cualquier aplicación para Windows. Con este lenguaje se puede crear desde una simple calculadora hasta un procesador de texto de la talla de Word, o una hoja de cálculo como Excel, o bien, cualquier aplicación que se le ocurra al programador.

Visual Basic es un *lenguaje visual* que se origina del lenguaje de programación **Basic**. La programación en *Visual Basic* se basa en un ambiente de desarrollo totalmente gráfico, que facilita la creación de interfaces gráficas, y en cierta medida, también la programación misma. Todos los programas que realicemos en *Visual Basic* serán por defecto, programas con todas las características de una aplicación típica de Windows.

2.2 Historia y versiones de Visual Basic

Hace poco más de 15 años, la realización de aplicaciones para el entorno de Windows era bastante complicada antes de la introducción de *Visual Basic* 1.0 en su primera versión en 1991. Esta versión incluyó poco más que la tecnología *Embedded Basic*, que había sido desarrollada originalmente en Microsoft QuickBasic 4.0 y una herramienta compiladora de diseño simple, originalmente diseñada para Windows 3.0 pero que nunca fue utilizada para tal fin. Aproximadamente 12 meses después, el desarrollo y mejora de la versión 1.0 comenzó, Microsoft sacó al mercado una herramienta desarrolladora para cubrir la exigencia en ese momento del mercado, cuyo nombre en clave fue "Thunder" (Trueno).

Después de la mejora de esta primera versión *Visual Basic* 1.0, se propaga a través de toda la comunidad en unos pocos meses. Poco después surge la fiebre de por *Visual Basic*, un grupo pequeño pero fuerte. Programadores empiezan a transformar las bibliotecas de código en controles. Estos controles se distribuían como componentes de *Visual Basic*, llamados VBXs, o los controles personalizados.

Cuando aún no había pasado un año de su salida inicial al mercado, Microsoft había comenzado a utilizar *Visual Basic* para la creación de algunos de sus propios proyectos. A medida que la demanda de Visual Basic aumentaba, los desarrolladores requerirían un *Visual Basic* más potente. Para tratar a esta necesidad creciente, Microsoft anunció la disponibilidad de *Visual Basic* 2.0 en noviembre de 1992.

La segunda versión de *Visual Basic*, distribuida en la edición estándar y profesional, proveía a los desarrolladores un funcionamiento perceptiblemente mejorado y mayor capacidad para crear aplicaciones de mayor tamaño y más sofisticadas. Incluía también una ayuda para mejorar la puesta a punto y depuración; proveía de la capacidad de conectarse a bases de datos mediante ODBC (conectividad de base de datos abierta), y nuevas y productivas herramientas, por ejemplo, *la ventana de propiedades*, sintaxis del código en color, y completo soporte para un *Interfaz de Múltiples Documentos* (MDI).

Mientras la adopción de *Visual Basic* en las corporaciones se expandía, también lo hacía la necesidad de una herramienta para el desarrollador que permitiera aplicaciones data-aware robustas. *Visual Basic* 3.0, anunciado solamente seis meses después de la salida al mercado de la versión 2.0, solucionaba esta necesidad combinando el motor de la base de datos de Microsoft Access 1.1 con un conjunto rico de controles data-aware.

Se anuncian en marzo de 1997 y en junio de 1998 las versiones de *Visual Basic* 5.0 y 6.0, respectivamente. Representaron un paso importante hacia posibilitar a los desarrolladores en *Visual Basic*, programar en los nuevos niveles del funcionamiento, en el ambiente libre que representa Internet. Las características tales como el compilador del código nativo, introdujeron aumentos del funcionamiento de hasta el 2.000 por ciento. El Webclass designer (diseñador de cla-



Capítulo 2

ses Web), simplificó la creación de las aplicaciones de la Web, proporcionando un modelo intuitivo del objeto para el servidor web, y el diseñador de páginas DHTML permitió la creación de aplicaciones para Microsoft Internet Explorer 4.0, que combinaron la riqueza de HTML dinámico (DHTML), con el funcionamiento y la productividad de *Visual Basic*. Finalmente, con la Control Creation Edition (Edición de Creación de Controles), los desarrolladores de *Visual Basic* podrían construir fácilmente los controles Microsoft ActiveX de alto rendimiento y con un amplio alcance que estaban destinados a Internet.

Hoy en día, se encuentra en el mercado la versión .NET de *Visual Basic*. Con *Visual Basic* .NET, *Visual Basic* sigue siendo la herramienta más productiva para la creación de aplicaciones que se ejecutan en el sistema operativo Microsoft Windows. Esta versión, incluye todas las herramientas de programación rápida de aplicaciones que los programadores esperan de Microsoft, como la creación, con arrastrar y colocar, de aplicaciones para Windows que aprovechan totalmente las bases de datos y los servicios Web XML. También *Visual Basic* .NET, ofrece características de Internet móvil que permiten a los programadores crear una interfaz Web móvil única. Esta versión no solo es más potente, sino que también, ofrece un entorno grafico cien por ciento mejorado, en relación a las versiones anteriores de *Visual Basic*. El *editor de código* y otros elementos del **EID** poseen nuevas características y mejoras que facilitan la lectura y escritura de los *procedimientos* escritos en la aplicación.

2.3 El EID (Entorno Integrado de Desarrollo) de Visual Basic 6.0

El EID de *Visual Basic 6.0*, posee un sin numero de herramientas que hacen de la programación de aplicaciones en Visual Basic mucho más fácil e interactiva. Este entorno incluye elementos tales como: barra de menús, barra de controles, barra de herramientas, ventana de propiedades, ventana de proyectos, depurador, formularios, etc. Estos elementos los podemos apreciar en el momento que ejecutamos *Visual Basic* en nuestro ordenador. Para ejecutar *Visual Basic 6.0*, haga clic en el menú **Inicio>Programas>Microsoft Visual Studio 6.0>Microsoft Visual Basic 6.0**. Cuando se ha ejecutado *Visual Basic 6.0* aparece en la pantalla una ventana similar a la mostrada en la siguiente figura:

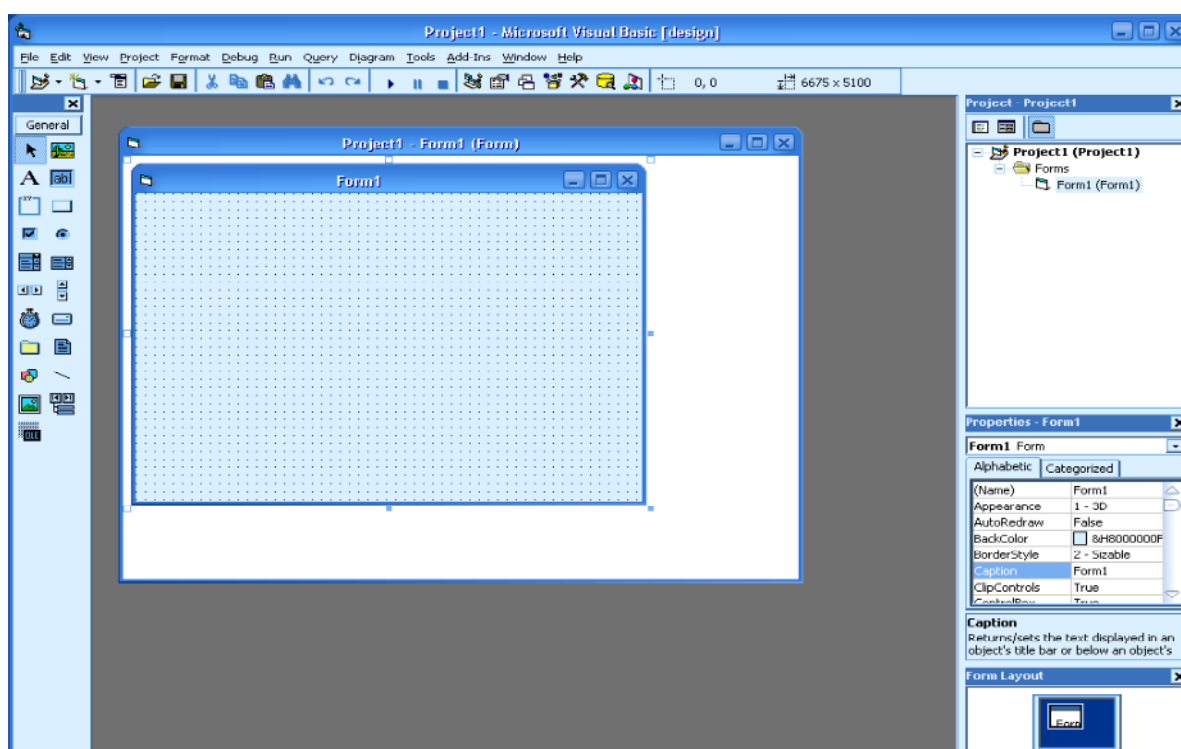


Figura 2.1. Entorno de Desarrollo de Visual Basic 6.0

Ing. Carlos Manuel Rodríguez Bucarely



En esta ventana se pueden distinguir los siguientes elementos:

- 1.- **La barra de título, la barra de menús y la barra de herramientas estándar.**
- 2.- **La caja de herramientas** (ToolBox) con los *controles* más comunes.
- 3.- **Los formularios** (forms) en gris, en el que se colocan los *controles* de la aplicación. Está dotado de una rejilla (grid)), para facilitar la alineación de los *controles* en el *formulario*.
- 4.- **El explorador de proyecto** (Project Explorer), donde se muestran los *formularios* y otros *módulos de programas* que forman parte de la aplicación.
- 5.- **La ventana de propiedades** (Properties Windows), en la que se pueden ver las *propiedades* de un *objeto* seleccionado sobre un *formulario* de la aplicación.
- 6.- **La ventana de esquema de formularios** (Form Layout Window), que muestra y determina la posición inicial de los formularios cuando la aplicación se este ejecutando.
- 7.- **La ventana explorador de formularios** (Forms Explorer Windows), en esta se muestra el formulario activo con el cual se esta trabajando en la aplicación.

Existen otros elementos tales como: **La ventana editor de códigos** (Code Editor) y **la ventana depurador** (Debugger Windows) para ver valores en variables en tiempo de ejecución. Todo este conjunto de herramientas y de ventanas es lo que se llama Entorno Integrado de Desarrollo o IDE (Integrated Development Environment).

- 2.3.1 La barra de menús

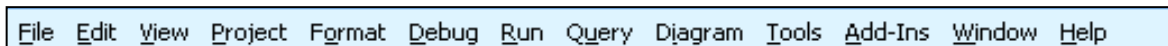


Figura 2.2. La barra de menús de Visual Basic 6.0.

La barra de menús de Visual Basic 6.0 resulta muy similar a la de cualquier otra aplicación de Windows, tal y como se puede apreciar en la *Figura 2.2*. Algunos de los menús de esta barra tienen muy poca novedades, es decir, algunos incluyen las opciones típicas de los menús de cualquier aplicación de Windows.

El menú **File** tiene pocas novedades. Lo más importante es la distinción entre *proyectos*. Un *proyecto* reúne y organiza todos los ficheros que componen el programa o aplicación. Estos ficheros pueden ser *formulario*, *módulos*, *clases*, *recursos*, etc. *Visual Basic 6.0* permite tener más de un proyecto abierto simultáneamente, lo cual puede ser útil en ocasiones. Con el comando **Add Project...** se añade un nuevo proyecto en la ventana **Project Manager**. Con los comandos **Open Project...** o **New Project** se abre o se crea un nuevo proyecto, pero cerrando el o los *proyectos* que estuvieran abiertos previamente. En este menú está el comando **Make ProjectName.exe...**, que permite crear ejecutables de los *proyectos*.

Tampoco el menú **Edit** aporta cambios importantes sobre lo que es lo habitual. Por el contrario el menú **View**, generalmente de poca utilizada, es bastante propio de *Visual Basic 6.0*. Este permite hacer aparecer en pantalla las distintas ventanas del entorno de desarrollo, así como acceder a un *formulario* o al código relacionado con un *control* (que también aparece al hacer doble clic sobre él), y manejar *funciones* y *procedimientos*.

El menú **Project** permite añadir distintos tipos de elementos a un proyecto. Con **Project Properties...** se puede elegir el tipo de proyecto y determinar el formulario con el que se arrancará la aplicación (*Startup Object*). Con el comando **Components** se pueden añadir nuevos *controles* a la *barra de controles* (Toolbox) que aparece a la izquierda de la pantalla.

En menú **Tools** se encuentran los comandos para arrancar el **Menu Editor** y para establecer opciones del programa. En **Tools/Options...** se encuentran una serie de opciones que permiten configurar el **EID de Visual Basic 6.0**. En páginas siguientes veremos con más detalles los elementos que componen la ventana **Options** del **EID de Visual Basic 6.0**.



Por último, la **Ayuda (Help)** (siempre imprescindible y en el caso de **Visual Basic 6.0** particularmente muy bien hecha) que se encuentra en el menú **Help**, se basa fundamentalmente en una clasificación temática ordenada de la información disponible (**Contents**), en una clasificación alfabética de la información (**Index**) y en la búsqueda de información sobre algún tema por el nombre (**Search**).

- 2.3.2 La barra de herramientas estándar



Figura 2.3. La barra de herramientas estándar de Visual Basic 6.0.

La **Barra de Herramientas Estándar** aparece debajo de la barra de menús. Esta permite acceder a las opciones más importantes de los menús de *Visual Basic*. En *Visual Basic 6.0* existen cuatro barras de herramientas que son: *Debug*, *Edit*, *FormEditor* y *Estándar*, por defecto sólo aparece la **barra de herramientas estándar**. Haciendo clic con el botón derecho del mouse sobre cualquier parte de la *barra de herramientas*, aparece un menú contextual con el que se puede hacer aparecer y ocultar cualquiera de las barras. Esta barra posee algunos elementos típicos de Windows como: *nuevo*, *abrir*, *guardar*, *copiar*, *cortar*, *pegar*, *buscar*, *deshacer* y *rehacer*, aunque también posee elementos que son exclusivos del **EID** de *Visual Basic*.

Esta barra posee dos elementos que son propios del **EID** de Visual Basic. El primer elemento marcado en rojo, representa el *indicador de posición* de los *controles* en la aplicación. Cuando el programador mueve un *control* sobre el *formulario* en modo de diseño podrá observar que los valores del indicador cambian. El primer valor, indica la distancia entre el borde *izquierdo* (Left) del contenedor y el borde interno izquierdo del *control* y, el segundo valor, indica la distancia entre la parte superior del contenedor y el borde interno *superior* (Top) del *control*.

El segundo elemento marcado en rojo, representa el *indicador de tamaño* de los *controles* en la aplicación. El primer valor indica la **Altura (Height)** de un control seleccionado en la aplicación y, el segundo valor, indica la **Anchura (Width)** del control. Estos valores se representan en una unidad de medida llamada **Twips**. Un *Twips* es una unidad independiente de la pantalla utilizada para asegurar que la colocación y la proporción de los elementos de la pantalla de la aplicación son los mismos en todos los sistemas de pantallas. Un *Twip* es una unidad de medida de la pantalla igual a 1/20 de un punto de impresora. Existen aproximadamente 1,440 *Twips* en una pulgada lógica y 567 *Twips* en un centímetro.

- 2.3.3 La caja de herramientas (Toolbox)



Figura 2.4. Toolbox.

La *Figura 2.4* muestra la *caja de herramientas* (Toolbox), que incluye los *controles* con los que se puede diseñar la pantalla de la aplicación. Estos *controles* son por ejemplo, botones de comando, etiquetas, cajas de texto, imágenes, etc. Si usted no ve la caja de herramientas, entonces, haga clic en el menú **View (Ver)**, y seleccione, la opción **Toolbox (Caja de herramientas)**.

Para introducir un *control* en un *formulario*, simplemente hay que hacer clic con el botón izquierdo del *mouse* sobre el *control* deseado y arrastrarlo hacia el formulario, o bien, haciendo doble clic sobre el *control* de la *caja de herramientas*.

El número de *controles* que pueden aparecer en esta ventana varían con la configuración del sistema, pero usted puede agregar nuevos componente a la caja de controles. Para agregar nuevos componentes se utiliza el comando **Components...** del menú **Project**. Otra forma de activar la ventana **Componentes**, es haciendo un clic derecho sobre cualquier parte de la *caja de herramientas*, y seleccionar la opción **Componentes** del menú contextual.



- 2.3.4 Los formularios (Forms)

Los *formularios* son las zonas de la pantalla sobre las que se diseña el programa y sobre los que se sitúan los *controles* o *herramientas* del **ToolBox**. Al ejecutar el programa, el *Formulario* se convertirá en la ventana principal de la aplicación, donde aparecerán los botones, las cajas de texto, los gráficos, etc. En *Visual Basic 6.0* existen dos tipos de formularios que son: *formulario estándar* y *formulario de múltiple interfaz de documento (MDI)*. En este apartado solo tratamos con los *formularios estándar*. La *Figura 2.5* muestra *formulario* típico de *Visual Basic 6.0*.

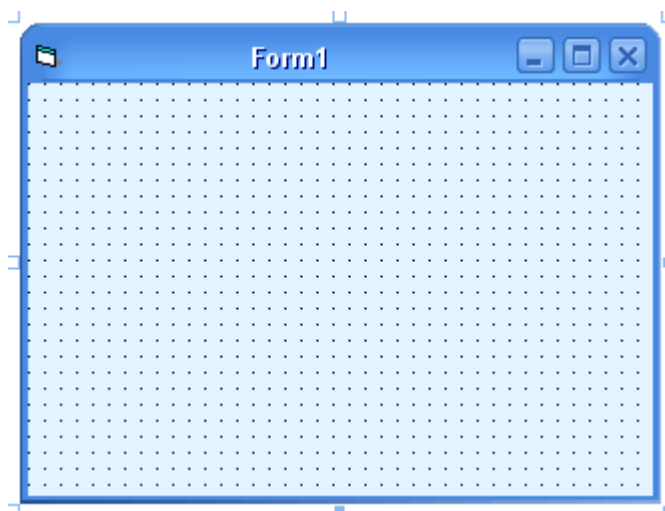


Figura 2.5. Formulario de Visual Basic 6.0.

El conjunto de puntos que aparecen sobre el formulario se llama *mall* o *retícula (grid)* que permite alinear los *controles* manualmente de una forma precisa, evitando tener que introducir coordenadas continuamente. Esta *mall* (grid) sólo será visible en el proceso de diseño del programa; al ejecutarlo la *mall* automáticamente desaparece.

Un *formulario* de *Visual Basic* posee todas las características de una ventana de Windows. Si usted tiene instalado Windows XP o algún Skin que modifique la apariencia de las ventanas de Windows, verá también esos cambios en los *formularios* de *Visual Basic*. Puede ver que el *formulario* que se muestra en la *Figura 2.5* hereda las características de Windows XP.

En *modo de diseño* usted puede modificar el tamaño de las ventanas especificando sus respectivas propiedades de altura y anchura. También puede modificarlo con unas pequeñas asas que aparecen de forma cuadrada , cuando el *formulario* tiene el enfoque.

- 2.3.5 El explorador de proyectos (Project Explorer)

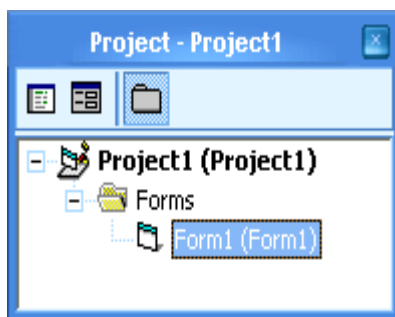
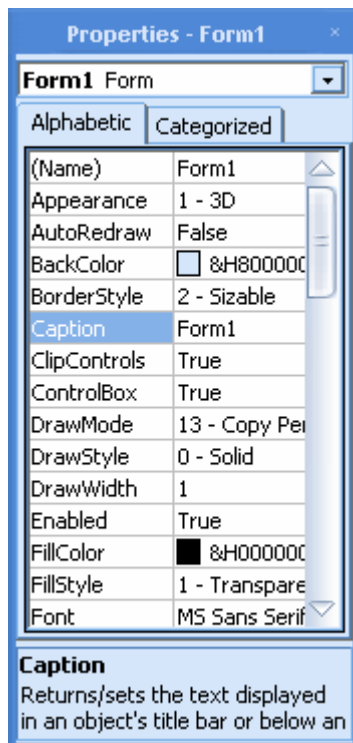


Figura 2.6. Explorador de proyectos.

El *explorador de proyectos* visualiza lo distintos *formularios* y *módulos* que componen un proyecto. Estos *módulos* están representados por un icono que lo diferencia de los demás *módulos*. Para visualizar uno de estos *módulos* o *formularios*, solo debe hacer doble click en el icono que lo representa, o bien, hacer click en el botón **View Object** (Ver Objeto) , que se encuentra en la parte superior del explorador de proyectos. También puede ver el código fuente de un *módulo* seleccionado, haciendo click en el botón **View Code** (Ver Código)  del *explorador de proyectos*. Si usted no ve el *explorador de proyectos*, entonces, pulse la combinación de teclas **Ctrl+R**.



- 2.3.6 La ventana de propiedades (Properties Windows)



En la ventana de propiedades se muestran todas las propiedades de un *control* o un *módulo* seleccionado en la aplicación. Mediante esta ventana es posible cambiar los valores de las *propiedades* de cada uno de los *controles*, indicando así, sus *características* y *estados* antes de ejecutar la aplicación.

A medida que se van seleccionando los *controles* en la aplicación, se puede apreciar, que las *propiedades* que antes mostraba un *tipo de control* desaparecen y se muestran las *propiedades* del *control* que ha sido seleccionado actualmente. La cantidad de *propiedades* que aparecen en la *ventana de propiedades*, depende de las *características* propias de ese *control*.

En la parte superior de esta ventana se puede apreciar una *caja combinada*, que permite seleccionar cada uno de los *controles* que se encuentran en la aplicación. También posee dos solapas, una de ellas permite ordenar alfabéticamente las *propiedades* de los *Controles* y la otra permite organizarla por categoría. En la parte inferior se encuentra un recuadro que describe la función de una propiedad seleccionada.

En caso de no ver esta ventana en el **EID** de *Visual Basic*, pulse la tecla **[F4]**, o bien, haga clic en **Properties Windows** del menú **View (Ver)**.

Figura 2.7. Ventana de propiedades.

- 2.3.7 La ventana de esquema de formularios (Form Layout Window)

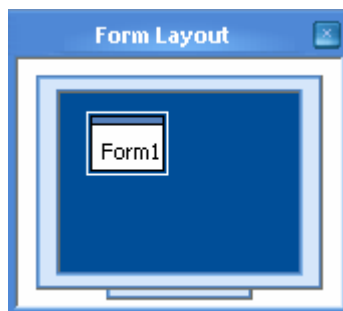
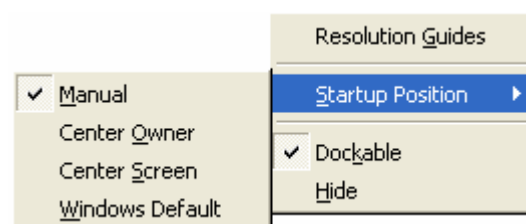


Figura 2.8. Form Layout Window.

Esta ventana permite observar y alinear en la pantalla cada uno de los *formularios* de una aplicación, es decir, permite indicar la posición donde debe de aparecer cada uno de los *formularios* cuando la aplicación este corriendo o se este ejecutando.

Para indicar la posición de un *formulario* este debe seleccionarse manteniendo pulsado el botón izquierdo del mouse y ubicarlo en la posición deseada de la pantalla que aparece en la *ventana de distribución de formularios*. También puede indicar la posición inicial del formulario de la siguiente manera: Desde la *Ventana de esquema de formularios* haga un clic derecho sobre el formulario al cual quiere establecer una posición inicial y, a continuación, ubique el puntero del mouse en la opción **Startup Posición (Posición Inicial)**. Haga clic sobre una de las posiciones que aparecen en el submenú contextual.





- 2.3.8 La ventana explorador de formulario (Form Explorer Window)

La *ventana explorador de formulario* (Form Explorer Windows) muestra el *formulario* con el cual se está trabajando en la aplicación. Esta ventana aparece por defecto cada vez que se carga una aplicación y es la única ventana que permite al usuario interactuar con los *formularios* y *controles* que se encuentran en la aplicación. La siguiente figura muestra la *ventana explorador de formularios*:

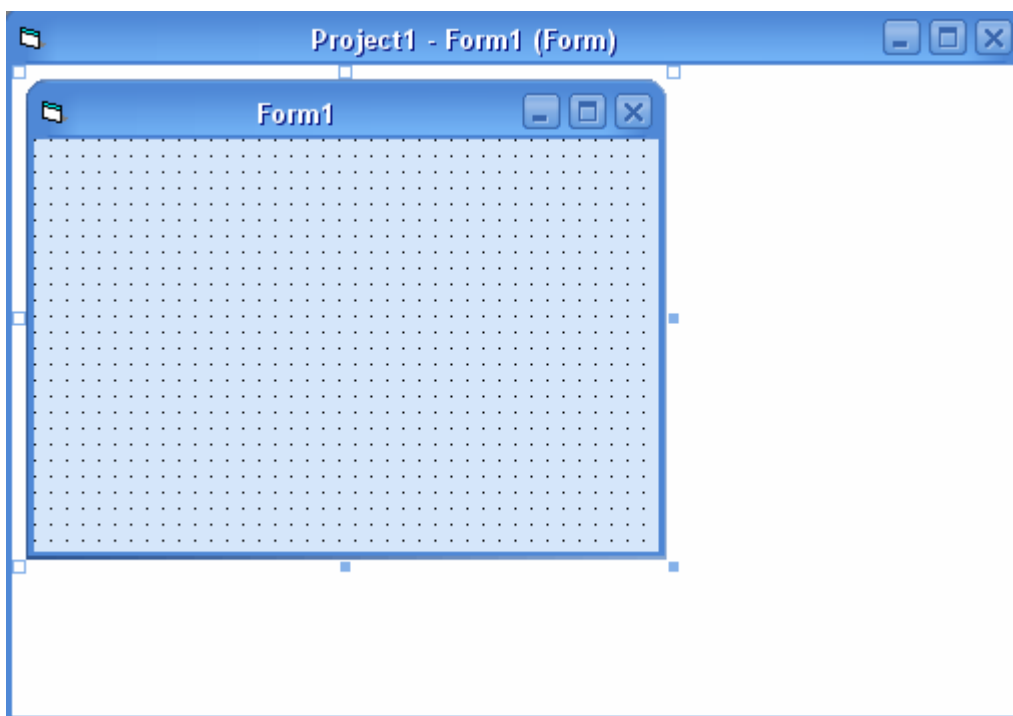


Figura 2.9. La ventana explorador de formularios.

En la parte superior de la *ventana explorador de formularios*, se especifica el nombre del proyecto seguido del nombre del *formulario* activo que en este caso es Project1 – Form1 (Form). Dentro de esta ventana se puede apreciar el *formulario* activo con el cual se está trabajando en la aplicación.

2.4 La ayuda de MSDN Library

MSDN, Microsoft Developer Network, es la referencia esencial para programadores que utilizan las herramientas de desarrollo de Microsoft o tienen como objetivo cualquier plataforma de Internet o Windows. *MSDN Library* contiene más de 1,1 GB de información de programación técnica, que incluye código de ejemplo, Developer Knowledge Base, la documentación *Visual Studio*, la documentación de SDK, artículos técnicos, anotaciones de seminarios y conferencias, y especificaciones técnicas.

Esta herramienta es muy importante en el momento que se está programando en cualquier plataforma de *Visual Studio*, debido a que es posible obtener cualquier tipo de información, ya sea de un comando, de algún mensaje de error o de un tema de interés en específico. *MSDN Library* proporciona toda la ayuda necesaria referente a cada uno de los productos de *Visual Studio*. *MSDN Library* tiene la característica de permitir seleccionar información de un producto en específico, independientemente de los demás productos de *Visual Studio*. A continuación se muestra la ventana de ayuda de *MSDN Library*:

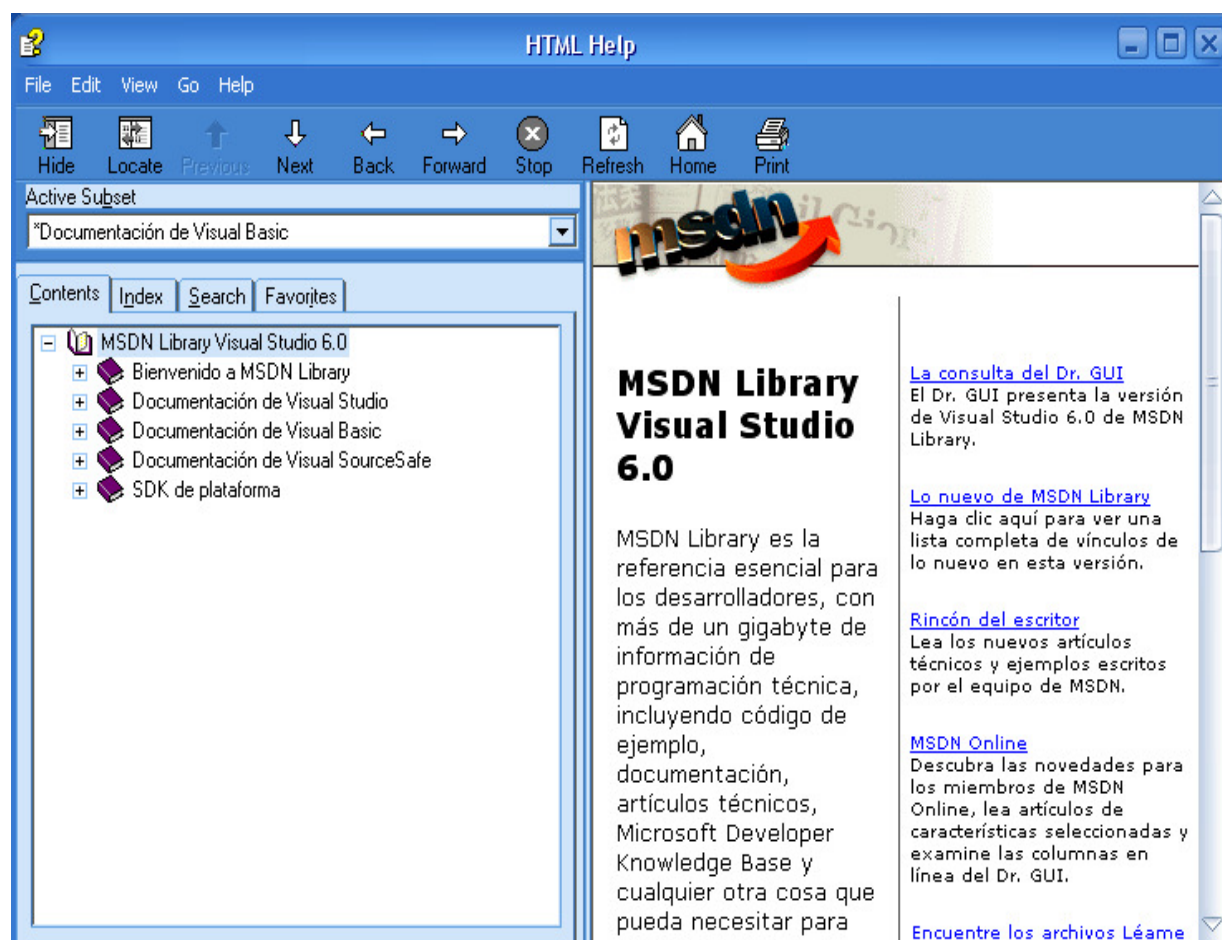


Figura 2.10. La ventana de ayuda de MSDN Library.

- 2.4.1 Instalación de MSDN Library

Para instalar el *MSDN Library* inserte el CD en su unidad de CD-ROM. Durante el programa de instalación se le pedirá que elija entre las opciones de instalación Personalizada, Completa y Típica. La opción de configuración Típica permite ejecutar *MSDN Library* desde el CD. El programa de instalación copiará el conjunto mínimo de archivos al disco duro local. Estos archivos incluyen los archivos de sistema del visor MSDN, los archivos del índice de contenido y los archivos de Ayuda utilizados por los productos de desarrollo de *Visual Studio*.

La instalación personalizada permite especificar la instalación local de un subconjunto de *MSDN Library*. Se copiarán al disco duro local los archivos que seleccione y los archivos mencionados en la descripción de la Instalación típica. Todavía seguirá viendo la tabla de contenido de toda la biblioteca. Si selecciona un contenido que no se instaló localmente, se le pedirá que inserte el CD de *MSDN Library*.

La instalación completa copia todos los archivos a su disco duro local. Este tipo de instalación tiene la ventaja de no solicitar que se inserte el CD de *MSDN Library* para buscar algún tipo de información, debido a que todos los archivos se copian en su disco duro local.

- 2.4.2 Comprender el visor de MSDN Library

Esta versión de *MSDN Library* se genera mediante el sistema de Ayuda HTML de Microsoft. Los archivos de Ayuda HTML se muestran en una ventana semejante a un explorador, no en la versión



completa de Internet Explorer con todas sus barras de herramientas, listas de favoritos e iconos visibles para el usuario final, sino en una ventana de Ayuda de tres paneles. El panel superior contiene la barra de herramientas, el panel izquierdo contiene los métodos de exploración y el panel derecho muestra el tema, con toda la funcionalidad del explorador intacta. Todo lo que se puede mostrar en Internet Explorer también se puede mostrar en la Ayuda HTML. El panel de exploración contiene las fichas **Contenido**, **Índice**, **Búsqueda** y **Favoritos** como pudimos observar en la *Figura 2.10* de la página anterior. Si hace clic en los temas en la tabla de contenido, el Índice o la lista de Favoritos, puede explorar cualquier información de *MSDN Library*. En la ficha **Búsqueda** puede encontrar cada aparición de una palabra o frase contenida en cualquier tema.

- 2.4.3 Seleccionar la documentación adecuada

Anteriormente, se comentó que una de las características importantes de la ayuda de *MSDN Library* es que permite seleccionar información de un producto en específico de *Visual Studio*. Para seleccionar un tipo de documentación solo debe hacer clic en la caja **Subconjunto Activo** (*Active Subset*) de la parte superior del visor *MSDN Library*, como se muestra en la siguiente figura:

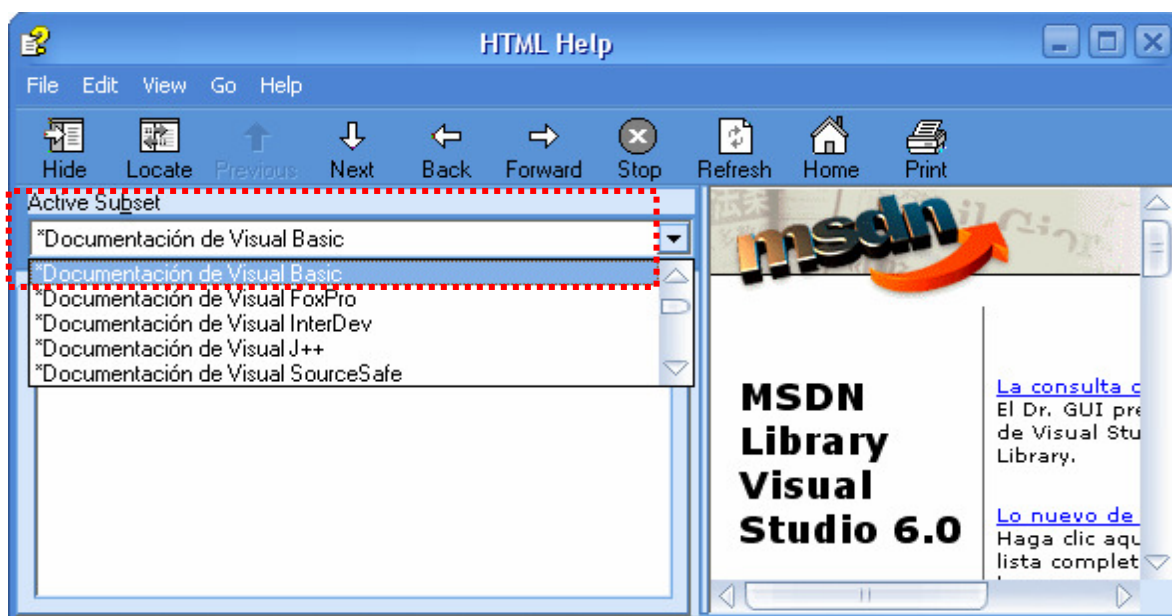


Figura 2.11. Selección del tipo de documentación en la ventana de ayuda de *MSDN Library*.

En la figura anterior se muestra seleccionada la *Documentación de Visual Basic*. Al seleccionar este tipo de documentación usted solo podrá obtener información acerca de *Visual Basic*, pero puede cambiar el tipo de documentación cuantas veces usted desee, o bien, puede seleccionar la opción *toda la colección* para obtener información de todos los productos, en vez de un producto en específico.

Es recomendable que usted configure el tipo de documentación de acuerdo con el producto que usted este trabajando, por ejemplo, si esta trabajando con *Visual Basic*, seleccione la *Documentación de Visual Basic*, de esta manera obtendrá resultados más explícito.

- 2.4.4 Buscar información en el visor de *MSDN Library*

Hay varios métodos para encontrar la información que necesita y muchas maneras de combinar métodos para que la búsqueda sea rápida y eficiente. Los métodos de que dispone el visor de *MSDN Library* son: Tabla de contenido, Índice de palabras clave, Búsqueda de texto completo y Favoritos.



- 2.4.4.1 Explorar mediante la Tabla de contenido

Este tipo de búsqueda permite obtener información examinando cada uno de los temas por título. Es muy útil cuando se quiere obtener información completa y detallada sobre algún tema en específico. Los temas disponibles se muestran en una lista expandible con sus ramificaciones como se muestra en la siguiente figura:

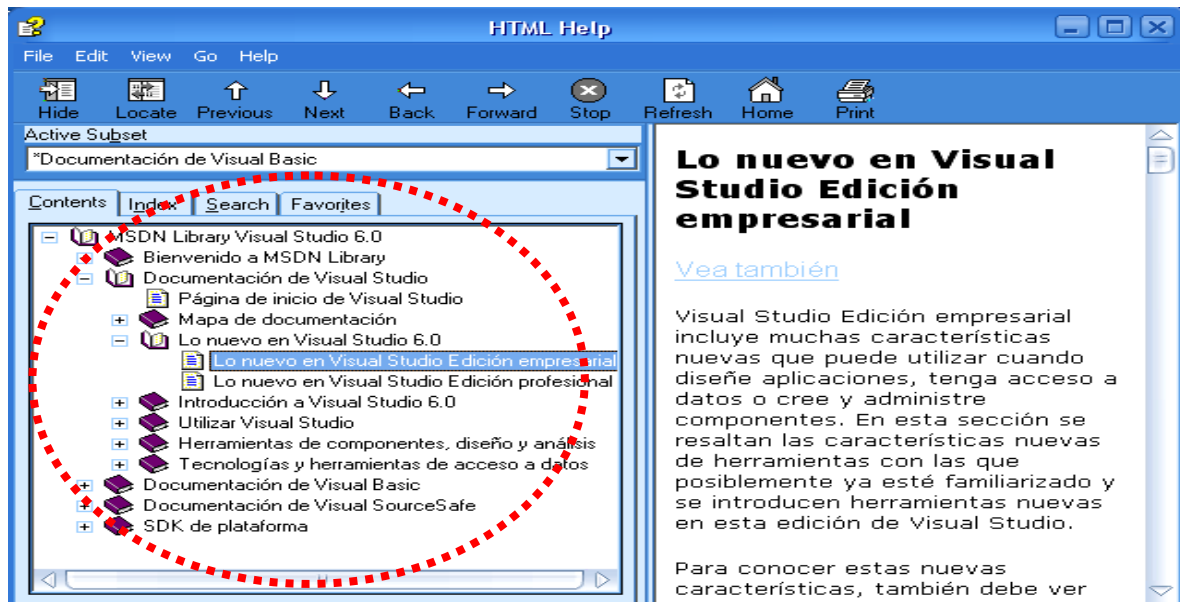


Figura 2.12. Representación gráfica de la tabla de contenido en la ayuda de MSDN Library.

- 2.4.4.2 Encontrar información con el índice

La ficha **Index (Índice)** contiene una lista de palabras claves relacionadas con los muchos de los temas de MSDN Library. Este índice es similar a la de un libro

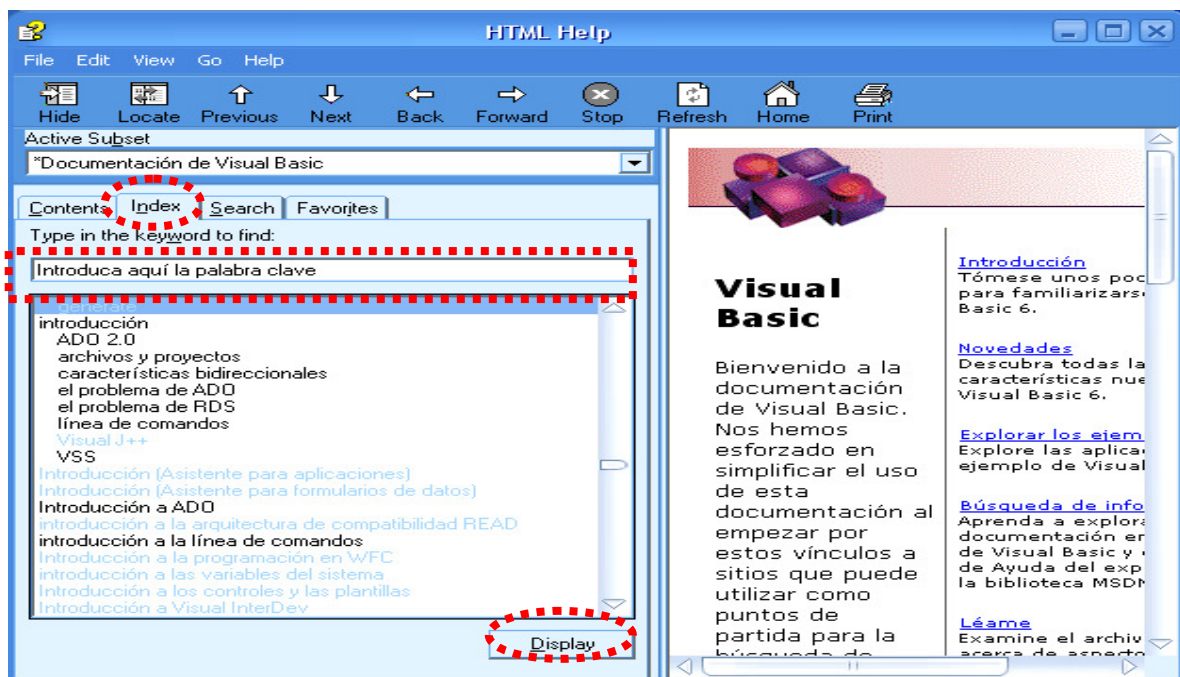


Figura 2.13. Explorar temas mediante la ficha Índice.

Ing. Carlos Manuel Rodríguez Bucarely



Para buscar un tema mediante el índice

1. En el panel de exploración, haga clic en la ficha **Index (Índice)** y después, escriba o seleccione una palabra clave acerca de la que desee encontrar información.
2. Cuando haya seleccionado una palabra clave, haga clic en **Display (Mostrar)**.
3. En la lista de temas encontrados, seleccione el tema que desee y, después, haga clic en **Display (Mostrar)**.

- 2.4.4.3 Encontrar información con la búsqueda de texto completo

Una búsqueda básica de temas se compone de la palabra o frase que desea encontrar. Puede utilizar expresiones comodín, expresiones anidadas, operadores booleanos, coincidencias de palabras similares, la lista de resultados anterior o títulos de temas para afinar la búsqueda.

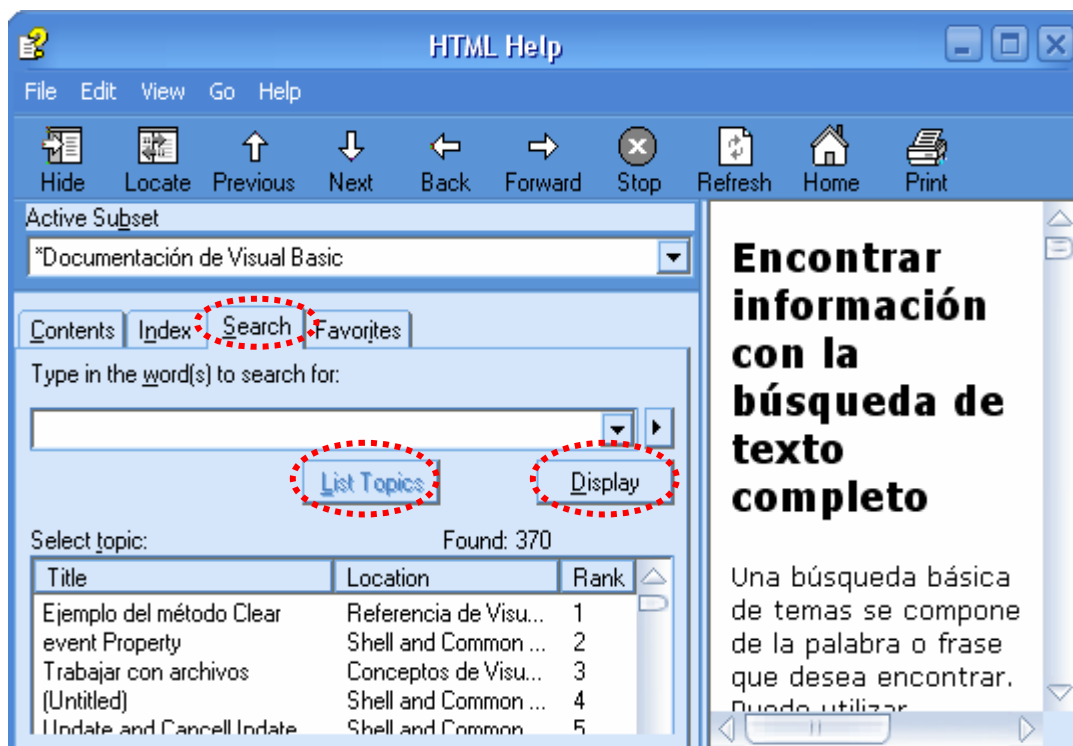


Figura 2.14. Búsqueda de información mediante texto completo.

Para realizar una búsqueda de texto completo haga lo siguiente:

1. En el panel de exploración, haga clic en la ficha **Search (Búsqueda)** y, después, escriba la palabra o frase que desee encontrar. Utilice el botón de flecha a la derecha para agregar operadores booleanos a la búsqueda.
2. Haga clic en **List Topics (Mostrar temas)**. La búsqueda devolverá las primeras 500 coincidencias encontradas.
3. Resalte el tema que desee y, después, haga clic en **Display (Mostrar)**. (Opcionalmente, puede mostrar cualquiera si hace doble clic en él.)



- 2.4.4.4 Crear una lista de temas favoritos

Para crear una lista de temas favoritos realice los siguientes pasos:

1. En el panel de exploración, haga clic en la ficha **Contents (Contenido)**, **Index (Índice)** o **Search (Búsqueda)**, y abra un tema que desee convertir en tema favorito.
2. Haga clic en la ficha **Favorites (Favoritos)**. El tema que abrió en el paso 1 se mostrará en el área **Topics (Tema actual)**.
3. Haga clic en **Add (Agregar)** para agregar el tema a su lista de favoritos.

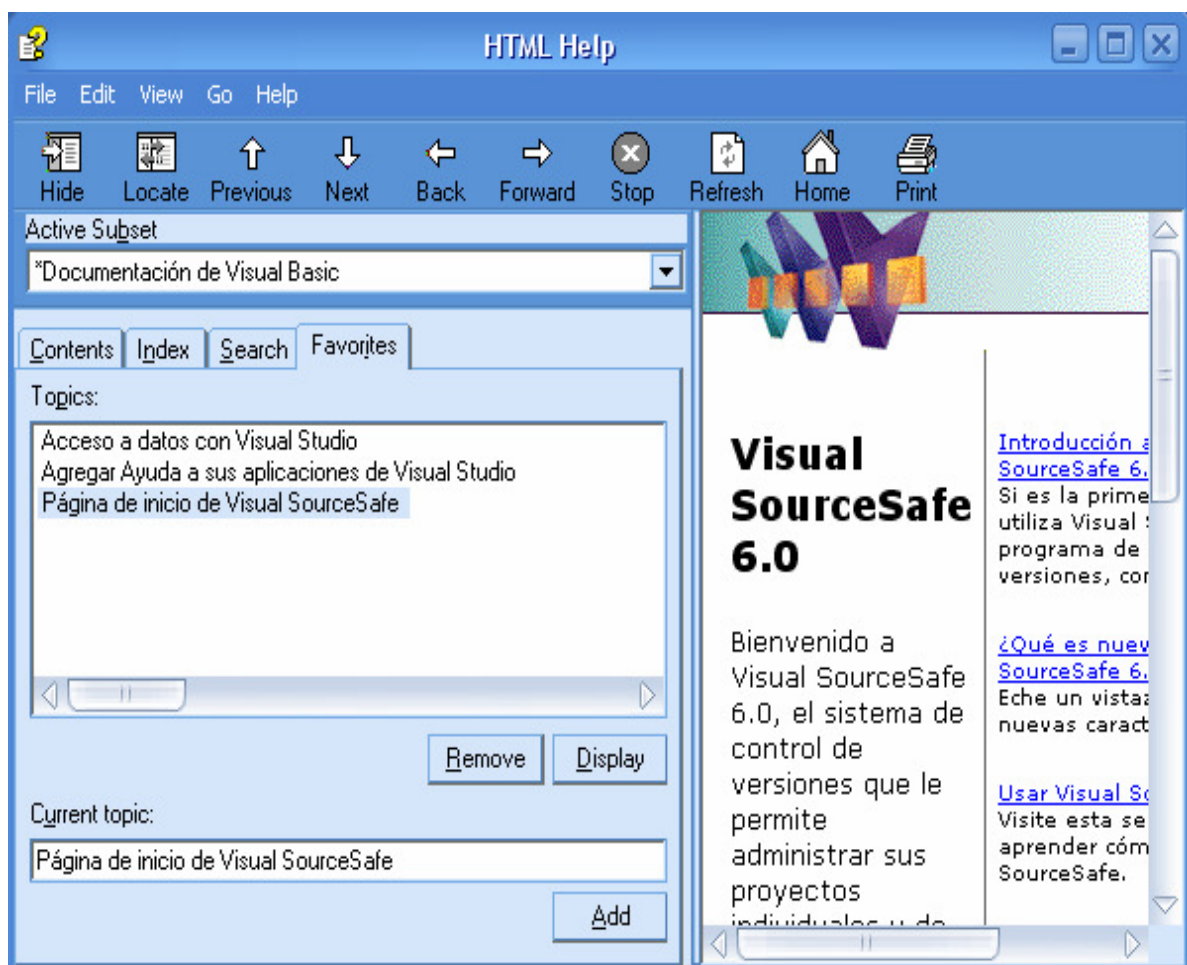


Figura 2.15. Agregar temas a la lista de favoritos.

Para volver a un tema favorito:

- Resalte el tema y, después, haga clic en **Display (Mostrar)**, o bien, haga doble clic en el nombre del tema.

Para quitar un tema favorito:

- Seleccione el tema y, después, haga clic en **Remove (Quitar)**.



2.5 El Editor de Código (Code Editor)

El **Editor de Código** de *Visual Basic 6.0* es la ventana en la cual se escriben las sentencias de los *procedimientos* y *módulos* de la aplicación. Esta ventana se activa de formas diferentes, una de las principales es haciendo doble clic sobre un *formulario* o *sobre* cualquiera de sus *controles*. También existen otras formas que son muy poco habituales. Una de ellas es seleccionar la opción **Code (Código)**, del menú **View (Ver)**, o bien, haciendo clic en el botón **View Code (Ver Código)**, del **Explorador de Proyectos**. La siguiente figura muestra el aspecto físico del *Editor de Código*:

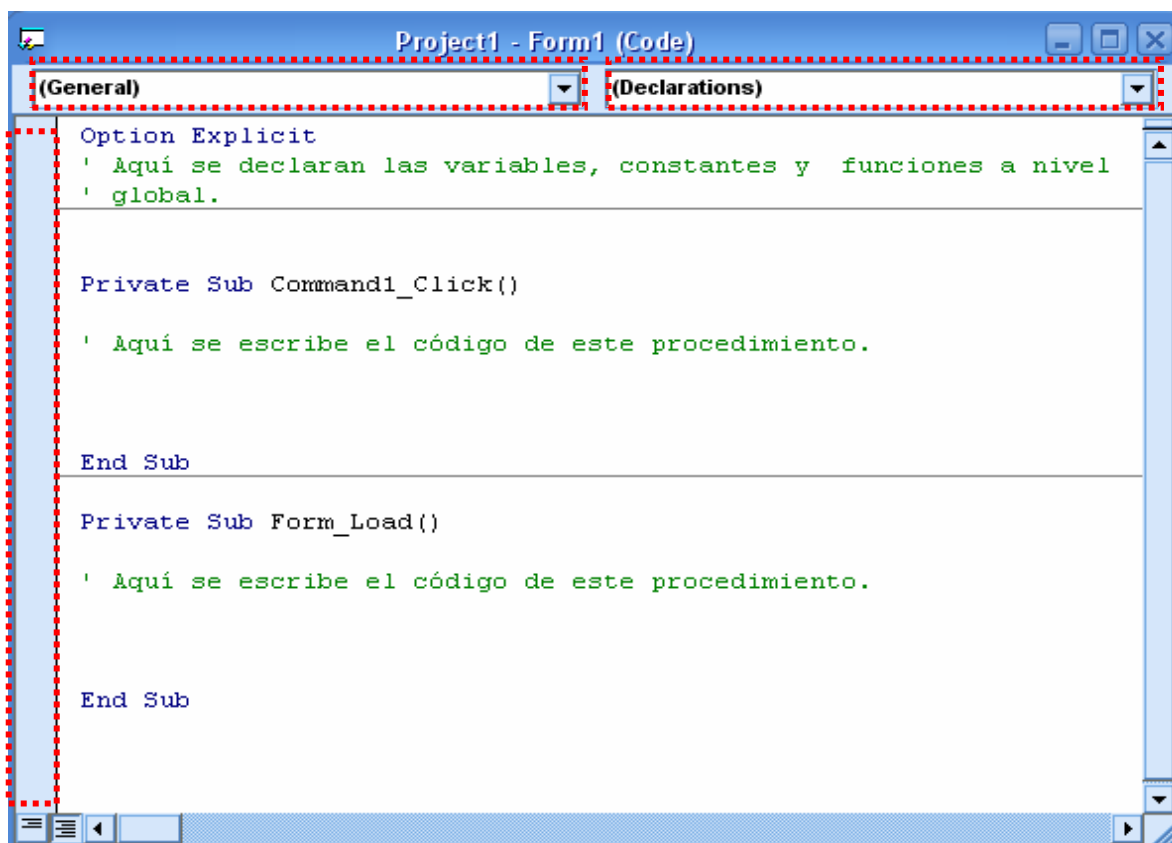


Figura 2.16. Ventana del Editor de Código.

- 2.5.1 Características del Editor de Código (Code Editor)

Esta ventana posee algunos elementos y características que son importantes conocer. El primer recuadro marcado con rojo representa una lista desplegable que posee todos los *controles* que se encuentran en el *formulario*. Puede ver el código de cualquier *control* del *formulario* haciendo clic sobre el *nombre del control* cuando la lista ha sido desplegada. El segundo recuadro representa otra lista desplegable que posee los *eventos* de un *control* seleccionado. Puede cambiar de evento cada vez que sea necesario haciendo clic sobre el *nombre del evento* cuando el *control* este seleccionado. Un *control* esta seleccionado en el *Editor de Código* cuando el cursor se encuentra parpadeando dentro del *procedimiento*. El tercer recuadro de forma vertical representa el *margen izquierdo* del *Editor de Código*. En este se marcan los *Breakpoints (puntos de parada)* en las líneas de códigos de Visual Basic.

Una de las características más destacadas en el *Editor de Código* es la utilización de colores para hacer diferencia entre las líneas de código. El código escrito por el programador aparece en negro, las palabras claves o sentencias aparecen en **azul**, los comentarios en **verde**, los errores en **rojo**, etc. Esta característica juega un papel importante en el *Editor de Código*, ya que permite detectar y corregir problemas con mayor facilidad.



En la parte superior de la *Figura 2.16* encontramos la declaración **Option Explicit** que obliga al programador a declarar todas las variables que vaya a utilizar en la aplicación. Si utiliza una variable que no ha sido declarada, entonces, el programa abortará con un mensaje de error en *modo de ejecución*.

También encontramos dos *procedimientos*, uno para el **Evento Click** de un botón de comando y otro para el **Evento Load** de un *formulario*. Dentro de estos dos *procedimientos* se ha utilizado algunos *comentarios* que aparecen de color verde para indicar que en esta zona se escribe el código del *procedimiento*. Los *comentarios* son parte del código fuente, pero no tienen ningún valor al momento de compilación, es decir, los errores ortográficos y los caracteres utilizados después de la comilla simple (') o de la palabra clave **Rem** no tienen ninguna validez. Los comentarios se utilizan únicamente para incluir notas aclaratorias en un programa.

En esta ventana aparecen dos barras de desplazamiento, una vertical y otra horizontal. Estas barras permiten visualizar el código escrito por el programador cuando este sobrepasa el límite del tamaño de la ventana del *Editor de Código*. Estas barras de desplazamiento son similares a las barras del explorador de Internet.

- 2.5.2 Personalizar el Editor de Código (Code Editor)

Visual Basic 6.0 posee una herramienta que permite al usuario personalizar el *Editor de Código*. Puede cambiar totalmente la apariencia que tendrá el código de *Visual Basic*. También esta herramienta permite configurar gran parte del **EID** de *Visual Basic* de una forma muy sencilla.

Puede personalizar el *Editor de Código* realizando los siguientes pasos:

1. Haga clic en el menú **Tools (Herramientas)**.
2. Seleccione el comando **Options... (Opciones...)**.
3. Seleccione la ficha **Editor Format (Formato del editor)**.
4. Aparecerá la siguiente configuración:

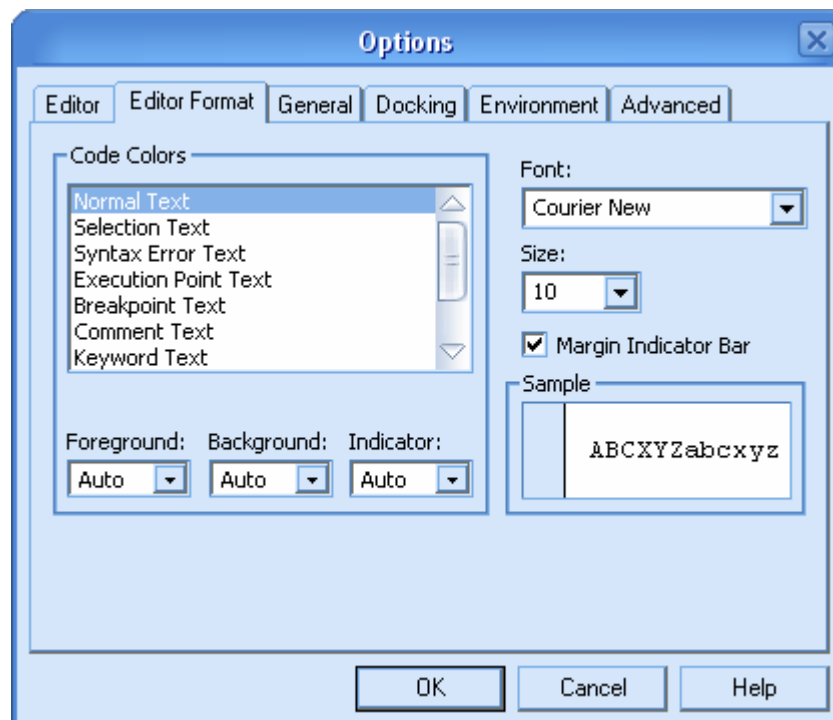


Figura 2.17. Ventana de opciones.



En esta ventana puede observar los siguientes elementos:

1. Colores de código (Code Colors)

Determina los colores de primer plano y de fondo utilizados para el tipo de texto seleccionado en el cuadro de lista.

- **Lista de texto:** enumera los elementos de texto que tienen colores que se pueden personalizar.
- **Foreground (Primer plano):** especifica el color de primer plano para el texto seleccionado en **Lista de color de texto**.
- **Background (Fondo):** especifica el color de fondo para el texto seleccionado en la **Lista de color de texto**.
- **Indicador (Indicator):** especifica el color del indicador de margen.

2. Fuente (Font)

Especifica la fuente utilizada para todo el código.

3. Tamaño

Especifica el tamaño de la fuente utilizada para todo el código.

4. Barra indicadora al margen

Hace que la barra indicadora de margen sea visible o no.

5. Ejemplo

Muestra un texto de ejemplo con la fuente, tamaño y colores seleccionados.

2.6 El Depurador (Debugger)

El **Depurador** es una herramienta utilizada para la corrección y detección de errores en la aplicación. Hoy en día la mayoría de los entornos de programación poseen potentes herramientas que facilitan la depuración de los programas realizados. La característica principal del *Depurador* es que permite ejecutar parcialmente el programa, deteniendo la ejecución en una línea de código determinada. También permite ver valores de *variables* y *procesos* en tiempo de ejecución. De esta manera es posible detectar la fuente principal de los errores en la aplicación.

Visual Basic 6.0 posee una barra de herramienta destinada a la depuración de los programas. Esta barra se muestra en la siguiente figura:

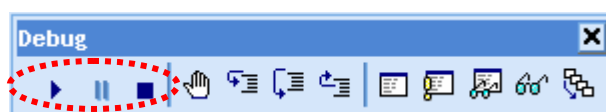


Figura 2.18. Barra de herramienta de depuración.

Esta barra posee 12 botones cada uno de los cuales tiene una función específica en el proceso de depuración. Los primeros tres botones que aparecen marcados en rojo, se utilizan para ejecutar, detener y finalizar una aplicación. El botón **Start (Ejecutar)** ►, se utiliza para poner en ejecución una



aplicación, es decir, pasar del *modo de diseño* al *modo de ejecución*. El botón **Pausar (Break)** , se utiliza para detener temporalmente la ejecución de un programa. El botón **Finalizar (End)** , se utiliza para cancelar o finalizar la ejecución un programa. Los demás botones se detallan en las próximas secciones.

- 2.6.1 Depurar utilizando puntos de parada (Breakpoints).

Para ejecutar parcialmente un programa se pueden utilizar varias formas. Una de ellas consiste en incluir **Breakpoints** en determinadas líneas de código. Los *Breakpoints* se indican con un punto grueso delante de la línea de código y un cambio de color, tal como se ve en la *Figura 2.19*. El colocar un *Breakpoint* en una línea de código implica que la ejecución del programa se detendrá al llegar a esa línea. Para insertar *Breakpoints* debe posicionar el cursor en la línea de código que desea colocar el Breakpoint y, a continuación, ejecutar una de estas acciones:

1. Haga click en el botón **Toggle Breakpoint** , de la barra de herramientas de depuración.
2. Seleccione el comando **Toggle Breakpoint** del menú **Debug (Depurar)**.
3. Utilice el método abreviado pulsando la tecla **[F9]**.
4. Haga clic sobre el margen izquierdo del *Editor de Código*.

Cualquiera de estos pasos enumerados anteriormente, se utilizan para insertar y quitar *Breakpoint* en una línea de código. Puede utilizar el más ergonómico que es pulsando la tecla **[F9]**.

- 2.6.2 Depurar paso a paso por instrucciones (Step Into)

Este método ejecuta el código instrucción a instrucción. Ejecuta la instrucción en el punto de ejecución actual. Si la instrucción llama a un *procedimiento*, la siguiente instrucción que se muestra es la primera instrucción del *procedimiento*.

Para depurar utilizando este método realice una de las siguientes acciones:

- 1.- Haga clic en el botón **Step Into** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Step Into** del menú **Debug (Depurar)**.
- 3.- Utilice el método abreviado pulsando la tecla **[F8]**.

- 2.6.3 Depurar paso a paso por procedimientos (Step Over)

Es similar a *Paso a paso por instrucciones*. La diferencia en el uso es evidente cuando la instrucción actual contiene una llamada a un procedimiento. *Paso a paso por procedimientos* ejecuta el procedimiento como una unidad y después pasa a la siguiente instrucción del procedimiento actual. Por tanto, la siguiente instrucción mostrada es la próxima instrucción del procedimiento actual, independientemente de si la instrucción actual es una llamada a otro procedimiento. Sólo está disponible en modo de interrupción.

Para depurar utilizando este método realice una de las siguientes acciones:

- 1.- Haga clic en el botón **Step Over** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Step Over** del menú **Debug (Depurar)**.
- 3.- Utilice el método abreviado pulsando **Mayús-[F8]**.

- 2.6.4 Depurar paso a paso para salir (Step Out)

Ejecuta las demás líneas de una función en la que reside el punto de ejecución actual. La siguiente



Capítulo 2

instrucción mostrada es la instrucción que sigue a la llamada a *procedimiento*. Todo el código se ejecuta entre los puntos de ejecución actual y final. Sólo está disponible en modo de interrupción.

Para depurar utilizando este método realice una de las siguientes acciones:

- 1.- Haga clic en el botón **Step Out** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Step Out** del menú **Debug (Depurar)**.
- 3.- Utilice el método abreviado pulsando **Ctrl-Mayús-[F8]**.

- 2.6.5 Ventana de Locales (Locals Window)

La **Ventana de Locales** presenta automáticamente todas las { CONTROL Internet.HHCtrl.1 }variables de la pila actual y sus valores cuando la aplicación esta corriendo en *modo de depuración* o pasa a *modo de interrupción*. La *Ventana de Locales* se actualiza automáticamente cada vez que cambia de { CONTROL Internet.HHCtrl.1 }modo de ejecución a { CONTROL Internet.HHCtrl.1 }modo de interrupción y cada vez que cambia el contexto de pila.

A continuación, se muestra un procedimiento codificado para sumar dos valores contenidos en dos variables, donde cada variable tiene un valor inicial. También se ha insertado un *Breakpoint* para interrumpir el *procedimiento* y así poder observar los valores de cada variable en la *Ventana de Locales*.

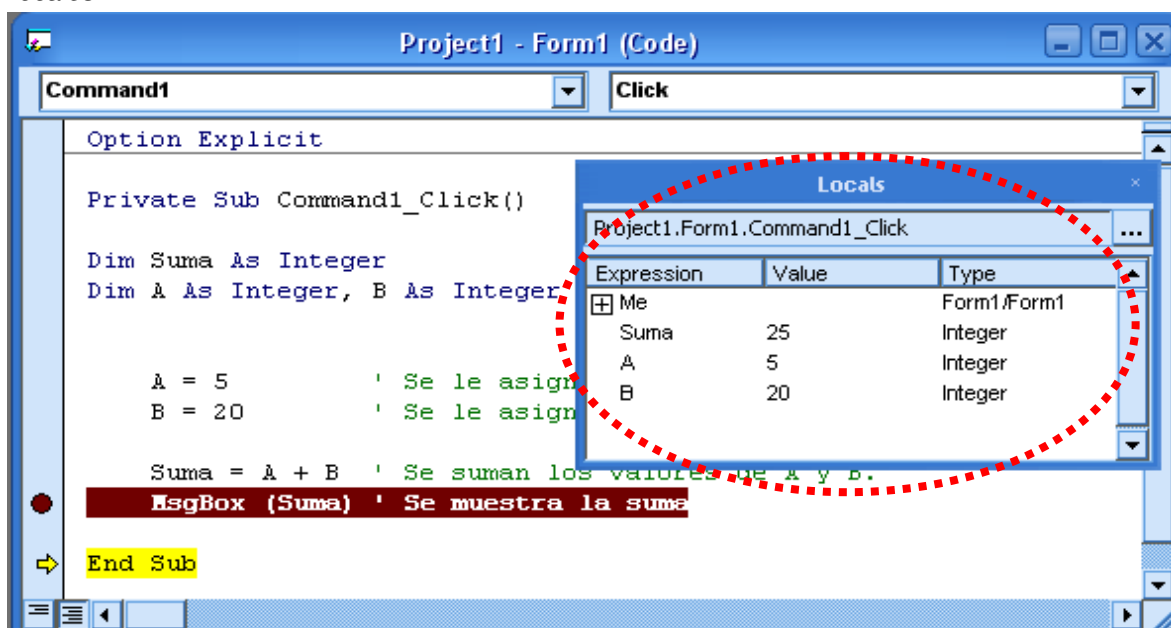


Figura 2.19. Ventana Locales.

En la *Figura 2.18* puede observar que en la parte superior de la ventana *Locales* existen tres campos que son: **Expression (Expresión)**, **Value (Valor)** y **Type (Tipo)**. En el campo **Expresión** se listan las variables declaradas en el *procedimiento*, en el campo **Valor** se listan los valores actuales de cada variable y, en el campo **Tipo** se muestra el tipo de datos que pueden almacenar las variables.

También se puede observar que la variable **Suma** obtiene el valor 25 como resultado del proceso **Suma = A + B**. Donde **A** tiene asignado el valor 5 y **B** el valor 20.

Para activar la ventana *Locales* siga uno de estos pasos:

- 1.- Haga clic en el botón **Locals Window** , de la barra de herramientas de depuración.

Ing. Carlos Manuel Rodríguez Bucarely



2.- Seleccione el comando **Locals Window** del menú **View (Ver)**.

- 2.6.6 Ventana Inmediato (Immediate Window)

La **Ventana Inmediato** muestra información resultante de las instrucciones de depuración del código o de los comandos introducidos directamente en la ventana. Está disponible en { **CONTROL Internet.HHCtrl.1** } modo de interrupción.

La *Ventana Inmediato* permite:

- Probar código problemático o que se haya escrito recientemente.
- Consultar o cambiar el valor de una variable mientras se ejecuta una aplicación. Mientras la ejecución se detiene, asigne a las variables un nuevo valor igual que se haría en el código.
- Consultar o cambiar el valor de una propiedad mientras se ejecuta una aplicación.
- Llamar a *procedimientos* exactamente de la misma forma a como se haría en el código.
- Ver los resultados de la depuración durante la ejecución del programa.

Para activar la *Ventana Inmediato* siga uno de estos pasos:

- 1.- Haga clic en el botón **Immediate Window** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Immediate Window** del menú **View (Ver)**.
- 3.- Utilice el método abreviado pulsando **Ctrl-G**.

- 2.6.7 Ventana Inspección (Watch Window)

La **Ventana Inspección** muestra las expresiones de inspección actuales. La **Ventana Inspección** aparece automáticamente si las *expresiones de inspección* se definen en el proyecto.

Si el contexto de la expresión no está en { **CONTROL Internet.HHCtrl.1** } alcance cuando se entra en el { **CONTROL Internet.HHCtrl.1** } modo de interrupción, el valor actual no se muestra.

Para activar la *Ventana Inspección* siga uno de estos pasos:

- 1.- Haga clic en el botón **Watch Window** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Watch Window** del menú **View (Ver)**.

- 2.6.8 Inspección rápida (Quick Watch)

Muestra el cuadro de diálogo **Inspección rápida** con el valor actual de la expresión seleccionada. Sólo disponible en *modo de interrupción*. Utilice este comando para comprobar el valor actual de una variable, propiedad, u otra expresión para la que no haya definido una *expresión de inspección*. Seleccione la expresión de la ventana Código o de la ventana Inmediato y después elija el comando **Inspección rápida**. Para agregar una *expresión de inspección* basada en la expresión en el cuadro de diálogo **Inspección rápida**, elija el botón **Agregar**.

Para activar el cuadro de diálogo *Inspección rápida* siga uno de estos pasos:

- 1.- Haga clic en el botón **Quick Watch** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Quick Watch** del menú **Debug (Depurar)**.



3.- Utilice el método abreviado pulsando **Mayús-F9**.

- 2.6.9 Pila de llamadas (Call Stack)

El cuadro de diálogo **Pila de llamadas**, enumera los procedimientos que se llaman en la aplicación. Solo los procedimientos que han iniciado pero no terminado son los que se muestran en el cuadro de diálogo **Pila de llamadas**. Esta opción está disponible únicamente en { *CONTROL Internet.HHCtrl.1* } modo de interrupción.

Para activar el cuadro de dialogo *Pila de llamadas* siga uno de estos pasos:

- 1.- Haga clic en el botón **Call Stack** , de la barra de herramientas de depuración.
- 2.- Seleccione el comando **Call Stack** del menú **View (Ver)**.
- 3.- Utilice el método abreviado pulsando **Ctrl-L**.

2.7 PREGUNTAS PARA CONTESTAR

I.- Conteste:

1. ¿Qué es Visual Basic?
2. ¿Qué es un Entorno Integrado de Desarrollo?
3. ¿Cuál es el lenguaje de programación que da evolución a Visual Basic?
4. Diga cuantas versiones de Visual Basic ha lanzado la Microsoft.

II.- Hable brevemente de la historia de Visual Basic.

III.- Habla del EID de Visual Basic 6.0 y mencione cada uno de sus elementos.

IV.- Conteste:

1. ¿Cuál es la diferencia que existe entre la barra de menús del EID de Visual Basic 6.0 y la barra de menús estándar?
2. Hable de la barra de herramientas estándar.
3. ¿Qué son los formularios?
4. ¿Qué es la malla o retícula?
5. ¿Qué es del Explorador de Proyectos?
6. ¿Qué es la Ventana de Propiedades?
7. ¿Para que se utiliza la ventana de esquema de formulario?
8. ¿Qué es la ventana de Explorador de Formularios?
9. Hable de la ayuda de MSDN Library.
10. Diga los pasos para instalar la ayuda de MSDN Library.
11. Explique porque es recomendable seleccionar la documentación adecuada.
12. Explique las distintas formas de buscar información en la ventana de ayuda de MSDN Library.
13. Diga los pasos para crear una lista de temas favoritos.
14. Hable del Editor de Código.
15. Mencione las características del Editor de Código.
16. ¿Qué es el Depurador?
17. Mencione los distintos modos de depuración.
18. Hable de la Ventana de Locales y la Ventana Inspección.
19. ¿En que consiste la Inspección Rápida?
20. Hable del cuadro de dialogo Pila de llamadas.



CONTENIDO

3.1 Los controles más usuales en Visual Basic 6.0.

- 3.1.1 Los botones de comando (CommandButton).
 - 3.1.1.1 Propiedades de los botones de comando.
 - 3.1.1.2 Uso de las propiedades en el Editor de Código.
 - 3.1.1.3 Eventos sobre los botones de comando.
 - 3.1.1.4 Métodos de los botones de comando.
 - 3.1.1.5 Ejercicios prácticos.
- 3.2.1 Las etiquetas (Labels).
 - 3.2.1.1 Propiedades de las etiquetas.
 - 3.2.1.2 Eventos sobre las etiquetas.
 - 3.2.1.3 Ejercicios propuestos.
- 3.3.1 Las cajas de texto (TextBox).
 - 3.3.1.1 Propiedades de las cajas de texto.
 - 3.3.1.2 Eventos sobre las cajas de texto.
 - 3.3.1.3 Métodos de las cajas de texto.
 - 3.3.1.4 Ejercicios prácticos.
- 3.4.1 Los botones de opción (OptionButton).
 - 3.4.1.1 Propiedades de las botones de opción.
 - 3.4.1.2 Eventos sobre los botones de opción.
 - 3.4.1.3 Métodos de los botones de opción.
 - 3.4.1.4 Ejercicios prácticos.
- 3.5.1 Las cajas de verificación (CheckBox).
 - 3.5.1.1 Propiedades de las cajas de verificación.
 - 3.5.1.2 Eventos sobre las cajas de verificación.
 - 3.5.1.3 Métodos de las cajas de verificación.
 - 3.5.1.4 Ejercicios prácticos.
- 3.6.1 Las barras de desplazamiento (ScrollBar).
 - 3.6.1.1 Propiedades de las barras de desplazamiento.
 - 3.6.1.2 Eventos sobre las barras de desplazamiento.
 - 3.6.1.3 Ejercicios prácticos.
- 3.7.1 Las cajas de lista (ListBox).
 - 3.7.1.1 Propiedades de las cajas de lista.
 - 3.7.1.2 Eventos sobre las cajas de lista.
 - 3.7.1.3 Métodos de las cajas de lista.
 - 3.7.1.4 Ejercicios prácticos.
 - 3.7.1.5 Ejercicios propuestos.
- 3.8.1 Las cajas combinadas (ComboBox).
 - 3.8.1.1 Ejercicios prácticos.
- 3.9.1 El control tiempo (Timer).
 - 3.9.1.1 Propiedades del control tiempo.
 - 3.9.1.2 Ejercicios prácticos.
- 3.10.1 Controles relacionados con ficheros (FileList, DirList y DriveList).
 - 3.10.1.1 Conectar los controles de ficheros.
 - 3.10.1.2 Ejercicios prácticos.

3.2 El control de cuadros de diálogo (CommonDialog).

- 3.2.1 Los cuadros de dialogo Abrir y Guardar (Open/Save).
 - 3.2.1.1 Ejercicios prácticos.
- 3.2.2 El cuadro de diálogo Imprimir (Print).
 - 3.2.2.1 Ejercicios prácticos.
- 3.2.3 El cuadro de diálogo Fuente (Font).
 - 3.2.3.1 Ejercicios prácticos.
- 3.2.4 El cuadro de diálogo Color.



- 3.2.4.1 Ejercicios prácticos.

3.3 Arreglo en los controles.

3.4 Figura con los controles más usuales de Visual Basic 6.0.



3.1 Los controles más usuales en Visual Basic 6.0

Los *controles* más usuales son aquellos que usamos con mayor frecuencia al momento de crear una aplicación. Estos *controles* son por ejemplo: *botones de comando*, *botones de opción*, *cajas de texto*, *etiquetas*, *barras de desplazamiento*, *listas*, *cajas combinadas*, *botones de verificación*, etc.

- 3.1.1 Los botones de comando (CommandButton)

 Los *botones de comando* son aquellos botones típicos que vemos frecuentemente en las aplicaciones de Windows, que realizan una operación en específico, por ejemplo, salir, imprimir, cancelar, etc.

Los *botones de comando* se utilizan principalmente para iniciar, interrumpir o terminar un proceso. Estos controles pueden ser activados de múltiples formas como son: Haciendo clic sobre el, presionando la tecla barra espaciadora o la tecla ENTRAR cuando este tiene el *enfoque*, o bien, pulsando desde el teclado su respectivo *acceso directo*.

Estos botones poseen una gran cantidad de *propiedades*, *métodos* y *eventos* que definiremos a continuación. Es importante recordar que todos los valores de las *propiedades* de un *control* seleccionado en la aplicación pueden ser observados y modificados desde la *ventana de propiedades*.

- 3.1.1.1 Propiedades de los botones de comando

Antes de definir cada una de las *propiedades* de los *botones de comando*, ejecute *Visual Basic 6.0*, y, a continuación, seleccione la opción **Standard EXE** de la ventana **New Project (Nuevo Proyecto)**, y haga clic en el botón **Open (Abrir)**.

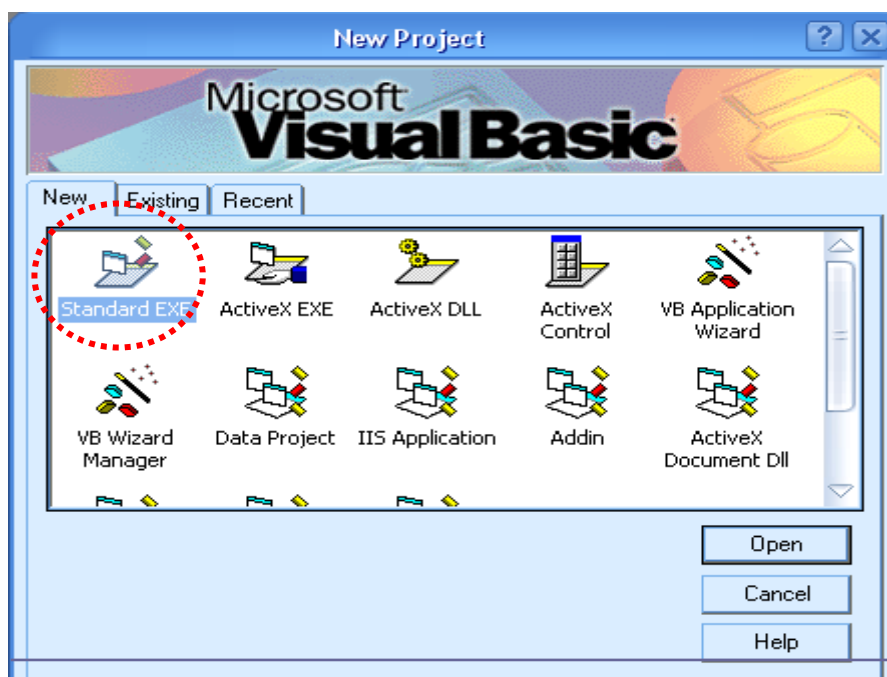


Figura 3.1. Ventana de Nuevo Proyecto.

Si ya había iniciado *Visual Basic*, entonces, haga clic en el menú **File (Archivo)** y seleccione la opción **New Project (Nuevo Proyecto)**. Después de haber iniciado un nuevo proyecto, inserte un **botón de comando (Command)** en el *formulario*, tal y como se ve en la siguiente figura:



Capítulo 3

Si ya había iniciado *Visual Basic*, entonces, haga clic en el menú **File (Archivo)** y seleccione la opción **New Project (Nuevo Proyecto)**. Después de haber iniciado un nuevo proyecto, inserte un **botón de comando (Command)** en el *formulario*, tal y como se ve en la siguiente figura:

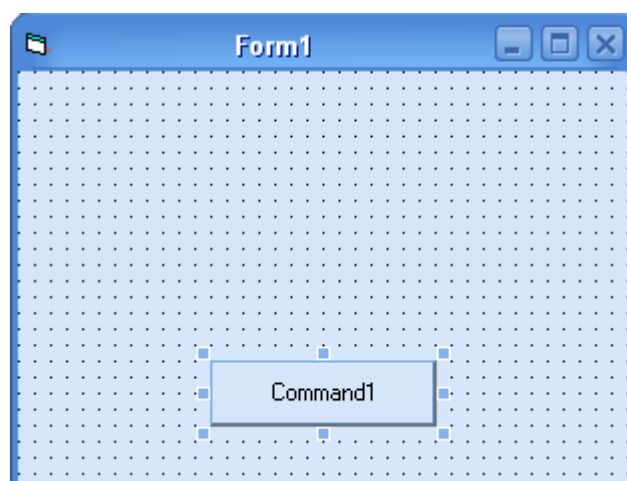


Figura 3.2. Botón de comando.

NOTA: Recuerde que para insertar un *control* de la *caja de controles*, solo debe mantener presionado el botón izquierdo del mouse sobre el *control* y luego arrastrarlo hasta el *formulario*. También puede hacer doble clic sobre el *control*, y este quedará ubicado en el centro del *formulario*.

Ahora observe en la *ventana de propiedades*, todas las *propiedades* que posee un *botón de comando*. Podrá observar *propiedades* tales como: **Name**, **Appearance**, **BackColor**, **Cancel**, **Caption**, *etc.* Como ha podido notar, un *botón de comando* posee un gran número de *propiedades*, pero en este libro solo trabajaremos con las *propiedades* más importantes de cada *control*.

Propiedad	Descripción
Name	Se utiliza para asignarle el nombre al <i>control</i>. Este nombre va a permitir identificar el <i>control</i> de los demás <i>controles</i> de la aplicación. Esta <i>propiedad</i> solo esta disponible en <i>modo de diseño</i>. Los nombres de objetos (object name) se clasifican en dos tipos que son: <i>nombres por defectos</i> y <i>nombres definidos por el usuario</i>. Los <i>nombres por defecto</i> son aquellos que <i>Visual Basic</i> les asigna cuando son incluidos como nuevos objetos en una aplicación, es decir, cuando son añadidos por primera vez a un proyecto o un formulario. Estos vienen acompañados por un entero único que representa el número de orden con el cual se han ido insertando objetos de un mismo tipo. Por ejemplo, un nuevo objeto Formulario (Form) será Form1, un objeto MDIForm nuevo será MDIForm1 y el cuarto control TextBox que cree en un formulario será Text4. Por otro lado, los <i>nombres definidos por el usuario</i> son nombres especificados por el usuario y escritos cómodamente para diferenciarlos de otros objetos de la aplicación.
Ejercicio: - Verifique que el <i>botón de comando</i> este seleccionado. Cuando el botón esta seleccionado aparecen unos pequeños cuadros llamados asas, tal como se muestra en la <i>Figura 3.2</i>. Estas asas permiten reducir y aumentar el tamaño del botón. - Busque la propiedad (Name) (<i>la primera propiedad de la ventana de propiedades</i>). Cambie el <i>nombre por defecto</i> Command1 y escriba cmdSalir y, a continuación, presione la tecla [ENTER]. Este será ahora el nuevo nombre del <i>botón de comando</i>.	

Ing. Carlos Manuel Rodríguez Bucarely



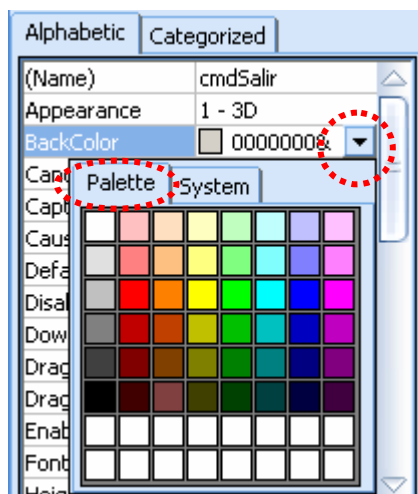
Capítulo 3

Al asignar el nombre (**Name**) al *botón de comando*, usted no notará ningún cambio en el botón. Este nombre será utilizado para identificar el *botón de comando* de los demás *controles* de la aplicación.

BackColor	Cambia el color del <i>botón de comando</i> . Para que el botón tome el color seleccionado en la propiedad BackColor , usted debe establecer el valor “1- Graphical ” en la propiedad Style del <i>botón de comando</i> .
------------------	--

Ejercicio:

- Busque la propiedad **Style** del botón de comando y seleccione el valor “1 - **Graphical**”.
- Seleccione la propiedad **BackColor** y luego haga clic en el botón *fecha abajo* .
- Seleccione la ficha **Palette (Paleta)**.



- Aparecerá la paleta de colores de Windows.
- Seleccione el color que quiere aplicar al *botón de comando*.
- Puede observar como el *botón de comando* cambia de color.

NOTA 1: El *botón de comando* solo cambiará de color si ha seleccionado el valor “1 - **Graphical**” en la propiedad **Style**.

NOTA 2: Usted puede utilizar los colores predeterminados del sistema de Windows haciendo clic en la ficha **System (Sistema)**.

COMENTARIO: Cuando usted ha seleccionado un color en la propiedad **BackColor** un valor hexadecimal se escribe en la caja de esta propiedad. Este valor puede ser seleccionado y copiado como texto con el mouse para utilizarse desde código en el *Editor de Código*.

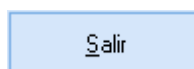
3.3. Activación de la propiedad BackColor.

Caption	Establece el texto que aparece escrito sobre el objeto, en este caso sobre el <i>botón de comando</i> . Si utiliza el carácter (&) ampersand delante de cualquier carácter del texto escrito en la propiedad Caption , entonces, ese carácter va a permitir acceder a la función del botón como si hubiese echo click sobre el, con solo presionar la tecla control (Ctrl) más el carácter que tiene colocado delante el <i>ampersand</i> .
----------------	--

COMENTARIO: Cuando crea un nuevo objeto, su título predeterminado es el valor predeterminado de la propiedad **Name**. Este título predeterminado incluye el nombre de objeto y un entero, como Comando1 o Formulario1. Para obtener una etiqueta más descriptiva, modifique la propiedad **Caption**.

Ejercicio:

- Seleccione la propiedad **Caption** del *botón de comando* y escriba **&Salir**. El *ampersand* utilizado delante de la letra “S”, indica el *acceso directo* al botón. Podrá observar que el texto escrito en el botón aparece con un *underscore* () debajo de la letra “**S**”.



COMENTARIO: Si no encuentra el carácter (&) **ampersand** en su teclado, usted puede obtenerlo mediante los valores **ANSI**, pulsando **ALT + 38**.



Capítulo 3

Enabled	Habilita o deshabilita el objeto, es decir, indica si el objeto responderá a los eventos del usuario. Esta propiedad puede tomar los valores lógicos: True y False. Si el valor de esta propiedad esta en False, entonces, el objeto no responderá a ninguna acción que realice el usuario sobre el. El valor por defecto de esta propiedad es True. COMENTARIO: La propiedad Enabled permite activar o desactivar formularios y controles en <i>tiempo de ejecución</i>. Por ejemplo, puede desactivar objetos que no se aplican al estado actual de la aplicación. También puede desactivar un control empleado fundamentalmente con fines de presentación, como un cuadro de texto que proporciona información de sólo lectura.
Ejercicio: ▪ Seleccione la propiedad Enabled del <i>botón de comando</i>. ▪ Haga clic en el botón flecha abajo , y seleccione el valor lógico False. ▪ Usted no notará ningún cambio en el <i>botón de comando</i> hasta que corra la aplicación. Para correr la aplicación pulse la tecla [F5]. ▪ La apariencia del <i>botón de comando</i> será la siguiente: ▪ Haga clic sobre el <i>botón de comando</i> y notará que el botón no responde a la acción que usted esta realizando sobre el. ▪ Haga clic sobre el botón End (Finalizar) , de la <i>barra de herramientas estándar</i> para finalizar la aplicación, o bien, haga clic en el botón cerrar  de la ventana de la aplicación. ▪ Seleccione la propiedad Enabled y establezca el valor True, para habilitar el <i>botón de comando</i>.	
DisabledPicture	Establece una imagen que aparecerá en el <i>botón de comando</i> cuando este deshabilitado, es decir, cuando la propiedad Enabled este establecida a False. Para que la imagen aparezca en el <i>botón de comando</i> debe establecer el valor "1 - Graphical" en la propiedad Style.
Ejercicio: ▪ Seleccione la propiedad Enabled del <i>botón de comando</i>, y seleccione el valor lógico False. ▪ Seleccione la propiedad Style y seleccione el valor "1-Graphical". ▪ Seleccione la propiedad DisabledPicture y haga clic en los tres puntos suspensivos . ▪ En la ventana que aparece, busque la imagen Img01Cap3.jpg. Esta imagen se encuentra en la carpeta Imágenes del disco que se distribuye con este libro. ▪ Después de haber seleccionado la imagen, haga clic en el botón Open (Abrir). ▪ Usted no notará ningún cambio hasta que corra la aplicación. ▪ Pulse la tecla [F5] para correr la aplicación. ▪ El <i>botón de comando</i> aparecerá con una imagen como se muestra a continuación: ▪ Finalice la aplicación, haciendo clic en el botón Cerrar de la ventana. ▪ Seleccione la propiedad Enabled y establezca el valor True, para habilitar el <i>botón de comando</i>.	



NOTA: Para quitar la imagen establecida en la propiedad **DisabledPicture** haga lo siguiente:

- 1) Seleccione la propiedad **DisabledPicture**.
- 2) Haga clic en la caja blanca que dice **(Bitmap)**.
- 3) Presione la tecla **Supr** o **Delete**.
- 4) Cuando el la caja blanca aparezca **(None)**, entonces, se habrá quitado la imagen.

Font	Permite cambiar el tipo de fuente del texto que aparece en el objeto. El tipo de fuente se refiere a las operaciones o efectos que se pueden efectuar sobre un texto, por ejemplo, tamaño de letra, negrita, cursiva, subrayado, tachado, etc.
-------------	--

Ejercicio:

- Seleccione la propiedad **Font** del *botón de comando*.
- Haga clic en los tres puntos suspensivos .
- Aparecerá la ventana típica de la fuente de Windows.
- Ahora, cambie el tamaño de la letra a 14 y seleccione **Negrita (Bold)** en **Estilo de fuente (Font Style)**.
- Haga clic en el botón **Aceptar (OK)**.
- El *botón de comando* tomará una apariencia como esta:

Salir

COMENTARIO: Esta propiedad no requiere que se establezca el valor “1 – Graphical” en la propiedad **Style**. El tipo de fuente se activa en cualquiera de los valores posibles de la propiedad **Style**.

Height y Width	Permite cambiar la altura y anchura del objeto. Donde Height representa la altura y Width la anchura expresada en <i>Twips</i> (unidad de medida de la pantalla).
-----------------------	---

Ejercicio:

- Seleccione la propiedad **Height** y escriba el valor 600 y, a continuación, pulse la tecla **[ENTER]**.
- Notará que la altura del *botón de comando* ha cambiado.
- Seleccione la propiedad **Width** y escriba el valor 1500 y, a continuación, pulse la tecla **[ENTER]**.
- Notará que ahora la anchura del *botón de comando* también ha cambiado.
- El *botón de comando* tendrá una apariencia como esta:

Salir

NOTA: Los valores de las propiedades **Height** y **Width** cambian cada vez que usted modifica su tamaño por medio de las asas del control.

COMENTARIO: Utilice estas propiedades cada vez que necesite asignar un tamaño en específico a los controles de la aplicación. En ocasiones se dificulta en gran manera aumentar o disminuir el tamaño de un control por medio de sus asas, debido, a que existen tamaños en los controles que no es posible asignar por medio de las asas.

También utilice estas propiedades, cuando quiera tener botones del mismo tamaño en una aplicación. Para hacer esto, solo debe especificar los mismos valores para ambas propiedades.



Capítulo 3

Left y Top	Permite cambiar la posición a la izquierda y superior del <i>control</i> sobre su <i>contenedor</i> (<i>Formulario</i> , <i>PictureBox</i> o <i>Frame</i>). Donde Left indica la posición a la izquierda y Top la posición superior. Cuando el valor de la propiedad Left aumenta, entonces, el control se desplaza hacia la derecha y cuando el valor disminuye, el control se desplaza a la izquierda. Lo mismo pasa con la propiedad Top . Cuando el valor de la propiedad Top aumenta, entonces, el control se desplaza hacia abajo y cuando el valor disminuye, el control se desplaza hacia arriba.
-------------------	--

Ejercicio:

- Seleccione la propiedad **Left** del *botón de comando* y escriba el valor 0.
- El *botón de comando* se desplaza hacia la esquina del formulario.
- Seleccione la propiedad **Top** del *botón de comando* y escriba el valor 0.
- El *botón de comando* se desplaza hacia la esquina superior del formulario, es decir, hacia arriba.



NOTA: Los valores de las propiedades **Left** y **Top** cambian cada vez que el control se mueve con el mouse sobre su contenedor.

Picture	Asigna una imagen (Bitmap) en el objeto. En un <i>botón de comando</i> esta propiedad tendrá efecto siempre y cuando el valor de la propiedad Style este establecido a “1 – Graphical”.
----------------	--

Ejercicio:

- Seleccione la propiedad **Style** y asigne el valor “1 – Graphical”.
- Seleccione la propiedad **Picture** del *botón de comando*.
- Haga clic en los tres puntos suspensivos .
- En la ventana que aparece, busque la imagen **Img02Cap3.jpg**. Esta imagen se encuentra en la carpeta **Imágenes** del disco que se distribuye con este libro.
- Después de haber seleccionado la imagen, haga clic en el botón **Abrir (Open)**.
- El botón tendrá una apariencia como se muestra en la siguiente figura:



NOTA 1: Para quitar la imagen del *botón de comando*, aplique los mismos procedimientos especificados en la propiedad **DisabledPicture**.

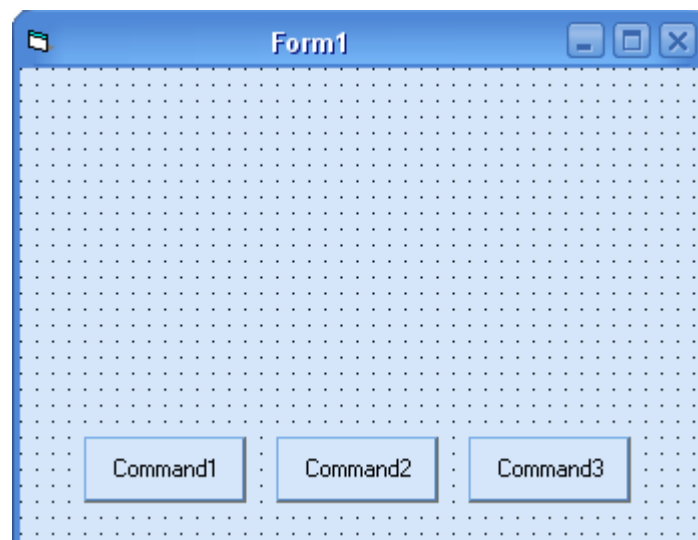
NOTA 2: Algunos formatos de imágenes no son soportados por esta propiedad. Los formatos de imágenes que pueden ser asignados son los de tipo BMP, GIF, JPEG, ICO y metaarchivos (.WMF, .EMF).



TabIndex	Indica el orden o el número de orden a que le corresponde el <i>control</i> recibir el <i>foco</i> o la atención del programa a medida que se pulse la tecla Tab sobre los <i>controles</i> de la aplicación.
-----------------	--

Ejercicio:

- Abra un nuevo proyecto haciendo clic en el menú **File (Archivo)** y seleccione **New Project (Nuevo proyecto)**.
- Si aparece una ventana solicitando guardar el proyecto anterior, entonces, haga clic en **Yes** y, a continuación, escriba el nombre del formulario, por ejemplo, **FormEjercicio1-3**. Después de haber escrito el nombre del formulario, haga clic en el botón **Save (Guardar)**.
- Aparecerá otra ventana solicitando el nombre del proyecto. Escriba **Ejercicio1-3** y, a continuación, haga clic en el botón **Save (Guardar)**.
- Si aparece la ventana **Source Code Control (Control de código fuente)**, entonces, haga clic en el botón **No**.
- En la ventana **New Project (Nuevo Proyecto)**, seleccione la opción **Standard EXE** y, a continuación, haga clic en **OK (Aceptar)**.
- En el formulario inserte tres *botones de comando*, tal y como se muestra en la siguiente figura:



- Pulse la tecla **[F5]** para correr la aplicación.
- Mientras la aplicación este en ejecución, pulse la tecla **Tab** { EMBED PBrush } tres veces.
- Podrá observar que cada vez que pulsa la tecla **Tab**, el *foco* pasa por cada uno de los *botones de comando* de forma secuencial, es decir, en orden. Esto es porque el primer *botón de comando* tiene el valor 0 en la propiedad **TabIndex**, el segundo botón el valor 1 y el tercer botón el valor 2.
- Pare la aplicación, haciendo clic en el botón **Cerrar** de la ventana de la aplicación.

NOTA 1: El orden de tabulación en la propiedad **TabIndex** comienza por el valor 0. Donde 0 representa el primer elemento del orden de tabulación.

COMENTARIO: Utilice siempre la propiedad **TabIndex** en cada una de las ventanas de las aplicaciones que usted realice, debido a que una gran parte de los usuarios utilizan la tecla **Tab** para desplazarse por cada uno de los elementos de una aplicación. Llevar un orden correcto de tabulación es característico de un buen programador.

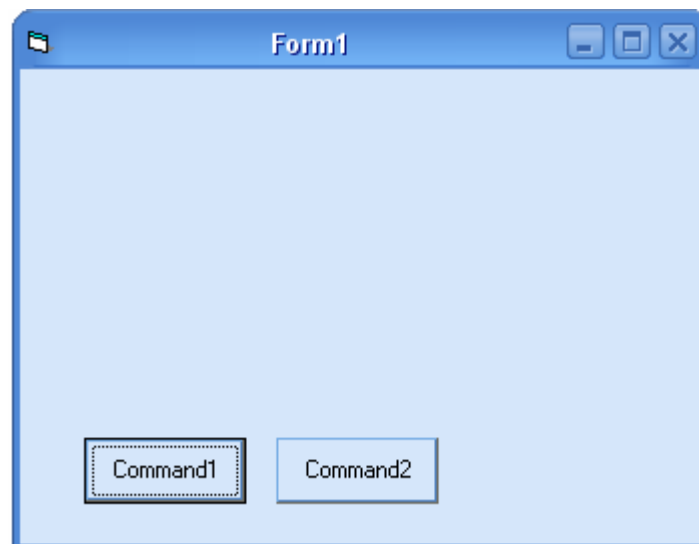


NOTA 2: Cuando usted va a guardar un proyecto, Visual Basic solicita que se especifique el nombre de cada uno de los formularios que haya utilizado en la aplicación, incluyendo el nombre del proyecto. Por ejemplo, si usted utiliza diez formularios, entonces, tendrá que escribir un nombre distinto para los diez formularios y el nombre del proyecto.

Visible	Establece si el <i>control</i> estará Visible cuando se ejecute la aplicación. Si el valor de esta propiedad esta en True (verdadero) , entonces, el <i>botón de comando</i> estará visible, de lo contrario, estará Invisible. Esta propiedad puede tomar los valores lógicos True o False .
----------------	--

Ejercicio:

- Seleccione el tercer *botón de comando* (Command3) del formulario.
- Seleccione la propiedad **Visible** y asigne el valor lógico **False**.
- Pulse la tecla **[F5]** para correr la aplicación.
- Podrá observar que el tercer *botón de comando* no aparece en la aplicación:



- Pare la aplicación, haciendo clic en el botón **Cerrar** de la ventana de la aplicación.
- Seleccione nuevamente el tercer *botón de comando*.
- Busque la propiedad **Visible** y asigne el *valor por defecto* **True**.

NOTA: Los *valores por defecto*, son aquellos valores iniciales asignados a las propiedades de los objetos. Estos valores son especificados por los programadores de objetos en el momento de su construcción.

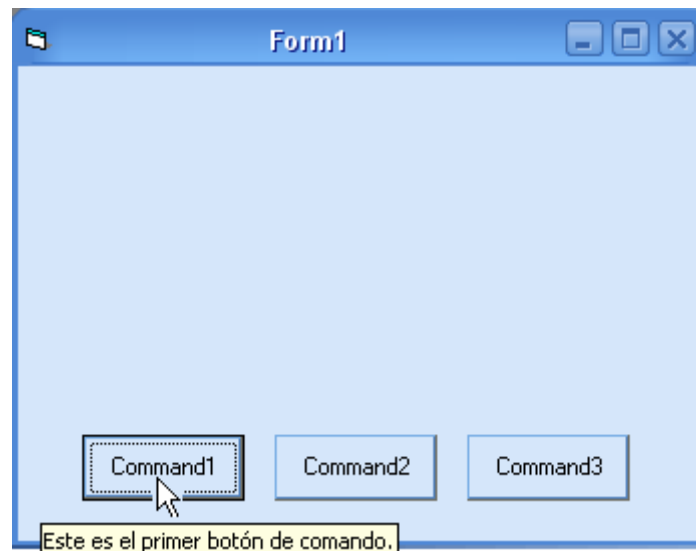
ToolTipText	Se utiliza para mostrar el texto contextual que aparece cuando se coloca el puntero del mouse sobre el <i>objeto</i> o <i>control</i> .
--------------------	---

Ejercicio:

- Seleccione el primer *botón de comando* (Command1) del formulario.
- Seleccione la propiedad **ToolTipText** y escriba: **Este es el primer botón de comando**.
- Seleccione el segundo *botón de comando* (Command2).
- En la propiedad **ToolTipText** escriba: **Este es el segundo botón de comando**.



- Seleccione el tercer *botón de comando* (Command3).
- En la propiedad **ToolTipText** escriba: **Este es el tercer botón de comando**.
- Pulse la tecla **[F5]** para correr la aplicación.
- Coloque el puntero del mouse sobre cada uno de los *botones de comando* y verá el texto escrito en la propiedad **ToolTipText**.



- Pare la aplicación, haciendo clic en el botón **Cerrar** de la ventana de la aplicación.
- Haga clic en el menú **File (Archivo)** y, a continuación, haga clic en **Save Project (Guardar proyecto)**.
- En la ventana que aparece escriba **FormEjercicio2-3** y haga clic en el botón **Save (Guardar)**. En el nombre del proyecto escriba **Ejercicio2-3** y haga clic en el botón **Save (Guardar)**.
- Si aparece la ventana **Source Code Control (Control de código fuente)**, entonces, haga clic en el botón **No**.
- Inicie un nuevo proyecto, haciendo clic en **New Project (Nuevo proyecto)** del menú **File (Archivo)**.
- En la ventana **New Project (Nuevo Proyecto)**, seleccione la opción **Standard EXE** y, a continuación, haga clic en **OK (Aceptar)**.

COMENTARIO 1: La propiedad **ToolTipText** se utiliza principalmente en barras de menús para indicar la función de las imágenes presentadas en la barra.

COMENTARIO 2: No se recomienda utilizar la propiedad **ToolTipText** en *botones de comando* que tenga textos explicativos, por ejemplo, un botón para salir que tenga como texto **Salir**, un botón de imprimir que tenga por texto **Imprimir**, etc. Si utiliza esta propiedad en estos botones con textos explicativo, solo hágalo para expresar ayuda adicional acerca del botón, por ejemplo, pulse ALT-S para salir.

También puede utilizar esta propiedad en *botones de comando* que estén representado por imágenes, por ejemplo, un botón que tenga una imagen de una impresora. En este caso se podría escribir en la propiedad **ToolTipText** el texto: **Imprimir**.



- 3.1.1.2 Uso de las propiedades en el Editor de Código

En capítulos anteriores comentamos que la mayoría de las *propiedades* pueden cambiarse en *modo de diseño* y que también casi siempre en *modo de ejecución*. Con mucha frecuencia el programador necesitará cambiar el *estado de un objeto* por medio de sus *propiedades* en *modo de ejecución*. Esto se hace dentro del *procedimiento* de un *objeto* especificando el *nombre del objeto*, seguido de un punto (.) y el nombre de la *propiedad* y el valor o el nuevo *estado del objeto* separado por el signo de igualdad (=). Por ejemplo, si tenemos dos *botones de comando* en una aplicación uno para salir con el texto **Salir** y el otro para cancelar con el texto **Cancelar**. Queremos que el texto se muestre cuando la aplicación comience a correr y no cuando la estemos diseñando, entonces, tendríamos que programar dentro del *procedimiento* del evento **Load (Carga)** del *formulario*. Para realizar este ejemplo, haga lo siguiente:

- En un nuevo proyecto, inserte dos *botones de comando*.
- Haga doble clic en cualquier parte del *formulario* (**no sobre los botones**).
- Aparecerá la ventana **Code Editor** con el cursor parpadeando dentro del *procedimiento Load* del *formulario*.
- Dentro de este *procedimiento* usted va a escribir estas dos líneas de código:

```
Command1.Caption = "&Salir"  
Command2.Caption = "&Cancelar"
```

- La siguiente figura ilustra esta codificación:

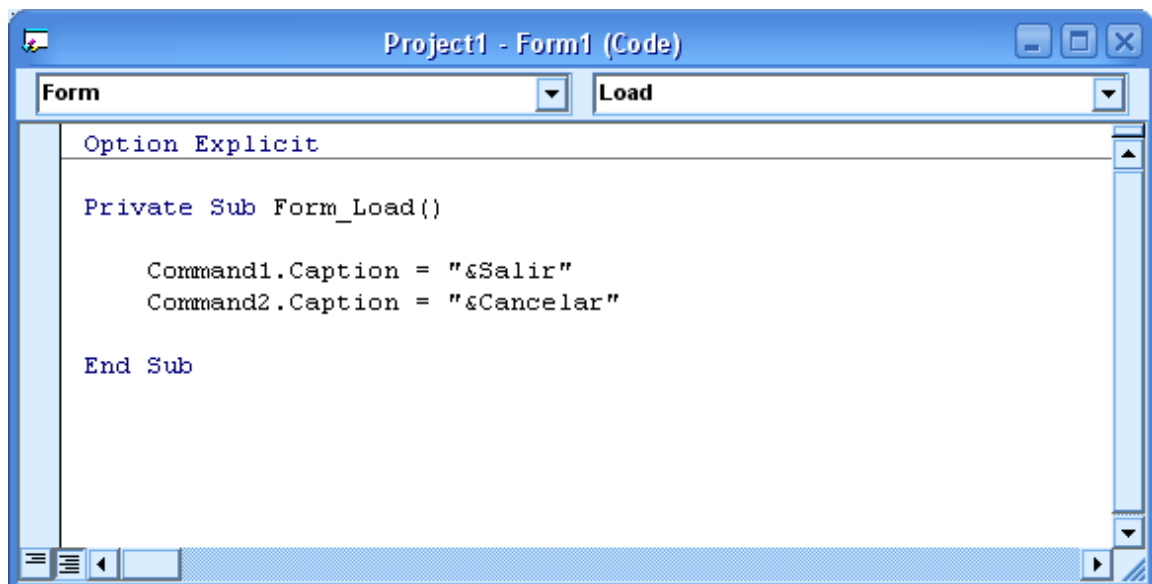


Figura 3.4. Evento **Load (Carga)** de un formulario.

NOTA: Después de escribir el **nombre del objeto** y luego digitar el **punto**, automáticamente aparece una ventana emergente que te permite explorar todas las propiedades de dicho objeto. Si esta ventana no aparece cuando usted digita el **punto**, entonces, el nombre del objeto no está bien escrito en la ventana del *Editor de Código* o simplemente, no existe en el formulario.

COMENTARIO: Utilice siempre el evento **Load** de los formularios siempre y cuando quiera realizar alguna operación antes de iniciar la aplicación. Más adelante estudiaremos más acerca de los eventos de los formularios.



- Pulse la tecla **[F5]** para correr la aplicación.

Ambos botones aparecen con el texto escrito como si se hubiese especificado en la propiedad **Caption** en *modo de diseño*. Esto es, porque se ha codificado dentro del *procedimiento* del *evento Load* del *formulario*. El evento **Load (Carga)** indica que la codificación dentro de ese *procedimiento* debe ser evaluada o ejecutada mientras cargue el *formulario*. Este *evento* ejecuta el código antes que el *formulario* se muestre en pantalla. Si ocurre algún error el *formulario* no se muestra.

- Pare la aplicación, haciendo clic en el botón **Cerrar** de la ventana de la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio3-3** para el *formulario* y **Ejercicio3-3** para el proyecto.

NOTA: Recuerde que todos los *objetos* o *controles* están representados por un *procedimiento*. Para ver más acerca de los *procedimientos* puede consultar el subtema **1.4.5 Procedimientos** del **Capítulo 1**.

- 3.1.1.3 Eventos sobre los botones de comando

Se ha dicho que los *eventos* son las acciones que realizan los usuarios sobre un determinado tipo de *objeto*. Estos *eventos* están definidos como *procedimientos* dentro de cada *objeto*, donde cada *procedimiento* deber ser codificado para representar dicho evento.

Todos los objetos poseen una serie de eventos que responden a las diferentes acciones del usuario, pero no todos los eventos de los objetos debe de ser codificados, solo los que el programador considere adecuados. Al igual que con las *propiedades*, trabajaremos únicamente con los *eventos* más utilizados de los *objetos*.

Evento	Descripción
Click	Ocurre cuando el usuario presiona y suelta un botón del mouse. En un <i>botón de comando</i> este <i>evento</i> ocurre cuando se presiona y se suelta el botón izquierdo del mouse. En otros <i>objetos</i> este <i>evento</i> ocurre sin importar cuales de los botones del mouse fue presionado. Sin duda alguna este es uno de los eventos más consultados en cualquier tipo de aplicación, por ende, se debe de codificar adecuadamente el procedimiento que representa este evento en cada uno de los objetos de la aplicación.
Ejercicio: <u>Esta aplicación muestra dos <i>botones de comando</i> que son activados por el <i>evento Click</i>. El primer botón muestra un mensaje de bienvenida y el segundo botón permite salir de la aplicación.</u> ▪ Abra un nuevo proyecto e inserte dos <i>botones de comando</i>. ▪ Seleccione la <i>propiedad (Name)</i> del primer <i>botón de comando</i> y escriba cmdAceptar, y en la <i>propiedad Caption</i> escriba: &Aceptar. ▪ Seleccione el segundo <i>botón de comando</i>, y en la <i>propiedad (Name)</i> escriba cmdSalir, y en la <i>propiedad Caption</i> escriba: &Salir. ▪ Haga doble clic en el primer <i>botón de comando</i>. ▪ Aparecerá la ventana del Code Editor (Editor de código) con el cursor parpadeando dentro del <i>procedimiento Click</i> del <i>botón de comando</i>.	



- Dentro del *procedimiento Click* escriba la siguiente línea de código:

```
MsgBox ("Bienvenido a Visual Basic 6.0")
```

- El procedimiento codificado quedaría de la siguiente manera:

```
Private Sub cmdAceptar_Click()  
    MsgBox ("Bienvenido a Visual Basic 6.0")  
End Sub
```

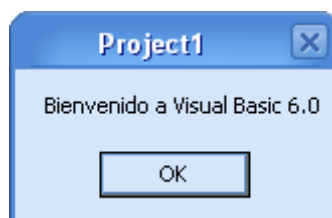
- Cierre la ventana del Editor de Código.
- Haga doble clic en el segundo *botón de comando*.
- Dentro del *procedimiento Click* del segundo *botón de comando* escriba la siguiente sentencia:

```
End
```

- El *procedimiento* codificado quedaría de la siguiente manera:

```
Private Sub cmdSalir_Click()  
    End  
End Sub
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Haga clic en el botón **Aceptar**.
- Aparecerá una caja de mensaje dando la bienvenida a *Visual Basic 6.0*:



- Haga clic en el botón **OK** para cerrar la ventana.
- Haga clic en el botón **Salir** para salir de la aplicación.

La sentencia **MsgBox** se utiliza para mostrar un mensaje en la pantalla. El texto que se muestra, se especifica entre comillas y dentro de dos paréntesis, como se mostró anteriormente. Más adelante



hablaremos detalladamente sobre esta sentencia.

Por otro lado, la sentencia **End** se utiliza para finalizar la aplicación. Esta sentencia cierra todos los *formularios* abierto en la aplicación, sin importar desde que *formulario* se halla ejecutado la sentencia **End**.

NOTA 1: Un evento **Click** no solo se activa haciendo clic con un botón del mouse. Este evento también puede activarse al presionar la tecla **BARRA ESPACIADORA** o la tecla **ENTER** sobre un objeto *botón de comando* cuando este tiene el enfoque. Otra forma de activar el evento **Click**, es asignado mediante código el valor **True** en la propiedad **Value** del *botón de comando*.

NOTA 2: Tenga presente que los *eventos* en un *procedimiento* vienen separados por un **underscore** (**_**) o **subrayado** que lo separa del nombre del objeto. Es importante recordar esto, para que tenga claro con cual *evento* usted esta trabajando en un *procedimiento*.

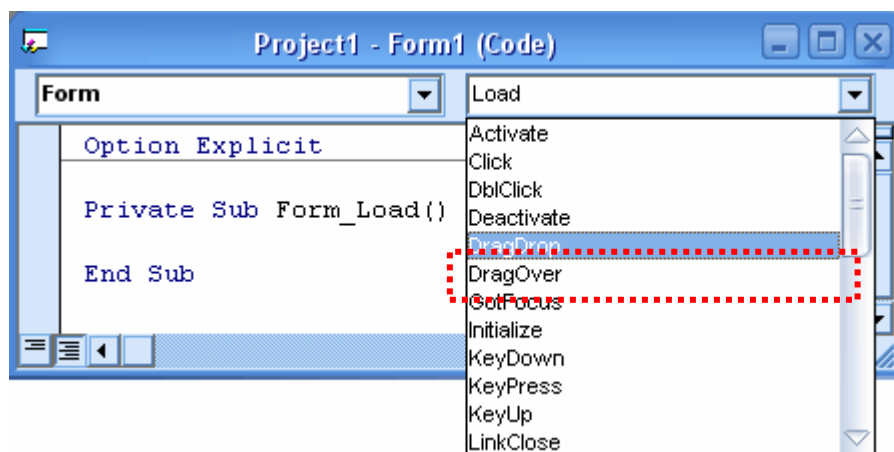
- Guarde la aplicación con los nombres **FormEjercicio4-3** para el *formulario* y, **Ejercicio4-3** para el proyecto.

DragDrop	Ocurre cuando finaliza una operación de arrastrar y soltar un <i>objeto</i> sobre otro <i>objeto</i>. En los <i>botones de comando</i> este <i>evento</i> posee tres <i>argumentos</i> que son: Source, coordenadas X, Y del puntero del mouse. Para que un <i>objeto</i> se pueda arrastra debe tener establecido el valor 1 – Automatic en la propiedad DragMode, en caso contrario la operación de arrastrar y soltar nunca iniciará. El <i>procedimiento DragDrop</i> viene representado de la siguiente manera: <pre>Private Sub Objeto_DragDrop(Source As Control, X As Single, Y As Single) End Sub</pre> Donde: Source: Representa el control que se esta arrastrando. Esta variable que al castellano significa origen, hace referencia al <i>objeto</i> como si se tratase del <i>objeto</i> mismo, es decir, se puede hacer referencia a cualquier propiedad del <i>objeto</i> especificando la variable Source, seguido del nombre de la <i>propiedad</i>. Por ejemplo, Source.Visible = False, oculta el objeto que identifica el argumento Source. X: Es un número que representa la coordenada horizontal actual del puntero del mouse dentro del <i>formulario</i> o <i>control de destino</i>. Y: Es un número que representa la coordenada vertical actual del puntero del mouse dentro del <i>formulario</i> o <i>control de destino</i>. COMENTARIO: Utilice un procedimiento de evento DragDrop para controlar qué ocurre tras completarse una operación de arrastrar. Por ejemplo, puede mover el control de origen a una nueva ubicación o copiar un archivo de una ubicación a otra. Cuando varios controles puedan usarse potencialmente en un argumento origen: ▪ Utilice la palabra clave TypeEnum con la instrucción If para determinar el tipo de control utilizado con origen. ▪ Utilice la propiedad Tag del control para identificar un control y después utilice un procedimiento de evento DragDrop.
-----------------	--

**Ejercicio:**

El siguiente ejercicio muestra una aplicación con un *botón de comando* que se puede mover de posición manteniendo pulsado el botón izquierdo del Mouse y soltándolo en cualquier parte del *formulario*.

- Abra un nuevo proyecto e inserte un *botón de comando*.
- Seleccione el botón de comando y busque la propiedad **DragMode**. Seleccione el valor **1 – Automatic** para que el botón se puede arrastrar.
- Haga doble clic en cualquier parte del *formulario* para activar el *Editor de código*. Tenga en cuenta no hacer doble clic en el *botón de comando*.
- Aparecerá el cursor parpadeando en el procedimiento que representa el evento **Load (Carga)** del formulario. Este evento no nos interesa, por tal razón, debemos hacer clic en el evento **DragDrop** de la lista de *eventos*, ubicada en la parte superior del *Editor de Código*.



- Aparecerá el *procedimiento* que representa el evento **DragDrop** del *formulario*:

```
Private Sub Form_DragDrop(Source As Control, X As Single, Y As Single)

End Sub
```

- Dentro del *procedimiento* **DragDrop** escriba las siguientes líneas de código:

```
Source.Top = Y
Source.Left = X
```

- El *procedimiento* codificado quedaría de la siguiente manera:

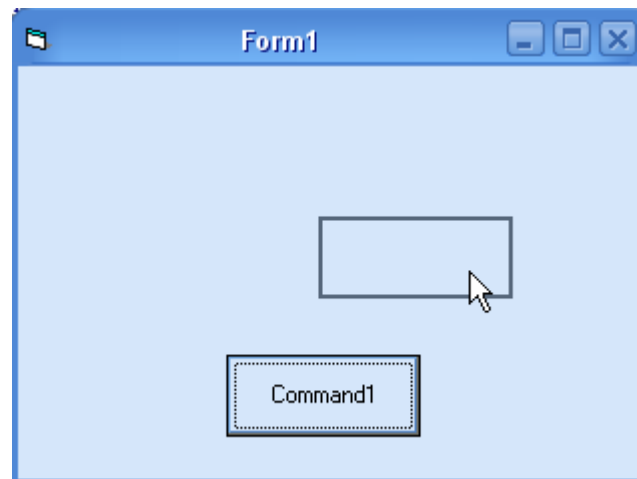
```
Private Sub Form_DragDrop(Source As Control, X As Single, Y As Single)

    Source.Top = Y           'Coloca el objeto en la coordenada Y del Mouse
    Source.Left = X          'Coloca el objeto en la coordenada X del Mouse

End Sub
```



- Pulse la tecla **[F5]** para correr la aplicación.
- Ahora mueva el *botón de comando* a cualquier parte del *formulario*. Podrá observar que el *botón de comando* toma la posición actual del puntero del Mouse.



- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio5-3** para el *formulario* y, **Ejercicio5-3** para el proyecto.

Explicación:

Anteriormente, explicamos que la variable **Source** identifica al objeto como si se tratase del mismo. Cuando escribimos **Source.Top = Y** estamos queriendo decir **Command1.Top = Y**. De igual manera, **Source.Left = X** estamos queriendo decir **Command1.Left = X**. Dijimos que **X** es un número que representa la posición horizontal del puntero del Mouse y, la variable **Y** la posición vertical. Asignando estos valores en las propiedades **Top** (**Arriba**) y **Left** (**Izquierda**) del *botón de comando*, hace posible cambiar la posición del botón. Para más información acerca de las *propiedades Top y Left* retrocedan algunas páginas atrás, a las *propiedades de los botones de comando* de este mismo capítulo.

Utilizar la variable **Source** tiene algunas ventajas que se enumeran a continuación:

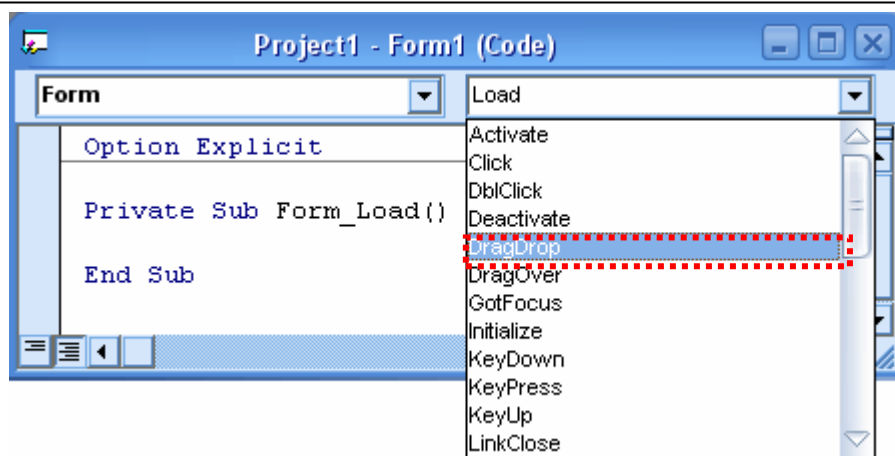
1. Cualquier *control* que se agregue en el *formulario* se podrá cambiar de posición, siempre y cuando el valor **1 – Automatic** este establecido en la propiedad **DragMode**.
2. La variable **Source** identifica el *objeto* que se esta arrastrando, sin importar cual sea el *tipo de objeto*.
3. Es posible identificar cual es el *tipo de objeto* que se arrastra, utilizando la sentencia **TypeOf** combinada con la instrucción **If**. Ejemplo: **If TypeOf Source Is CommandButton Then**. Este ejemplo verifica si el objeto que se arrastra es un *botón de comando*.

COMENTARIO: Este evento combinado con el evento **DragOver** es muy utilizado en la mayoría de los juegos de mesa. Por ejemplo, los juegos de carta, ajedrez, damas, etc. Estos juegos requieren manejar frecuente mente estos dos eventos, debido a que consisten realmente en arrastrar y soltar algún objeto.



Capítulo 3

DragOver	Ocurre mientras se realiza una operación de arrastrar y soltar. Mientras el usuario este arrastrando un objeto sobre su contenedor, el evento DragOver se efectuará continuamente. Este evento permite realizar variedades de operaciones con el objeto, principalmente en aplicaciones de video juegos. Este evento posee los mismos argumentos del evento DragDrop (Source, X, Y), a diferencia del argumento State. Este argumento representa un entero que corresponde al estado de transición del control que se está arrastrando en relación al formulario o control de destino. Puede tomar entre tres valores 0, 1 y 2. Cuando el <i>Estado</i> se encuentra en 0 (Entrar), indica que el <i>objeto origen</i>, se encuentra dentro del intervalo de un destino, es decir, el objeto iniciará la operación de arrastre. El <i>Estado 1 (Salir)</i>, indica que el <i>objeto origen</i> se esta arrastrando fuera del <i>intervalo de un destino</i>, es decir, fuera del intervalo del <i>objeto origen</i>. El <i>Estado 2 (Sobre)</i>, indica que el <i>objeto origen</i> se ha movido de una posición del destino a otra, es decir, se encuentra dentro del <i>intervalo destino</i> pero en otra posición. El <i>procedimiento DragOver</i> viene representado de la siguiente manera: <pre>Private Sub Objeto_DragOver(Source As Control, X As Single, Y As Single, State As Integer) End Sub</pre> NOTA: El termino <i>objeto origen</i> representa el objeto que se esta arrastrando. Un <i>intervalo destino</i> representa el conjunto de valores que están comprendidos entre la posición (x, y) del <i>objeto origen</i> que se arrastra con relación a su contenedor y el tamaño (altura y anchura) de dicho objeto. COMENTARIO: Utilice un procedimiento de evento DragOver para determinar lo que ocurre tras iniciarse el arrastre y antes de que un control se coloque en un destino. Por ejemplo, puede comprobar un intervalo de destino válido si resalta el destino (establezca la propiedad BackColor o ForeColor desde el código) o muestra un puntero de arrastre especial (establezca la propiedad DragIcon o MousePointer desde el código). Utilice el argumento <i>estado</i> para determinar las acciones de los puntos de transición claves. Por ejemplo, podría resaltar un posible destino cuando <i>estado</i> se establezca a 0 (Entrar) y restaurar la apariencia anterior del objeto cuando <i>estado</i> se establezca a 1 (Salir).
Ejercicio: El siguiente ejercicio muestra una aplicación con un <i>botón de comando</i> que se puede mover de posición manteniendo pulsado el botón izquierdo del Mouse y soltándolo en cualquier parte del <i>formulario</i>. A diferencia de la aplicación anterior, este ejemplo muestra un icono especial cuando se esta arrastrando el objeto. ▪ Abra un nuevo proyecto e inserte un <i>botón de comando</i>. ▪ Seleccione el botón de comando y busque la propiedad DragMode. Seleccione el valor 1 – Automatic para que el botón se puede arrastrar. ▪ Haga doble clic en cualquier parte del <i>formulario</i> para activar el <i>Editor de código</i>. Tenga en cuenta no hacer doble clic en el <i>botón de comando</i>. ▪ Seleccione el evento DragDrop de la <i>lista de eventos</i> ubicada en la esquina superior derecha del <i>Editor de código</i>.	



- Aparecerá el *procedimiento* que representa el evento **DragDrop** del *formulario*:

```
Private Sub Form_DragDrop(Source As Control, X As Single, Y As Single)

End Sub
```

- Dentro del *procedimiento* **DragDrop** escriba las siguientes líneas de código:

```
Source.Top = Y
Source.Left = X
```

- El *procedimiento* codificado quedaría de la siguiente manera:

```
Private Sub Form_DragDrop(Source As Control, X As Single, Y As Single)

    Source.Top = Y           'Coloca el objeto en la coordenada Y del Mouse
    Source.Left = X          'Coloca el objeto en la coordenada X del Mouse
    Source.MousePointer = 0  'Asigna el puntero normal o predeterminado del Mouse

End Sub
```

Como ha podido notar es la misma codificación del programa anterior, que permite posicionar el *objeto* que se arrastra en cualquier parte del *formulario*, con la única diferencia en que se ha agregado una nueva línea de código **Source.MousePointer = 0**. La propiedad **MousePointer** permite cambiar la forma del puntero del Mouse en tiempo de ejecución. Esta propiedad puede tomar una serie de valores que se muestran en la siguiente tabla:

Valor	Puntero	Valor	Puntero	Valor	Puntero	Valor	Puntero
0	Default (↖)	5	Size (↕↔)	10	Up Arrow (↑)	15	Size↕↔
1	Arrow (↖)	6	Size NE SW (↗↙)	11	Hourglass (⌚)	99	Desde archivo
2	Cross (⊕)	7	Size N S (↕)	12	No Drop (⊘)		
3	I-Beam (⌵)	8	Size NWSE (↖↗)	13	ArrowHourglass (↖⌚)		
4	Icon (🖱)	9	Size W E (↔)	14	ArrowQuestion (↖?)		



Ahora codificaremos para que cuando el usuario este moviendo el objeto sobre el formulario, este cambie el aspecto del puntero del Mouse. Esto debe ser codificado en el evento **DragOver** del *botón de comando* y no del formulario.

- Haga doble clic sobre el *botón de comando* y, a continuación, seleccione el evento **DragOver** de la *lista de eventos* ubicada en la esquina superior derecha del *Editor de código*.

- Aparecerá el procedimiento que representa el evento **DragOver** del *botón de comando*:

```
Private Sub Command1_DragOver(Source As Control, X As Single, Y As Single, State As Integer)

End Sub
```

- Dentro del *procedimiento DragOver* escriba las siguientes líneas de código:

```
' Si el Estado del argumento (State) se encuentra en 0 (Entrar), entonces, cambia
' el puntero del Mouse

If State = 0 Then Source.MousePointer = 15
```

- El *procedimiento* codificado quedaría de la siguiente manera:

```
Private Sub Command1_DragOver(Source As Control, X As Single, Y As Single, State As Integer)

    ' Si el Estado del argumento (State) se encuentra en 0, entonces, cambia
    ' el puntero del Mouse

    If State = 0 Then Source.MousePointer = 15

End Sub
```

- Pulse la tecla **[F5]** para correr la aplicación.

- Ahora mueva el *botón de comando* a cualquier parte del *formulario*. Podrá observar que el *botón de comando* toma la posición actual del puntero del Mouse. Además, puede observar como el puntero del Mouse cambia de apariencia mientras se esta arrastrando el *botón de comando*.

- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.

- Guarde la aplicación con los nombres **FormEjercicio6-3** para el *formulario* y, **Ejercicio6-3** para el proyecto.

Explicación:

Esta aplicación se puede dividir en dos partes representadas por dos eventos, el primero **DragDrop** para posicionar el objeto en la coordenada **X, Y** del Mouse, y el segundo **DragOver** para cambiar el



puntero del Mouse mientras se arrastra el *botón de comando*.

Se ha dicho anteriormente, que el evento **DragDrop** ocurre cuando termina una operación de arrastrar y soltar un *objeto* sobre su *contenedor*. En esta aplicación se ha codificado este evento con tres líneas de código donde las dos primeras la tratamos en la aplicación anterior. La línea de código **Source.MousePointer = 0** permite establecer el puntero por defecto o predeterminado del Mouse.

La propiedad **MousePointer (Puntero del Mouse)**, es una propiedad muy utilizada en los *controles* de Visual Basic 6.0. Esta permite cambiar la apariencia del puntero del Mouse, permitiendo seleccionar entre 16 constantes o valores ya explicados en la tabla anterior.

Usted podría preguntarse que hace esta línea de código dentro de este procedimiento (evento). La razón es muy sencilla, el evento **DragDrop** permite realizar cualquier función, siempre y cuando termine la operación de arrastrar y soltar. Por ende, el puntero del Mouse debe establecerse a su forma normal o predeterminada cuando el objeto sea colocado en la posición deseada, si no lo hace, entonces, el puntero del Mouse quedaría cambiado aún cuando halla terminado la operación de arrastrar y soltar el objeto.

Por otro lado, el evento **DragOver** esta codificado únicamente para cambiar la apariencia del puntero del Mouse cuando comience a arrastrar el *botón de comando*. La única línea de código que podemos ver en este procedimiento es:

```
If State = 0 Then Source.MousePointer = 15
```

Traducido literalmente al castellano:

Si Estado del objeto es Igual a Cero Entonces Puntero del botón de comando es Igual a 15

Se ha dicho que al iniciar la operación de arrastrar y soltar un objeto, el *argumento State* se encontrará en **0 (Entrar)**; esto permite realizar cualquier operación antes de soltar o terminar de arrastrar el objeto. Se ha utilizado el *estado 0* y no el **1** o **2** por la siguiente razón: “Se quiere cambiar la apariencia del puntero del Mouse cuando se inicie la operación de arrastrar y soltar, y esto solo lo proporciona el *estado 0 (Entrar)*”.

NOTA: Utilice la sentencia **If** o **Case** para verificar cual es el estado actual del objeto.

COMENTARIO: Utilice la propiedad **MousePointer** siempre que quiera cambiar la apariencia del puntero del Mouse sobre un objeto. Establezca el valor 99 en la propiedad **MousePointer** cuando quiera cargar un cursor o icono desde un disco.



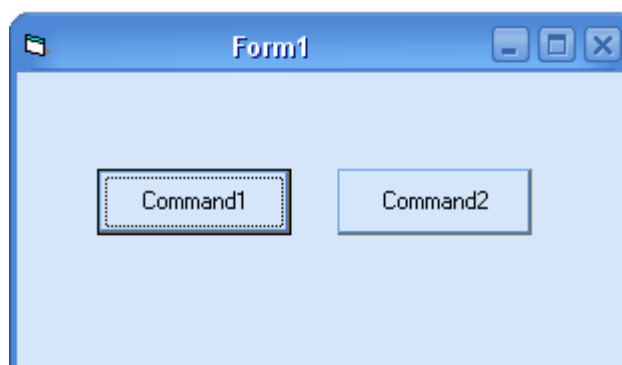
NOTA 3: Establezca el valor 99 en la propiedad **MousePointer** cuando quiera cargar un cursor o icono desde un disco. Para esto debe cargar el archivo desde la propiedad **Mouselcon** del objeto o utilizando la sentencia **LoadPicture** desde el *Editor de código*.

GotFocus	Este evento ocurre cuando un objeto recibe el <i>enfoque</i> o <i>focus</i>. Un objeto tiene el enfoque cuando el usuario a realizado un evento sobre el y no lo ha hecho sobre otro objeto hasta ese momento. Cuando usted hace clic sobre un <i>botón de comando</i> u otro objeto de la aplicación, en ese momento el objeto recibe el enfoque o la atención del usuario. El termino GotFocus significa “obtener el enfoque”, ocurre exactamente cuando el usuario realiza una acción o evento sobre el objeto, causado de esta manera que el objeto anterior pierda el enfoque o la atención del usuario. Este <i>evento</i> es muy poco utilizado en los <i>botones de comando</i>, pero de todos modos mostraré un ejemplo de su uso.
-----------------	---

Ejercicio:

Este ejemplo muestra una aplicación que posee dos *botones de comando*, cuando se hace clic sobre uno de ellos entonces se muestra un mensaje diciendo cual de los dos botones ha recibido el enfoque.

- Abra un nuevo proyecto e inserte dos *botón de comando*, tal y como se muestra en la siguiente figura:



- Haga doble clic sobre el primer *botón de comando* para que aparezca el *Editor de código*.
- Seleccione el evento **GotFocus** de la *lista de eventos*.
- Dentro de este procedimiento escriba las siguientes líneas de código:



'Muestra un mensaje diciendo que el primer botón recibió el enfoque.

MsgBox ("El primer botón recibió el enfoque")

- El *procedimiento* codificado quedaría de la siguiente manera:

```
Private Sub Command1_GotFocus()
```

'Muestra un mensaje diciendo que el primer botón recibió el enfoque.

MsgBox ("El primer botón ha recibido el enfoque")

```
End Sub
```

- Haga clic en **Form1** desde el *Explorador de Proyecto* para salir del *Editor de código* y visualizar el formulario de la aplicación.
- Haga doble clic en el segundo *botón de comando*.
- Seleccione el evento **GotFocus** de la *lista de eventos*.

'Muestra un mensaje diciendo que el primer botón recibió el enfoque.

MsgBox ("El segundo botón recibió el enfoque")

- El *procedimiento* codificado quedaría de la siguiente manera:

```
Private Sub Command2_GotFocus()
```

'Muestra un mensaje diciendo que el segundo botón recibió el enfoque.

MsgBox ("El segundo botón ha recibido el enfoque")

```
End Sub
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Aparecerá un mensaje diciendo "El primer botón ha recibido el enfoque", esto es, porque el primer *botón de comando* tiene establecido el valor 1 en la *propiedad TabIndex*, es decir, tiene establecido como el primer objeto que recibe el enfoque cuando inicia la aplicación.
- Con la aplicación ejecutándose, haga clic en el segundo *botón de comando*. Aparecerá otro mensaje diciendo "El segundo botón ha recibido el enfoque".

Cada vez que haga clic sobre uno de los *botones de comando* aparecerá un mensaje, la razón es, que el mensaje se ha codificado en el evento **GotFocus**, es decir, cuando se obtiene el enfoque.



- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio7-3** para el *formulario* y, **Ejercicio7-3** para el proyecto.

NOTA: Un objeto puede recibir el enfoque sólo si sus *propiedades* **Enabled** y **Visible** están establecidas a **True**. Para personalizar la interfaz de teclado en Visual Basic para mover el enfoque, establezca el { CONTROL Internet.HHCtrl.1 }orden de tabulación O especifique { CONTROL Internet.HHCtrl.1 }teclas de acceso para controles de un formulario.

COMENTARIO: Utilice este evento para especificar las acciones que ocurren cuando un control o un formulario recibe primero el enfoque. Por ejemplo, si adjunta un procedimiento de evento **GotFocus** a cada control de un formulario puede guiar al usuario mostrándole instrucciones breves o mensajes en la barra de estado. También puede proporcionar avisos visuales activando, desactivando o mostrando otros controles que dependan del control que tiene el enfoque.

LostFocus	El evento LostFocus es el opuesto del evento GotFocus. Este evento ocurre cuando un objeto pierde el enfoque o focus. El termino LostFocus significa “perdida del enfoque”, ocurre exactamente cuando el usuario realiza una acción o evento sobre otro objeto, causado de esta manera que el objeto anterior pierda el enfoque. Este evento es muy utilizado por los programadores, para verificar si un usuario realizo o no alguna operación requerida por el programa, por ejemplo, un programa que requiere que se introduzca el nombre y el apellido de una persona para registrarlo. Es obvio que para esto se necesitan dos <i>cajas de texto</i> (<i>TextBox</i>) y, por ende, no pueden dejarse vacías porque son datos importantes. Puede verificar con este evento si el usuario dejó o no las cajas vacías. Al igual que el <i>evento</i> GotFocus, este <i>evento</i> es muy poco utilizado en los <i>botones de comando</i>. Puede utilizar el ejercicio anterior para practicar este <i>evento</i>. Codifique el <i>evento</i> LostFocus en vez del <i>procedimiento</i> GotFocus y, cambie los mensajes “El primer botón ha recibido el enfoque”, “El segundo botón ha recibido el enfoque”, por: “El primer botón ha perdido el enfoque”, “El segundo botón ha perdido el enfoque”. COMENTARIO: Un procedimiento de evento LostFocus resulta especialmente útil para comprobar y validar actualizaciones. Usar LostFocus puede hacer que la validación tenga lugar conforme el usuario mueve el enfoque del control. Otro uso para este tipo de procedimiento de evento es activar, desactivar, ocultar y mostrar otros objetos, como en un procedimiento de evento GotFocus. También puede invertir o cambiar condiciones que estableció en el procedimiento de evento GotFocus del objeto.
KeyDown, KeyUp	Ocurre cuando el usuario presiona (KeyDown) y suelta (KeyUp) una tecla mientras un <i>objeto</i> tiene el <i>enfoque</i>. Aunque son dos <i>eventos</i> que suelen usarse combinados, son eventos distintos y, por consiguiente, serán definidos de manera independiente.

COMENTARIO: Utilice los eventos **KeyDown** y **KeyUp** siempre y cuando quiera realizar una operación distinta para cuando el usuario presiona una tecla y otra operación para cuando la suelta. También utilice estos eventos cuando quiera leer teclas especiales como las teclas de función, el teclado numérico, etc. En caso contrario utilice el evento **KeyPress**.



Capítulo 3

El evento **KeyDown (Tecla Abajo)** ocurre cuando el usuario presiona una tecla. Es un evento que se gestiona rápidamente y en un plazo de corto tiempo. La razón es, porque ocurre inmediatamente el usuario presiona una tecla antes que esta vuelva a levantarse. Si el usuario mantiene presionada cualquier tecla, entonces, este evento se efectuará constantemente.

El *procedimiento* para el evento **KeyDown** viene representado de la siguiente manera:

```
Private Sub Objeto_KeyDown(KeyCode As Integer, Shift As Integer)

End Sub
```

Como se puede apreciar el *procedimiento* para este *evento* esta compuesto por dos *argumentos* que son: **KeyCode** y **Shift**. El *argumento KeyCode* devuelve un código de tecla que representa la tecla pulsada por el usuario mientras el objeto tiene el enfoque. Los valores aceptados para este *argumento* se muestran en las siguientes tablas:

Constante	Valor	Descripción
vbKeyBack	8	Tecla RETROCESO
vbKeyTab	9	Tecla TAB
vbKeyClear	12	Tecla SUPR
vbKeyReturn	13	Tecla ENTRAR
vbKeyShift	16	Tecla MAYÚS
vbKeyControl	17	Tecla CTRL
vbKeyMenu	18	Tecla MENU
vbKeyPause	19	Tecla PAUSA
vbKeyCapital	20	Tecla BLOQ MAYUS
vbKeyEscape	27	Tecla ESC
vbKeySpace	32	Tecla BARRA ESPACIADORA
vbKeyPageUp	33	Tecla RE PÁG
vbKeyPageDown	34	Tecla AV PÁG
vbKeyEnd	35	Tecla FIN
vbKeyHome	36	Tecla INICIO
vbKeyLeft	37	Tecla FLECHA IZQUIERDA
vbKeyUp	38	Tecla FLECHA ARRIBA
vbKeyRight	39	Tecla FLECHA DERECHA
vbKeyDown	40	Tecla FLECHA ABAJO
vbKeySelect	41	Tecla SELECT



Constante	Valor	Descripción
vbKeyPrint	42	Tecla IMPRIMIR PANTALLA
vbKeyExecute	43	Tecla EXECUTE
vbKeySnapshot	44	Tecla SNAPSHOT
vbKeyInsert	45	Tecla INS
vbKeyDelete	46	Tecla SUPR
vbKeyHelp	47	Tecla AYUDA
vbKeyNumlock	144	Tecla BLOQ NUM

Teclado alfanumérico y Teclado numérico

Constante	Valor	Descripción	Constante	Valor	Descripción
vbKeyA	65	Tecla A	vbKey0	48	Tecla 0
vbKeyB	66	Tecla B	vbKey1	49	Tecla 1
vbKeyC	67	Tecla C	vbKey2	50	Tecla 2
vbKeyD	68	Tecla D	vbKey3	51	Tecla 3
vbKeyE	69	Tecla E	vbKey4	52	Tecla 4
vbKeyF	70	Tecla F	vbKey5	53	Tecla 5
vbKeyG	71	Tecla G	vbKey6	54	Tecla 6
vbKeyH	72	Tecla H	vbKey7	55	Tecla 7
vbKeyI	73	Tecla I	vbKey8	56	Tecla 8
vbKeyJ	74	Tecla J	vbKey9	57	Tecla 9
vbKeyK	75	Tecla K	vbKeyNumpad0	96	Tecla 0
vbKeyL	76	Tecla L	vbKeyNumpad1	97	Tecla 1
vbKeyM	77	Tecla M	vbKeyNumpad2	98	Tecla 2
vbKeyN	78	Tecla N	vbKeyNumpad3	99	Tecla 3
vbKeyO	79	Tecla O	vbKeyNumpad4	100	Tecla 4
vbKeyP	80	Tecla P	vbKeyNumpad5	101	Tecla 5
vbKeyQ	81	Tecla Q	vbKeyNumpad6	102	Tecla 6
vbKeyR	82	Tecla R	vbKeyNumpad7	103	Tecla 7
vbKeyS	83	Tecla S	vbKeyNumpad8	104	Tecla 8
vbKeyT	84	Tecla T	vbKeyNumpad9	105	Tecla 9
vbKeyU	85	Tecla U	vbKeyMultiply	106	Tecla MULTIPLICACION (*)
vbKeyV	86	Tecla V	vbKeyAdd	107	Tecla SIGNO SUMA (+)
vbKeyW	87	Tecla W	vbKeySeparator	108	Tecla INTRO
vbKeyX	88	Tecla X	vbKeySubtract	109	Tecla SIGNO MENOS (-)
vbKeyY	89	Tecla Y	vbKeyDecimal	110	Tecla PUNTO DECIMAL (.)
vbKeyZ	90	Tecla Z	vbKeyDivide	111	Tecla SIGNO DIVISION (/)



Teclas de función

Constante	Valor	Descripción
vbKeyF1	112	Tecla F1
vbKeyF2	113	Tecla F2
vbKeyF3	114	Tecla F3
vbKeyF4	115	Tecla F4
vbKeyF5	116	Tecla F5
vbKeyF6	117	Tecla F6
vbKeyF7	118	Tecla F7
vbKeyF8	119	Tecla F8
vbKeyF9	120	Tecla F9
vbKeyF10	121	Tecla F10
vbKeyF11	122	Tecla F11
vbKeyF12	123	Tecla F12

Las tablas anteriores muestran cada uno de los valores proporcionados por el *evento* **KeyDown** en su *argumento* **KeyCode**. Estos valores pueden ser manejados y comprobados utilizando la sentencia **If** o **Case**. Es posible verificar cual tecla el usuario ha presionado sobre el objeto cuando este tiene el enfoque, por ejemplo, dentro del *procedimiento* **KeyDown** es posible escribir una línea de código parecida a la siguiente: **If** KeyCode = vbKeyF9 **Then**. Esta línea de código proporcionará un valor verdadero cuando el usuario presione la **Tecla F9**. Esto hace posible realizar operaciones distintas para cada tecla pulsada por el usuario.

Por otro lado, el argumento **Shift** devuelve un valor entero que representa el estado de las teclas MAYUS, CTRL y ALT en el momento del evento. Estos valores corresponden a los enteros 1, 2 y 4, respectivamente. La siguiente tabla muestra las constantes y valores que identifican cada uno de estos estados:

Constante	Valor	Descripción
vbShiftMask	1	Tecla MAYÚS
vbCtrlMask	2	Tecla CTRL
vbAltMask	4	Tecla ALT

No es recomendable tratar los valores proporcionados por el *argumento* **Shift** por sus valores correspondientes en la tabla anterior, debido a que sería imposible detectar cualquier combinación de estas tres teclas. Es recomendable almacenar los valores proporcionados por el *argumento* **Shift** en una variable de *tipo entera* (**As Integer**), y luego verificar si su valor es mayor que cero (0).



Capítulo 3

Para realizar lo dicho anteriormente, asigne primero cada resultado a una variable temporal entera y después compare **Mayús (Shift)** con una *máscara de bits*. Utilice el operador **And (Y)** con el argumento **Mayús (Shift)** para comprobar si la condición es mayor que 0, lo que indica que el modificador se presionó, como en este ejemplo:

```
Dim TeclaShiftPresionada As Integer
```

```
Dim TeclaAltPresionada As Integer
```

```
Dim TeclaCtrlPresionada As Integer
```

```
TeclaShiftPresionada = (Shift And vbShiftMask)
```

```
TeclaAltPresionada = (Shift And vbAltMask)
```

```
TeclaCtrlPresionada = (Shift And vbCtrlMask)
```

```
If TeclaShiftPresionada > 0 Then
```

```
MsgBox ("Usted presiono la tecla MAYUS o SHIFT")
```

```
Elseif TeclaAltPresionada > 0 Then
```

```
MsgBox ("Usted presiono la tecla ALT")
```

```
Elseif TeclaCtrlPresionada > 0 Then
```

```
MsgBox ("Usted presiono la tecla CTRL")
```

```
End If
```

Si agrega este bloque de código dentro del evento **KeyDown** de cualquier objeto, se mostrará un mensaje cada vez que se pulse una de las teclas MAYUS, ALT o CTRL mientras el objeto tiene el *enfoque*.

Se puede observar en el ejemplo anterior que se han utilizado tres variables de tipo **Integer** (*entera*) para almacenar cada uno de los valores de las teclas MAYUS, ALT y CTRL, respectivamente. Cuando el usuario pulsa una de estas tres teclas, la variable correspondiente a esa tecla toma un valor mayor que cero. Por ejemplo, si el usuario pulsa la tecla ALT la variable *TeclaAltPresionada* toma el valor 4, si se pulsa la tecla SHIFT o MAYUS la variable *TeclaShiftPresionada* toma el valor 1 y, si se pulsa la tecla CTRL la variable *TeclaCtrlPresionada* toma el valor 2. Después de esto, se han establecido tres condiciones con una sentencia **If (Si)** *anidada* (*sentencias If dentro de otro If*), para verificar cuáles de las tres variables tiene un valor mayor que cero. Cuando se comprueba la condición y resulta verdadera, es decir, que la variable tiene un valor mayor que cero, entonces, se muestra un mensaje diciendo cual fue la tecla pulsada.



Para detectar cualquier combinación de tecla, utilice el operador lógico **And (Y)** para verificar si los valores de las demás variables son iguales o mayores que cero.

Ejemplo:

```
Dim TeclaShiftPresionada As Integer
Dim TeclaAltPresionada As Integer
Dim TeclaCtrlPresionada As Integer

TeclaShiftPresionada = (Shift And vbShiftMask)
TeclaAltPresionada = (Shift And vbAltMask)
TeclaCtrlPresionada = (Shift And vbCtrlMask)

If TeclaShiftPresionada > 0 And TeclaAltPresionada > 0 Then
    MsgBox ("Usted presiono la tecla MAYUS y la tecla ALT")

    ElseIf TeclaAltPresionada > 0 And TeclaCtrlPresionada > 0 Then
        MsgBox ("Usted presiono la tecla ALT y la tecla CTRL")

        ElseIf TeclaCtrlPresionada > 0 And TeclaShiftPresionada > 0 Then
            MsgBox ("Usted presiono la tecla CTRL y la tecla MAYUS")

End If
```

Si se comprueba que dos o más variables son mayores que cero usando la sentencia **If (Si)** y el operador lógico **And (Y)**, entonces, el usuario ha presionado una combinación de tecla. Puede verificar las combinaciones de tres o más teclas utilizando como parámetro el ejemplo anterior.

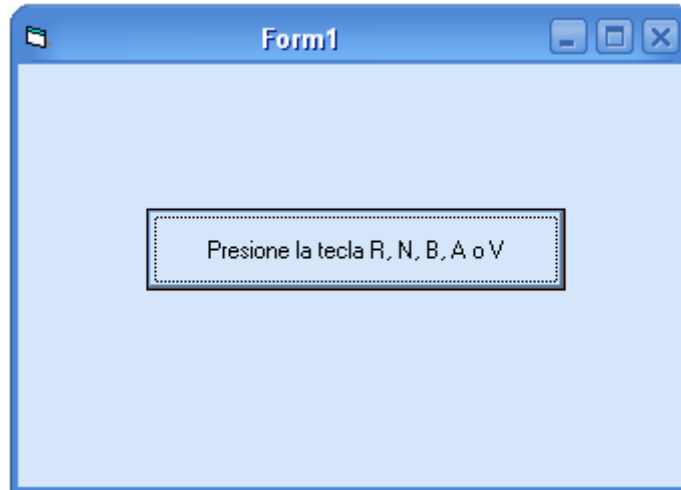
Por otro lado, el *evento* **KeyUp (Tecla Arriba)** ocurre cuando el usuario suelta la tecla presionada. Este evento ocurre exactamente cuando finaliza el evento **KeyDown (Tecla Abajo)**. Estos dos eventos mayormente se utilizan combinados para realizar una operación cuando el usuario pulsa una tecla y otra cuando la suelta.

A continuación, se muestra un ejercicio donde se combinan ambos eventos. Este ejercicio esta compuesto por un *botón de comando* que cambia su color cada vez que se pulsa una tecla y vuelve a su color original cuando se suelta la tecla. Se ha codificado para que cambie entre cinco colores distintos si se pulsa una de las letras que representan el color **R, N, B, A o V** (rojo, negro, blanco, azul o verde).



Capítulo 3

- Abra un nuevo proyecto e inserte un *botón de comando*, tal y como se muestra en la siguiente figura:



- Busque la *propiedad* **Caption** y escriba: **Presione la tecla R, N, B, A o V**. El texto aparecerá dentro del *botón de comando*, tal y como se muestra en la figura anterior.
- Busque la *propiedad* **Style** y establezca el valor **1-Graphical**. Recuerde que esto es necesario para que un *botón de comando* pueda cambiar de color. Usted puede retroceder algunas páginas si desea saber más acerca de la propiedad **Style**.
- Haga doble clic sobre el *botón de comando* para invocar el *Editor de código*.
- Estando dentro del procedimiento **Click** del *botón de comando* seleccione el evento **KeyDown** de la *lista de eventos* y, a continuación, escriba el siguiente bloque de código:

```
'Si se pulsa la tecla R
If KeyCode = vbKeyR Then
Command1.BackColor = vbRed      ' Pone el color rojo al botón
End If

'Si se pulsa la tecla N
If KeyCode = vbKeyN Then
Command1.BackColor = vbBlack    ' Pone el color negro al botón
End If

'Si se pulsa la tecla B
If KeyCode = vbKeyB Then
Command1.BackColor = vbWhite    ' Pone el color blanco al botón
End If

'Si se pulsa la tecla A
If KeyCode = vbKeyA Then
Command1.BackColor = vbYellow   ' Pone el color amarillo al botón
End If
```

Ing. Carlos Manuel Rodríguez Bucarellly



'Si se pulsa la tecla V

```
If KeyCode = vbKeyV Then  
Command1.BackColor = vbGreen 'Pone el color verde al botón  
End If
```

- El procedimiento codificado quedaría de la siguiente manera:

```
Private Sub Command1_KeyDown(KeyCode As Integer, Shift As Integer)
```

'Si se pulsa la tecla R

```
If KeyCode = vbKeyR Then  
Command1.BackColor = vbRed 'Pone el color rojo al botón  
End If
```

'Si se pulsa la tecla N

```
If KeyCode = vbKeyN Then  
Command1.BackColor = vbBlack 'Pone el color negro al botón  
End If
```

'Si se pulsa la tecla B

```
If KeyCode = vbKeyB Then  
Command1.BackColor = vbWhite 'Pone el color blanco al botón  
End If
```

'Si se pulsa la tecla A

```
If KeyCode = vbKeyA Then  
Command1.BackColor = vbYellow 'Pone el color amarillo al botón  
End If
```

```
If KeyCode = vbKeyV Then  
Command1.BackColor = vbGreen 'Pone el color verde al botón  
End If
```

```
End Sub
```

- Tenga en cuenta no haber codificado dentro del *evento Click*, sino dentro del *procedimiento KeyDown*.

Hasta este momento solo hemos codificado dentro del procedimiento **KeyDown**, esto nos va a permitir probar la aplicación antes de codificar el *procedimiento KeyUp*.

- Pulsa la tecla **[F5]** para correr la aplicación.
- Pulse cada una de las teclas que muestra el *botón de comando*.

Usted puede observar que el *botón de comando* permanece con el color seleccionado hasta que se pulsa otra tecla. Esto es, porque no se ha codificado aun el procedimiento **KeyUp** del *botón de comando*, que nos va a permitir restablecer el color inicial o el color por defecto que tenía el botón antes de ser cambiado.



- Pare la aplicación haciendo clic en el botón **Cerrar** de la ventana.
- Ahora estando en *modo de diseño* vamos a codificar el procedimiento **KeyUp** del *botón de comando*.
- Haga doble clic sobre el *botón de comando* y, a continuación, seleccione el evento **KeyUp**.
- Aparecerá el procedimiento **KeyUp** como se muestra a continuación:

```
Private Sub Command1_KeyUp(KeyCode As Integer, Shift As Integer)
```

```
End Sub
```

- Dentro del procedimiento escriba la siguiente línea de código:

```
'Pone el color por defecto del botón de comando
```

```
Command1.BackColor = vbButtonFace
```

- El procedimiento codificado quedaría de la siguiente manera:

```
Private Sub Command1_KeyUp(KeyCode As Integer, Shift As Integer)
```

```
'Pone el color por defecto del botón de comando
```

```
Command1.BackColor = vbButtonFace
```

```
End Sub
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Pulse cada una de las teclas que muestra el *botón de comando*.
- Puede observar que el botón de comando toma su color inicial cuando se suelta la tecla que ha sido pulsada.
- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio8-3** para el *formulario* y, **Ejercicio8-3** para el proyecto.

Explicación:

Como se mencionó anteriormente, en esta aplicación hemos combinados los *eventos* **KeyDown** (**Tecla Abajo**) y **KeyUp** (**Tecla Arriba**). Dentro del procedimiento **KeyDown** utilizamos el argumento **KeyCode** combinado con la sentencia **If** para verificar cada una de las teclas pulsada por el usuario. Estas teclas están representadas por constantes que ya hemos visto en las páginas anteriores. Después de verificar cual fue la tecla presionada, entonces, hemos cambiado el *estado del objeto* utilizando la propiedad **BackColor** y el nombre del color que representa la letra pulsada.

Por otro lado, se ha codificado el procedimiento del *evento* **KeyUp** para restablecer el color inicial del *botón de comando* cuando el usuario suelta la tecla. Para esto también se ha cambiado el *estado del objeto* utilizando la propiedad **BackColor** y la constante **vbButtonFace** (color por defecto del *botón de comando*).



KeyPress	Este evento ocurre cuando el usuario presiona y suelta una tecla. Es un evento prácticamente similar al evento KeyUp con la única diferencia de sus argumentos. El procedimiento KeyPress esta representado de la siguiente manera: <pre>Private Sub Objeto_KeyPress(KeyAscii As Integer) End Sub</pre> Este evento solo posee un único <i>argumento</i> llamado KeyAscii. Este <i>argumento</i> devuelve un entero que representa un código ANSI de la tecla pulsada por el usuario. Es posible detectar cual es la tecla pulsada por el usuario combinado este argumento con la sentencia If o Case. Por ejemplo: <pre>If KeyAscii = 65 Then MsgBox ("Se pulso la tecla A en mayúscula")</pre> Cuando se pulse la tecla A en mayúscula desde el teclado cuando el objeto tenga el enfoque, entonces, aparecerá un mensaje diciendo “Se pulso la tecla A en mayúscula”. Esto es porque el valor 65 es un código ANSI que representa la letra A en mayúscula. Este evento difícilmente usted tendrá que utilizarlo en los <i>botones de comando</i>, por tal razón me limitaré a dar ejemplo de este evento en este tipo de control. Más adelante veremos una gama de ejemplos con este evento en otros tipos de controles.
MouseDown y MouseUp	Estos dos eventos son similares a los eventos KeyDown y KeyUp, la única diferencia es que MouseDown y MouseUp se gestionan con el Mouse y no con el teclado. El evento MouseDown ocurre cuando el usuario presiona un botón de Mouse y cuando lo suelta ocurre el evento MouseUp. Estos dos eventos suelen utilizarse combinados para realizar operaciones distintas cuando el usuario presiona y suelta un botón del Mouse. Estos dos procedimientos para vienen representado de la siguiente manera: <pre>Private Sub Objeto_MouseDown(Button As Integer, Shift As Integer, X As Single, Y As Single) End Sub</pre> <pre>Private Sub Objeto_MouseUp(Button As Integer, Shift As Integer, X As Single, Y As Single) End Sub</pre> Donde: Button: Este argumento almacena o devuelve un valor que representa el botón del



Mouse que ha sido pulsado o soltado por el usuario. El argumento **Button (Botón)** es un campo de bits que puede tomar los valores 1, 2 y 4. Donde 1 representa el botón izquierdo del Mouse, 2 el botón derecho y 4 el botón central. Estos valores permiten identificar cual de los botones del Mouse fue pulsado por el usuario sobre un objeto en una aplicación y, para realizar operaciones distintas para cada botón.

Shift: Devuelve un entero que corresponde al estado de las teclas MAYÚS, CTRL y ALT cuando el botón especificado en el argumento botón se presionó o se soltó. Estos valores corresponden a los enteros 1, 2 y 4, respectivamente. El valor 1 representa la tecla MAYÚS, el 2 la tecla CTRL y el 4 la tecla ALT. Puede retroceder algunas páginas y repasar este argumento en los eventos **KeyDown** y **KeyUp** estudiados anteriormente.

X: Es un número que representa la coordenada horizontal actual del puntero del mouse dentro del formulario o contenedor.

Y: Es un número que representa la coordenada vertical actual del puntero del mouse dentro del formulario o contenedor.

COMENTARIO: Utilice un procedimiento de evento **MouseDown** o **MouseUp** para especificar acciones que ocurrirán cuando se presione o se suelte un determinado botón del mouse. A diferencia de los eventos **Click** y **DbClick**, los eventos **MouseDown** y **MouseUp** le permiten distinguir entre los botones primario, secundario y central. También puede escribir código para combinaciones de teclado y mouse que utilicen los modificadores de teclado MAYÚS, CTRL y ALT.

NOTA 1: No utilice los eventos **MouseDown** y **MouseUp** para dar efectos de pulsación a *botones de comando*. Utilice controles de imágenes para realizar dichos efectos. Puede utilizar un control **Image** o **PictureBox** que responden de la misma manera que los *botones de comando*.

NOTA 2: No utilice los eventos **MouseDown** para realizar las operaciones que se deben realizar con el evento **Click** de un determinado tipo de objeto. Si utiliza el evento **MouseDown** y el evento **Click** en un mismo objeto, entonces, no puede utilizar el evento **MouseDown** para mostrar mensajes en otras ventanas o mostrar algún formulario, debido a que el código que esta dentro del evento **Click** no se ejecutará. Esto es porque el evento **MouseDown** intercepta el evento **Click**, y como resultado el código de este último evento no se producirá.

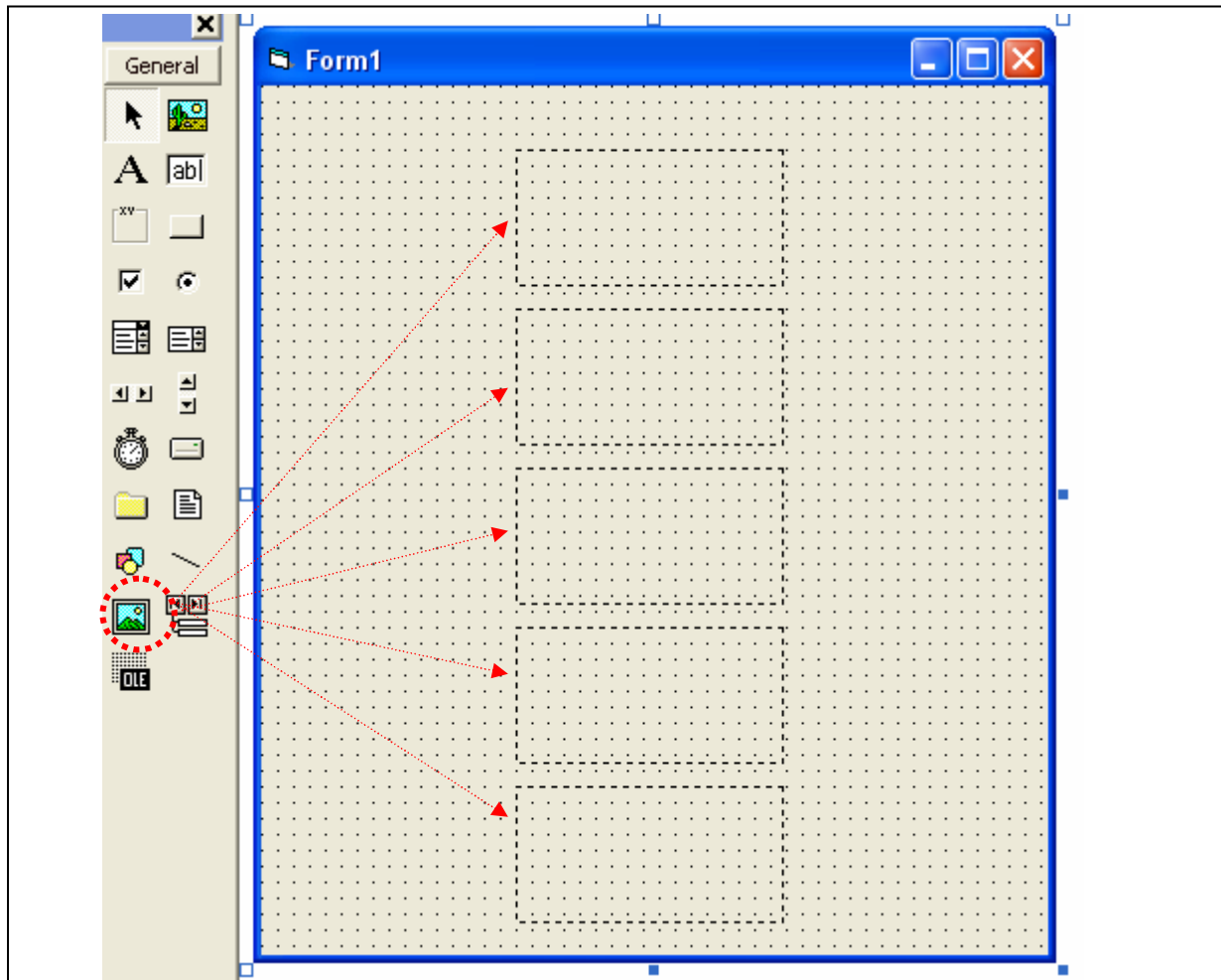
Ejercicio:

Para este ejemplo utilizaremos un tipo de control que aun no hemos definido ni utilizado en las aplicaciones anteriores realizadas en este libro. Este es el control **Image**  que permite seleccionar una imagen del disco duro y establecerla en el formulario como un objeto que responde a los mismos eventos de un *botón de comando*.

La ventaja de utilizar un control **Image** es que podemos utilizar nuestros propios *botones de comando* que hayamos creado en cualquier editor gráfico, ya sea PhotoShop, Paint, FreeHand, etc. Por ejemplo, podemos dibujar dos imágenes que representen un *botón de comando*, uno para su estado inicial y otro para cuando se pulse, y luego cambiar estas imágenes con los eventos **MouseDown** y **MouseUp**.

El siguiente ejemplo muestra una aplicación con cinco imágenes que representan los botones de comando de la aplicación. Estas imágenes serán establecidas con cinco controles **Image**.

- Abra un nuevo proyecto.
- Dibuje en el formulario cinco controles **Image** del mismo tamaño (**sin hacer Copy – Paste**), tal y como se muestra en la siguiente página:



Observe bien en la imagen anterior el tamaño del formulario y establézcale el mismo tamaño al formulario de su aplicación. Si desea puede escribir en la propiedad **Height** el valor **7065** y en la propiedad **Width** el valor **6000** que es el tamaño exacto del formulario de este ejercicio. Puede hacer lo mismo con los cinco controles **Image**. Sus tamaños exactos son **Height = 1035** y **Width = 2025**. Establezca el mismo valor en cada una de las propiedades **Height** y **Width** de cada control **Image** para que queden del mismo tamaño.

- Ahora insertemos otros dos controles **Image** en cualquier parte del formulario (preferiblemente en una esquina). No importa el tamaño de estos últimos dos controles.

NOTA: Debemos de tener un total de siete controles **Image** agregado al formulario cuyos *nombres por defecto* son Image1, Image2, Image3, Image4, Image5, Image6, Image7. Estos nombres lo utilizaremos con mucha frecuencia en la codificación de esta aplicación.

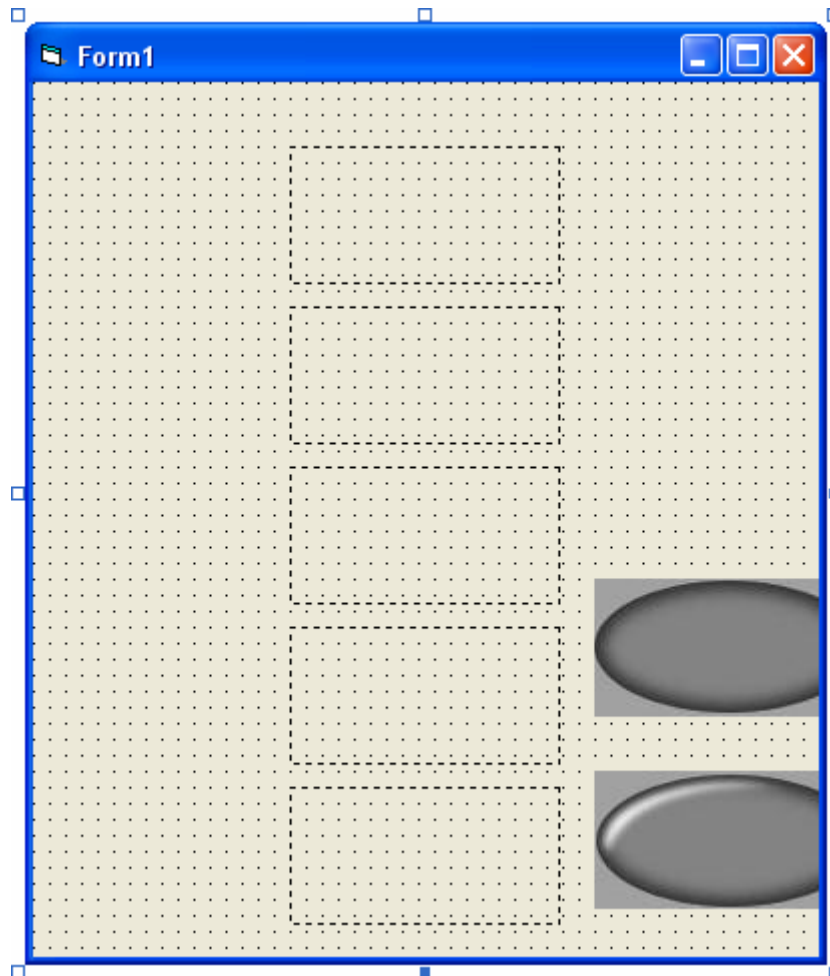
- Haga un solo clic en el sexto control **Image (Image6)**. Seleccione la propiedad **Visible** y establezca el valor **False**. Haga lo mismo con el séptimo control **Image (Image7)**.

- Seleccione nuevamente el sexto control **Image (Image6)**. Busque y seleccione la propiedad **Picture** y haga clic en el botón con los tres puntos suspensivos.

- Busque la imagen llamada **Img03Cap3.jpg** que se encuentra la carpeta **Imágenes** del disco de este libro.



- Seleccione nuevamente el séptimo control **Image (Image7)**. Busque y seleccione la propiedad **Picture** y haga clic en el botón con los tres puntos suspensivos.
- Busque la imagen llamada **Img04Cap3.jpg** que se encuentra en la carpeta **Imágenes** del disco de este libro.
- El formulario tendrá un aspecto como el que se muestra a continuación:



Estas dos imágenes serán las imágenes que se establecerán cuando el usuario presione un botón del Mouse (**MouseDown**) y otra para cuando lo suelte (**MouseUp**). La imagen asignada en el séptimo control **Image (Image7)** será la imagen que tendrá todos los botones de la aplicación cuando esta se inicie, y también cuando el usuario suelte un botón del mouse. Esta será la imagen principal, debido a que tiene el aspecto gráfico de que está arriba, es decir, sin haberse pulsado. Por otro lado, la imagen que posee el sexto control **Image (Image6)** será la imagen que se establecerá en cada evento **MouseDown** de los demás controles **Image**. Esta será la imagen que indicará que el usuario ha pulsado uno de los botones de la aplicación.

Para establecer la imagen que aparecerá en los controles Image1, Image2, Image3, Image4, Image5 cuando la aplicación se ejecute se tendrá que codificar el procedimiento del evento **Load (carga)** del formulario. Recuerde que estudiamos anteriormente que el evento **Load (carga)** ocurre antes que la aplicación sea ejecutada completamente, esto permite realizar un sin número de operaciones antes que el formulario principal de la aplicación se muestre en la pantalla.

- Haga doble clic en cualquier parte del formulario. Aparecerá la ventana del *Editor de Código* y el cursor parpadeando dentro del procedimiento **Load** del formulario.



- Dentro del procedimiento **Load** escriba las siguientes líneas de código:

' Pone el título que aparece en la barra de título de la ventana

```
Form1.Caption = "Ejercicio 9 - Botones grafico"
```

' Cambia el color de fondo del formulario y le pone el color de los botones

```
Form1.BackColor = RGB(162, 162, 162)
```

' Asigna la imagen del séptimo control Image (Image7) a los botones principales

```
Image1.Picture = Image7.Picture ' En el primer botón se pone la séptima imagen
```

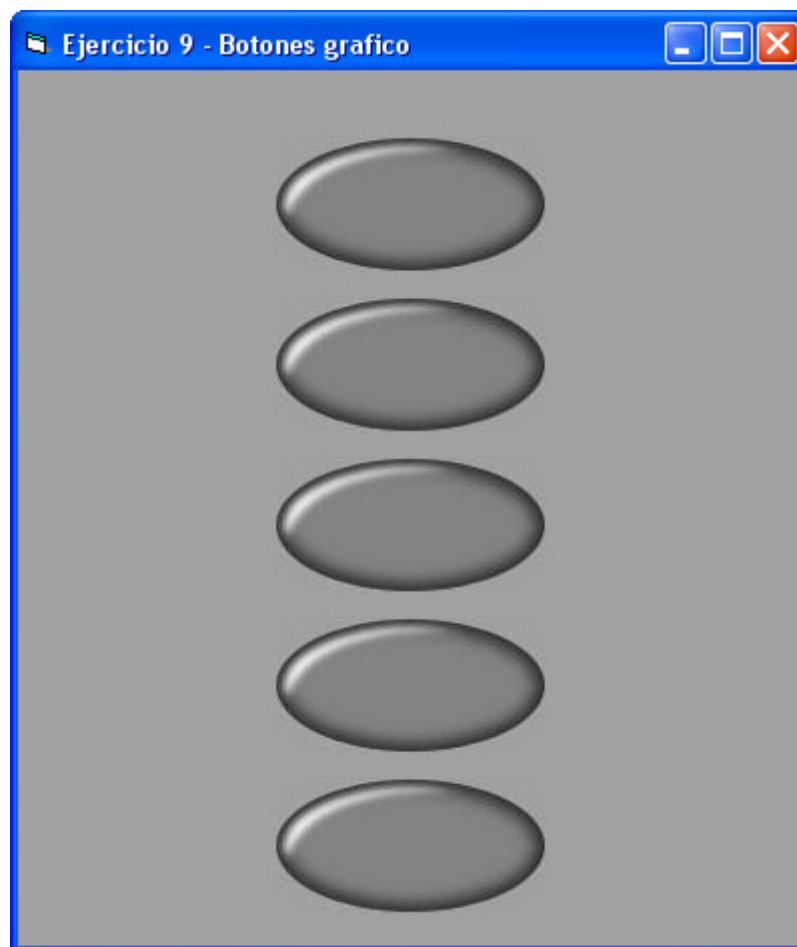
```
Image2.Picture = Image7.Picture ' En el segundo botón se pone la séptima imagen
```

```
Image3.Picture = Image7.Picture ' En el tercer botón se pone la séptima imagen
```

```
Image4.Picture = Image7.Picture ' En el cuarto botón se pone la séptima imagen
```

```
Image5.Picture = Image7.Picture ' En el quinto botón se pone la séptima imagen
```

- Pulse la tecla **[F5]** para correr la aplicación.
- La aplicación se verá como se muestra a continuación:



Si hace clic en cada uno de los botones del formulario no verá ningún cambio, debido a que no se ha codificado su respectivo procedimiento de evento **MouseDown**. Simplemente hemos puesto la imagen del séptimo control **Image** (**Image7**) en los botones principales. Ahora vamos a codificar para que cuando se haga clic en cada uno de los botones se cambie la imagen contenida en dicho botón y se asigne la imagen del sexto control **Image** (**Image6**).



- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Haga clic en **Form1** del *explorador de proyecto* para ver el formulario y sus controles.
- Haga doble clic en el primer control **Image (Image1)**. Aparecerá el procedimiento del evento **Click** de este control. Este evento no nos interesa, sino, el evento **MouseDown**. Seleccione el evento **MouseDown** de la *lista de eventos* y escriba la siguiente línea de código:

```
Image1.Picture = Image6.Picture ' En el primer botón se pone la sexta imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el segundo control **Image (Image2)** y seleccione el evento **MouseDown**. Dentro del procedimiento escriba lo siguiente:

```
Image2.Picture = Image6.Picture ' En el segundo botón se pone la sexta imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el tercer control **Image (Image3)** y seleccione el evento **MouseDown**. Dentro del procedimiento escriba lo siguiente:

```
Image3.Picture = Image6.Picture ' En el tercer botón se pone la sexta imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el cuarto control **Image (Image4)** y seleccione el evento **MouseDown**. Dentro del procedimiento escriba lo siguiente:

```
Image4.Picture = Image6.Picture ' En el cuarto botón se pone la sexta imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el quinto control **Image (Image5)** y seleccione el evento **MouseDown**. Dentro del procedimiento escriba lo siguiente:

```
Image5.Picture = Image6.Picture ' En el quinto botón se pone la sexta imagen
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Haga clic sobre cada uno de los botones de la aplicación.

Podrá observar que los botones cambian su imagen por la imagen que posee el sexto control **Image (Image6)**. Esto es porque se ha codificado cada uno de los procedimientos del evento **MouseDown** de cada control **Image**. También puede observar que cuando hace clic en cada uno de los botones de la aplicación, la imagen que se asigna al botón permanece aún si el usuario ha soltado el botón del mouse. Es necesario que el botón vuelva a su estado inicial cuando el usuario suelte el botón del mouse para que el efecto de pulsación se realice correctamente. Para esto debemos codificar cada uno de los procedimientos del evento **MouseUp** de cada control **Image** y asignar dentro de estos la imagen que posee el séptimo control **Image (Image7)**.

- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.



- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el primer objeto **Image (Image1)** y busque el evento **MouseUp**. Dentro de este procedimiento escriba:

```
Image1.Picture = Image7.Picture ' En el primer botón se pone la séptima imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el segundo objeto **Image (Image2)** y busque el evento **MouseUp**. Dentro de este procedimiento escriba:

```
Image2.Picture = Image7.Picture ' En el segundo botón se pone la séptima imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el tercer objeto **Image (Image3)** y busque el evento **MouseUp**. Dentro de este procedimiento escriba:

```
Image3.Picture = Image7.Picture ' En el tercer botón se pone la séptima imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el cuarto objeto **Image (Image4)** y busque el evento **MouseUp**. Dentro de este procedimiento escriba:

```
Image4.Picture = Image7.Picture ' En el cuarto botón se pone la séptima imagen
```

- Haga clic en **Form1** del *explorador de proyecto*.
- Haga doble clic en el quinto objeto **Image (Image5)** y busque el evento **MouseUp**. Dentro de este procedimiento escriba:

```
Image5.Picture = Image7.Picture ' En el quinto botón se pone la séptima imagen
```

- Pulse la tecla **[F5]** para correr la aplicación.
 - Haga clic sobre cada uno de los botones de la aplicación.
- Podrá observar que los botones vuelven a su estado normal después que el usuario suelta el botón del mouse.
- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
 - Guarde la aplicación con los nombres **FormEjercicio9-3** para el *formulario* y, **Ejercicio9-3** para el *proyecto*.

Explicación:

En esta aplicación hemos codificado solo tres eventos: el evento **Load** del formulario y los eventos **MouseDown** y **MouseUp** de los controles **Image**. Dentro del procedimiento **Load** hemos escrito una codificación muy fácil de entender, además de los comentarios agregados dentro de estos.



La primera línea de código **Form1.Caption = "Ejercicio 9 - Botones gráfico"** pone el título que aparece en la barra de título de la ventana de la aplicación. El texto escrito dentro de las comillas es el texto que se muestra en la barra. La propiedad **Caption** la habíamos estudiado con anterioridad y explicábamos que se utiliza para poner el texto que aparece dentro o sobre un control o formulario.

La segunda línea de código **Form1.BackColor = RGB(162, 162, 162)** pone el color de fondo al formulario. En esta línea de código hemos utilizado una función que le puede parecer muy extraña. Esta es la función **RGB (Red Green Blue)** que permite hacer combinaciones de colores utilizando los colores primarios (Rojo, Verde y Azul). Esta función es muy utilizada para obtener colores que *Visual Basic* no proporciona mediante constantes o valores en hexadecimal. Estos tipos de combinaciones puede obtenerlos mediante opciones de los programas de edición de gráficos e incluso, en Microsoft Word desde el menú Formato – Fondo – Más colores... - Personalizado.

El próximo bloque de código permite establecer la imagen que aparece en cada uno de los botones de la aplicación:

Image1.Picture = Image7.Picture

Image2.Picture = Image7.Picture

Image3.Picture = Image7.Picture

Image4.Picture = Image7.Picture

Image5.Picture = Image7.Picture

Cada una de estas líneas de código tiene algo similar y es que todas tienen asignada la imagen del séptimo control **Image (Image7)**. Esta imagen se almacena en la propiedad **Picture** de los controles **Image**. Si traducimos literalmente la primera línea de este bloque de código estaríamos diciendo lo siguiente: **La imagen de Image1 Es igual a La imagen de Image7**.

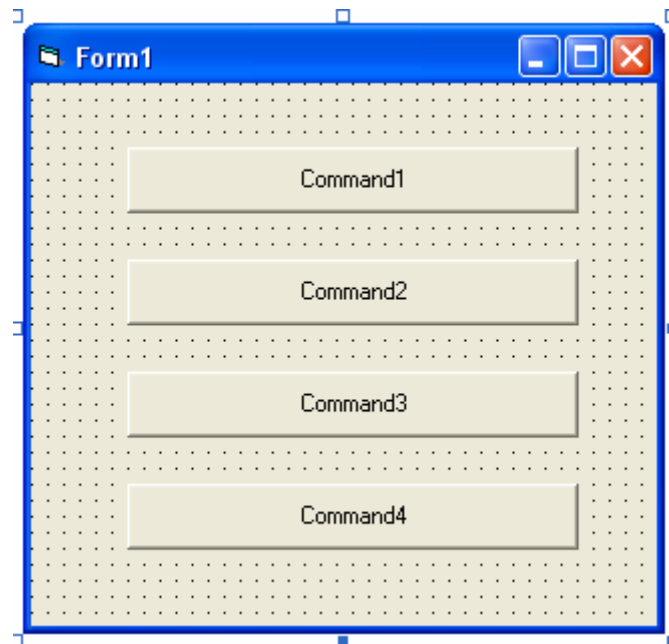
Por otro lado, se ha codificado el evento **MouseDown** de cada objeto **Image** para establecer la imagen que debe de tener dicho botón cuando el usuario presiona un botón del Mouse. También hemos codificado el evento **MouseUp** de cada objeto **Image** para establecer la imagen que deben tener los botones de la aplicación cuando el usuario suelte el botón del Mouse. Para estos procedimientos utilizamos el mismo método del bloque de código anterior.

MouseMove	Este evento ocurre mientras el usuario mueve el puntero del Mouse sobre un objeto. Este evento es muy utilizado para dar efectos a imágenes y botones gráficos. El procedimiento para este evento posee los mismos <i>argumentos</i> que conforman los procedimientos de los eventos MouseDown y MouseUp. Puede utilizar estos argumentos de la misma manera que lo hemos estudiado anteriormente. Este evento es muy utilizado para dar efectos a imágenes y botones gráficos. Por ejemplo, se puede crear una aplicación con un menú compuesto de <i>botones de comando</i>, y que estos cambien de color cada vez que el usuario coloque el puntero del mouse sobre cada uno de ellos.
------------------	---

Ejercicio:

El siguiente ejercicio muestra una aplicación con cuatro botones de comando que cambian de color cada vez que el usuario mueve el puntero del Mouse sobre cada uno de ellos.

- Abra un nuevo proyecto e inserte cuatro *botones de comando*. Los botones de comando deben estar uno de bajo de otro y del mismo tamaño, tal y como se muestra en la figura de la página siguiente:



- Seleccione uno por uno los botones de la aplicación y establezca el valor **1 – Graphical** en la propiedad **Style**. Esto es necesario para que los *botones de comando* puedan cambiar de color.
- Haga doble clic en cualquier parte del formulario. Aparecerá el cursor parpadeando dentro del procedimiento **Load** del formulario. Dentro de este procedimiento escriba el siguiente bloque de código:

```
' Pone el título de la ventana
Form1.Caption = "Ejercicio 10 - Botones gráfico"

' Cambia el color de fondo del formulario y lo pone de color blanco
Form1.BackColor = vbWhite

' El siguiente bloque de código cambia el color de los botones

Command1.BackColor = vbCyan ' Pone el color Cyan al primer botón
Command2.BackColor = vbCyan ' Pone el color Cyan al segundo botón
Command3.BackColor = vbCyan ' Pone el color Cyan al tercer botón
Command4.BackColor = vbCyan ' Pone el color Cyan al cuarto botón
```

NOTA: Si quiere puede cambiar el texto que aparece dentro de cada *botón de comando* usando la propiedad **Caption**.

- Pulse la tecla **[F5]** para ejecutar la aplicación.

Usted puede observar que el color del formulario ha cambiado y también el color de los *botones de comando*, esto es porque hemos modificado el *estado* de cada uno de estos controles utilizando la propiedad **BackColor (Color de Fondo)**. Ahora necesitamos que los *botones de comando* cambien de color cuando se coloque el puntero del mouse sobre cada uno de ellos. Para esto debemos codificar el procedimiento del evento **MouseMove** de cada *botón de comando*.

- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.



Capítulo 3

- Haga clic en **Form1** del *explorador de proyecto* para ver el formulario y sus controles.
- Haga doble clic sobre el primer *botón de comando* y seleccione el evento **MouseMove**. Dentro de este evento escriba la siguiente línea de código.

```
' Verifica si el color es Cyan y le pone el color Amarillo
If Command1.BackColor = vbCyan Then Command1.BackColor = vbYellow
```

- Haga doble clic sobre el segundo *botón de comando* y seleccione el evento **MouseMove**. Dentro del evento escriba la siguiente línea de código.

```
' Verifica si el color es Cyan y le pone el color Amarillo
If Command2.BackColor = vbCyan Then Command2.BackColor = vbYellow
```

- Haga doble clic sobre el tercer *botón de comando* y seleccione el evento **MouseMove**. Dentro del evento escriba la siguiente línea de código.

```
' Verifica si el color es Cyan y le pone el color Amarillo
If Command3.BackColor = vbCyan Then Command3.BackColor = vbYellow
```

- Haga doble clic sobre el cuarto *botón de comando* y seleccione el evento **MouseMove**. Dentro del evento escriba la siguiente línea de código.

```
' Verifica si el color es Cyan y le pone el color Amarillo
If Command4.BackColor = vbCyan Then Command4.BackColor = vbYellow
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Desplace el puntero del Mouse sobre cada uno de los *botones de comando*. Puede observar que a medida que coloca el Mouse sobre cada uno de ellos estos cambian del color Cyan al color Amarillo. Esto es porque hemos codificado dentro del evento **MouseMove (Movimiento del Mouse)** una línea de código que permite evaluar cual es el color actual del *botón de comando* y luego establecer otro si se cumple la condición. Por ejemplo, la línea de código contenida en el primer *botón de comando*:

If Command1.BackColor = vbCyan Then Command1.BackColor = vbYellow

Si traducimos esta línea queremos decir:

Si Boton1.ColorFondo = Cyan Entonces Boton2.ColorFondo = Amarillo

Es necesario utilizar una condición que nos permita verificar el color actual del *botón de comando*, debido a que si escribimos solamente **Command1.BackColor = vbYellow** cada vez que pasemos el puntero del Mouse sobre el *botón de comando*, entonces, el programa le asignará el color Amarillo sin importar que este lo tenga. Entonces, se preguntará usted ¿Cuál es el problema?; el problema es el siguiente: Aunque el programa se lo asigne correctamente y usted no note ningún error, si se detiene un poco y mueve constantemente el puntero del Mouse sobre cualquier botón de la aplicación notará un pequeño parpadeo que indica que se ha vuelto a asignar el color. Este pequeño problema es poco visible con colores, pero si estamos cambiando alguna imagen gráfica constantemente el problema será mucho más notorio. En resumen, es recomendable utilizar siempre una condición para evaluar el *estado* actual del objeto, ya sea un color, un texto, un archivo de imagen u otro elemento.

En esta aplicación es necesario que el color de los *botones de comando* vuelva a Cyan cada vez que el usuario desplace el puntero del Mouse fuera de la superficie de cualquier botón. Para esto,



debemos codificar el evento **MouseMove** del formulario.

- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Haga doble clic en cualquier parte del formulario y seleccione el evento **MouseMove**.
- Dentro del procedimiento **MouseMove** del formulario escriba:

```
' Verifica si el color de los botones es Amarillo y lo pone a Cyan

If Command1.BackColor = vbYellow Then Command1.BackColor = vbCyan

If Command2.BackColor = vbYellow Then Command2.BackColor = vbCyan

If Command3.BackColor = vbYellow Then Command3.BackColor = vbCyan

If Command4.BackColor = vbYellow Then Command4.BackColor = vbCyan
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Mueva el puntero del Mouse sobre cada uno de los botones de la aplicación.

Puede observar que los *botones de comando* toman su color inicial cada vez que el usuario desplaza el puntero del Mouse fuera de la superficie de cualquier botón. Esto es porque se ha codificado dentro del evento **MouseMove** del formulario un bloque de código que verifica si los *botones de comando* tienen el color amarillo, en caso de ser así le asigna el color Cyan que es su color inicial. Cada vez que el usuario mueve el puntero del Mouse sobre el formulario, esta codificación se ejecuta y verifica si alguno de los *botones de comando* tiene el color Amarillo, si encuentra que alguno tiene el color Amarillo, entonces, modifica su *estado BackColor (Color de Fondo)* con el color Cyan.

- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio10-3** para el *formulario* y, **Ejercicio10-3** para el proyecto.

- 3.1.1.4 Métodos de los botones de comando.

En el **Capítulo 1** de este libro, definíamos los *métodos* como funciones que viene programadas por el creador del objeto o control. Estos son utilizado para realizar una tarea en es específico, sin la necesidad de ser programadas por el usuario. Todos los métodos son llamados desde código en tiempo de ejecución a diferencia de algunas propiedades de los objetos. Al igual que existen muchas propiedades compartidas por la mayoría de los objetos, también existen muchos métodos comunes. A continuación, examinaremos algunos de ellos:

Método	Descripción
Move	Permite modificar las propiedades Left, Top, Width y Height en una única operación. Puede modificar todas al mismo tiempo o algunas de ellas. El siguiente ejemplo modifica las propiedades Left, Top y Width de un <i>botón de comando</i>. ' Sitúa el botón de comando en la esquina superior del formulario y modifica su anchura a 2000 twips. Command1.Move 1, 1, 2000 El primer valor es asignado a la propiedad Left, el segundo a la propiedad Top y el tercero a la propiedad Width del <i>botón de comando</i>. Esta sentencia lo que hace es



resumir el uso de estas cuatro propiedades. Por ejemplo, el siguiente código es similar al ejemplo anterior:

```
Command1.Top = 1      ' Modifica la posición superior del objeto.  
Command1.Left = 1     ' Modifica la posición lateral del objeto.  
Command1.Width = 2000 ' Modifica la anchura del objeto a 2000 twips.
```

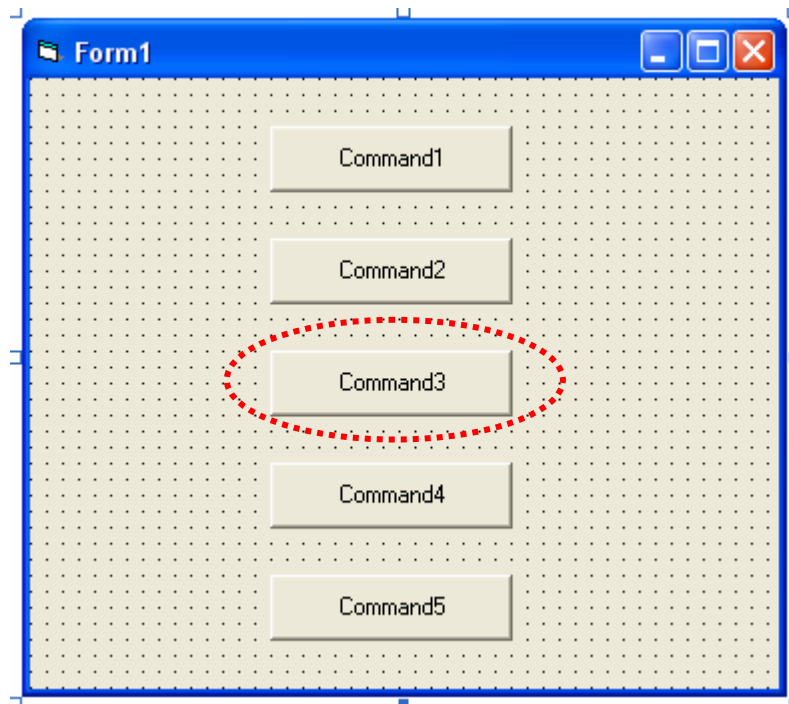
Es preferible utilizar el método **Move** que realizar asignaciones individuales de propiedades por, al menos dos razones: Esta operación es más rápida que realizar cuatro o tres asignaciones distintas y si desea modificar las propiedades **Width** y **Height** de un formulario, cada asignación individual de la propiedad pondrá en marcha un evento **Resize** distinto, con lo que se sobrecargará mucho su código.

COMENTARIO: El método **Move** Sólo se requiere el argumento **Left** (izquierda). Sin embargo, para especificar otros argumentos debe especificar todos los argumentos que le anteceden a dichos argumentos. Por ejemplo, no puede especificar **Width** (ancho) sin especificar **Left** (izquierda) y **Top** (superior). Los argumentos siguientes que no se especifiquen permanecen sin cambios.

Ejercicio:

El siguiente ejercicio muestra una aplicación con cinco *botones de comando*, donde uno de ellos ubicará los otros cuatro en cada esquina de la ventana.

- Abra un nuevo proyecto e inserte cinco *botones de comando*, tal y como se ve en la siguiente figura:



- Haga doble clic sobre el tercer *botón de comando*.
- Dentro del procedimiento **Click** del tercer *botón de comando*, escriba el siguiente bloque de código:

```
Command1.Move 1, 1, 1000 ' Posiciona el primer botón en la esquina superior izquierda  
Command2.Move 4600, 1, 1000 ' Posiciona el segundo botón en la esquina superior derecha  
Command4.Move 1, 4100, 1000 ' Posiciona el cuarto botón en la esquina inferior izquierda  
Command5.Move 4600, 4100, 1000 ' Posiciona el quinto botón en la esquina inferior derecha
```




▪ Pulse la tecla **[F5]** para ejecutar la aplicación.

▪ Haga clic en el tercer *botón de comando* (Command3).

Puede observar como los demás *botones de comando* se posicionan en cada esquina del formulario. También hemos codificado para cambiar el tamaño de cada *botón de comando*.

▪ Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.

▪ Guarde la aplicación con los nombres **FormEjercicio11-3** para el *formulario* y, **Ejercicio11-3** para el proyecto.

Explicación:

En el bloque de código anterior no hemos escrito nada complejo o difícil de entender. Simplemente, se ha utilizado el método **Move** para modificar las propiedades **Left**, **Top** y **Width** de cada *botón de comando*. Pero bien, se preguntará ¿Como he obtenido los valores que posicionan cada *botón de comando* en cada esquina del formulario?, esto es algo muy sencillo; usted puede obtener los valores de cualquier propiedad de un objeto en *Modo de diseño*. Por ejemplo, para realizar el ejercicio anterior, he colocado en *Modo de diseño* cada uno de los *botones de comando* en las esquinas del formulario y he copiado en hoja de papel los valores de las propiedades **Left** y **Top**. Luego situé los *botones de comando* donde estaban anteriormente. Después de obtener estos valores, se ha codificado dentro del procedimiento **Click** del tercer *botón de comando* que es el botón que modifica los estados de los demás *botones de comando*.

En cada línea de código del bloque anterior, podrá observar que el último valor de cada sintaxis no varía. Esto es, porque es el valor de la propiedad **Width (Anchura)** debe de ser el mismo para que cada *botón de comando* tenga el mismo tamaño. Este valor lo he tomado por conveniencia, no es un valor constante. Usted puede modificar este valor y asignar el tamaño deseado a cada *botón de comando*.

Refresh	Este <i>método</i> dibuja nuevamente un control o formulario, es decir, actualiza el aspecto gráfico del control. En muy pocas ocasiones es necesario utilizar este <i>método</i> debido a que <i>Visual Basic</i> refresca automáticamente el aspecto gráfico del control siempre que tiene la oportunidad. Utilice este método cuando desee hacer lo siguiente: • Mostrar información de una variable u otro objeto mientras se este ejecutando un proceso o un procedimiento. • Actualizar rápidamente el texto de un Label o un TextBox para mostrar los valores obtenidos en un <i>proceso repetitivo (Bucle)</i>. • Presentar completamente un formulario mientras se carga otro. • Actualizar el contenido de un cuadro de lista del sistema de archivos, como un control FileListBox. • Actualizar las estructuras de datos de un control Data.
SetFocus	Mueve el <i>foco</i> de entrada al control especificado, es decir, hace que el programa coloque su atención en dicho control. Un objeto tiene el <i>foco</i> cuando el programa muestra de una u otra forma que el objeto esta esperando alguna acción por el usuario. Por ejemplo, una <i>caja de texto</i> tiene el enfoque cuando el cursor esta parpadeando sobre ella. Este es un <i>método</i> muy utilizado en la mayoría de los controles, pero muy pocas veces en los <i>botones de comando</i>.

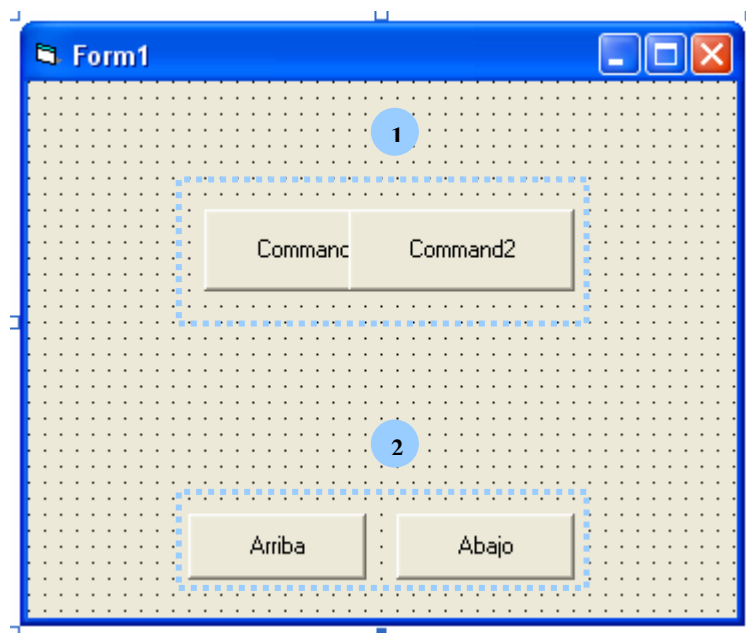


ZOrder	Permite colocar un control o un formulario por debajo o por encima de otros objetos dentro de su nivel grafico. El orden de colocación de los objetos sobre un contenedor recibe el nombre de <i>orden-z</i>. Su formato es: Objeto.ZOrder <i>Posición</i> Donde: Objeto: Especifica el nombre del objeto al cual se quiere cambiar el <i>orden-z</i>. Posición: Un numero entero que indica si el objeto estará sobre otros objetos o por debajo de otros objetos. Si <i>Posición</i> es 0 o se omite, entonces, el objeto o control se coloca por encima de todos los objetos. Si <i>Posición</i> es 1, el objeto se coloca por debajo de todos los objetos. NOTA: No podrá colocar un control lightweight sobre un control estándar, debido a que los controles lightweight se colocan en una dimensión menor a los controles estándar. Entre los principales controles lightweight tenemos: Label, Shape, Image y Line.
---------------	---

Ejercicio:

El siguiente ejercicio muestra una aplicación que permite cambiar el orden-z de un *botón de comando* que se encuentra en conjunto con otros *botones de comando*.

- Abra un nuevo proyecto e inserte dos *botones de comando*, uno encima del otro, tal y como se ve en la primera selección de la siguiente figura:



- Coloque ahora otros dos *botones de comando* la parte inferior del formulario (selección 2).
- Seleccione el tercer *botón de comando* y en la propiedad **Caption** escriba **Arriba**.
- Seleccione el cuarto *botón de comando* y en la propiedad **Caption** escriba **Abajo**.
- Haga doble clic sobre el tercer *botón de comando* y en el procedimiento **Click** escriba la siguiente línea de comando:



Command1.ZOrder 0 ' Coloca el botón 1 por encima del botón 2

- Haga doble clic sobre el cuarto *botón de comando* y en el procedimiento **Click** escriba:

Command1.ZOrder 1 ' Coloca el botón 1 por debajo del botón 2

- Pulse la tecla **[F5]** para correr la aplicación.
- Haga clic en los botones Abajo y Arriba.

Podrá observar como el primer *botón de comando* se coloca por encima o por debajo del segundo *botón de comando*. Esto es porque se ha modificado el *orden-z* del primer *botón de comando*. En caso de haber modificado el orden-z del segundo *botón de comando* en vez del primero el resultado hubiese sido lo mismo.

- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio12-3** para el *formulario* y, **Ejercicio12-3** para el proyecto.

COMENTARIO: En modo de diseño puede utilizar la combinación **Ctrl+J** para situar delante el control seleccionado y la combinación **Ctrl+K** para situar detrás el control.

- 3.1.1.5 Ejercicios prácticos

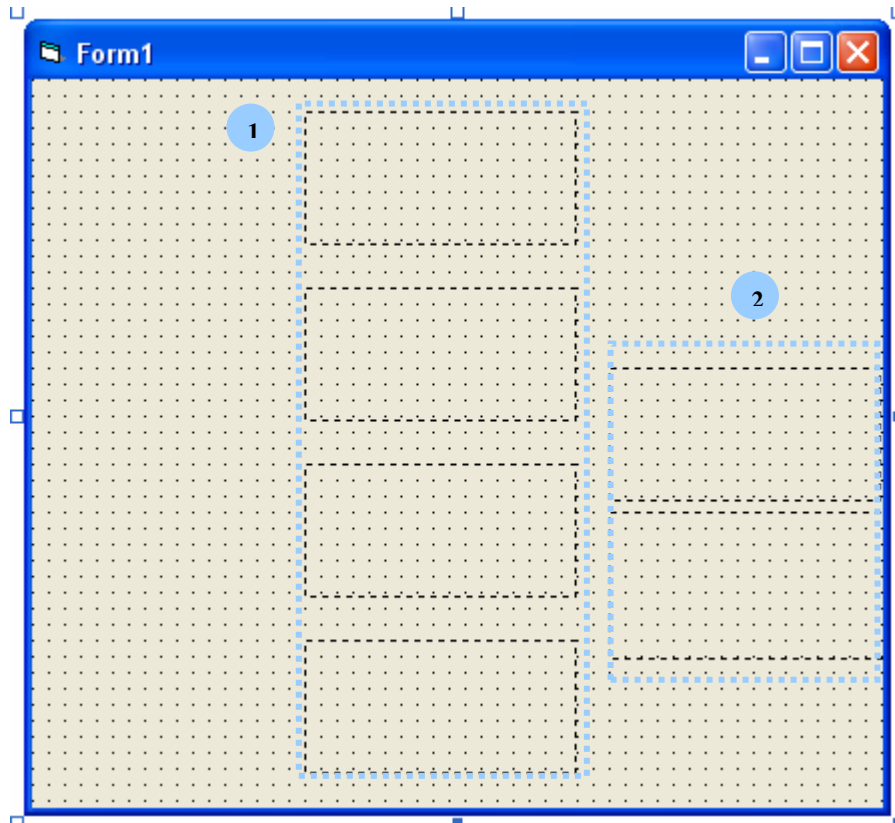
- El siguiente ejercicio muestra una aplicación con cuatro controles **Image** que cambian la imagen cada vez que el usuario desplaza el puntero del Mouse sobre cada uno de ellos.





Para realizar esta aplicación realice los siguientes pasos:

- Abra un nuevo proyecto.
- Modifique el tamaño del formulario y dibuje cuatro controles **Image** del mismo tamaño, cada uno debajo del otro (no muy junto), tal y como se ve en la primera selección de la siguiente figura:



- Dibuje otros dos controles **Image** (Image5, Image6), tal y como se ve en la segunda selección de la figura anterior.
- Seleccione el quinto control **Image** (Image5) y en la propiedad **Visible** establezca el valor **False**. En la propiedad **Picture** especifique la imagen llamada **Img05Cap3.jpg** que se encuentra la carpeta **Imágenes** del disco de este libro.
- Seleccione el sexto control **Image** (Image6) y en la propiedad **Visible** establezca el valor **False**. En la propiedad **Picture** especifique la imagen llamada **Img06Cap3.jpg** de la carpeta **Imágenes**.
- Haga doble clic sobre cualquier parte del formulario y dentro del procedimiento **Load** escriba el siguiente bloque de código:

'Asigna el color de fondo al formulario utilizando la función RGB.

```
Form1.BackColor = RGB(162, 162, 162)
```

'Establece la imagen inicial a los primeros cuatro controles Image.

'La imagen inicial se encuentra asignada en el quinto control Image (Image5).

'Estas imágenes aparecerán cuando inicie o cargue (Load) la aplicación.

```
Image1.Picture = Image5.Picture 'Pone en el control Image1 la imagen inicial.
```

```
Image2.Picture = Image5.Picture 'Pone en el control Image2 la imagen inicial.
```

```
Image3.Picture = Image5.Picture 'Pone en el control Image3 la imagen inicial.
```

```
Image4.Picture = Image5.Picture 'Pone en el control Image4 la imagen inicial.
```



Capítulo 3

- Seleccione el evento **MouseMove** del formulario y escriba el siguiente bloque de código:

```
' Asigna la imagen inicial a los primeros cuatros controles Image.
' La imagen es asignada cuando el puntero del Mouse pasa por el formulario.

If Image1.Picture = Image6.Picture Then Image1.Picture = Image5.Picture
If Image2.Picture = Image6.Picture Then Image2.Picture = Image5.Picture
If Image3.Picture = Image6.Picture Then Image3.Picture = Image5.Picture
If Image4.Picture = Image6.Picture Then Image4.Picture = Image5.Picture
```

- Haga doble clic sobre el primer control **Image** (Image1) y seleccione el evento **MouseMove**. Dentro de este evento escriba la siguiente línea de código:

```
' Cambia y verifica la imagen contenida en Image1 y asigna la imagen del
' sexto control Image (Image6).

If Image1.Picture = Image5.Picture Then Image1.Picture = Image6.Picture
```

- Haga doble clic sobre el segundo control **Image** (Image2) y seleccione el evento **MouseMove**. Dentro de este evento escriba la siguiente línea de código:

```
' Cambia y verifica la imagen contenida en Image2 y asigna la imagen del
' sexto control Image (Image6).

If Image2.Picture = Image5.Picture Then Image2.Picture = Image6.Picture
```

- Haga doble clic sobre el tercer control **Image** (Image3) y seleccione el evento **MouseMove**. Dentro de este evento escriba la siguiente línea de código:

```
' Cambia y verifica la imagen contenida en Image3 y asigna la imagen del
' sexto control Image (Image6).

If Image3.Picture = Image5.Picture Then Image3.Picture = Image6.Picture
```

- Haga doble clic sobre el cuarto control **Image** (Image4) y seleccione el evento **MouseMove**. Dentro de este evento escriba la siguiente línea de código:

```
' Cambia y verifica la imagen contenida en Image4 y asigna la imagen del
' sexto control Image (Image6).

If Image4.Picture = Image5.Picture Then Image4.Picture = Image6.Picture
```

- Pulse la tecla **[F5]** para correr la aplicación.
- Desplace el puntero del Mouse sobre cada uno de los botones de la aplicación. Podrá observar como cada *botón de comando* cambian de apariencia a medida que el usuario desplaza el puntero sobre cada uno de ellos.
- Haga clic en el botón **Cerrar** de la ventana para detener la aplicación.
- Guarde la aplicación con los nombres **FormEjercicio13-3** para el *formulario* y, **Ejercicio13-3** para el proyecto.



Capítulo 3

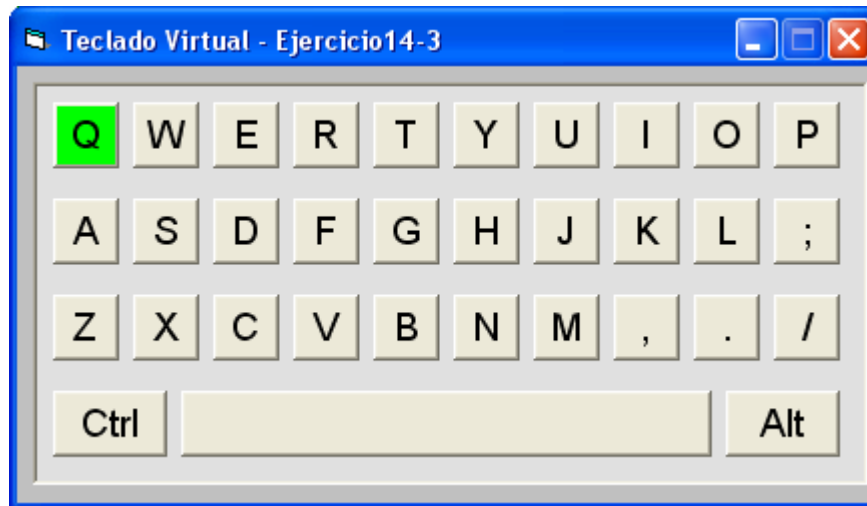
Analice cada uno de los procedimientos codificados en el programa anterior y conteste las siguientes preguntas:

1. ¿Cuáles eventos fueron codificados en la aplicación?
2. ¿Cuándo ocurre el evento **Load** de los formularios y establezca la importancia de este?
3. Enumere la función de cada una de las líneas de código escrita dentro del procedimiento del evento **Load**.
4. ¿Cuánto controles **Image** fueron utilizados en la aplicación?
5. Diga la función de los primeros cuatros controles **Image**.
6. Diga la función de los dos últimos controles **Image** (Image5, Image6).
7. ¿Por qué hemos puesto invisible los dos últimos controles **Image**?
8. ¿Cuál es la utilidad que hacemos de la propiedad **Picture** en la aplicación?
9. ¿Por qué hemos codificado el evento **MouseMove** del formulario?
10. Explique cada una de las líneas de código escrita dentro del procedimiento **MouseMove** del formulario.
11. ¿Cuál evento fue codificado en los primeros cuatro controles **Image**?
12. Explique cada una de las líneas escritas dentro de estos procedimientos.
13. ¿Cuál es la utilidad de la sentencia If - Then?
14. ¿Para que utilizamos la sentencia RGB?
15. ¿Cuál es la importancia de la sentencia RGB?



Capítulo 3

- En la siguiente aplicación hemos creado un teclado virtual similar al que vemos en los programas de mecanografía.



Para realizar esta aplicación ejecute los siguientes pasos:

- Abra un nuevo proyecto.
- Ahora modifiquemos el tamaño del formulario. Escriba en la propiedad **Height** el valor **3780** y en la propiedad **Width** el valor **6540**. **NOTA:** Estos valores no son constantes ni obligatorio, les estoy especificando el valor para que la aplicación quede tal y como la he realizado en mi computador. Cuando usted tenga como proyecto realizar algún programa de mecanografía, estos valores serán el que usted considere para su teclado virtual.
- Inserte un control **PictureBox**  y dibújelo casi del mismo tamaño del formulario. O bien, escriba en **Height: 3015** y en **Width: 6255**.
- Dibuje 33 **CommandButton** dentro del control **PictureBox**. Todos del mismo tamaño, menos el de la barra espaciadora. Este debe ser mucho más grande, tal y como se ve en la imagen. **NOTA:** Si usted opta por hacer **Copy – Paste**, entonces, cuando haga **Paste** aparecerá una ventana preguntándole si desea crear un **Arrays** (arreglo) de controles, responda que **NO**, para que no se creen índices en los botones.
- Establezca los siguientes valores en cada una de las propiedades de los botones de comando.

Control	Propiedad	Valor
Command1	Name	cmdQ
	Caption	Q
	Font	Arial, Tamaño 14
	Style	1 - Graphical
Command2	Name	cmdW
	Caption	W
	Font	Arial, Tamaño 14
	Style	1 - Graphical



Capítulo 3

Control	Propiedad	Valor	Control	Propiedad	Valor
Command3	Name Caption Font Style	cmdE E Arial, Tamaño 14 1 - Graphical	Command13	Name Caption Font Style	cmdD D Arial, Tamaño 14 1 - Graphical
Command4	Name Caption Font Style	cmdR R Arial, Tamaño 14 1 - Graphical	Command14	Name Caption Font Style	cmdF F Arial, Tamaño 14 1 - Graphical
Command5	Name Caption Font Style	cmdT T Arial, Tamaño 14 1 - Graphical	Command15	Name Caption Font Style	cmdG G Arial, Tamaño 14 1 - Graphical
Command6	Name Caption Font Style	cmdY Y Arial, Tamaño 14 1 - Graphical	Command16	Name Caption Font Style	cmdH H Arial, Tamaño 14 1 - Graphical
Command7	Name Caption Font Style	cmdU U Arial, Tamaño 14 1 - Graphical	Command17	Name Caption Font Style	cmdJ J Arial, Tamaño 14 1 - Graphical
Command8	Name Caption Font Style	cmdI I Arial, Tamaño 14 1 - Graphical	Command18	Name Caption Font Style	cmdK K Arial, Tamaño 14 1 - Graphical
Command9	Name Caption Font Style	cmdO O Arial, Tamaño 14 1 - Graphical	Command19	Name Caption Font Style	cmdL L Arial, Tamaño 14 1 - Graphical
Command10	Name Caption Font Style	cmdP P Arial, Tamaño 14 1 - Graphical	Command20	Name Caption Font Style	cmdPuntoYComa ; Arial, Tamaño 14 1 - Graphical
Command11	Name Caption Font Style	cmdA A Arial, Tamaño 14 1 - Graphical	Command21	Name Caption Font Style	cmdZ Z Arial, Tamaño 14 1 - Graphical
Command12	Name Caption Font Style	cmdS S Arial, Tamaño 14 1 - Graphical	Command22	Name Caption Font Style	cmdX X Arial, Tamaño 14 1 - Graphical



Capítulo 3

Control	Propiedad	Valor	Control	Propiedad	Valor
Command23	Name Caption Font Style	cmdC C Arial, Tamaño 14 1 - Graphical	Command33	Name Caption Font Style	cmdAlt Alt Arial, Tamaño 14 1 - Graphical
Command24	Name Caption Font Style	cmdV V Arial, Tamaño 14 1 - Graphical			
Command25	Name Caption Font Style	cmdB B Arial, Tamaño 14 1 - Graphical			
Command26	Name Caption Font Style	cmdN N Arial, Tamaño 14 1 - Graphical			
Command27	Name Caption Font Style	cmdM M Arial, Tamaño 14 1 - Graphical			
Command28	Name Caption Font Style	cmdComa , Arial, Tamaño 14 1 - Graphical			
Command29	Name Caption Font Style	cmdPunto . Arial, Tamaño 14 1 - Graphical			
Command30	Name Caption Font Style	cmdSlash / Arial, Tamaño 14 1 - Graphical			
Command31	Name Caption Font Style	cmdCtrl Ctrl Arial, Tamaño 14 1 - Graphical			
Command32	Name Caption Font Style	cmdBarra Arial, Tamaño 14 1 - Graphical			



Capítulo 3

Después de haber colocado todos los controles necesitamos codificar el evento **KeyDown** del **PictureBox** para leer las teclas que son pulsadas por el usuario. A partir de las teclas leídas el programa irá cambiando el color de cada *botón de comando*.

- Haga doble clic sobre el **PictureBox** (Picture1), y seleccione el evento **KeyDown**.
- Escriba el siguiente bloque de código:

'En este procedimiento esta codificado todo lo relacionado
'para cuando se pulse una tecla como lo dice su evento
'KeyDown (Tecla Abajo).

'El siguiente bloque de código pone el color del botón en verde
'según la tecla pulsada.

```
If KeyCode = vbKeyQ Then cmdQ.BackColor = vbGreen
If KeyCode = vbKeyW Then cmdW.BackColor = vbGreen
If KeyCode = vbKeyE Then cmdE.BackColor = vbGreen
If KeyCode = vbKeyR Then cmdR.BackColor = vbGreen
If KeyCode = vbKeyT Then cmdT.BackColor = vbGreen
If KeyCode = vbKeyY Then cmdY.BackColor = vbGreen
If KeyCode = vbKeyU Then cmdU.BackColor = vbGreen
If KeyCode = vbKeyI Then cmdI.BackColor = vbGreen
If KeyCode = vbKeyO Then cmdO.BackColor = vbGreen
If KeyCode = vbKeyP Then cmdP.BackColor = vbGreen
If KeyCode = vbKeyA Then cmdA.BackColor = vbGreen
If KeyCode = vbKeyS Then cmdS.BackColor = vbGreen
If KeyCode = vbKeyD Then cmdD.BackColor = vbGreen
If KeyCode = vbKeyF Then cmdF.BackColor = vbGreen
If KeyCode = vbKeyG Then cmdG.BackColor = vbGreen
If KeyCode = vbKeyH Then cmdH.BackColor = vbGreen
If KeyCode = vbKeyJ Then cmdJ.BackColor = vbGreen
If KeyCode = vbKeyK Then cmdK.BackColor = vbGreen
If KeyCode = vbKeyL Then cmdL.BackColor = vbGreen
If KeyCode = 192 Then cmdPuntoYComa.BackColor = vbGreen
If KeyCode = vbKeyZ Then cmdZ.BackColor = vbGreen
If KeyCode = vbKeyX Then cmdX.BackColor = vbGreen
If KeyCode = vbKeyC Then cmdC.BackColor = vbGreen
If KeyCode = vbKeyV Then cmdV.BackColor = vbGreen
If KeyCode = vbKeyB Then cmdB.BackColor = vbGreen
If KeyCode = vbKeyN Then cmdN.BackColor = vbGreen
If KeyCode = vbKeyM Then cmdM.BackColor = vbGreen
If KeyCode = 188 Then cmdComa.BackColor = vbGreen
If KeyCode = 190 Then cmdPunto.BackColor = vbGreen
If KeyCode = 111 Then cmdSlash.BackColor = vbGreen
If KeyCode = vbKeyControl Then cmdCtrl.BackColor = vbGreen
If KeyCode = vbKeySpace Then cmdBarra.BackColor = vbGreen
If Shift = vbAltMask Then cmdAlt.BackColor = vbGreen
```



Capítulo 3

- Pulse la tecla **[F5]** para correr la aplicación. Podrá observar que en la pantalla aparece una especie de teclado virtual que espera a que el usuario pulse alguna tecla. Si usted pulsa cualquier tecla desde su teclado verá como las teclas se tornan de color verde. También notará que cuando la tecla es pulsada al menos una sola vez esta permanece con el color verde. Lo ideal sería que el color del botón vuelva a su color original cuando la tecla pulsada sea soltada desde su teclado.
- Para que las teclas tornen a su color normal, detenga la aplicación y codifique el evento **KeyUp (Tecla arriba)** del **PictureBox** con el siguiente bloque de código:

'El siguiente bloque de código pone el color normal de las teclas pulsadas.

```
If KeyCode = vbKeyQ Then cmdQ.BackColor = vbButtonFace
If KeyCode = vbKeyW Then cmdW.BackColor = vbButtonFace
If KeyCode = vbKeyE Then cmdE.BackColor = vbButtonFace
If KeyCode = vbKeyR Then cmdR.BackColor = vbButtonFace
If KeyCode = vbKeyT Then cmdT.BackColor = vbButtonFace
If KeyCode = vbKeyY Then cmdY.BackColor = vbButtonFace
If KeyCode = vbKeyU Then cmdU.BackColor = vbButtonFace
If KeyCode = vbKeyI Then cmdI.BackColor = vbButtonFace
If KeyCode = vbKeyO Then cmdO.BackColor = vbButtonFace
If KeyCode = vbKeyP Then cmdP.BackColor = vbButtonFace
If KeyCode = vbKeyA Then cmdA.BackColor = vbButtonFace
If KeyCode = vbKeyS Then cmdS.BackColor = vbButtonFace
If KeyCode = vbKeyD Then cmdD.BackColor = vbButtonFace
If KeyCode = vbKeyF Then cmdF.BackColor = vbButtonFace
If KeyCode = vbKeyG Then cmdG.BackColor = vbButtonFace
If KeyCode = vbKeyH Then cmdH.BackColor = vbButtonFace
If KeyCode = vbKeyJ Then cmdJ.BackColor = vbButtonFace
If KeyCode = vbKeyK Then cmdK.BackColor = vbButtonFace
If KeyCode = vbKeyL Then cmdL.BackColor = vbButtonFace
If KeyCode = 192 Then cmdPuntoYComa.BackColor = vbButtonFace
If KeyCode = vbKeyZ Then cmdZ.BackColor = vbButtonFace
If KeyCode = vbKeyX Then cmdX.BackColor = vbButtonFace
If KeyCode = vbKeyC Then cmdC.BackColor = vbButtonFace
If KeyCode = vbKeyV Then cmdV.BackColor = vbButtonFace
If KeyCode = vbKeyB Then cmdB.BackColor = vbButtonFace
If KeyCode = vbKeyN Then cmdN.BackColor = vbButtonFace
If KeyCode = vbKeyM Then cmdM.BackColor = vbButtonFace
If KeyCode = 188 Then cmdComa.BackColor = vbButtonFace
If KeyCode = 190 Then cmdPunto.BackColor = vbButtonFace
If KeyCode = 111 Then cmdSlash.BackColor = vbButtonFace
If KeyCode = vbKeyControl Then cmdCtrl.BackColor = vbButtonFace
If KeyCode = vbKeySpace Then cmdBarra.BackColor = vbButtonFace
If Shift = vbAltMask Then cmdAlt.BackColor = vbButtonFace
```



- Ejecute nuevamente la aplicación.

Pulse algunas teclas desde su teclado y observará que los botones que representan las teclas pulsadas vuelven a su color original cuando esta es soltada desde su teclado.

- Detenga la aplicación y guárdela con los nombres **FormEjercicio14-3** para el *formulario* y, **Ejercicio14-3** para el proyecto.

Analice cada uno de los procedimientos codificados en el programa anterior y conteste cada una de las preguntas de la página siguiente:

1. ¿Cuáles eventos fueron codificados en el programa anterior?
2. ¿Por qué se codificó el evento **KeyUp** y no solo el evento **KeyDown**?
3. ¿Cuál es la función del bloque de código del evento **KeyDown**?
4. ¿Qué cambio de código usted ve en los procedimientos de los eventos **KeyUp** y **KeyDown**?
5. Explique porque las teclas vuelven a su color original.
6. ¿Por qué hubo la necesidad de utilizar algunos números delante del argumento **KeyCode**?
7. ¿Por qué utilizamos el argumento **Shift** para leer la tecla ALTERNA en vez de utilizar el evento **KeyCode**?
8. ¿Cuál es la función de las constantes **vbGreen** y **vbButtonFace**?
9. Explique la necesidad de utilizar un objeto **PictureBox** para agregar los *botones de comando*.
10. ¿Por qué no codificamos el evento **KeyPress** en vez de los eventos **KeyUp** y **KeyDown**?



- 3.2.1 Las etiquetas (Labels)

Una *Etiqueta* o *Label* es un objeto gráfico que se coloca en una parte determinada de un contenedor para mostrar un texto que no puede ser modificado directamente por el usuario. Las *etiquetas* son utilizadas principalmente para mostrar textos en la pantalla. Pero no están limitadas solo a esto, pueden ser utilizadas para múltiples operaciones como: opciones de menús, botones de comando, barras de progreso, marquesinas, banners, etc. En este libro no limitaremos el uso de las etiquetas, les daremos múltiples uso tal y como se mencionó anteriormente.

Las *etiquetas*, al igual que los *botones de comando* poseen una gran cantidad de propiedades, métodos y eventos. La mayoría de las propiedades, métodos y eventos de las *etiquetas* son las mismas que las de los *botones de comando* y su comportamiento es también el mismo. Nos limitaremos a definir las propiedades, métodos y eventos más importantes de las *etiquetas*, debido a que las demás han sido definidas en los *botones de comando*.

- 3.2.1.1 Propiedades de las etiquetas

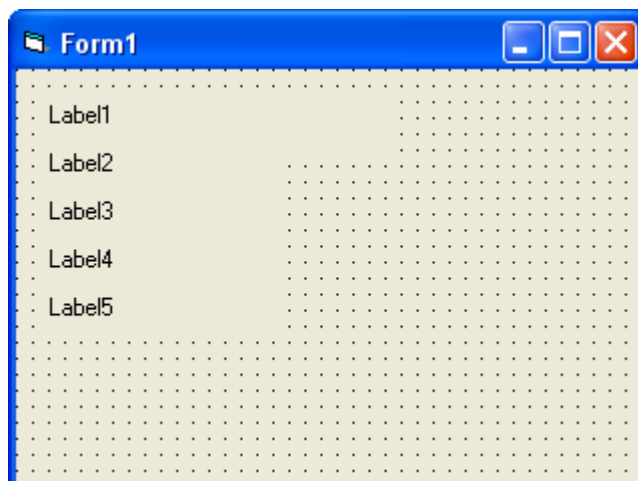
Las *Etiquetas* o *Label* poseen un gran número de propiedades como cualquier otro objeto de *Visual Basic*, pero solo algunas de estas son utilizadas frecuentemente por el programador. Entre las propiedades más importantes de las *Etiquetas* tenemos: **Caption**, **AutoSize**, **BackColor**, **BackStyle**, **Font**, **ForeColor** y **WordWrap**. Usted puede notar que algunas de las propiedades mencionadas anteriormente se encuentran también en los *botones de comando* (*Caption*, *BackColor*, *Font*, *ForeColor*), pero otras son propias de las *Etiquetas* (*AutoSize*, *WordWrap*).

A continuación, definiremos las propiedades más utilizadas e importantes de las *Etiquetas*:

Propiedad	Descripción
Caption	Al igual que en los <i>botones de comando</i> , esta propiedad se utiliza para mostrar un texto sobre un objeto o control colocado sobre un <i>contenedor</i> (Formulario, Frame, Picture, etc). El texto que se escribe en esta propiedad puede ser un campo de una base de datos, un mensaje, algún estado de progreso, una combinación de teclas, un enlace a una página Web, una opción de menú, etc. COMENTARIO: Utilice la propiedad Alignment para justifica el texto escrito en la propiedad Caption . 0 – Justifica a la izquierda, 1 – Justifica a la derecha y 2 – Justifica en el centro.

Ejercicio:

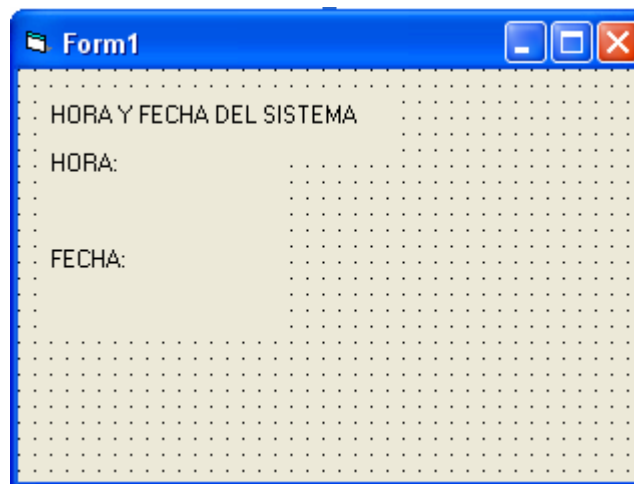
- Abra un nuevo proyecto e inserte cinco Etiquetas (Label1, Label2, Label3, Label4, Label5) en el formulario. Tal y como se ve en la siguiente imagen:





- Seleccione la primera Etiqueta (Label1) y en la propiedad **Caption** escriba: HORA Y FECHA DEL SISTEMA.
- Seleccione la segunda Etiqueta (Label2) y en la propiedad **Caption** escriba: HORA:
- Seleccione la tercera Etiqueta (Label3) y borre el texto que tiene en la propiedad **Caption**: Label3.
- Seleccione la cuarta Etiqueta (Label4) y en la propiedad **Caption** escriba: FECHA:
- Seleccione la quinta Etiqueta (Label5) y borre el texto que tiene en la propiedad **Caption**: Label5.

Los controles sobre el formulario deben tener una apariencia similar a esta:



- Ahora utilizaremos las dos Etiquetas (Label3 y Label4) que no contienen ningún texto para mostrar la hora y la fecha del sistema. Para esto utilizaremos las sentencias **Time** y **Date**. La primera obtiene la hora del sistema y la segunda la fecha. Los valores obtenidos por estas dos sentencias serán asignados en la propiedad **Caption** de la etiqueta correspondiente a la fecha y a la hora.

Necesitamos que la fecha y la hora aparezcan en los Label3 y Label4 respectivamente cuando se ejecute la aplicación. Como usted recordará el único evento que ocurre cuando el formulario es totalmente mostrado en la pantalla es el evento **Load (Carga)** del formulario. Nuestro código debe ser escrito dentro de este evento.

- Haga doble clic sobre cualquier parte del formulario teniendo en cuenta que no puede hacer doble clic sobre ninguna de las Etiquetas.

- Dentro del procedimiento **Load** del formulario escriba las siguientes líneas de código:

```
Label3.Caption = Time    ' Fija la hora en el Label3  
Label5.Caption = Date    ' Fija la fecha en el Label5
```

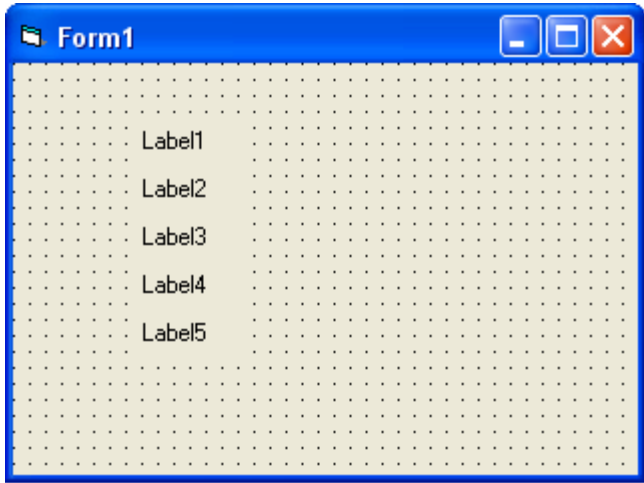
- Pulse la tecla **[F5]** para correr la aplicación.

Podrá notar como aparece la hora y la fecha del sistema en las etiquetas Label3 y Label5. Esto es porque hemos especificado que el valor obtenido por las sentencias **Time** y **Date** se asignen en la propiedad **Caption** de cada una de estas etiquetas.

COMENTARIO: Comúnmente utilizará las etiquetas para mostrar textos o valores obtenidos de algún proceso o alguna función, tal y como se pudo apreciar en el ejercicio anterior.

- Guarde la aplicación con los nombres **FormEjercicio15-3** para el *formulario* y, **Ejercicio15-3** para el proyecto.

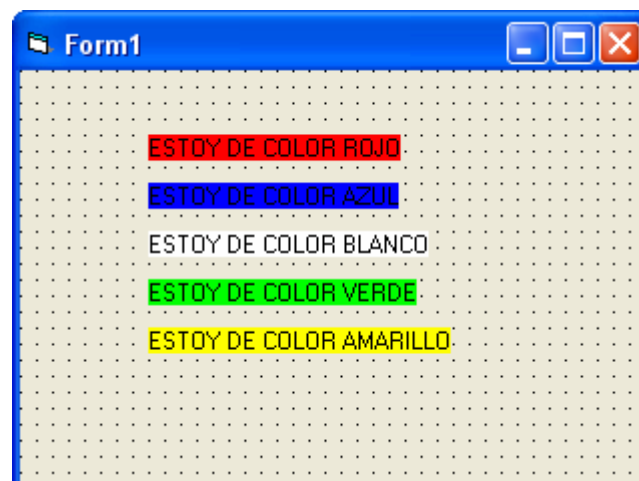


Propiedad	Descripción
AutoSize	En ocasiones el texto escrito en la propiedad Caption de una <i>Etiqueta</i> excede al tamaño de la misma esto dificulta en gran manera la escritura de textos muy largos. Para que una etiqueta se ajuste al tamaño del texto escrito en la propiedad Caption debe establecer el valor True en la propiedad AutoSize de la <i>Etiqueta</i>. Por ejemplo, si en la propiedad Caption de una <i>Etiqueta</i> usted escribe el texto "MICROSOFT VISUAL BASIC 6.0" y el tamaño de la <i>Etiqueta</i> no es suficientemente grande para mostrar el texto, entonces, pasaría algo similar a lo mostrado en la siguiente figura:  Esto ocurriría si el valor de la propiedad AutoSize esta establecido a False. Por otro lado, si establece el valor True en la propiedad AutoSize el control se ajustaría al tamaño del texto. Véalo en la siguiente figura:  COMENTARIO: A medida que va escribiendo en la propiedad Caption de una <i>Etiqueta</i> el valor de la propiedad Width aumenta o disminuye si el valor de la propiedad AutoSize esta establecido a True.
BackColor	Establece el color de fondo de la <i>Etiqueta</i> cuando el valor de la propiedad BackStyle esta establecido a 1 – Opaque. Siempre que inserte una nueva <i>Etiqueta</i> a un formulario podrá cambiar el color de fondo en la propiedad BackColor sin tener que modificar el valor de la propiedad BackStyle, debido a que el valor por defecto de esta propiedad es 1 – Opaque.
Ejercicio: Abra un nuevo proyecto e Inserte 5 Etiquetas tal y como se ve en la siguiente figura: 	



- Seleccione una por una las *Etiquetas* y busque la propiedad **AutoSize** y establezca su valor a **True**.
- En la propiedad **Caption** de la primera *Etiqueta* escriba: ESTOY DE COLOR ROJO.
- En la propiedad **Caption** de la segunda *Etiqueta* escriba: ESTOY DE COLOR AZUL.
- En la propiedad **Caption** de la tercera *Etiqueta* escriba: ESTOY DE COLOR BLANCO.
- En la propiedad **Caption** de la cuarta *Etiqueta* escriba: ESTOY DE COLOR VERDE.
- En la propiedad **Caption** de la quinta *Etiqueta* escriba: ESTOY DE COLOR AMARILLO.
- Seleccione la primera *Etiqueta* y busque la propiedad **BackColor** y desde la ficha *Palette* seleccione cualquier color rojo.
- Seleccione la segunda *Etiqueta* y busque la propiedad **BackColor** y desde la ficha *Palette* seleccione cualquier color azul.
- Seleccione la tercera *Etiqueta* y busque la propiedad **BackColor** y desde la ficha *Palette* seleccione el color blanco.
- Seleccione la cuarta *Etiqueta* y busque la propiedad **BackColor** y desde la ficha *Palette* seleccione cualquier color verde.
- Seleccione la quinta *Etiqueta* y busque la propiedad **BackColor** y desde la ficha *Palette* seleccione cualquier color amarillo.

Las etiquetas sobre el formulario deben tener una apariencia similar a la siguiente:



- Guarde la aplicación con los nombres **FormEjercicio16-3** para el *formulario* y, **Ejercicio16-3** para el proyecto.

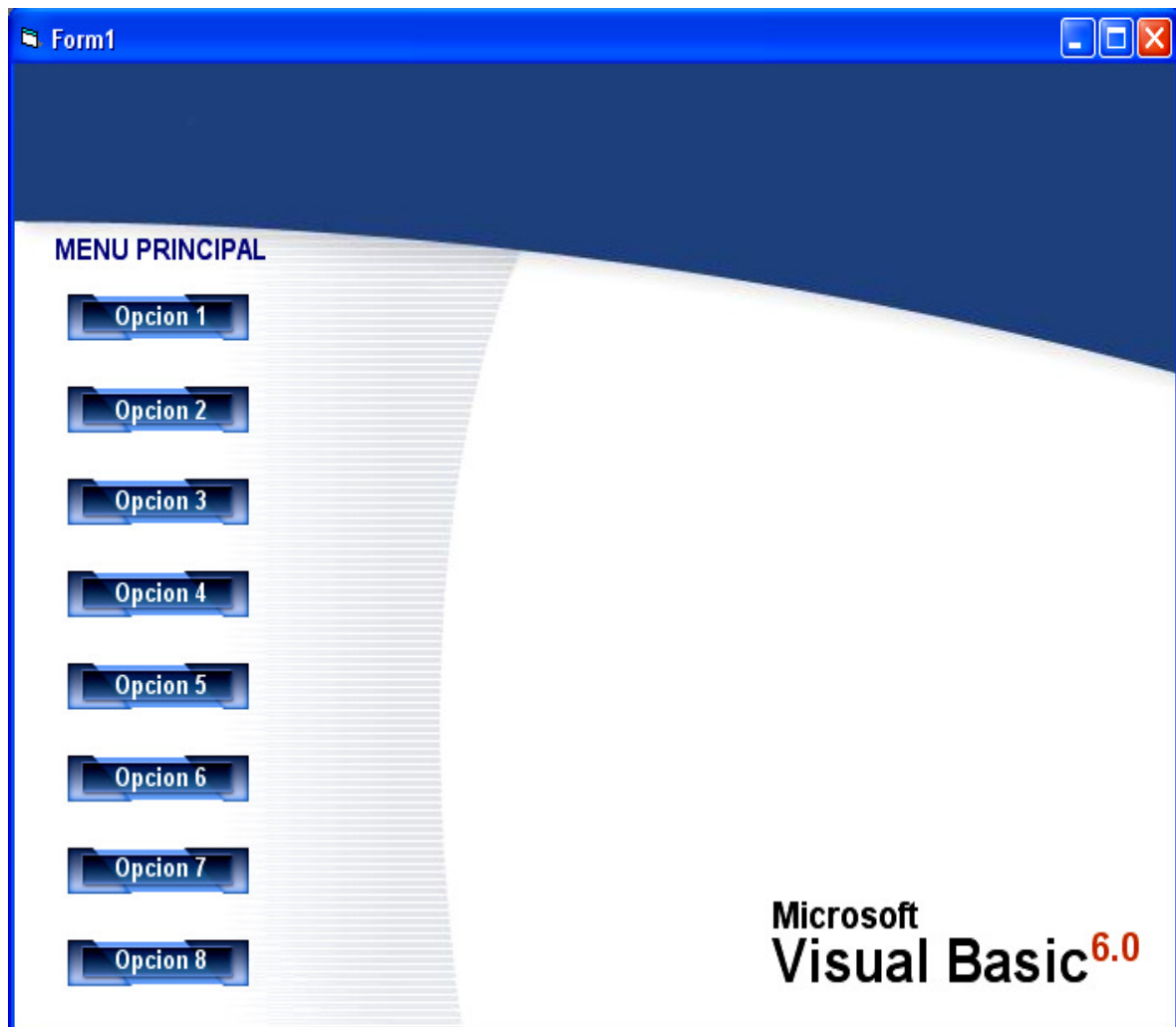
BackStyle	Esta propiedad especifica si la <i>Etiqueta</i> tendrá color de fondo o si totalmente transparente. Puede tomar los valores 1 – Opaque y 0 – Transparent. El valor 1 – Opaque es el valor por defecto de esta propiedad y especifica que el color que se especifique en la propiedad BackColor se asignará y se mostrará en la <i>Etiqueta</i>. El segundo valor de esta propiedad indica que la <i>Etiqueta</i> no tendrá ningún color de fondo. Este valor es bastante usado por los programadores debido a que permite asignar textos sobre imágenes sin afectar ni opacar la parte de la imagen donde la <i>Etiqueta</i> es colocada.
------------------	--

NOTA: La propiedad **BackColor** de la etiqueta se pasa por alto cuando el valor **0 – Transparent** esta establecido en la propiedad **BackStyle**.

**Ejercicio:**

La siguiente aplicación muestra un entorno gráfico compuesto de un menú con varias opciones. Donde cada opción está representado por una imagen gráfica en forma de botón y sobre cada imagen una Etiqueta que describe cada una de las opciones del menú.

Al concluir el ejercicio tendremos una aplicación similar a la que se presenta en la siguiente imagen:



Para realizar la aplicación anterior siga los siguientes pasos:

- Abra un nuevo proyecto.
- En la propiedad **Height** del formulario escriba 3600 y en la propiedad **Width** escriba 4800.
- En la propiedad **Picture** del formulario establezca la imagen **Img07Cap3.jpg** de la carpeta **Imágenes** que se distribuye en el disco de este libro.
- Inserte una **Etiqueta (Label1)** en cualquier parte del formulario. Establezca los siguientes valores en las propiedades de la Etiqueta:

Propiedad	Valor	Propiedad	Valor	Propiedad	Valor
AutoSize	True	Caption	MENU PRINCIPAL	Font	Arial 11, negrita.
Top	1320	Left	360	ForeColor	{ EMBED PBrush }

g. Carlos Manuel Rodríguez



Capítulo 3

Propiedad	Valor
BackStyle	0 - Transparent

▪ Inserte un control **Image** en cualquier parte del formulario y en la propiedad **Picture** establezca la imagen **Img08Cap3.Gif** de la carpeta **Imágenes**.

▪ Busque la propiedad **Left** de la imagen y establezca el valor 480. En la propiedad **Top** establezca el valor 1800.

▪ Ahora hagamos una copia de la misma imagen para tener un segundo botón. Haga un clic derecho sobre la imagen (**Image1**) y seleccione la opción **Copy** del menú contextual.

▪ Haga clic derecho en cualquier parte del formulario y seleccione la opción **Paste** y en la pregunta "Do you want to create a control array?", "¿Quiere crear un arreglo control?" conteste que **No**.

▪ En la nueva imagen (**Image2**) establezca en la propiedad **Left** el valor 480. En la propiedad **Top** establezca el valor 2520.

▪ Agreguemos el tercer botón del menú. Haga clic derecho en cualquier parte del formulario y seleccione la opción **Paste**, y a continuación, conteste **No** a la pregunta.

▪ En la tercera imagen (Image3) establezca en la propiedad **Left** el valor 480. En la propiedad **Top** establezca el valor 2340.

▪ Agreguemos el cuarto botón del menú. Haga clic derecho en cualquier parte del formulario y seleccione la opción **Paste**, y a continuación, conteste **No** a la pregunta.

▪ En la cuarta imagen (Image4) establezca en la propiedad **Left** el valor 480. En la propiedad **Top** establezca el valor 3960.

▪ Agreguemos el quinto botón del menú como lo ha hecho anteriormente. En la propiedad **Left** establezca el valor 480 y en la propiedad **Top** el valor 4680.

▪ Agregue el sexto botón del menú. En la propiedad **Left** establezca el valor 480 y en la propiedad **Top** el valor 5400.

▪ Agregue el séptimo botón del menú. En la propiedad **Left** establezca el valor 480 y en la propiedad **Top** el valor 6120.

▪ Agregue el octavo y último botón del menú. En la propiedad **Left** establezca el valor 480 y en la propiedad **Top** el valor 6840.

Después de haber agregado los botones del menú principal es necesario escribir un texto sobre cada uno de ellos para diferenciarlos de las demás opciones del menú. Para esto utilizaremos ocho Etiquetas y en cada Etiqueta se especificará el texto correspondiente a cada opción del menú.

▪ Inserte una Etiqueta en cualquier parte del formulario y establezca los siguientes valores en las propiedades especificadas:

Propiedad	Valor	Propiedad	Valor	Propiedad	Valor
AutoSize	True	Caption	Opción 1	Font	Arial 10, negrita.
Top	1850	Left	840	ForeColor	{ EMBED PBrush } &H00FFFFFF& (Blanco)



Capítulo 3

- Haga una copia de la Etiqueta y péguela en cualquier parte del formulario. En la propiedad **Caption** escriba **Opción 2**, en la propiedad **Left** especifique el valor 840 y en la propiedad **Top** escriba 2560.
- Haga una tercera copia y péguela en cualquier parte del formulario. En la propiedad **Caption** escriba **Opción 3**, en la propiedad **Left** especifique el valor 840 y en la propiedad **Top** 3285.
- Repita los mismos pasos hasta completar las ocho Etiquetas. Los valores para las propiedades **Left** y **Top** de las demás Etiquetas se muestran en siguiente tabla:

Control	Propiedad	Valor
Label5	Left	840
	Top	4005
Label6	Left	840
	Top	4725
Label7	Left	840
	Top	5445
Label8	Left	840
	Top	6180
Label10	Left	840
	Top	6885

Ahora diseñemos el eslogan que aparece en la esquina inferior derecha del formulario de la aplicación. Para esto utilizaremos tres Etiquetas, la primera para escribir "Microsoft", la segunda para escribir "Visual Basic" y la tercera para escribir "6.0".

- Inserte una Etiqueta en cualquier parte del formulario y establezca en las propiedades especificadas los valores que se muestran en la siguiente tabla:

Propiedad	Valor
Caption	Microsoft
AutoSize	True
BackStyle	0 - Transparent
Left	6120
Top	6480
Font	Arial, Negrita, Tamaño 16.

- Inserte otra Etiqueta sobre el formulario y establezca los siguientes valores en las propiedades especificadas:

Propiedad	Valor
Caption	Visual Basic
AutoSize	True
BackStyle	0 - Transparent
Left	6120
Top	6720
Font	Arial, Negrita, Tamaño 26.

- Inserte la una ultima etiqueta sobre el formulario y establezca los siguientes valores en las propiedades especificadas:

Propiedad	Valor
Caption	6.0



Propiedad	Valor
AutoSize	True
BackStyle	0 - Transparent
Left	9120
Top	6705
Font	Arial, Negrita, Tamaño 16.
ForeColor	{ EMBED PBrush } &H000040C0& (Mamey)

- Pulse la tecla **F5** para correr la aplicación.

Al correr la aplicación notará que si hacemos clic sobre cada una de las opciones del menú principal estas no realizan ninguna operación, esto es, porque no se han programado los eventos necesarios para cada una de las opciones del menú. Más adelante programaremos estos eventos para que cada opción del menú principal tenga alguna función de utilidad en la aplicación.

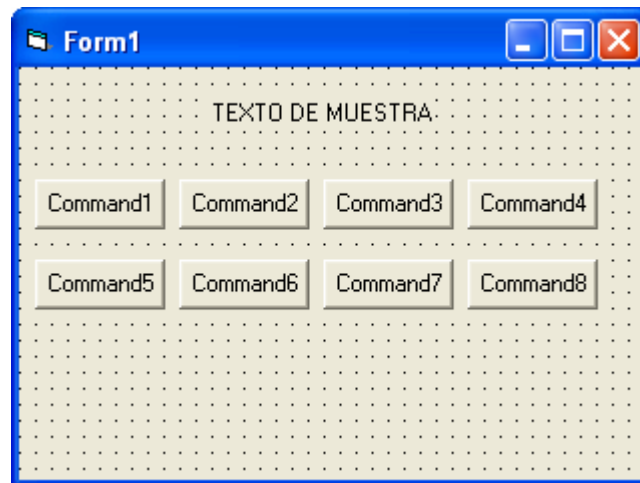
- Guarde la aplicación con los nombres **FormEjercicio17-3** para el *formulario* y, **Ejercicio17-3** para el proyecto.

Font	Como pudimos notar en la aplicación anterior, esta propiedad es utilizada para cambiar el <i>tipo de fuente</i> del texto especificado en la propiedad Caption de la Etiqueta. El <i>tipo de fuente</i> corresponde al tipo de letra, estilo de fuente, efectos y tamaño de la letra contenida en un control que soporte esta propiedad. COMENTARIO 1: La propiedad Font desde el <i>Editor de código</i> representa un objeto cuyas propiedades puede usar de forma independiente. Por ejemplo, si desea poner en negrita el texto contenido en una etiqueta llamada Label1: <pre>Label1.Font.Bold = True</pre> Las propiedades más utilizadas del objeto Font son: Bold (Negrita), Italic (Cursiva), Name (Nombre del tipo de letra), Size (Tamaño), StrikeThrough (Tachado) y COMENTARIO 2: Utilice la propiedad Font en <i>Modo de diseño</i> cuando quiera darle formato al texto contenido en los objetos de la aplicación y no espera cambiarlo muy a menudo. Por otro lado, cuando utilice el objeto Font desde el <i>Editor de código</i> hágalo cuando desee cambiar el formato del texto con mucha frecuencia o cuando el formato del texto tenga que cambiar por algún evento realizado por el usuario. Por ejemplo, cambiar el color de una opción de un menú cuando el usuario coloca el puntero del Mouse sobre dicha opción.
-------------	--

Ejercicio:

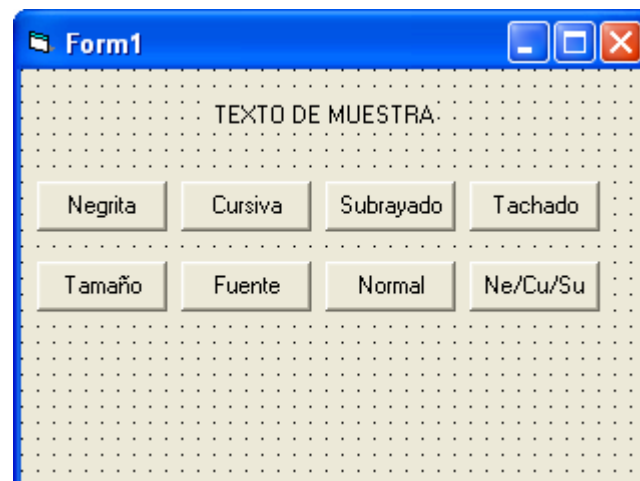
A continuación, se muestra una aplicación compuesta por ocho *botones de comando*. Cada *botón de comando* modificará una de las propiedades del objeto **Font** de una Etiqueta (Label) que estará colocada en la parte más arriba de los *botones de comando*.

- Abra un nuevo proyecto.
- Inserte una Etiqueta en la parte más arriba del formulario y en el centro.
- En la propiedad **Caption** escriba TEXTO DE MUESTRA. En la propiedad **AutoSize** establezca el valor **True**.
- Inserte ocho *botones de comando* del mismo tamaño uno al lado del otro y debajo de la Etiqueta



- Seleccione el primer *botón de comando* y en la propiedad **Caption** escriba Negrita.
- Seleccione el segundo *botón de comando* y en la propiedad **Caption** escriba Cursiva.
- Seleccione el tercer *botón de comando* y en la propiedad **Caption** escriba Subrayado.
- Seleccione el cuarto *botón de comando* y en la propiedad **Caption** escriba Tachado.
- Seleccione el quinto *botón de comando* y en la propiedad **Caption** escriba Tamaño.
- Seleccione el sexto *botón de comando* y en la propiedad **Caption** escriba Fuente.
- Seleccione el séptimo *botón de comando* y en la propiedad **Caption** escriba Normal.
- Seleccione el octavo *botón de comando* y en la propiedad **Caption** escriba Ne/Cu/Su.

La aplicación debe tener un aspecto similar a la que se muestra en la siguiente imagen:



Ahora codifiquemos el *evento Click* de cada uno de los **botones de comando** para que cuando hagamos clic sobre cada uno de ellos este realice la operación indicada en su propiedad **Caption**.

- Haga doble clic sobre el primer *botón de comando* (el que tiene por texto Negrita) y en el procedimiento del *evento Click* la siguiente línea de código:

```
Label1.Font.Bold = True ' Hace que la Etiqueta se ponga en negrita.
```

- Haga doble clic sobre el segundo *botón de comando* (el que tiene por texto Cursiva) y escriba dentro del procedimiento del *evento Click* la siguiente línea de código:



Capítulo 3

Label1.Font.Italic = **True** ' Hace que la Etiqueta se ponga en cursiva.

- Dentro del *evento Click* del botón Subrayado:

Label1.Font.Underline = **True** ' Hace subrayar la Etiqueta.

- Dentro del *evento Click* del botón Tachado:

Label1.Font.Strikethrough = **True** ' Hace tachar la Etiqueta.

- Dentro del *evento Click* del botón Tamaño:

```
Dim valor As Integer      ' Declara la variable numérica para almacenar el tamaño.
On Error Resume Next     ' Si el usuario introduce un valor incorrecto no se detiene la aplicación

' Muestra una caja que permite que el usuario escriba el tamaño de la letra y lo almacena en la
' variable valor.
valor = InputBox("Introduzca el tamaño de la letra:", "Tamaño de letra")

Label1.Font.Size = valor   ' Establece el tamaño introducido a la Etiqueta.
```

- Dentro del *evento Click* del botón Fuente:

```
Dim Tipo As String       ' Declara la variable de texto para almacenar el tipo de letra.
On Error Resume Next     ' Si el usuario introduce un valor incorrecto no se detiene la aplicación

' Muestra una caja que permite introducir el nombre del tipo de letra y lo almacena en la
' variable Tipo.
Tipo = InputBox("Introduzca nombre del tipo de letra", "Tipo de letra")

Label1.Font.Name = Tipo   ' Establece el tipo de letra a la Etiqueta.
```

- Dentro del *evento Click* del botón Normal:

Label1.Font.Bold = False	'Le quita la negrita a la Etiqueta
Label1.Font.Italic = False	'Le quita la cursiva a la Etiqueta
Label1.Font.Underline = False	'Le quita el subrayado a la Etiqueta
Label1.Font.Strikethrough = False	'Le quita el tachado a la Etiqueta
Label1.Font.Name = "MS Sans Serif"	'Pone el tipo de fuente predeterminado
Label1.Font.Size = 8	'Pone el tamaño predeterminado

- Dentro del *evento Click* del botón Ne/Cu/Su (Negrita/Cursiva/Subrayado):

Label1.Font.Bold = **True** ' Pone Negrita a la Etiqueta.
 Label1.Font.Italic = **True** ' Pone cursiva a la Etiqueta.
 Label1.Font.Underline = **True** ' Hace subrayar la Etiqueta.

- Pulse la tecla **F5** para correr la aplicación.

- Pruebe cada uno de los *botones de comando* de la aplicación y observe como la Etiqueta cambia de apariencia cada vez que pulsamos uno de los botones. En el caso del botón **Tamaño** introduzca un valor numérico, por ejemplo, 14. En el botón **Fuente** un tipo de fuente, por ejemplo, Arial.



Capítulo 3

- Guarde la aplicación con los nombres **FormEjercicio18-3** para el *formulario* y, **Ejercicio18-3** para el proyecto.

ForeColor	La propiedad ForeColor devuelve o establece el color del texto contenido en un control. En una Etiqueta esta propiedad afecta el color del texto escrito en la propiedad Caption. Al igual que la propiedad BackColor esta propiedad puede tomar valores hexadecimales, constantes de colores del sistema y constantes de colores básicos. COMENTARIO: En las Etiquetas esta propiedad es muy utilizada por los programadores para crear efectos gráficos en menús y en botones gráficos. Cuando coloca una Etiqueta sobre un botón gráfico puede hacer que esta cambie de color al usuario colocar el puntero Mouse sobre el, o bien, puede hacer que cambie de color cuando el usuario haga clic sobre el botón. Esto es posible codificando los eventos MouseMove y Click del botón gráfico o de la Etiqueta.
------------------	--

- 3.2.1.2 Eventos sobre las etiquetas

La mayoría de los controles estándar de *Visual Basic* se ven afectados por los mismos eventos, aunque algunos eventos son usados en otros controles más que en otros. Este es el caso de las Etiquetas que poseen las mismas propiedades de los *botones de comando* pero solo algunas de ellas son realmente útiles al momento de programarlas.

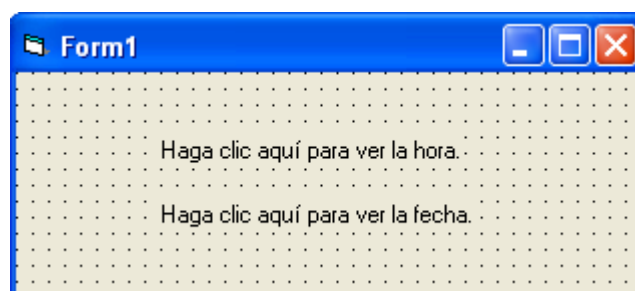
En las Etiquetas los eventos más utilizados son: **Click**, **DbClick**, **MouseMove**, **MouseDown** y **MouseUp**. Estos eventos tienen el mismo comportamiento en los demás controles de Visual Basic y su uso dependerá de la necesidad del programador. Por tal razón, daremos una mínima definición de estos, ya que lo hemos definido con anterioridad en los *botones de comando*.

Click	Ocurre cuando el usuario presiona uno de los botones del Mouse. En las Etiquetas el procedimiento para este evento es programado cuando la Etiqueta es utilizada como un botón de comando, o bien, para ejecutar una acción específica cuando el usuario pulse un botón del Mouse. Por ejemplo, mostrar una ventana, mostrar un mensaje, ejecutar un programa, abrir una página Web, etc.
--------------	---

Ejercicio:

A continuación, se muestra una aplicación que muestra la hora y la fecha cuando se hace clic sobre una de las Etiquetas.

- Abra un nuevo proyecto.
- Inserte dos Etiquetas una debajo de otra.
- Establezca el valor **True** en la propiedad **AutoSize** de ambas Etiquetas.
- En la propiedad **Caption** de la primera Etiqueta escriba "Haga clic aquí para ver la hora", y en la propiedad **Caption** de la segunda Etiqueta escriba "Haga clic aquí para ver la fecha". La aplicación tendría un aspecto similar al de la siguiente imagen:





Capítulo 3

- Haga doble clic sobre la primera Etiqueta y escriba:

```
MsgBox("La hora del sistema es:" & Time) ' Muestra una ventana con la hora del sistema.
```

- Haga doble clic sobre la segunda Etiqueta y escriba:

```
MsgBox("La fecha del sistema es:" & Date) ' Muestra una ventana con la fecha del sistema.
```

- Pulse la tecla **F5** para correr la aplicación.

Haga clic sobre cada uno de las Etiquetas y podrá observar que cuando ocurre el *evento Click* sobre una de las Etiquetas esta muestra la hora o la fecha del sistema.

Explicación:

Para esta aplicación hemos utilizado tres sentencias que ya la habíamos visto en aplicaciones anteriores que son las sentencias **MsgBox**, **Time** y **Date**. La primera es utilizada para mostrar rápidamente una caja de mensaje, la segunda para mostrar la hora del sistema y la tercera para mostrar la fecha del sistema. En esta aplicación hemos combinado la sentencia **Time** y **Date** con la sentencia **MsgBox** para mostrar un mensaje de texto adjunto a la hora o a la fecha del sistema.

En la primera línea de código vemos lo siguiente:

```
MsgBox("La hora del sistema es:" & Time)
```

La sentencia **MsgBox** muestra la cadena de texto "**La hora del sistema es:**". Después de esto vemos un carácter llamado **Ampersand (&)** que se utiliza para unir dos o más cadenas de texto. Este carácter servirá para unir la hora del sistema proporcionada por la sentencia **Time** con la primera cadena de caracteres. Esto mismo lo hacemos en el evento **Click** de la segunda Etiqueta con la sentencia **Date** que devuelve la fecha del sistema.

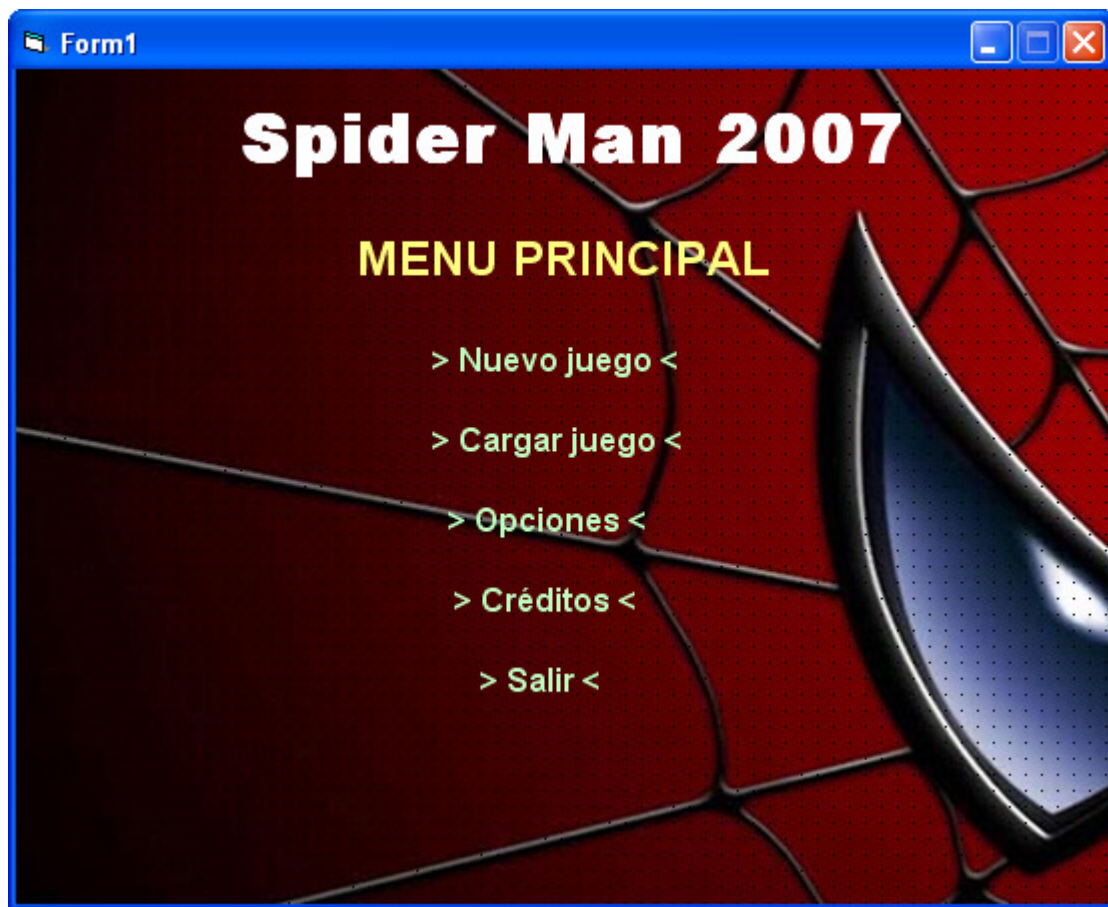
- Detenga la aplicación y guárdela con los nombres **FormEjercicio19-3** para el *formulario* y, **Ejercicio19-3** para el proyecto.

DbClick	Al igual que en los <i>botones de comando</i> este evento ocurre cuando el usuario pulsa dos veces un botón del Mouse. Este evento no es muy utilizado en las Etiquetas, pero en algunas ocasiones encontrará alguna función de utilidad el programar este evento.
MouseMove	Ocurre cuando el usuario mueve el puntero del Mouse sobre la zona donde esta dibujada la Etiqueta. Es uno de los eventos de las Etiquetas más utilizados por los programadores ya que permite crear y mostrar inmensidades de efectos gráficos cuando el usuario coloca el puntero del Mouse sobre la Etiqueta. COMENTARIO: Utilice siempre este evento cuando quiera realizar efectos gráficos sobre menús personalizados, o bien, cuando quiera crear efectos animados mientras el usuario mueva el puntero del Mouse en la zona donde este dibujada la Etiqueta.

Ejercicio:

1.- El siguiente ejercicio muestra una aplicación compuesta por un menú principal compuesto de cinco opciones, donde cada opción del menú cambia de apariencia cada vez que el usuario coloca el puntero del Mouse sobre una de ellas.

La aplicación tendrá una apariencia como la que se muestra en la figura de la siguiente página:



- Abra un nuevo proyecto.
- En la propiedad **Height** establezca el valor 6765 y en la propiedad **Width** el valor 8310.
- Busque la propiedad **StartPosition** del formulario y seleccione el valor **2 – CenterScreen**. Esto permite que el formulario aparezca en el centro de la pantalla cuando se ejecute la aplicación.
- En la propiedad **Picture** establezca la imagen **Img09Cap3.jpg** de la carpeta **Imágenes**.
- Inserte una Etiqueta en cualquier parte del formulario y establezca los siguientes valores en las propiedades especificadas:

Propiedad	Valor
Caption	Spider Man 2007
AutoSize	True
BackStyle	0 - Transparent
Left	1680
Top	120
Font	Arial Black, Negrita, Tamaño 26.
ForeColor	☐ &H00FFFFFF& (Blanco)

- Inserte otra Etiqueta en cualquier parte del formulario y establezca los siguientes valores en las propiedades especificadas:



Propiedad	Valor
Caption	MENU PRINCIPAL
AutoSize	True
BackStyle	0 - Transparent
Left	2550
Top	1200
Font	Arial, Negrita, Tamaño 18.
ForeColor	 &H0080FFFF& (Amarillo claro)

▪ Ahora dibujemos las cinco opciones del menú principal insertando cinco Etiquetas una debajo de la otra.

▪ Seleccione cada una de las opciones del menú y establezca los valores mostrados en las siguientes tablas para la opción correspondiente:

Opción 1

Propiedad	Valor
Caption	> Nuevo juego <
AutoSize	True
BackStyle	0 - Transparent
Left	3120
Top	2040
Font	Arial, Negrita, Tamaño 12.
ForeColor	 &H00C0FFC0& (Verde lumínico)

Opción 2

Propiedad	Valor
Caption	> Cargar juego <
AutoSize	True
BackStyle	0 - Transparent
Left	3120
Top	2640
Font	Arial, Negrita, Tamaño 12.
ForeColor	 &H00C0FFC0& (Verde lumínico)

Opción 3

Propiedad	Valor
Caption	> Opciones <
AutoSize	True
BackStyle	0 - Transparent
Left	3240
Top	3240
Font	Arial, Negrita, Tamaño 12.
ForeColor	 &H00C0FFC0& (Verde lumínico)

Opción 4

Propiedad	Valor
Caption	> Créditos <
AutoSize	True
BackStyle	0 - Transparent
Left	3285
Top	3840
Font	Arial, Negrita, Tamaño 12.
ForeColor	 &H00C0FFC0& (Verde lumínico)

Opción 5

Propiedad	Valor
Caption	> Salir <
AutoSize	True
BackStyle	0 - Transparent
Left	3480
Top	4440
Font	Arial, Negrita, Tamaño 12.
ForeColor	 &H00C0FFC0& (Verde lumínico)



- Haga doble clic sobre la primera opción y seleccione el evento **MouseMove** y, a continuación, escriba el siguiente bloque de código:

'Verifica si la Etiqueta NO tiene el color de fondo activado

```
If Label3.BackStyle = 0 Then
Label3.BackStyle = 1           ' Permite que se establezca el color de fondo
Label3.BackColor = &H8000&    ' Pone el color verde
```

'Desactiva el color de fondo de las demás opciones

```
Label4.BackStyle = 0
Label5.BackStyle = 0
Label6.BackStyle = 0
Label7.BackStyle = 0
```

End If

- Haga doble clic sobre la segunda opción y seleccione el evento **MouseMove** y, a continuación, escriba:

'Verifica si la Etiqueta NO tiene el color de fondo activado

```
If Label4.BackStyle = 0 Then
Label4.BackStyle = 1           ' Permite que se establezca el color de fondo
Label4.BackColor = &H8000&    ' Pone el color verde
```

'Desactiva el color de fondo de las demás opciones

```
Label3.BackStyle = 0
Label5.BackStyle = 0
Label6.BackStyle = 0
Label7.BackStyle = 0
```

End If

- Haga doble clic sobre la tercera opción y seleccione el evento **MouseMove** y, a continuación, escriba:

'Verifica si la Etiqueta NO tiene el color de fondo activado

```
If Label5.BackStyle = 0 Then
Label5.BackStyle = 1           ' Permite que se establezca el color de fondo
Label5.BackColor = &H8000&    ' Pone el color verde
```

'Desactiva el color de fondo de las demás opciones

```
Label3.BackStyle = 0
Label4.BackStyle = 0
Label6.BackStyle = 0
Label7.BackStyle = 0
```

End If

- Haga doble clic sobre la cuarta opción y seleccione el evento **MouseMove** y, a continuación, escriba:



'Verifica si la Etiqueta NO tiene el color de fondo activado

If Label6.BackStyle = 0 Then

Label6.BackStyle = 1

' Permite que se establezca el color de fondo

Label6.BackColor = &H8000&

' Pone el color verde

'Desactiva el color de fondo de las demás opciones

Label3.BackStyle = 0

Label4.BackStyle = 0

Label5.BackStyle = 0

Label7.BackStyle = 0

End If

- Haga doble clic sobre la cuarta opción y seleccione el evento **MouseMove** y, a continuación, escriba:

'Verifica si la Etiqueta NO tiene el color de fondo activado

If Label7.BackStyle = 0 Then

Label7.BackStyle = 1

' Permite que se establezca el color de fondo

Label7.BackColor = &H8000&

' Pone el color verde

'Desactiva el color de fondo de las demás opciones

Label3.BackStyle = 0

Label4.BackStyle = 0

Label5.BackStyle = 0

Label6.BackStyle = 0

End If

- Haga doble clic en cualquier parte del formulario. Seleccione el evento **MouseMove** del formulario y escriba:

' Desactiva el color de fondo de las opciones

If Label3.BackStyle = 1 Then Label3.BackStyle = 0

If Label4.BackStyle = 1 Then Label4.BackStyle = 0

If Label5.BackStyle = 1 Then Label5.BackStyle = 0

If Label6.BackStyle = 1 Then Label6.BackStyle = 0

If Label7.BackStyle = 1 Then Label7.BackStyle = 0

- Pulse la tecla **[F5]** para correr la aplicación.
- Mueva el puntero del Mouse sobre cada una de las opciones y verá como estas se tornan en verde.

Explicación:

Como pudimos notar en la aplicación anterior hemos codificado el evento **MouseMove** de cada opción del menú principal para que este cambie de apariencia cada vez que el usuario coloque el puntero del Mouse sobre cada una de ellas. Empezaremos explicando el código escrito en el evento **MouseMove** de la primera opción del menú principal.

La primera línea de código: `If Label3.BackStyle = 0 Then`. En esta línea verificamos si la Etiqueta puede tomar color de fondo. Cuando la propiedad **BackStyle** esta establecida a **0** **cero** indica que la Etiqueta no puede tomar el color establecido en la propiedad **BackColor** y en consecuencia este tendrá un aspecto transparente.

Si traducimos lógicamente esta línea de código quedaría expresada de la siguiente manera:



`If Label3.BackStyle = 0` `Then`
 Si Etiqueta 3 es transparente Entonces

Si se comprueba que la Etiqueta 3, es decir, la primera opción del menú principal esta transparente o no tiene color de fondo, entonces, se ejecutan las próximas líneas de código:

Segunda línea de código: `Label3.BackStyle = 1`. Esta línea le quita la transparencia a la Etiqueta 3 permitiéndole tener color de fondo, es decir, el color que se asigna en la propiedad **BackColor**. Cuando la propiedad **BackStyle** se establece a uno, entonces, la Etiqueta puede tener color de fondo.

Tercera línea de código: `Label3.BackColor = &H8000&`. Esta línea pone el color de fondo en la Etiqueta. Recuerde que la propiedad **BackColor** (Color de fondo) permite asignar los colores mediante constantes y valores hexadecimales. En este caso hemos establecido el valor hexadecimal **&H8000&** que es el color verde. Esta línea de código es la que hace que cuando el usuario coloca el puntero del Mouse sobre la opción esta se ponga en verde.

Las últimas cuatro líneas de código:

```
Label4.BackStyle = 0
Label5.BackStyle = 0
Label6.BackStyle = 0
Label7.BackStyle = 0
```

Estas líneas desactivan el color de fondo de las demás opciones, ya que la única que lo debe de tener activado es la primera opción del menú. Es necesario deshabilitar el color de fondo de las demás opciones del menú para que cuando el usuario coloque el puntero del Mouse sobre cada una de ellas se pueda comprobar que estas están transparentes.

Las demás opciones están compuestas prácticamente con un bloque de código similar, con la única diferencia de la comparación en la sentencia `If` y las últimas cuartas líneas de código. En estas se excluye la Etiqueta que se compara con la sentencia `If`. Analizaremos esto explicando el código escrito en la segunda opción del menú principal.

Primera línea de código: `If Label4.BackStyle = 0` `Then`. Al igual que con la primera opción del menú, verificamos si la segunda opción esta transparente, o bien, no puede tomar color de fondo.

Segunda línea de código: `Label4.BackStyle = 1`. Hace que la segunda opción que es la Etiqueta 4 pueda tomar el color de fondo.

Tercera línea de código: `Label4.BackColor = &H8000&`. Esta línea le asigna el color de fondo verde a la Etiqueta 4.

Las últimas cuatro líneas de código:

```
Label3.BackStyle = 0
Label5.BackStyle = 0
Label6.BackStyle = 0
Label7.BackStyle = 0
```

Este bloque de código es similar a la de la primera opción del menú con la única diferencia de que no asigna transparencia a la Etiqueta que se compara en la sentencia `If`. En este caso no incluimos la Etiqueta 4 debido a que el objetivo es quitarle la transparencia cuando el usuario coloque el puntero del Mouse sobre esta. Tenga en cuenta que no puede poner transparente la Etiqueta que se esta codificando.



Aunque no hemos contado la línea de código **End If** en las explicaciones anteriores por no considerarlo necesario, tenga en cuenta que cuando utiliza la sentencia **If** acompañado con más de una línea de código, entonces, debe de finalizar con un **End If**.

Las demás opciones están codificadas de la misma manera que las dos primeras opciones teniendo en cuenta lo siguiente: 1) La Etiqueta comparada en la sentencia **If** es la misma que se esta codificando, 2) Se debe quitar la transparencia a la Etiqueta que se compara con la sentencia **If**, 3) Asignar el color de fondo con la propiedad **BackColor** a la Etiqueta que se esta comparando con la sentencia **If**, y 4) No debe de incluir la Etiqueta que se compara con la sentencia **If** en las últimas cuatro línea de código.

En la aplicación anterior también hemos codificado el evento **MouseMove** del formulario para desactivar o poner transparente cualquiera de las opciones del menú principal cada vez que el usuario mueva el puntero del Mouse sobre el formulario y no sobre las opciones. Si el usuario ha colocado el puntero del Mouse sobre una de las opciones del menú esta se activa, pero es necesario desactivarla cuando el usuario desplaza el puntero del Mouse sobre otra opción. Note que el puntero del Mouse debe primero de pasar sobre el formulario antes de colocarse sobre otra opción, aprovechando esto, hemos codificado el evento **MouseMove** del formulario para poner transparente todas las opciones del menú.

Primera línea de código:

If Label3.BackStyle = 1 **Then** Label3.BackStyle = 0. En esta línea se verifica si la primera opción no esta transparente, es decir, si esta activada. En caso de estar activada se desactiva.

Segunda línea de código:

If Label4.BackStyle = 1 **Then** Label4.BackStyle = 0. En esta línea se verifica si la segunda opción no esta transparente, es decir, si esta activada. En caso de estar activada se desactiva.

Tercera línea de código:

If Label5.BackStyle = 1 **Then** Label5.BackStyle = 0. En esta línea se verifica si la tercera opción no esta transparente, es decir, si esta activada. En caso de estar activada se desactiva.

Cuarta línea de código:

If Label6.BackStyle = 1 **Then** Label6.BackStyle = 0. En esta línea se verifica si la cuarta opción no esta transparente, es decir, si esta activada. En caso de estar activada se desactiva.

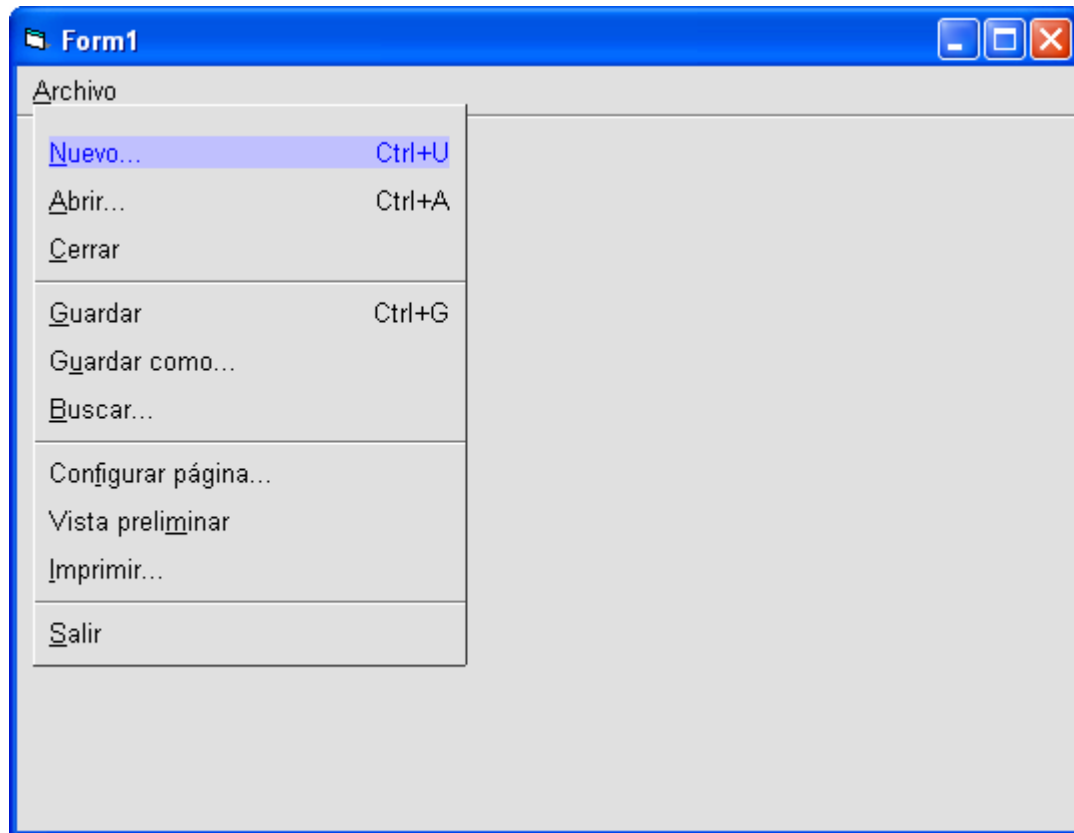
Quinta línea de código:

If Label7.BackStyle = 1 **Then** Label7.BackStyle = 0. En esta línea se verifica si la quinta opción no es transparente, es decir, si esta activada. En caso de estar activada se desactiva.

▪ Detenga la aplicación y guárdela con los nombres **FormEjercicio20-3** para el *formulario* y, **Ejercicio20-3** para el proyecto.

2.- En este segundo ejercicio crearemos un menú típico que vemos en la mayoría de las aplicaciones de Windows. Aunque **Visual Basic** proporciona una herramienta para crear menús de forma sencilla, crear menús personalizados crea efectos visuales más atractivos a la vista del usuario. Para este ejercicio hemos tomado como ejemplo el menú *Archivo* de Microsoft Word. Crearemos este menú con todas las opciones que lo componen.

La aplicación tendrá un aspecto gráfico como el que se muestra en la figura de la siguiente página:



- Inicie un nuevo proyecto.
- Establezca los siguientes valores en cada una de las propiedades indicada del formulario:

Propiedad	Valor
BackColor	☐ &H00E0E0E0& (Gris claro)
Height	8505
StartPosition	2 – CenterScreen
Width	10410

- Inserte una Etiqueta en cualquier parte del formulario y establezca los siguientes valores:

Propiedad	Valor
Name	MenuArchivo
AutoSize	True
BackStyle	0 – Transparent
Caption	&Archivo
Font	Arial, Normal, 10.
Left	120
Top	60

- Busque en la *caja de herramientas* el control **Line** , y dibuje una línea desde una esquina del formulario hasta la otra esquina, o bien, establezca los siguientes valores en las propiedades del objeto **Line** para tener mejores resultados:

Propiedad	Valor
BorderColor	{ EMBED PBrush } &H00808080& (Gris oscuro)
X1	0
X2	10320



Capítulo 3

- Inserte otro objeto **Line** y dibújelo debajo del primero desde un extremo del formulario hasta el otro extremo. Para mejores resultados establezca los siguientes valores:

Propiedad	Valor
BorderColor	☐ &H00FFFFFF& (Blanco)
X1	0
X2	10320
Y1, Y2	370

Podrá notar que al unir estas dos líneas una de color gris y otra de color blanco crea un efecto visual de profundidad. Estos efectos son utilizados muy a menudo por la mayoría de los programadores para separar opciones de menús y barras de herramientas en las aplicaciones.

- Inserte un control **PictureBox**  en cualquier parte del formulario y establezca los siguientes valores en las propiedades indicadas:

Propiedad	Valor
Appearance	0 - Flat
BackColor	☐ &H00E0E0E0& (Gris claro)
Height	4215
Width	3255
Left	120
Top	280

- Dibujemos la primera opción del menú Archivo. Para esto inserte una Etiqueta dentro del **PictureBox**. Establezca los siguientes valores en las propiedades indicadas:

Propiedad	Valor	Propiedad	Valor
Name	OpcionNuevo	Font	Arial, Normal, 10.
BackStyle	0 - Transparent		
Height	240		
Width	3000		
Left	120		
Top	240		
Caption	&Nuevo... (29 espacios) Ctrl+U		

NOTA: Después de los tres puntos suspensivos presione 29 veces la barra espaciadora en la propiedad **Caption** para separar el nombre de la opción del acceso directo y, a continuación, escriba el texto **Ctrl+U**. Estos espacios no son reglamentos de **Visual Basic**, sino, que lo he utilizado para ubicar al final de la opción el acceso directo. Usted puede optar por dar los espacios que usted considere a la hora de crear menús personalizados.

- Dibujemos las demás opciones del menú principal haciendo copia de la primera. De esta manera ahorraremos tiempo evitando tener que modificar tantas propiedades. Los valores para la demás opciones son las mismas menos los valores de la propiedad **Name**, **Caption**, **Left** y **Top**. Para hacer copias de la primera opción del menú haga clic derecho sobre la opción **Nuevo...** y seleccione la opción **Copy (Copiar)** y, a continuación, péguela dentro del **PictureBox**. Modifique ahora las siguientes propiedades:

Propiedad	Valor
Name	OpcionAbrir
Caption	&Abrir... (31 espacios) Ctrl+A
Left	120
Top	600

Ing. Carlos Manuel Rodríguez Bucarely



Capítulo 3

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la tercera opción. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionCerrar
Caption	&Cerrar
Left	120
Top	960

Separemos las tres primeras opciones con objetos **Line**, tal y como lo vemos en el menú Archivo de Microsoft Word.

- Haga clic en el objeto **Line** de la caja de herramientas y dibujela debajo de la opción **Cerrar** de un extremo del **PictureBox** hacia el otro. Puede optar por asignar los siguientes valores al objeto **Line** para que el resultado sea similar al de este ejercicio:

Propiedad	Valor
BorderColor	☐ &H00E0E0E0& (Gris claro)
X1	20
X2	3240
Y1, Y2	1320

- Inserte otro objeto **Line** justamente debajo de este para crear un efecto visual de profundidad. Modifique las propiedades de este objeto **Line** con los siguientes valores:

Propiedad	Valor
BorderColor	☐ &H00FFFFFF& (Blanco)
X1	20
X2	3240
Y1, Y2	1330

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la cuarta opción. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionGuardar
Caption	&Guardar (26 espacios) Ctrl+G
Left	120
Top	1440

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la quinta opción. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionGuardarComo
Caption	G&uardar como...
Left	120
Top	1800

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la sexta opción. Modifique las siguientes propiedades de la Etiqueta pegada:



Propiedad	Valor
Name	OpcionBuscar
Caption	&Buscar...
Left	120
Top	2160

Separemos estas opciones de las siguientes con objetos **Line** como lo hicimos anteriormente.

- Inserte un objeto **Line** debajo de la opción **Buscar...** y dibujela de un extremo al otro del **PictureBox**. Modifique las propiedades de este objeto con los siguientes valores:

Propiedad	Valor
BorderColor	☐ &H00E0E0E0& (Gris claro)
X1	20
X2	3240
Y1, Y2	2520

- Inserte otro objeto **Line** justamente debajo de este para crear un efecto visual de profundidad. Modifique las propiedades de este objeto **Line** con los siguientes valores:

Propiedad	Valor
BorderColor	☐ &H00FFFFFF& (Blanco)
X1	20
X2	3240
Y1, Y2	2530

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la séptima opción. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionConfigurar
Caption	Con&figurar página...
Left	120
Top	2640

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la octava opción. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionVista
Caption	Vista preli&minar
Left	120
Top	3000

- Haga clic derecho sobre el **PictureBox** y seleccione la opción Paste (Pegar) para crear la novena opción. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionImprimir
Caption	&Imprimir...
Left	120
Top	3360



Separaremos estas opciones de las siguientes con objetos **Line** como lo hicimos anteriormente.

- Inserte un objeto **Line** debajo de la opción **Imprimir...** y dibujela de un extremo al otro del **PictureBox**. Modifique las propiedades de este objeto con los siguientes valores:

Propiedad	Valor
BorderColor	☐ &H00E0E0E0& (Gris claro)
X1	20
X2	3240
Y1, Y2	3720

- Inserte otro objeto **Line** justamente debajo de este para crear un efecto visual de profundidad. Modifique las propiedades de este objeto **Line** con los siguientes valores:

Propiedad	Valor
BorderColor	☐ &H00FFFFFF& (Blanco)
X1	20
X2	3240
Y1, Y2	3730

- Haga clic derecho sobre el **PictureBox** y seleccione la opción **Paste** (Pegar) para crear la décima y última opción del menú principal. Modifique las siguientes propiedades de la Etiqueta pegada:

Propiedad	Valor
Name	OpcionSalir
Caption	&Salir
Left	120
Top	3840

Hemos indicado minuciosamente los valores para cada propiedad de los controles colocados en los ejercicios de este capítulo, con el objetivo de que usted se familiarice con las propiedades y para obtener mejores resultados en los ejercicios. Pero todo esto es un principio, a medida que avance tendrá que dibujar los objetos por su propia cuenta y observar como hemos colocado los objetos sobre los ejercicios para que ahorre tiempo y no tenga que modificar tantas propiedades.

Después de haber diseñado el menú **Archivo** con todas sus opciones, es ahora el momento de programar las opciones para crear el efecto visual que vemos en los menús típicos de Windows cuando desplazamos el puntero del Mouse sobre cada una de las opciones de un menú.

- Haga doble clic en cualquier parte del formulario.
- Seleccione el evento **Load** del formulario y escriba la siguiente línea de código.

```
Picture1.Visible = False ' Oculta las opciones del menú Archivo
```

Esta línea hace que al iniciar la aplicación el menú **Archivo** y sus opciones no aparezcan hasta que se haga clic sobre el.

- Haga doble clic sobre la etiqueta del menú **Archivo** y dentro del evento **Click** escriba el bloque de código que se muestra en la siguiente página:



```
If Picture1.Visible = False Then ' Si las opciones no se ven Entonces
Picture1.Visible = True         ' Visualiza las opciones
Else                             ' Sino (si las opciones se ven)
Picture1.Visible = False       ' Oculta las opciones
End If
```

Este bloque de código hace que cuando se haga clic sobre el menú Archivo las opciones del menú aparezcan o se desaparezcan. Si las opciones se ven cuando usted hace clic sobre el menú Archivo, entonces, se ocultarán. Pero si las opciones no se ven cuando usted hace clic sobre el menú Archivo, entonces, aparecerán. Por eso hemos utilizado la sentencia **If** para verificar el estado de la propiedad **Visible** del **PictureBox** cuando se hace clic sobre el menú Archivo.

▪ Haga doble clic sobre el formulario y seleccione el evento **Click**. Dentro del procedimiento de este evento escriba:

```
If Picture1.Visible = True Then ' Si las opciones se ven Entonces
Picture1.Visible = False       ' las oculta
End If
```

Este bloque de código hace que cuando se haga clic en cualquier parte del formulario se desaparezcan las opciones del menú, es decir, hace que se oculte el menú Archivo. Este bloque de código es muy importante, debido a que es necesario ocultar los menús cuando se hace clic sobre otro objeto dentro de una misma aplicación. Si existieran otros objetos dentro de esta aplicación como una caja de texto, una imagen, un botón de comando, etc., entonces, habría que escribir el código anterior en el evento **Click** de estos otros objetos para ocultar el menú cuando se haga clic sobre estos. Si usted está visualizando este documento desde Word pruebe abriendo el menú Archivo de Word y verá que cuando hace clic en cualquier parte fuera del menú este automáticamente se oculta.

Lo último para terminar nuestro menú es hacer que las opciones cambien de color cada vez que el usuario coloque el puntero del Mouse sobre una de ellas. Para esto tendremos que codificar el evento **MouseMove** de cada una de las opciones del menú, haciendo que cuando el puntero del Mouse pase sobre una de ellas, entonces, estas cambien automáticamente el color de fondo. También es necesario codificar el evento **MouseMove** de **PictureBox** que contiene todas las opciones, para que cuando el usuario retire el puntero del Mouse de cualquiera de ellas, entonces, estas vuelvan a tener su color inicial.

▪ Haga doble clic sobre la primera opción del menú y, a continuación, seleccione el evento **MouseMove**. Dentro de este evento escriba el siguiente bloque de código:

```
If OpcionNuevo.BackStyle = 0 Then
OpcionNuevo.BackStyle = 1 ' Permite que se le ponga color de fondo
OpcionNuevo.BackColor = &HDE552E ' Pone el color azul celeste al fondo de la opción
OpcionNuevo.ForeColor = vbWhite ' Pone el color blanco al texto de la opción
End If
```

▪ Corra la aplicación y haga clic sobre el menú Archivo. Ahora mueva el puntero del Mouse sobre la opción Nuevo. Podrá observar que la opción automáticamente cambia su color inicial al color Azul, esto se hizo posible con la línea: `OpcionNuevo.BackColor = &HDE552E`. También puede notar que cuando se cambia el color del fondo, también se cambia el color del texto a blanco con el código: `OpcionNuevo.ForeColor = vbWhite`. Puede establecer colores distintos cambiando los valores especificados en este ejercicio.



Capítulo 3

Podrá notar que al mover el puntero del Mouse sobre otra opción, esta se mantiene con su color inicial. Esto es, porque no hemos codificado el evento **MouseMove** de dicha opción. Para hacer que las demás opciones se comporten como la primera debemos de codificar una por una cada opción del menú, utilizando el mismo código de la primera y cambiando el nombre de la opción.

- Detenga la aplicación y haga doble clic sobre la segunda opción. Seleccione el evento **MouseMove** y escriba:

```
If OpcionAbrir.BackStyle = 0 Then
OpcionAbrir.BackStyle = 1           ' Permite que se le ponga color de fondo
OpcionAbrir.BackColor = &HDE552E   ' Pone el color azul celeste al fondo de la opción
OpcionAbrir.ForeColor = vbWhite     ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la tercera opción:

```
If OpcionCerrar.BackStyle = 0 Then
OpcionCerrar.BackStyle = 1           ' Permite que se le ponga color de fondo
OpcionCerrar.BackColor = &HDE552E   ' Pone el color azul celeste al fondo de la opción
OpcionCerrar.ForeColor = vbWhite     ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la cuarta opción:

```
If OpcionGuardar.BackStyle = 0 Then
OpcionGuardar.BackStyle = 1           ' Permite que se le ponga color de fondo
OpcionGuardar.BackColor = &HDE552E   ' Pone el color azul celeste al fondo de la opción
OpcionGuardar.ForeColor = vbWhite     ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la quinta opción:

```
If OpcionGuardarComo.BackStyle = 0 Then
OpcionGuardarComo.BackStyle = 1       ' Permite que se le ponga color de fondo
OpcionGuardarComo.BackColor = &HDE552E ' Pone el color azul celeste al fondo de la opción
OpcionGuardarComo.ForeColor = vbWhite  ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la sexta opción:

```
If OpcionBuscar.BackStyle = 0 Then
OpcionBuscar.BackStyle = 1           ' Permite que se le ponga color de fondo
OpcionBuscar.BackColor = &HDE552E   ' Pone el color azul celeste al fondo de la opción
OpcionBuscar.ForeColor = vbWhite     ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la séptima opción:

```
If OpcionConfigurar.BackStyle = 0 Then
OpcionConfigurar.BackStyle = 1       ' Permite que se le ponga color de fondo
OpcionConfigurar.BackColor = &HDE552E ' Pone el color azul celeste al fondo de la opción
OpcionConfigurar.ForeColor = vbWhite  ' Pone el color blanco al texto de la opción
End If
```



- Dentro del evento **MouseMove** de la octava opción:

```
If OpcionVista.BackStyle = 0 Then
OpcionVista.BackStyle = 1      ' Permite que se le ponga color de fondo
OpcionVista.BackColor = &HDE552E ' Pone el color azul celeste al fondo de la opción
OpcionVista.ForeColor = vbWhite  ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la novena opción:

```
If OpcionImprimir.BackStyle = 0 Then
OpcionImprimir.BackStyle = 1      ' Permite que se le ponga color de fondo
OpcionImprimir.BackColor = &HDE552E ' Pone el color azul celeste al fondo de la opción
OpcionImprimir.ForeColor = vbWhite  ' Pone el color blanco al texto de la opción
End If
```

- Dentro del evento **MouseMove** de la décima opción:

```
If OpcionSalir.BackStyle = 0 Then
OpcionSalir.BackStyle = 1      ' Permite que se le ponga color de fondo
OpcionSalir.BackColor = &HDE552E ' Pone el color azul celeste al fondo de la opción
OpcionSalir.ForeColor = vbWhite  ' Pone el color blanco al texto de la opción
End If
```

Ahora que hemos codificado cada uno de los eventos de las opciones del menú, vamos a probar el código que escribimos corriendo la aplicación. Corra la aplicación y desplace el puntero del Mouse sobre cada una de las opciones. Puede notar que las opciones se ponen de color azul y el texto de cada opción de color blanco, pero es necesario hacer que cada opción vuelva a su color inicial cuando se mueva el puntero del Mouse sobre otra opción, o cuando el puntero del Mouse se desplace fuera de la opción. Para hacer lo dicho anteriormente, debemos codificar el evento **MouseMove** del **PictureBox** que contiene las opciones.

- Detenga la aplicación.
- Haga doble clic en el **PictureBox** que contiene las opciones del menú. Tenga cuidado de no hacer doble clic sobre una de las opciones.
- Seleccione el evento **MouseMove** del control **PictureBox** y dentro de este escriba:

```
' Para la primera opción
If OpcionNuevo.BackStyle = 1 Then
OpcionNuevo.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionNuevo.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la segunda opción
If OpcionAbrir.BackStyle = 1 Then
OpcionAbrir.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionAbrir.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la tercera opción
If OpcionCerrar.BackStyle = 1 Then
OpcionCerrar.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionCerrar.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If
```



```
' Para la cuarta opción
If OpcionGuardar.BackStyle = 1 Then
OpcionGuardar.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionGuardar.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la quinta opción
If OpcionGuardarComo.BackStyle = 1 Then
OpcionGuardarComo.BackStyle = 0  ' Hace que se le quite el color de fondo.
OpcionGuardarComo.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la sexta opción
If OpcionBuscar.BackStyle = 1 Then
OpcionBuscar.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionBuscar.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la séptima opción
If OpcionConfigurar.BackStyle = 1 Then
OpcionConfigurar.BackStyle = 0  ' Hace que se le quite el color de fondo.
OpcionConfigurar.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la octava opción
If OpcionVista.BackStyle = 1 Then
OpcionVista.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionVista.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la novena opción
If OpcionImprimir.BackStyle = 1 Then
OpcionImprimir.BackStyle = 0   ' Hace que se le quite el color de fondo.
OpcionImprimir.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If

' Para la décima opción
If OpcionSalir.BackStyle = 1 Then
OpcionSalir.BackStyle = 0      ' Hace que se le quite el color de fondo.
OpcionSalir.ForeColor = vbBlack 'Hace que el texto de la opción se ponga de color negro.
End If
```

- Corra la aplicación.
- Desplace el puntero del Mouse sobre cada una de las opciones del menú y podrá ver que estas vuelven a su color original cuando se pasa de una opción a otra.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio21-3** para el *formulario* y, **Ejercicio21-3** para el proyecto.

Pudo notar, que crear una aplicación con menús personalizados requiere de mucho código por parte del programador, pero esto tiene una ventaja, y es que permite crear menús a nuestro estilo y con los efectos gráficos que se nos ocurran que vayan acorde al estilo de nuestra aplicación.



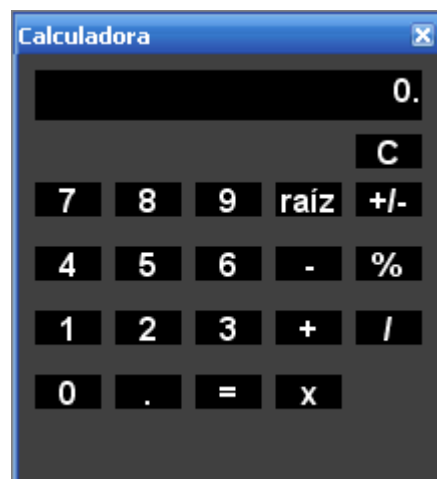
Capítulo 3

MouseDown y MouseUp	Se ha dicho anteriormente que los eventos MouseDown y MouseUp ocurren respectivamente cuando el usuario pulsa y suelta un botón del Mouse, ya sea el izquierdo, el derecho o el del centro. Usted puede programar ambos eventos para realizar operaciones distintas en cada uno de ellos en una misma acción llamada Click. Por ejemplo, si queremos que una Etiqueta cambie de color cuando el usuario pulse el botón izquierdo del Mouse y vuelva a su color original cuando el usuario suelte el botón, entonces, debemos programar el evento MouseDown para asignar el color de la pulsación, y luego programar el evento MouseUp para volver a poner el color original o color inicial de la Etiqueta. NOTA: Utilice el argumento Button (Botón) para identificar cual es el botón del Mouse presionado por el usuario. Este argumento devuelve un campo de bits que puede tomar los valores 1, 2 y 4. Donde 1 representa el botón izquierdo del Mouse, 2 el botón derecho y 4 el botón central.
----------------------------	---

Ejercicio:

La siguiente aplicación muestra el entorno de una calculadora típica donde cada botón de la calculadora cambia de color cuando el usuario pulsa el botón izquierdo del Mouse sobre el y toma su color inicial cuando lo suelta.

NOTA: La calculadora no realiza las operaciones de cálculo que tiene que realizar, es simplemente el entorno. En los siguientes capítulos veremos como hacer funcional adecuadamente la calculadora.



Para realizar la aplicación anterior y ver el comportamiento de los eventos **MouseDown** y **MouseUp** ejecute los siguientes pasos:

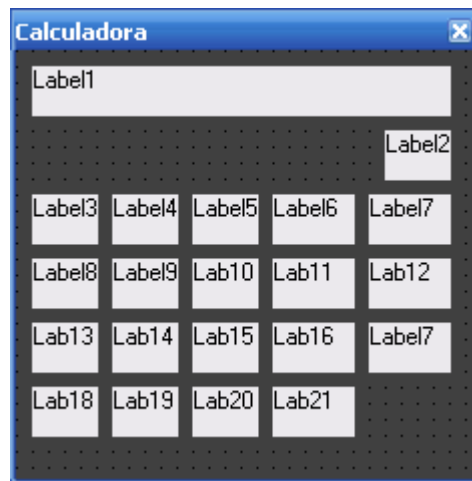
- Abra un nuevo proyecto.
- Establezca los siguientes valores en las propiedades indicadas del formulario:

Control	Propiedad	Valor
Form1	BackColor Caption BorderStyle Height Width StartPosition	{ EMBED PBrush } &H00404040& (Gris oscuro) Calculadora 4 – Fixed ToolWindow 3570 3240 2 - CenterScreen

Ing. Carlos Manuel Rodríguez Bucarely



- Inserte 21 etiquetas sobre el formulario, tal y como se muestra en la siguiente imagen:



- Mantenga la tecla CTRL (control) presionada y mientras esta pulsada haga clic sobre cada una de las Etiquetas del formulario. Cuando todas las Etiquetas estén seleccionadas modificaremos algunos valores en algunas propiedades de las Etiquetas. **NOTA:** Cuando seleccionamos controles del mismo tipo como lo hicimos anteriormente, podremos aplicar los mismos valores en cada una de las propiedades de las Etiquetas seleccionadas. Esto tiene una gran ventaja, y es que, nos ahorra mucho tiempo y evita la redundancia de tener que repetir los mismos procedimientos para todas las Etiquetas.

- Ahora que están todas las Etiquetas seleccionadas vamos a modificar los valores de las siguientes propiedades con el valor indicado en la siguiente tabla:

Propiedad	Valor
BackColor	{ EMBED PBrush } &H00000000& (Negro)
Font	Arial, Negrita, 12.
ForeColor	{ EMBED PBrush } &H00FFFFFF& (Blanco)

- Haga clic en cualquier parte del formulario para deseleccionar las Etiquetas.
- Aplique los siguientes valores en cada una de las propiedades de los controles indicados en la siguiente tabla:

Control	Propiedad	Valor
Label1	Name	lbCajaResultado
	Alignment	1 – Right Justify
	Caption	0.
	Height	375
	Width	2895
Label2	Name	lbBotonC
	Alignment	2 – Center
	Caption	C
	Left	2520
	Top	600
	Height	255
	Width	495



Capítulo 3

Control	Propiedad	Valor	Control	Propiedad	Valor
Label3	Name	lbBoton7	Label10	Name	lbBoton6
	Alignment	2 – Center		Alignment	2 – Center
	Caption	7		Caption	6
	Left	120		Left	1320
	Top	960		Top	960
	Height	255		Height	255
	Width	495		Width	495
Label4	Name	lbBoton8	Label11	Name	lbBotonMenos
	Alignment	2 – Center		Alignment	2 – Center
	Caption	8		Caption	-
	Left	720		Left	1920
	Top	960		Top	960
	Height	255		Height	255
	Width	495		Width	495
Label5	Name	lbBoton9	Label12	Name	lbBotonPorcentaje
	Alignment	2 – Center		Alignment	2 – Center
	Caption	9		Caption	%
	Left	1320		Left	2520
	Top	960		Top	960
	Height	255		Height	255
	Width	495		Width	495
Label6	Name	lbBotonRaiz	Label13	Name	lbBoton1
	Alignment	2 – Center		Alignment	2 – Center
	Caption	raíz		Caption	1
	Left	1920		Left	120
	Top	960		Top	1920
	Height	255		Height	255
	Width	495		Width	495
Label7	Name	lbBotonMM	Label14	Name	lbBoton2
	Alignment	2 – Center		Alignment	2 – Center
	Caption	+/-		Caption	2
	Left	2520		Left	720
	Top	960		Top	1920
	Height	255		Height	255
	Width	495		Width	495
Label8	Name	lbBoton4	Label15	Name	lbBoton3
	Alignment	2 – Center		Alignment	2 – Center
	Caption	4		Caption	3
	Left	120		Left	1320
	Top	1440		Top	1920
	Height	255		Height	255
	Width	495		Width	495
Label9	Name	lbBoton5	Label16	Name	lbBotonSuma
	Alignment	2 – Center		Alignment	2 – Center
	Caption	5		Caption	+
	Left	720		Left	1920
	Top	1440		Top	1920
	Height	255		Height	255
	Width	495		Width	495



Control	Propiedad	Valor
Label17	Name	lbBotonDivision
	Alignment	2 – Center
	Caption	/
	Left	2520
	Top	1920
	Height	255
	Width	495
Label18	Name	lbBoton0
	Alignment	2 – Center
	Caption	0
	Left	120
	Top	2400
	Height	255
	Width	495
Label19	Name	lbBotonPunto
	Alignment	2 – Center
	Caption	.
	Left	720
	Top	2400
	Height	255
	Width	495
Label20	Name	lbBotonIgual
	Alignment	2 – Center
	Caption	=
	Left	1320
	Top	2400
	Height	255
	Width	495
Label21	Name	lbBotonMultiplicacion
	Alignment	2 – Center
	Caption	X
	Left	1920
	Top	2400
	Height	255
	Width	495

▪ El siguiente paso es hacer que las Etiquetas cambien de color cuando el usuario haga clic con el botón derecho del Mouse sobre una de ellas. Para esto vamos a codificar el evento **MouseDown** y **MouseUp** de cada Etiqueta.

▪ Haga doble clic sobre la segunda Etiqueta del formulario, o más bien, sobre el botón C. A continuación, seleccione el evento **MouseDown** y escriba el siguiente bloque de código:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBotonC.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp** de la misma etiqueta escriba:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBotonC.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```



Capítulo 3

- Haga doble clic sobre la tercera Etiqueta del formulario (botón 7). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.  
lbBoton7.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.  
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.  
lbBoton7.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.  
End If
```

- Haga doble clic sobre la cuarta Etiqueta del formulario (botón 8). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.  
lbBoton8.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.  
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.  
lbBoton8.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.  
End If
```

- Haga doble clic sobre la quinta Etiqueta del formulario (botón 9). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.  
lbBoton9.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.  
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.  
lbBoton9.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.  
End If
```

- Haga doble clic sobre la sexta Etiqueta del formulario (botón raíz). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.  
lbBotonRaiz.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.  
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.  
lbBotonRaiz.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.  
End If
```



Capítulo 3

- Haga doble clic sobre la séptima Etiqueta del formulario (botón +/-). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBotonMM.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBotonMM.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la octava Etiqueta del formulario (botón 4). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBoton4.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBoton4.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la novena Etiqueta del formulario (botón 5). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBoton5.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBoton5.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la décima Etiqueta del formulario (botón 6). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBoton6.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBoton6.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la décima primera Etiqueta del formulario (botón -). Seleccione el evento **MouseDown** y escriba:



Capítulo 3

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBotonMenos.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.
lbBotonMenos.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

▪ Haga doble clic sobre la décima segunda Etiqueta del formulario (botón %). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBotonPorciento.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.
lbBotonPorciento.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

▪ Haga doble clic sobre la décima tercera Etiqueta del formulario (botón 1). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBoton1.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.
lbBoton1.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

▪ Haga doble clic sobre la décima cuarta Etiqueta del formulario (botón 2). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBoton2.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.
lbBoton2.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

▪ Haga doble clic sobre la décima quinta Etiqueta del formulario (botón 3). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBoton3.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```



En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBoton3.BackColor = vbBlack  ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la décima sexta Etiqueta del formulario (botón +). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBotonSuma.BackColor = vbBlue  ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBotonSuma.BackColor = vbBlack  ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la décima séptima Etiqueta del formulario (botón /). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBotonDivision.BackColor = vbBlue  ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBotonDivision.BackColor = vbBlack  ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la décima octava Etiqueta del formulario (botón 0). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBoton0.BackColor = vbBlue  ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBoton0.BackColor = vbBlack  ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la décima novena Etiqueta del formulario (botón punto). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then      ' Verifica se pulsa el botón izquierdo.
lbBotonPunto.BackColor = vbBlue  ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then      ' Verifica si se pulso el botón izquierdo.
lbBotonPunto.BackColor = vbBlack  ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```



- Haga doble clic sobre la vigésima Etiqueta del formulario (botón =). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBotonIgual.BackColor = vbBlue ' Pone el color verde cuando se pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.
lbBotonIgual.BackColor = vbBlack ' Pone el color negro cuando se suelta el botón izquierdo.
End If
```

- Haga doble clic sobre la vigésima primera Etiqueta del formulario (botón X). Seleccione el evento **MouseDown** y escriba:

```
If Button = 1 Then          ' Verifica se pulsa el botón izquierdo.
lbBotonMultiplicacion.BackColor = vbBlue ' Pone el color verde si pulsa el botón izquierdo.
End If
```

En el evento **MouseUp**:

```
If Button = 1 Then          ' Verifica si se pulso el botón izquierdo.
lbBotonMultiplicacion.BackColor = vbBlack ' Pone el color negro si suelta el botón izquierdo.
End If
```

- Corra la aplicación.
- Haga clic sobre cada uno de los botones de la calculadora y verá como estos cambian de color y vuelven a su color original.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio22-3** para el *formulario* y, **Ejercicio22-3** para el proyecto.

- 3.2.1.3 Ejercicios propuestos

1. Diseñar una aplicación que contenga un menú principal en el centro del formulario. El menú debe contener siete opciones cualesquiera. Cuando el usuario coloque el puntero del Mouse sobre cada una de las opciones, estas deben cambiar de color, y cuando el usuario retire el puntero del Mouse de las opciones estas deben tonar a su color original. Utilice el evento **MouseMove**.
2. Coloque varias Etiquetas en un formulario y haga que estas cambien de color cuando el usuario haga clic con el botón izquierdo del Mouse sobre una de estas, y que luego tomen su color original cuando el usuario suelte el botón del Mouse. Utilice los eventos **MouseDown** y **MouseUp**.
3. Diseñar utilizando Etiquetas una aplicación que contenga los siguientes menús:

Archivo	
Nuevo	Ctrl+N
Abrir...	Ctrl+A
Guardar	Ctrl+G
Guardar como...	
Configurar página...	
Imprimir...	Ctrl+P
Salir	

Edición	
Deshacer	Ctrl+Z
Cortar	Ctrl+X
Copiar	Ctrl+C
Pegar	Ctrl+V
Eliminar	Supr

Ing. Carlos Manuel Rodríguez Bucarellly



- 3.3.1 Las cajas de texto (TextBox)

CAJA DE TEXTO

Un control **TextBox**, llamado también caja de edición, es aquella que permite al usuario introducir datos en tiempo de ejecución. También son muy utilizadas para mostrar datos contenidas en una base de datos o en archivos de texto. Siempre que necesite solicitar datos del usuario utilice las cajas de texto, por ejemplo, el registro de un empleado, registro de un cliente, registrar un producto en un almacén, ingresos de calificaciones de estudiantes, registro de un formulario, etc.

- 3.3.1.1 Propiedades de las cajas de texto

Las cajas de texto poseen las propiedades comunes (Name, BackColor, Enabled, Font, Height, Width, Left, Top, TabIndex, Visible y ToolTipText) ya vistas anteriormente en el control **CommandButton**. Aparte de estas propiedades las cajas de texto poseen características especiales, es decir, muy propias de ellas. Estas propiedades se detallan a continuación:

Propiedad	Descripción
Alignment	Esta propiedad permite justificar el texto que el usuario ingresara por el teclado o por medio de otra fuente de información. Puede tomar los valores 0 – Left Justify (justificación a la izquierda), 1 – Right Justify (justificación a la derecha) y 2 – Center (justificación en el centro). El valor por defecto es 0 – Left Justify, ya que la caja de texto coloca automáticamente el texto a la izquierda. COMENTARIO: No se modifica muy a menudo esta propiedad, debido a que se tiene por costumbre escribir en una caja de texto a partir de su extremo izquierdo. En algunas ocasiones se justifica a la izquierda o a la derecha como es el caso de una calculadora.
Ejercicio: - Abra un nuevo proyecto. - Inserte tres cajas de texto una debajo de la otra. - Seleccione la primera caja y establezca el valor 0 – Left Justify en la propiedad Alignment. - Seleccione la segunda caja y establezca el valor 1 – Right Justify en la propiedad Alignment. - Seleccione la tercera caja y establezca el valor 2 – Center en la propiedad Alignment. - Seleccione nuevamente la primera caja de texto y borre el valor que contiene la propiedad Text. - Haga lo mismo con las otras dos cajas. - Corra la aplicación y escriba en cada una de las cajas de texto. Podrá entonces ver el comportamiento de cada uno de los valores especificados en la propiedad Alignment. - Detenga la aplicación y guárdela con los nombres FormEjercicio23-3 para el <i>formulario</i> y, Ejercicio23-3 para el proyecto.	
Appearance	Establece si la caja de texto tendrá o no apariencia 3D. Cuando la caja tiene apariencia 3D se nota un efecto de profundidad en el interior de la caja. Esta es la apariencia que distingue una caja de texto de otros controles como, por ejemplo, las Etiquetas. Esta propiedad puede tomar los valores 0 – Flat (sin bordes 3D) y 1 – 3D (con bordes 3D), este último es el valor por defecto. COMENTARIO: En muchas ocasiones usted verá la necesidad de quitar la apariencia 3D de una caja de texto, debido a que es posible que no se adapte a la apariencia gráfica que usted haya seleccionado para su aplicación.

**Ejercicio:**

El siguiente ejercicio muestra dos formularios de ingreso de datos para una agenda telefónica. Ambos formulario contienen los mismos campos de edición pero uno de ellos contiene apariencia 3D en la cajas de texto y el otro de apariencia simple (sin 3D). De esta manera usted podrá notar la diferencia.

- Abra un nuevo proyecto.
- Modifique manualmente el tamaño del formulario para que pueda diseñar los dos formularios de ingreso de datos. Observe detenidamente la imagen y trate de poner el formulario del mismo tamaño.
- En la propiedad **Caption** del formulario escriba Formularios.
- Identifique cuales son los controles que están en la imagen y trate de colocarlos de la misma manera que usted lo puede ver. En la imagen puede notar que hemos modificado la propiedad **Font** de las dos Etiquetas que están como encabezados en cada formulario. También modificamos la propiedad **Font** de las demás Etiquetas para hacer resaltar el texto contenido en ellas.
- Recuerde que debe establecer el valor **0 – Flat** en la propiedad **Appearance** de las cajas de texto del segundo formulario.
- Cuando haya concluido con el ejercicio guárdelo con los nombres **FormEjercicio23-3** para el formulario y, **Ejercicio23-3** para el proyecto.

BorderStyle

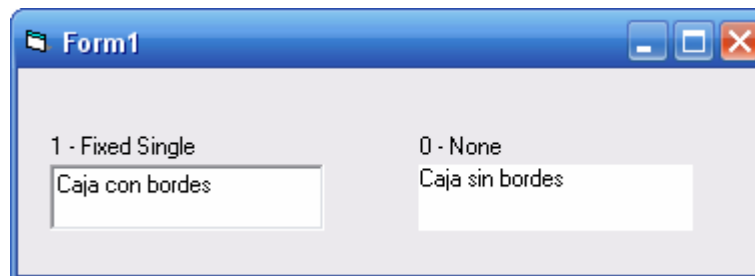
Esta propiedad devuelve o establece el estilo de borde de la caja de texto, es decir, indica si la caja de texto tendrá o no bordes en sus extremos. Puede tomar los valores **0 – None** (sin bordes) y **1 – Fixed Single** (con borde simple fijo).

COMENTARIO 1: Esta propiedad tiene el mismo efecto en algunos controles, como por ejemplo en las Etiquetas, pero en otros controles como es el caso de los Formularios determinan características claves que lo identifican visualmente como una ventana de propósito general o como un cuadro de dialogo.



Capítulo 3

Las siguientes imágenes muestran dos cajas de texto, una con borde simple fijo y otra sin bordes:



COMENTARIO 2: Utilice el valor **0-None** en la propiedad **BorderStyle** de las cajas de texto cuando quiera dar una apariencia más profesional a las cajas. En ocasiones los programadores diseñan imágenes gráficas con apariencia de cajas de texto y con bordes espectaculares, pero son simple imágenes y no se puede escribir sobre estas. El método utilizado para resolver este problema es colocar una caja de texto sin bordes sobre la imagen.

DataField, DataFormat, DataMember y DataSource	Estas propiedades están relacionadas a la manipulación de bases de datos y se tratan en el Capítulo 5 de este libro.
Locked	Devuelve o establece un valor que indica si la caja de texto se puede modificar. Puede tomar los valores True o False . Si es True el usuario no podrá escribir en la caja de texto y si es False (valor por defecto) el usuario podrá escribir en la caja de texto. COMENTARIO: Utilice el valor True en la propiedad Locked si desea mostrar datos que el usuario no podrá modificar en la caja de texto. Por ejemplo, puede utilizar una Caja de texto para mostrar la hora o la fecha del sistema y establecer el valor True en la propiedad Locked de la Caja de texto para que el usuario no la pueda modificar. Otro caso podría ser, que al leer los datos de una base de datos usted solamente desee mostrarlo al usuario y no permitirle modificarlos.
Text	Sin duda alguna esta es la propiedad más importante de las cajas de texto, debido a que es la encargada de manipular el texto que se escribe en la caja de texto. Todo lo que sea escrito en la caja de texto se almacena en la propiedad Text en tiempo de ejecución. También puede escribir en la caja de texto mediante el editor de código como se muestra en el siguiente ejercicio para mostrar la fecha del sistema.

Ejercicio:

- Abra un nuevo proyecto.
- Inserte una caja de texto.
- Seleccione la propiedad **Locked** de la caja de texto y establezca el valor **True**.
- Haga doble clic en cualquier parte del formulario y en el evento **Load** escriba la siguiente línea de código:

```
Text1.Text = Date      ' Muestra la fecha del sistema.
```

- Corra la aplicación.

Se mostrará la fecha del sistema en la caja de texto. Intente modificarla. Podrá observar que la caja de texto no le permite modificar su contenido debido a que la hemos bloqueado (**Locked**).

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio24-3** para el *formulario* y **Ejercicio24-3** para el *proyecto*.

Ing. Carlos Manuel Rodríguez Bucarely

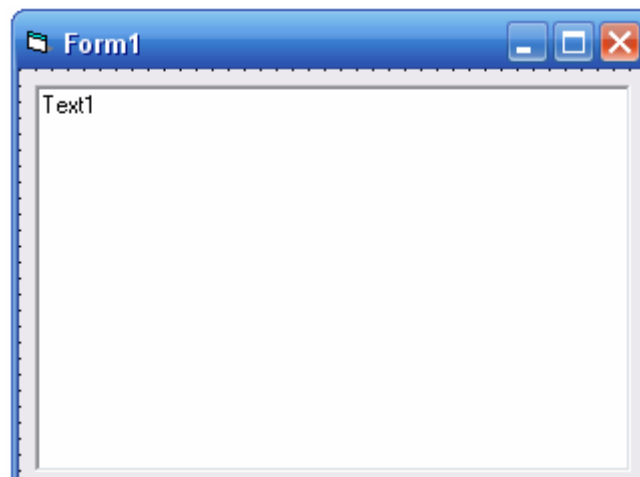


Capítulo 3

MaxLength	Devuelve o establece un valor numérico que indica la cantidad máxima de caracteres que puede aceptar una caja de texto en el área de edición. El valor máximo para una caja de texto es de aproximadamente 65,535 caracteres o 32 KB. El valor por defecto de la propiedad MaxLength es 0, que indica que la caja de texto podrá aceptar el límite máximo de caracteres. Un valor mayor de 0 especificado, indicará la cantidad de caracteres que el usuario podrá escribir en la caja de texto. Por ejemplo, si especifica el valor 15 en la propiedad MaxLength el usuario no podrá escribir más de 15 caracteres en la caja de texto. COMENTARIO: Utilice esta propiedad cuando la aplicación este vinculada o relacionada con archivo de bases de datos para limitar las cajas de texto a los valores máximos admitidos por los campos de la base de datos que están relacionados con las cajas de texto.
MultiLine	Devuelve o establece un valor que indica si la caja de texto admitirá más de una línea de texto en el área de edición. Puede tomar los valores True o False. Si es True la caja de texto permitirá más de una línea de texto en el área de edición, es decir, podrá pasar a la otra línea de texto con el simple hecho de pulsar la tecla ENTER. Si es False solo admitirá una línea de texto y no saltará a la siguiente línea aunque pulse la tecla ENTER. El valor por defecto de esta propiedad es False. COMENTARIO: Utilice múltiples líneas cuando requiera mostrar o solicitar gran cantidad de datos en la caja de texto. Por ejemplo, mostrar el contenido de un archivo o para solicitar gran cantidad de datos del usuario como: comentario, nota, explicación, detalles, etc. Los procesadores de texto (Word, WordPad, NotePad, etc.) son ejemplos de aplicaciones que manejan cajas de texto con múltiples líneas.
ScrollBars	Devuelve o establece un valor que indica si la caja de texto tendrá barra de desplazamiento horizontal o vertical. Puede tomar los valores 0- None (si barra), 1- Horizontal (barra horizontal), 2- Vertical (barra vertical) y 3- Both (ambas barras). COMENTARIO: Utilice esta propiedad si la caja de texto admite más de una línea de texto en el área de edición. Esto permite que el usuario pueda explorar fácilmente el contenido de la caja haciendo uso de las barras de desplazamiento.

Ejercicio:

- Abra un nuevo proyecto. Coloque una caja de texto que cubra casi todo el formulario:



Ing. Carlos Manuel Rodríguez Bucarely



- Seleccione la caja de texto y modifique las siguientes propiedades:

Propiedad	Valor
MultiLine	True
ScrollBars	0 - Transparent
Text	

- Corra la aplicación.

Escriba varias líneas en las cajas de texto hasta que las barras de desplazamiento se activen. Podrá notar que las barras de desplazamiento son únicamente útiles cuando las cajas de texto pueden admitir más de una línea de texto.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio25-3** para el *formulario* y **Ejercicio25-3** para el proyecto.

PasswordChar	Devuelve o establece un valor que se muestra cada vez que se escribe en la caja de texto. Esta propiedad solo admite un carácter. Es utilizada para ocultar los datos que se escriban en la caja de texto. Los caracteres escritos se sustituyen por el carácter especificado en la propiedad PasswordChar . Por ejemplo, si usted tiene designada en su aplicación una caja de texto para escribir una contraseña de acceso, puede escribir un asterisco * en la propiedad PasswordChar para que en vez de mostrarse la contraseña se muestren asteriscos.
	NOTA: Las cajas de textos protegidas mediante contraseñas desactivan las combinaciones de tecla Ctrl+X y Ctrl+C para evitar que se copie su contenido.

Ejercicio:

- Diseñe una pantalla de acceso o de identificación de usuario. Solicite el nombre y la clave o contraseña de acceso mediante dos cajas de texto. Utilice dos etiquetas para especificar los datos requeridos en cada caja de texto. Recuerde proteger los datos que se escriban en la caja de texto correspondiente a la contraseña.

- 3.3.1.2 Eventos sobre las cajas de texto

Las cajas de texto poseen los eventos estándares de los controles (**Click**, **DbClick**, **DragDrop**, **DragOver**, **GotFocus**, **KeyDown**, **KeyUp**, **KeyPress**, **LostFocus**, **MouseDown**, **MouseMove**) ya estudiados anteriormente. Aparte de estos eventos las cajas de texto incluyen un evento muy propio de ellas, este es el evento **Change**, que ocurre cuando se modifica el texto contenido en el área de edición.

Todos los eventos ya mencionados anteriormente y algunas propiedades especiales la estudiaremos en los siguientes tópicos. Los tópicos son una colección de aplicaciones y trucos que comúnmente utilizaremos en las cajas de texto. Muchos de estos tópicos lo utilizarán al momento en que este diseñando una aplicación.

• Cómo leer las teclas pulsadas desde el teclado a través de una caja de texto

Las cajas de texto cuentan con los eventos estándares **KeyPress**, **KeyDown** y **KeyUp**. Cada uno de estos eventos es utilizado para atrapar las actividades del usuario sobre una caja de texto. Se ha dicho anteriormente que el evento **KeyPress** ocurre cuando el usuario presiona y suelta una tecla. Tiene un argumento llamado **KeyAscii** que devuelve el valor **ASCII** correspondiente a la tecla pulsada por el usuario. Este argumento permite realizar una serie de operaciones sobre las cajas de texto. Algunos casos se tratan en la siguiente página.

**- Hacer que una caja de texto solo acepte números:**

Dentro del evento **KeyPress** de la caja de texto escriba el siguiente bloque de código:

```
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then  
    KeyAscii = 0  
End If
```

El bloque de código hace que una caja de texto solo acepte números. Existen otros métodos para hacer esto y lo trataremos más adelante.

Explicación del código anterior:

```
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then
```

En esta línea se verifica si el valor ASCII de la tecla pulsada se encuentran en el intervalo de los valores ASCII correspondiente a los números naturales del 0 al 9. Los números naturales del 0 al 9 se encuentran entre los valores 48 hasta el 57 de la tabla ASCII. El valor ASCII 48 corresponde al número 0, el valor 49 al número 1, el valor 50 al número 2 y así sucesivamente. El valor 8 de la tabla ASCII corresponde a la tecla de borrar (BackSpace).

Si traducimos literalmente el bloque de código anterior obtendríamos más o menos lo siguiente:

Si No (se pulsa una tecla con valor ASCII mayor o igual que 48 **Y** menor o igual que 57 **O** la tecla BackSpace) **Entonces**

Valor ASCII es igual a 0, es decir, asume que no se pulso ninguna tecla.

Fin Si

Cuando igualamos a cero el argumento **KeyAscii** la aplicación asume que no se ha pulsado ninguna tecla y como consecuencia no se muestra la tecla pulsada en la caja de texto.

Otro ejemplo para hacer que una caja de texto admita solamente números:

```
If (KeyAscii < 48) Or (KeyAscii > 57) Then  
    If KeyAscii <> 8 Then  
        KeyAscii = 0  
    End If  
End If
```

Este es otro ejemplo pero mucho más profesional:

```
If InStr("0123456789" & Chr(8) & Chr(13), Chr(KeyAscii)) = 0 Then  
    KeyAscii = 0  
End If
```

Este último ejemplo es mucho más recomendable cuando se va a permitir introducir otros caracteres a parte de los números. Los caracteres que la caja de texto admitirá lo pueden incluir dentro de las comillas donde se encuentran los números. Por ejemplo, si la caja de texto admitirá puntos, entonces, incluya el punto dentro de las comillas: "0123456789.". Más adelante estudiaremos la función **InStr**.

**- Hacer que todo lo que se escriba en una caja de texto se convierta a mayúscula**

En muchas ocasiones necesitamos que todo lo que escriba el usuario en una caja de texto sea convertido a mayúscula. Para esto utilizamos la función **UCase** dentro del evento **KeyPress**. La función **UCase** se encarga de convertir una cadena de caracteres en mayúscula. Utilizaremos también la función **Chr** para convertir el valor **ASCII** de la tecla pulsada en el carácter correspondiente a dicho valor. También utilizaremos la función **Asc** para obtener el valor **ASCII** del carácter convertido a mayúscula. Esta última función es la inversa a la función **Chr**.

Escriba la siguiente línea de código dentro del evento **KeyPress** de la caja de texto:

```
KeyAscii = Asc(UCase(Chr(KeyAscii)))      'Convierte todo a mayúscula.
```

Existen muchos métodos para hacer que todo lo que se escriba en una caja de texto sea convertido a mayúscula, pero este es un código muy profesional que he creado y probablemente no lo encuentre en otros libros o en Internet.

Explicación del código anterior:

Para entender esta línea de código explicaremos por parte cada una de las funciones que se emplean en ella. Como en todos los lenguajes de programación los últimos paréntesis son los primeros que se ejecutan. Para esta línea de código podríamos decir, que las últimas funciones son las primeras que se ejecutan. Por consiguiente, en esta línea de código la primera función que se ejecuta es **Chr**. Lo primero que hacemos es convertir en una letra o carácter el valor que tiene el argumento **KeyAscii** con la función **Chr**. Por ejemplo, si el usuario pulsa la tecla "a" el argumento **KeyAscii** del evento **KeyPress** toma el valor 97, esto es, porque 97 es el valor **ASCII** para la letra "a". Luego que se obtiene el valor 97, la función **Chr** lo convierte en una "a". Después de esto, utilizamos la función **UCase** para convertir esa "a" en mayúscula. Cuando convertimos el carácter en mayúscula, obtenemos el valor **ASCII** de ese carácter convertido a mayúscula, utilizando la función **Asc**. Recuerde que todos los caracteres tienen un valor distinto aunque sea el mismo, por ejemplo, el valor para la "a" en minúscula es 97 y, para la "A" en mayúscula es 65. Luego que obtenemos el valor **ASCII** de la letra convertida a mayúscula, asignamos ese valor al argumento **KeyAscii** para que asuma que se pulso esa letra sobre la caja de texto.

- Hacer que todo lo que se escriba en una caja de texto se convierta a minúscula

Para esto, hacemos lo mismo que explicamos anteriormente pero cambiamos la función **UCase** por **LCase**.

```
KeyAscii = Asc(LCase(Chr(KeyAscii)))      'Convierte todo a minúscula.
```

- Pasar de una caja de texto a otra cuando se pulse la tecla ENTER

Con frecuencia las aplicaciones posicionan el cursor en la siguiente caja de texto cuando el usuario pulsa la tecla ENTER. Para esto verificamos si el usuario pulso la tecla ENTER mediante el evento **KeyPress** de la caja de texto. Cuando el usuario pulsa la tecla ENTER el argumento **KeyAscii** del evento **KeyPress** toma el valor 13 que corresponde a la tecla ENTER en la tabla **ASCII**. Luego enviamos al teclado la pulsación de la tecla TAB o tabulación mediante la sentencia **SendKeys**.

```
If KeyAscii = 13 Then      'Verifica que se pulsa la tecla ENTER.  
    SendKeys "{tab}"      'Se pulsa la tecla TAB.  
End If
```



NOTA: Tiene que tener en cuenta si el orden de tabulación de cada control esta correctamente enumerado. Utilice la propiedad **TabIndex** de cada control para controlar el orden de tabulación.

Los KeyDown y KeyUp en las cajas de texto

Los eventos **KeyDown** y **KeyUp** son utilizando también para capturar las actividades realizadas por el usuario en las cajas de texto. A diferencia del evento **KeyPress**, los eventos **KeyDown** y **KeyUp** pueden capturar teclas especiales que no son posibles capturar por el evento **KeyPress**. Por ejemplo, las teclas ALT, CTRL, SHIFT, tecla de funciones (F1 hasta F12), Delete (Supr), Home (Inicio), PageUp (Re Pág), PageDown (Av Pág), End (Fin), Teclas de desplazamiento (Arriba, Abajo, Izquierda, Derecha), entre otras.

Los eventos **KeyDown** y **KeyUp** son mucho más complejos que el evento **KeyPress**, esto puede notarlo en lista de argumentos de dichos eventos.

• Algunas propiedades de las cajas de texto manejadas en tiempo de ejecución

Además, de la propiedad **Text**, existen tres propiedades que son muy utilizadas en las cajas de texto que son: **SelStart**, **SelLength** y **SelText**.

» La propiedad **SelStart** devuelve o establece un valor numérico de la posición del signo de intercalación parpadeante (punto de inserción que indica donde aparecerá el texto que teclee el usuario). Por ejemplo, si desea posicionar el punto de inserción al principio de la caja de texto, entonces, utilizaríamos un código similar a este: `nombre_caja_texto.SelStart = 0`. Cuando el punto de inserción se encuentra al principio de la caja de texto su valor es 0. Si desea posicionarlo al final de la caja de texto escriba: `nombre_caja_texto.SelStart = Len(nombre_caja_texto.Text)`. La longitud del texto indicará el valor máximo que puede alcanzar el punto de inserción en una caja de texto. Mediante la sentencia **Len** puede obtener la longitud de una cadena de texto, como lo pudimos en el ejemplo anterior.

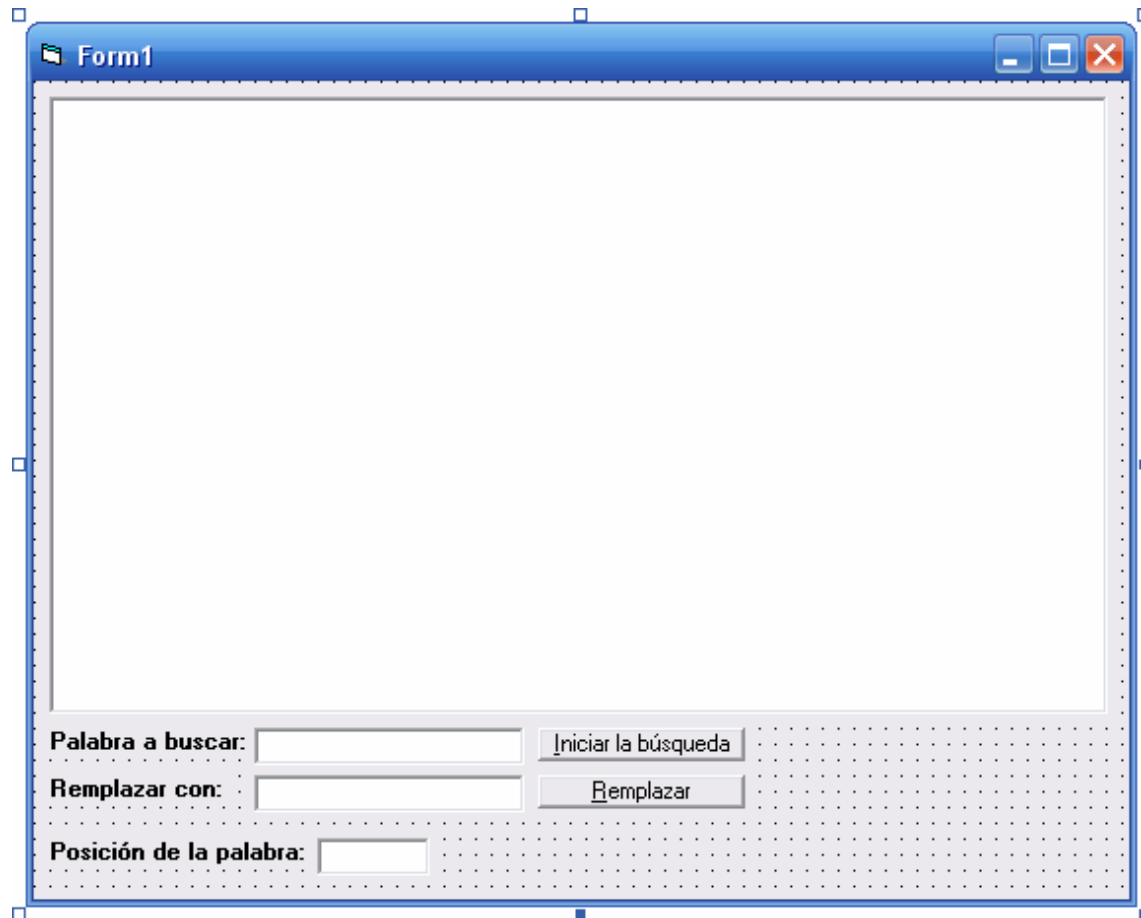
» La propiedad **SelLength** devuelve o establece la cantidad de caracteres seleccionados en la caja de texto. También puede seleccionar una cantidad de caracteres utilizando código de programa. Por ejemplo, si desea seleccionar mediante código todo el contenido de una caja de texto, utilice un código similar a este: `nombre_caja_texto.SelLength = Len(nombre_caja_texto.Text)`.

» La propiedad **SelText** devuelve o establece el texto seleccionado en la caja de texto, ya sea por el usuario o mediante código. Esta propiedad es muy utilizada para realizar búsquedas y reemplazar textos.

A continuación, se muestra una aplicación donde usted podrá notar el comportamiento de cada una de estas tres propiedades. La aplicación consiste en un sistema de búsqueda y reemplazo de palabras. La aplicación tendrá cuatro cajas de texto: la primera para el contenido principal, la segunda para que el usuario especifique la palabra a buscar, la tercera para que el usuario especifique la palabra con la cual desea reemplazar la palabra buscada, y en la cuarta la aplicación mostrará la posición donde se encuentra la palabra buscada.

Para la realización de esta aplicación haga lo siguiente:

- Abra un nuevo proyecto.
- Modifique el tamaño del formulario en ambos extremos para poder colocar adecuadamente los controles. Si desea, puede especificar el valor 6735 en la propiedad **Height** y el valor 8460 en la propiedad **Width**, así tendrá el tamaño exacto con la cual he diseñado la aplicación.
- Inserte una caja de texto que cubra casi todo el formulario, dejando un espacio de tres pulgadas en la parte inferior, tal y como se ve en la figura de la siguiente página.



- Coloque tres etiquetas, tres cajas de texto y dos botones de comando, tal y como se muestra en la imagen anterior. Recuerde modificar la propiedad **Caption** de cada etiqueta y de cada botón de comando. También elimine el contenido de la propiedad **Text** de cada caja de texto.
- Cambie los nombres a las cajas de texto y a los botones de comando para tener mejor control de estos en la aplicación. A la caja principal asigne el nombre: txtContenido, a la caja de búsqueda: txtBuscar, a la caja para reemplazar: txtReemplazar, a la caja posición de la palabra: txtPosicion, al botón para iniciar la búsqueda: cmdBuscar, y al botón de reemplazar: cmdReemplazar.
- Ahora vamos a hacer que la caja de texto principal tenga algo de contenido para poder buscar alguna palabra dentro de ella. Para esto, haga doble clic en cualquier parte del formulario y dentro del evento **Load** del formulario escriba la siguiente línea de código:

```
txtContenido.Text = "La computadora es una máquina electrónica que acepta y procesa datos."
```

Esta línea de código hará que al iniciar la aplicación la caja de texto principal tenga como contenido el texto especificado en la propiedad **Text**. Recuerde que el evento **Load** ocurre cuando la aplicación es totalmente cargada, por tal razón, hemos puesto esta línea de código en dicho evento.

- Seleccione la caja de texto principal (txtContenido) y en la propiedad **HideSelection** especifique el valor **False**. Esto hace que la palabra buscada desde código se vea resaltada o seleccionada.
- Haga doble clic en el botón *Iniciar la búsqueda*. Dentro del evento **Click** escriba el siguiente bloque de código:



```
Dim i As Integer ' Variable que controla el bucle.

'Iniciamos un bucle para buscar por toda la caja.
For i = 0 To Len(txtContenido.Text)

    'Establecemos el comienzo de la búsqueda.
    txtContenido.SelStart = i

    'Especificamos la longitud del texto a buscar.
    txtContenido.SelLength = Len(txtBuscar.Text)

    'Verificamos si el texto seleccionado es igual al especificado en la caja Buscar y paramos la bus.
    If txtContenido.SelText = txtBuscar.Text Then Exit For

Next i
txtPosicion.Text = txtContenido.SelStart 'Pone la posición donde se encontró la palabra.
```

▪ Antes de seguir agregando código corra la aplicación. En la caja de texto de búsqueda escriba una palabra que se encuentre en la caja del contenido, por ejemplo, computadora. Después haga clic en el botón *Iniciar la búsqueda*. Podrá observar como el código anterior busca la palabra y la selecciona.

Explicación del código:

Línea 1: Dim i As Integer

En esta línea hemos declarado una variable de tipo entera llamada i. Esta variable es necesaria para controlar el bucle. Si tiene conocimientos en otros lenguajes de programación sabrá que todo bucle tiene una variable de control de tipo entera.

Línea 2: For i = 0 To Len(txtContenido.Text)

En esta línea hemos iniciado el bucle para que se repita hasta que lea carácter por carácter el contenido de la caja principal. Hemos iniciado el bucle que comience desde el valor 0, para que comience a leer desde el primer carácter de la caja principal. El bucle se repetirá hasta que se lean todos los caracteres de la caja y esto está dado por: **To** Len(txtContenido.Text). Si traducimos literalmente esta porción de código obtendríamos algo como: **Hasta** la Longitud(de la caja contenido).

Línea 3: txtContenido.SelStart = i

Esta línea hace que el punto de inserción se coloque en cada uno de los caracteres de la caja principal. Esto es necesario para comenzar la selección desde un punto diferente, si no se ha encontrado la palabra en una posición anterior.

Línea 4: txtContenido.SelLength = Len(txtBuscar.Text)

Esta línea se encarga de seleccionar los caracteres a la derecha a partir de la posición actual establecida por **SelStart**. El total de caracteres a seleccionar va a depender de la cantidad de caracteres que posee la palabra a buscar.

Línea 5: If txtContenido.SelText = txtBuscar.Text Then Exit For

Esta línea verifica si el texto seleccionado es igual al texto que se está buscando. Si el texto almacenado en la propiedad **SelText** es idéntico al buscado, entonces, se finaliza el bucle. En caso contrario se comienza la búsqueda desde otra posición (**SelStart**).

**Línea 6:** `Next i`

Esta línea es requerida por Visual Basic al final de la utilización de un bucle. Traducida literalmente significa: Próxima repetición o siguiente repetición.

Línea 7: `txtPosicion.Text = txtContenido.SelStart`

Esta última línea se ejecuta después de haberse concluido con la búsqueda. Se encarga de poner en la caja de texto *posición*, la posición donde **SelStart** pudo conseguir la palabra, o más bien, a partir de donde se comenzó a seleccionar la palabra encontrada.

- Detenga la aplicación y codifiquemos el botón de remplazar. Dentro del evento **Click** del botón remplazar escriba:

```
If Len(txtContenido.SelText) > 0 Then ' Verifica que halla texto seleccionado.  
    'Reemplaza el texto seleccionado por el texto escrito en la caja remplazar.  
    txtContenido.SelText = txtRemplazar.Text  
End If
```

- Corra la aplicación.
- Busque nuevamente la palabra computadora. Después que la palabra se encuentre seleccionada, escriba en la caja de texto *remplazar* la palabra “ordenador”, y a continuación, haga clic en el botón remplazar. Podrá notar como es remplazada la palabra.

Explicación del código

Primero verificamos que exista algún texto seleccionado con la línea de código: `If Len(txtContenido.SelText) > 0 Then`. Si no se hace esta evaluación, entonces, aunque no exista selección alguna el programa insertará el texto escrito en la caja remplazar a partir de la posición que se encuentra el punto de inserción.

En la segunda línea reemplazamos el texto el texto seleccionado por el texto escrito en la caja de texto remplazar. Si traducimos literalmente esta línea de código obtendríamos algo como: El texto seleccionado en la caja contenido es igual al texto contenido en la caja remplazar.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio26-3** para el *formulario* y **Ejercicio26-3** para el proyecto.

Realicemos otra aplicación para familiarizarnos más con las propiedades **SelStart**, **SelLength** y **SelText**.

La siguiente aplicación muestra un programa típico de mecanografía. En este no hemos agregado un teclado virtual, solo calculamos los errores y las pulsaciones correctas. La aplicación consta de cinco cajas de texto, la primera para el texto que se tiene que digitar, la segunda para el texto digitado, la tercera para mostrar las pulsaciones erróneas, la cuarta para mostrar las correctas y la quinta para mostrar el total de pulsaciones.

- Abra un nuevo proyecto.
- Modifique el tamaño del formulario para colocar adecuadamente los controles en la aplicación. Puede utilizar los siguientes valores para el formulario: **Height** = 7245 y **Width** = 8115.
- Diseñe la aplicación, tal y como se muestra en la figura de la página siguiente.



- Seleccione la primera caja de texto y modifique las siguientes propiedades:

Propiedad	Valor
Name	txtContenido
MultiLine	True
ScrollBars	2 - Vertical
Font	Arial, 12.
HideSelection	False
Locked	True
Text	

- Seleccione la segunda caja de texto y modifique las siguientes propiedades:

Propiedad	Valor
Name	txtDigitado
MultiLine	True
ScrollBars	2 - Vertical
Font	Arial, 12.
Text	

- Seleccione una por una las últimas tres cajas de texto y borre el contenido de la propiedad **Text**. A la tercera caja asigne el nombre txtErrores, a la segunda txtCorrectas y a la tercera txtTotalPulsaciones. Estos nombres permitirán trabajar mucho mejor en la aplicación.

Ahora, hagamos que la primera caja de texto contenga alguna información para que el usuario la digite. Para esto haga doble clic en cualquier parte del formulario y dentro del evento **Load** escriba:



Capítulo 3

```
txtcontenido.Text = "Visual Basic es un lenguaje de programación orientado a objetos."
```

```
' Selecciona el primera carácter a digitar.
```

```
txtContenido.SelStart = 0
```

```
txtContenido.SelLength = 1
```

```
' Hace que el punto de inserción se coloque en la caja de digitación.
```

```
txtDigitado.TabIndex = 0
```

- Haga doble clic sobre la caja de contenido y dentro del evento **Click** escriba:

```
'Estas dos líneas, impide que el usuario coloque manualmente el punto de inserción
```

```
'en otra posición de la caja de contenido al hacer clic sobre esta.
```

```
txtContenido.SelStart = Len(txtDigitado.Text)
```

```
txtContenido.SelLength = 1
```

- Haga doble clic sobre la caja de digitación y dentro del evento **Click** escriba:

```
'Verifica si el ejercicio ha concluido.
```

```
If Len(txtContenido.Text) = Len(txtDigitado.Text) Then
```

```
    MsgBox ("Usted ha finalizado el ejercicio.") 'Muestra al usuario un mensaje de finalización.
```

```
    KeyAscii = 0 ' Cancela cualquier tecla pulsada si el ejercicio termino.
```

```
'Si el ejercicio no ha concluido entonces:
```

```
Elseif KeyAscii = 8 Then
```

```
    KeyAscii = 0
```

```
' Verifica si se pulsa la tecla BackSpace (tecla de borrar).
```

```
' Si se pulsa la tecla BackSpace la cancela.
```

```
Else
```

```
    'Sino se pulsa la tecla BackSpace.
```

```
    'Verificamos las pulsaciones correctas.
```

```
    If txtContenido.SelText = Chr(KeyAscii) Then
```

```
        'Calculamos el total de pulsaciones correctas.
```

```
        txtCorrectas.Text = Val(txtCorrectas.Text) + 1
```

```
    Else
```

```
        'Calculamos las pulsaciones incorrectas.
```

```
        txtErrores.Text = Val(txtErrores.Text) + 1
```

```
    End If
```

```
'Selecciona la letra que corresponde pulsar.
```

```
txtContenido.SelStart = Len(txtDigitado.Text) + 1
```

```
txtContenido.SelLength = 1
```

```
'Calcula el total de pulsaciones.
```

```
txtTotalPulsaciones.Text = Val(txtCorrectas.Text) + Val(txtErrores.Text)
```

```
End If
```



- En el evento **Click** del botón *Salir* escriba:

```
End      'Finaliza el programa.
```

- Corra la aplicación.

Ahora en la caja de digitación trate de escribir lo que se muestra en la caja de contenido. Observe, que a medida que va escribiendo se van colocando las pulsaciones correctas, las pulsaciones erróneas y el total de pulsaciones. Otra nota muy importante es, que cada vez que usted escribe un carácter, en la caja contenido se selecciona el próximo carácter a pulsar.

Explicación del código:

Empezamos explicando el bloque de código del evento **Load** del formulario. En este evento, hemos codificado lo que la aplicación debe de realizar al momento de ejecutarse. Definiremos línea por línea el bloque de código:

Línea 1: `txtContenido.Text = "Visual Basic es un lenguaje de programación orientado a objetos."`

Esta línea hace que la caja de contenido tenga algo cuando inicia la aplicación. El texto que esta entre las comillas es lo que aparece en el área de edición de la caja de texto.

Línea 2 y 3:

```
txtContenido.SelStart = 0  
txtContenido.SelLength = 1
```

Estas dos líneas trabajan de forma combinada, para seleccionar el primer carácter de la caja contenido al momento de ejecutarse la aplicación. La primera de estas dos líneas hace que el punto de inserción se coloque al inicio de la caja de texto. La segunda hace que el primer carácter aparezca seleccionado.

Línea 4:

```
txtDigitado.TabIndex = 0
```

Esta línea hace que al iniciar la aplicación el punto de inserción se coloque en la caja de digitación. No hemos utilizado el método **SetFocus**, porque no es posible utilizarlo en el momento que se esta cargando la aplicación.

Continuamos nuestra explicación con el bloque de código escrito en el evento **Click** de la caja contenido. Este evento lo hemos codificado con dos líneas de código muy importantes. Estas dos líneas, hacen que al usuario hacer clic sobre la caja contenido no afecte el carácter que está seleccionado para su pulsación, es decir, hace que se mantenga el punto de inserción en el carácter que tiene que ser pulsado por el usuario. Si estas dos líneas no se hubiesen agregado en la aplicación, entonces, cada vez que el usuario haga clic en la caja contenido, la tecla a pulsar se deselecta. En consecuencia, al pulsar la tecla que corresponde digitar se tomaría como un error y no como correcta.

Línea 1: `txtContenido.SelStart = Len(txtDigitado.Text)`

Esta línea hace que el punto de inserción se coloque en el carácter que corresponde digitar. Esto se hace asignando en la propiedad **SelStart** de la caja de contenido la longitud o el número de caracteres de la caja de digitación. Por ejemplo, si usted ha efectuado seis pulsaciones, la longitud de la caja será seis. Asignando esta longitud en la propiedad **SelStart** de la caja contenido, el punto inserción se colocará en el carácter que corresponde pulsar. Este será igual al número de pulsaciones realizadas por el usuario.

Ing. Carlos Manuel Rodríguez Bucarely



Línea 2: txtContenido.SelLength = 1

Selecciona con una sombra el carácter que tiene que pulsarse en la caja de contenido. El número 1 indica que solo se debe seleccionar un carácter.

Ahora explicaremos el código más complejo de esta aplicación. Este es el bloque de código escrito en el evento **KeyPress** de la caja de digitación. En este verificamos la tecla que corresponde pulsar, el número de pulsaciones correctas, el número de pulsaciones incorrectas y el total de pulsaciones realizadas. También hemos codificado para cancelar la pulsación de la tecla BackSpace (tecla de borrar).

Línea 1, Línea 2 y Línea 3:

```
If Len(txtContenido.Text) = Len(txtDigitado.Text) Then
```

```
    MsgBox ("Usted ha finalizado el ejercicio.")
```

```
    KeyAscii = 0
```

Estas tres líneas están codificadas exclusivamente para verificar y notificar cuando el usuario haya finalizado el ejercicio. En la primera línea verificamos si la longitud de la caja contenido es igual a la longitud de la caja de digitación. Si ambas cajas tienen la misma cantidad de caracteres, quiere decir, que el usuario ha finalizado el ejercicio. Luego con la segunda línea notificamos al usuario con un cuadro de mensaje que ha finalizado el ejercicio. La tercera línea hace que se cancele cualquier pulsación que haya realizado el estudiante al finalizar el ejercicio.

Línea 4 y Línea 5:

```
Elseif KeyAscii = 8 Then
```

```
    KeyAscii = 0
```

Con estas dos líneas verificamos si el usuario ha pulsado la tecla BackSpace (tecla de borrar) y la cancelamos. En consecuencia, el usuario no podrá borrar de la caja de digitación los caracteres que ha pulsado. Cuando el usuario pulsa la tecla BackSpace, el argumento **KeyAscii** del evento **KeyPress** toma el valor 8 ocho que representa dicha tecla en la tabla de valores **ASCII**. Al asignar luego el valor 0 cero al argumento **KeyAscii** cancelamos cualquier tecla pulsada por el usuario, en este caso la tecla BackSpace.

Si el ejercicio no ha finalizado y el usuario no ha pulsado la tecla BackSpace, entonces, se ejecuta el bloque de código siguiente:

```
    If txtContenido.SelText = Chr(KeyAscii) Then
```

```
        txtCorrectas.Text = Val(txtCorrectas.Text) + 1
```

```
    Else
```

```
        txtErrores.Text = Val(txtErrores.Text) + 1
```

```
    End If
```

```
    txtContenido.SelStart = Len(txtDigitado.Text) + 1
```

```
    txtContenido.SelLength = 1
```

```
    txtTotalPulsaciones.Text = Val(txtCorrectas.Text) + Val(txtErrores.Text)
```

```
End If
```




Con la primera línea verificamos si la tecla que se tiene que pulsar es igual a la tecla que fue pulsada por el usuario. El texto almacenado en la propiedad **SelText** de la caja contenido corresponde a la tecla que se tiene que pulsar. Luego con la sentencia **Chr** convertimos el valor que tiene el argumento **KeyAscii** de la tecla pulsada por el usuario al carácter correspondiente a dicho valor.

La segunda línea `txtCorrectas.Text = Val(txtCorrectas.Text) + 1`, la utilizamos para almacenar e incrementar las pulsaciones correctas. Si sumamos 1 al valor actual de la caja de texto, entonces, incrementaremos el valor. La sentencia **Val** se utiliza para convertir el valor actual de la caja a un número entero o real para luego poderla incrementar. Si la tecla pulsada por el usuario no coincide con la tecla que se tiene que pulsar, entonces, se calculan los errores: `txtErrores.Text = Val(txtErrores.Text) + 1`.

Las últimas tres líneas que se ejecutan sin importar que la pulsaciones sean correctas e incorrectas, se utilizan para seleccionar el carácter que se tiene que pulsar y para calcular el total de pulsaciones.

```
txtContenido.SelStart = Len(txtDigitado.Text) + 1  
txtContenido.SelLength = 1
```

```
txtTotalPulsaciones.Text = Val(txtCorrectas.Text) + Val(txtErrores.Text)
```

La primera línea coloca el punto de inserción en el carácter que se tiene que pulsar. Esto se hace asignando en la propiedad **SelStart** de la caja contenido, la longitud de la caja de texto incrementada en uno. Con la segunda línea seleccionamos con una sombra el carácter que se tiene que pulsar. Y por último, sumando el total de pulsaciones correctas y el total de pulsaciones incorrectas, obtenemos el total de pulsaciones realizadas por el usuario.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio27-3** para el *formulario* y **Ejercicio27-3** para el proyecto.

• Definición de una máscara de entrada en una caja de texto

En muchas ocasiones el programador verá la necesidad de obligar al usuario a introducir los datos según un formato preestablecido. Esto podría evitar muchos problemas futuros en el almacenamiento de los datos. Además, una máscara de entrada facilita la entrada de los datos al usuario. Por ejemplo, si al diseñar la aplicación usted considera que el usuario debe introducir el número telefónico de un individuo mediante el formato (Código_de_área)-NNN-NNNN, usted puede optar por facilitar esta entrada y obligar a que la entrada sea de esa manera. Por ejemplo: (809)-699-5853, (829)-698-5236, (416)-612-3210.

Para hacer esto se requiere de códigos de programación complejos y de muchísima experiencia en operaciones de entrada y salida de datos. Para evitar que usted tenga que escribir tantos códigos para definir una máscara de entrada he creado una función especial y única que le permitirá definir la mayoría de formatos de entrada. Esta función contiene todos los códigos necesarios para definir cualquier máscara de entrada con solo especificar algunos argumentos. Podrá copiar el código de la función en su proyecto y llamarla cada vez que sea necesario.

Para crear una máscara de entrada utilizando la función que he creado, siga los siguientes pasos:

- Abra un nuevo proyecto.
- Haga clic en el menú **View (Ver)** y, a continuación, ejecute la opción **Code (Código)**.
- Escriba **cuidadosamente** el código de la función si está leyendo este libro en un material impreso. En caso de tener el libro en formato digital, copie la función que se muestra en la página siguiente y péguela en el Editor de código de Visual Basic.



```

'Función para definir mascara de entrada creada por: Ing. Carlos M. Rodríguez Bucarely.
Function MascaraBucarely(TextBoxName As String, MascFormat As String, KeyAscii As Integer, DataType As String)
Dim Formato As String
Dim Tecla As Integer
Dim Posiciones(100) As Integer
Dim E As Integer
Dim ES As Boolean
Dim I As Integer
Dim C As String
Dim II, ObjectNumber As Integer
Dim dd As Boolean
For I = 0 To 255: On Error GoTo x: If UCase(Me.Controls(I).Name) = UCase(TextBoxName) Then ObjectNumber = I
Next I
x:
If UCase(DataType) = "NUMBER" Then
    If InStr("0123456789" & Chr(8) & Chr(13), Chr(KeyAscii)) = 0 Then
        KeyAscii = 0
    End If
End If
On Error GoTo ControlNoEncontrado
Dim Control As Object
Set Control = Me.Controls(ObjectNumber)
ES = False
Formato = MascFormat
Tecla = KeyAscii
KeyAscii = 0

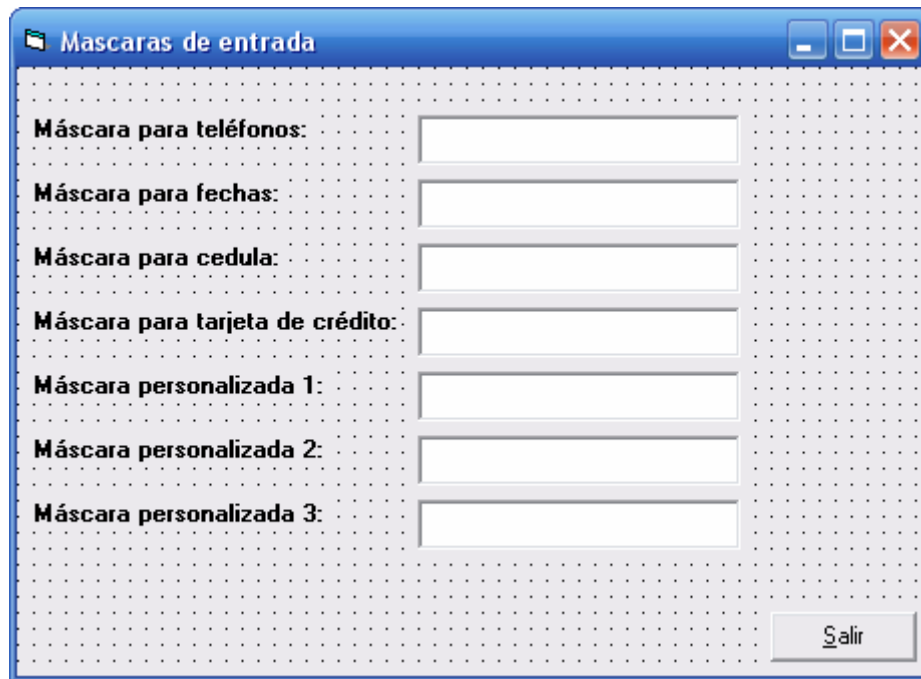
If Len(Control.Text) = 0 Then
    Control.Text = MascFormat
End If
For I = 1 To 100
    C = Mid(Formato, I, 1)
    If C = "_" Or C = "#" Then
        E = E + 1
        Posiciones(E) = I
    End If
Next I
dd = False
Control.SelLength = 1
If Tecla <> 8 Then
    For I = 1 To 100
        For II = 1 To 100
            If Control.SelStart + 1 = Posiciones(II) Then dd = True
        Next II
        If dd = True Then
            Exit For
        Else
            dd = False
            Control.SelStart = Control.SelStart + 1
        End If
    Next I
End If
If Tecla = 8 Then
    If Control.SelStart > 0 Then
        Control.SelStart = Control.SelStart - 1
    End If
End If
For I = 1 To 100
    If Control.SelStart + 1 = Posiciones(I) Then
        If Tecla = 8 Then
            Control.SelLength = 1
        Else
            KeyAscii = Tecla
        End If
    End If
Next I
Exit Function
ControlNoEncontrado:
End Function

```



Capítulo 3

- Agregue cinco cajas de texto. Cada caja de texto tendrá una máscara de entrada diferente, tal y como se muestra en la siguiente imagen:



- Haga doble clic sobre la primera caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.
If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada para Teléfonos.
MaskBucarely "text1", "(____) ____ ____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:

'Bloquea la utilización de la tecla DELETE o SUPR.
If KeyCode = 46 Then KeyCode = 0

- Haga doble clic sobre la segunda caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.
If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada para Fechas.
MaskBucarely "text2", "__/__/____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:

'Bloquea la utilización de la tecla DELETE o SUPR.
If KeyCode = 46 Then KeyCode = 0



Capítulo 3

- Haga doble clic sobre la tercera caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.
If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada para Cédulas.
MascaraBucarely "text3", "____ - ____ - ____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:

'Bloquea la utilización de la tecla DELETE o SUPR.
If KeyCode = 46 Then KeyCode = 0

- Haga doble clic sobre la cuarta caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.
If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada para Tarjetas de crédito.
MascaraBucarely "text4", "____ ____ ____ ____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:

'Bloquea la utilización de la tecla DELETE o SUPR.
If KeyCode = 46 Then KeyCode = 0

- Haga doble clic sobre la quinta caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.
If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada personalizada.
MascaraBucarely "text5", "SSN ____ - ____ - ____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:

'Bloquea la utilización de la tecla DELETE o SUPR.
If KeyCode = 46 Then KeyCode = 0

- Haga doble clic sobre la sexta caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.
If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada personalizada.
MascaraBucarely "text6", "RNC-_____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:



'Bloquea la utilización de la tecla DELETE o SUPR.

If KeyCode = 46 Then KeyCode = 0

- Haga doble clic sobre la séptima caja de texto. Seleccione el evento **KeyPress** de la lista de eventos. Dentro de este evento escriba:

'Hace que cuando se pulse la tecla ENTER pase a la siguiente caja.

If KeyAscii = 13 Then SendKeys "{tab}"

'Define una máscara de entrada personalizada.

MascaraBucarely "text7", "NCF: _____", KeyAscii, "NUMBER"

- Dentro del evento **KeyDown** de esta misma caja escriba:

'Bloquea la utilización de la tecla DELETE o SUPR.

If KeyCode = 46 Then KeyCode = 0

- Haga doble clic sobre el formulario y dentro del evento **Load** escriba:

'Hace que todas las máscaras de entrada se muestren al iniciar la aplicación.

Text1_KeyPress (0)

Text2_KeyPress (0)

Text3_KeyPress (0)

Text4_KeyPress (0)

Text5_KeyPress (0)

Text6_KeyPress (0)

Text7_KeyPress (0)

- Corra la aplicación. Podrá observar que cada caja de texto contiene una máscara de entrada distinta, tal y como se ve en la siguiente figura:



- Escriba en cada una de las cajas de texto. Podrá notar, que las cajas de texto facilitan la entrada de los datos y obliga a que sean introducidos de una determinada manera. También hemos codificado para que la pulsación de la tecla ENTER desplace el punto de inserción a la próxima caja de texto.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio28-3** para el *formulario* y **Ejercicio28-3** para el proyecto.

Explicación de la función **MascaraBucarelly**

Como pudo haber notado, la función **MascaraBucarelly** es llamada desde el evento **KeyPress** de cada caja de texto. Esta función es utilizada de acuerdo al siguiente formato:

MascaraBucarelly [*TextBoxName*], [*MascFormat*], [*KeyAscii*], [*Data Type*]

Donde:

TextBoxName: Una cadena de caracteres que representa el nombre de la caja de texto dentro del formulario. Este nombre debe ser especificado entre comillas, por ejemplo, "Text1", "txtTelefono", "txtCedula", etc.

MascFormat: Una cadena de caracteres que representa el formato de máscara de entrada en la caja de texto. Debe ser especificada entre comillas.

KeyAscii: Es una constante que representa el valor en la tabla **ASCII** de la tecla pulsada. Este es necesario para el funcionamiento correcto de la función. No requiere que se especifique entre comillas.

Data Type: Una cadena de caracteres que representa el tipo de datos que admitirá la caja de texto. Este puede tomar los valores **NUMBER** O **STRING**. Cuando es **NUMBER** la caja de texto solo podrá admitir números. Si es **STRING**, la caja de texto podrá admitir texto y números a la vez. Requiere que sea especificado entre comillas.

NOTA: Podrá hacer que la máscara de entrada aparezca cuando la caja de texto reciba el enfoque y no cuando inicie la aplicación. Esto puede hacerlo llamando el evento **KeyPress** de la caja correspondiente mediante el evento **GotFocus** de dicha caja. Si opta por hacer esto tendrá que omitir el código escrito en el evento **Load** del formulario.

• Filtrar el contenido de una caja de texto o una cadena de caracteres

En muchas ocasiones nos veremos en la necesidad de depurar el contenido de una caja de texto o una cadena de caracteres, eliminando símbolos, caracteres, formatos o mascara de entrada establecidos en la misma. Por ejemplo, usted puede definir una mascara de entrada para solicitar un número telefónico como (809)-569-5445, pero al momento de almacenarlo no le interesa los paréntesis, o bien, solo desea almacenar los números sin los paréntesis ni los guiones. Para filtrar estos tipos de cadenas personalizadas Visual Basic no posee rutinas predefinidas, en consecuencia, usted se verá en la obligación de construir una función especial para realizar dicha operación. Pero de la misma manera que dije anteriormente, estas operaciones requieren de mucha experiencia de parte del programador. Por tal razón, he creado una función especial llamada **FiltrarCadenaBucarelly**. Esta le permitirá filtrar todo tipo de cadena de caracteres, e incluso, sustituir caracteres por otros caracteres dentro de la misma cadena. Vamos a crear un proyecto donde mostraremos varios ejemplos de algunos filtros de cadenas utilizando la función que he creado.

- Abra un nuevo proyecto.
- Haga clic en el menú **View (Ver)** y, a continuación, ejecute la opción **Code (Código)**.

Ing. Carlos Manuel Rodríguez Bucarelly



Capítulo 3

- Escriba **cuidadosamente** el código de la función si esta leyendo este libro en un material impreso. En caso de tener el libro en formato digital, copie la función que se muestra más abajo y péguela en el Editor de código de Visual Basic.

'Función para filtrar cadenas de caracteres creada por: Ing. Carlos M. Rodríguez Bucarelly.

Function FiltrarCadenaBucarelly(Texto As String, Caracteres As String, RemplazarCaracteres As String) As String

Dim i As Integer

Dim C As String

Dim StringResult As String

For i = 1 To Len(Texto)

C = Mid(Texto, i, 1)

If InStr(1, Caracteres, C, vbTextCompare) Then

StringResult = StringResult & RemplazarCaracteres

Else

StringResult = StringResult & C

End If

Next i

FiltrarCadenaBucarelly = StringResult

End Function

- Inserte ocho etiquetas, ocho cajas de texto y ocho botones de comando, tal y como se muestra en la siguiente figura:

Ing. Carlos Manuel Rodríguez Bucarelly



Capítulo 3

▪ Modifique la propiedad **Caption** de cada etiqueta. En la propiedad **Caption** de la primera etiqueta escriba Texto 1, en la segunda Texto 2, en la tercera Texto 3, así hasta la etiqueta ocho.

▪ En la propiedad **Text** de cada texto escriba los siguientes valores:

Control	Propiedad	Valor
Text1	Text	(809)-659-5478
Text2	Text	(809)-778-5426
Text3	Text	Visual Basic 6.0
Text4	Text	5465 4455 4564 8984
Text5	Text	809 597 6997
Text6	Text	255.365.215.9856
Text7	Text	AXZBT543HGJ841
Text8	Text	BXD54D11593UJJKL

▪ En la propiedad **Caption** de cada botón de comando escriba los siguientes valores:

Control	Propiedad	Valor
Command1	Caption	Reemplazar Guiones "-" por Espacios
Command2	Caption	Quitar Guiones "-" y Paréntesis ()
Command3	Caption	Quitar Espacios
Command4	Caption	Reemplazar los Espacios por Ceros
Command5	Caption	Reemplazar los Espacios por Guiones
Command6	Caption	Quitar los Puntos (.)
Command7	Caption	Quitar cualquier número
Command8	Caption	Eliminar cualquier letra

▪ Haga doble clic sobre el primer botón de comando y dentro del evento **Click** escriba:

Reemplaza los guiones por espacios.
Text1.Text = FiltrarCadenaBucarely(Text1.Text, "-", " ")

▪ Haga doble clic sobre el segundo botón de comando y dentro del evento **Click** escriba:

Quita los guiones y los paréntesis.
Text2.Text = FiltrarCadenaBucarely(Text2.Text, "-()", "")

▪ Haga doble clic sobre el tercer botón de comando y dentro del evento **Click** escriba:

Quita los espacios.
Text3.Text = FiltrarCadenaBucarely(Text3.Text, " ", "")

▪ Haga doble clic sobre el cuarto botón de comando y dentro del evento **Click** escriba:



'Reemplazar los espacios por ceros.

```
Text4.Text = FiltrarCadenaBucarely(Text4.Text, " ", "0")
```

- Haga doble clic sobre el quinto botón de comando y dentro del evento **Click** escriba:

'Reemplaza los espacios por guiones.

```
Text5.Text = FiltrarCadenaBucarely(Text5.Text, " ", "-")
```

- Haga doble clic sobre el sexto botón de comando y dentro del evento **Click** escriba:

'Quita los puntos.

```
Text6.Text = FiltrarCadenaBucarely(Text6.Text, ".", "")
```

- Haga doble clic sobre el séptimo botón de comando y dentro del evento **Click** escriba:

'Quita cualquier número.

```
Text7.Text = FiltrarCadenaBucarely(Text7.Text, "0123456789", "")
```

- Haga doble clic sobre el octavo botón de comando y dentro del evento **Click** escriba:

'Elimina cualquier letra.

```
Text8.Text = FiltrarCadenaBucarely(Text8.Text, "qwertyuiopasdfghjklñzxcvbnm", "")
```

- Corra la aplicación.
- Haga clic en cada uno de los botones de comando. Podrá observar que las cadenas escritas son filtradas según el mandato especificado en el botón de comando.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio29-3** para el *formulario* y **Ejercicio29-3** para el proyecto.

Explicación de la función FiltrarCadenaBucarely

La función **FiltrarCadenaBucarely** devuelve una cadena filtrada según un filtro especificado. Esta función es utilizada de acuerdo al siguiente formato:

FiltrarCadenaBucarely ([*Texto*], [*Caracteres*], [*ReemplazarCaracteres*])

Donde:

Texto: Representa la cadena de caracteres a la cual se le va a aplicar el filtro. Esta puede ser una variable o una referencia a alguna propiedad de un objeto que devuelva una cadena de caracteres.

Caracteres: Es una cadena de caracteres que contiene todos los caracteres que será eliminados o filtrados de la cadena principal. Esta puede ser una variable o un conjunto de caracteres especificados entre comillas.

ReemplazarCaracteres: Representa un carácter o un conjunto de caracteres que serán utilizados para reemplazar los caracteres filtrados. Esta puede ser una variable, un carácter o una cadena de caracteres especificada entre comillas.

NOTA: Puede almacenar la cadena filtrada en una variable o en un objeto que soporte propiedades texto. Utilizar una variable es una muy buena opción a la hora que desee utilizar la cadena filtrada para operaciones de cálculo o almacenamiento en archivos de base de datos.



• Formato del texto

Muchas aplicaciones empresariales les permiten introducir datos mediante un formato y luego lo muestran con otro. Por ejemplo, usted introduce un valor numérico y automáticamente la aplicación le asigna separadores de millares y un número fijo de cifras decimales. Otras aplicaciones que trabajan con valores monetarios agregan automáticamente el símbolo de moneda (\$, US\$, RD\$, pta, etc.) correspondiente a un país determinado. También podrá establecer formatos a las fechas, expresándola en formato largo o corto.

Para hacer todo lo mencionado anteriormente disponemos de la función **Format**. Es una de las mejores opciones cuando queremos aplicar formatos a los valores numéricos o de texto. Para aplicar estos formatos a una caja de texto se recomienda codificar el evento **LostFocus** de dicha caja. Por ejemplo, si queremos agregar separadores de millares a una caja de texto llamada Text1, codifique el evento **LostFocus** con el siguiente código:

On Error Resume Next

```
Text1.Text = Format(CDbl(Text1.Text), "#,###,###,##0.#####")
```

O bien, puede utilizar el siguiente código:

On Error Resume Next

```
Text1.Text = Format(CDbl(Text1.Text), "##,##0.00")
```

Estos dos bloques de código agregan separadores de millares a los valores numéricos. La diferencia entre ambos códigos es que el segundo agrega la parte decimal “.00” al final de cada valor numérico. Por ejemplo, usted introduce el valor 52 este se convierte a 52.00 cuando la caja pierde el enfoque, si introduce 1524 se convierte a 1,524.00, si introduce 3425633 se convierte a 3,425,633.00. El primer código hace lo mismo pero no agrega la parte decimal “.00”, sino solamente los separadores de millares (las comas).

En los dos bloques de código anterior y en los siguientes hemos agregado la línea de código **On Error Resume Next** para que Visual Basic no produzca un error al intentar dar formato a una caja de texto que este vacía o a un valor numérico nulo. La función **CDbl** convierte el valor escrito en la caja de texto en un tipo de datos numérico **Double** (*doble precisión*), permitiendo de esta manera que los valores numéricos muy grandes escritos en la caja de texto sean entendidos y tratados como tal dentro de un intervalo numérico válido en Visual Basic.

Si ha aplicado un formato numérico como el que explicamos anteriormente, usted puede hacer que el número vuelva a tener su formato original, es decir, quitar los separadores de millares. Si quiere que el número vuelva a su estado original cuando la caja reciba nuevamente el enfoque, entonces, dentro del evento **GotFocus** de la caja de texto escriba:

On Error Resume Next

```
Text1.Text = CDbl(Text1.Text)
```

Si ha agregado otro tipo de símbolo al valor numérico, por ejemplo, un símbolo monetario (\$, RD\$, US\$, pta), entonces, la mejor opción es utilizar la función **FiltrarCadenaBucarely** antes de ejecutar el código anterior. De esta manera podrá primero quitar los caracteres no deseados.

Vamos a crear un programa donde se mostrarán algunos tipos de conversiones utilizando la función **Format**.



- Abra un nuevo proyecto.
- Diseñe la siguiente pantalla:

- Establezca los siguientes nombres a las cajas de texto:

Control	Propiedad	Valor
Text1	Name	txtNumeroGeneral
Text2	Name	txtFormatoMonetario1
Text3	Name	txtFormatoMonetario2
Text4	Name	txtFormatoMonetario3
Text5	Name	txtFormatoMonetario4
Text6	Name	txtFechaCorta
Text7	Name	txtFechaLarga
Text8	Name	txtHoraMilitar
Text9	Name	txtHoraMeridiana



Capítulo 3

- Dentro del evento **Load** del formulario escriba:

'Este bloque de código agrega valores a cada caja de texto para luego aplicar los formatos.

```
txtNumeroGeneral.Text = CDb(321545)
txtFormatoMonetario1.Text = CDb(15860)
txtFormatoMonetario2.Text = CDb(14500)
txtFormatoMonetario3.Text = CDb(16850)
txtFormatoMonetario4.Text = CDb(25000)
```

- Copie la función **FiltrarCadenaBucarely** en la sección general del editor de código:

'Función para filtrar cadenas de caracteres creada por: Ing. Carlos M. Rodríguez Bucarely.

Function FiltrarCadenaBucarely(Texto As String, Caracteres As String, RemplazarCaracteres As String) As String

```
Dim i As Integer
Dim C As String
Dim StringResult As String

For i = 1 To Len(Texto)

    C = Mid(Texto, i, 1)

    If InStr(1, Caracteres, C, vbTextCompare) Then
        StringResult = StringResult & RemplazarCaracteres
    Else
        StringResult = StringResult & C
    End If

Next i
```

FiltrarCadenaBucarely = StringResult

End Function

- Dentro del primer botón **Aplicar** escriba:

'Aplica el formato de número general.

On Error Resume Next

txtNumeroGeneral.Text = Format(CDb(txtNumeroGeneral.Text), "#,###,###,##0.#####")

- Dentro del primer botón **Quitar** escriba:

'Quita el formato de número general.

On Error Resume Next

txtNumeroGeneral.Text = CDb(txtNumeroGeneral.Text)

- Dentro del segundo botón **Aplicar** escriba:

'Aplica el formato monetario simple.

On Error Resume Next

txtFormatoMonetario1.Text = Format(CDb(txtFormatoMonetario1.Text), "##,##0.00")



- Dentro del segundo botón Quitar escriba:

```
'Quita el formato monetario simple.  
On Error Resume Next  
txtFormatoMonetario1.Text = CDb(txtFormatoMonetario1.Text)
```

- Dentro del tercer botón Aplicar escriba:

```
'Aplica el formato monetario de República Dominicana.  
On Error Resume Next  
txtFormatoMonetario2.Text = "RD$" & Format(CDb(txtFormatoMonetario2.Text), "##,##0.00")
```

- Dentro del tercer botón Quitar escriba:

```
'Quita el formato monetario de República Dominicana.  
On Error Resume Next  
txtFormatoMonetario2.Text = CDb(FiltrarCadenaBucarely(txtFormatoMonetario2.Text, "RD$", ""))
```

- Dentro del cuarto botón Aplicar escriba:

```
'Aplica el formato monetario de los Estados Unidos.  
On Error Resume Next  
txtFormatoMonetario3.Text = "US$" & Format(CDb(txtFormatoMonetario3.Text), "##,##0.00")
```

- Dentro del cuarto botón Quitar escriba:

```
'Quita el formato monetario de los Estados Unidos.  
On Error Resume Next  
txtFormatoMonetario3.Text = CDb(FiltrarCadenaBucarely(txtFormatoMonetario3.Text, "US$", ""))
```

- Dentro del quinto botón Aplicar escriba:

```
'Aplica el formato monetario del país actual.  
On Error Resume Next  
txtFormatoMonetario4.Text = FormatCurrency(CDb(txtFormatoMonetario4.Text))
```

- Dentro del quinto botón Quitar escriba:

```
'Quita el formato monetario del país actual.  
On Error Resume Next  
txtFormatoMonetario4.Text = CDb(txtFormatoMonetario4.Text)
```

- Dentro del primer botón Mostrar:

```
'Pone la fecha en formato corto.  
txtFechaCorta.Text = Format(Date, "Short Date")
```

- Dentro del segundo botón Mostrar:

```
'Pone la fecha en formato largo.  
txtFechaLarga.Text = Format(Date, "Long Date")
```



- Dentro del tercer botón Mostrar:

```
'Mostrar la hora en formato militar.  
txtHoraMilitar.Text = Format(Time, "h:m:s")
```

- Dentro del cuarto botón Mostrar:

```
'Mostrar la hora en formato meridiano.  
txtHoraMeridiana.Text = Format(Time, "hh:mm:ss AMPM")
```

- Corra la aplicación. Utilice los botones Aplicar y Quitar para que vea el funcionamiento del código escrito anteriormente.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio30-3** para el *formulario* y **Ejercicio30-3** para el proyecto.

• Validación del contenido de una caja de texto

La validación de una caja de texto es un proceso que consiste en verificar que el contenido de una caja de texto cumpla con una determinada condición o criterio. Por ejemplo, se puede verificar que no se deje vacía una determinada caja de texto, o verificar que el valor de una caja de texto sea positivo, etc.

Para validar el contenido de una caja de texto el programador podrá elegir el momento en que desee hacerlo. Podrá hacerlo inmediatamente la caja de texto intenta perder el enfoque utilizando el evento **Validate**, o podrá validar el contenido de la(s) caja(s) de texto y otros controles antes de cerrar o descargar la ventana donde se encuentran los datos. Evaluar en ambos momentos podría resultar el método más adecuado pero esto dependerá de su criterio como programador.

Vamos a realizar una aplicación donde veremos los momentos en el que podemos validar el contenido de las cajas de textos. La aplicación que vamos a crear simula un simple formulario donde se solicitan algunos datos para el registro de una persona en una determinada empresa. Pero antes de registrar a la persona hay que tener en cuenta un criterio: *no se registran personas menores de 18 años y que el nombre, apellido, edad, dirección y la cédula deben ser especificada obligatoriamente antes de efectuar el registro.*

En esta aplicación validamos cuando las cajas pierden el enfoque y cuando hacemos clic en el botón Registrar.

- Abra un nuevo proyecto y diseñe la siguiente pantalla:

Ing. Carlos Manuel Rodríguez Bucarellly



- Cambie los nombres por defectos de las cajas de texto por los que se muestran a continuación:

Control	Propiedad	Valor
Text1	Name	txtNombre
Text2	Name	txtApellido
Text3	Name	txtCedula
Text4	Name	txtEdad
Text5	Name	txtDireccion
Text6	Name	txtTelefono

Recuerde borrar el contenido que tienen las cajas de texto en la propiedad **Text**.

- Seleccione el botón Registrar y establezca el valor **False** en la propiedad **CauseValidation**. Esto es para que no se validen las cajas de texto cuando el botón de comando reciba el enfoque. Haga lo mismo con el botón Cancelar.

- Haga doble clic sobre la primera caja de texto y, a continuación, seleccione el evento **CauseValidation**. Dentro de este evento escriba:

```
'Verifica si la caja de texto esta vacía.
If Len(Trim(txtNombre.Text)) = 0 Then
MsgBox ("Debe introducir el Nombre.")
Cancel = True ' Hace que el enfoque NO pase a otro control.
End If
```

- Haga doble clic sobre la segunda caja de texto y, a continuación, seleccione el evento **CauseValidation**. Dentro de este evento escriba:

```
'Verifica si la caja de texto esta vacía.
If Len(Trim(txtApellido.Text)) = 0 Then
MsgBox ("Debe introducir el Apellido.")
Cancel = True ' Hace que el enfoque NO pase a otro control.
End If
```

- Haga doble clic sobre la tercera caja de texto y, a continuación, seleccione el evento **CauseValidation**. Dentro de este evento escriba:

```
'Verifica si la caja de texto esta vacía.
If Len(Trim(txtCedula.Text)) = 0 Then
MsgBox ("Debe introducir la Cédula.")
Cancel = True ' Hace que el enfoque NO pase a otro control.
End If
```

- Haga doble clic sobre la cuarta caja de texto y, a continuación, seleccione el evento **CauseValidation**. Dentro de este evento escriba:

```
'Verifica si la caja de texto esta vacía.
If Len(Trim(txtEdad.Text)) = 0 Then
MsgBox ("Debe introducir la Edad.")
Cancel = True ' Hace que el enfoque NO pase a otro control.
```



```
'Verifica que la edad cumpla con el criterio establecido.  
Elseif Val(txtEdad.Text) < 18 Then  
MsgBox ("No se registran menores de edad.")  
Cancel = True ' Hace que el enfoque NO pase a otro control.  
End If
```

- Haga doble clic sobre la quinta caja de texto y, a continuación, seleccione el evento **CauseValidation**. Dentro de este evento escriba:

```
'Verifica si la caja de texto esta vacía.  
If Len(Trim(txtDireccion.Text)) = 0 Then  
MsgBox ("Debe introducir la Dirección.")  
Cancel = True ' Hace que el enfoque NO pase a otro control.  
End If
```

NOTA: No hemos validado el teléfono porque no esta establecido en uno de los criterios de la aplicación.

- En el evento **Click** del botón Registrar escriba:

```
'Verifica si la caja de texto Nombre esta vacía.  
If Len(Trim(txtNombre.Text)) = 0 Then  
MsgBox ("Debe introducir el Nombre.")  
txtNombre.SetFocus ' Hace que la caja reciba el enfoque.  
'Verifica si la caja de texto Apellido esta vacía.  
Elseif Len(Trim(txtApellido.Text)) = 0 Then  
MsgBox ("Debe introducir el Apellido.")  
txtApellido.SetFocus ' Hace que la caja reciba el enfoque.  
'Verifica si la caja de texto Cédula esta vacía.  
Elseif Len(Trim(txtCedula.Text)) = 0 Then  
MsgBox ("Debe introducir la Cedula.")  
txtCedula.SetFocus ' Hace que la caja reciba el enfoque.  
'Verifica si la caja de texto Edad esta vacía.  
Elseif Len(Trim(txtEdad.Text)) = 0 Then  
MsgBox ("Debe introducir la Edad.")  
txtEdad.SetFocus  
'Verifica si la persona es menor de Edad.  
Elseif val(txtEdad.text) < 18 Then  
MsgBox("No se registran personas menores de edad.")  
txtEdad.SetFocus  
'Verifica si la caja de texto Dirección esta vacía.  
Elseif Len(Trim(txtDireccion.Text)) = 0 Then  
MsgBox ("Debe introducir la Dirección.")  
txtDireccion.SetFocus ' Hace que la caja reciba el enfoque.  
  
Else  
'Aquí se escribe la codificación para almacenar los datos en la Base de datos.  
'En nuestro caso mostraremos un mensaje para hacer un simulacro.  
MsgBox ("El registro ha sido almacenado satisfactoriamente.")  
'Limpiamos las cajas.  
txtNombre.Text = ""  
txtApellido.Text = ""  
txtCedula.Text = ""  
txtEdad.Text = ""  
txtTelefono.Text = ""  
txtDireccion.Text = ""  
End If
```



- En el evento **Click** del botón Cancelar escriba:

```
'Sale del programa.  
End
```

- Corra la aplicación. Deje la caja de texto Nombre vacía y haga clic en la caja de texto Apellido. Podrá observar que no podrá pasar a la siguiente caja sin antes haber escrito algo en la caja Nombre. Por igual, si hace clic en el botón Registrar y deja uno de los campos requeridos vacío, se le notificará que debe introducir algún dato en la caja.

NOTA: NO todas las cajas de texto en una aplicación son validadas. Decidir cuales cajas de texto validar dependerá de análisis y estudios realizados por el programador, y en la mayoría de los casos por requerimientos internos del sistema.

COMENTARIO: En muchas ocasiones el programador coloca un texto descriptivo, un símbolo o una pequeña imagen en los campos que son requeridos en un formulario. Esto notifica al usuario los campos que tiene que completar obligatoriamente.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio31-3** para el *formulario* y **Ejercicio31-3** para el proyecto.

- 3.3.1.3 Métodos de las cajas de texto

Incluye los métodos más usuales de los controles (**Drag**, **Refresh**, **SetFocus**). También posee otros métodos que no son usados con tanta frecuencia en una aplicación de Visual Basic.

- 3.3.1.4 Ejercicios prácticos

- El siguiente ejercicio muestra como realizar operaciones matemáticas utilizando cajas de texto. El ejercicio esta compuesto por tres cajas de texto y cuatro botones de comando que permitirán realizar las operaciones básicas de matemática.

- Cambie los nombres por defecto de las cajas de texto: txtPrimerValor para la primera caja, txtSegundoValor para la segunda caja y txtResultado para la tercera caja.



- Dentro del evento **KeyPress** de la primera caja de texto escriba:

```
'Hace que la caja de texto solo acepte números.
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then
KeyAscii = 0
End If
```

- Dentro del evento **KeyPress** de la segunda caja de texto escriba:

```
'Hace que la caja de texto solo acepte números.
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then
KeyAscii = 0
End If
```

- Haga doble clic sobre el botón Sumar y dentro del evento **Click** escriba:

```
'Validación de las cajas de texto verificando que no estén vacías.
'Verificamos la primera caja.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
    MsgBox ("Debe introducir un valor en la primera caja.")
    txtPrimerValor.SetFocus      ' Hace que la caja reciba el enfoque.
    'Verificamos la segunda caja.
    ElseIf Len(Trim(txtSegundoValor.Text)) = 0 Then
        MsgBox ("Debe introducir un valor en la segunda caja.")
        txtSegundoValor.SetFocus  ' Hace que la caja reciba el enfoque.

Else
    'Suma el valor de las dos cajas de texto y lo muestra en la caja resultado.
    'Utilizamos la sentencia Val para indicar que es con números que estamos trabajando.
    txtResultado.Text = Val(txtPrimerValor.Text) + Val(txtSegundoValor.Text)

End If
```

- Haga doble clic sobre el botón Restar y dentro del evento **Click** escriba:

```
'Validación de las cajas de texto verificando que no estén vacías.
'Verificamos la primera caja.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
    MsgBox ("Debe introducir un valor en la primera caja.")
    txtPrimerValor.SetFocus      ' Hace que la caja reciba el enfoque.
    'Verificamos la segunda caja.
    ElseIf Len(Trim(txtSegundoValor.Text)) = 0 Then
        MsgBox ("Debe introducir un valor en la segunda caja.")
        txtSegundoValor.SetFocus  ' Hace que la caja reciba el enfoque.

Else
    'Resta el valor de las dos cajas de texto y lo muestra en la caja resultado.
    'Utilizamos la sentencia Val para indicar que es con números que estamos trabajando.
    txtResultado.Text = Val(txtPrimerValor.Text) - Val(txtSegundoValor.Text)

End If
```

- Haga doble clic sobre el botón Multiplicar y dentro del evento **Click** escriba:



```
'Validación de las cajas de texto verificando que no estén vacías.
'Verificamos la primera caja.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
    MsgBox ("Debe introducir un valor en la primera caja.")
    txtPrimerValor.SetFocus      ' Hace que la caja reciba el enfoque.
'Verificamos la segunda caja.
Elseif Len(Trim(txtSegundoValor.Text)) = 0 Then
    MsgBox ("Debe introducir un valor en la segunda caja.")
    txtSegundoValor.SetFocus     ' Hace que la caja reciba el enfoque.

Else
    'Multiplica el valor de las dos cajas de texto y lo muestra en la caja resultado.
    'Utilizamos la sentencia Val para indicar que es con números que estamos trabajando.
    txtResultado.Text = Val(txtPrimerValor.Text) * Val(txtSegundoValor.Text)

End If
```

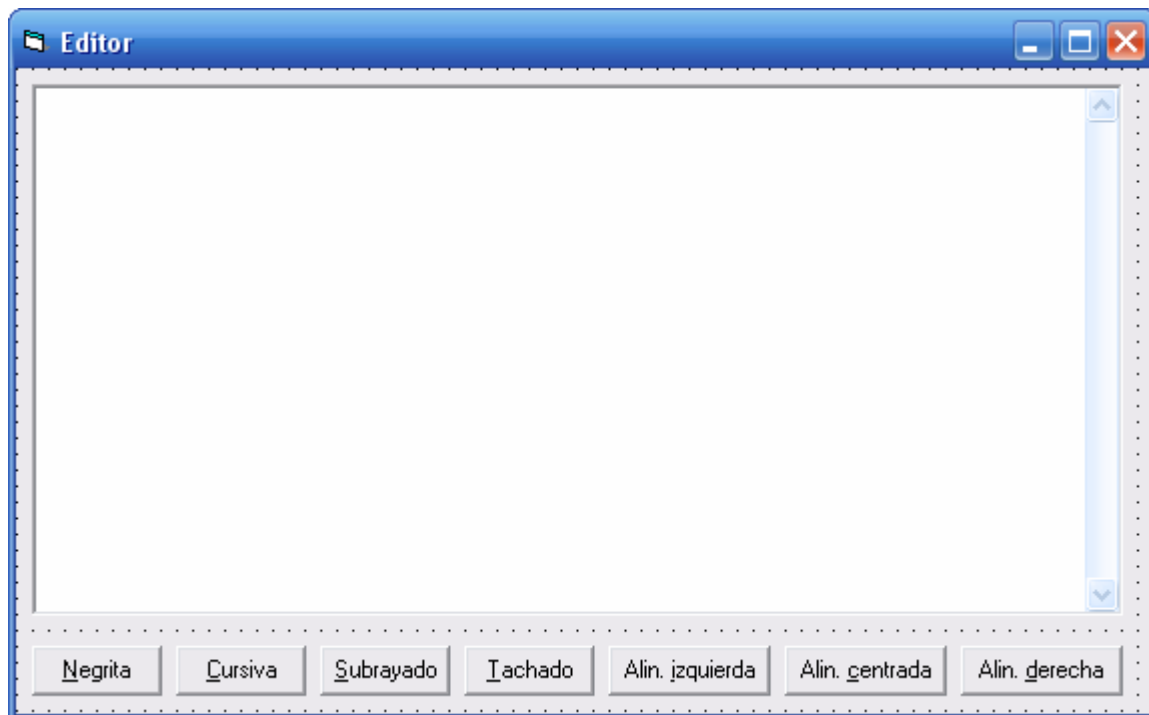
- Haga doble clic sobre el botón Dividir y dentro del evento **Click** escriba:

```
'Validación de las cajas de texto verificando que no estén vacías.
'Verificamos la primera caja.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
    MsgBox ("Debe introducir un valor en la primera caja.")
    txtPrimerValor.SetFocus      ' Hace que la caja reciba el enfoque.
'Verificamos la segunda caja.
Elseif Len(Trim(txtSegundoValor.Text)) = 0 Then
    MsgBox ("Debe introducir un valor en la segunda caja.")
    txtSegundoValor.SetFocus     ' Hace que la caja reciba el enfoque.

Else
    'Divide el valor de las dos cajas de texto y lo muestra en la caja resultado.
    'Utilizamos la sentencia Val para indicar que es con números que estamos trabajando.
    txtResultado.Text = Val(txtPrimerValor.Text) / Val(txtSegundoValor.Text)

End If
```

- Corra la aplicación.
- Agrega algún valor numérico en la primera y en la segunda caja de texto. Haga clic sobre los botones de comando para efectuar una operación matemática y ver los resultados.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio32-3** para el *formulario* y **Ejercicio32-3** para el proyecto.
- El siguiente ejercicio muestra como cambiar el tipo de fuente de una caja de texto mediante botones de comando. El ejercicio consiste en un pequeño editor compuesto por una caja de texto con barras de desplazamiento y ocho botones de comando, que permitirán realizar las siguientes operaciones: asignar y quitar negrita, asignar y quitar subrayado, asignar y quitar cursiva, asignar y quitar tachado y alinear el texto.
- Diseñe la pantalla que se muestra en la siguiente página:



- Cambie el nombre por defecto de la caja de texto y asígnele: txtContenido.
- Establezca en valor **True** en la propiedad **MultiLine** de la caja de texto y seleccione **2 – Vertical** en la propiedad **ScrollBars**.
- Borre el contenido de la propiedad **Text** de la caja de texto.
- Dentro del botón Negrita escriba:

```
'Verifica si la caja no tiene negrita. Si no la tiene se la pone.  
If txtContenido.FontBold = False Then  
    txtContenido.FontBold = True  
Else  
    'Si tiene negrita se la quita.  
    txtContenido.FontBold = False  
End If  
'Esto que hicimos anteriormente es necesario para que el botón sirva para poner y quitar la negrita.
```

- Dentro del botón Cursiva escriba:

```
'Verifica si la caja no tiene cursiva. Si no la tiene se la pone.  
If txtContenido.FontItalic = False Then  
    txtContenido.FontItalic = True  
Else  
    'Si tiene cursiva se la quita.  
    txtContenido.FontItalic = False  
End If  
'Esto que hicimos anteriormente es necesario para que el botón sirva para poner y quitar la cursiva.
```

- Dentro del botón Subrayado escriba:



```
'Verifica si la caja no tiene subrayado. Si no lo tiene se lo pone.  
If txtContenido.FontUnderline = False Then  
    txtContenido.FontUnderline = True  
Else  
    'Si tiene subrayado se lo quita.  
    txtContenido.FontUnderline = False  
End If  
'Esto que hicimos anteriormente es necesario para que el botón sirva para poner y quitar el  
'subrayado.
```

- Dentro del botón Tachado escriba:

```
'Verifica si la caja no tiene tachado. Si no lo tiene se lo pone.  
If txtContenido.FontStrikethru = False Then  
    txtContenido.FontStrikethru = True  
Else  
    'Si tiene tachado se lo quita.  
    txtContenido.FontStrikethru = False  
End If  
'Esto que hicimos anteriormente es necesario para que el botón sirva para poner y quitar el  
'tachado.
```

- Dentro del botón Alineación Izquierda escriba:

```
'Alinea el texto a la izquierda. El valor 0 alinea a la izquierda.  
txtContenido.Alignment = 0
```

- Dentro del botón Alineación Centrada escriba:

```
'Alinea el texto en el centro. El valor 2 alinea el texto en el centro.  
txtContenido.Alignment = 2
```

- Dentro del botón Alineación Derecha escriba:

```
'Alinea el texto a la derecha. El valor 1 alinea el texto a la derecha.  
txtContenido.Alignment = 1
```

- Corra la aplicación.
- Escriba algo en la caja de texto y utilice los botones para aplicar y quitar el tipo de fuente.

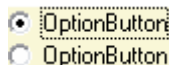
NOTA 1: Los efectos se aplican a todo el contenido de la caja de texto, debido a que las cajas de texto estándar no aplican fuente a porciones de texto seleccionado. Si desea crear aplicaciones potentes como un procesador de palabra utilice el control **RichTextBox**. Este permite aplicar fuente a porciones de texto seleccionado. También permite insertar imágenes, definir márgenes, realizar búsquedas, etc.

NOTA 2: Para cambiar el tipo de letra utilice la propiedad **FontName** y para el tamaño de la letra **FontSize**.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio33-3** para el *formulario* y **Ejercicio33-3** para el proyecto.



- 3.4.1 Los botones de opción (OptionButton)



Un botón de opción muestra una opción que se puede activar o desactivar.

Generalmente, los *controles* **OptionButton** se utilizan en un grupo de opciones para mostrar opciones entre las cuales el usuario solo puede seleccionar una sola. Los *controles* **OptionButton** se agrupan si los dibuja dentro de un contenedor como un *control* **Frame**, un control **PictureBox** o un **Formulario**. Al seleccionar una de las opciones dentro de un contenedor se activa con un punto negro, indicando que la opción ha sido seleccionada. Si hace clic sobre otra opción del mismo contenedor se desactiva la opción que anteriormente estaba activada y se marca con el punto negro la opción que actualmente selecciono. Para agrupar controles **OptionButton** en un **Frame** o **PictureBox**, dibuje en primer lugar el **Frame** o el **PictureBox** y, a continuación, dibuje dentro los controles **OptionButton**.

- 3.4.1.1 Propiedades de los botones de opción

Los botones de opción poseen prácticamente las mismas propiedades de los botones de comando. En los botones de opción al igual que en los botones de comando se destaca la propiedad **Caption** que especifica un texto descriptivo de la función de ese botón en la aplicación. Posee también la propiedad **Value** que indica si el botón de opción está activado o no, puede tomar los valores **True** (activado) o **False** (desactivado). Cuando selecciona una opción dentro de un contenedor la propiedad **Value** de la opción seleccionada almacena el valor **True** y cuando selecciona otra opción dentro del mismo contenedor la opción seleccionada anteriormente almacena **False** en la propiedad **Value**. Para verificar la opción seleccionada se utiliza la sentencia **If** como veremos en los próximos ejercicios.

- 3.4.1.2 Eventos sobre los botones de opción

Los botones de opción poseen los mismos eventos estudiados anteriormente en los botones de comando. Al igual que en los botones de comando el evento más importante de los controles **OptionButton** es el evento **Click**. Este ocurre cuando el usuario hace clic sobre la opción y es prácticamente el único evento que se programa en los botones de opción.

- 3.4.1.3 Métodos de los botones de opción

Los botones de opción también se complementan con los métodos estándar ya estudiados anteriormente (**Drag**, **Refresh**, **SetFocus**, **Move**, **Zorder**). Los métodos son muy poco usados en los controles que usamos frecuentemente, por tal razón, nos limitaremos en hacer énfasis en cada uno de ellos.

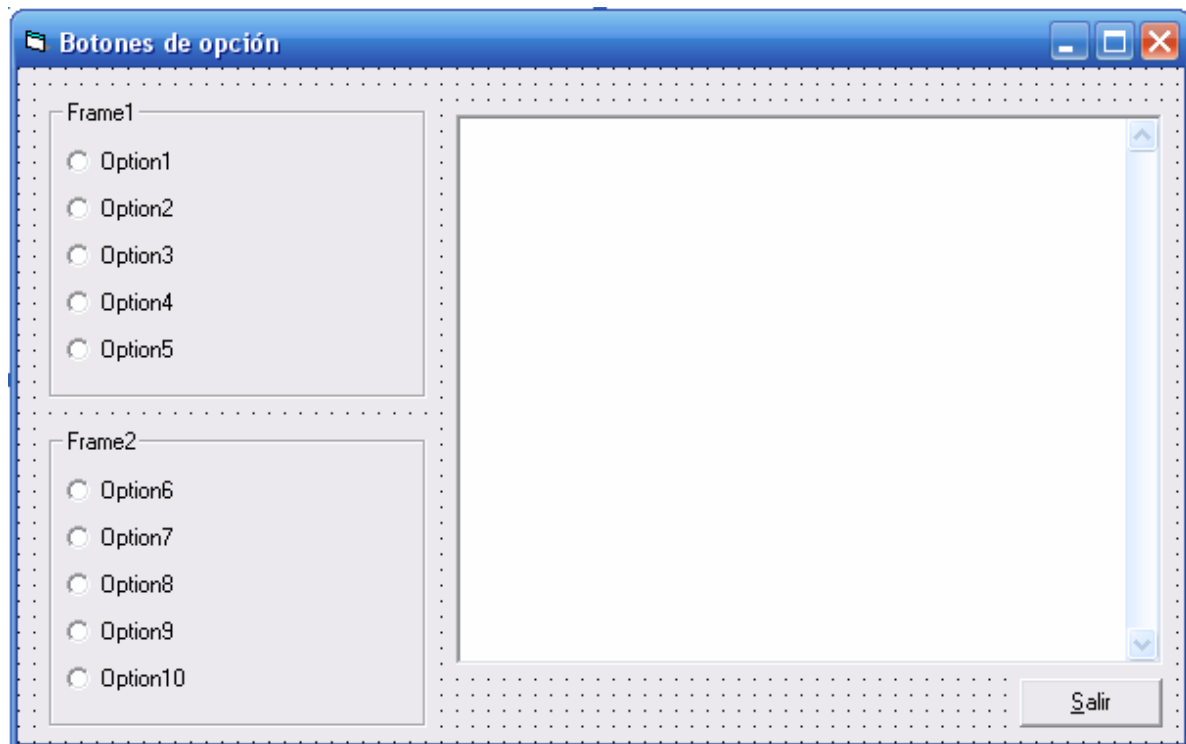
- 3.4.1.4 Ejercicios prácticos

• La siguiente aplicación posee un conjunto de botones de opción dentro de varios contenedores que indicarán los colores que se le va aplicar al fondo y a la letra de una caja de texto. Cada conjunto de colores estará por separado en controles **Frame**. Estos tendrán por encabezado la función que realizará ese conjunto de botones dentro del control **Frame**.

▪ Abra un nuevo proyecto.

▪ En la propiedad **Caption** del formulario escriba Botones de opción. En la propiedad **Height** escriba el valor 5685 y en la propiedad **Width** escriba 9060 para fijar el tamaño adecuado al formulario.

▪ Inserte dos **Frame**  en el formulario y dentro de cada **Frame** cinco botones de opción, tal y como se ve en la imagen de la página siguiente:



- Inserte una caja de texto a la derecha de los *controles* **Frame** y un botón de comando en la esquina inferior izquierda del formulario tal y como se puede apreciar en la imagen anterior. En la propiedad **Caption** del botón de comando escriba **&Salir**.
- Aplique los siguientes valores en las propiedades indicada de los controles que se muestran en la siguiente tabla:

Control	Propiedad	Valor
Frame1	Caption	Colores del fondo
Frame2	Caption	Colores de la letra
Option1	Caption	Rojo
Option2	Caption	Verde
Option3	Caption	Azul
Option4	Caption	Amarillo
Option5	Caption	Blanco
Option6	Caption	Cyan
Option7	Caption	Magenta
Option8	Caption	Blanco
Option9	Caption	Negro
Option10	Caption	Azul
Text1	Name MultiLine ScrollBars Text	txtContenido True 2 – Vertical



- Haga doble clic sobre la primera opción y dentro del evento **Click** escriba:

```
'Pone el color rojo al fondo de la caja de texto.  
txtContenido.BackColor = vbRed
```

- Haga doble clic sobre la segunda opción y dentro del evento **Click** escriba:

```
'Pone el color verde al fondo de la caja de texto.  
txtContenido.BackColor = vbGreen
```

- Haga doble clic sobre la tercera opción y dentro del evento **Click** escriba:

```
'Pone el color azul al fondo de la caja de texto.  
txtContenido.BackColor = vbBlue
```

- Haga doble clic sobre la cuarta opción y dentro del evento **Click** escriba:

```
'Pone el color amarillo al fondo de la caja de texto.  
txtContenido.BackColor = vbYellow
```

- Haga doble clic sobre la quinta opción y dentro del evento **Click** escriba:

```
'Pone el color blanco al fondo de la caja de texto.  
txtContenido.BackColor = vbWhite
```

- Haga doble clic sobre la sexta opción y dentro del evento **Click** escriba:

```
'Pone el color cyan al texto de la caja de texto.  
txtContenido.ForeColor = vbCyan
```

- Haga doble clic sobre la séptima opción y dentro del evento **Click** escriba:

```
'Pone el color magenta al texto de la caja de texto.  
txtContenido.ForeColor = vbMagenta
```

- Haga doble clic sobre la octava opción y dentro del evento **Click** escriba:

```
'Pone el color blanco al texto de la caja de texto.  
txtContenido.ForeColor = vbWhite
```

- Haga doble clic sobre la novena opción y dentro del evento **Click** escriba:

```
'Pone el color negro al texto de la caja de texto.  
txtContenido.ForeColor = vbBlack
```

- Haga doble clic sobre la décima opción y dentro del evento **Click** escriba:

```
'Pone el color azul al texto de la caja de texto.  
txtContenido.ForeColor = vbBlue
```

- Corra la aplicación.
- Utilice los primeros cinco botones de comando para poner el color de fondo a la caja de texto.
- Escriba algún texto en la caja de texto y utilice los últimos cinco botones de comando para cambiar el color del texto.



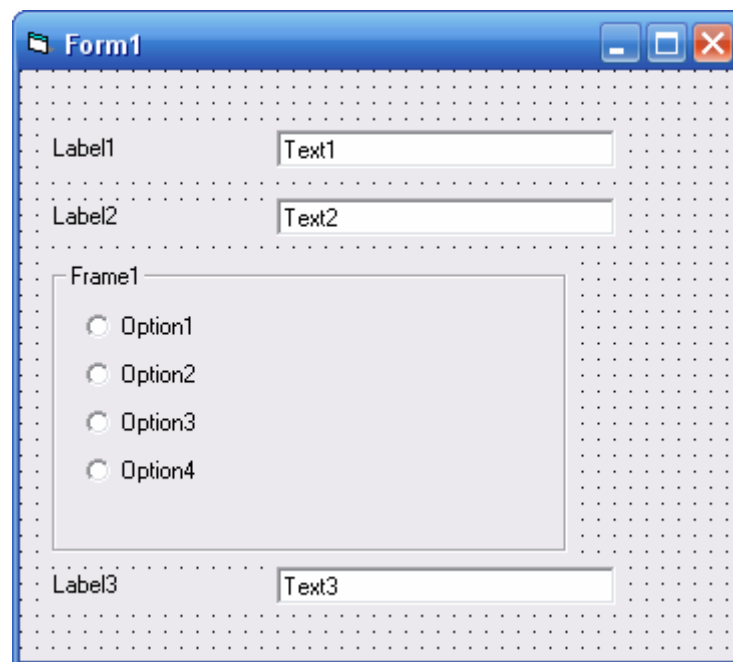
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio34-3** para el *formulario* y **Ejercicio34-3** para el proyecto.

Explicación del programa

En el programa anterior hemos visto dos controles **Frame** compuestos cada uno por cinco opciones. Las primeras cinco opciones para cambiar el color de fondo de la caja de texto y las últimas cinco opciones para cambiar el color del texto de la caja. Dentro de las primeras cinco opciones hemos utilizado la propiedad **BackColor** de las cajas de texto. Esta propiedad como usted ha podido estudiar, se utiliza para cambiar el color de fondo de un control. Para cada opción hemos especificado un color distinto mediante las constantes de color de Visual Basic, pero usted puede usar valores hexadecimales para representar los colores como lo hemos visto anteriormente.

En las últimas cinco opciones hemos utilizado la propiedad **ForeColor** de las cajas de texto. Dicha propiedad nos permite cambiar el color del texto de la caja. Al igual que las primeras cinco opciones cada opción tiene un color establecido que es indicado en la propiedad **Caption** de dicha opción.

- La siguiente aplicación realiza las cuatro operaciones básicas de matemática mediante cuatro botones de opción.



- Inserte primero dos etiquetas y dos cajas de texto, luego inserte un control **Frame** y dentro de ese **Frame** inserte cuatro botones de opción. Luego inserte una tercera etiqueta y una tercera caja de texto, tal y como se ve en la imagen.

- Aplique los siguientes valores en las propiedades indicadas de los controles que se muestran en la siguiente tabla:

Control	Propiedad	Valor
Label1	AutoSize	True
	Caption	Primer Valor:
Label2	AutoSize	True
	Caption	Segundo Valor:



Control	Propiedad	Valor
Text1	Name Text	txtPrimerValor
Text2	Name Text	txtSegundoValor
Frame1	Caption	Seleccione una operación
Opcion1	Caption	&Suma
Opcion2	Caption	&Resta
Opcion3	Caption	&Multiplicación
Opcion4	Caption	&División
Label3	AutoSize Caption	True Resultado:
Text3	Name Locked Text	txtResultado True
Form1	Caption	Operaciones Matemáticas

La apariencia de los controles sobre el formulario debe ser la siguiente:

Ahora codifiquemos las dos primeras cajas de texto para que solo admitan números.

- Dentro del evento **KeyPress** de ambas cajas de texto escriba el siguiente código:



```
'Hace que la caja de texto solo acepte números.
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then
KeyAscii = 0
End If
```

- Haga doble clic sobre el primer botón de opción y dentro del evento **Click** escriba:

```
'Verificamos que las cajas tengan valores.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
  MsgBox ("Debe introducir un valor en la primera caja.")
  txtPrimerValor.SetFocus      'Hace que la primera caja reciba el enfoque si esta vacía.
  Option1.Value = False        'Desactiva la opción suma.

Elseif Len(Trim(txtSegundoValor.Text)) = 0 Then
  MsgBox ("Debe introducir un valor en la segunda caja.")
  txtSegundoValor.SetFocus     'Hace que la segunda caja reciba el enfoque si esta vacía.
  Option1.Value = False        'Desactiva la opción suma.

Else

  'Suma las dos cajas de texto y muestra el resultado en la tercera caja.
  txtResultado.Text = Val(txtPrimerValor.Text) + Val(txtSegundoValor.Text)

End If
```

- Haga doble clic sobre el segundo botón de opción y dentro del evento **Click** escriba:

```
'Verificamos que las cajas tengan valores.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
  MsgBox ("Debe introducir un valor en la primera caja.")
  txtPrimerValor.SetFocus      'Hace que la primera caja reciba el enfoque si esta vacía.
  Option2.Value = False        'Desactiva la opción resta.

Elseif Len(Trim(txtSegundoValor.Text)) = 0 Then
  MsgBox ("Debe introducir un valor en la segunda caja.")
  txtSegundoValor.SetFocus     'Hace que la segunda caja reciba el enfoque si esta vacía.
  Option2.Value = False        'Desactiva la opción resta.

Else

  'Resta las dos cajas de texto y muestra el resultado en la tercera caja.
  txtResultado.Text = Val(txtPrimerValor.Text) - Val(txtSegundoValor.Text)

End If
```

- Haga doble clic sobre el tercer botón de opción y dentro del evento **Click** escriba:

```
'Verificamos que las cajas tengan valores.
If Len(Trim(txtPrimerValor.Text)) = 0 Then
  MsgBox ("Debe introducir un valor en la primera caja.")
  txtPrimerValor.SetFocus      'Hace que la primera caja reciba el enfoque si esta vacía.
  Option3.Value = False        'Desactiva la opción multiplicar.

Elseif Len(Trim(txtSegundoValor.Text)) = 0 Then
```



```
MsgBox ("Debe introducir un valor en la segunda caja.")
txtSegundoValor.SetFocus      'Hace que la segunda caja reciba el enfoque si esta vacía.
Option3.Value = False         'Desactiva la opción multiplicar.
```

Else

```
'Multiplica las dos cajas de texto y muestra el resultado en la tercera caja.
txtResultado.Text = Val(txtPrimerValor.Text) * Val(txtSegundoValor.Text)
```

End If

- Haga doble clic sobre el cuarto botón de opción y dentro del evento **Click** escriba:

'Verificamos primero si el divisor es distinto de cero. Recuerde no se puede dividir por cero.

```
If Val(txtSegundoValor.Text) > 0 Then
```

```
    'Verificamos que las cajas tengan valores.
```

```
    If Len(Trim(txtPrimerValor.Text)) = 0 Then
```

```
        MsgBox ("Debe introducir un valor en la primera caja.")
```

```
        txtPrimerValor.SetFocus      'Hace que la primera caja reciba el enfoque si esta vacía.
```

```
        Option4.Value = False         'Desactiva la opción dividir.
```

```
    ElseIf Len(Trim(txtSegundoValor.Text)) = 0 Then
```

```
        MsgBox ("Debe introducir un valor en la segunda caja.")
```

```
        txtSegundoValor.SetFocus      'Hace que la segunda caja reciba el enfoque si esta vacía.
```

```
        Option4.Value = False         'Desactiva la opción dividir.
```

Else

```
    'Divide las dos cajas de texto y muestra el resultado en la tercera caja.
```

```
    txtResultado.Text = Val(txtPrimerValor.Text) / Val(txtSegundoValor.Text)
```

End If

Else

```
MsgBox ("No se puede dividir por cero.")
```

```
txtSegundoValor.Text = ""
```

```
'Borra el cero para que el usuario escriba otro número.
```

```
txtSegundoValor.SetFocus      'Hace que la segunda caja reciba el enfoque.
```

End If

- Corra la aplicación y escriba un valor en las dos primeras cajas de texto.
- Utilice uno por uno los cuatro botones de opción para ver los resultados de cada operación matemática aplicada en ambos números.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio35-3** para el *formulario* y **Ejercicio35-3** para el proyecto.

El código anterior no requiere de mucha explicación, debido a que hemos visto anteriormente como realizar operaciones de cálculo matemáticos con las cajas de texto. La única diferencia, es que estamos utilizando botones de opción y no botones de comando para realizar los cálculos matemáticos. Recomiendo que analice detenidamente el código anterior para que tome en cuenta algunas reglas de validación y modificaciones aplicadas antes de efectuar operaciones de cálculo, principalmente en la división.



- 3.5.1 La cajas de verificación (CheckBox)



La única diferencia entre los botones de opción (OptionButton) y las cajas de verificación (CheckBox) es que dentro de un mismo contenedor se pueden seleccionar más de una opción. Otra diferencia entre los botones de opción y las cajas de verificación es que en estas no se programa muy a menudo el evento **Click**, sino, que mediante la sentencia **If** se verifica si la propiedad **Value** esta establecida a **1 – Checked**. Cuando una caja de verificación esta activada la propiedad **Value** toma el valor 1 (uno) y cuando esta desactivada toma el valor 0 (cero).

- 3.5.1.1 Propiedades de las cajas de verificación

Al igual que en los botones de opción, las propiedades más importantes de las cajas de verificación son: **Caption** (Título) y la propiedad **Value** (Valor) que indica si el elemento esta activado o desactivado.

- 3.5.1.2 Eventos sobre las cajas de verificación

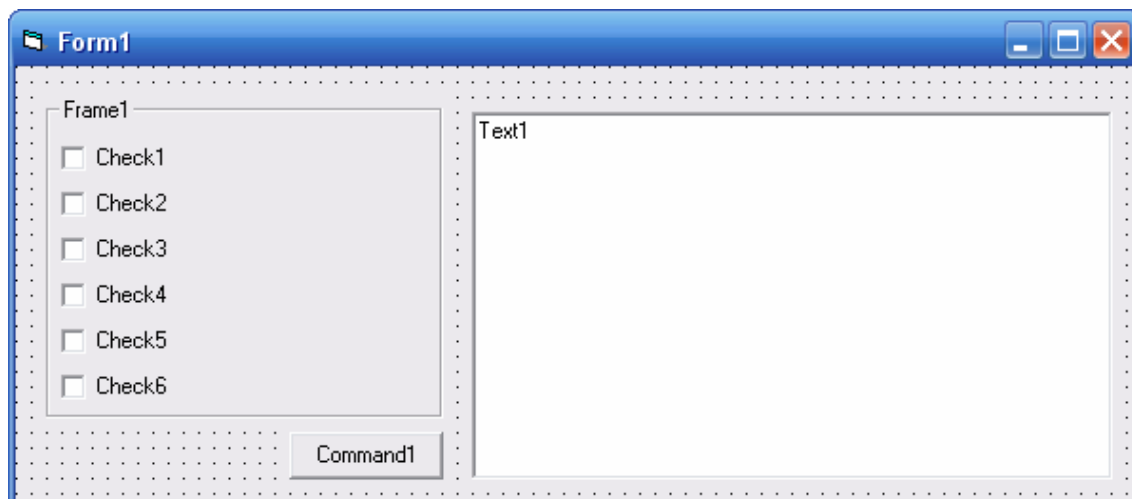
Las cajas de verificación no se diferencian en nada de los demás controles con respecto a los eventos estándar de los controles. Puede utilizar los eventos ya vistos anteriormente. Como se dijo anteriormente, en las cajas de verificación no se programa muy a menudo el evento **Click**, sino, que se verifica si la caja de verificación esta activada o desactivada mediante la propiedad **Value**. Si mediante la sentencia **If** usted verifica que esta activada, entonces, usted programa después de la cláusula **Then** y si no esta activada y desea realizar otra acción después de haber echo la comprobación, entonces, programe después de la cláusula **Else**.

- 3.5.1.3 Métodos de las cajas de verificación

Las cajas de verificación poseen todos los métodos estándar ya estudiados anteriormente (**Drag**, **Refresh**, **SetFocus**, **Move**, **Zorder**).

- 3.5.1.4 Ejercicios prácticos

- La siguiente aplicación permite seleccionar entre varios efectos que se le aplicarán a una caja de texto mediante botones de verificación después haberse hecho clic sobre un botón de comando.



Ing. Carlos Manuel Rodríguez Bucarely



Capítulo 3

- Abra un nuevo proyecto e inserte un control **Frame** y dentro de este seis cajas de verificación. Debajo de estas un botón de comando y a la derecha una caja de texto, tal y como se ve en la imagen de la página anterior.
- Aplique los siguientes valores en las propiedades indicada de los controles que se muestran en la siguiente tabla:

Control	Propiedad	Valor
Frame1	Caption	Efectos
Check1	Caption	Fondo azul, &texto blanco
Check2	Caption	&Letras grandes
Check3	Caption	&Cursiva
Check4	Caption	&Negrita
Check5	Caption	&Subrayado
Check6	Caption	T&achado
Command1	Caption	A&plicar
Text1	Name MultiLine ScrollBars Text	txtContenido True 2 – Vertical

- La apariencia de los controles sobre el formulario debe ser la siguiente:



- Ahora codifiquemos para que cada opción seleccionada aplique el efecto seleccionado a la caja de texto. Cada vez que el usuario haga seleccione un efecto deberá hacer clic sobre el botón Aplicar, por consiguiente, debemos codificar el evento **Click** del botón Aplicar.
- Haga doble Clic sobre el botón Aplicar y dentro del evento **Click** escriba:

Ing. Carlos Manuel Rodríguez Bucarely



```
'Verificamos si la primera caja de verificación esta activada.
If Check1.Value = 1 Then
    'Aplicamos el efecto.
    txtContenido.BackColor = vbBlue ' Fondo azul.
    txtContenido.ForeColor = vbWhite ' Texto blanco.
Else
    'Si no esta activada pone la caja en su estado normal:
    'Fondo blanco y texto negro.
    txtContenido.BackColor = vbWhite 'Fondo blanco.
    txtContenido.ForeColor = vbBlack 'Texto negro.
End If

'Verificamos si la segunda caja de verificación esta activada.
If Check2.Value = 1 Then
    'Aplicamos el efecto.
    txtContenido.FontSize = 18 'Letras grandes.
Else
    'Si esta desactivada pone el tamaño inicial de la caja.
    txtContenido.FontSize = 8 'Letras normal.
End If

'Verificamos si la tercera caja de verificación esta activada.
If Check3.Value = 1 Then
    'Aplicamos el efecto.
    txtContenido.FontItalic = True 'Pone cursiva.
Else
    'Si no esta activada desactiva la cursiva.
    txtContenido.FontItalic = False 'Quita cursiva.
End If

'Verificamos si la cuarta caja de verificación esta activada.
If Check4.Value = 1 Then
    'Aplicamos el efecto.
    txtContenido.FontBold = True 'Pone negrita.
Else
    'Si no esta activada desactiva la negrita.
    txtContenido.FontBold = False 'Quita negrita.
End If

'Verificamos si la quinta caja de verificación esta activada.
If Check5.Value = 1 Then
    'Aplicamos el efecto.
    txtContenido.FontUnderline = True 'Pone subrayado.
Else
    'Si no esta activada desactiva la negrita.
    txtContenido.FontUnderline = False 'Quita subrayado.
End If

'Verificamos si la sexta caja de verificación esta activada.
If Check6.Value = 1 Then
    'Aplicamos el efecto.
    txtContenido.FontStrikethru = True 'Pone tachado.
Else
    'Si no esta activada desactiva la negrita.
    txtContenido.FontStrikethru = False 'Quita tachado.
End If
```



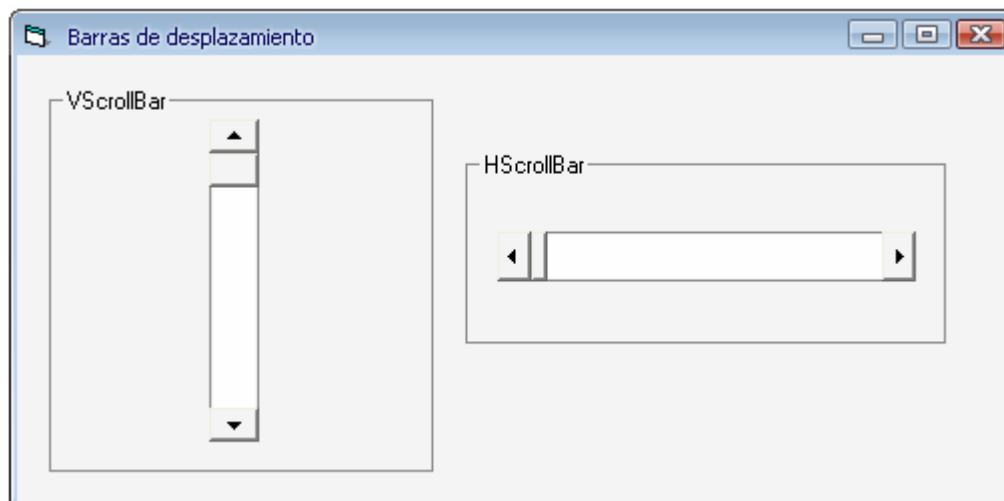
- Corra la aplicación.
- Escriba algo en la caja de texto.
- Active el efecto que quiera aplicar a la caja de texto y haga clic en el botón Aplicar. Podrá observar que la caja de texto toma el efecto seleccionado. Si desea quitar el efecto seleccionado, entonces, desactive la caja de verificación y, a continuación, haga clic en el botón Aplicar.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio36-3** para el *formulario* y **Ejercicio36-3** para el proyecto.

Explicación del código anterior

He agregado suficientes comentarios en el código anterior para que no requiera de mucha explicación. Usted puede ir interpretando paso a paso la función de cada línea de código escrita en el programa anterior. No obstante quiero resaltar algunos puntos importantes. Usted pudo haber notado que cuando aplicamos un efecto cualquiera también programamos para quitarlo. Esto es necesario para permitirle al usuario quitar un efecto que ya había aplicado a la caja de texto. Cada vez que utilice cajas de verificación tenga en cuenta que el usuario querrá en algún momento deshacer algún cambio realizado en la aplicación. Esto es válido para cualquier situación. Permítale siempre al usuario rehacer y deshacer una operación realizada por él, que implique algún cambio en la aplicación.

- 3.6.1 La barras de desplazamiento (ScrollBar)

Visual Basic posee 2 tipos de barras de desplazamiento o Scrollbar, una barra de tipo Vertical llamada **VScrollBar** y otro de tipo Horizontal llamada **HScrollBar**:



Ambas barras son exactamente iguales en lo que se refiere a las propiedades, métodos y eventos salvo su distinta orientación. La función principal de estos controles es la de devolver un valor para luego utilizarlo ya sea para una función o rutina que tengamos en nuestro programa.

- 3.6.1.1 Propiedades de las barras de desplazamiento

Las barras de desplazamiento poseen la mayoría de propiedades, métodos y eventos comunes de los controles. Pero al igual que en todos los controles solo algunas de estas son consideradas realmente importantes.



Capítulo 3

Las barras de desplazamiento poseen seis propiedades que son realmente importantes. Estas se describen a continuación:

Value	Es la propiedad más importante de las barras de desplazamiento. Esta Establece o devuelve el valor actual del control cuando movemos el cuadro deslizable de la barra de desplazamiento. Esta propiedad también cambia su valor cuando hacemos clic en las flechas arriba y abajo de la barra de desplazamiento.
Max	Devuelve o establece el valor máximo de la propiedad Value de la barra de desplazamiento cuando el cuadro deslizable se encuentra en el extremo inferior o derecho. El valor máximo para esta propiedad es 32767.
Min	Devuelve o establece el valor mínimo de la propiedad Value de la barra de desplazamiento cuando el cuadro deslizable se encuentra en el extremo superior o izquierdo.
LargeChange	Devuelve o establece el cambio que se producirá en el valor de la propiedad Value de la barra de desplazamiento cuando el usuario haga clic en el área situada entre el cuadro deslizable y la flecha de desplazamiento. El mínimo valor de esta propiedad es 1.
SmallChange	Devuelve o establece el cambio que se producirá en el valor de la propiedad Value de un control de barra de desplazamiento cuando el usuario haga clic en la flecha de desplazamiento. El mínimo valor de esta propiedad es 1.

- 3.6.1.2 Eventos sobre las barras de desplazamiento

Las barras de desplazamiento posee todos los eventos estándar de los controles pero solo dos de estos son los más importantes. Estos son los eventos **Change** y **Scroll**. El evento **Change** ocurre cuando el usuario mueve el cuadro deslizable o hace clic sobre las flechas de la barra de desplazamiento. Cuando este evento ocurre se modifica el valor de la propiedad **Value**.

El evento **Scroll** ocurre cuando el usuario mueve el cuadro deslizable de la barra de desplazamiento. Al igual que el evento **Change** el evento **Scroll** modifica el valor de la propiedad **Value**.

- 3.6.1.3 Ejercicios prácticos

En esta sección mostraremos un ejercicio práctico donde usted podrá apreciar el uso correcto de las propiedades y eventos de las barras de desplazamiento. Para este ejercicio hemos creado una aplicación que permite crear colores utilizando la función **RGB** de Visual Basic. Esta función permite crear colores haciendo combinaciones con los colores básicos *rojo*, *verde* y *azul*. Para crear un color con esta función solo se debe especificar como entrada un valor comprendido entre 0 y 255 para cada color de la función. El valor 0 indica la menor intensidad y 255 la máxima.

En este ejercicio hemos utilizado tres barras de desplazamiento para establecer en cada color un valor comprendido entre 0 y 255.

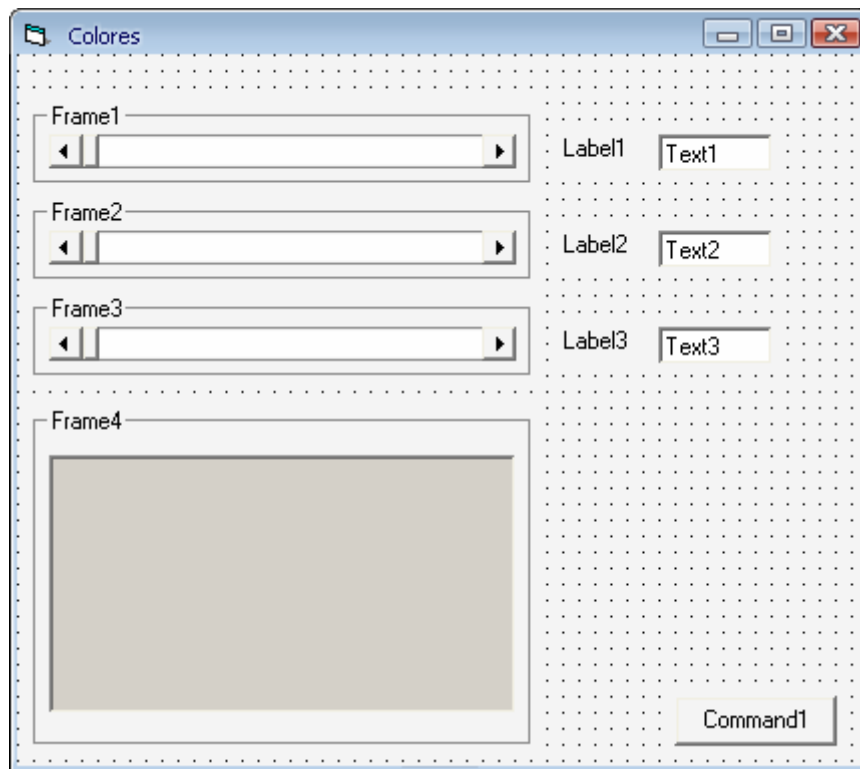
Para crear el ejercicio siga detalladamente los pasos que se le indican a continuación:

- Abra un nuevo proyecto.



Capítulo 3

- Inserte tres **Frame** y dentro de cada uno una barra de desplazamiento horizontal (HScrollBar). (Observe la figura).
- Inserte tres etiquetas y al lado de cada etiqueta una caja de texto.
- Inserte otro control **Frame** y dentro de este un **PictureBox**.
- Inserte un botón de comando en la esquina inferior izquierda del formulario.



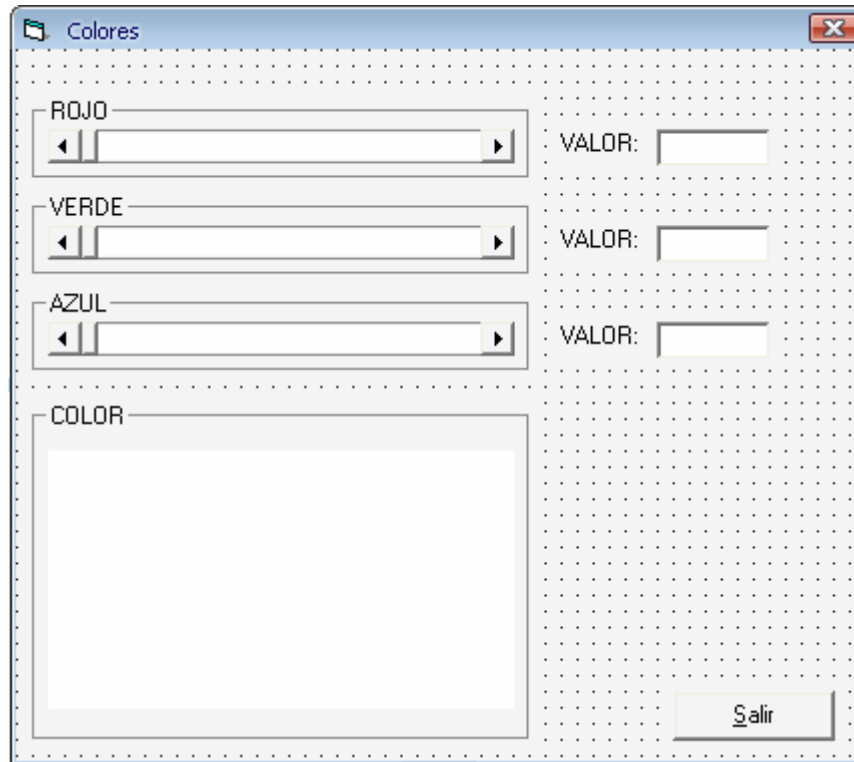
- Establezca los siguientes valores en las propiedades indicadas en cada control:

Control	Propiedad	Valor
Form	Caption BorderStyle StartPosition	Colores 1 – Fixed Single 2 - CenterScreen
Frame1	Caption	ROJO
Frame2	Caption	VERDE
Frame3	Caption	AZUL
HScroll1	Max	255
HScroll2	Max	255
HScroll3	Max	255
Label1	Caption	VALOR:
Label2	Caption	VALOR:
Label3	Caption	VALOR:

Control	Propiedad	Valor
Text1	Text	
Text2	Text	
Text3	Text	
Frame4	Caption	COLOR
Picture1	BorderStyle	0 – None
Command1	Caption	&Salir



- La apariencia de los controles sobre el formulario debe ser la siguiente:



- Haga doble clic sobre la primera barra de desplazamiento y dentro del evento **Change** escriba:

Dim Rojo, Verde, Azul

'Almacena el valor actual de cada barra en la variable correspondiente a cada color.

Rojo = HScroll1.Value

Verde = HScroll2.Value

Azul = HScroll3.Value

'Asigna el color al cuadro de imagen utilizando la función RGB y los valores de las barras.

Picture1.BackColor = RGB(Rojo, Verde, Azul)

'Establece el valor actual de cada barra en las cajas de texto.

Text1.Text = Rojo

Text2.Text = Verde

Text3.Text = Azul

- Haga doble clic sobre la segunda barra de desplazamiento y dentro del evento **Change** escriba:

Dim Rojo, Verde, Azul

'Almacena el valor actual de cada barra en la variable correspondiente a cada color.

Rojo = HScroll1.Value

Verde = HScroll2.Value

Azul = HScroll3.Value



Capítulo 3

'Asigna el color al cuadro de imagen utilizando la función RGB y los valores de las barras.

```
Picture1.BackColor = RGB(Rojo, Verde, Azul)
```

'Establece el valor actual de cada barra en las cajas de texto.

```
Text1.Text = Rojo
```

```
Text2.Text = Verde
```

```
Text3.Text = Azul
```

- Haga doble clic sobre la tercera barra de desplazamiento y dentro del evento **Change** escriba:

'Almacena el valor actual de cada barra en la variable correspondiente a cada color.

```
Dim Rojo, Verde, Azul
```

```
Rojo = HScroll1.Value
```

```
Verde = HScroll2.Value
```

```
Azul = HScroll3.Value
```

'Asigna el color al cuadro de imagen utilizando la función RGB y los valores de las barras.

```
Picture1.BackColor = RGB(Rojo, Verde, Azul)
```

'Establece el valor actual de cada barra en las cajas de texto.

```
Text1.Text = Rojo
```

```
Text2.Text = Verde
```

```
Text3.Text = Azul
```

- Haga doble clic sobre la primera barra de desplazamiento y, a continuación, seleccione el evento **Scroll**. Dentro de este evento escriba:

'Llama el evento Change de la primera barra de desplazamiento.

```
HScroll1_Change
```

- Haga doble clic sobre la segunda barra de desplazamiento y, a continuación, seleccione el evento **Scroll**. Dentro de este evento escriba:

'Llama el evento Change de la segunda barra de desplazamiento.

```
HScroll2_Change
```

- Haga doble clic sobre la tercera barra de desplazamiento y, a continuación, seleccione el evento **Scroll**. Dentro de este evento escriba:

'Llama el evento Change de la tercera barra de desplazamiento.

```
HScroll3_Change
```

- En el evento **Click** del botón Salir escriba:

'Sale de la aplicación.

```
End
```

- Corra la aplicación y mueva el cuadro deslizable de cada una de las barras de desplazamiento hacia la derecha. Cada vez que mueve uno de estos cuadros deslizable se crea un nuevo color y el valor actual de la barra de desplazamiento se asigna en las cajas de texto de la derecha.



- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio37-3** para el *formulario* y **Ejercicio37-3** para el proyecto.

Explicación:

La primera operación a explicar en la aplicación anterior es el valor 255 aplicado en la propiedad **Max** de cada barra de desplazamiento. Hemos asignado este valor, debido a que el valor máximo que puede aceptar la función **RGB** para cada color es 255. Recuerde que el valor máximo que alcanza una barra de desplazamiento cuando se mueve su cuadro deslizable hacia su límite es el que especificamos en la propiedad **Max**.

Lo segundo en explicar es el mismo bloque de código escrito dentro del evento **Change** de cada barra de desplazamiento.

Primera línea: Dim Rojo, Verde, Azul

En esta línea de código se declaran tres variables. Cada variable es utilizada para almacenar el valor actual de cada barra de desplazamiento cada vez que movamos el cuadro deslizable de cada una de ellas.

Segunda línea: Rojo = HScroll1.Value

Esta línea de código permite almacenar el valor actual de la primera barra de desplazamiento en la variable Rojo. Este valor servirá para especificarlo en la función **RGB**. El valor máximo que podrá almacenar esta variable es 255 porque es el límite para cada barra de desplazamiento.

Tercera línea: Verde = HScroll2.Value

Esta línea de código almacena el valor actual de la segunda barra de desplazamiento en la variable Verde.

Cuarta línea: Azul = HScroll3.Value

Esta línea de código almacena el valor actual de la tercera barra de desplazamiento en la variable Azul.

Quinta línea: Picture1.BackColor = RGB(Rojo, Verde, Azul)

Asigna el color creado mediante la función **RGB** en el cuadro de imagen. Cada una de las variables se especifica dentro de los paréntesis de la función **RGB**. Esas variables poseen el valor de la propiedad **Value** de cada barra. Esos valores se le pasan a la función **RGB** para crear un color combinando los tres colores básicos.

Sexta línea: Text1.Text = Rojo

Esta línea de código almacena en la primera caja de texto el valor actual de la variable Rojo, es decir, el valor actual de la primera barra de desplazamiento.

Séptima línea:

Esta línea de código almacena en la segunda caja de texto el valor actual de la variable Verde, es decir, el valor actual de la segunda barra de desplazamiento.

Octava línea:

Esta línea de código almacena en la tercera caja de texto el valor actual de la variable Azul.

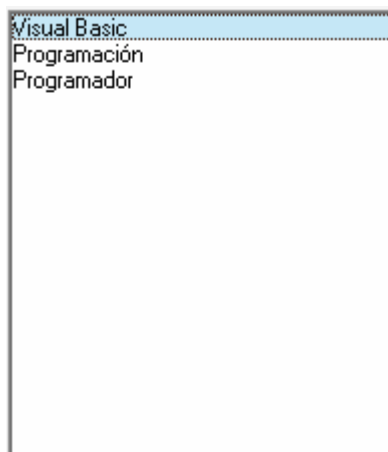


Capítulo 3

El bloque de código anterior lo hemos escrito en el evento **Change** de cada barra de desplazamiento para que cada vez que se mueva el cuadro deslizable de una de las barras se ejecute la función **RGB** y modifique el valor de las variables. De esta manera todas las barras de desplazamiento estarán conectadas.

La tercera operación a explicar es el código escrito dentro del evento **Scroll** de cada barra de desplazamiento. Se explico anteriormente, que el evento **Scroll** ocurre cuando el usuario esta moviendo el cuadro deslizable de una barra de desplazamiento. A diferencia del evento **Change** que ocurre cuando el usuario mueve y suelta el cuadro deslizable o cuando hace clic sobre las flechas. Dentro del evento **Scroll** de cada barra de desplazamiento hemos llamado el evento **Change** de la barra correspondiente para que a medida que se muevan los cuadros deslizables se combine y se muestre los colores.

- 3.7.1 Las cajas de lista (ListBox)



Una **ListBox** es un control en el que se pueden tomar varios registros de líneas, teniendo uno o varios de ellos seleccionado. Si en la lista hay más registros de los que se pueden mostrar al mismo tiempo, se añade automáticamente una barra de desplazamiento.

Para añadir o eliminar registros de la lista en modo de Ejecución se utilizan los métodos **AddItem** y **RemoveItem**. El contenido de una **ListBox** suele inicializarse desde el evento **Load** de los formularios, de tal manera, las listas obtienen sus elementos antes de que la aplicación cargue totalmente. También, puede especificar los elementos de la **ListBox** en modo de diseño mediante la propiedad **List** desde la ventana de propiedades.

- 3.7.1.1 Propiedades de las cajas de lista

Las cajas de listas poseen la mayoría de las propiedades comunes de los controles, pero además de estas poseen propiedades muy propias de ellas. A continuación, se detallan las propiedades más importantes de las cajas de lista.

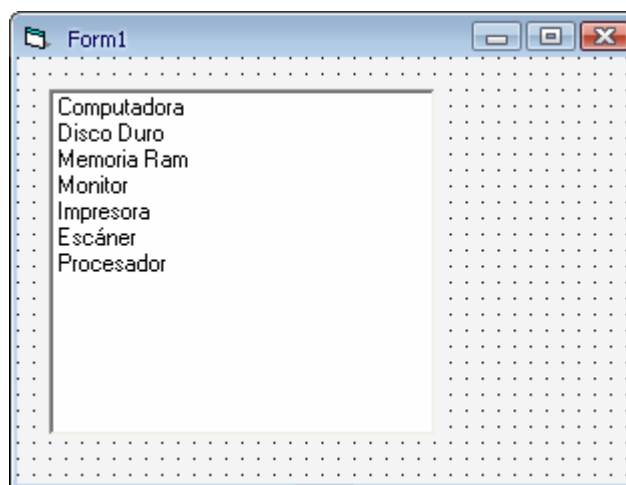
Propiedad	Descripción
List	Esta propiedad es utilizada para agregar elementos o registros al ListBox en modo de diseño. En modo de ejecución esta propiedad es utilizada para leer cada uno de los elementos del ListBox .
Ejercicio: - Abra un nuevo proyecto. - Inserte un ListBox { EMBED PBrush } desde la caja de herramientas. - Dibuje el ListBox casi del mismo alto del formulario. - Seleccione la propiedad List y haga clic en la flecha abajo. Aparecerá un pequeño recuadro donde usted tendrá que escribir uno por uno los elementos de la caja de lista. Cada vez que usted escriba un elemento deberá presionar la tecla ENTER. Al presionar la tecla ENTER el recuadro se esconde y, como consecuencia, tendrá que hacer clic nuevamente en el botón flecha abajo para agregar otro elemento.	



- Agregue los siguientes elementos:

Computadora
Disco Duro
Memoria Ram
Monitor
Impresora
Escáner
Procesador

- Su aplicación deberá tener una apariencia similar a la mostrada en la siguiente imagen:



MultiSelect	Devuelve o establece un valor que indica si el usuario podrá seleccionar más de un elemento en el ListBox. Puede tomar los siguientes valores: 0 – None: Valor predeterminado. No permite seleccionar más de un elemento. 1 – Simple: Selección múltiple simple. Al hacer clic con el Mouse o presionar la BARRA ESPACIADORA se selecciona o se desactiva un elemento de la lista. (Las teclas de dirección desplazan el enfoque.) 2 – Extended: Selección múltiple extendida. Al presionar MAYÚS y hacer clic, o al presionar MAYÚS y una de las teclas de dirección (FLECHA ARRIBA, FLECHA ABAJO, FLECHA IZQUIERDA o FLECHA DERECHA) se extiende la selección desde el elemento seleccionado anteriormente hasta el elemento actual. Al presionar CTRL y hacer clic con el Mouse se selecciona o desactiva un elemento de la lista.
Sorted	Establece un valor que indica si los elementos de un ListBox aparecen ordenados alfabéticamente. Puede tomar los valores True o False (<i>Valor por defecto</i>). Cuando es True los elementos aparecerán ordenados alfabéticamente, y solo lo podrá apreciar los resultados en modo de ejecución. Cuando es False los elementos se muestran tal y como se introducen en el ListBox. NOTA: Al establecer el valor True en la propiedad Sorted posiblemente se cambie el orden en que introduce los elementos.



Style	Devuelve o establece un valor que indica si los elementos de la ListBox poseen casillas de verificación. Los valores admitidos por esta propiedad son: 0 – Standard: Valor predeterminado. Los elementos de la ListBox se muestran de forma de lista de texto. 1 – CheckBox: Casilla de verificación. El control ListBox se muestra con una marca de verificación junto a cada elemento de texto. Es posible seleccionar múltiples elementos del ListBox si activa la casilla de verificación que hay junto a ellos. Observe el gráfico: { EMBED PBrush }
--------------	---

- 3.7.1.2 Eventos sobre las cajas de lista

Las cajas de lista poseen los eventos comunes de los controles pero solo uno de ellos es el más utilizado. El evento **Click** es el que comúnmente programaremos en una **ListBox** para determinar algún elemento que halla sido seleccionado. También, podrá determinar los elementos seleccionados en una **ListBox** desde cualquier otro control de la aplicación.

- 3.7.1.3 Métodos de las cajas de lista

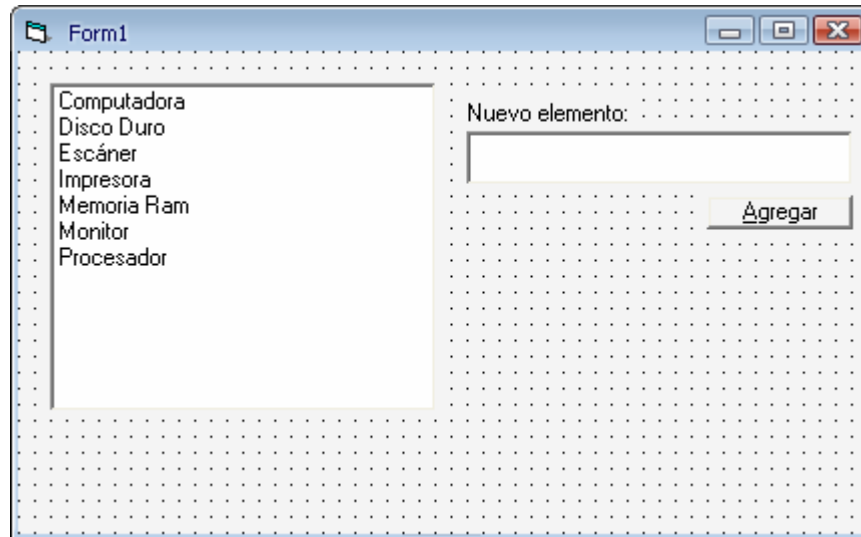
A continuación, se muestra una tabla donde se listan los métodos más importantes de una **ListBox**:

AddItem	Uno de los métodos más importante de una ListBox. Este es utilizado para agregar registros o elementos a una ListBox en <i>modo de ejecución</i>. Su formato es: Objeto.AddItem <i>Elemento</i>, [<i>Posición</i>] Donde: <i>Objeto:</i> Requerido. Indica el nombre de la ListBox, es decir, el valor indicado en la propiedad Name. <i>Elemento:</i> Requerido. Una expresión de cadena que especifica el elemento que se va a agregar al objeto. <i>Posición:</i> Opcional. Un entero que especifica la posición dentro del objeto donde se insertan el elemento o la fila nuevos.
----------------	---



Ejercicio:

- Agregue una etiqueta y debajo una caja de texto en el formulario de nuestro primer ejercicio. Debajo de esta coloque un botón de comando, tal y como se muestra en la imagen:



- Dentro del botón *Agregar* escriba:

```

'Verificamos que la caja de texto tenga información. Si no tiene, no se agrega el elemento.
If Len(Trim(Text1.Text)) > 0 Then
    List1.AddItem Text1.Text 'Agrega el contenido de la caja de texto a la ListBox.
    Text1.Text = ""          'Limpiamos la caja.
End If

Text1.SetFocus              'Hace que la caja reciba el enfoque.
    
```

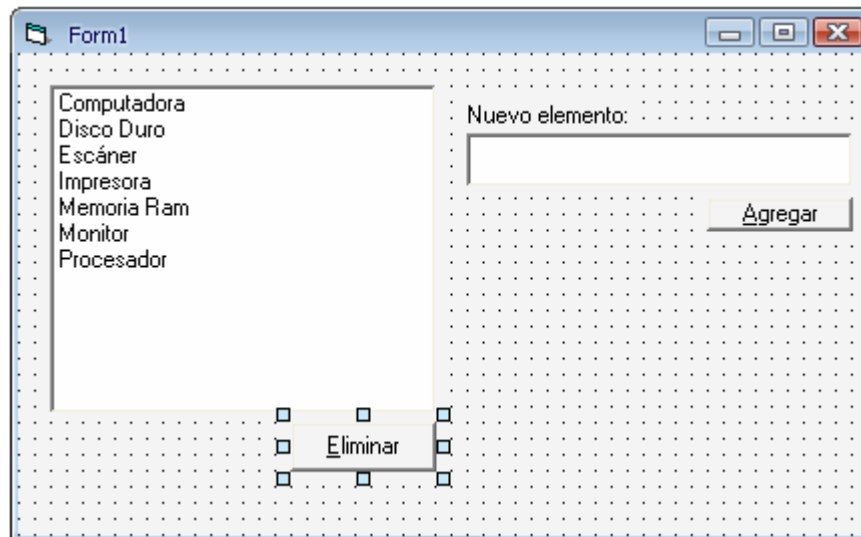
- Corra la aplicación.
- Escriba algo en la caja de texto y, a continuación, haga clic en el botón *Agregar*. Podrá Observar que el texto escrito en la caja de texto se agrega a la **ListBox** mediante el método **AddItem**.

RemoveItem	Este método es utilizado para eliminar un elemento o registro del ListBox. Su formato es: Objeto.RemoveItem (<i>Posición</i>) Donde: <i>Objeto:</i> Requerido. Una cadena de caracteres que indica el nombre del control ListBox. <i>Posición:</i> Requerido. Un valor numérico que indica la posición del elemento que se desea borrar. La posición del primer elemento de un control ListBox esta representada por el valor 0. Podrá utilizar el método ListIndex para determinar la posición del elemento seleccionado.
-------------------	---



Ejercicio:

- Agregue un botón de comando en la esquina inferior derecha del control **ListBox**, tal y como se muestra en la imagen:



- En la propiedad **Caption** del botón escriba **&Eliminar**.
- Haga doble clic sobre el botón de comando y en el evento **Click** escriba:

```
On Error Resume Next      'En caso de no haber ningún elemento seleccionado.
'Elimina el elemento seleccionado.
List1.RemoveItem List1.ListIndex
```

- Corra la aplicación.
- Seleccione un elemento de la **ListBox** y, a continuación, haga clic en el botón *Eliminar*. Podrá notar que el elemento seleccionado se elimina al hacer clic en el botón eliminar. Esto es posible, debido a que el método **ListIndex** devuelve un valor numérico que indica la posición del elemento seleccionado. Luego, ese valor se le pasa al método **RemoveItem** para que elimine dicho elemento.
- Detenga la aplicación.

Clear	Este método borra todos los elementos del control ListBox. Su formato es bastante sencillo, solo se debe especificar el nombre del control ListBox seguido del método Clear. Ejemplo: <pre>List1.Clear 'Elimina todo el contenido del ListBox.</pre>
ListCount	Devuelve un valor numérico que indica la cantidad de elemento que contiene la ListBox. Este método es muy utilizado para leer todo el contenido de un control ListBox. Para esto se debe combinar con un bucle for o while. Su formato es similar al del método anterior: <code>List1.ListCount</code>.

**Ejemplo 1:**

'Muestra en un cuadro de mensaje la cantidad de elementos de una ListBox.

```
MsgBox(List1.ListCount)
```

Ejemplo 2:

'Pasa todos los elementos de una ListBox a otra ListBox.

```
Dim i As Long
```

'Inicia un bucle que se repite según la cantidad de elementos de la ListBox.

```
For i = 0 To List1.ListCount - 1 'Restamos 1 porque una ListBox comienza desde 0.
```

```
List2.AddItem List1.List(i) 'Agregamos cada elemento de la primera Lista a la segunda Lista.
```

```
Next i
```

ListIndex	Devuelve un valor numérico que indica la posición del elemento seleccionado dentro del control ListBox. Ejemplo 1: 'Muestra en un cuadro de mensaje la posición del elemento seleccionado. MsgBox(List1.ListIndex) Ejemplo 2: 'Muestra en un cuadro de mensaje el texto del elemento seleccionado. MsgBox(List1.List(List1.ListIndex))
SelCount	Devuelve un valor numérico que indica la cantidad de elementos seleccionados en una ListBox. Esta propiedad es muy útil cuando un control ListBox permite seleccionar varios elementos a la vez. El valor devuelto por esta función se puede utilizar conjuntamente con un bucle para leer únicamente la cantidad de elementos seleccionados en una ListBox. Su formato es similar al del método ListCount.
Selected	Devuelve o establece un valor lógico (True o False) que indica si un elemento está o estará seleccionado. Su formato es: Objeto.Selected(<i>Indice</i>) = [True/False] Donde: <i>Objeto</i>: Es el nombre de la ListBox. <i>Indice</i>: Indica la posición del elemento en la ListBox. <i>True/False</i>: Valores que devuelve el método. True indica que elemento está seleccionado. False indica que elemento no está seleccionado.

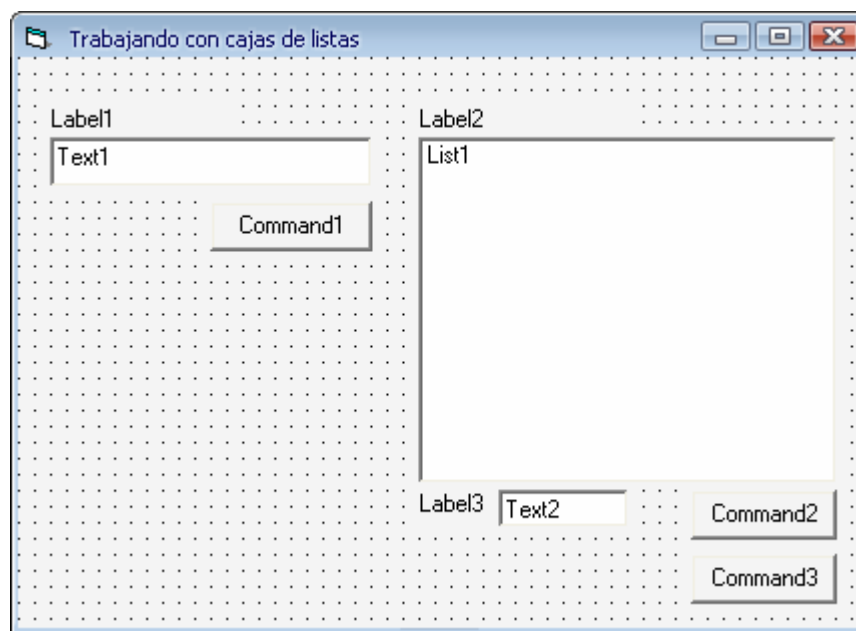


- Guarde nuestro primer ejercicio con los nombres **FormEjercicio38-3** para el *formulario* y **Ejercicio38-3** para el proyecto.

- 3.7.1.4 Ejercicios prácticos

1.- Nuestra primera aplicación muestra como agregar nombre de países a una **ListBox**. También permite eliminar un país seleccionado.

- Abra un nuevo proyecto.
- Inserte los controles que se muestran en la siguiente imagen:



- Aplice los siguientes valores a las propiedades de los controles indicado en la siguiente tabla:

Control	Propiedad	Valor
Form	Caption StartPosition	Trabajando con cajas de listas 2 – CenterScreen
Label1	AutoSize Caption Font	True Introduzca el país: Arial, Normal, 10.
Text1	Name Text	txtPais
Command1	Name Caption	cmdAgregarPais &Agregar
Label2	AutoSize Caption	True Lista de países:
List1	Name	lstPaises
Label3	AutoSize Caption	True Total:
Text2	Name Text	txtTotalPaises



Control	Propiedad	Valor
Command2	Name Caption	cmdEliminar &Eliminar
Command3	Name Caption	cmdBorrarLista &Borrar lista

- Dentro del evento **Click** del botón *Agregar* escriba:

```
'Verifica si la caja de texto se deja vacía.
If Len(Trim(txtPais.Text)) = 0 Then
    MsgBox ("Introduzca el nombre de un país en la caja.")
    txtPais.SetFocus
Else
    'Si la caja tiene algún país.
    lstPaises.AddItem txtPais.Text 'Agregue el país.

    txtPais.Text = ""           'Limpia la caja de texto.
    txtPais.SetFocus           'Hace que la caja reciba el enfoque.

    'Pone el número de países agregados en la caja de lista.
    txtTotalPaises.Text = lstPaises.ListCount

End If
```

- Dentro del evento **Click** del botón *Eliminar* escriba:

```
'En caso de que ocurra un error al eliminar un elemento de la lista.
On Error Resume Next

lstPaises.RemoveItem (lstPaises.ListIndex)    'Borra el elemento seleccionado.

txtTotalPaises.Text = lstPaises.ListCount      'Actualiza el total de países.
```

- Dentro del evento **Clic** del botón *Borrar Lista* escriba:

```
'Elimina todo el contenido del ListBox.
lstPaises.Clear

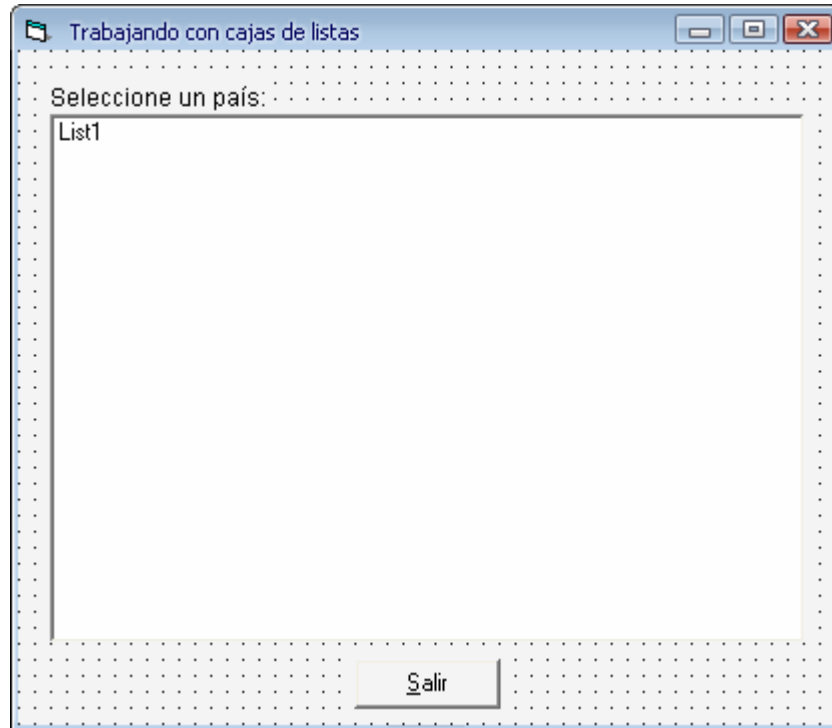
txtTotalPaises.Text = lstPaises.ListCount    'Actualiza el total de países.
```

- Corra la aplicación.
- En la caja de texto de la aplicación introduzca algún país, y a continuación, haga clic en el botón *Agregar*. Agregue todos los países que desee y podrá observar que todos los países se agregan al control **ListBox**.
- Seleccione algunos de los países ya agregados y luego haga clic en el botón *Eliminar*.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio39-3** para el *formulario* y **Ejercicio39-3** para el proyecto.



Capítulo 3

2.-) Nuestra segunda aplicación muestra en un cuadro de dialogo la capital de un país seleccionado en un control **ListBox**.



- Abra un nuevo proyecto.
- Inserte una etiqueta en la parte superior del formulario. (*Observe la imagen*).
- Debajo de la etiqueta coloque un control **ListBox**.
- Coloque un botón de comando debajo del control **ListBox**.
- Aplique los siguientes valores a las propiedades de los controles indicado en la siguiente tabla:

Control	Propiedad	Valor
Form	Caption StartPosition	Trabajando con cajas de listas 2 - CenterScreen
Label1	AutoSize Caption Font	True Seleccione un país Arial, Normal, 10.
List1	Name	IstPaíses
Command1	Name Caption	cmdSalir &Salir

- Dentro del evento **Click** del botón *Salir* escriba:

```
'Cierra la aplicación.  
End
```

- Dentro del evento **Load** del formulario escriba:

**'Agrega los países al control**

```
IstPaises.AddItem "República Dominicana"  
IstPaises.AddItem "Perú"  
IstPaises.AddItem "Salvador"  
IstPaises.AddItem "México"  
IstPaises.AddItem "Puerto Rico"  
IstPaises.AddItem "Ecuador"
```

- Dentro del evento **Click** del control **ListBox** escriba:

'Verifica el país seleccionado y muestra la capital en un cuadro de mensaje.

```
If UCase(IstPaises.List(IstPaises.ListIndex)) = UCase("República Dominicana") Then  
    MsgBox ("Santo Domingo")  
Elseif UCase(IstPaises.List(IstPaises.ListIndex)) = UCase("Perú") Then  
    MsgBox ("Lima")  
Elseif UCase(IstPaises.List(IstPaises.ListIndex)) = UCase("Salvador") Then  
    MsgBox ("San Salvador")  
Elseif UCase(IstPaises.List(IstPaises.ListIndex)) = UCase("México") Then  
    MsgBox ("México")  
Elseif UCase(IstPaises.List(IstPaises.ListIndex)) = UCase("Puerto Rico") Then  
    MsgBox ("San Juan")  
Elseif UCase(IstPaises.List(IstPaises.ListIndex)) = UCase("Ecuador") Then  
    MsgBox ("Quito")  
End If
```

- Corra la aplicación.
- Seleccione cualquier país de la lista para que el programa muestre su capital en una ventana de diálogo.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio40-3** para el *formulario* y **Ejercicio40-3** para el proyecto.

- 3.7.1.5 Ejercicios propuestos

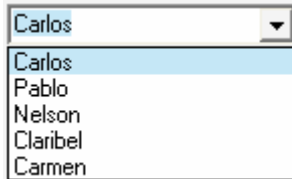
- 1.- Crear una aplicación que permita agregar y eliminar libros en un control **ListBox** mediante una caja de texto. El programa debe tener una *etiqueta* donde muestre la cantidad de libros que contiene el **ListBox**.
- 2.- Crear una aplicación que contenga dos *controles* **ListBox** que permitan cambiar el color de fondo y el color de letra de una caja de texto que se encuentre sobre un formulario. Uno de los **ListBox** debe tener la lista de los colores de fondo que se le aplicará a la *caja de texto* y el otro control **ListBox** los colores para la letra de la *caja de texto*.
- 3.- Crear una aplicación que en un control **ListBox** contenga veinte números cuales quiera. Cuando un número de lo de la lista sea seleccionado debe mostrarse ese número en una ventana de dialogo.



4.- Crear una aplicación que permita agregar y eliminar nombres de personas en un control **ListBox** y que permita organizarlos alfabéticamente.

5.- Crear una aplicación que mediante una lista de colores en un control **ListBox** permita cambiar el color de la ventana de la aplicación.

- 3.8.1 Las cajas combinadas (ComboBox)



Un **ComboBox** no tiene muchas diferencias en relación con un control **ListBox**. La diferencia que existe entre ambos *controles* es que un **ComboBox** oculta la lista de elementos y solo se muestra cuando se hace clic en el *botón flecha abajo* [▼] que contiene el *control*, mientras que el *control* **ListBox** muestra la lista de elementos sin ocultarla.

Otra de la diferencia principal es que un **ComboBox** tiene una propiedad llamada **Style**, que puede adoptar tres valores (1, 2 ó 3) que corresponden con tres distintas formas de presentar una lista:

1.) **0- DropDownCombo**: Éste es el valor más habitual y corresponde con el caso en el que sólo se muestra el registro seleccionado, que es editable por el usuario, permaneciendo el resto de los elementos oculto hasta que el usuario despliega la lista completa haciendo clic sobre *el botón flecha abajo* [▼].

2.) **1- Simple Combo**: En este caso el registro seleccionado también es editable, y se muestra una lista no desplegable dotada si es necesario de una **ScrollBar**.

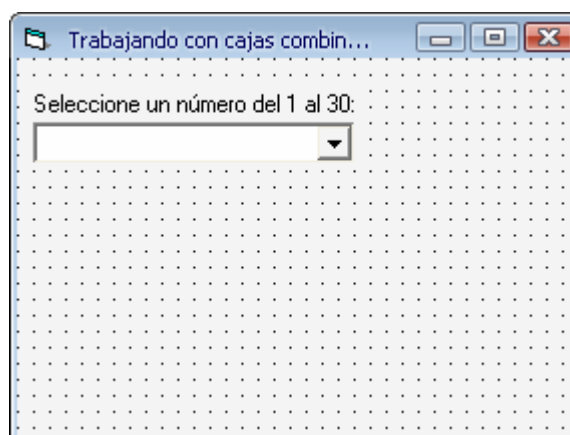
3.) **2- DropDown List**: En este último caso el registro seleccionado no es editable y la lista es desplegable.

A pesar de estas dos grandes diferencias, existen muchas relaciones con respecto a los métodos y las propiedades entre ambos *controles*. Por ejemplo los *métodos* **AddItem**, **RemoveItem** o **Clear** y las *propiedades* **List**, **ListIndex** o **ListCount**.

La propiedad **Text** corresponde con lo que aparece en el *área de edición* del **ComboBox** que es por lo general el primer elemento de la lista desplegable.

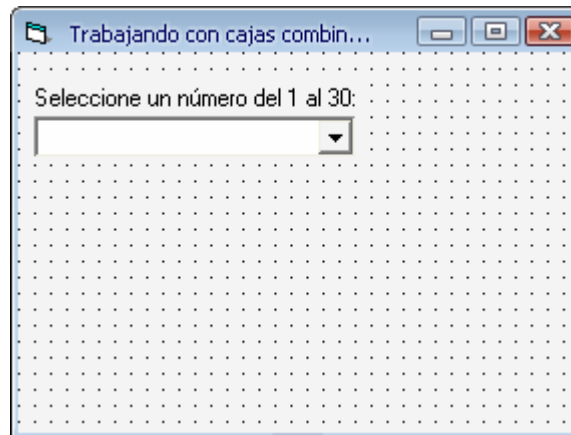
- 3.8.1.1 Ejercicios prácticos

▪ A continuación se muestra una Aplicación que muestra los número del 1 a 30 en un *control* **ComboBox**.





- Abra un nuevo proyecto.
- Inserte una etiqueta y debajo de la etiqueta una caja combinada , tal y como se muestra en la siguiente imagen:



- Dentro del evento **Load** del formulario escriba:

```
'Muestra los números 1 al 30 en la caja combinada.  
Dim i As Integer  
  
For i = 1 To 30 'Inicia un bucle del 1 al 30.  
    Combo1.AddItem i 'Agrega el número actual al ComboBox.  
Next i
```

- Corra la aplicación.
- Haga clic en el botón con la fecha abajo. Podrá observar que los números del 1 al 30 se han agregado en la caja combinada.

Al seleccionar uno de los números este se coloca en el área de edición de la caja combinada. Usted podrá verificar o leer el valor escrito en el área de edición mediante la propiedad **Text** de la caja combinada.

- Detenga la aplicación y guárdela con los nombres **FormEjercicio41-3** para el *formulario* y **Ejercicio41-3** para el proyecto.

- 3.9.1 El control tiempo (Timer)

Un control **Timer** puede ejecutar código a intervalos periódicos produciendo un *evento Timer*, que ocurre cuando ha transcurrido un *Intervalo* preestablecido para un control **Timer**. La frecuencia del intervalo se almacena en la propiedad **Interval** del control que especifica el tiempo en milisegundos.

- 3.9.1.1 Propiedades del control tiempo

De todas las propiedades de un *control tiempo* las más importantes son **Enabled** para habilitar o deshabilitar el *control* y la propiedad **Interval** para especificar el intervalo de tiempo en que el control realizará el evento.

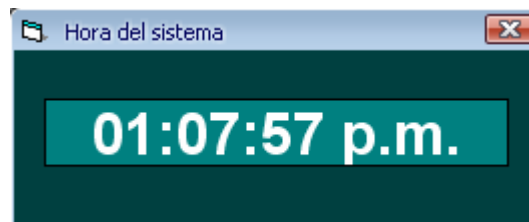


Capítulo 3

Para representar los segundos en *milisegundos* solo debe multiplicar la cantidad de segundos por mil. Por ejemplo, 2 segundos sería $2 \times 1000 = 2000$ *milisegundos*, un minuto sería $60 \times 1000 = 60000$ *milisegundos*, así sucesivamente.

- 3.9.1.2 Ejercicios prácticos

1.- El siguiente ejercicio consiste en una aplicación que muestra un reloj digital en una ventana.



- Abra un nuevo proyecto.
- Reduzca el tamaño del formulario similar al tamaño mostrado en la imagen anterior.
- Inserte una etiqueta y dibújela casi del tamaño del formulario. (*Observe la imagen*).
- Inserte un control **Timer**  en cualquier parte del formulario.
- Aplique los siguientes valores a las propiedades de los controles indicado en la siguiente tabla:

Control	Propiedad	Valor
Form	Caption StartPosition BorderStyle	Reloj digital 2-CenterScreen 1 – Fixed Single
Label1	Caption BackColor BorderStyle Font ForeColor	00:00:00 { EMBED PBrush } 1 – Fixed Single Arial, Negrita, 22. { EMBED PBrush } Blanco.
Timer1	Interval	1000

Ponemos la hora en la etiqueta.

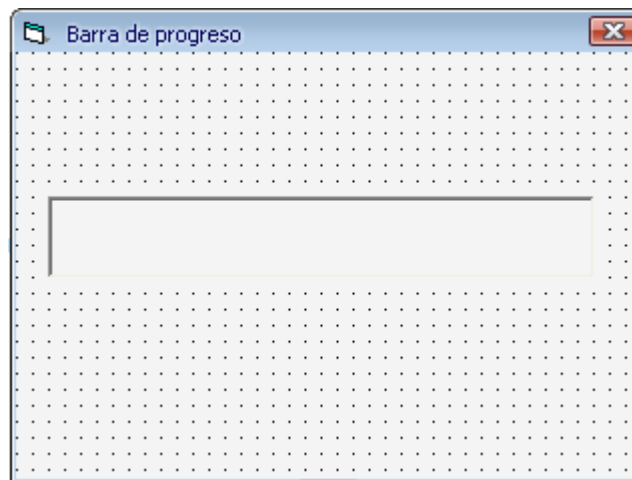
```
Label1.Caption = Format(Time, "HH:MM:SS AMPM")
```

- Corra la aplicación. Podrá ver que la hora del sistema puesta en la etiqueta se actualiza cada segundo. Esto es, porque el control se ejecuta cada 1000 milisegundos, es decir, cada segundo.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio42-3** para el *formulario* y **Ejercicio42-3** para el proyecto.

2.- En nuestra segunda aplicación simularemos una barra de progreso utilizando un control **Timer**.

- Abra un nuevo proyecto.
- Inserte un control **Picture**  sobre el formulario, tal y como se muestra en la imagen de la

Ing. Carlos Manuel Rodríguez Bucarely



- Dentro del control **Picture** dibuje otro control **Picture**. Tenga en cuenta que debe de dibujarlo dentro del primer control **Picture**. No se preocupe por el tamaño ni la posición del segundo control **Picture** la disposición de este control la especificamos en la siguiente tabla:

Control	Propiedad	Valor
Form	Caption	Barra de progreso
	BorderStyle	1 – Fixex Single
	StartPosition	2 – CenterScreen
Picture1	Appearance	0 – Flat
	Height	615
	Width	4095
Picture2	Appearance	0 - Flat
	BorderStyle	0 – None
	Height	615
	Width	15

- Inserte un control **Timer** en el formulario. En la propiedad **Interval** escriba 300.
- Haga doble clic sobre el control **Timer** y escriba:

```

'Verifica si la anchura del primer Picture es menor que la anchura de su contenedor. Si es menor se
'incrementa.
If Picture2.Width < Picture1.Width Then
    Picture2.Width = Picture2.Width + 100    'Incrementa el segundo Picture.
Else
    'Desactiva el tiempo cuando el primer Picture alcanza la anchura de su contenedor.
    Timer1.Enabled = False
End If

```

- Corra la aplicación. Podrá observar que la barra se incrementa hasta cubrir todo el espacio de su contenedor (Picture1).
- Detenga la aplicación y guárdela con los nombres **FormEjercicio43-3** para el *formulario* y **Ejercicio43-3** para el proyecto.



- 3.10.1 Controles relacionados con ficheros (FileList, DirList y DriveList)

Visual Basic proporciona una serie de controles especializados en el acceso al sistema de archivos de Windows. Estos controles son el **Cuadro de lista de unidades**, el **Cuadro de lista de directorios** y el **Cuadro de lista de archivos**. Cada uno de estos controles permite acceder a los correspondientes elementos del sistema de archivos.



Al **cuadro de lista de unidades (DriveList)** muestra cualquier unidad que tengas en tu ordenador. En tiempo de diseño se muestra la etiqueta de la unidad del disco duro de forma que puedas ajustar el control al tamaño de la misma. Este control incluye las unidades de disco duro, disco flexible, Cd-Rom, etc.

Este *control* tiene la propiedad llamada **Drive** que recoge la unidad seleccionada por el usuario (puede ser una unidad física como el disco **c:** o una unidad lógica asignada por el usuario a otro disco o directorio en un servidor o en otro ordenador de la red.)



El **cuadro de lista de directorios (DirList)** nos permite mostrar los directorios del sistema de archivos del ordenador. Es conveniente que este control muestre tres o cuatro carpetas o directorios. En tiempo de diseño muestra la carpeta en la que se inicia la aplicación y en la que por defecto se guarda el proyecto. Este *control* posee la propiedad **Path** que determina y asigna la unidad que se mostrarán en dicha caja.



El **cuadro de lista de archivos (FileList)** nos muestra los archivos de un determinado directorio o carpeta. Su propiedad más interesante es **Pattern** que nos permite especificar qué tipo de archivos son mostrados en dicho control. Para utilizar esta propiedad se pueden utilizar los comodines * y ? al establecer la propiedad. Estos caracteres tienen el mismo significado que en MS-DOS o Windows para especificar los nombres de los archivos. Si estableces la propiedad **Pattern** con la cadena *.txt, estás indicando que se muestren sólo los archivos que tengan esta extensión. Se pueden mostrar más de un tipo de archivos separándolos con punto y coma (;).

- 3.10.1.1 Conectar los controles de ficheros

En tiempo de diseño, al dibujar los distintos controles del sistema de archivos, estos muestran la unidad y el directorio en la que se crea el proyecto como he comentado anteriormente. En tiempo de ejecución el usuario puede cambiar la unidad y el directorio o carpeta y esta situación no se verá reflejada si no se escribe código. Para que los controles estén sincronizados es necesario conectarlos.

El evento predeterminado del control cuadro de lista de unidades es **Change**. Este evento sucede cuando el usuario despliega la lista de unidades y selecciona una unidad distinta a la actual, por lo que es el evento adecuado para actualizar la lista de directorios de la siguiente forma:

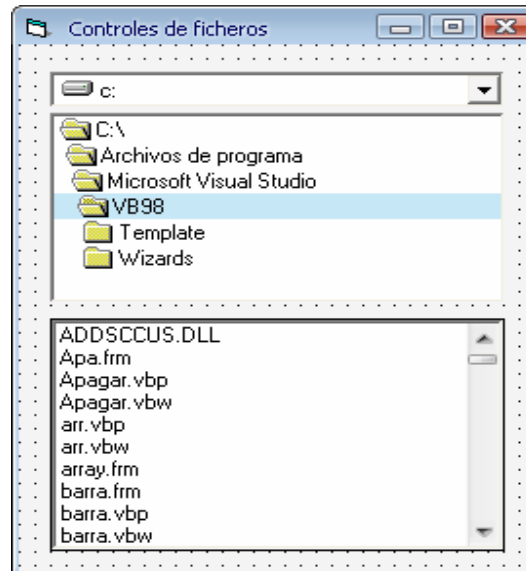
```
Private Sub Dir1_Change ( )  
    Dir1.Path = Drive1.Drive  
End Sub
```

Para el control cuadro de lista de directorios deberemos hacer algo parecido, el código será el siguiente:

```
Private Sub Dir1_Change ( )  
    File1.Path = Dir1.Path  
End Sub
```

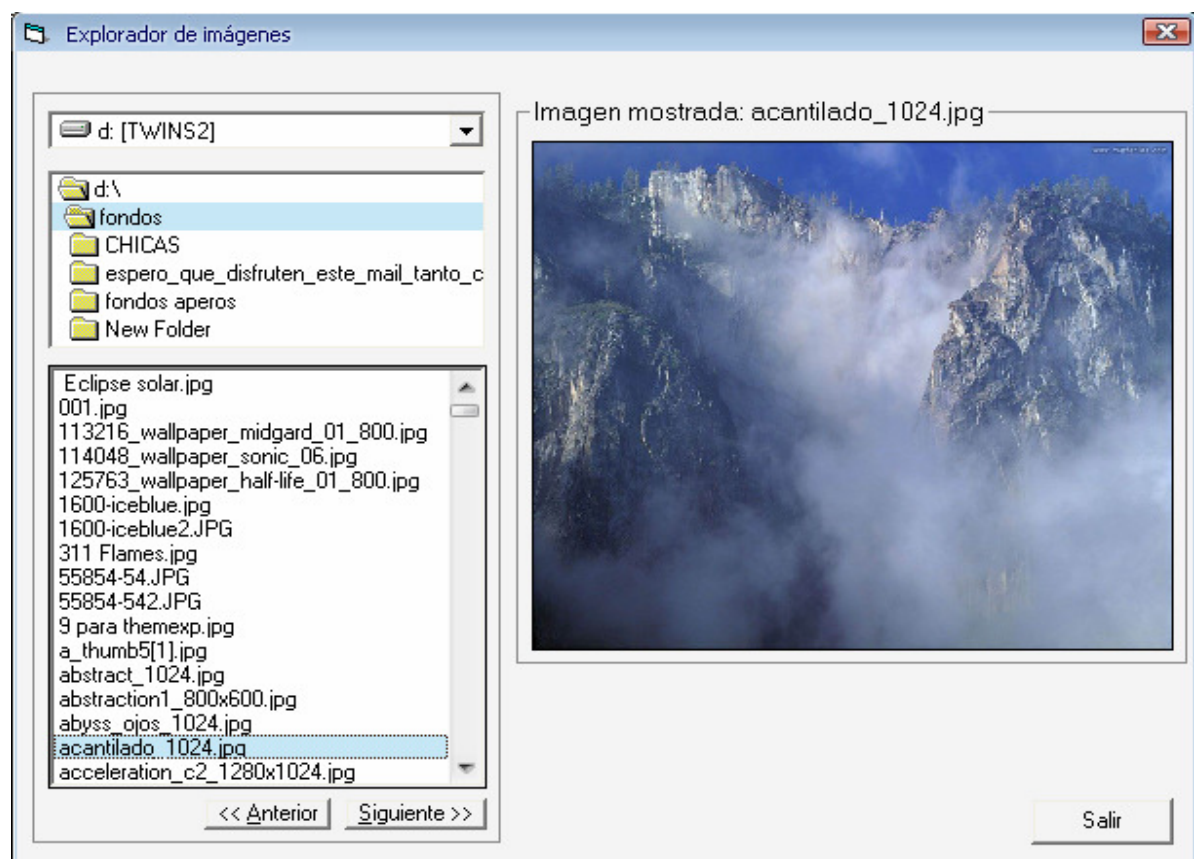


De esta forma tenemos conectados los tres controles de acceso al sistema de archivos. A continuación, se muestra una imagen con los tres controles relacionados:



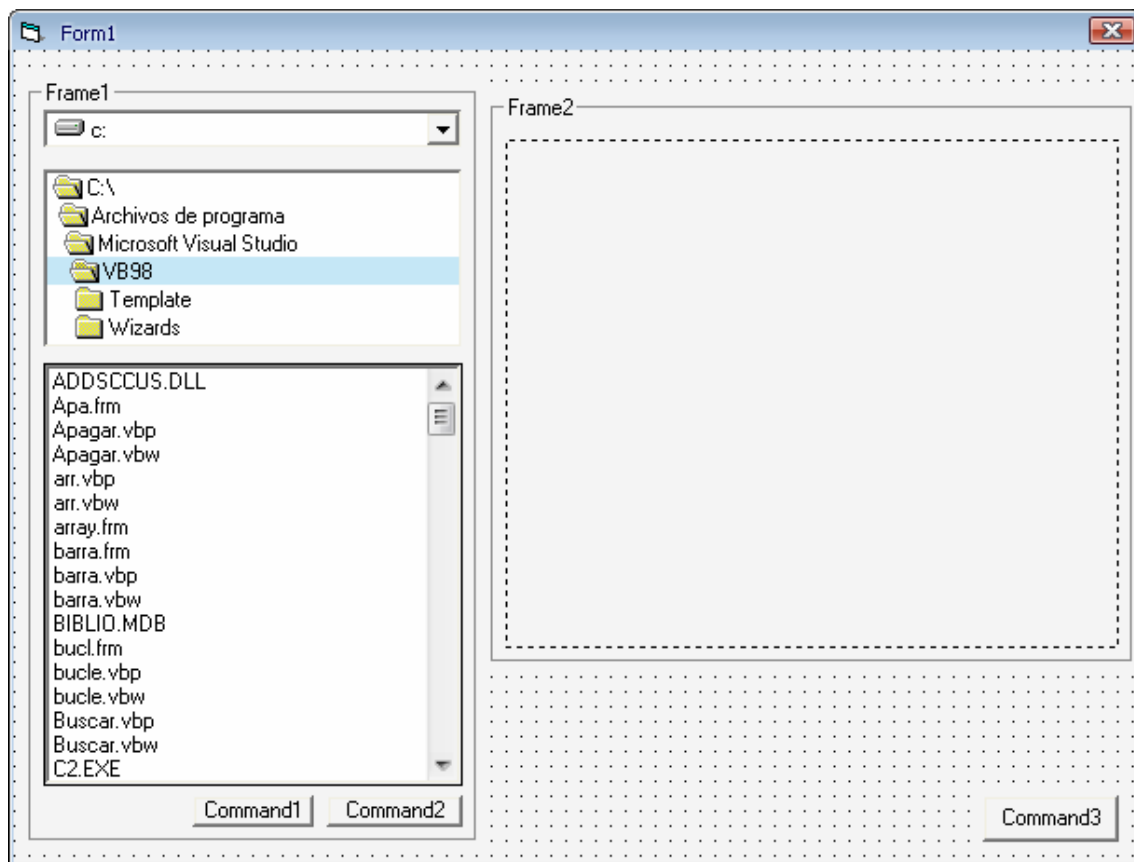
- 3.10.1.2 Ejercicios prácticos

En el siguiente ejemplo utilizamos los tres controles vistos anteriormente para navegar por las unidades del sistema y visualizar los archivos de imágenes en un control **Image**.





- Abra un nuevo proyecto.
- Modifique el tamaño del formulario con los siguientes valores en las propiedades **Height** = 6915 y **Width** = 9135.
- Inserte un control **Frame** y dentro de este coloque un control **DriveListBox**, un **DirListBox** y un **FileListBox**. Coloque debajo de estos tres controles dos botones de comando. (*Observe la imagen*).
- Inserte a la derecha otro control **Frame** y dentro un control **Image**. (*Observe la imagen*).
- En la esquina inferior derecha coloque un botón de comando. (*Observe la imagen*).



- Establezca los siguientes valores en las propiedades indicadas de cada control:

Control	Propiedad	Valor
Form1	Caption BorderStyle StartPosition	Explorador de imágenes 1 - FixedSingle 2 - CenterScreen
Frame1	Caption	-
Frame2	Caption	-
Command1	Caption	<< &Anterior
Command2	Caption	&Siguiente >>
Command3	Caption	Salir
Image1	Appearance BorderStyle	0 - Flat 1 - FixedSingle



- Haga doble clic sobre el control **DriveListBox** y dentro del evento **Change** escriba:

```
On Error Resume Next      'En caso de que cualquier unidad tenga algún error.  
'Hace que la lista de directorios mostrada sea la de la unidad seleccionada.  
Dir1.Path = Drive1.Drive
```

- Haga doble clic sobre el control **DirListBox** y dentro del evento **Change** escriba:

```
'Hace que la lista de archivo muestre los archivos de la carpeta seleccionada  
'en la lista de directorios.  
File1.Path = Dir1.Path
```

- Haga doble clic sobre el control **FileListBox** y dentro del evento **Click** escriba:

```
'Coloca la imagen seleccionada en el control Image.  
Image1.Picture = LoadPicture(File1.Path & "\" & File1.FileName)  
  
'Pone información de la imagen.  
Frame2.Caption = "Imagen mostrada: " & File1.FileName
```

- Dentro del botón de comando << *Anterior* escriba:

```
On Error Resume Next      'En caso de que la lista este vacía o se llegue al final de la lista.  
File1.Selected(File1.ListIndex - 1) = True 'Regresa a la imagen anterior.
```

- Dentro del botón de comando *Siguiente* >> escriba:

```
On Error Resume Next      'En caso de que la lista este vacía o se llegue al final de la lista.  
File1.Selected(File1.ListIndex + 1) = True 'Pasa a la siguiente imagen.
```

- Dentro del botón *Salir* escriba:

```
'Sale del programa.  
End
```

- Haga doble clic en cualquier parte del formulario y dentro del evento **Load** escriba:

```
'Hace que la lista de archivos solo muestre archivos de imágenes bmp, gif y jpg.  
File1.Pattern = "*.bmp;*.gif;*.jpg"  
  
'Hace que la imagen mostrada se ajuste al tamaño del control Image.  
Image1.Stretch = True
```

- Corra la aplicación.

- Utilice el control **DriveListBox** y seleccione una unidad de su sistema donde tenga carpetas con imágenes. En el control **DirListBox** aparecerán las carpetas de la unidad seleccionada.

- Seleccione la carpeta que contenga las imágenes en sus disco. En el control **FileListBox** aparecerá las imágenes de la carpeta seleccionada. Haga clic sobre una de las imágenes y observe como aparece en el control **Image**.



- Haga clic sobre los botones *Siguiente* y *Anterior* para mostrar cada una de las imágenes de la carpeta seleccionada.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio44-3** para el *formulario* y **Ejercicio44-3** para el proyecto.

3.2 El control de cuadros de diálogo (CommonDialog)

 El control **CommonDialog** proporciona un conjunto de cuadros de diálogo estándar para realizar operaciones como abrir y guardar archivos, establecer las opciones de impresión y seleccionar colores y fuentes. El control también tiene la posibilidad de presentar Ayuda ejecutando el motor de Ayuda de Windows.

Formato:

CommonDialog.**Metodo**

El control **CommonDialog** proporciona una interfaz entre Visual Basic y las rutinas de la biblioteca de vínculos dinámicos **Commdlg.dll** de Microsoft Windows. Para crear un cuadro de diálogo utilizando este control, **Commdlg.dll** debe encontrarse en el directorio **SYSTEM** de Microsoft Windows.

Para usar el control **CommonDialog** en una aplicación, agréguelo a un formulario y establezca sus propiedades. El cuadro de diálogo presentado por el control está determinado por los métodos del control.

En `{ CONTROL Internet.HHCtrl.1 } { HYPERLINK "JavaScript:alink_6.Click()" }` se presenta un cuadro de diálogo o se ejecuta el motor de Ayuda, cuando se invoca el método apropiado; en `{ CONTROL Internet.HHCtrl.1 } { HYPERLINK "JavaScript:alink_7.Click()" }`, el control **CommonDialog** se presenta como un icono dentro de un formulario. No se puede cambiar el tamaño de dicho icono.

El control **CommonDialog** puede presentar los cuadros de diálogo siguientes utilizando el método especificado.

Método	Cuadro de diálogo presentado
ShowOpen	Cuadro de diálogo Abrir .
ShowSave	Cuadro de diálogo Guardar como .
ShowColor	Cuadro de diálogo Color .
ShowFont	Cuadro de diálogo Fuente .
ShowPrinter	Cuadro de diálogo Imprimir u Opciones de impresión .
ShowHelp	Cuadro de diálogo Invoca el motor de Ayuda de Windows .

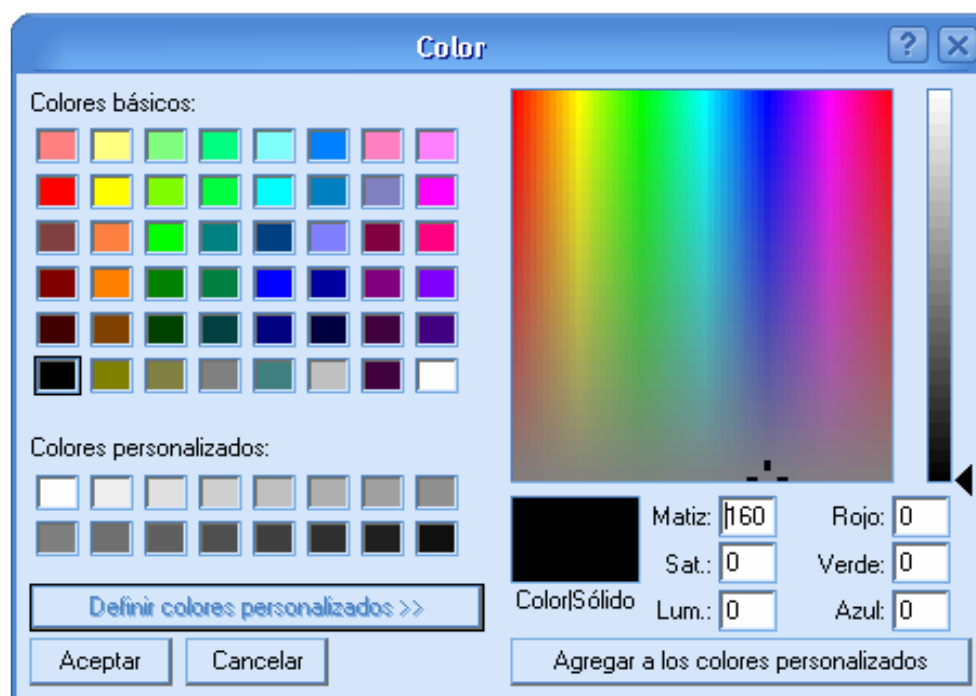
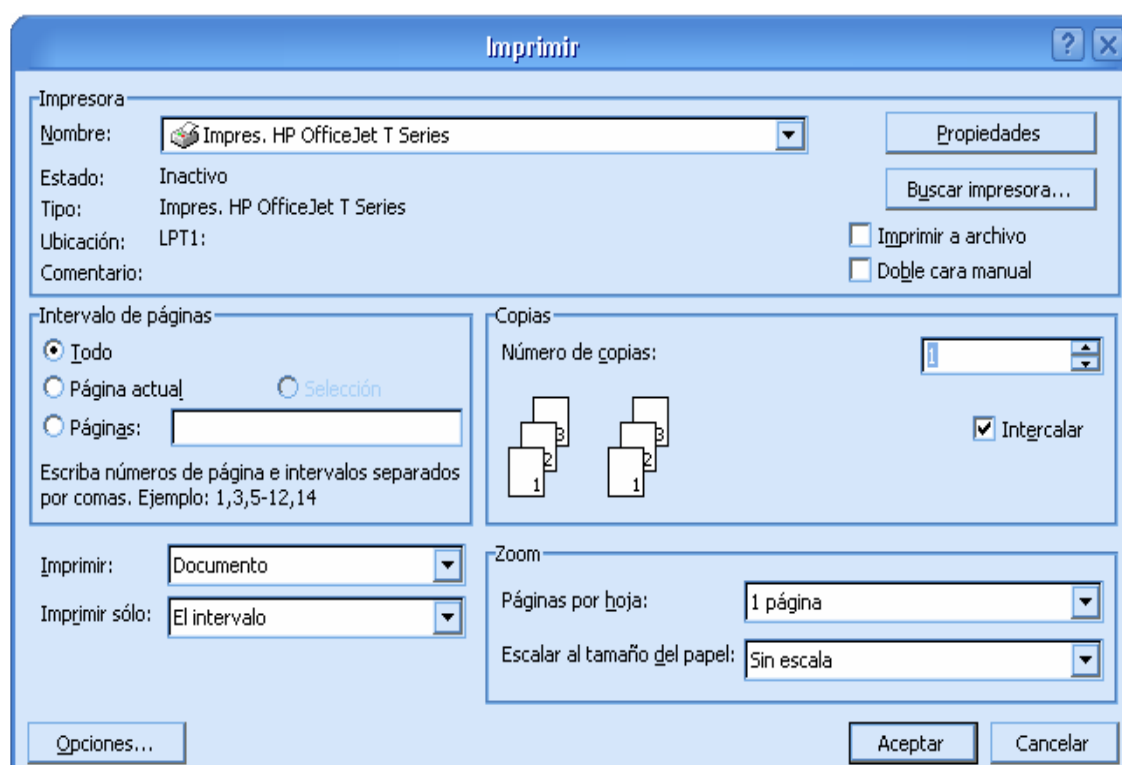
Para mostrar cada uno de los cuadros de diálogos deberá especificar el nombre del control de dialogo seguido del **método** correspondiente al cuadro de dialogo que desea abrir, por ejemplo:

CommonDialog1.**ShowOpen** 'Muestra el cuadro de diálogo Abrir.

CommonDialog1.**ShowSave** 'Muestra el cuadro de diálogo Guardar como.

CommonDialog1.**ShowColor** 'Muestra el cuadro de diálogo Color.

Ing. Carlos Manuel Rodríguez Bucarely

Figura 3.7. Cuadro de diálogo **Color**.Figura 3.8. Cuadro de diálogo **Imprimir**.

El control **CommonDialog** no aparece por defecto en la barra de herramientas no estándar. Para agregar este control a la *caja de herramientas*, haga clic en el menú **Project** y luego seleccione la opción **Components...** o simplemente presione **Ctrl + T**. Aparecerá el siguiente cuadro de diálogo:

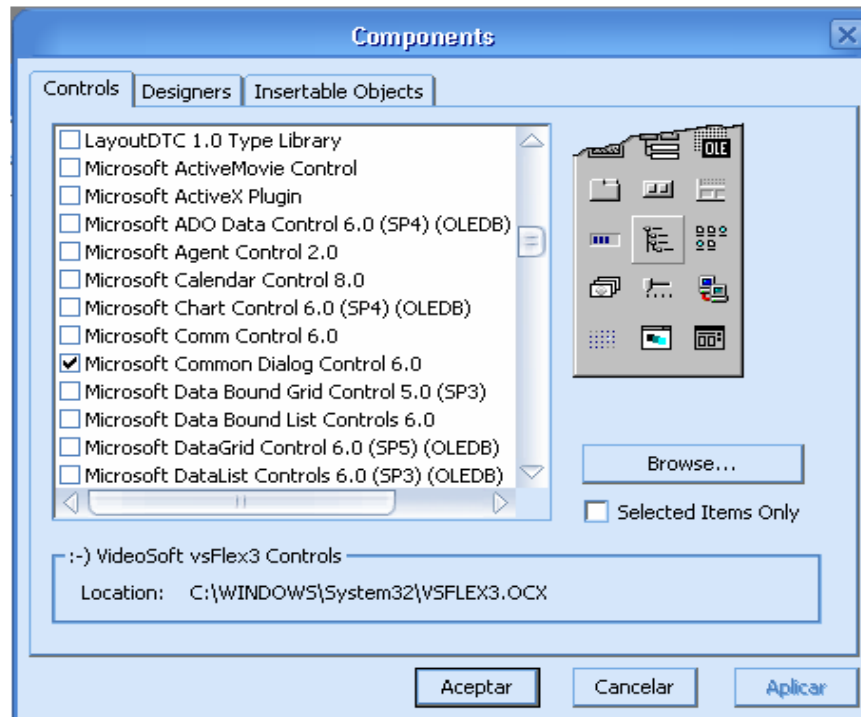


Figura 3.9. Cuadro de diálogo de Componentes.

En la ventana **Components** seleccione el elemento **Microsoft Common Dialog Control 6.0** tal y como se puede apreciar en la imagen y luego, haga clic en el botón **Aceptar**. El control **CommonDialog** se agregará a la *caja de herramientas*.

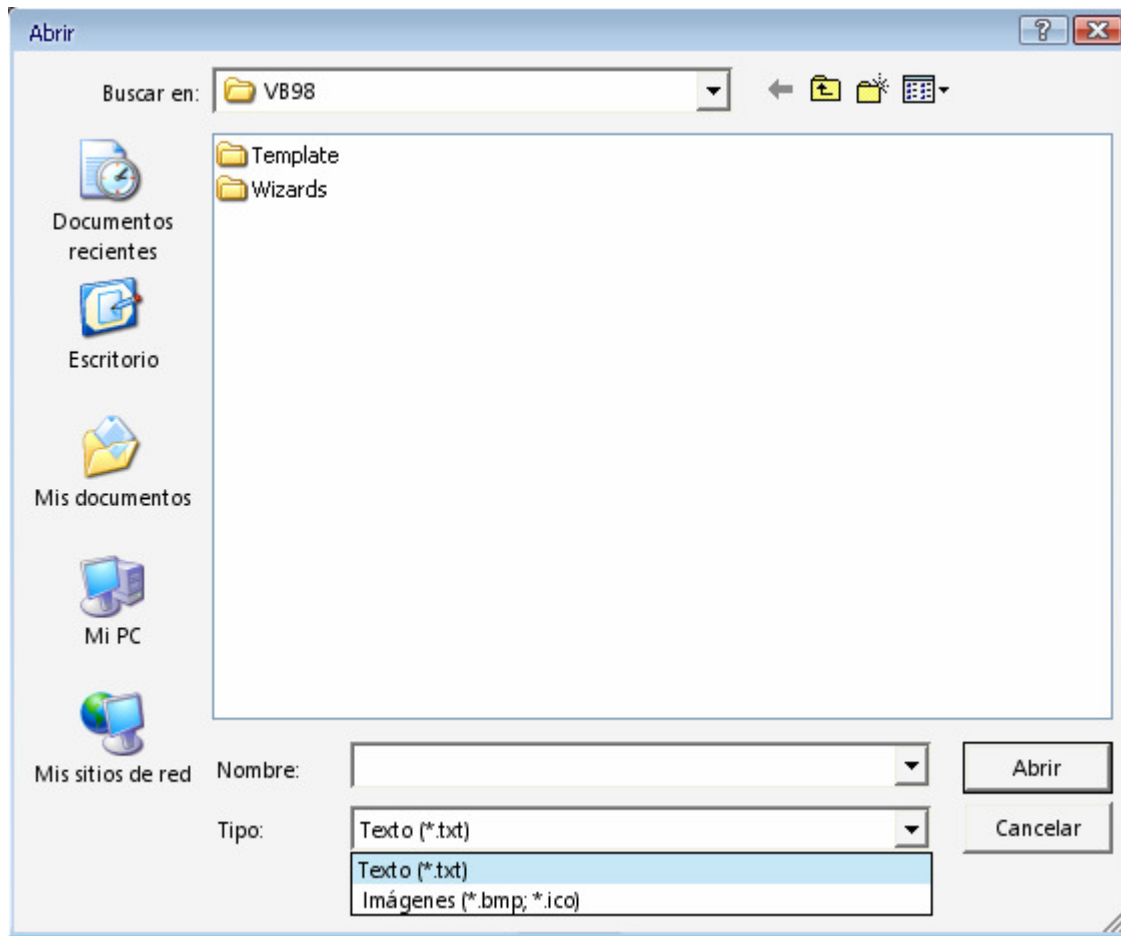
- 3.2.1 Los cuadros de dialogo Abrir y Guardar (Open/Save)

Los cuadros de diálogos **Abrir** y **Guardar** son aquellos que vemos típicamente cuando abrimos o guardamos un archivo desde un programa. A continuación se muestra una tabla con las propiedades más importantes para los métodos **ShowOpen** y **ShowSave**.

Propiedad	Descripción
DefaultExt	Es la extensión por defecto a utilizar para abrir/salvar archivos. Con Save , si el nombre del fichero se teclea sin extensión, se añade esta extensión por defecto.
DialogTitle	Devuelve o da valor al título de la caja de diálogo.
FileName	Nombre completo del archivo a abrir o guardar , incluyendo el path .
FileTitle	Nombre del archivo a abrir/salvar sin la ruta de acceso correspondiente.
Filter	Contiene los filtros de selección que aparecerán indicados en la parte inferior de la pantalla en la lista de tipos de archivo. Pueden indicarse múltiples tipos de archivo, separándolos mediante una barra vertical “ ” que se puede obtener pulsando las teclas Ctrl + Alt + 1 o pulsando la combinación de teclas ALT + 0166. Su sintaxis es la siguiente: Objeto.Filter = “(descripción a aparecer en la caja de lista) filtro” Ejemplo: CommonDialog1.Filter = “Texto (*.txt) *.txt Imágenes (*.bmp; *.ico) *.bmp; *.ico” CommonDialog1.ShowOpen



Aparecerá la siguiente ventana con la lista **Tipo** de la siguiente manera:



FilterIndex	Indica el índice de los elementos del filtro. Por defecto empieza a enumerar por "1".							
InitDir	Contiene el nombre del directorio por defecto. Si no se especifica, se utiliza el directorio actual. Los archivos del directorio especificado en esta propiedad serán los que se muestren al cargar el cuadro de dialogo Abrir. Ejemplo: CommonDialog1.InitDir = "C:\Windows" 'Muestra los archivos de la carpeta Windows.							
Flags	Devuelve o establece las opciones de los cuadros de dialogo que muestra el control CommonDialog. Los valores admitidos para la propiedad Flags para el cuadro de dialogo Abrir se muestran en la siguiente tabla: <table border="1"> <thead> <tr> <th>Constante</th><th>Valor</th><th>Descripción</th></tr> </thead> <tbody> <tr> <td>cdIOFNAIAllowMultiselect</td><td>&H200</td><td>Especifica que el <i>cuadro de lista</i> Nombre de archivo permite varias selecciones. El usuario puede seleccionar varios archivos en tiempo de ejecución presionando la tecla Mayús y utilizando las teclas Flecha arriba y Flecha abajo para seleccionar los archivos deseados.</td></tr> </tbody> </table>		Constante	Valor	Descripción	cdIOFNAIAllowMultiselect	&H200	Especifica que el <i>cuadro de lista</i> Nombre de archivo permite varias selecciones. El usuario puede seleccionar varios archivos en tiempo de ejecución presionando la tecla Mayús y utilizando las teclas Flecha arriba y Flecha abajo para seleccionar los archivos deseados.
Constante	Valor	Descripción						
cdIOFNAIAllowMultiselect	&H200	Especifica que el <i>cuadro de lista</i> Nombre de archivo permite varias selecciones. El usuario puede seleccionar varios archivos en tiempo de ejecución presionando la tecla Mayús y utilizando las teclas Flecha arriba y Flecha abajo para seleccionar los archivos deseados.						



Capítulo 3

Constante	Valor	Descripción
cdIOFNCreatePrompt	&H2000	Especifica que el cuadro de diálogo solicita al usuario la creación de un archivo que no existe actualmente. Este indicador establece automáticamente los indicadores cdIOFNPathMustExist y cdIOFNFileMustExist .
cdIOFNExplorer	&H80000	Usa la plantilla del cuadro de diálogo Abrir archivo de tipo Explorador. Funciona en Windows 95 y en Windows NT 4.0.
cdIOFNExtensionDifferent	&H400	Indica que la extensión del nombre de archivo devuelto es distinta de la extensión especificada por la propiedad DefaultExt . Este indicador no está definido si la propiedad DefaultExt es Null , si las extensiones coinciden o si el archivo no tiene extensión. El valor de este indicador se puede comprobar después de cerrar el cuadro de diálogo.
cdIOFNFileMustExist	&H1000	Especifica que el usuario sólo puede introducir nombres de archivos existentes en el cuadro de texto Nombre de archivo. Si este indicador está activado y el usuario escribe un nombre de archivo no válido, se mostrará una advertencia. Este indicador establece automáticamente el indicador cdIOFNPathMustExist .
cdIOFNHelpButton	&H10	Hace que el cuadro de diálogo presente el botón Ayuda.
cdIOFNHideReadOnly	&H4	Oculta la <i>casilla de verificación</i> Sólo lectura.
cdIOFNLongNames	&H200000	Usa nombres de archivo largos.
cdIOFNNoChangeDir	&H8	Hace que el cuadro de diálogo restablezca como directorio actual el que lo era en el momento de abrirse el cuadro de diálogo.
cdIOFNNoDereferenceLinks	&H100000	No resuelve la referencia en vínculos del sistema (también conocidos como accesos directos). De forma predeterminada, la elección de un vínculo hace que el sistema resuelva la referencia que contiene.
cdIOFNNoLongNames	&H40000	No utiliza nombres de archivo largos.
cdIOFNNoReadOnlyReturn	&H8000	Especifica que el archivo devuelto no tendrá establecido el atributo de Sólo lectura y no estará en un directorio protegido contra escritura.
cdIOFNNoValidate	&H100	Especifica que el cuadro de diálogo común permite caracteres no válidos en el nombre de archivo devuelto.
cdIOFNOverwritePrompt	&H2	Hace que el cuadro de diálogo Guardar como genere un cuadro de mensajes si el archivo seleccionado ya existe. El usuario tiene que confirmar si desea sobrescribir el archivo.

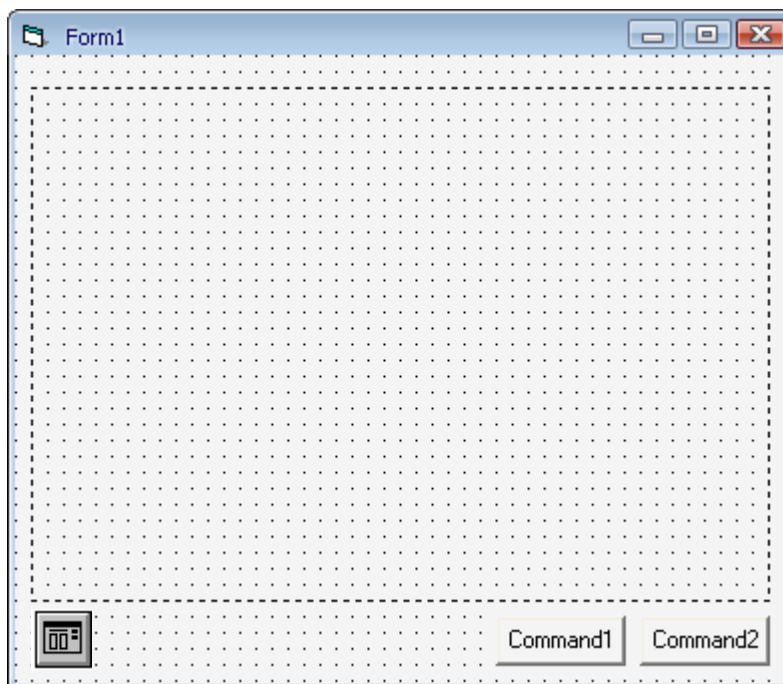


Constante	Valor	Descripción
cdIOFNPathMustExist	&H800	Especifica que el usuario sólo puede escribir rutas de acceso válidas. Si este indicador está activado y el usuario escribe una ruta no válida, se mostrará un mensaje de advertencia.
cdIOFNReadOnly	&H1	Hace que la casilla de verificación Sólo lectura esté activada inicialmente cuando se crea el cuadro de diálogo. Este indicador también indica el estado de dicha casilla de verificación cuando se cierra el cuadro de diálogo.
cdIOFNShareAware	&H4000	Especifica que se pasarán por alto los errores de violación al compartir.

- 3.2.1.1 Ejercicios prácticos

1.- El siguiente ejercicio muestra una aplicación que permite cargar una imagen en un control **Image** mediante el cuadro de dialogo **Abrir**.

- Abra un nuevo proyecto e inserte un control **CommonDialog**.
- Inserte un control **Image** y dos botones de comando, tal y como se muestra en la siguiente figura:



- Establezca los siguientes valores en las propiedades indicadas de cada control:

Control	Propiedad	Valor
Form1	Caption	Explorador de imágenes
	BorderStyle	1 – FixedSingle
	StartPosition	2 - CenterScreen
Image	Appearance	0 – Flat
	BorderStyle	1 – Fixed Single
Command1	Caption	&Abrir



Control	Propiedad	Valor
Command2	Caption	&Salir

- Dentro del evento **Click** del botón *Abrir* escriba:

On Error Resume Next 'En caso de que el usuario no seleccione ningún archivo.

'Especifica el tipo de archivo que se podrán abrir.

CommonDialog1.Filter = "Archivos de imagen (*.BMP)|*.BMP|Archivos de imagen (*.JPG)|*.JPG"

CommonDialog1.ShowOpen 'Abre el cuadro de diálogo Abrir.

'Muestra la imagen seleccionada en el control Image.

Image1.Picture = LoadPicture(CommonDialog1.FileName)

- Dentro del evento **Click** del botón *Salir* escriba:

'Sale del programa.

End

- Dentro del evento **Load** del formulario escriba:

Image1.Stretch = True 'Hace que la imagen se adapte al tamaño del control Image.

- Corra la aplicación.
- Haga clic sobre el botón *Abrir*. Aparecerá la *ventana de diálogo Abrir*.
- Busque una carpeta que contenga imágenes y seleccione una de ellas. Podrá observar como la imagen se coloca sobre el control **Image**.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio45-3** para el *formulario* y **Ejercicio45-3** para el proyecto.

- 3.2.2 El cuadro de diálogo Imprimir (Print)

El **cuadro de dialogo imprimir** es el que comúnmente vemos antes de imprimir un documento. En este aparecen las impresoras instaladas en su sistema, un botón de propiedades que permite configurar la impresora seleccionada y un conjunto de opciones disponibles antes de imprimir un documento. Observe la **Figura 3.8** de este capítulo.

Para mostrar el cuadro de diálogo imprimir utilizamos el método **ShowPrint** del control **CommonDialog**. A continuación, se muestra una tabla con las propiedades más importantes para el método de impresión (**ShowPrint**).

Propiedad	Descripción
Copies	Devuelve o establece un valor que determina el número de copias que se van a imprimir. Para el objeto Printer , no está disponible en tiempo de diseño.
FromPage	Devuelve o establece un valor que indica el número de la página donde comenzará la impresión. Este valor el usuario lo especifica en la caja de texto DE del cuadro de diálogo Imprimir .
ToPage	Devuelve o establece un valor que indica el número de la página donde terminará la impresión. Este valor el usuario lo especifica en la caja de texto A del cuadro de diálogo Imprimir .



Capítulo 3

Propiedad	Descripción		
Copies	Devuelve o establece un valor que determina el número de copias que se van a imprimir. Para el objeto Printer , no está disponible en tiempo de diseño.		
FromPage	Devuelve o establece un valor que indica el número de la página donde comenzará la impresión. Este valor el usuario lo especifica en la caja de texto DE del cuadro de diálogo Imprimir .		
ToPage	Devuelve o establece un valor que indica el número de la página donde terminará la impresión. Este valor el usuario lo especifica en la caja de texto A del cuadro de diálogo Imprimir .		
Flags	Devuelve o establece las opciones del cuadro de diálogo Imprimir .		
	Los valores disponibles en la propiedad Flags para el cuadro de diálogo Imprimir son los siguientes:		
	Constante	Valor	Descripción
	cdIPDAIIPages	&H0	Devuelve o establece el estado del botón de opción Todo.
	cdIPDCollate	&H10	Devuelve o establece el estado de la casilla de verificación Intercalar.
	cdIPDDisablePrintToFile	&H80000	Deshabilita la casilla de verificación Imprimir en un archivo.
	cdIPDHelpButton	&H800	Hace que el cuadro de diálogo presente el botón Ayuda.
	cdIPDHidePrintToFile	&H100000	Oculta la casilla de verificación Imprimir en un archivo.
	cdIPDNoPageNums	&H8	Deshabilita el botón de opción Páginas y el control de edición asociado.
	cdIPDNoSelection	&H4	Deshabilita el botón de opción Selección.
	cdIPDNoWarning	&H80	Evita que aparezca un mensaje de advertencia cuando no hay ninguna impresora predeterminada.
	cdIPDPageNums	&H2	Devuelve o establece el estado del botón de opción Páginas.
	cdIPDPrintSetup	&H40	Hace que el sistema presente el cuadro de diálogo Configurar impresora en vez del cuadro de diálogo Imprimir .
	cdIPDPrintToFile	&H20	Devuelve o establece el estado de la casilla de verificación Imprimir en un archivo.

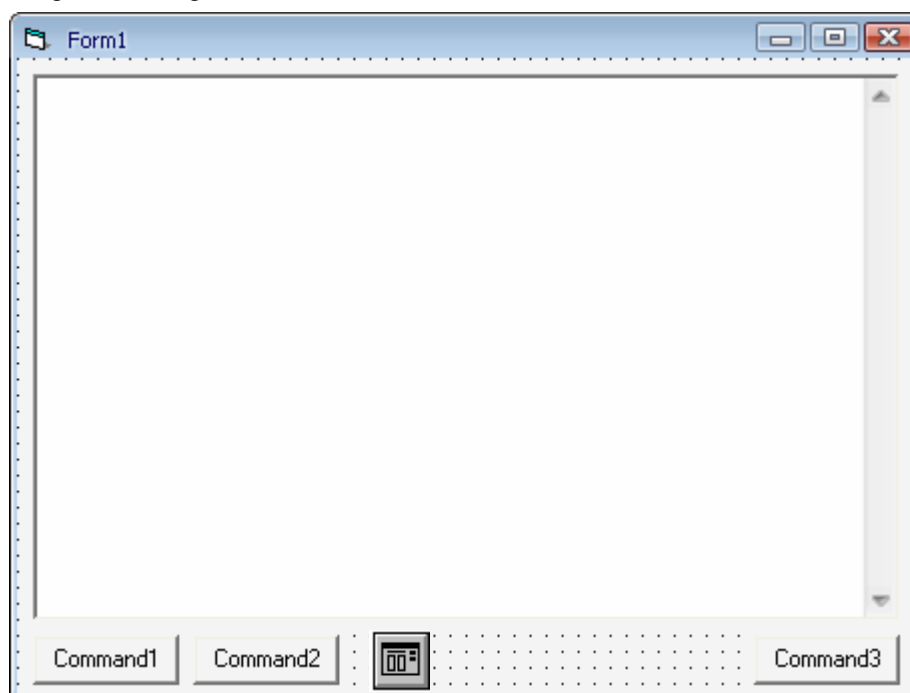


Constante	Valor	Descripción
cdIPDReturnDC	&H100	Devuelve un contexto de dispositivo para la impresora seleccionada en el cuadro de diálogo. El contexto de dispositivo se devuelve en la propiedad hDC del cuadro de diálogo.
cdIPDReturnDefault	&H400	Devuelve el nombre de la impresora predeterminada.
cdIPDReturnIC	&H200	Devuelve un contexto de información para la impresora seleccionada en el cuadro de diálogo. El contexto de información proporciona una manera rápida de obtener información acerca del dispositivo sin crear un contexto de dispositivo. El contexto de información se devuelve en la propiedad hDC del cuadro de diálogo.
cdIPDSelection	&H1	Devuelve o establece el estado del botón de opción Selección. Si no se especifican cdIPDPageNums ni cdIPDSelection , el botón de opción Todo estará en el estado seleccionado.
CdIPDUseDevModeCopies	&H40000	Si un controlador de impresora no acepta varias copias, el establecimiento de este indicador deshabilita el control Número de copias del cuadro de diálogo Imprimir .

- 3.2.2.1 Ejercicios prácticos

El siguiente ejercicio muestra como utilizar el cuadro de diálogo **Imprimir** para imprimir el contenido de una caja de texto.

- Abra un nuevo proyecto.
- Inserte una caja de texto, tres botones de comando y un control **CommonDialog**, tal y como se muestra en la siguiente imagen:



Ing. Carlos Manuel Rodríguez Bucarely



- Establezca los siguientes valores en las propiedades indicadas de cada control:

Control	Propiedad	Valor
Form1	Caption BorderStyle StartPosition	Trabajando con la impresora 1 – FixedSingle 2 - CenterScreen
Text1	MultiLine ScrollBars	True 2 - Vertical
Command1	Caption	&Abrir
Command2	Caption	&Imprimir
Command3	Caption	&Salir

- Dentro del evento **Click** del botón *Salir* escriba:

```
'Sale del programa.
End
```

- En el evento **Click** del botón de comando *Abrir* escriba:

```
On Error GoTo noArchivo 'En caso de que el usuario no seleccione ningún archivo.

CommonDialog1.Filter = "Archivos de texto (*.txt)|*.txt" 'Muestra solo los archivos de texto.

CommonDialog1.ShowOpen 'Abre la ventana de diálogo Abrir.

Dim LineaLeida As String

'Abre el archivo seleccionado por el usuario.
Open CommonDialog1.FileName For Input As #1

While Not EOF(1) 'Inicia un bucle que se repite hasta el fin del archivo.

    Line Input #1, LineaLeida 'Lee la línea actual.

    'Muestra la línea leída en la caja de texto.
    Text1.Text = Text1.Text & LineaLeida & Chr(13) & Chr(10)

Wend

Close #1 'Cierra el archivo.

Exit Sub
noArchivo:

'Muestra un cuadro de mensaje si el usuario no selecciona ningún archivo.
MsgBox ("No se ha seleccionado ningún archivo.")
```



- Corra la aplicación y haga clic sobre el botón *Abrir*.
- Busque un archivo de texto en el disco de su sistema y haga clic en el botón *Abrir* del cuadro de diálogo **Abrir**.
- Haga clic sobre el botón *Imprimir*. Aparecerá el cuadro de diálogo Imprimir.
- Haga clic sobre el botón *Imprimir* del cuadro de diálogo **Imprimir**. Los datos serán enviados a la impresora si tiene una instalada en su sistema.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio46-3** para el *formulario* y **Ejercicio46-3** para el proyecto.

- 3.2.3 El cuadro de diálogo Fuente (Font)

El **cuadro de diálogo Fuente** permite que el usuario seleccione un determinado tipo de fuente especificado una fuente, un tamaño, un color y un estilo.

Para mostrar el **cuadro de diálogo Fuente**, establezca las opciones relativas al cuadro de diálogo **Fuente** utilizando la propiedad **Flags**. Después, utilice el método **ShowFont** para presentar realmente el cuadro de diálogo. Una vez que el usuario realice sus selecciones en el cuadro de diálogo **Fuente**, las propiedades siguientes contienen información acerca de la selección del usuario:

Propiedad	Determina
Color	El color seleccionado. Para usar esta propiedad, tiene que establecer primero la propiedad Flags a cdICFEffects .
FontBold	Si ha seleccionado negrita .
FontItalic	Si ha seleccionado <i>cursiva</i> .
FontStrikethru	Si ha seleccionado tachado. Para usar esta propiedad, tiene que establecer primero la propiedad Flags a cdICFEffects .
FontUnderline	Si ha seleccionado subrayado. Para usar esta propiedad, tiene que establecer primero la propiedad Flags a cdICFEffects .
FontName	El nombre de fuente seleccionado.
FontSize	El tamaño de fuente seleccionado.

La propiedad **Flags** del **cuadro de diálogo Fuente** puede tomar los siguientes valores:

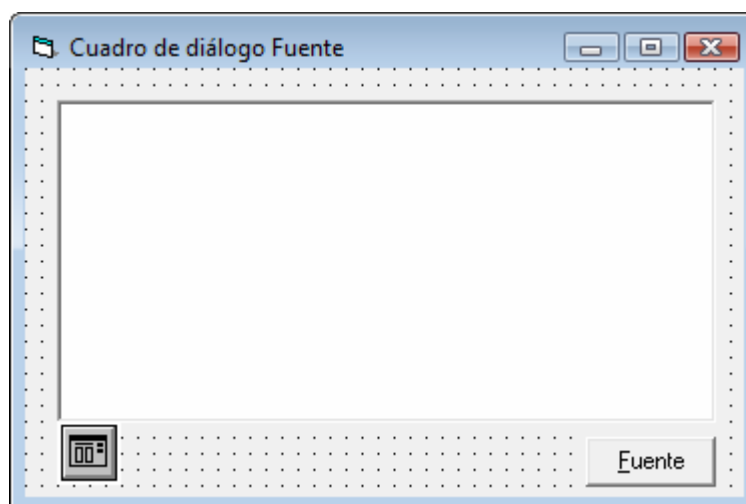
Constante	Valor	Descripción
cdICFANSIOnly	&H400	Especifica que el cuadro de diálogo sólo permite la selección de las fuentes que utilicen el juego de caracteres de Windows. Si este indicador está activado, el usuario no podrá seleccionar una fuente que sólo contenga símbolos.
cdICFApply	&H200	Activa el botón Aplicar del cuadro de diálogo.
cdICFBoth	&H3	Hace que el cuadro de diálogo enumere las fuentes de impresora y de pantalla disponibles.



Constante	Valor	Descripción
cdICFEffects	&H100	Especifica que el cuadro de diálogo permite los efectos de tachado, subrayado y color.
cdICFFixedPitchOnly	&H4000	Especifica que el cuadro de diálogo selecciona sólo fuentes de densidad fija.
cdICFForceFontExist	&H10000	Especifica que se presentará un mensaje de error si el usuario intenta seleccionar una fuente o un estilo que no exista.
cdICFHelpButton	&H4	Hace que el cuadro de diálogo presente un botón Ayuda.
cdICFLimitSize	&H2000	Especifica que el cuadro de diálogo selecciona sólo tamaños de fuente dentro del intervalo especificado por las propiedades Min y Max .
cdICFScreenFonts	&H1	Hace que el cuadro de diálogo muestre únicamente las fuentes de pantalla compatibles con el sistema.

- 3.2.3.1 Ejercicios prácticos

- A continuación se muestra una aplicación que nos permite cambiar el tipo de fuente y el color del texto escrito en una caja de texto. También, nos permite agregar efectos al texto.
- Abra un nuevo proyecto.
- Inserte una caja de texto, un botón de comando y el control **CommonDialog**. Tal y como se ve en la siguiente imagen:



- Seleccione la caja de texto y borre el contenido de la propiedad **Text**.
- En la propiedad **Caption** del botón de comando escriba &Fuente.
- En el evento **Click** del botón de comando escriba el bloque de código que se muestra en la siguiente página...



```
'Establecer CancelError a True.
CommonDialog1.CancelError = True

'Si el usuario hace clic en el botón cancelar no se provoca un error.
On Error GoTo ErrorCancelar

'Establecer la propiedad Flags.
CommonDialog1.Flags = cdlCFEffects Or cdlCFBoth

'Presentar el cuadro de diálogo Fuente
CommonDialog1.ShowFont

'Asigna a la caja de texto el tipo de letra seleccionada en el cuadro de dialogo.
Text1.Font.Name = CommonDialog1.FontName

'Asigna a la caja de texto el tamaño de letra seleccionado en el cuadro de dialogo.
Text1.Font.Size = CommonDialog1.FontSize

'Si el usuario seleccionada negrita se asigna a la caja.
Text1.Font.Bold = CommonDialog1.FontBold

'Si el usuario selecciona cursiva se asigna a la caja.
Text1.Font.Italic = CommonDialog1.FontItalic

'Si el usuario selecciona subrayado se asigna a la caja.
Text1.Font.Underline = CommonDialog1.FontUnderline

'Si el usuario selecciona tachado se asigna a la caja.
Text1.Font.Strikethru = CommonDialog1.FontStrikethru

'Asigna el color seleccionado al texto de la caja.
Text1.ForeColor = CommonDialog1.Color

Exit Sub
ErrorCancelar:
'El usuario ha hecho clic en el botón Cancelar.
Exit Sub
```

- Corra la aplicación.
- Escriba algo en la caja de texto.
- Haga clic en el botón fuente.
- Seleccione el tipo de fuente que desea aplicar al texto de la caja de texto. Cuando haga clic en el botón **Aceptar** el tipo de fuente seleccionado se aplica a la caja.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio47-3** para el *formulario* y **Ejercicio47-3** para el proyecto.



- 3.2.4 El cuadro de diálogo Color

El cuadro de diálogo **Color** permite que el usuario seleccione un color de una paleta o que cree y seleccione un color personalizado. Este cuadro de diálogo se muestra mediante el método **ShowColor** del control **CommonDialog**.

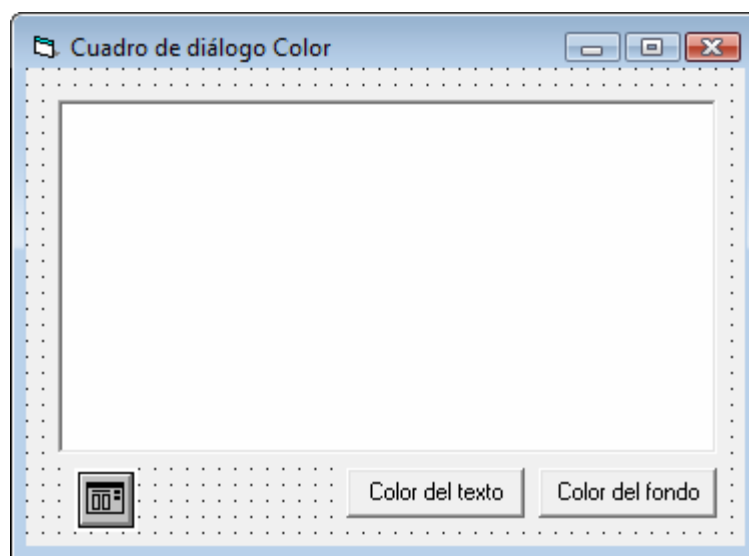
La propiedad **Color** del control **CommonDialog** almacena el color seleccionado por el usuario en la paleta de colores.

La propiedad **Flags** del **cuadro de diálogo Color** puede tomar los siguientes valores:

Constante	Valor	Descripción
cdCCFullOpen	&H2	Se presenta todo el cuadro de diálogo, incluyendo la sección Definir colores personalizados.
cdICCHelpButton	&H8	Hace que el cuadro de diálogo presente un botón Ayuda.
cdICCPreventFullOpen	&H4	Desactiva el botón de comando Definir colores personalizados y evita que el usuario defina colores personalizados.
cdICCRGBInit	&H1	Establece el valor de color inicial del cuadro de diálogo.

- 3.2.4.1 Ejercicios prácticos

- En la siguiente aplicación veremos como utilizar el cuadro de diálogo **Color** para establecer el color de fondo y del texto de una caja de texto.
- Abra un nuevo proyecto.
- Inserte una caja de texto, dos botones de comando y el control **CommonDialog**. Tal y como se ve en la siguiente imagen:



- Seleccione la caja de texto y borre el contenido de la propiedad **Text**. En la propiedad **Caption** del primer botón de comando escriba *Color del texto* y en la del segundo botón escriba *Color del fondo*.



- Dentro del evento **Click** del primer botón de comando escriba:

```
'Establecer CancelError a True.  
CommonDialog1.CancelError = True  
  
On Error GoTo ErrorCancelar  
  
'Establecer la propiedad Flags.  
CommonDialog1.Flags = cdlCCRGBInit  
  
'Presentar el cuadro de diálogo Color.  
CommonDialog1.ShowColor  
  
'Establece el color de seleccionado al texto de la caja.  
Text1.ForeColor = CommonDialog1.Color  
  
Exit Sub  
ErrorCancelar:  
'El usuario ha hecho clic en el botón Cancelar.
```

- Dentro del evento **Click** del segundo botón de comando escriba:

```
'Establecer CancelError a true.  
CommonDialog1.CancelError = True  
  
On Error GoTo ErrorCancelar  
  
'Establecer la propiedad Flags.  
CommonDialog1.Flags = cdlCCRGBInit  
  
'Presentar el cuadro de diálogo Color.  
CommonDialog1.ShowColor  
  
'Establece el color de seleccionado al texto de la caja.  
Text1.BackColor = CommonDialog1.Color  
  
Exit Sub  
ErrorCancelar:  
'El usuario ha hecho clic en el botón Cancelar.
```

- Corra la aplicación.
- Escriba algo en la caja de texto.
- Haga clic en el botón **Color del texto**. Seleccione un color de la paleta de colores y haga clic en el botón aceptar. Podrá observar que el color se le aplica al texto de la caja.
- Haga clic en el botón **Color del fondo**. Seleccione un color de la paleta de colores y haga clic en el botón aceptar. Podrá observar que el color se le aplica al fondo de la caja de texto.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio48-3** para el *formulario* y **Ejercicio48-3** para el proyecto.



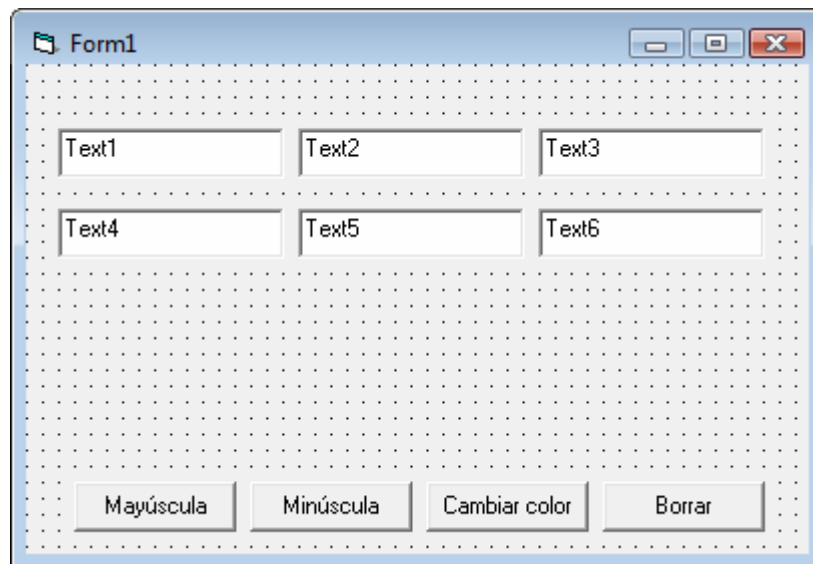
3.3 ARREGLO EN LOS CONTROLES

Un **Arreglo** de controles constituye un conjunto de controles del mismo tipo y con el mismo nombre que solo se diferencia por un índice. Son utilizados frecuentemente para realizar una misma operación sobre un mismo conjunto de controles. Cada elemento del Arreglo contiene un valor numérico indicado en la propiedad **Index** de dicho elemento. Este valor es utilizado para identificar a ese elemento dentro del arreglo.

Vemos todo esto dicho anteriormente con un ejemplo:

En el siguiente ejemplo utilizaremos seis cajas de texto a la cual le aplicaremos una operación al mismo tiempo. Cada caja de texto tendrá el mismo nombre (**Name**) pero un índice (**Index**) distinto.

- Abra un nuevo proyecto.
- Agregue seis cajas de texto y cuatro botones de comando, tal y como se muestra en la siguiente imagen:



- Para crear el Arreglo de controles debemos especificar un valor en la propiedad **Index** de cada caja de texto. Para esto seleccione la primera caja y en la propiedad **Index** escriba 1, en la misma propiedad de la segunda caja escriba 2, en la tercera caja escriba 3, en la cuarta caja escriba 4, en la quinta caja escriba 5 y en la sexta caja escriba 6.
- Ahora debemos asignar el mismo nombre a cada caja de texto. Para esto seleccione cada una de las cajas de texto y en su respectiva propiedad **Name** escriba el texto **caja**.
- En la propiedad **Caption** del primer botón escriba *Mayúscula*, en la del segundo botón escriba *Minúscula*, en la del tercer botón escriba *Cambiar color* y en la del último botón escriba *Borrar*.
- Dentro del evento **Click** del botón Mayúscula escriba:

```
Dim i As Long
```

```
For i = 1 To caja.Count 'Repite hasta que lea todas las cajas del Arreglo.
```

```
    caja(i).Text = UCase(caja(i).Text) 'Convierte el contenido de todas las cajas a mayúscula.
```

```
Next i
```



- Dentro del evento **Click** del botón *Minúscula* escriba:

```
Dim i As Long  
  
For i = 1 To caja.Count 'Repite hasta que lea todas las cajas del Arreglo.  
    caja(i).Text = LCase(caja(i).Text) 'Convierte el contenido de todas las cajas a minúscula.  
Next i
```

- Dentro del evento **Click** del botón *Cambiar color* escriba:

```
Dim i As Long  
  
For i = 1 To caja.Count 'Repite hasta que lea todas las cajas del Arreglo.  
    caja(i).BackColor = vbBlue 'Asigna el color azul al fondo de todas las cajas.  
Next i
```

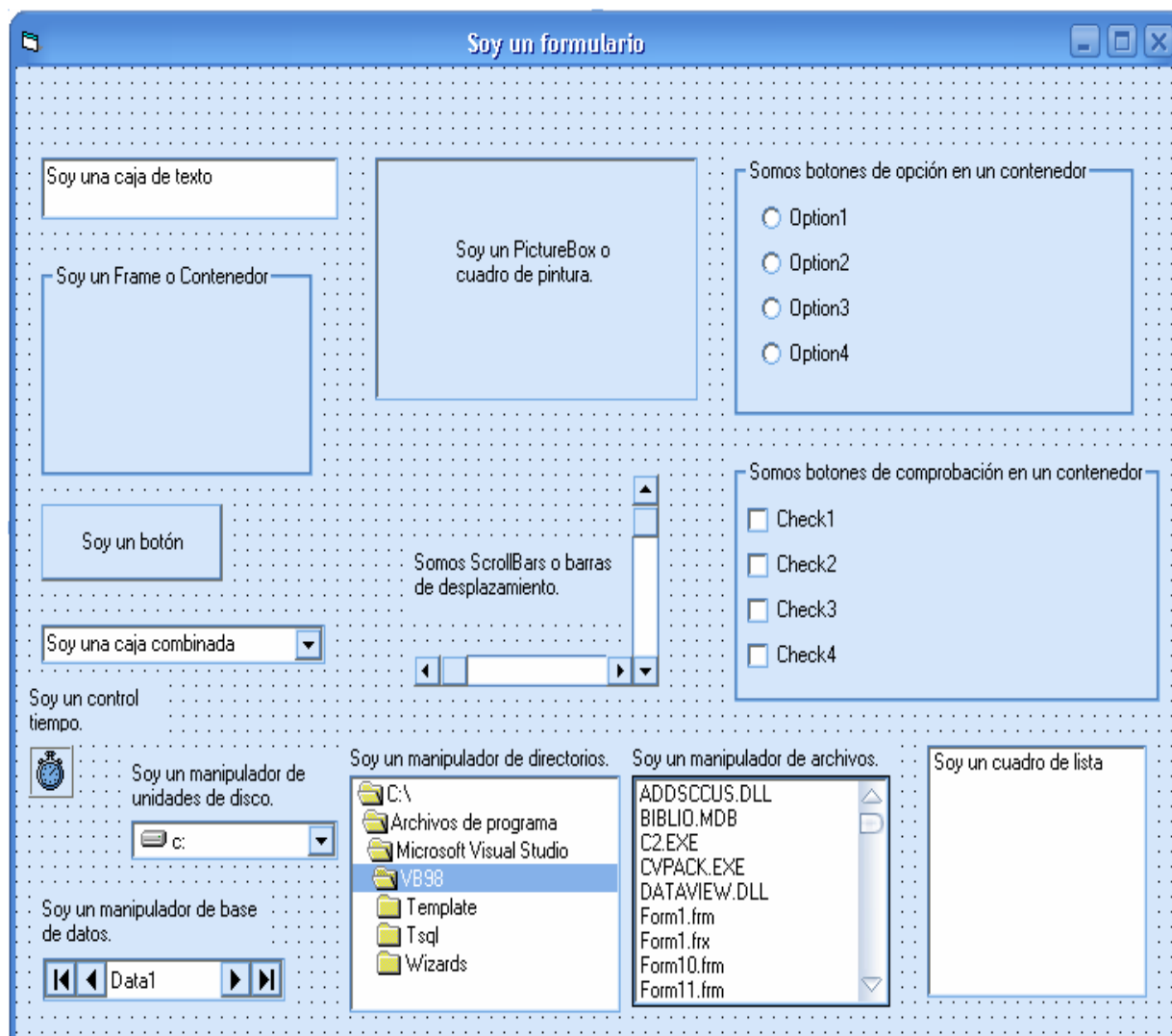
- Dentro del evento **Click** del botón *Borrar* escriba:

```
Dim i As Long  
  
For i = 1 To caja.Count 'Repite hasta que lea todas las cajas del Arreglo.  
    caja(i).Text = "" 'Borra el contenido de todas las cajas.  
Next i
```

- Corra la aplicación.
- Haga clic sobre cada uno de los botones de comando. Podrá observar que el mismo código es aplicable a cada una de las cajas de texto. Esto es debido a que cada caja de texto posee un índice que lo representa dentro del Arreglo. Este índice cambia cada vez que se repite el bucle recorriendo así por todas las cajas que contiene el Arreglo.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio49-3** para el *formulario* y **Ejercicio49-3** para el proyecto.



3.4 FIGURA CON LOS CONTROLES MÁS USUALES DE VISUAL BASIC 6.0





CONTENIDO

4.1 El Lenguaje Basic.

- 4.1.1 Antecedentes.
- 4.1.2 Nacimiento y primeros años.
- 4.1.3 Crecimiento Explosivo.
- 4.1.4 Perfeccionamiento.

4.2 Conceptos básicos.

- 4.2.1 Identificadores.
- 4.2.2 Palabras reservadas.
- 4.2.3 Tipos de datos.
 - 4.2.3.1 Clasificación de los tipos de datos.
 - 4.2.3.1.1 Tipos enteros (Byte, Integer, Long).
 - 4.2.3.1.2 Tipos reales (Single, Double, Currency).
 - 4.2.3.1.3 Tipos cadena (String).
 - 4.2.3.1.4 Tipos lógicos (Boolean).
 - 4.2.3.1.5 Tipos variados (Variant).
- 4.2.4 Constantes.
 - 4.2.4.1 Declaración de constantes.
 - 4.2.4.2 Declaración de constantes públicas.
 - 4.2.4.3 Declaración de constantes privadas.
 - 4.2.4.4 Declaración de constantes locales.
- 4.2.5 Variables.
 - 4.2.5.1 Declaración de una variable.
 - 4.2.5.1.1 Declaración de una variable pública.
 - 4.2.5.1.2 Declaración de una variable privada.
 - 4.2.5.1.3 Declaración de una variable local.
 - 4.2.5.2 Nombre descriptivos de las variables.
 - 4.2.5.3 Almacenar y recuperar datos en variables.
 - 4.2.5.4 Operaciones aritméticas con variables.
 - 4.2.5.5 Ejercicios propuestos.
- 4.2.6 Operaciones de entrada y salida.
 - 4.2.6.1 Función InputBox.
 - 4.2.6.2 Función MsgBox.

4.3 Estructuras de control selectivas.

- 4.3.1 Expresiones lógicas.
 - 4.3.1.1 Operadores aritméticos.
 - 4.3.1.2 Operadores de relación.
 - 4.3.1.3 Operadores lógicos.
 - 4.3.1.3.1 Operador lógico AND.
 - 4.3.1.3.2 Operador lógico OR.
 - 4.3.1.3.3 Operador lógico NOT.
- 4.3.2 La sentencia If.
- 4.3.3 La sentencia Case.

4.4 Estructuras de control repetitivas.

- 4.4.1 El Bucle For...Next.
- 4.4.2 El Bucle Do...Loop.
- 4.4.3 El Bucle While...Wend.
- 4.4.4 El Bucle For Each...Next.
- 4.4.5 Ejercicios propuestos.

4.5 Las funciones en el lenguaje Basic.

- 4.5.1 Funciones predefinidas.
 - 4.5.1.1 Funciones aritméticas.
 - 4.5.1.2 Funciones trigonométricas.
 - 4.5.1.3 Funciones matemáticas derivadas.
 - 4.5.1.4 Funciones de conversión de tipo de datos.



CONTENIDO

- 4.5.1.5 Funciones de manipulación de cadenas de caracteres.
 - 4.5.1.5.1 Concatenación de cadenas.
 - 4.5.1.5.2 Obtener subcadenas.
 - 4.5.1.5.3 Funciones especiales.
 - 4.5.1.5.4 Longitud de una cadena.
- 4.5.1.6 Ejercicios prácticos.
- 4.5.2 Funciones definidas por el usuario.
- 4.6 Introducción a las estructuras de datos.**
 - 4.6.1 Los Arrays.
 - 4.6.1.1 Arrays unidimensionales: vectores.
 - 4.6.1.1.1 Declaración de un Array unidimensional.
 - 4.6.1.1.2 Almacenar y leer datos en un Array unidimensional.
 - 4.6.1.1.3 Operaciones con los elementos de un Array unidimensional.
 - 4.6.1.1.4 Ejercicios prácticos.
 - 4.6.1.2 Arrays multidimensionales: tablas y matrices.
 - 4.6.1.2.1 Declaración de un Array multidimensional.
 - 4.9.1.2.2 Recorrido por las filas y columnas de un Array multidimensional.
 - 4.6.1.2.3 Almacenando y leer datos en un Array multidimensional.
 - 4.6.1.2.4 Ejercicios prácticos.
 - 4.6.2 Ejercicios propuestos.
- 4.7 Preguntas para contestar.**



4.1 El Lenguaje Basic

BASIC es una familia de lenguajes de programación. Fue originalmente ideado como una herramienta de enseñanza, se diseminó entre los microcomputadores caseros en la década de 1980, y sigue siendo popular hoy en día en muchos dialectos bastante distintos del original.

BASIC es el acrónimo de **B**eginners **A**ll-purpose **S**ymbolic **I**nstruction **C**ode (en español: "código de instrucciones simbólicas de propósito general para principiantes") y está ligado al nombre de un trabajo sin publicar del coinventor del lenguaje, **Thomas Kurtz**.

- 4.1.1 Antecedentes

Antes de mediados de la década de 1960, las computadoras eran herramientas sumamente caras que eran utilizadas únicamente para propósitos especiales, ejecutando una sola "tarea" a la vez. Sin embargo, durante esa década, los precios comenzaron a bajar al punto que incluso las pequeñas empresas podían costearlas. La velocidad de las máquinas se incrementó al punto que a menudo quedaban ociosas porque no había suficientes tareas para ellas. Los lenguajes de programación de aquellos tiempos estaban diseñados como las máquinas en las que corrían: para propósitos específicos como el procesamiento de fórmulas. Como las máquinas para una sola tarea eran caras, se consideraba que la velocidad de ejecución era la característica más importante de todas. En general, todas eran difíciles de utilizar, y aportaban poca estética.

Fue en aquellos tiempos que el concepto de sistema de Tiempo compartido comenzó a popularizarse. En uno de estos sistemas, el tiempo de procesamiento de la computadora principal se dividía, y a cada usuario se le otorgaba una pequeña porción en una secuencia. Las máquinas eran lo suficientemente rápidas como para engañar a la mayoría de usuarios, dándoles la ilusión de que disponían de una máquina entera solo para ellos. En teoría la distribución del tiempo entre los usuarios redujo considerablemente el costo de la computación, ya que una sola máquina podía ser compartida, al menos en teoría, entre cientos de usuarios.

- 4.1.2 Nacimiento y primeros años

El lenguaje BASIC original fue inventado en 1964 por **John George Kemeny** (1926-1993) y **Thomas Eugene Kurtz** (1928) en el **Dartmouth College**. En los años subsiguientes, mientras que otros dialectos de BASIC aparecían, el BASIC original de Kemeny y Kurtz era conocido como **BASIC Dartmouth**.

BASIC fue diseñado para permitir a los estudiantes escribir programas usando terminales de computador de tiempo compartido. BASIC estaba intencionado para facilitar los problemas de complejidad de los lenguajes anteriores, con un nuevo lenguaje diseñado específicamente para la clase de usuarios que los sistemas de tiempo compartido permitían: un usuario más sencillo, a quien no le interesaba tanto la velocidad, sino el hecho de ser capaz de usar la máquina. Los diseñadores del lenguaje también querían que permaneciera en el dominio público, lo que contribuyó a que se diseminara.

Los ocho principios de diseño de BASIC fueron:

1. Ser fácil de usar para los principiantes.
2. Ser un lenguaje de propósito general.
3. Permitir que los expertos añadieran características avanzadas, mientras que el lenguaje permanecía simple para los principiantes.
4. Ser interactivo.
5. Proveer mensajes de error claros y amigables.
6. Responder rápido a los programas pequeños.



7. No requerir un conocimiento del hardware de la computadora.
8. Proteger al usuario del sistema operativo.

El lenguaje fue en parte basado en **FORTRAN II** y en parte en **Algol 60**, con adiciones para hacerlo apropiado para tiempo compartido y aritmética de matrices, BASIC fue implementado por primera vez en la **mainframe GE-265**, que soportaba múltiples terminales. Contrario a la creencia popular, era un lenguaje compilado al momento de su introducción. Casi inmediatamente después de su lanzamiento, los profesionales de computación comenzaron a alegar que BASIC era muy lento y simple. Tal argumento es un tema recurrente en la industria de las computadoras.

Aun así, BASIC se expandió hacia muchas máquinas, y se popularizó moderadamente en los minicomputadores como la serie **DEC PDP y la Data General Nova**. En estos casos, el lenguaje era implementado como un intérprete, en vez de un compilador, o alternativamente, de ambas formas.

- 4.1.3 Crecimiento Explosivo

Sin embargo, fue con la introducción de la **Microcomputador Altair 8800** en 1975 que BASIC se diseminó ampliamente. La mayoría de lenguajes de programación eran demasiado grandes para caber en la pequeña memoria que la mayoría de usuarios podía pagar para sus máquinas, y con el lento almacenamiento que era la cinta de papel, y más tarde la cinta de audiocasete (los discos magnéticos aún no existían), y la falta de editores de texto adecuados, un lenguaje pequeño como BASIC era una buena opción. Uno de los primeros en aparecer fue **Tiny BASIC**, una implementación simple de BASIC escrita originalmente por el **Dr. Li-Chen Wang**, y portada más tarde a la **Altair** por **Dennis Allison**, a petición de **Bob Albrecht** (quien después fundó el *Dr. Dobbs Journal*). El diseño de Tiny BASIC y el código fuente completo fue publicado en 1976 en DDJ.

En 1975 Microsoft (entonces constaba de dos personas: **Bill Gates y Paul Allen**) lanzó Altair BASIC. Luego comenzaron a aparecer bajo licencia versiones para otras plataformas, y millones de copias y variantes pronto estarían en uso. Se convirtió en uno de los lenguajes estándar en la Apple II. Para 1979 Microsoft estaba negociando con varios vendedores de microcomputadores, incluyendo a IBM, para licenciar un intérprete de BASIC para sus computadores. Una versión se incluyó en los chips ROM de las PCs IBM, para PCs sin discos, y en las que disponían de unidad de diskettes el BASIC era iniciado automáticamente si es que no se colocaba ningún diskette de arranque de sistema operativo.

Mientras que las nuevas compañías intentaban seguir los pasos del éxito de Altair, IMSAI, North Star, y Apple, creando la revolución de la computadora casera. BASIC se convirtió en una característica estándar para casi todas las computadoras caseras; la mayoría venía con un intérprete de BASIC en ROM (algo hecho por primera vez por la Commodore PET en 1977). Pronto había muchos millones de computadores corriendo BASIC alrededor del mundo, un número mucho más grande que el de todos los usuarios de otros lenguajes juntos. Muchos programas, especialmente los de la Apple II e IBM PC, dependían de la presencia del intérprete de BASIC de Microsoft y no correrían sin éste; por lo que Microsoft usó la licencia de copyright en los intérpretes de BASIC para influir en las negociaciones con los vendedores de computadores.

El BASIC fue también el lenguaje prefijado en los computadores caseros europeos de la década de los 80 como el **ZX Spectrum**, **MSX** o el **Commodore 64**, haciendo muchas veces la función de intérprete y sistema operativo primitivo ya que venían implementados en ROM.

- 4.1.4 Perfeccionamiento

En este período se crearon versiones de BASIC nuevas y más poderosas. Microsoft vendió varias



versiones de BASIC para **MS-DOS/PC-DOS**, incluyendo **BASICA**, **GW-BASIC** (una versión compatible con BASICA que no necesitaba la ROM de IBM), y **Quick BASIC**. El fabricante de **Turbo Pascal**, **Borland**, publicó **Turbo BASIC** 1.0 en 1985 (versiones sucesoras aún se venden bajo el nombre de **PowerBASIC** por otra compañía). Aparecieron varias extensiones de BASIC para computadores caseras, típicamente con gráficos, sonido, y comandos DOS, así como facilidades para Programación estructurada. Otros lenguajes usan la sintaxis de BASIC como base para otros sistemas totalmente diferentes, como por ejemplo GRASS.

Sin embargo a finales de la década de 1980 las computadoras nuevas eran mucho más complejas, e incluían características (como la Interfaz gráfica de usuario) que hacían a BASIC menos apropiado para programarlas. Al mismo tiempo las computadoras progresaban de ser un interés para aficionados a herramientas usadas principalmente para ejecutar aplicaciones escritas por otros, y la programación en sí se fue haciendo menos importante para la creciente mayoría de usuarios. BASIC comenzó a desvanecerse, aunque numerosas versiones aún estaban disponibles.

La suerte de BASIC dio un giro nuevamente con la introducción de **Visual Basic** de Microsoft. Aunque es algo difícil considerar este lenguaje como BASIC (a pesar de que usa muchas palabras clave conocidas de BASIC) se ha convertido en uno de los lenguajes más usados en la plataforma Windows, y se dice que representa del 70 al 80% del desarrollo comercial de aplicaciones. **Visual Basic for Applications (VBA)** fue añadido a **Microsoft Excel 5.0** en 1993 y al resto de la línea de productos de Microsoft Office en 1997. Windows 98 incluyó un intérprete de **VBScript**. La versión más reciente de Visual Basic es llamada **VB.NET**. La suite **OpenOffice.org** incluye una variante de BASIC menos poderosa que su contraparte de Microsoft.

4.1 Conceptos básicos

Todos los lenguajes de programación están compuestos por una serie de objetos que hacen posible su funcionamiento entre los cuales tenemos: variables, constantes, tipos de datos, sentencias, expresiones, funciones y estructuras. Sin estos los lenguajes de programación serían totalmente inservibles.

- 4.2.1 Identificadores

Los identificadores representan una expresión que hace referencia a una variable o una constantes. Un identificador es una secuencia de caracteres que puede tener una longitud máxima de 255 caracteres.

Un identificador se caracteriza por estas reglas:

1. Debe comenzar con una letra (A a Z, mayúsculas o minúsculas) y no puede contener blancos.
2. Letras, dígitos y caracteres subrayados (_) están permitidos después del primer carácter.
3. No se puede utilizar una palabra reservada como identificador.
4. El @ **Arroba** es valido después del último carácter y representa un tipo de datos.
5. El & **Ampersand** es valido después del último carácter y representa un tipo de datos.

A continuación, se muestra una lista de identificadores validos:

- Nombre



- Area_Rectangulo
- Lado1
- Impuesto_Sobre_La_Renta
- ImpuestoTotales

Estos son identificadores no validos:

- Area&Rectangulo : Tiene un & Ampersand antes del último carácter. Es valido solo al final.
- Lado 1 : Tiene un espacio.
- Impuesto@Renta : Tiene un @ Arroba antes del último carácter. Es valido solo al final.
- For : Es una palabra clave del lenguaje Basic.
- 5Nombre : Tiene un número al inicio. Es valido después del primer carácter.

- 4.2.2 Palabras reservadas

Las palabras reservadas del lenguaje Basic no se pueden utilizar como identificadores, ya que tienen significado especial en Visual Basic y no se utilizan para otros propósitos.

A continuación, se muestra una lista con las palabras reservadas del lenguaje Basic:

Abs	Add	And	App
Activate	Array	Ascatn	Base
Beep	Cbool	Cbyte	Ccur
Cdate	Cdbl	Cdec	Cint
Clng	Csng	Cstr	Cvar
Cverr	Call	Case	Cdh
Choose	Chdir	Chdrive	Clear
Close	Collection	Command	Compare
Const	Cos	CreateObject	Curdir
Dateserial	Datevalue	Day	Ddb
Defftype	Dim	Deletesetting	Dir
Do	Doevents	Each	End
Environ	Eof	Eqv	Erase
Err	Error	Exit	Exp
Explicit	Fileattr	Filecopy	Filedatetime
Filelen	Fix	For	Format
Freefile	Function	Fv	Get
Getattr	GetObject	Getsetting	Getallsetting
Gosub	Goto	Hex	Hour
Imp	Input	Instr	Int
Integer	Ipmt	Irr	Is
Isarray	Isdate	Isempty	Iserror
Ismissing	IsNull	Isnumeric	Isobject
Item	Kill	Lbound	Lcase



Left	Let	Like	Loc
Lock	Lof	Log	Loop
Lset	Ltrim	Me	Mid
Minute	Mirr	Mkdir	Mod
Month	Name	New	Next
Not	Nper	Npv	Oct
On	Onerror	Open	Or
Option	Print	Ppmt	Print#
Private	Property	Public	Put
Pv	Qbcolor	Raise	Randomize
Rate	Redim	Remove	Reset
Resume	Return	Rgb	Right
Rmdir	Rnd	Rset	Rtrim
Savesettings	Second	Selectcase	Seek
Shell	SendKeys	Setattr	Sgn
Sin	Single	Sln	Space
Spc	Sqr	Static	Stop
Str	Strcomp	Strconv	String
Sub	Syd	Switch	Tab
Tan	Timer	TimeSerial	TimeValue
Trim	TypeName	Ubound	Ucase
Unlock	Val	Vartype	Weekday
Wend	While	Width	Write#
Xor	Yeqr	#if	#else

- 4.2.3 Tipos de datos

Los tipos de datos son los distintos objetos de información con los que trabaja una aplicación en Visual Basic. Todos los datos tienen un tipo asociado con ellos. Un dato puede ser un simple carácter como un "B", una cadena de caracteres como "La casa de pedro", un valor entero como 242, un número real como 3.1415 o un valor lógico como True o False.

- 4.2.3.1 Clasificación de los tipos de datos

Se dijo anteriormente, que cada tipo de información tiene un tipo de datos asociados. Cualquier tipo de datos estará definido dentro de la siguiente clasificación: **tipos enteros (Byte, Integer, Long)**, **tipos reales (Single, Double, Currency)**, **tipos cadena (String)**, **tipos lógicos (Boolean)**, **tipos fecha (Date)** y **tipos variados (Variant)**. Al seleccionar un tipo de datos adecuado para las variables de su programa ahorraría mucho espacio en la memoria del computador donde se ejecute la aplicación y como resultado esta trabajaría mucho más rápido. El tipo de datos a seleccionar va a depender del tipo de información que usted valla a almacenar en una variable. A continuación, se muestran todos los tipos de datos disponibles en Visual Basic y el tipo de información que cada uno de ellos pueden almacenar, así como los valores máximos y mínimos que estos soportan.

- 4.2.3.1.1 Tipos enteros (Byte, Integer, Long)

Visual Basic tiene tres tipos de datos predefinidos para representar los números enteros: **Byte**, **Integer** y **Long**.

- **Byte:** Las { CONTROL Internet.HHCtrl.1 } { HYPERLINK "JavaScript:alink_4.Click()" } se almacenan como números de 8 bits (1 byte). No pueden tener signos y debe estar dentro de los valores 0 y 255. El { CONTROL Internet.HHCtrl.1 } { HYPERLINK "JavaScript:alink_5.Click()" } Byte es útil para almacenar datos binarios.

- **Integer:** Las variables de tipo Integer se almacenan como número de 16 bits (2 bytes). Pueden



tener signos negativo o positivo dentro del intervalo -32.768 a 32.767. Esto quiere decir, que el mayor número negativo que puede almacenar es -32.768 y el mayor número positivo es 32,767. El carácter de declaración de tipo para el tipo **Integer** es el signo de porcentaje (%).

Las variables tipo **Integer** también se pueden utilizar para representar valores enumerados. Un valor enumerado puede contener un conjunto finito de números enteros únicos, cada uno de los cuales tiene un significado especial en el contexto en el que se utiliza. Los valores enumerados proporcionan una forma cómoda de seleccionar entre un número conocido de opciones. Por ejemplo, cuando se pregunta al usuario que elija un color de una lista, se podría tener 0 = negro, 1 = blanco y así sucesivamente. Es una buena práctica de programación definir constantes utilizando la instrucción **Const** para cada valor enumerado.

- **Long:** Las variables **Long** (enteros largos) se almacenan como números con signo de 32 bits (4 bytes) con un valor comprendido entre -2.147.483.648 y 2.147.483.647. El carácter de declaración de tipo para **Long** es el Ampersand (&).

- 4.2.3.1.2 Tipos reales (Single, Double, Currency)

Visual Basic también posee tres tipos de datos para representar los números reales: **Single**, **Double** y **Currency**.

- **Single:** Las variables **Single** (punto flotante de precisión simple) se almacenan como números IEEE de coma flotante de 32 bits (4 bytes) con valores que van de -3,402823E38 a -1,401298E-45 para valores negativos y de 1,401298E-45 a 3,402823E38 para valores positivos. El carácter de declaración de tipo para **Single** es el signo de exclamación (!).

- **Double:** Las variables **Double** (punto flotante de doble precisión) se almacenan como números IEEE de coma flotante de 64 bits (8 bytes) con valores de -1,79769313486232E308 a -4,94065645841247E-324 para valores negativos y de 4,94065645841247E-324 a 1,79769313486232E308 para valores positivos. El carácter de declaración de tipo para **Double** es el signo de número (#).

- **Currency:** Las variables tipo **Currency** se almacenan como números de 64 bits (8 bytes) en un formato de número entero a escala de 10,000 para dar un número de punto fijo con 15 dígitos a la izquierda del signo decimal y 4 dígitos a la derecha. Esta representación proporciona un intervalo de -922.337.203.685.477,5808 a 922.337.203.685.477,5807. El carácter de declaración de tipo para **Currency** es el signo arroba @.

El tipo de datos **Currency** es útil para cálculos monetarios y para cálculos de punto fijo, en los cuales la precisión es especialmente importante.

- 4.2.3.1.3 Tipos cadena (String)

Hay dos clases de cadenas: cadenas de longitud variable y cadenas de longitud fija.

- Las cadenas de longitud variable pueden contener hasta 2,000 millones de caracteres (2^{31}).
- Las cadenas de longitud fija que pueden contener de 1 a 64 KB (2^{16}) caracteres.

Nota: No se puede usar una cadena { **CONTROL Internet.HHCtrl.1** } { **HYPERLINK** "JavaScript:alink_4.Click()" } de longitud fija en un { **CONTROL Internet.HHCtrl.1** } { **HYPERLINK** "JavaScript:alink_5.Click()" }.

Los códigos para caracteres de tipo **String** varían desde 0 a 255. Los primeros 128 caracteres (0–127) del juego de caracteres corresponden a los letras y los símbolos de un teclado estándar de

Ing. Carlos Manuel Rodríguez Bucarely



ASCII. Los siguientes 128 caracteres (128–255) representan caracteres especiales, como letras de alfabetos internacionales, acentos, símbolos de moneda y fracciones. El carácter de declaración de tipo para **String** es el signo de dólar (\$).

- 4.2.3.1.4 Tipos lógicos (Boolean)

Las { `CONTROL Internet.HHCtrl.1` } { `HYPERLINK "JavaScript:alink_4.Click()"` } se almacenan como números de 16 bits (2 bytes), pero sólo pueden ser **True** o **False**. Las variables tipo **Boolean** se presentan como **True** o **False** (cuando se utiliza **Print**) o `#TRUE#` o `#FALSE#` (cuando se utiliza **Write #**). Utilice las { `CONTROL Internet.HHCtrl.1` } { `HYPERLINK "JavaScript:alink_5.Click()"` } **True** y **False** para asignar uno de los dos estados a las variables tipo **Boolean**.

Cuando se convierten a tipo **Boolean** otros { `CONTROL Internet.HHCtrl.1` } { `HYPERLINK "JavaScript:alink_6.Click()"` }, 0 se convierte en **False**, y el resto de los valores se convierten en **True**. Cuando los valores tipo **Boolean** se convierten a otros { `CONTROL Internet.HHCtrl.1` } { `HYPERLINK "JavaScript:alink_7.Click()"` } numéricos, **False** se convierte en 0 y **True** se convierte en -1.

- 4.2.3.1.5 Tipos variados (Variant)

Una variable **Variant** es capaz de almacenar todos los tipos de datos definidos en el sistema. No tiene que convertir entre esos tipos de datos si los asigna a una variable **Variant**; **Visual Basic** realiza automáticamente cualquier conversión necesaria.

- 4.2.4 Constantes

A menudo verá que el código contiene *valores constantes* que reaparecen una y otra vez. O puede que el código dependa de ciertos números que resulten difíciles de recordar (números que, por sí mismos, no tienen un significado obvio).

En estos casos, puede mejorar mucho la legibilidad del código y facilitar su mantenimiento si utiliza constantes. Una *constante* es un nombre significativo que sustituye a un número o una cadena que no varía. Aunque una *constante* recuerda ligeramente a una variable, no puede modificar una constante o asignarle un valor nuevo como ocurre con una variable. Hay dos orígenes para las *constantes*:

Constantes intrínsecas o definidas por el sistema: proporcionadas por aplicaciones y controles. Las constantes de Visual Basic se muestran en **Visual Basic (VB)** y **Visual Basic** para aplicaciones (**VBA**) y las bibliotecas de objetos en el Examinador de objetos. Otras aplicaciones que proporcionan bibliotecas de objetos, como Microsoft Excel y Microsoft Project, también proporcionan una lista de constantes que puede usar con sus objetos, métodos y propiedades. También se definen constantes en la biblioteca de objetos de cada control **ActiveX**. Para obtener más detalles acerca de la utilización del Examinador de objetos, vea "Programar con objetos".

Las constantes simbólicas o definidas por el usuario: se declaran mediante la instrucción **Const**. Las constantes definidas por el usuario se describen en la próxima sección, "Crear sus propias constantes".

En Visual Basic, los nombres de *constantes* tienen un formato que combina mayúsculas y minúsculas, con un prefijo que indica la biblioteca de objetos que define la constante. Las *constantes* de las bibliotecas de objetos de **Visual Basic** y **Visual Basic** para aplicaciones tienen el prefijo "vb"; por ejemplo, **vbTileHorizontal**.

- 4.2.4.1 Declaración de constantes

La sintaxis para declarar una constante es la siguiente:

Ing. Carlos Manuel Rodríguez Bucarellly



El argumento **NombreConstante** es un nombre simbólico válido (Las reglas son las mismas que para crear variables), y **Expresión** está compuesta por constantes y operadores de cadena o números. Sin embargo, no puede usar llamadas a funciones en expresión.

La declaración de una constante puede ser de distintos tipos: **Pública** (dentro de un módulo), **Privada** (en el área de declaraciones general de un formulario) o **Local** (dentro de un procedimiento).

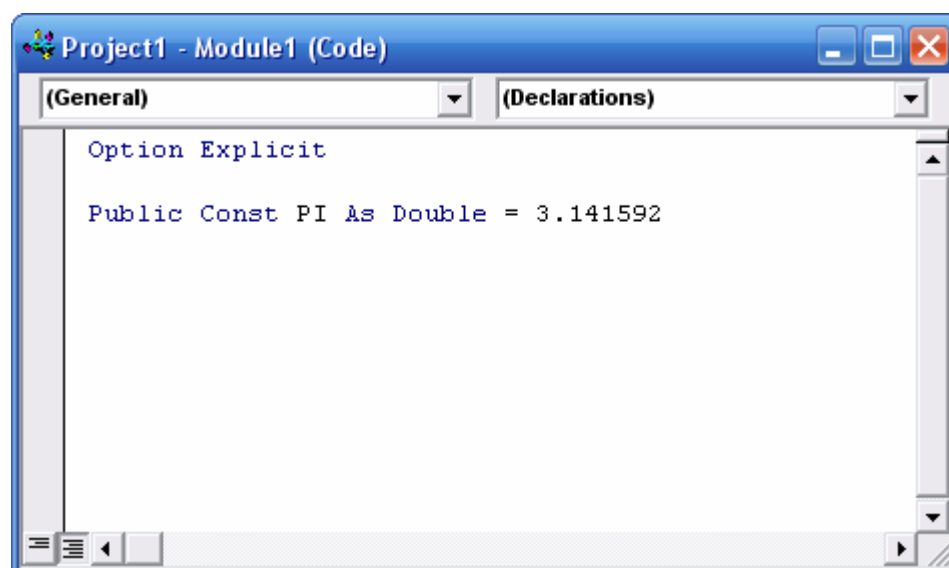
- 4.2.4.2 Declaración de constantes públicas

Declarar una *constante pública* significa que esa constante podrá ser utilizada desde cualquier procedimiento, formulario o módulo que contenga la aplicación. Para declarar una constante pública siga los siguientes pasos:

Para declarar una constante pública siga los siguientes pasos:

1. Inserte un módulo desde el menú **Project/Add Module**.
2. Dentro de ese módulo escriba la instrucción **Public** seguida de la instrucción **Const** y, a continuación, el *nombre de la constante* con su *tipo asociado*.
3. Por último, ponga el valor que tendrá la constante después del signo de igual.

Ejemplo:



Después de hacer esta declaración puede usar la constante **PI** para hacer cualquier cálculo matemático dentro de cualquier procedimiento (un botón de comando, una caja de lista, un formulario, etc.) sin la necesidad de hacer referencia al módulo que la contiene.

Para probar esto hagamos una aplicación que nos permita calcular el volumen de un cilindro. El volumen de un cilindro se calcula mediante la siguiente fórmula: $V = PI * Radio * Radio * Altura$. El radio y la altura deberán ser introducidos por el usuario mediante cajas de texto. El programa mostrará el volumen calculado en otra caja de texto.

- En el mismo proyecto que creó la constante inserte tres etiquetas y al lado de cada etiqueta una caja de texto. En la parte inferior derecha del formulario inserte dos botones de comando, uno para Calcular el volumen y el otro para salir de la aplicación, tal y como se muestra en la figura de la siguiente página:



- Aplique los siguientes valores en las propiedades indicadas de los controles que se muestran en la siguiente tabla:

Control	Propiedad	Valor
Form1	Caption	Volumen de un cilindro
Label1	AutoSize Caption	True Introduzca el radio:
Text1	Name Text	txtRadio
Label2	AutoSize Caption	True Introduzca la altura:
Text2	Name Text	txtAltura
Label3	AutoSize Caption	True Resultado:
Text3	Name Locked Text	txtResultado True
Command1	Name Caption	cmdCalcular &Calcular
Command2	Name Caption	cmdSalir &Salir

- Hagamos que las dos primeras cajas solo admitan números. Dentro del evento **KeyPress** de las dos primeras cajas de texto escriba:

```

'Hace que la caja de texto solo acepte números.
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then
KeyAscii = 0
End If
    
```



- Haga doble clic sobre el primer botón de comando y dentro del evento **Click** escriba:

```
'Verificamos que se introduzca el radio.  
If Len(Trim(txtRadio.Text)) = 0 Then  
    MsgBox ("Debe introducir el radio.")  
    txtRadio.SetFocus ' Hace que la caja del radio reciba el enfoque.  
  
'Verificamos que se introduzca la altura.  
Elseif Len(Trim(txtAltura.Text)) = 0 Then  
    MsgBox ("Debe introducir la altura.")  
    txtAltura.SetFocus ' Hace que la caja de la altura reciba el enfoque.  
  
'Si todo esta bien calculamos el volumen.  
Else  
  
    Dim Radio As Double          ' Definimos la variable para el radio.  
    Dim Altura As Double         ' Definimos la variable para la altura.  
    Dim Resultado As Double      ' Definimos la variable para el resultado.  
  
    'Almacenamos el valor de la caja del Radio en la variable Radio.  
    Radio = CDb(txtRadio.Text)  
  
    'Almacenamos el valor de la caja de la altura en la variable altura.  
    Altura = CDb(txtAltura.Text)  
  
    'Calculamos el volumen del cilindro.  
    Resultado = PI * Radio * Radio * Altura  
  
    'Mostramos el resultado en la caja del resultado.  
    txtResultado.Text = Resultado  
  
End If
```

- En el evento **Click** del segundo botón de comando escriba:

```
'Sale de la aplicación.  
End
```

- Corra la aplicación.
- Escriba un valor en la caja del radio y un valor en la caja de la altura. Después, haga clic en el botón Calcular. Podrá observar que el programa le muestra el resultado, es decir, el volumen del cilindro.

Lo importante en esta aplicación es la utilización de la constante **PI** que declaramos en el modulo. Si observa en la formula que utilizamos dentro del botón de comando podrá observar que especificamos el nombre de la constante en vez de especificar el valor que esta contiene aunque la formula interpreta el nombre de la constante PI por el valor 3.141592. Tenga en cuenta que podrá utilizar esta constante cuantas veces desee dentro de la misma aplicación.

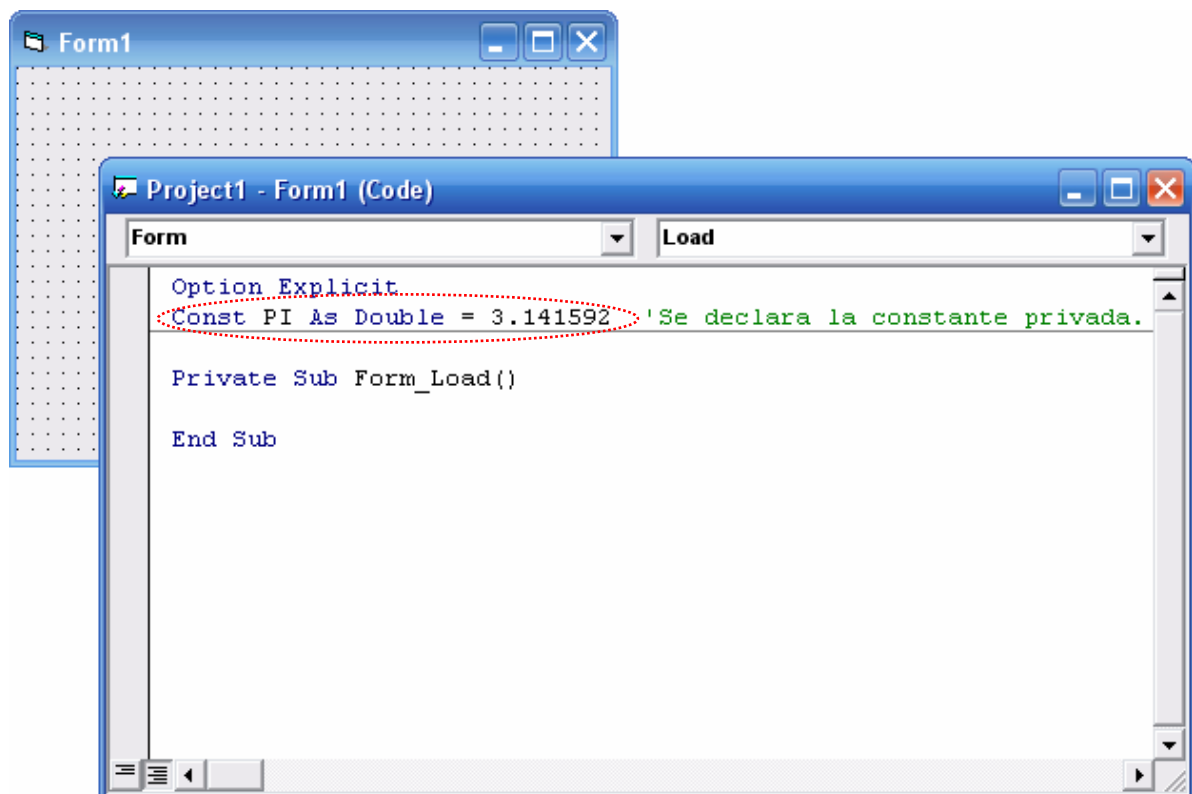
- Detenga la aplicación y guarde el ejercicio con los nombres **ModuloEjercicio1-4** para el módulo, **FormEjercicio1-4** para el formulario y **Ejercicio1-4** para el proyecto.



- 4.2.4.3 Declaración de constantes privadas

Declarar una *constante privada* significa que esa *constante* puede ser usada dentro de todos los procedimientos de un mismo formulario o dentro del formulario donde se declara la *constante*. Para declarar una *constante privada* siga los siguientes pasos:

a) Haga doble clic sobre cualquier parte del formulario y en la sección **(General)** del formulario escriba la instrucción **Const** seguida del nombre de la constante, su tipo asociado y su valor. Tenga mucho cuidado de no declarar la constante dentro del evento **Load** del formulario.



La zona encerrada en el círculo es donde debe de declarar la variable privada. Esta es la sección general de un formulario. Todas las variables y constantes que se declaren en esta zona serán de índole privada.

b) Al hacer esto puede utilizar la constante PI desde cualquiera de los procedimientos dentro de ese formulario, es decir, puede agregar un botón de comando, una caja de lista, una etiqueta, en fin, cualquier control en general y hacer referencia a esta constante solo por su nombre desde cualquiera de los eventos de dicho control. Por ejemplo, agregue un botón de comando al formulario y dentro del evento **Click** escriba:

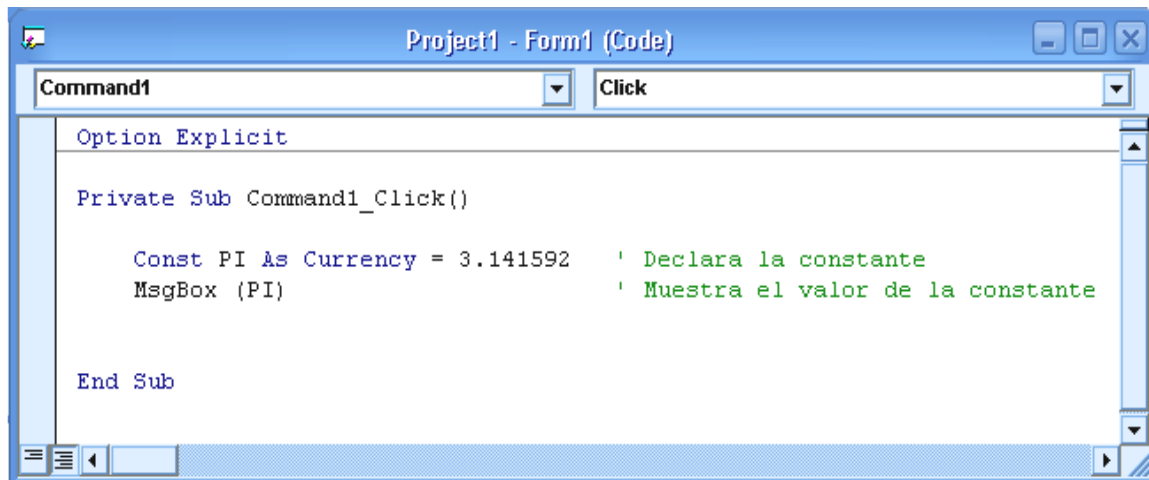
```
MsgBox (PI * 3) 'Multiplica el valor del PI por tres y muestra el resultado en la pantalla.
```

- 4.2.4.4 Declaración de constantes locales

Declarar una *constante local* significa que esa *constante* solo puede ser usada dentro del procedimiento donde se declara. Para declarar una *constante local* escriba la instrucción **Const** seguida del nombre de la constante, su tipo asociado y su valor dentro de un procedimiento o evento.



Ejemplo:



```
Project1 - Form1 (Code)
Command1 Click
Option Explicit

Private Sub Command1_Click()

    Const PI As Currency = 3.141592    ' Declara la constante
    MsgBox (PI)                        ' Muestra el valor de la constante

End Sub
```

- 4.2.5 Variables

En Visual Basic las *variables* se utilizan para almacenar temporalmente valores durante la ejecución de la aplicación. Las variables tienen un nombre que nos permite acceder al valor que tiene la variable y un tipo de dato asociado que nos permite determinar la clase de datos que la variable puede almacenar. Una variable se puede definir también como un marcador de posición en la memoria del computador donde se almacena un valor desconocido. Por ejemplo, si ha creado un programa para gestionar las ventas de computadoras de una empresa y usted no sabe el precio de las computadoras ni la cantidad de computadora a vender, puede utilizar dos variables para almacenar los valores desconocidos, vamos a llamarles a esas variables *PrecioComputadora* y *CantidadComputador*. Cada vez que se ejecute el programa, el usuario deberá especificar los valores para estas dos variables posiblemente mediante dos cajas de texto. Para calcular el total de la venta y mostrarlo en una caja de texto llamada *txtTotal*, el código debería parecerse al siguiente.

'Almacena el precio de la computadora en la variable PrecioComputadora contenido en la caja.

PrecioComputadora = CDb1(txtPrecioComputadora.Text)

'Almacena la cantidad de computadora a comprar en la variable PrecioComputadora contenida en la caja.

CantidadComputador = CInt(txtCantidadComputadora.Text)

'Calcula el precio total y almacena el resultado en la caja txtTotal.

txtTotal.Text = PrecioComputadora * CantidadComputadora

La expresión devuelve un total distinto cada vez, dependiendo de los valores que indique el usuario. Las *variables* le permiten realizar un cálculo sin tener que saber antes cuáles son los valores especificados.

- 4.2.5.1 Declaración de una variable

Declarar una variable consiste en indicarle de antemano al programa el lugar donde se almacenaran los datos desconocidos. Al declarar una variable se debe especificar *el nivel de alcance de la variable, el nombre y el tipo de datos asociado a dicha variable*.

Antes de declarar una variable debe de conocer los distintos tipos de *nivel de alcance* de una variable. El *nivel de alcance* le va a permitir conocer las distintas zonas desde donde podrá utilizar las variables. Existen tres tipos de *nivel de alcance* que son: *público, privado y local*. En los siguientes temas tratamos cada uno de estos y su forma de establecerlo en una variable.



- 4.2.5.1.1 Declaración de una variable pública

Al igual que las constantes públicas una *variable pública* podrá ser utilizada desde cualquier parte de la aplicación. Estas se declaran a nivel de módulos mediante el identificador **Public**. Una variable pública es declarada mediante el siguiente formato:

Public nombre_variable **As** Tipo_de_datos

Donde:

Public : Indica que la variable es de nivel público.

nombre_variable: Es un identificador valido para Visual Basic. Este es el nombre que le permitirá acceder a los valores desconocidos.

As : Palabra clave para indicar el tipo de datos.

Tipo_de_datos : Indica el tipo de datos que podrá almacenar las variables.
Este corresponde a la clasificación de los tipos de datos.

Tiene que tener en cuenta que no todas las variables pueden ser de tipo pública por el simple echo de que sea posible hacerlo. El *nivel de alcance* de la variable va a depender del uso que le va a dar a la variable. Las variables públicas son utilizadas principalmente para almacenar valores que serán utilizados en la aplicación a nivel general. Por ejemplo, si crea una aplicación que va a requerir el uso de la impresora y desea informar al usuario cuando esta no se encuentre disponible en el computador cada vez que el programa requiera su uso. Podrá verificar al iniciar la aplicación, si el computador dispone de una impresora y almacenar un valor en una variable pública que luego va a servir para comprobar o consultar si el computador tiene o no una impresora disponible.

- 4.2.5.1.2 Declaración de una variable privada

Son variables que pueden ser utilizadas dentro de un mismo módulo o formulario. El alcance de este tipo de variable será de nivel general dentro de un mismo modulo o formulario. Se declaran a nivel de módulos o en la *sección general* de un formulario mediante el identificador **Dim**. Estas variables son muy utilizadas pero debe de controlar su uso. Tiene que tener en cuenta, que si declara una variable de *nivel privado* no podrá declarar la misma variable de *nivel público* o *local* en la misma aplicación. De esta misma forma, si declara una *variable pública* no podrá declara la misma variable a *nivel privado* o *local*. Una variable de *nivel privado* se declara bajo el siguiente formato:

Dim nombre_variable **As** Tipo_de_datos

Ejemplos:

Dim TotalVentasDelDia **As** Integer

Dim FacturasImpresa **As** Integer

- 4.2.5.1.3 Declaración de una variable local

Sin lugar a dudas, es el nivel de alcance más utilizado y recomendado para una variable. Estas *variables* se declaran en un procedimiento o evento mediante la instrucción **Dim** y sólo existen mientras se ejecuta el procedimiento. Cuando termina el procedimiento, desaparece el valor de la variable. Además, el valor de una variable de un procedimiento es local de dicho procedimiento; es decir, no puede tener acceso a una variable de un procedimiento desde otro procedimiento. Estas características le permiten usar los mismos nombres de *variables* en distintos procedimientos sin preocuparse por posibles conflictos o modificaciones accidentales.



- 4.2.5.2 Nombre descriptivos de las variables

No existe un modelo a seguir para establecer o asignar un nombre adecuado a una variable, esto va a depender de su criterio como programador. Pero por experiencia considere algunos consejos que le doy como programador antes de definir una variable: 1) No utilice nombres muy largos para las variables, 2) Utilice nombres que describan de forma muy reducida la función de la variable, 3) No utilice variables incoherentes en procesos de cálculos matemáticos exactos, 4) Utilice variables incoherentes para procesos de calculo aleatorios. Evitará muchos problemas futuros si considera los consejos anteriores. Debe de tener en cuenta que una variable esta sometida a las reglas de los identificadores en Visual Basic.

- 4.2.5.3 Almacenar y recuperar datos en variables

Para almacenar un valor en una variable utilice el signo de igualdad "=", que es considerado también como operador de asignación.

Almacenando valores en las variables:

- 1) ManzanasVendidas = 10 ' Se pasa el valor 10 a la variable.
- 2) ManzanasVendidas = ManzanasVendidas + 1 ' Se incrementa la variable.
- 3) CantidadComprar = Cint(txtCantidadComprar.Text) ' Se almacena el valor contenido en la caja.

Leyendo los datos de una variable:

Para leer los datos de una variable especifique primero el lugar donde se almacenará los valores que contiene la variable, tal y como se muestra en los siguientes ejemplos:

- 1) MsgBox(ManzanasVendidas) ' Muestra el valor de la variable en una caja de mensaje.
- 2) txtMazanasVendias.Text = MazanasVendias 'Pone el valor de la variable en la caja de texto.
- 3) txtCantidadComprar.Text = CantidadComprar 'Pone el valor de la variable en la caja de texto.

- 4.2.5.4 Operaciones aritméticas con variables

Las operaciones aritméticas se realizan a través de una expresión o formula matemática. Las formulas matemáticas están compuesta por un conjunto de variables, escritas mediante operadores de igualdad y los operadores matemáticos.

Toda expresión o formula matemática en todas las áreas del saber son representadas mediante símbolos o variables que luego son sustituidas por valores reales proporcionados por el usuario. De esta misma manera se realizan las operaciones aritméticas con variables en Visual Basic. El usuario deberá sustituir los valores de cada variable por valores reales utilizando la operación de asignación.

Por ejemplo:

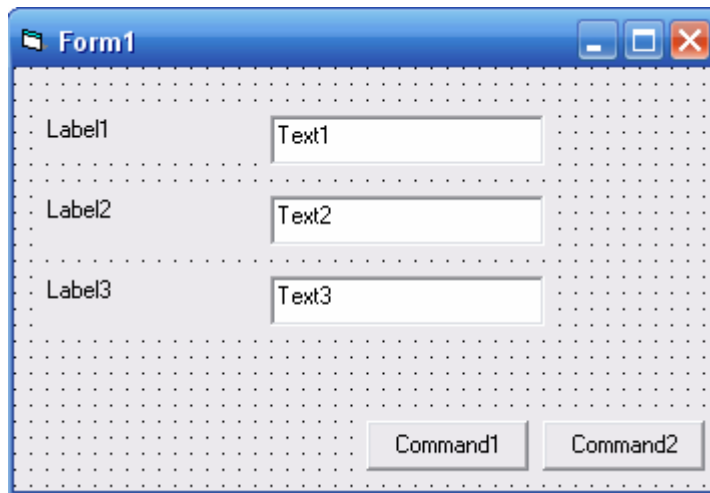
Si usted requiere diseñar un programa que permita calcular el sueldo bruto de un empleado a partir de las horas trabajadas y el precio por hora, sabiendo que el sueldo bruto es obtenido multiplicando las horas trabajadas por el precio por hora. Deberá expresarlo con una formula similar a la siguiente: SueldoBruto = HorasTrabajadas * PrecioPorHoras, o más resumido, SB = HT * PH.



Para resolver este problema deberá definir tres variables y permitir al usuario introducir los valores desconocidos, que en este caso son las horas trabajadas y el precio por hora. Podrá utilizar cajas de texto para asignar en las variables los valores proporcionados por el usuario.

Para crear la aplicación anterior siga los siguientes pasos:

- Abra un nuevo proyecto.
- Inserte tres etiquetas, tres cajas de texto y dos botones de comando, tal y como se muestra en la siguiente figura:



- Aplique los siguientes valores en las propiedades indicada de los controles que se muestran en la siguiente tabla:

Control	Propiedad	Valor
Label1	AutoSize Caption	True Horas trabajadas:
Label2	AutoSize Caption	True Precio por hora:
Label3	AutoSize Caption	True Sueldo Bruto:
Text1	Name Text	txtHorasTrabajadas
Text2	Name Locked Text	txtPrecioHora True
Text3	Name Text	txtSueldoBruto
Command1	Name Caption	cmdCalcular &Calcular
Command2	Name Caption	cmdSalir &Salir
Form1	Caption	Sueldo Bruto



- Dentro del evento **KeyPress** de las dos primeras cajas de texto escriba el siguiente bloque de código para que solo admitan números:

```
'Hace que la caja de texto solo acepte números.  
If Not (KeyAscii >= 48 And KeyAscii <= 57 Or KeyAscii = 8) Then  
KeyAscii = 0  
End If
```

- Haga doble clic sobre el botón Calcular y dentro del evento **Click** escriba:

```
'Verificamos que las cajas no se dejen vacías.  
If Len(Trim(txtHorasTrabajadas.Text)) = 0 Then  
MsgBox ("Introduzca las horas trabajadas.")  
txtHorasTrabajadas.SetFocus  
Elseif Len(Trim(txtPrecioHora.Text)) = 0 Then  
MsgBox ("Introduzca el precio por hora.")  
txtPrecioHora.SetFocus  
Else  
  
Dim horas_trabajadas As Integer      ' Variable para almacenar las horas trabajadas.  
Dim precio_horas As Currency         ' Variable para almacenar el precio por horas.  
Dim sueldo_bruto As Currency         ' Variable para almacenar el sueldo bruto.  
  
horas_trabajadas = CInt(txtHorasTrabajadas.Text) ' Almacena las horas escrita en la caja en la  
                                                    ' variable.  
  
precio_horas = CCur(txtPrecioHora.Text)         ' Almacena el precio por hora escrita en la caja en la  
                                                    ' variable.  
  
sueldo_bruto = horas_trabajadas * precio_horas    ' Calcula el sueldo bruto.  
txtSueldoBruto.Text = FormatCurrency(CDbl(sueldo_bruto)) ' Escribe el sueldo bruto en la caja.  
  
End If
```

- Dentro del evento **Click** del botón salir.

```
'Sale de la aplicación.  
End
```

- Corra la aplicación.
- Introduzca un valor en la caja horas trabajadas y un valor en la caja precio por hora.
- Haga clic en el botón calcular y podrá ver los resultados.

Explicación del código:

En esta aplicación anterior, cabe destacar la forma en que almacenamos en las variables los valores contenidos en las cajas de texto. Para las dos primeras cajas de texto hemos declarado una variable independiente. En estas variables se almacenan los valores introducidos por el teclado, en nuestro caso, las horas trabajadas y el precio por hora. Cuando asignamos estos valores en las variables mediante el operador de asignación "=", procedemos a calcular el sueldo bruto mediante la formula `sueldo_bruto = horas_trabajadas * precio_horas`. Cuando obtenemos el resultado, lo mostramos en la tercera caja de texto, que es la caja destinada para escribir el sueldo bruto calculado.



▪ Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio2-4** para el formulario y **Ejercicio2-4** para el proyecto.

- 4.2.5.5 Ejercicios propuestos

1) Se requiere calcular el área de un triángulo: **area = (base * altura)/2**, la base y la altura deben ser introducidas mediante cajas de texto. El programa debe mostrar el área del triángulo en una caja de texto.

2) Se requiere calcular el volumen de una esfera: **volumen = 4/3 * PI * radio * radio * radio**.

3) Se requiere evaluar la siguiente formula: $X1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$ (es la solución positiva de una ecuación de segundo grado). Los datos deben ser proporcionados mediante cajas de texto.

4) Se requiere calcular el área y el volumen de un cilindro: **area = (2 * (PI * radio*radio)) + ((2 * PI * radio) * h)** y **volumen = (PI * radio*radio) * h**.

5) Se requiere calcular las raíces de una ecuación de segundo grado: $x1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$ y $x2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$. Los datos deben ser proporcionados mediante cajas de texto.

- 4.2.6 Operaciones de entrada y salida

Visual Basic proporciona dos herramientas rápidas y simples para permitir que el usuario introduzca datos y lo muestre en la pantalla. Estas dos herramientas son las funciones **InputBox** y **MsgBox**.

- 4.2.6.1 Función InputBox

Muestra un mensaje en un cuadro de diálogo, espera que el usuario escriba un texto o haga clic en un botón y devuelve un tipo *String* con el contenido del cuadro de texto.

Su formato es:

InputBox(prompt[, title][, default][, xpos][, ypos][, helpfile, context])

Donde:

Prompt	Es una expresión de tipo cadena requerida por la función. Este parámetro indica el mensaje que aparecerá en el cuadro de diálogo. La longitud máxima de prompt es de aproximadamente 1024 caracteres, según el ancho de los caracteres utilizados. Si prompt consta de más de una línea, puede separarlos utilizando un carácter de retorno de carro (Chr(13)), un carácter de avance de línea (Chr(10)) o una combinación de los caracteres de retorno de carro-avance de línea (Chr(13) y Chr(10)) entre cada línea y la siguiente.
title	Opcional. Expresión de cadena que indica el texto que aparecerá en la barra de título del cuadro de diálogo. Si se omite title , se muestra en la barra de título el nombre de la aplicación.
default	Opcional. Expresión de cadena que se muestra en el cuadro de texto como respuesta predeterminada cuando no se suministra una cadena. Si omite default , se muestra el cuadro de texto vacío.



xpos	Opcional. Expresión numérica que especifica, en twips, la distancia en sentido horizontal entre el borde izquierdo del cuadro de diálogo y el borde izquierdo de la pantalla. Si se omite xpos , el cuadro de diálogo se centra horizontalmente.
ypos	Opcional. Expresión numérica que especifica, en twips, la distancia en sentido vertical entre el borde superior del cuadro de diálogo y el borde superior de la pantalla. Si se omite ypos , el cuadro de diálogo se coloca a aproximadamente un tercio de la altura de la pantalla, desde el borde superior de la misma.
helpfile	Opcional. Expresión de cadena que identifica el archivo de Ayuda que se utilizará para proporcionar ayuda interactiva para el cuadro de diálogo. Si se especifica helpfile , también deberá especificarse context .
context	Opcional. Expresión numérica que es el número de contexto de Ayuda asignado por el autor al tema de Ayuda correspondiente. Si se especifica context , también deberá especificarse helpfile .

Ejercicio:

- Abra un nuevo proyecto e inserte un botón de comando.
- Dentro del evento **Click** del botón de comando escriba:

'Se declaran las variables donde se almacenará el nombre y el apellido.

Dim nombre As String

Dim apellido As String

'Permite que el usuario introduzca su nombre y lo almacena en la variable nombre.

nombre = InputBox("Introduzca su nombre:", "Datos personales")

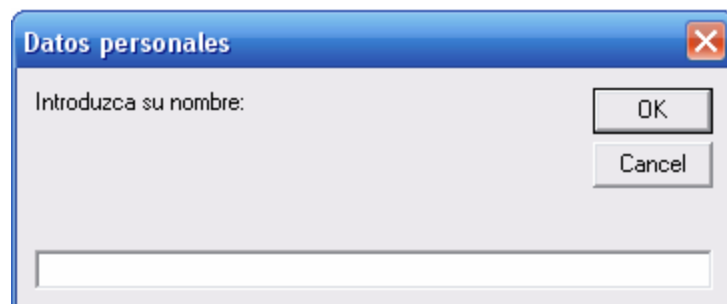
'Permite que el usuario introduzca su apellido y lo almacena en la variable apellido.

apellido = InputBox("Introduzca su apellido:", "Datos personales")

'Muestra en una caja de mensaje el nombre completo.

MsgBox ("Su nombre completo es: " & nombre & " " & apellido)

- Corra la aplicación.
- Aparecerá el siguiente cuadro de diálogo:



- Escriba su nombre en la caja de texto y, a continuación, haga clic en el botón OK.
- Aparecerá otro cuadro de diálogo similar solicitando su apellido. Escriba el apellido y haga clic en el botón OK.
- Aparecerá luego en una caja de mensaje su nombre y su apellido.



- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio3-4** para el formulario y **Ejercicio3-4** para el proyecto.

Explicación:

Lo importante en esta aplicación es la forma rápida y sencilla de almacenar en variable datos proporcionados por el usuario. Mediante un cuadro personalizado generado por la función **InputBox** se puede almacenar en una variable cualquier valor proporcionado por el usuario.

En nuestra aplicación hemos definido dos variables: una para el nombre y otra para el apellido, ambas de tipo *String*. Después de esto, hemos utilizado la función **InputBox** delante de cada variable unidos mediante el *operador de asignación* **igualdad** “=” para que el valor proporcionado por el usuario se asigne en las variables.

Por último, hemos utilizado la función **MsgBox** para mostrar el nombre y el apellido proporcionado por el usuario. Hemos combinado los valores de cada variable mediante el *operador de concatenación* **ampersand** “&”.

NOTA: Podrá utilizar la función **InputBox** para solicitar todo tipo de datos y luego utilizar *funciones de conversión de tipo de datos* para realizar operaciones de cálculo de forma adecuada.

- 4.2.6.2 Función MsgBox

Muestra un mensaje en un cuadro de diálogo, espera a que el usuario haga clic en un botón y devuelve un tipo **Integer** correspondiente al botón elegido por el usuario.

Su formato es:

MsgBox(*prompt*[, *buttons*][, *title*][, *helpfile*, *context*])

Donde:

<i>prompt</i>	Es una expresión de tipo cadena requerida por la función. Este parámetro indica el mensaje que aparecerá en el cuadro de diálogo. La longitud máxima de <i>prompt</i> es de aproximadamente 1024 caracteres, según el ancho de los caracteres utilizados. Si <i>prompt</i> consta de más de una línea, puede separarlos utilizando un carácter de retorno de carro (Chr(13)), un carácter de avance de línea (Chr(10)) o una combinación de los caracteres de retorno de carro-avance de línea (Chr(13) y Chr(10)) entre cada línea y la siguiente.
<i>buttons</i>	Opcional. Expresión numérica que corresponde a la suma de los valores que especifican el número y el tipo de los botones que se pretenden mostrar, el estilo de icono que se va a utilizar, la identidad del botón predeterminado y la modalidad del cuadro de mensajes. Si se omite este argumento, el valor predeterminado para <i>buttons</i> es 0.
<i>title</i>	Opcional. Expresión de cadena que se muestra en la barra de título del cuadro de diálogo. Si se omite <i>title</i> , en la barra de título se coloca el nombre de la aplicación.
<i>helpfile</i>, <i>context</i>	Se utiliza de la misma manera que en la función InputBox .



El argumento **buttons** tiene los siguientes valores:

Constante	Valor	Descripción
VbOKOnly	0	Muestra solamente el botón Aceptar .
VbOKCancel	1	Muestra los botones Aceptar y Cancelar .
VbAbortRetryIgnore	2	Muestra los botones Anular , Reintentar e Ignorar .
VbYesNoCancel	3	Muestra los botones Sí , No y Cancelar .
VbYesNo	4	Muestra los botones Sí y No .
VbRetryCancel	5	Muestra los botones Reintentar y Cancelar .
VbCritical	16	Muestra el icono de mensaje crítico .
VbQuestion	32	Muestra el icono de pregunta de advertencia .
VbExclamation	48	Muestra el icono de mensaje de advertencia .
VbInformation	64	Muestra el icono de mensaje de información .
VbDefaultButton1	0	El primer botón es el predeterminado.
VbDefaultButton2	256	El segundo botón es el predeterminado.
VbDefaultButton3	512	El tercer botón es el predeterminado.
VbDefaultButton4	768	El cuarto botón es el predeterminado.
VbApplicationModal	0	Aplicación modal; el usuario debe responder al cuadro de mensajes antes de poder seguir trabajando en la aplicación actual.
VbSystemModal	4096	Sistema modal; se suspenden todas las aplicaciones hasta que el usuario responda al cuadro de mensajes.
VbMsgBoxHelpButton	16384	Agrega el botón Ayuda al cuadro de mensaje.
VbMsgBoxSetForeground	65536	Especifica la ventana del cuadro de mensaje como la ventana de primer plano.
VbMsgBoxRight	524288	El texto se alinea a la derecha.
VbMsgBoxRtlReading	1048576	Especifica que el texto debe aparecer para ser leído de derecha a izquierda en sistemas hebreo y árabe.

Los valores devueltos por la función según la respuesta proporcionada por el usuario se muestran en la siguiente tabla:

Constante	Valor	Descripción
vbOK	1	Aceptar
vbCancel	2	Cancelar
vbAbort	3	Anular
vbRetry	4	Reintentar
vbIgnore	5	Ignorar
vbYes	6	Sí
vbNo	7	No

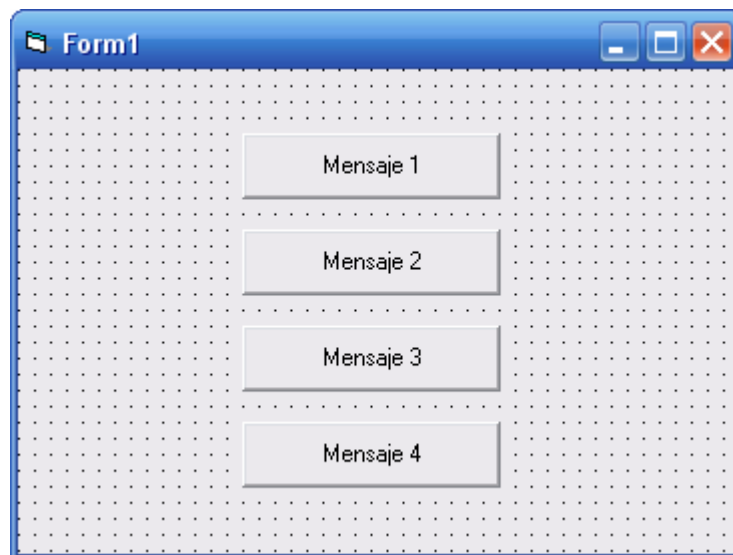


Ejercicio:

El siguiente ejercicio muestra como utilizar la función **MsgBox** para mostrar cuadro de diálogos personalizados.

- Abra un nuevo proyecto.
- Inserte cuatro botones de comando uno debajo del otro.
- En la propiedad **Caption** del primer botón de comando escriba: "Mensaje 1".
- En la propiedad **Caption** del segundo botón de comando escriba: "Mensaje 2".
- En la propiedad **Caption** del tercer botón de comando escriba: "Mensaje 3".
- En la propiedad **Caption** del cuarto botón de comando escriba: "Mensaje 4".

La aplicación debe tener una apariencia similar a la mostrada en la siguiente imagen:



- Dentro del evento **Click** del primer botón de comando escriba:

'Muestra un mensaje en un cuadro de diálogo sencillo con los botones Si y No, y el icono de mensaje crítico.

Dim respuesta

respuesta = MsgBox("¿Esta seguro que desea formatear su computador?", vbYesNo + vbCritical + vbDefaultButton2, "Formatear Disco")

'Selecciona la respuesta del usuario.

If respuesta = vbYes Then

MsgBox ("Usted selecciono la respuesta Sí.")

End If

If respuesta = vbNo Then

MsgBox ("Usted selecciono la respuesta No.")

End If

- Dentro del evento **Click** del segundo botón de comando escriba:



'Muestra un mensaje en un cuadro de diálogo personalizado con los botones, si, no y cancelar.
'En el cuadro se muestra el icono de mensaje de advertencia.

```
Dim respuesta  
respuesta = MsgBox("¿Desea guardar los cambios en el documento?", vbYesNoCancel +  
vbExclamation + vbDefaultButton1, "Editor")
```

'Selecciona la respuesta del usuario.

```
If respuesta = vbYes Then  
MsgBox ("Usted selecciono la respuesta Sí.")  
End If
```

```
If respuesta = vbNo Then  
MsgBox ("Usted selecciono la respuesta No.")  
End If
```

```
If respuesta = vbCancel Then  
MsgBox ("Usted selecciono la respuesta Cancelar.")  
End If
```

- Dentro evento **Click** del tercer botón de comando:

'Muestra un mensaje en un cuadro de diálogo personalizado con los botones, Abortar, Reintentar e Ignorar. En el cuadro se muestra el icono de mensaje de crítico.

```
Dim respuesta  
respuesta = MsgBox("No se puede copiar el archivo.", vbAbortRetryIgnore + vbCritical, "Copiando  
archivo")
```

'Selecciona la respuesta del usuario.

```
If respuesta = vbAbort Then  
MsgBox ("Usted selecciono la respuesta Abortar.")  
End If
```

```
If respuesta = vbRetry Then  
MsgBox ("Usted selecciono la respuesta Reintentar.")  
End If
```

```
If respuesta = vbIgnore Then  
MsgBox ("Usted selecciono la respuesta Ignorar.")  
End If
```

- Dentro del evento **Click** del cuarto botón de comando:

'Muestra un mensaje en un cuadro de diálogo personalizado el botón Aceptar y Cancelar.
'En el cuadro se muestra el icono de información.

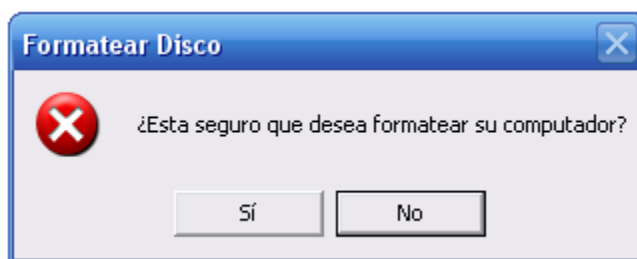
```
MsgBox ("Los datos se almacenaron satisfactoriamente."), vbOKOnly, "Guardar"
```



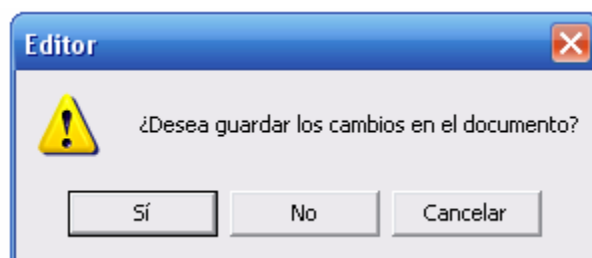
- Corra la aplicación.
- Haga clic sobre cada uno de los botones para que observe los distintos cuadro de dialogo.
- Analice el código escrito en cada botón de comando y observe las diferentes formas de personalización de la función **MsgBox**.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio4-4** para el formulario y **Ejercicio4-4** para el proyecto.

En caso de no tener un computador disponible los cuadros de diálogo que se muestran en el ejercicio anterior son los siguientes:

Primer botón:



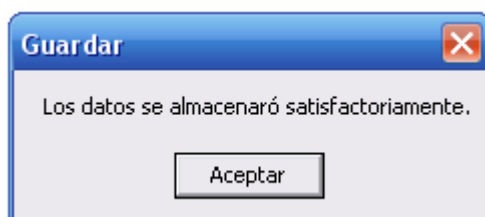
Segundo botón:



Tercer botón:



Cuarto botón:





4.3 ESTRUCTURAS DE CONTROL SELECTIVAS

Con mucha frecuencia nos encontraremos con la necesidad de hacer comparaciones lógicas y tomar decisiones según los datos proporcionados por el usuario. Estas decisiones y comparaciones las realizamos utilizando las *estructuras de control selectivas*. Gracias a estas, el programador podrá evaluar entre varias alternativas y decidir que hacer para cada alternativa dentro del programa.

Las estructuras de control selectivas están presentes en cuanto a conceptos en todos los lenguajes de programación. Son utilizadas para tomar decisiones con los datos proporcionados por el usuario o, bien, con la información obtenida mediante algún proceso. Vienen representadas comúnmente por las palabras en inglés **if-then-else** y **case**, aunque en algunos lenguajes de programación presentan mínimas variaciones.

- 4.3.1 Expresiones lógicas

Una **expresión lógica** es una combinación de variables, constantes, valores literales, funciones y operadores de relación que dan como resultado un valor único, *verdadero* o *falso*. Las expresiones lógicas se clasifican en: **expresiones lógicas simples** y **expresiones lógicas compuestas**.

Las **expresiones lógicas simples** son aquellas que están compuestas por una única expresión lógica, tal y como se muestran en los siguientes ejemplos:

1. $(a > b)$ 'Esta expresión evalúa si el valor de la variable **a** es mayor que el de la variable **b**. Si el valor de **a** es mayor que el de **b**, devuelve el valor lógico **true**, de lo contrario, devuelve el valor lógico **false**.
2. $(5 + 3 = b)$ 'Esta expresión evalúa si la suma de **5+3** es igual al valor de la variable **b**. Si la suma de **5+3** es igual al valor que contiene la variable **b**, devuelve el valor lógico **true**, de lo contrario, devuelve **false**.
3. $(a + b > c)$ 'Esta expresión evalúa si la suma de **a+b** es mayor que el valor de la variable **c**. Devuelve **true** si **a+b** es mayor que **c**, de lo contrario, devuelve **false**.
4. $(n \neq 0)$ 'Esta expresión evalúa si el valor de la variable **n** es distinto de **cero**. Devuelve **true** si el valor de **n** es distinto de **cero**, de lo contrario, devuelve **false**.
5. $(n + 1 = 3)$ 'Esta expresión evalúa si la suma de **n+1** es igual a **tres**. Devuelve **true** si la suma de del valor de la variable **n + 1** es igual a **tres**, de lo contrario, devuelve **false**.

Las **expresiones lógicas compuestas** son aquellas que contienen varias expresiones lógicas relacionadas mediante operadores lógicos (**and**, **or**, **not**).

Ejemplos:

1. $(a > b) \text{ and } (a > c)$ 'Esta expresión evalúa si el valor de **a** es mayor que el valor de **b**, y si el valor de **a** es mayor que **c**. Si ambas expresiones son verdaderas, devuelve el valor lógico **true**, de lo contrario, devuelve **false**.
2. $(a = 3) \text{ or } (a = 4)$ 'Esta expresión evalúa si **a** es igual a tres ó **a** es igual a 4. Si uno de las dos expresiones lógicas es verdadera devuelve el valor lógico **true**, de lo contrario, devuelve **false**.



- 4.3.1.1 Operadores aritméticos

Son aquellos que se utilizan para realizar las operaciones básicas de las matemáticas. En las operaciones básicas tenemos: suma, resta, multiplicación, división, residuo y exponenciación.

Operador	Significado	Ejemplo	Resultado
+	Suma	$a + b$	Suma de a y b.
-	Resta	$a - b$	Diferencia entre a y b.
*	Multiplicación	$a * b$	Producto de a por b.
/	División	a / b	Cociente de a sobre b.
\	División entera	$a \setminus b$	Cociente entero de a sobre b.
Mod	Módulo o Resto	$a \text{ mod } b$	Resto de a sobre b.
^	Exponenciación	$a ^ b$	Potencia de a elevado a b.

- 4.3.1.2 Operadores de relación

Los *operadores de relación* son utilizados para expresar condiciones y describir una relación entre dos valores. Los operadores de relación se muestran en la siguiente tabla:

Operador	Significado	Equivalente matemático
>	Mayor que	>
<	Menor que	<
=	Igual a	=
>=	Mayor o igual que	≥
<=	Menor o igual que	≤
<>	Distinto a	≠

Ejemplos de algunas relaciones:

1. $5 > 3$ Devuelve verdadero (true)
2. $3 = 2 + 1$ Devuelve verdadero (true)
3. $2 <> 1 + 1$ Devuelve falso (false)
4. $8 < 5$ Devuelve falso (false)
5. $'A' < 'B'$ Devuelve verdadero (true)
6. $'Z' < 'H'$ Devuelve falso (false)

- 4.3.1.3 Operadores lógicos

Se utilizan para combinar las expresiones lógicas, es decir, que nos permiten evaluar más de una expresión lógica a la vez. Los operadores **AND**, **OR** y **NOT** trabajan con operandos que son expresiones lógicas.

Su formato es:

[Operando 1] operador lógico [Operando 2] ...operador lógico [Operando N]

Donde:

operando1, *operando2* y *operandoN* pueden ser cualquier expresión lógica.



Ejemplos:

5 > 6 **AND** 7 > 20 Devuelve falso (false)
 3 > 1 **AND** 6 < 20 Devuelve verdadero (true)

- 4.3.1.3.1 Operador lógico AND

El *operador lógico AND (Y)* combina dos o más expresiones lógicas y produce un resultado verdadero sólo si todos los operandos son verdadero.

Operando 1		Operando 2	Resultado
True	AND	True	True
True		False	False
False		True	False
False		False	False

- 4.3.1.3.2 Operador lógico OR

El *operador lógico OR (O)* Devuelve verdadero si al menos una de las expresiones lógicas produce un resultado verdadero.

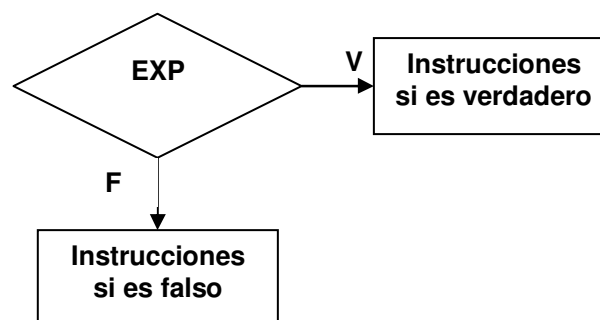
Operando 1		Operando 2	Resultado
True	OR	True	True
True		False	True
False		True	True
False		False	False

- 4.3.1.3.3 Operador lógico NOT

El operador lógico **NOT(NO)** niega el valor original de una expresión, si es verdadero será falso, si es falso será verdadero.

- 4.3.2 La sentencia If

Dado que una condición produce un valor verdadero o falso, se necesita una sentencia de control que ejecute determinada sentencia si la condición es verdadera, y otra si es falsa. En Pascal esta alternativa se realiza con la sentencia **if-then-else**. A continuación se describe el diagrama de flujo y el formato de la sentencia.





La sentencia **if** es utilizada mediante el siguiente formato:

```
If (Condición) Then
    [instrucciones si es verdadero]
    .
    .
    .
Else
    [instrucciones si es falso]
    .
    .
    .
End If
```

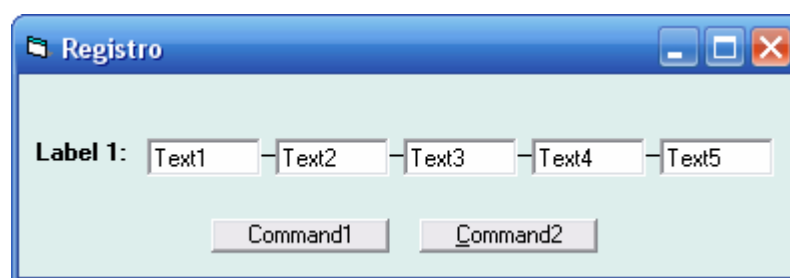
La *condición* es una *expresión Booleana* que puede ser verdadera o falsa (**true o false**). Una *expresión Booleana* se forma comparando valores de las expresiones utilizando operadores de relación (relacionales) o comparación y los operadores lógicos vistos anteriormente.

En la mayoría de los ejercicios realizados hasta ahora hemos utilizado la estructura de control selectiva **if-then-else** para tomar dediciones y responder a situaciones previstas de antemano. Por ejemplo, comprobar si una *caja de texto* se deja vacía, verificar si un valor de una caja o una variable cumplen con una determinada condición, evaluar el valor de una propiedad de un objeto **image**, etc.

Ejercicio:

En el siguiente ejercicio crearemos una pantalla que vemos comúnmente al instalar un programa. Esta es la pantalla para registrar el producto. En esta verificaremos que cada una de las cajas de texto que debe completar el usuario no se dejen vacía. También, verificaremos si la clave escrita por el usuario coincide con la requerida por el programa.

- Abra un nuevo proyecto.
- Modifique el tamaño del formulario e inserte una etiqueta, cinco cajas de texto y dos botones de comando, tal y como se ve en la siguiente pantalla:



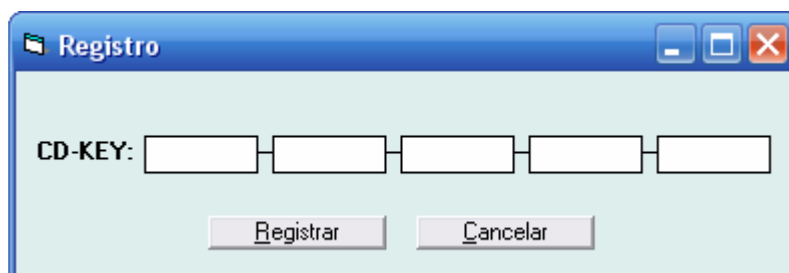
- Establezca los siguientes valores en las propiedades indicadas en cada uno de los controles mostrados en la siguiente tabla:

Control	Propiedad	Valor
Form	Caption StartPosition	Registro 2 – CenterScreen
Label1	AutoSize Caption Font	True CD-KEY MS Sans Serif, Tamaño 10 y Negrita.



Control	Propiedad	Valor
Text1	Name Appearance MaxLength Text	txtCaja1 0 - Flat: 5
Text2	Name Appearance MaxLength Text	txtCaja2 0 - Flat: 5
Text3	Name Appearance MaxLength Text	txtCaja3 0 - Flat: 5
Text4	Name Appearance MaxLength Text	txtCaja4 0 - Flat: 5
Text5	Name Appearance MaxLength Text	txtCaja5 0 - Flat: 5
Command1	Name Caption	cmdRegistrar &Registrar
Command2	Name Caption	cmdCancelar &Cancelar

- La ventana debe tener la siguiente apariencia:



- Dentro del **evento Click** del botón *Registrar* escriba el siguiente bloque de código:

```

Dim Clave As String
Clave = "1234512345123451234512345"

'Verifica que la contraseña es igual a la que contiene las cajas.
If txtCaja1.Text & txtCaja2.Text & txtCaja3.Text & txtCaja4.Text & txtCaja5.Text = Clave Then
    MsgBox ("Contraseña Correcta.")
    Unload Me ' Cierra esta ventana.
Else
    MsgBox ("Contraseña Incorrecta."), vbCritical, "Registro"
    txtCaja1.SetFocus ' Hace que la primera caja reciba el enfoque.
End If
    
```



Una de las características de las cajas que contienen una ventana para registro de contraseña es, que cada vez que se complete el número de caracteres en una caja, el curso se coloque automáticamente en la siguiente caja. En nuestro caso, programaremos el salto a la siguiente caja cuando se haya escrito cinco caracteres en una caja determinada. Para esto tendremos que programar el evento **Change** de cada una de las cajas de texto.

- Dentro del evento **Change** de la primera caja escriba:

```
'Verificamos que Si la primera caja de texto tiene cinco caracteres.  
If Len(txtCaja1.Text) = 5 Then  
    txtCaja2.SetFocus ' Saltamos a la segunda caja.  
End If
```

- Dentro del evento **Change** de la segunda caja escriba:

```
'Verificamos que Si la segunda caja de texto tiene cinco caracteres.  
If Len(txtCaja2.Text) = 5 Then  
    txtCaja3.SetFocus ' Saltamos a la tercera caja.  
End If
```

- Dentro del evento **Change** de la tercera caja escriba:

```
'Verificamos que Si la tercera caja de texto tiene cinco caracteres.  
If Len(txtCaja3.Text) = 5 Then  
    txtCaja4.SetFocus ' Saltamos a la cuarta caja.  
End If
```

- Dentro del evento **Change** de la cuarta caja escriba:

```
'Verificamos que Si la cuarta caja de texto tiene cinco caracteres.  
If Len(txtCaja4.Text) = 5 Then  
    txtCaja5.SetFocus ' Saltamos a la quinta caja.  
End If
```

- Dentro del evento **Change** de la quinta caja escriba:

```
'Verificamos que Si la quinta caja de texto tiene cinco caracteres.  
If Len(txtCaja5.Text) = 5 Then  
    cmdRegistrar.SetFocus ' Hacemos que el botón Registrar reciba el enfoque.  
End If
```

- Dentro del **evento Click** del botón *Cancelar* escriba:

```
'Sale del programa.  
End
```

- Corra la aplicación y en las cajas de texto escriba una contraseña cualquiera.
- Luego haga clic en el botón *Registrar*. Podrá observar que se muestra un mensaje diciendo que la contraseña es incorrecta.



▪ Ahora escriba la contraseña correcta: 12345-12345-12345-12345. Podrá observar que aparece una ventana diciendo que la contraseña es correcta.

Pudo notar en la aplicación que es posible elegir entre varias alternativas. Una de ellas es proporcionar un mensaje al usuario si la contraseña es incorrecta y la segunda es informar al usuario de que su contraseña es correcta.

▪ Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio5-4** para el formulario y **Ejercicio5-4** para el proyecto.

- 4.3.3 La sentencia Case

La sentencia **Case** es una variante de la sentencia **If**. Esta es utilizada también para elegir entre varias alternativas. Su formato es muy distinto al de la sentencia **If**, tal y como se puede apreciar a continuación:

Select Case Expresión

Case vpe1

[sentencias1]

Case vpe2

[sentencias2]

Case vpe...N

[Sentencias...N]

Case Else

[Sentencias-sino]

End Select

Selecione según expresión

Sea vpe1

[sentencias1]

Sea vpe2

[sentencias2]

Sea vpe...N

[Sentencias...N]

No sea vpe1, vpe2, vpe...N

[Sentencias-sino]

Fin selección

Donde **expresión** es una expresión numérica o alfanumérica que puede proporcionar una serie de valores distintos y uno de ellos puede o no encontrarse en la lista. A estos valores en una **sentencia Case** se les podrían llamar "**valores proporcionados por la expresión (vpe)**". Las **etiquetas vpe1, vpe2, vpe...N** representan valores que puede o no proporcionar la expresión. Según sea el valor se ejecutan las **sentencias** seguidas a la **etiqueta (vpeN)** correspondiente. La cláusula **opcional Case Else** indica que si los valores proporcionados por la **expresión** no se encuentran en las listas de las etiquetas (**vpeN**) entonces se ejecutarán las [**Sentencias-sino**].

Ejemplo:

Dim Numero **As Integer**

Numero = InputBox("Introduzca un número del 1 al 4:") 'Permite que el usuario introd. un numero.

Select Case Numero

Case 1

MsgBox ("Este es el número Uno.")

Case 2

MsgBox ("Este es el número Dos.")

Case 3

MsgBox ("Este es el número Tres.")



Case 4

MsgBox ("Este es el número cuatro.")

Case Else

MsgBox ("No se a proporcionado los valores 1, 2, 3 o 4")

End Select

La *variable* **numero** almacena el número introducido por el usuario. Si el usuario introduce el número uno, entonces, en la variable **numero** se almacena valor **1** y, a continuación, se ejecuta el caso uno (**vpe1**), donde se encuentra la sintaxis **MsgBox ("Este es el número Uno.")**, si **numero** toma el valor de **2**, entonces, se ejecuta el caso dos (**vpe2**), y a si sucesivamente.

4.4 ESTRUCTURAS DE CONTROL REPETITIVAS

Las **Estructuras de Control Repetitivas** llamadas también **Bucles**, son aquellas que nos permiten repetir un determinado bloque de código mientras se cumple una determinada condición. Los **Bucles** están compuestos por una condición o expresión que se puede evaluar a falso o verdadero. Mientras la condición se evalúe a verdadero (**true**), el cuerpo de bucle se ejecutará.

En Visual Basic existen cuatro tipos de bucles que son: **For...Next**, **Do...Loop**, **While...Wend** y **For Each...Next**.

- 4.4.1 El Bucle For...Next

La sentencia **For ... Next** repite una determinada serie de sentencias dado un valor inicial y un valor final. Este *bucle* se utiliza cuando se conoce por anticipado el número de repeticiones requerida por el programa. Si el número de repeticiones no se conoce por anticipado entonces debe utilizar las sentencia **While ... Wend** o **Do ... Loop** en lugar de **For**.

Su formato es:

a)

```
For variable = valor inicial To valor final
```

```
    [Sentencias...]
```

```
Next variable
```

b)

```
For variable = valor inicial To valor final Step x
```

```
    [Sentencias...]
```

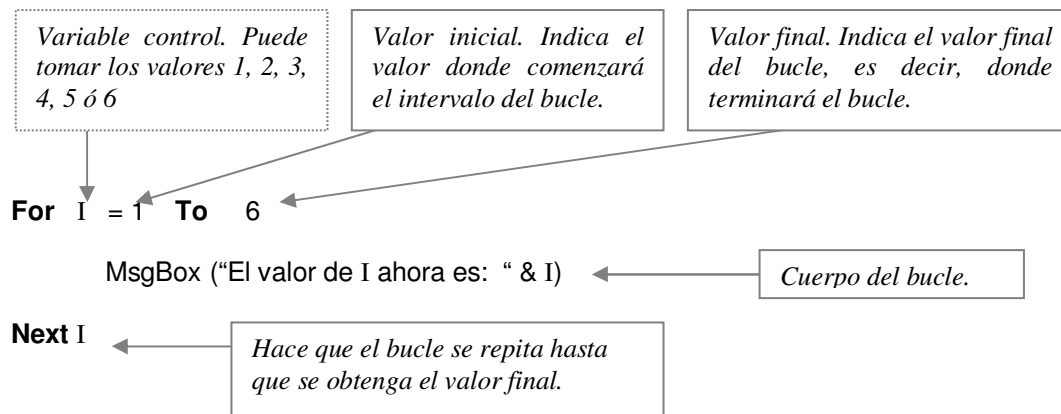
```
Next variable
```



Capítulo 4

Al ejecutarse la sentencia **For** por primera vez, el *valor inicial* se asigna a *variable* que se denomina *variable de control*, y a continuación se ejecuta la sentencia del interior del bucle hasta que la *variable de control* toma el *valor final*.

Ejemplo:



En el segundo formato vemos la cláusula **Step x**. Esta indica que el bucle se decrementa o incrementa en **x**.

Ejemplo:

```
For I = 6 To 1 Step - 1  
    MsgBox ("El valor de I ahora es: " & I)  
Next I
```

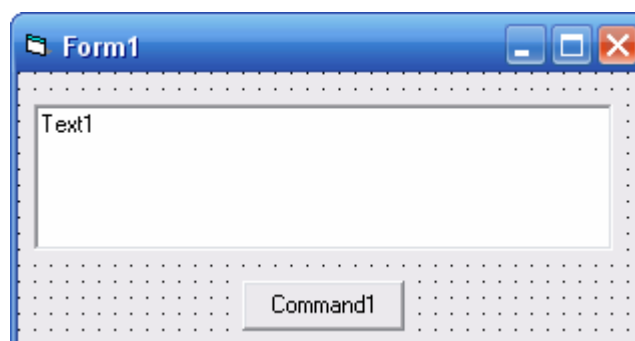
En este caso **I** tomará valores comenzando desde 6 hasta llegar a 1 (6, 5, 4, 3, 2, 1). Si en caso de haber especificado **-2** en vez de **-1**, entonces, se decrementa de dos en dos (6, 4, 2, 0).

Al usar la cláusula **Step** debe tener en cuenta, que el *valor inicial* del bucle es menor que el *valor final* del bucle cuando se decrementa.

Ejercicio 1:

En este primer ejercicio mostramos los números del 1 al 200 en una caja de texto utilizando un bucle For...Next.

- Abra un nuevo proyecto.
- Inserte una caja de texto y un botón de comando, tal y como se muestra en la siguiente imagen:





- Establezca los siguientes valores en las propiedades indicadas en cada uno de los controles mostrados en la siguiente tabla:

Control	Propiedad	Valor
Text	Name MultiLine ScrollBars Text	txtNumeros True 2 – Vertical
Command1	Name Caption	cmdMostrar &Mostrar

- En el evento **Click** del botón de comando escriba:

```
Dim I As Integer 'Declaramos la variable de control para el bucle.
For I = 1 To 200 'Iniciamos el bucle del 1 al 200.
    txtNumeros.Text = txtNumeros.Text & I & ", " 'Mostramos el valor actual de I en la caja.
Next I 'Repetimos el bucle.
```

- Corra la aplicación y haga clic en el botón *Mostrar*.

Podrá observar como los números del 1 al 200 se muestran sobre la caja de texto, tal y como se muestra en la siguiente imagen:



Puede hacer que los números se muestren uno de bajo del otro haciendo un salto de línea. Para hacer un salto de línea solo debe combinar los valores **ASCII 13** y **10**. Ejemplo:

```
txtNumeros.Text = txtNumeros.Text & I & Chr(13) & Chr(10)
```

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio6-4** para el formulario y **Ejercicio6-4** para el proyecto.

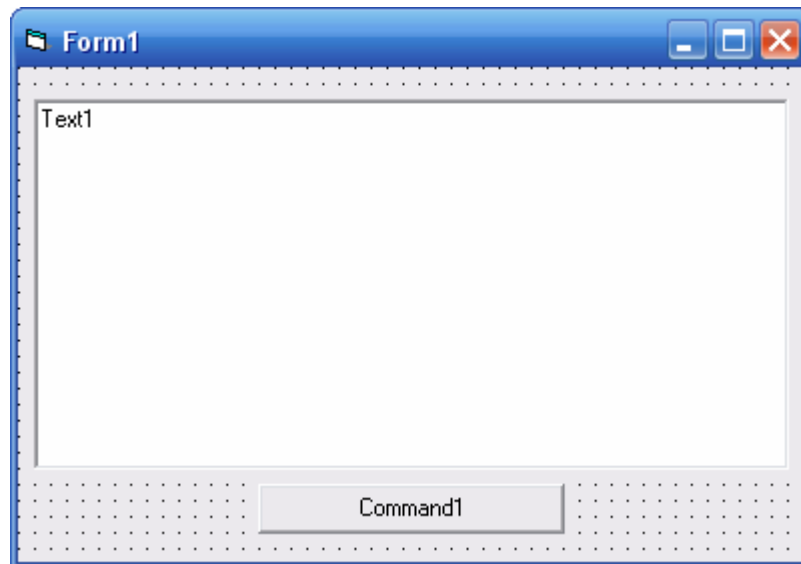
Ejercicio 2:

En este segundo ejercicio utilizaremos un bucle For...Next para sustituir todos los espacios que contenga una caja de texto por comas.

- Abra un nuevo proyecto.



- Inserte una caja de texto y un botón de comando, tal y como se muestra en la siguiente imagen:



- Establezca los siguientes valores en las propiedades indicadas en cada uno de los controles mostrados en la siguiente tabla:

Control	Propiedad	Valor
Text	Name	txtContenido
	MultiLine	True
	ScrollBars	2 – Vertical
	Text	
Command1	Name	cmdCambiar
	Caption	&Cambiar

- Dentro del evento **Click** del botón de comando escriba:

```

Dim I As Integer 'Declaro la variable control para el bucle.
Dim C As String 'Variable para almacenar cada carácter leído de la caja.

For I = 1 To Len(txtContenido.Text) 'Inicio el bucle del 1 hasta la longitud de la caja.

    C = Mid(txtContenido.Text, I, 1) 'Se lee carácter por carácter.

    'Seleccionamos uno por uno los caracteres de la caja.
    txtContenido.SelStart = I - 1
    txtContenido.SelLength = 1

    'Verificamos si el caracter seleccionado es un espacio.
    If txtContenido.SelText = Chr(32) Then
        txtContenido.SelText = "," ' Si es un espacio se sustituye por una coma.
    End If

Next I 'Se repite el bucle.
    
```



- Corra la aplicación y escriba varias palabras en la caja de texto y, a continuación, haga clic en el botón *Cambiar*.

Podrá ver como cada uno de los espacios que separan las palabras son sustituidas por una coma.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio7-4** para el formulario y **Ejercicio7-4** para el proyecto.

- 4.4.2 El Bucle Do...Loop

Utilice el bucle **Do** para ejecutar un bloque de instrucciones un número indefinido de veces. Hay algunas variantes en la instrucción **Do...Loop**, pero cada una evalúa una condición numérica para determinar si continúa la ejecución. Como ocurre con **If...Then**, la *condición* debe ser un valor o una expresión que dé como resultado **False** (cero) o **True** (distinto de cero).

El bucle **Do...Loop** tiene el siguiente formato:

Do While *condición*

Instrucciones...

Loop

Hacer Mientras *condición = verdadera*

Instrucciones...

Repite

Cuando Visual Basic ejecuta este bucle **Do**, primero evalúa *condición*. Si *condición* es **False** (cero), se salta todas las instrucciones. Si es **True** (distinto de cero), Visual Basic ejecuta las instrucciones, vuelve a la instrucción **Do While** y prueba la condición de nuevo.

Por tanto, el bucle se puede ejecutar cualquier número de veces, siempre y cuando *condición* sea distinta de cero o **True**. Nunca se ejecutan las instrucciones si *condición* es **False** inicialmente.

Ejemplo:

El siguiente ejemplo convierte todo el contenido de una caja de texto a mayúscula:

'Variable para almacenar el total de los caracteres leídos.

Dim TotalCaracterLeido As Integer

TotalCaracterLeido = 0 'Iniciamos la variable en cero.

'Repetimos mientras la longitud de la caja de texto es mayor que el total de caracteres leído.

Do While Len(txtContenido.Text) > TotalCaracterLeido

TotalCaracterLeido = TotalCaracterLeido + 1 'Incrementamos el total de carácter leído en 1.

'Colocamos el punto de inserción delante del carácter a leer.

txtContenido.SelStart = TotalCaracterLeido - 1

txtContenido.SelLength = 1 'Seleccionamos el carácter.

'Convertimos el carácter seleccionado a mayúscula.

txtContenido.SelText = UCase(txtContenido.SelText)

Loop 'Volvemos a repetir hasta que se lean todos los caracteres de la caja.



Si en una aplicación tenemos una caja de texto llamada txtContenido, y colocamos el bloque de código anterior en un botón de comando. Al hacer clic sobre dicho botón se convertirán todos los caracteres escritos en la caja de texto a mayúscula.

El bucle **Do...Loop** también permite evaluar una condición después de la primera ejecución del bucle. Esta variante del bucle **Do...Loop** permite ejecutar el bucle al menos una vez, sin importar que la condición sea *verdadera* o *falsa*.

El formato para esta variante del bucle **Do...Loop** es colocar la sentencia **While** después de la cláusula **Loop**:

Do

Instrucciones...

Loop While *condición*

Ejercicio:

El siguiente bloque de código permite calcular la raíz cuadrada de un número positivo. Si el número introducido es negativo el bucle se repetirá.

```
Dim Numero As Integer 'Se declara la variable donde se almacenará el número introducido.
```

```
Do
```

```
'Solicita un número y se almacena en la variable numero.
```

```
Numero = InputBox("Introduzca un número:")
```

```
'Si el número es negativo se muestra un mensaje al usuario.
```

```
If Numero < 0 Then MsgBox ("Introduzca un número positivo.")
```

```
Loop While (Numero < -0) 'Se repite mientras el número sea negativo.
```

```
'Cuando el numero es positivo el bucle finaliza y se muestra la raíz.
```

```
MsgBox ("La raíz del número es:" & Sqr(Numero))
```

- 4.4.3 El Bucle While...Wend

La estructura repetitiva **While (Mientras)** es aquella en la que el número de iteraciones no se conoce por anticipado y el cuerpo del bucle se repite *mientras* se cumple una determinada condición. La forma de utilizar este bucle es similar al bucle **Do...Loop**. La única diferencia es, que no es posible evaluar la condición después de la primera ejecución del bucle.

Su formato es:

While *condición*

Instrucciones...

Wend

Mientras *condición* sea verdadera

Instrucciones...

Repite



Las reglas de funcionamiento de este bucle son las siguientes reglas:

1. La condición se evalúa antes y después de cada ejecución del bucle. Si la condición es verdadera, se ejecuta el bucle, y si es falsa, el bucle no se ejecuta.
2. Si la condición se evalúa a falso cuando se ejecuta el bucle por primera vez, el cuerpo del bucle no se ejecutará nunca. En este caso se dice que el bucle se ha ejecutado cero veces.
3. Mientras la condición sea verdadera el bucle se ejecutará. Esto significa que el bucle se ejecutará indefinidamente a menos que "algo" en el interior del bucle modifique la condición haciendo que su valor pase a falso.

Ejemplo:

En el siguiente ejemplo mostraremos como utilizar un bucle **While...Wend** para imprimir el cuadrado de los números del 1 al 20 en una caja de texto.

```
Dim Numero As Integer 'Variable para generar de forma incrementada los números del 1 al 200.
Numero = 0

While Numero < 20 'Mientras el numero sea menor que 20 se repite.

    Numero = Numero + 1 'Generamos los números uno por uno en cada repetición.

    'Almacenamos el número y su cuadrado al lado en cada línea de una caja de texto.
    txtContenido.Text = txtContenido.Text & Numero & " El cuadrado es: " & Numero ^ 2 & Chr(13)
    & Chr(10)

Wend 'Repetimos.
```

Cada vez que el bucle se repita la variable *Numero* se incrementa en uno para obtener el número siguiente. Mientras el número generado sea menor que 20 se imprimirá en la caja de texto el número y su cuadrado al lado.

- 4.4.4 El Bucle For Each...Next

Repita un grupo de { *CONTROL Internet.HHCtrl.1* } { *HYPERLINK "JavaScript:alink_4.Click()"* } para cada elemento de una { *CONTROL Internet.HHCtrl.1* } { *HYPERLINK "JavaScript:alink_5.Click()"* } o { *CONTROL Internet.HHCtrl.1* } { *HYPERLINK "JavaScript:alink_6.Click()"* }.

Su formato es:

For Each *elemento In grupo*

Instrucciones...

[Exit For]

Instrucciones...

Next [*elemento*]

La sintaxis de la instrucción **For Each...Next** consta de las siguientes partes:



Parte	Descripción
<i>elemento</i>	Requerido. { CONTROL Internet.HHCtrl.1 } Variable que se utiliza para iterar por los elementos del conjunto o matriz. Para conjuntos, <i>elemento</i> solamente puede ser una variable del { CONTROL Internet.HHCtrl.1 }tipo Variant , una variable de objeto genérica o cualquier variable de objeto específica. Para matrices, <i>elemento</i> solamente puede ser una variable tipo Variant .
<i>grupo</i>	Requerido. Nombre de un conjunto de objetos o de una matriz (excepto una matriz de { CONTROL Internet.HHCtrl.1 }tipos definidos por el usuario).
<i>instrucciones</i>	Opcional. Una o más instrucciones que se ejecutan para cada elemento de un <i>grupo</i> .

La entrada al bloque **For Each** se produce si hay al menos un elemento en *grupo*. Una vez que se ha entrado en el bucle, todas las instrucciones en el bucle se ejecutan para el primer elemento en *grupo*. Después, mientras haya más elementos en *grupo*, las instrucciones en el bucle continúan ejecutándose para cada elemento. Cuando no hay más elementos en el *grupo*, se sale del bucle y la ejecución continúa con la instrucción que sigue a la instrucción **Next**.

Se pueden colocar en el bucle cualquier número de instrucciones **Exit For**. La instrucción **Exit For** se utiliza a menudo en la evaluación de alguna condición (por ejemplo, **If...Then**) y transfiere el control a la instrucción que sigue inmediatamente a la instrucción **Next**.

Puede anidar bucles **For Each...Next**, colocando un bucle **For Each...Next** dentro de otro. Sin embargo, cada *elemento* del bucle debe ser único.

NOTA: Si omite *elemento* en una instrucción **Next**, la ejecución continúa como si se hubiera incluido. Si se encuentra una instrucción **Next** antes de su instrucción **For** correspondiente, se producirá un error.

No se puede utilizar la instrucción **For Each...Next** con una matriz de tipos definidos por el usuario porque un tipo **Variant** no puede contener un tipo definido por el usuario.

Ejemplo:

En el siguiente ejemplo creamos un objeto llamado *Nombres*. En este agregamos tres nombres y

```
Dim Nombres As New Collection 'Creamos el objeto.
Dim Leido As Variant         'Variable donde se almacenarán los elementos leídos.

'Agregamos datos a la colección.
Nombres.Add "Carlos"
Nombres.Add "Pablo"
Nombres.Add "Jose"

'Leemos cada uno de los elementos agregados.
For Each Leido In Nombres
    MsgBox (Leido) 'Mostramos los elementos leídos en una caja de mensaje.
Next
```



- 4.4.5 Ejercicios propuestos

- 1.- Utilizando un bucle **For...Next** imprimir en una caja de texto el cubo de los números del 1 al 20.
- 2.- Utilizando un bucle **For...Next** calcular e imprimir en un cuadro de mensaje la sumatoria de los números del 1 al 100.
- 3.- Utilizando un bucle **For...Next** calcular e imprimir el cuadrado de la sumatoria de los números del 1 al 50.
- 4.- Utilizando un bucle **Do...Loop** convertir el contenido de una caja de texto a minúscula.
- 5.- Utilizando un bucle **Do...Loop** sustituir todos los espacios contenidos en una caja de texto por puntos.
- 6.- Utilizando un bucle **While...Wend** calcular e imprimir en un cuadro de mensaje el número de espacios contenidos en una caja de texto.
- 7.- Utilizando un bucle **While...Wend** calcular e imprimir en una caja de texto el cuadrado de los números del 10 al 20.
- 8.- Utilizando un bucle **While...Wend** calcular e imprimir en un cuadro de mensaje el total de letras "a" contenidas en una caja de texto.
- 9.- Utilizando un bucle **While...Wend** convertir las letras contenidas en una caja de texto a mayúscula y minúsculas respectivamente. Ejemplo: La CaSa De PeDrO.
- 10.- Crear un objeto llamado Carros. Agregar cinco marcas de carros y utilizando un bucle **For Each...Next** leer y mostrar en un cuadro de mensaje todos los elementos del objeto.

4.5 LAS FUNCIONES EN EL LENGUAJE BASIC

Una función es un subprograma que recibe como argumentos o parámetros datos de tipo numérico (**Integer**, **Long**, **Double**), o no numérico (**String**, **Boolean**, **Variant**), y devuelve un resultado.

El formato de una función es el siguiente:

Nombre_de_la_función (*argumento 1, argumento 2,...*)

Argumento 1, Argumento2,... Son datos de tipo *Integer, Long, Double, String, Variant, etc.*

El *argumento* se conoce en términos de lenguajes de programación con el nombre de *parámetro*. Para poder calcular el valor o resultado de la función, todo lo que se debe conocer es el valor o los valores e los argumentos.

Ejemplos:

1. **Abs**(-5) = 5 La función **Abs** calcula el valor absoluto de un argumento. En este caso el argumento es -5 y el resultado de la función es 5.

2. **Cos**(6) = 0.994521895368273

La función **Cos** calcula el coseno de un argumento. En este caso el argumento es 6 y el resultado de la función es 0.994521895368273.



Las funciones son muy útiles al momento de resolver operaciones complejas matemáticas o de otras disciplinas afines. En Visual Basic existen dos grandes grupos de funciones:

- Funciones predefinidas, internas o estándar.
- Funciones definidas por el usuario.

- 4.5.1 Funciones predefinidas

Las funciones predefinidas son aquellas que incorpora Visual Basic para realizar operaciones especiales dentro de un programa. Las funciones predefinidas se dividen en los siguientes grupos:

- Funciones aritméticas.
- Funciones trigonométricas.
- Funciones de conversión de tipos de datos.
- Funciones de manipulación de cadena de caracteres.
- Funciones financieras.

- 4.5.1.1 Funciones aritméticas

Las *funciones aritméticas* son aquellas que realizan operaciones matemáticas simples tales como: raíz cuadrada, cuadrado, redondeo, logaritmo, etc.), y devuelven un resultado.

- Función Round:

Se utiliza para redondear un número, es decir, devuelve el entero más próximo al argumento.

Su formato es:

Round(*Expresión*, [*NúmLugaresDecimales*])

Donde:

Expresión: Es cualquier expresión que puede ser evaluada como un número. Esta puede ser una variable, el valor de una caja de texto, etc.

NúmLugaresDecimales: Es opcional. Indica que cuanto lugares a la derecha del decimal se incluye en el redondeo. Si se omite, la función **Round** devuelve un número entero.

Ejemplos:

Los siguientes ejemplos muestran como redondear un número y mostrar el resultado en un cuadro de mensaje:

MsgBox (Round(5.5))	'Devuelve 6.
MsgBox (Round(3.25))	'Devuelve 3.
MsgBox (Round(-7.15))	'Devuelve -7.
MsgBox (Round(9.526, 2))	'Devuelve 9.53.
MsgBox (Round(7.58, 1))	'Devuelve 7.6.



- Función Int y Fix:

Devuelven la parte entera de un número real. En el caso de que el argumento sea un número negativo, la función **Int** devuelve el primer número entero negativo menor o igual que el número; la función **Fix** devuelve el primer entero negativo mayor o igual que el número.

Formatos:

Int (número)

Fix (número)

El argumento *Número* es un tipo **Double** o cualquier expresión numérica válida.

Ejemplos:

```
Dim MiNumero
MiNumero = Int(99.8) ' Devuelve 99.
MiNumero = Fix(99.2) ' Devuelve 99.

MiNumero = Int(-99.8) ' Devuelve -100.
MiNumero = Fix(-99.8) ' Devuelve -99.

MiNumero = Int(-99.2) ' Devuelve -100.
MiNumero = Fix(-99.2) ' Devuelve -99.
```

La diferencia entre **Int** y **Fix** es que si el *número* es negativo, **Int** devuelve el primer entero negativo menor o igual a *número*, mientras que **Fix** devuelve el primer entero negativo mayor o igual a *número*. Por ejemplo, **Int** convierte -8.4 en -9 y **Fix** convierte -8.4 a -8.

- Función Abs:

Devuelve el valor absoluto de un número, es decir, el mismo número si es positivo, o su opuesto, si es negativo.

Formato:

Abs (número)

El argumento *Número* puede ser cualquier expresión numérica válida. Si *número* contiene **Null**, la función devolverá **Null**; si es una variable no inicializada, devolverá cero.

Ejemplos:

```
Dim MiNumero
MiNumero = Abs(5) 'Devuelve 5.
MiNumero = Abs(-5) 'Devuelve 5.
```

- Función Exp:

Devuelve un tipo **Double** que especifica **e** (la base de los logaritmos naturales) elevado a una potencia. El valor de la constante **e** es 2.718282 aproximadamente.

Formato:

Exp(número)



Ejemplos:

Dim MiNumero

MiNumero = Exp(1) 'Devuelve 2.71828182845905. Es equivalente al valor de la constante e.

MiNumero = Exp(2) 'Devuelve 7.38905609893065. Es equivalente al cuadrado de la constante.

MiNumero = Exp(4) 'Devuelve 54.5981500331442. Es equivalente a la constante a la cuarta.

- Función Log:

Devuelve un tipo **Double** que representa el logaritmo natural de un número.

Formato:

Log(número)

El argumento *número* es un tipo **Double** o cualquier expresión numérica válida mayor que cero.

Ejemplos:

Dim MiLogaritmo

MiLogaritmo = **Log**(3) 'Devuelve 1.09861228866811.

MiLogaritmo = **Log**(4) 'Devuelve 1.38629436111989.

MiLogaritmo = **Log**(5) 'Devuelve 1.6094379124341.

Puede calcular logaritmos en base-*n* para cualquier número *x* dividiendo el logaritmo natural de *x* por el logaritmo natural de *n* de la siguiente manera:

$$\text{Log}_n(x) = \text{Log}(x) / \text{Log}(n)$$

El siguiente ejemplo muestra como calcular el logaritmo en base 10:

Dim MiLogaritmo

MiLogaritmo = **Log**(3) / **Log**(10) 'Devuelve 0.477121254719662.

MiLogaritmo = **Log**(4) / **Log**(10) 'Devuelve 0.602059991327962.

MiLogaritmo = **Log**(5) / **Log**(10) 'Devuelve 0.698970004336019.

- Función Sqr:

Devuelve la raíz cuadrada de un número.

Formato:

Sqr(número)

El argumento *número* es un tipo **Double** o cualquier expresión numérica válida mayor o igual que cero.

Ejemplos:

Dim Raiz

Raiz = **Sqr**(25) 'Devuelve 5.

Raiz = **Sqr**(4) 'Devuelve 2.



- Función Sgn:

Devuelve un tipo **Integer** que indica el signo de un número.

Formato:

Sgn(número)

El argumento *número* puede ser cualquier expresión numérica válida.

Valores devueltos

Si <i>número</i> es	Sgn devuelve
Mayor que cero	1
Igual a cero	0
Menor que cero	-1

Ejemplos:

```
Dim Signo
Signo = Sgn(-5)    'Devuelve -1.
Signo = Sgn(4)     'Devuelve 1.
Signo = Sgn(0)     'Devuelve 0.
```

- 4.5.1.2 Funciones trigonométricas

Las funciones trigonométricas que Visual Basic soporta son: seno, coseno, tangente y arco tangente. Las demás funciones Secante, Cosecante, Cotangente, Seno Inverso, Coseno Inverso, Secante inversa, Cosecante inversa, Cotangente inversa, Seno hiperbólico, Coseno hiperbólico, Tangente hiperbólica, Secante hiperbólica, Cosecante hiperbólica, Cotangente hiperbólica, Seno hiperbólico inverso, Coseno hiperbólico inverso, Tangente hiperbólica inversa, Secante hiperbólica inversa, Cosecante hiperbólica inversa y Cotangente hiperbólica inversa, se derivan partir de las cuatros funciones trigonométricas básicas.

- **Función Sin:** Devuelve un tipo **Double** que especifica el seno de un ángulo expresado en radianes.

Formato:

Sin(número)

El argumento *número* es un tipo **Double** o cualquier expresión numérica válida que expresa un ángulo en radianes.

Ejemplo:

```
Dim MiAngulo, MiSeno
MiAngulo = 1.4    'Expresado en radianes.
MiSeno = Sin(MiAngulo) 'Devuelve 0.98544972998846 radianes.
MsgBox(MiSeno)    'Muestra el resultado en un cuadro de mensaje.
```

Para expresar el resultado en grado, multiplique el ángulo por el cociente que resulta de dividir el valor del **PI** entre 180.



Ejemplo:

```
Dim MiAngulo, MiSeno
MiAngulo = 3 * (3.141592 / 180)  'Multiplica el ángulo 3 por pi/180.
MiSeno = Sin(MiAngulo)          'Devuelve 5.23359453647093E-02 en grados.
'Muestra el seno en un cuadro de mensaje. Redondeamos para eliminar la notación científica.
MsgBox (Round(MiSeno, 14))      'Muestra 0.0523359454.
```

- Función Cos:

Devuelve el coseno de un ángulo expresado en radianes.

Formato:

Cos(*número*)

El argumento *número* es un tipo **Double** o cualquier expresión numérica válida que expresa un ángulo en radianes.

Ejemplos:

```
Dim MiAngulo, MiCoseno
MiAngulo = 3.5                'El ángulo expresado en radianes.
MiCoseno = Cos(MiAngulo)     'Devuelve -0.936456687290796.
MsgBox(MiCoseno)             'Muestra el resultado en un cuadro de mensaje.
```

Puede expresar el resultado en grados multiplicando el ángulo por $\text{Pi}/180$, tal y como hicimos con la función **Sin**.

- Función Tan:

Devuelve la tangente de un ángulo expresado en radianes.

Formato:

Tan(*número*)

El argumento *número* es un tipo **Double** o cualquier expresión numérica válida que expresa un ángulo en radianes. Su aplicación es similar a los ejemplos anteriores.

- Función Atn:

Formato:

Atn(*número*)

El argumento *número* es un tipo **Double** o cualquier expresión numérica válida que expresa un ángulo en radianes. Su aplicación es similar a los ejemplos anteriores.

- 4.5.1.3 Funciones matemáticas derivadas

En la siguiente página se muestra una lista de funciones matemáticas no básicas que pueden derivarse de funciones matemáticas básicas:



Función	Derivadas equivalentes
Secante	$\text{Sec}(X) = 1 / \text{Cos}(X)$
Cosecante	$\text{Cosec}(X) = 1 / \text{Sin}(X)$
Cotangente	$\text{Cotan}(X) = 1 / \text{Tan}(X)$
Seno inverso	$\text{Arcsin}(X) = \text{Atn}(X / \text{Sqr}(-X * X + 1))$
Coseno inverso	$\text{Arccos}(X) = \text{Atn}(-X / \text{Sqr}(-X * X + 1)) + 2 * \text{Atn}(1)$
Secante inversa	$\text{Arcsec}(X) = \text{Atn}(X / \text{Sqr}(X * X - 1)) + \text{Sgn}((X) - 1) * (2 * \text{Atn}(1))$
Cosecante inversa	$\text{Arccosec}(X) = \text{Atn}(X / \text{Sqr}(X * X - 1)) + (\text{Sgn}(X) - 1) * (2 * \text{Atn}(1))$
Cotangente inversa	$\text{Arccotan}(X) = \text{Atn}(X) + 2 * \text{Atn}(1)$
Seno hiperbólico	$\text{HSin}(X) = (\text{Exp}(X) - \text{Exp}(-X)) / 2$
Coseno hiperbólico	$\text{HCos}(X) = (\text{Exp}(X) + \text{Exp}(-X)) / 2$
Tangente hiperbólica	$\text{HTan}(X) = (\text{Exp}(X) - \text{Exp}(-X)) / (\text{Exp}(X) + \text{Exp}(-X))$
Secante hiperbólica	$\text{HSec}(X) = 2 / (\text{Exp}(X) + \text{Exp}(-X))$
Cosecante hiperbólica	$\text{HCosec}(X) = 2 / (\text{Exp}(X) - \text{Exp}(-X))$
Cotangente hiperbólica	$\text{HCotan}(X) = (\text{Exp}(X) + \text{Exp}(-X)) / (\text{Exp}(X) - \text{Exp}(-X))$
Seno hiperbólico inverso	$\text{HArcsin}(X) = \text{Log}(X + \text{Sqr}(X * X + 1))$
Coseno hiperbólico inverso	$\text{HArccos}(X) = \text{Log}(X + \text{Sqr}(X * X - 1))$
Tangente hiperbólica inversa	$\text{HArctan}(X) = \text{Log}((1 + X) / (1 - X)) / 2$
Secante hiperbólica inversa	$\text{HArcsec}(X) = \text{Log}((\text{Sqr}(-X * X + 1) + 1) / X)$
Cosecante hiperbólica inversa	$\text{HArccosec}(X) = \text{Log}((\text{Sgn}(X) * \text{Sqr}(X * X + 1) + 1) / X)$
Cotangente hiperbólica inversa	$\text{HArccotan}(X) = \text{Log}((X + 1) / (X - 1)) / 2$
Logaritmo en base N	$\text{LogN}(X) = \text{Log}(X) / \text{Log}(N)$

- 4.5.1.4 Funciones de conversión de tipo de datos

Las funciones de conversión de tipo de datos permiten convertir un tipo de datos en específico en otro tipo de datos. Son muy utilizados cuando leemos valores de controles que proporcionan un tipo de datos muy distinto al que necesitamos. Por ejemplo, si diseñamos una aplicación donde los datos leídos de una caja de texto se requieren para realizar algún calculo matemático, debemos de convertir el tipo de datos **String**(cadena) devuelto por la caja de texto al tipo de datos requerido en la operación matemática.

Ejemplo:

En el siguiente ejemplo se muestra como calcular el área de un triangulo donde la base y la altura son proporcionadas mediante cajas de texto.

```
Dim Area, Base, Altura As Double
Base = CDbI(txtBase.Text) 'Leemos la base de la caja de texto txtBase.
Altura = CDbI(txtAltura.Text) 'Leemos la altura de la caja de texto txtAltura.
Area = (Base*Altura)/2 'Calculamos el área del triángulo.
MsgBox(Area) 'Muestra el área en un cuadro de mensaje.
```

Mediante la función **CDbl** podemos convertir el tipo de datos **String (cadena)** devuelto por la caja de texto al tipo de datos numérico **Double**.



Existen funciones para convertir un tipo de datos a otro tipo de datos en específico. A continuación, se muestra una lista que contiene las funciones de conversión de tipos de datos disponibles en Visual Basic:

[illegible]

- 4.5.1.5 Funciones de manipulación de cadenas de caracteres

Una cadena de caracteres es una secuencia de caracteres que pueden ser numéricos o no numéricos. Las cadenas de caracteres pueden ser manipuladas por el usuario mediante un conjunto de funciones disponibles en Visual Basic. Entre las principales operaciones que podemos realizar con una cadena de caracteres tenemos la: concatenación de cadena, extracción de subcadenas, búsqueda de subcadenas y longitud de una cadena.

- 4.5.1.5.1 Concatenación de cadenas

La concatenación de cadenas consiste en la unión de dos o más cadenas de caracteres para formar una sola. Esta operación se realiza mediante el carácter **ampersand &**. En muchas ocasiones los datos se encuentran almacenados en distintas variables y queremos unir los valores contenidos en cada una de ellas. Para esto se realiza una operación de concatenación.



Por ejemplo, si usted solicita el nombre de una persona y lo almacena en una variable y luego solicita el apellido y lo almacena en otra variable, podrá obtener el nombre completo de la persona y almacenarlo en una única variable.

Ejemplo:

Dim Nombre, Apellido, NombreCompleto As String

Nombre = InputBox("Introduzca su nombre:")

Apellido = InputBox("Introduzca su apellido:")

NombreCompleto = Nombre & " " & Apellido

'Solicitamos el nombre.

'Solicitamos el apellido.

'Concatenamos las variables.

'Mostramos el resultado de la concatenación almacenada en la variable NombreCompleto.

MsgBox ("Su nombre completo: " & NombreCompleto)

Explicación:

En este ejemplo hemos declarado tres variables de tipo cadena. La primera variable *Nombre* para almacenar el nombre que el usuario introduzca por el teclado, la segunda *Apellido* para almacenar el apellido introducido por el usuario, y la tercera *NombreCompleto* para almacenar el resultado de la concatenación de las variables *Nombre* y *Apellido*.

En la segunda línea solicitamos el nombre del usuario mediante una caja de entrada (**InputBox**), y lo almacenamos en la variable *Nombre*. En la tercera línea solicitamos el apellido y lo almacenamos en la variable *Apellido*.

En la cuarta línea almacenamos en la variable *NombreCompleto* la concatenación o unión de las variables *Nombre* y *Apellido* mediante el operador **Ampersand &**. Las dos comillas con un espacio entre ambas son para separar el nombre del apellido para que no aparezcan juntos. Por último, mostramos el resultado en un cuadro de mensaje. Puede observar en la última línea que hemos utilizado también el operador ampersand para unir una cadena incluida entre comillas con una cadena almacenada en una variable.

- 4.5.1.5.2 Obtener subcadenas

Obtener una subcadena consiste en extraer una cantidad especificada de caracteres de una cadena principal. Para realizar esta operación se disponen de tres funciones que son: **Left**, **Right** y **Mid**.

- Función Left:

Permite extraer una cantidad de caracteres desde la parte izquierda de una cadena.

Su formato es:

Left(cadena, número)

Donde:

cadena: Es una expresión cadena desde donde se extraerán la cantidad de caracteres.

número: Un valor numérico que indica cuando caracteres se va a devolver. Si el valor excede la longitud de la cadena, se devuelve la cadena completa.



Ejemplo:

Dim CadenaPrincipal, MiCadena
CadenaPrincipal = "Hola a todos"

MiCadena = **Left**(CadenaPrincipal, 4) 'Devuelve Hola.
MiCadena = **Left**(CadenaPrincipal, 1) 'Devuelve H.
MiCadena = **Left**(CadenaPrincipal, 12) 'Devuelve Hola a todos

- Función Right:

Permite extraer una cantidad de caracteres desde la parte derecha de una cadena.

Su formato es:

Right(cadena, número)

Donde:

cadena: Es una expresión cadena desde donde se extraerán la cantidad de caracteres.

numero: Un valor numérico que indica cuando caracteres se va a devolver. Si el valor excede la longitud de la cadena, se devuelve la cadena completa.

Ejemplo:

Dim CadenaPrincipal, MiCadena
CadenaPrincipal = "Hola a todos"

MiCadena = **Right**(CadenaPrincipal, 5) 'Devuelve todos.
MiCadena = **Right**(CadenaPrincipal, 1) 'Devuelve s.
MiCadena = **Right**(CadenaPrincipal, 12) 'Devuelve Hola a todos.

- Función Mid:

Permite extraer una subcadena de caracteres de una cadena principal, dado un valor inicial y una longitud.

Su formato es:

Mid(cadena, inicio, [longitud])

Donde:

cadena: Es una expresión cadena desde donde se extraerán los caracteres.

inicio: Una expresión numérica que indica desde donde se comenzará a extraer los caracteres. Si inicio es mayor que la longitud de la cadena, **Mid** devolverá una cadena de longitud cero ("").

longitud: Opcional. Una expresión numérica que indica la cantidad de caracteres que se desea extraer. Si se omite, se devolverá todos los caracteres a partir de la posición *inicio* hasta el final de la cadena.



Ejemplo:

```
Dim CadenaPrincipal, MiCadena
```

```
CadenaPrincipal = "Visual Basic y las cadenas de caracteres"
```

```
MiCadena = Mid(CadenaPrincipal, 1, 6)    'Devuelve Visual.  
MiCadena = Mid(CadenaPrincipal, 8, 5)    'Devuelve Basic.  
MiCadena = Mid(CadenaPrincipal, 30, 11)  'Devuelve caracteres.
```

- 4.5.1.5.3 Funciones especiales

En la siguiente sesión se listan una serie de funciones especiales muy útiles en el tratamiento de cadena de caracteres.

- Funciones UCase y LCase:

Estas funciones se utilizan para convertir una cadena de caracteres a mayúscula y a minúscula respectivamente.

Formatos:

UCase(cadena)

LCase(cadena)

Ejemplo:

```
Dim CadenaPrincipal, MiCadena
```

```
CadenaPrincipal = "Programando en Visual Basic"
```

```
MiCadena = UCase(cadenaprincipal)  'Devuelve PROGRAMANDO EN VISUAL BASIC.
```

```
MiCadena = LCase(cadenaprincipal)  'Devuelve programando en visual basic.
```

- Función StrConv:

Esta función es similar a las funciones anteriores. Permite convertir a mayúscula o minúscula según el tipo de conversión indicada. También, permite hacer otros tipos de conversión.

Formatos:

StrConv(cadena, conversión, [LCID])

Donde:

cadena: Es una expresión cadena a la cual se aplicará la conversión.

conversión: Un valor numérico que indica el tipo de conversión que será aplicada a la cadena de caracteres.

LCID: Opcional. La LocaleID, si es diferente de la LocaleID del sistema. (La LocaleID del sistema es la predeterminada.)



Los valores del argumento **conversión** son:

Constante	Valor	Descripción
vbUpperCase	1	Convierte la cadena a caracteres en mayúscula.
vbLowerCase	2	Convierte la cadena a caracteres en minúscula.
vbProperCase	3	Convierte a mayúscula la primera letra de cada palabra de la cadena.
vbWide*	4*	Convierte los caracteres cortos (un único byte) de la cadena a caracteres anchos (doble byte).
vbNarrow*	8*	Convierte los caracteres anchos (doble byte) de la cadena a caracteres cortos (un único byte).
vbKatakana**	16**	Convierte los caracteres Hiragana de la cadena a caracteres Katakana.
vbHiragana**	32**	Convierte los caracteres Katakana de la cadena a caracteres Hiragana.
vbUnicode	64	Convierte la cadena a <i>Unicode</i> utilizando la página de códigos predeterminada del sistema.
vbFromUnicode	128	Convierte la cadena de <i>Unicode</i> a la página de códigos predeterminada del sistema.

Ejemplo:

Dim CadenaPrincipal, MiCadena

```

CadenaPrincipal = "Carlos Bucarely"
MiCadena = UCase(CadenaPrincipal)    'Devuelve CARLOS BUCARELLY
MiCadena = LCase(CadenaPrincipal)    'Devuelve carlos bucarely
MiCadena = StrConv(CadenaPrincipal, 1) 'Devuelve CARLOS BUCARELLY
MiCadena = StrConv(CadenaPrincipal, 2) 'Devuelve carlos bucarely
MiCadena = StrConv(CadenaPrincipal, 3) 'Devuelve Carlos Bucarely
    
```

- Función Asc:

La función **Asc** devuelve el código de carácter ANSI o DBCS del primer carácter de una cadena.

Formato:

Asc(cadena)

El argumento obligatorio *cadena* es una expresión de cadena válida. Si *cadena* no contiene caracteres, se produce un *error en tiempo de ejecución*.

Ejemplo:

```

Dim MiCodigo
MiCodigo = Asc("A")    'Devuelve 65.
MiCodigo = Asc("a")    'Devuelve 97.
MiCodigo = Asc("Adios") 'Devuelve 65.
    
```

- Función Chr:

Devuelve una cadena correspondiente al código ANSI o DBCS especificado como argumento.



Ejemplo:

```
Dim MiCadena
MiCadena = Chr(65) 'Devuelve A.
MiCadena = Chr(97) 'Devuelve a.
MiCadena = Chr(56) 'Devuelve 8.
```

- Función InStr:

Devuelve un valor numérico que especifica la posición de la primera aparición de una cadena en otra, desde el principio de la cadena.

Formato:

InStr([inicio], cadena1, cadena2, [comparar])

Donde:

<i>inicio</i>	Opcional. Es un valor numérico que indica la posición desde donde se comenzará a realizar la búsqueda. Si se omite, la búsqueda se hará desde el principio de la cadena principal.
<i>cadena1</i>	Una expresión cadena donde se realizará la búsqueda.
<i>cadena2</i>	Una expresión cadena que indica la cadena que se busca.
<i>comparar</i>	Opcional. Un valor numérico que indica la clase de comparación que se utilizará cuando se evalúen subcadenas. Si se omite, se realiza una comparación binaria.

El argumento *comparar* puede tener los valores siguientes:

Constante	Valor	Descripción
vbUseCompareOption	-1	Realizar una comparación utilizando el valor de la instrucción Option Compare .
vbBinaryCompare	0	Realizar una comparación binaria.
vbTextCompare	1	Realiza una comparación basada en el texto.
vbDatabaseCompare	2	Sólo para Microsoft Access. Realizar una comparación basada en la información de su base de datos.

Ejemplo:

```
Dim CadenaPrincipal, CadenaBuscada, PosicionCadena
```

```
CadenaPrincipal = "La computadora de pedro" 'Cadena donde se busca.
CadenaBuscada = "computadora" 'Cadena que se buscará.
```

```
'Realiza una búsqueda basada en una comparación textual comenzando desde la posición 1.
PosicionCadena = InStr(1, CadenaPrincipal, CadenaBuscada, 1) 'Devuelve 4.
```

```
'Realiza una búsqueda basada en una comparación textual comenzando desde la posición 15.
PosicionCadena = InStr(15, CadenaPrincipal, CadenaBuscada, 1) 'Devuelve 0.
```




- Función InStrRev:

Devuelve un valor numérico que especifica la posición de la primera aparición de una cadena en otra, desde el final de la cadena.

Formato:

InStrRev([inicio], cadena1, cadena2, [comparar])

Donde:

<i>inicio</i>	Opcional. Es un valor numérico que indica la posición desde donde se comenzará a realizar la búsqueda. Si se omite, la búsqueda se hará desde el principio de la cadena principal.
<i>cadena1</i>	Una expresión cadena donde se realizará la búsqueda.
<i>cadena2</i>	Una expresión cadena que indica la cadena que se busca.
<i>comparar</i>	Opcional. Un valor numérico que indica la clase de comparación que se utilizará cuando se evalúen subcadenas. Si se omite, se realiza una comparación binaria.

Ejemplo:

Dim CadenaPrincipal, CadenaBuscada, PosicionCadena

CadenaPrincipal = "La computadora de pedro" 'Cadena donde se busca.
CadenaBuscada = "computadora" 'Cadena que se buscará.

'Realiza una búsqueda basada en una comparación textual comenzando desde la posición 1.
PosicionCadena = **InStrRev**(1, CadenaPrincipal, CadenaBuscada, 1) 'Devuelve 4.

'Realiza una búsqueda basada en una comparación textual comenzando desde la posición 1.
PosicionCadena = **InStrRev**(1, CadenaPrincipal, "pedro", 1) 'Devuelve 19.

- 4.5.1.5.4 Longitud de una cadena

Determinar la longitud de una cadena juega un papel muy importante al momento que se esta manipulando cadenas de caracteres. En muchas ocasiones verá la necesidad de conocer la cantidad de caracteres que posee una cadena de caracteres. Para esto, disponemos de la función **Len**, que devuelve una expresión numérica que indica la cantidad de caracteres de una cadena pasada como argumento a la función.

Su formato es:

Len(cadena)

El argumento cadena es una expresión de cadena valida. Si la cadena es nula, devuelve cero.

Ejemplo:

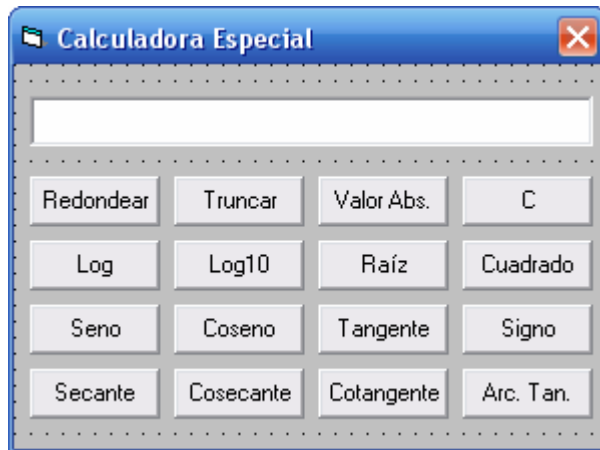
Dim LongitudCadena
LongitudCadena = **Len**("carlos") 'Devuelve 6.
LongitudCadena = **Len**("Pedro Manuel") 'Devuelve 12. El espacio se cuenta.
LongitudCadena = **Len**("") 'Devuelve 0.



- 4.5.1.6 Ejercicios prácticos

1. Nuestro primer ejercicio práctico consiste en una calculadora que permite calcular algunas de las funciones matemáticas y trigonométricas vistas en este capítulo.

- Abra un nuevo proyecto.
- Inserte una caja de texto y 16 botones de comando, tal y como se ve en la siguiente imagen:



- Establezca los siguientes valores en las propiedades de cada control indicado en la siguiente tabla:

Control	Propiedad	Valor
Form1	Caption BorderStyle	Calculadora Especial 1 – Fixed Single
Text1	Name Alignment Text	txtNumero 1 – Right Justify
Command1	Name Caption	cmdBoton1 Redondear
Command2	Name Caption	cmdBoton2 Truncar
Command3	Name Caption	cmdBoton3 Valor Abs.
Command4	Name Caption	cmdBoton4 C
Command5	Name Caption	cmdBoton5 Log
Command6	Name Caption	cmdBoton6 Log10
Command7	Name Caption	cmdBoton7 Raíz
Command8	Name Caption	cmdBoton8 Cuadrado
Command9	Name Caption	cmdBoton9 Seno
Command10	Name Caption	cmdBoton10 Coseno

Control	Propiedad	Valor
Command11	Name Caption	cmdBoton11 Tangente
Command12	Name Caption	cmdBoton12 Signo
Command13	Name Caption	cmdBoton13 Secante
Command14	Name Caption	cmdBoton14 Cosecante
Command15	Name Caption	cmdBoton15 Cotangente
Command16	Name Caption	cmdBoton16 Arc. Tan.



- Dentro del evento **KeyPress** de la caja de texto escriba:

```
'Hace que una caja solo acepte números, el punto y el signo de menos.  
If InStr("0123456789.-" & Chr(8) & Chr(13), Chr(KeyAscii)) = 0 Then  
KeyAscii = 0  
End If
```

- Dentro del evento **GotFocus** de la caja de texto escriba:

```
'Hace que el punto de inserción se coloque al final de la caja.  
txtNumero.SelStart = Len(txtNumero.Text)
```

- Dentro del evento **Click** del botón *Redondear* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Redondea el valor de la caja de texto.  
txtNumero.Text = Round(CDbl(txtNumero.Text))
```

- Dentro del evento **Click** del botón *Truncar* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Elimina la parte decimal del valor de la caja de texto.  
txtNumero.Text = Int(CDbl(txtNumero.Text))
```

- Dentro del evento **Click** del botón *Valor Abs.* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcular el valor absoluto del valor de la caja de texto.  
txtNumero.Text = Abs(CDbl(txtNumero.Text))
```

- Dentro del evento **Click** del botón *C* escriba:

```
'Limpia la caja de texto.  
txtNumero.Text = ""  
txtNumero.SetFocus
```

- Dentro del evento **Click** del botón *Log* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula el logaritmo natural del valor de la caja de texto.  
txtNumero.Text = Log(CDbl(txtNumero.Text))
```

- Dentro del evento **Click** del botón *Log10* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula el logaritmo base diez del valor de la caja de texto.  
txtNumero.Text = Log(CDbl(txtNumero.Text)) / Log(10)
```



- Dentro del evento **Click** del botón *Raíz* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula la raíz cuadrada del valor de la caja de texto.  
txtNumero.Text = Sqr(CDb(txtNumero.Text))
```

- Dentro del evento **Click** del botón *Cuadrado* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula el cuadrado del valor de la caja de texto.  
txtNumero.Text = CDb(txtNumero.Text) * CDb(txtNumero.Text)
```

- Dentro del evento **Click** del botón *Seno* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula el seno del valor de la caja de texto.  
txtNumero.Text = Round(Sin(CDb(txtNumero.Text) * (3.141592 / 180)), 14)
```

- Dentro del evento **Click** del botón *Coseno* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula el coseno del valor de la caja de texto.  
txtNumero.Text = Round(Cos(CDb(txtNumero.Text) * (3.141592 / 180)), 14)
```

- Dentro del evento **Click** del botón *Tangente* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula la tangente del valor de la caja de texto.  
txtNumero.Text = Round(Tan(CDb(txtNumero.Text) * (3.141592 / 180)), 14)
```

- Dentro del evento **Click** del botón *Signo* escriba:

```
'Te muestra en un cuadro de mensaje si el número es positivo o negativo.  
If Len(txtNumero.Text) > 0 Then  
If Sgn(CDb(txtNumero.Text)) = 1 Then MsgBox ("El número es positivo.")  
If Sgn(CDb(txtNumero.Text)) = -1 Then MsgBox ("El número es negativo.")  
End If
```

- Dentro del evento **Click** del botón *Secante* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula la secante del valor de la caja de texto.  
txtNumero.Text = Round(1 / Cos(CDb(txtNumero.Text) * (3.141592 / 180)), 14)
```

- Dentro del evento **Click** del botón *Cosecante* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.  
'Calcula la cosecante del valor de la caja de texto.  
txtNumero.Text = Round(1 / Sin(CDb(txtNumero.Text) * (3.141592 / 180)), 14)
```



- Dentro del evento **Click** del botón *Cotangente* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.
'Calcula la cotangente del valor de la caja de texto.
txtNumero.Text = Round(1 / Tan(CDbl(txtNumero.Text) * (3.141592 / 180)), 14)
```

- Dentro del evento **Click** del botón *Arc. Tan.* escriba:

```
On Error Resume Next      'En caso que la caja este vacía.
'Calcula el arcotangente del valor de la caja de texto.
txtNumero.Text = Round(Atn(CDbl(txtNumero.Text) * (3.141592 / 180)), 14)
```

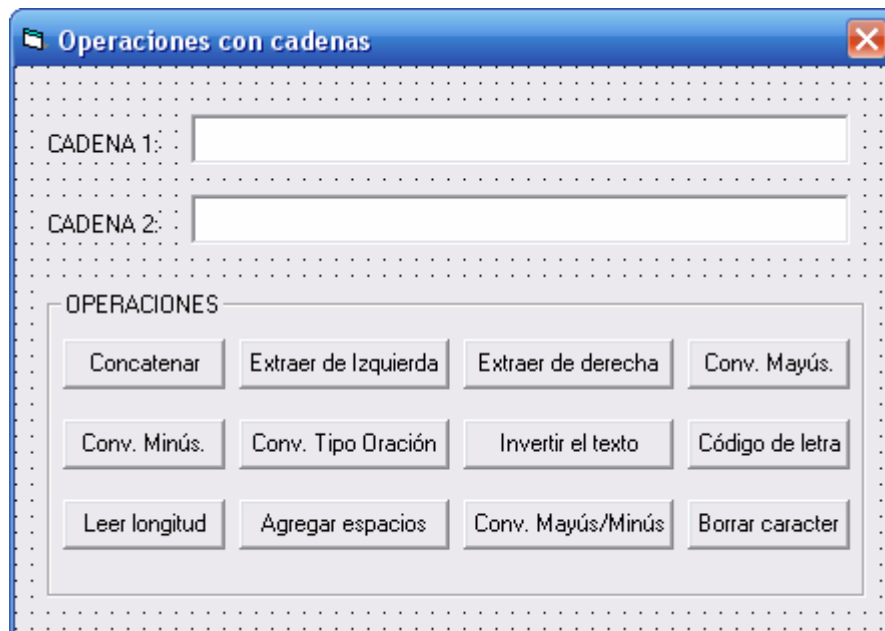
- Corra la aplicación.

Escriba algún valor en la caja de texto y, a continuación, haga clic sobre la función matemática que quiera ejecutar sobre ese número.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio8-4** para el formulario y **Ejercicio8-4** para el proyecto.

2. En nuestro segundo ejercicio veremos como utilizar algunas de las funciones de manipulación de cadenas vistas en este capítulo.

- Abra un nuevo proyecto.
- Inserte dos etiquetas, dos cajas de texto, un control frame y dentro de este doce botones de comando. Tal y como se muestra en la siguiente imagen:



- Establezca los siguientes valores en las propiedades de cada control indicado en la tabla de la siguiente página:



Capítulo 4

Control	Propiedad	Valor	Control	Propiedad	Valor
Form1	Caption BorderStyle	Operaciones con cadenas 1 – Fixed Single	Command6	Name Caption	cmdBoton6 Conv. Tipo Oración
Label1	AutoSize Caption	True CADENA 1:	Command7	Name Caption	cmdBoton7 Invertir el texto
Label2	AutoSize Caption	True CADENA 2:	Command8	Name Caption	cmdBoton8 Código de letra
Text1	Name Text	txtCadena1	Command9	Name Caption	cmdBoton9 Leer longitud
Text2	Name Text	txtCadena2	Command10	Name Caption	cmdBoton10 Agregar espacios
Frame1	Caption	OPERACIONES	Command11	Name Caption	cmdBoton11 Conv. Mayús/Minús
Command1	Name Caption	cmdBoton1 Concatenar	Command12	Name Caption	cmdBoton12 Borrar carácter
Command2	Name Caption	cmdBoton2 Extraer de izquierda			
Command3	Name Caption	cmdBoton3 Extraer de derecha			
Command4	Name Caption	cmdBoton4 Conv. Mayús.			
Command5	Name Caption	cmdBoton5 Conv. Minús.			

- Dentro del evento **Click** del botón *Concatenar* escriba:

```
Dim MiCadena As String
'Combina las dos cadenas escritas en las cajas de texto y la muestra en un MsgBox.
MiCadena = txtCadena1.Text & " " & txtCadena2.Text
MsgBox (MiCadena) 'Muestra la cadena resultante.
```

- Dentro del evento **Click** del botón *Extraer de izquierda* escriba:

```
'Extrae la cantidad de caracteres especificado del texto contenido en la primera caja.
On Error Resume Next 'En casos de que las cajas se dejen vacías.

Dim numero As Long
Dim MiCadena As String

'Solicita la cantidad de caracteres a leer.
numero = InputBox("Introduzca la cantidad de caracteres a extraer:")

'Se utiliza la función Left para leer desde la izquierda de la caja de texto.
MiCadena = Left(txtCadena1.Text, numero)

'Muestra la cadena resultante.
MsgBox (MiCadena)
```

- Dentro del evento **Click** del botón *Extraer de derecha* escriba:



'Extrae la cantidad de caracteres especificado del texto contenido en la primera caja.
On Error Resume Next 'En casos de que las cajas se dejen vacías.

Dim numero As Long
Dim MiCadena As String

'Solicita la cantidad de caracteres a leer.
numero = InputBox("Introduzca la cantidad de caracteres a extraer:")

'Se utiliza la función Right para leer desde la derecha de la caja de texto.
MiCadena = Right(txtCadena1.Text, numero)

'Muestra la cadena resultante.
MsgBox (MiCadena)

- Dentro del evento **Click** del botón *Conv. Mayús* escriba:

On Error Resume Next 'En caso de que las cajas se dejen vacías.

txtCadena1.Text = UCase(txtCadena1.Text) 'Convierte a mayúscula la primera caja.
txtCadena2.Text = UCase(txtCadena2.Text) 'Convierte a mayúscula la segunda caja.

- Dentro del evento **Click** del botón *Conv. Minús* escriba:

On Error Resume Next 'En caso de que las cajas se dejen vacías.

txtCadena1.Text = LCase(txtCadena1.Text) 'Convierte a minúscula la primera caja.
txtCadena2.Text = LCase(txtCadena2.Text) 'Convierte a minúscula la segunda caja.

- Dentro del evento **Click** del botón *Conv. Tipo Oración* escriba:

On Error Resume Next 'En caso de que las cajas se dejen vacías.

txtCadena1.Text = StrConv(txtCadena1.Text, 3) 'Convierte a tipo oración la primera caja.
txtCadena2.Text = StrConv(txtCadena2.Text, 3) 'Convierte a tipo oración la segunda caja.

- Dentro del evento **Click** del botón *Conv. Tipo Oración* escriba:

On Error Resume Next 'En caso de que las cajas se dejen vacías.

Dim CadenalInvertida As String
Dim i As Long

'Iniciamos el bucle para leer carácter por carácter el contenido de la caja.
For i = Len(txtCadena1.Text) To 1 Step -1

'Leemos carácter por carácter desde la derecha.
CadenalInvertida = CadenalInvertida & Mid(txtCadena1.Text, i, 1)

Next i

'Mostramos en la primera caja la cadena invertida.
txtCadena1.Text = CadenalInvertida



- Dentro del evento **Click** del botón *Invertir el texto* escriba:

```
'Invierte el texto contenido en la primera caja de texto.  
On Error Resume Next 'En caso de que las cajas se dejen vacías.  
Dim CadenaInvertida As String  
Dim i As Long  
  
'Iniciamos el bucle para leer caracter por carácter el contenido de la caja de texto.  
For i = Len(txtCadena1.Text) To 1 Step -1  
'Leemos caracter por caracter desde la derecha.  
CadenaInvertida = CadenaInvertida & Mid(txtCadena1.Text, i, 1)  
Next i  
  
'Muestra la cadena invertida en la primera caja de texto.  
txtCadena1.Text = CadenaInvertida
```

- Dentro del evento **Click** del botón *Código de letra* escriba:

```
'Lee el código ANSI del primer carácter de la primera caja de texto.  
On Error Resume Next 'En caso que la primera caja se deje vacía.  
Dim MiCodigo  
MiCodigo = Asc(txtCadena1.Text)  
'Muestra el código ANSI del carácter en un cuadro de mensaje.  
MsgBox (MiCodigo)
```

- Dentro del evento **Click** del botón *Leer longitud* escriba:

```
'Lee y muestra en un MsgBox la longitud de cada caja de texto.  
  
MsgBox ("La primera caja tiene " & Len(txtCadena1.Text) & " caracteres.")  
  
MsgBox ("La segunda caja tiene " & Len(txtCadena2.Text) & " caracteres.")
```

- Dentro del evento **Click** del botón *Agregar espacios* escriba:

```
'Agrega espacios entre cada carácter.  
Dim i As Long  
Dim CadenaEspacios As String  
  
'Iniciamos el bucle para leer carácter por carácter la caja de texto.  
For i = 1 To Len(txtCadena1.Text)  
'Leemos cada carácter y agregamos un espacio con Chr(32).  
CadenaEspacios = CadenaEspacios & Mid(txtCadena1.Text, i, 1) & Chr(32)  
Next i  
  
'Mostramos la cadena resultante.  
txtCadena1.Text = CadenaEspacios
```

- Dentro del evento **Click** del botón *Conv. Mayús/Minús* escriba:



```
'Convertimos a mayúscula y minúscula cada carácter de la primera caja.  
Dim i As Long  
Dim CadenaConvertida As String  
  
'Iniciamos el bucle para leer carácter por carácter el contenido de la caja.  
For i = 1 To Len(txtCadena1.Text)  
    If i Mod 2 <> 0 Then  
        CadenaConvertida = CadenaConvertida & UCase(Mid(txtCadena1.Text, i, 1))  
    Else  
        CadenaConvertida = CadenaConvertida & LCase(Mid(txtCadena1.Text, i, 1))  
    End If  
Next i  
  
'Mostramos la cadena resultante.  
txtCadena1.Text = CadenaConvertida
```

- Dentro del evento **Click** del botón *Borrar carácter* escriba:

```
'Buscar y borrar un carácter especificado.  
Dim i As Long  
Dim Cadena As String  
Dim Caracter As String  
  
'Lee el carácter que se quiere borrar.  
Caracter = InputBox("Introduzca el carácter que desea borrar de la caja:")  
  
'Iniciamos el bucle para leer carácter por carácter el contenido de la caja.  
For i = 1 To Len(txtCadena1.Text)  
    'Verifica si el carácter que se quiere borrar coincide con uno de la caja.  
    If Mid(txtCadena1.Text, i, 1) <> Caracter Then  
        'Vamos creando la cadena sin el carácter que se quiere borrar.  
        Cadena = Cadena & Mid(txtCadena1.Text, i, 1)  
    End If  
Next i  
  
'Mostramos la cadena resultante.  
txtCadena1.Text = Cadena
```

- Corra la aplicación.
- Escriba algún texto en la primera y la segunda caja de texto.
- Utilice cada uno de los botones de comando para probar los códigos escritos en cada uno de ellos.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio9-4** para el formulario y **Ejercicio9-4** para el proyecto.



- 4.5.2 Funciones definidas por el usuario

Además de las *funciones estándar* propias de Visual Basic usted podrá crear sus propias funciones y asignarle el nombre que usted considere adecuado. Las funciones definidas por el usuario juegan un papel muy importante en una aplicación. Estas permiten crear subprogramas dentro de la aplicación que pueden ser ejecutados con solo escribir su nombre y los argumentos que usted agregó cuando la definió.

La diferencia entre las *funciones estándar* y las *funciones definidas por el usuario* radica en que en la primera usted no tiene ninguna participación en el código que posee dicha función, sin embargo, las funciones definidas por el usuario pueden ser modificadas y perfeccionadas cada vez que usted lo considere adecuado.

Las funciones definidas por el usuario tienen las siguientes ventajas:

- Evitan la repetición de código.
- Pueden ser modificadas por el usuario.
- Se adaptan a la necesidad del usuario.
- Podrá crear funciones para realizar cálculos simples y complejos.
- Podrá indicar el tipo de alcance de la función.

Para crear una función Visual Basic dispone de la palabra reservada **Function**. Una función básica tiene el siguiente formato:

Function *Nombre*(*[argumento1]*, *[argumento2]*, ...*[argumentoN]*)

Sentencias...

End Function

Donde:

<i>Nombre</i>	Es un identificador válido que indica el nombre de la función.
<i>argumento1</i> , <i>argumento2</i> , <i>argumentoN</i> .	Opcional. Son variables que proporcionan valores requeridos por la función para su correcto funcionamiento.
<i>Sentencias...</i>	Conjunto de código que se ejecutan dentro de la función.

Vamos a crear nuestra primera función para que pueda observar el uso correcto de estas. Nuestra primera función va a consistir en una función que permita calcular el sueldo bruto de un empleado a partir de sus horas trabajadas y el precio por horas de trabajo. Es obvio que Visual Basic no posee una función para realizar dicho cálculo, por tal razón el usuario se verá en la necesidad de crearla. A esta función le recomendamos poner el nombre de *SueldoBruto*, tal y como se muestra en la siguiente página:



```
Function SueldoBruto(HorasTrabajadas As Long, PrecioPorHora As Currency)
    'Calcula el sueldo bruto.
    SueldoBruto = HorasTrabajadas * PrecioPorHora
End Function
```

Al escribir esta función en su proyecto, desde el editor de código podrá calcular el sueldo bruto de un empleado con solo escribir el *nombre* de la función y especificando algún valor para los argumentos *HorasTrabajadas* y *PrecioPorHora*.

Ejemplo:

```
Dim MiSueldoBruto As Currency

MiSueldoBruto = SueldoBruto(2, 200)    'Devuelve 400.

MiSueldoBruto = SueldoBruto(5, 100)    'Devuelve 500.
```

Podrá crear funciones que no requieran ningún tipo de argumento, por ejemplo:

```
'La siguiente función muestra un mensaje de bienvenida cuando la ejecute.
Function Mensaje()
```

```
    MsgBox("Bienvenido a Visual Basic 6.0")
```

```
End Function
```

Podrá ejecutar esta función desde cualquier procedimiento. Por ejemplo, dentro de un botón de comando:

```
Private Sub Command1_Click()

    Mensaje    'Llama la función. Se muestra el mensaje de bienvenida en un MsgBox.

End Sub
```

4.6 INTRODUCCIÓN A LAS ESTRUCTURAS DE DATOS

Una **estructura de datos** es una colección de datos organizados de un modo particular. Las estructuras de datos pueden ser de dos tipos: *estructuras de datos estáticas* y *estructuras de datos dinámicas*.

Las **estructuras de datos estáticas** son aquellas en las que se asigna una cantidad fija de memoria cuando se declara la variable. Las **estructuras de datos dinámicas** son aquellas cuya ocupación de memoria puede aumentar o disminuir en tiempo de ejecución del programa.

- 4.6.2 Los Arrays (Arreglos)

Un **Array** es una estructura de datos en la que se almacena una colección de datos del mismo tipo (por ejemplo, los salarios de los empleados de una empresa). De otro modo se puede decir que un **Array** es una lista de variables con el mismo nombre que solo se diferencia por un índice.

Ejemplo:



Salario [1]
Salario [2]
Salario [3]
.
.
.
Salario [n]

Donde *Salario* es el nombre del Array y los números dentro de los paréntesis llamados índices diferencia cada uno de los elementos del Array. Cada elemento del Array pueden tomar valores distintos y tratar de modo como datos independientes, es decir, ningún elemento afecta los datos que contienen los otros elementos del Array.

Los Arrays se clasifican en:

- Unidimensionales (vectores o listas)
- Multidimensionales (tablas o matrices)

- 4.6.1.1 Arrays unidimensionales: vectores

Un Array de una dimensión es aquel que cada elemento del Array solo puede almacenar un solo valor en una sola posición. En los Arrays de una dimensión se conoce de antemano el número de elementos que contiene ya que debe ser especificado en tiempo de diseño.

Todos los elementos del Array deben ser del mismo tipo. Cuando se declara un Array de *tipo Integer* esto quiere decir, que todos los elementos del Array serán de tipo entero.

Ejemplo:

Si se desea almacenar las calificaciones de 50 estudiantes se necesitaría un Array de 50 elementos y posiblemente el array lleve por nombre calificaciones. El nombre calificaciones porque sería el más lógico para este Array pero se le puede asignar otro nombre.

Para declarar este Array lo hacemos de la misma forma en que declaramos una variable, con la única diferencia de los paréntesis y el número de elementos del Array dentro de estos:

Dim Calificaciones(50) **As Integer**

De esta manera se declara el Array Calificaciones con 50 posiciones para almacenar valores de tipo entero. Para almacenar o leer algún dato en el Array, deberá especificar el nombre del Array y dentro de los paréntesis el número de posición de ese elemento dentro del Array:

Calificaciones(1)	'Primer elemento.
Calificaciones(2)	'Segundo elemento.
Calificaciones(3)	'Tercer elemento.
Calificaciones(4)	'Cuarto Elemento.
.	
.	
.	
Calificaciones(50)	'Último elemento.



- 4.6.1.1.1 Declaración de un Array unidimensional

Para declarar un Array unidimensional debe hacer lo siguiente:

1. Declarar el nivel de alcance del Array (Público o Local).
2. Especificar un identificador valido para el Array.
3. Dentro de paréntesis () escriba el número de posiciones que tendrá el Array.
4. Indicar el tipo de datos que se almacenarán en los elementos del Array.

Su formato es:

Dim nombre_del_array (elementos) As Tipo

Donde *nombre_del_array* es un identificador valido, es decir, un nombre valido para una variable, *elemento* es un rango ordinal, enumerado, lógico o tipo carácter y *tipo* es el tipo de datos de los elementos del Array (String, Long, Double, Integer, Boolean, Currency, etc.).

Ejemplos:

'Declara un Array llamado Nombres de tipo String de 100 elementos.

Dim Nombres(100) As String

'Declara un Array llamado Números de tipo Long de 50 elementos.

Dim Numeros(50) As Long

'Declara dos Arrays de 100 elementos, uno de tipo Currency y otro de tipo Integer.

Dim Sueldo(100) As Currency, HorasTrabajadas(100) As Integer

- 4.6.1.1.2 Almacenar y leer datos en un Array unidimensional

Para almacenar y leer datos en los elementos de un Array unidimensional, deberá indicar el nombre del Array y la posición del elemento donde quiere almacenar los datos. Por ejemplo, si queremos una aplicación que permita almacenar en dos Arrays los nombres y apellidos de cien personas introducidas mediante cajas de texto diseñaríamos una aplicación similar a la siguiente:



- Abra un nuevo proyecto y diseñe el entorno que vimos en la imagen de la página anterior.
- En la sección general del editor de código escriba:

```
Dim Nombre(30) As String
Dim Apellido(30) As String
```

- En el evento **Click** del botón **Agregar** escriba:

```
'Verifica que las cajas no se dejen vacías.
If Len(Trim(Text1.Text)) = 0 Then
    MsgBox ("Debe introducir un nombre.")
    Text1.SetFocus      'Hace que la caja reciba el enfoque.

Elseif Len(Trim(Text2.Text)) = 0 Then
    MsgBox ("Debe introducir el apellido.")
    Text2.SetFocus      'Hace que la caja reciba el enfoque.
Else
    'Contamos los elementos del Array que contienen información.
    Dim i As Long
    Dim TotalLeidos As Long

    For i = 1 To UBound(Nombre)      'Repite hasta que se lean todos los elementos del Arrays.
        If Len(Trim(Nombre(i))) > 0 Then
            TotalLeidos = TotalLeidos + 1      'Acumula los elementos del Arrays que tienen datos.
                                                'Esto es para saber cual será la posición del siguiente elemento.
        End If
    Next i

    Nombre(TotalLeidos + 1) = Text1.Text      'Agrega el nombre al Array.
    Apellido(TotalLeidos + 1) = Text2.Text      'Agrega el apellido al Array.

    'Limpiamos las cajas.
    Text1.Text = ""
    Text2.Text = ""
    Text1.SetFocus
End If
```

- En el evento **Click** del botón Leer arreglo escriba:

```
Dim i As Integer
'Leemos todos los elementos del Array y lo mostramos en el ListBox.
For i = 1 To UBound(Nombre)
    If Len(Trim(Nombre(i))) > 0 Then      'Verificamos los elementos que contienen datos.
        List1.AddItem Nombre(i) & " " & Apellido(i)      'Escribimos los elementos en la ListBox.
    End If
Next i
```

- Corra la aplicación.
- Escriba un nombre y su apellido, luego haga clic en el botón **Agregar**. Repite la misma operación varias veces. Después, haga clic en el botón leer Array. Podrá notar que los datos no se perdieron, sino que se almacenaron en un arreglo para su posterior lectura.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio10-4** para el formulario y **Ejercicio10-4** para el proyecto.



- 4.6.1.1.3 Operaciones con los elementos de un Array unidimensional

Una de las principales ventajas de los arreglos es que se puede realizar un sin número de operaciones con los elementos que este contiene. Entre las cuales tenemos: consultas, cálculos matemáticos, concatenación, eliminación y modificación de uno o varios elementos, etc. Para realizar un determinado tipo de operación, debemos de recorrer todos los elementos del Array utilizando un bucle **for**, y luego establecer los criterios u operaciones que desea realizar con dichos elemento. Para entender esto tenemos el siguiente problema:

Imaginase que usted tiene almacenado en una Array los nombres, apellidos y sueldos de 2000 personas y usted necesita obtener los siguientes reportes:

- a) Nombres y apellidos de personas que ganan más de 5000 pesos.
- b) Nombres de las personas cuyo apellido sea Rodríguez.
- c) Nombre y apellidos de las personas que ganan menos de 2000 pesos.

Para obtener estos reportes tendremos que hacer un recorrido por todos los elementos del Array, y mediante condiciones verificar que se cumplan los criterios especificados. Para esto necesitaríamos un código similar al que se muestra a continuación:

Dim i As Integer

'Suponemos que el arrays nombre ya exista y contenga los nombres de 2000 personas.

'Suponemos que existen también los arrays apellidos y sueldos.

For i = 1 To UBound(nombres)

'Obtenemos el primer reporte. Personas que ganen más de 5000 pesos.

If sueldos(i) > 5000 Then 'Verificamos los sueldos mayores de 5000 dentro del array.

'Si se encuentra un sueldo mayor de 5000 escribimos en una ListBox el nombre y el apellido de la persona.

List1.AddItem nombres(i) & " " & apellidos(i)

End If

'Obtenemos el segundo reporte. Nombres de las personas cuyo apellido sea Rodríguez.

If UCase(apellidos(i)) = UCase("rodriguez") Then 'Verificamos los apellidos Rodríguez.

'Si encontramos algún apellido igual a Rodríguez mostramos el nombre y el apellido de la persona en otra ListBox.

List2.AddItem nombre(i) & " " & apellidos(i)

End If

'Obtenemos el tercer reporte. Nombres y apellidos de las personas que ganan menos de 2000 pesos.

If sueldos(i) < 2000 Then 'Verificamos los sueldos menores de 2000 dentro del array Sueldos.

'Si se encuentra un sueldo menor de 2000 pesos escribimos en una ListBox el nombre y el apellido de la persona.

List3.AddItem nombres(i) & " " & apellidos(i)

End If

Next i



- 4.6.1.1.4 Ejercicios prácticos

1.- Hacer un programa que pida los datos de treinta empleados mediante 7 cajas de texto: *Nombre*, *Apellido*, *Departamento Encargado*, *Sueldo*, *Dirección* y *Teléfono*. Los datos deben ser mostrados de la siguiente manera:

<i>Nombre</i>	<i>Apellido</i>	<i>Dep. Encargado</i>	<i>Sueldo</i>	<i>Dirección</i>	<i>Teléfono</i>
xxxxxxx	xxxxxxx	xxxxxxxxxxx	xxxxxx	xxxxxxx	xxxxxxx
xxxxxxx	xxxxxxx	xxxxxxxxxxx	xxxxxx	xxxxxxx	xxxxxxx
xxxxxxx	xxxxxxx	xxxxxxxxxxx	xxxxxx	xxxxxxx	xxxxxxx

2.- Hacer un programa que mediante cajas de texto almacene en dos arrays 20 números. Luego se debe sumar los elementos de cada array y almacenarse en otro arrays. Mostrar los resultados en una ListBox.

108

3.- Hacer un programa que multiplique por tres los elementos de un array. El programa debe permitir almacenar 10 números mediante una caja de texto. Luego de estar almacenados, se debe multiplicar por tres cada uno de los elementos del array. Los productos deben ser almacenados en otro array. Mostrar los resultados en una ListBox.

Ejemplo:

10	30
20	60
5	15

4.- Escribir un programa que reste el contenido de dos arrays. Cada array debe tener 10 elementos y la diferencia debe ser almacenada en otro array. Mostrar los resultados en una ListBox.

5.- Escribir un programa que divida el contenido de tres arrays. Cada array debe tener 10 elementos y el cociente debe ser almacenado en otro array. Mostrar los resultados en una ListBox.

6.- Escribir un programa que calcule el resto de los valores de dos arrays. Cada array debe tener 10 elementos y el resto de la división debe ser almacenado en otro array. Mostrar los resultados en una ListBox.

7.- Hacer un programa que permita almacenar 15 valores en un array y verificar cuales de ellos están más cercano al número 100 y luego mostrarlos en una ListBox.

8.- Hacer un programa que permita almacenar 15 valores en un array y luego verificar cuales de ellos es positivo y luego mostrarlos en una ListBox.

9.- Hacer un programa que calcule el cuadrado de los elementos de un array. Los cuadrados deben ser almacenados en otro array y mostrados en una ListBox.

10.- Hacer un programa que pida 5 nombres y luego esos nombres deben ser mostrados de forma inversa. Mostrar los resultados en una ListBox.

Ejemplo:

<i>Datos de entrada</i>	<i>Datos de salida</i>
Carlos	José
Pablo	Nelson
Nelson	Pablo
José	Carlos



- 4.6.1.2 Arrays multidimensionales: tablas y matrices

Son aquellos Arrays que poseen dos índices y uno de los índices representa las columnas del Array y el otro representa las filas. Estos Arrays llamados también Arrays bidimensionales o tablas almacenan la información como registros de datos.

Para localizar y almacenar un valor en el Array se deben especificar dos posiciones, uno para la fila y otro para la columna. Un Array bidimensional tiene la siguiente estructura:

		Columnas							
		1	2	3	4	5	6	7	8
Filas	1	E(1,1)	E(1,2)	E(1,3)	Carlos	Pablo	Nelson		E(1,8)
	2								
	3								
	4								
	5								
	6	E(6,1)							E(6,8)

Los valores en la parte de arriba de la tabla representan las columnas del Array y los valores a la izquierda de la tabla representan las filas del Array. Para referirse a cada elemento del Array es necesario especificar el nombre del Array seguido del valor de la fila y el valor de la columna.

Ejemplo:

Para leer el valor que posee la primera fila y la cuarta columna del Array se haría de la siguiente manera:

E(1, 4) : Donde E represente el nombre del Array, el valor 1 el número de la fila y 4 el número de la columna.

En este caso el valor de este elemento del Array es *Carlos* ya que este es el dato que se encuentra en la posición especificada de la tabla.

- 4.6.1.2.1 Recorrido por las filas y columnas de un Array multidimensional

Como se ha dicho anteriormente es necesario especificar el valor de la fila y luego el valor de la columna para leer o almacenar un valor en cualquier elemento del Array. En muchas ocasiones nos veremos en la necesidad de leer el contenido de una fila del Array o el contenido de una columna del Array. Para explicar esto lo haremos con el siguiente ejemplo:

Primero declaramos un Array llamado agenda con 6 filas y 5 columnas. Este Array también es llamado Array 6x5, es decir, que posee 30 elementos.

Declaración de un Arrays multidimensional. Indicamos el número de filas y el número de columnas.
Dim agenda(6, 5) **As String**

Esta declaración crea una tabla lógica llamada **agenda** que posee seis filas y cinco columnas, es decir, una tabla con treinta celdas. En términos de base de datos, se podría decir que esta es una tabla que permite almacenar seis registros y cinco campos. La estructura sería como el que se muestra en la siguiente página...



- 4.6.1.2.2 Declaración de un Array multidimensional

Declarar un Array multidimensional es similar a la declaración de un Array unidimensional:

5. Declarar el nivel de alcance del Array (Público o Local).
6. Especificar un identificador valido para el Array.
7. Dentro de paréntesis () escriba el número de filas y el número de columnas que tendrá el Array.
8. Indicar el tipo de datos que se almacenarán en los elementos del Array.

Su formato es:

Dim nombre_del_array (filas, columnas) As Tipo

Donde *nombre_del_array* es un identificador valido, es decir, un nombre valido para una variable, *fila* es un valor numérico que indica la cantidad de registros que podrá almacenar la tabla, *columnas* es un valor numérico que indica la cantidad de campos que tendrá la tabla y *tipo* es el tipo de datos de los elementos del Array (String, Long, Double, Integer, Boolean, Currency, etc.).

Ejemplos:

'Declara un Array llamado empleados que permitirá almacenar 10 registros de cinco campos.
Dim empleados(10, 5) As String

'Declara un Array llamado clientes que permitirá almacenar 10 registros de cuatro campos.
Dim clientes(10, 4) As Long

El primer Array llamado empleados podría tener la siguiente estructura:

Nombre	Apellido	Cargo	Sueldo Bruto	Sueldo Neto

El segundo Array llamado clientes podría tener la siguiente estructura:

Nombre	Apellido	Teléfono	Dirección

El nombre de cada campo dependerá de los datos que el programador desea almacenar en la en el Array. Estos campos cambiarán dependiendo de los requerimientos de la aplicación.



	1	2	3	4	5
1	Yokasty	Abreu	597-4720	01/02/1980	Los Mina
2	Odalís	Merra	536-2152	05/04/1970	Las América
3	Ángel	Castillo	699-5235	02/03/1982	El Duarte
4	Rosa	Paulino	598-5566	09/10/1985	El Luperón
5	Andrés	Sánchez	866-5544	04/08/1950	Los tres ojos
6	Elvis	Santana	565-5251	02/08/1965	Lucerna

1. Si se desea leer el primer registro del Array que corresponde a la primera fila y a las columnas 1,2,3,4,5:

agenda(1, 1) agenda(1, 2) agenda(1, 3) agenda(1, 4) agenda(1, 5)

Como se puede notar el valor 1 no cambia, ya que este es una constante que representa la primera fila del Array.

Ejemplo:

'Agrega a una ListBox el primer registro de la tabla.

List1.AddItem agenda(1,1) & " " & agenda(1,2) & " " & agenda(1, 3) & " " & agenda(1, 4) & " " & _
agenda(1,5)

Con esta línea de código agregaríamos el registro Yokasty Abreu 597-4720 01/02/1980 Los Mina en una ListBox.

- 4.6.1.2.3 Almacenar y leer datos en un Array multidimensional

Para Almacenar y leer datos en un Array unidimensional haga lo siguiente:

1. Utilice un bucle **For** para leer todas las filas del Array.
2. Especifique la variable control del bucle para representar las filas del Array.
3. Especifique el número que representa cada columna que desea leer.

Por ejemplo, si queremos leer todos los registros del Array agenda escribimos un código similar a este:

Dim f As Integer

For f = 1 To UBound(agenda) *'Repite hasta que se lean todos los registros.*

'Leemos todos los campos. Las filas cambian cada vez que se repite el bucle.

'Los campos siempre serán los mismo. Lo único que debe de cambiar es el número de la fila.

List1.AddItem agenda(f,1) & " " & agenda(f,2) & " " & agenda(f, 3) & " " & agenda(f, 4) & " " & _
agenda(f,5)

Next f

Podrá notar que hemos especificado una variable para recorrer cada una de las filas del Arreglo. Cada vez que el bucle se repite el valor de la fila cambia pasando así por todos los registros de la tabla. Debe tener en cuenta que los valores de las columnas deben ser siempre los mismo, es decir, no deben cambiar como lo hicimos con las filas. No es necesario especificar todas las columnas, si solo desea leer dos campos, como por ejemplo, el nombre y el apellido, solo tendrá que especificar las columnas 1 y 2, las demás 3, 4, 5 se excluyen.



Ejercicio:

La siguiente aplicación permite almacenar en un arreglo multidimensional el nombre, apellido, teléfono y dirección de varias personal.

- Abra un nuevo proyecto.
- Diseñe la siguiente pantalla:

- En la sección general escriba:

```
Dim Agenda(200, 4) As String
```

'Define una tabla con 200 filas y cuatro columnas.

- En el evento **Click** del botón **Guardar** escriba:

'Verifica que se escriba el nombre y el apellido de la persona.

```
If Len(Trim(Text1.Text)) = 0 Then  
    MsgBox ("Debe escribir un nombre.")  
    Text1.SetFocus
```

```
Elseif Len(Trim(Text2.Text)) = 0 Then  
    MsgBox ("Debe escribir el apellido.")  
    Text2.SetFocus
```

```
Else
```



```

Dim i As Long
Dim NoElementos As Long

'Contamos la cantidad de registro que tiene el arreglo
For i = 1 To UBound(agenda)      'Repetimos hasta leer todos los elementos.
    If Len(Trim(agenda(i, 1))) > 0 Then 'Verificamos los elementos que tienen datos.
        NoElementos = noelementos + 1    'Almacenamos los registros que tienen datos.
    End If
Next i

'Almacenamos los nuevos datos en el arreglo. Para esto incrementamos en 1 el total
agenda(NoElementos + 1, 1) = Text1.Text    'Almacenamos el nombre.
agenda(NoElementos + 1, 2) = Text2.Text    'Almacenamos el apellido.
agenda(NoElementos + 1, 3) = Text3.Text    'Almacenamos el teléfono.
agenda(NoElementos + 1, 4) = Text4.Text    'Almacenamos la dirección.

'Limpiamos las cajas.
Text1.Text = ""
Text2.Text = ""
Text3.Text = ""
Text4.Text = ""
Text1.SetFocus

MsgBox ("El registro ha sido agregado.")

End If
    
```

- Dentro del evento **Click** del botón **Leer arreglo** escriba:

```

List1.Clear          'Limpiamos el ListBox.
Dim i As Long
For i = 1 To UBound(agenda)      'Lee todos los registros del Arreglo.
    If Len(Trim(agenda(i, 1))) > 0 Then

        'Agrega los elementos leídos al control ListBox.
        List1.AddItem agenda(i, 1) & " " & agenda(i, 2) & " " & agenda(i, 3) & " " & agenda(i, 4)
    End If
Next i
    
```

- Corra la aplicación.
- Escriba los datos de varias personas. Cuando escriba los datos de cada persona haga clic en el botón **Guardar** para almacenar los datos en el arreglo.
- Después que haya agregado los registros, haga clic en el botón **Leer arreglo**. Podrá notar que todos los datos se encuentran almacenados en el arreglo.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio11-4** para el formulario y **Ejercicio11-4** para el proyecto.



- 4.6.1.2.4 Ejercicios propuestos

- 1.- Hacer un programa que sume los elementos de dos tablas o matrices. La primera matriz llamada M1 y la segunda matriz llamada M2 se sumarán y sus resultados serán almacenados en una tercera matriz llamada MR.
- 2.- Hacer un programa que multiplique los elementos de dos matrices. La primera matriz llamada M1, la segunda matriz llamada M2 y la matriz producto MP.
- 3.- Escribir un programa que copie todos los elementos de una tabla en otra tabla.
- 4.- Escribir un programa que cambien los elementos de dos matrices. Todos los elementos de la primera matriz pasarán a la segunda matriz y todos los elementos de la segunda matriz pasarán a la primera matriz.
- 5.- Escribir un programa que multiplique por cuatro todos los elementos de dos matrices. Luego de ser multiplicados por cuatro deben ser sumados y almacenados en otra matriz su resultado.
- 6.- Escribir un programa que divida entre dos todos los elementos de dos matrices. Luego de ser divididos deben ser sumados y almacenados en otra matriz su resultado.
- 7.- Escribir un programa que divida dos matrices. La primera matriz llamada *dividendo* que tendrá los números a dividir y la segunda matriz llamada *divisora* que tendrá los números que dividirán los elementos de la primera matriz. Los cocientes o resultados serán almacenados en otra matriz llamada *cociente*.

4.7 PREGUNTAS PARA CONTESTAR

1. ¿Qué es un array?
2. ¿Qué es un arreglo unidimensional?
3. ¿Qué es un arreglo multidimensional?
4. ¿Cómo recorreremos las filas de un array unidimensional?
5. ¿Cómo recorreremos las filas de un array multidimensional?
6. ¿Qué se debe tomar en cuenta cuando recorreremos las filas de un array multidimensional?
7. ¿Qué tipo de datos pueden almacenar los arrays?
8. ¿Cuántas celdas tiene un array 5x5?
9. ¿Cuántas filas tiene un array 10x3?
10. ¿Qué se especifica en el primer índice de un array multidimensional?



CONTENIDO

5.4 ¿Qué son los menús?

5.5 Elementos de los menús.

5.6 El Editor de Menú (Menu Editor).

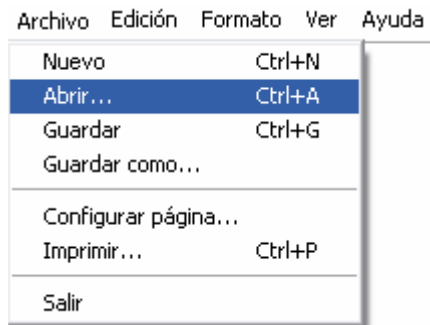
- 5.3.1 Descripción de los elementos del Editor de Menús.
- 5.3.2 Creación de menús en Visual Basic 6.0.
- 5.3.3 Creación de submenús.

5.4 Eventos sobre los menús.

5.5 Ejercicios propuestos.



5.1 ¿QUÉ SON LOS MENÚS?



Al conjunto de opciones presentadas al usuario para su selección en una zona determinada de la pantalla se les llaman **menús**. La gran mayoría de las aplicaciones de Windows poseen menús que contienen todas las herramientas necesarias que hacen posible la utilidad de la aplicación.

Los programas tales como Word, Excel, WordPad, incluyen **Barras de Menús** y dentro de estas *barras* se encuentran los menús y dentro de los menús las opciones para cada menú.

Por ejemplo, el menú **Archivo (File)** de Microsoft Word incluye opciones o comandos tales como: Nuevo, Abrir, Cerrar, Guardar, Guardar como, Imprimir, etc.

Los menús presentan sobre los demás *controles* la ventaja de que ocupan menos espacio en pantalla, pero tienen la limitante de que las opciones o comandos de los menús no están visibles hasta que se despliega totalmente el menú.

5.2 ELEMENTOS DE LOS MENÚS

Entre los elementos principales de los menús tenemos los *accesos directos*, los *indicadores de cuadro de dialogo* (...), el *indicador de submenú* (►), las *líneas divisoras* y las *imágenes*. Todos estos elementos permiten una mejor legibilidad al trabajar con los menús.

- *Los accesos directos*: son aquellos que mediante combinaciones de teclas nos permiten acceder a un menú o a una opción de un menú. Por ejemplo, para desplegar el menú **Archivo (File)** de Microsoft Word basta con pulsar las combinaciones de teclas **Alt + A** (en español) ó **Alt + F** (en inglés), o para activar la *ventana de dialogo* **Abrir** se pulsan las teclas **Ctrl + A** (en español) ó **Ctrl + O** (en inglés).
- *Los indicadores de cuadro de dialogo (...)*: Estos están representados por *tres puntos suspensivos* (...) que indican que al seleccionar esta opción se mostrará una ventana de dialogo dónde se requerirá de algún *evento* proporcionado por el usuario.
- *El indicador de submenú (►)*: El indicador de submenú esta representado por una *flecha a la derecha*, que indica que el elemento de ese menú posee otras opciones de menú llamado submenú.
- *Las líneas divisoras*: Así como su nombre lo indica, son *líneas divisoras* que dividen entre un conjunto de opciones y otro conjunto de opciones dentro de un mismo menú. Las líneas divisoras no tienen ninguna función especial dentro de un menú, solo indican la división entre un conjunto de opciones dentro de un mismo menú.
- *Las imágenes*: Las *imágenes* en los menús juegan un papel muy importante, ya que ilustran gráficamente la función de un elemento dentro de ese menú. Por ejemplo, la opción **Imprimir** del menú **Archivo** de Microsoft Word posee la imagen de una impresora , que indica que es la opción de imprimir.



5.3 EL EDITOR DE MENÚ (MENU EDITOR)

Visual Basic posee una potente herramienta para la creación de menús y todos los elementos que estos los componen. El **Editor de Menú (Menu Editor)** es la herramienta que permite la creación de estos. Para activar o mostrar la ventana del **Menu Editor**, haga clic en la opción **Menu Editor** del menú **Tools** (*herramientas*) o bien, haga clic en el botón  correspondiente al **Menu Editor** de la barra de herramientas estándar. Al seleccionar esta opción se mostrará la siguiente ventana:

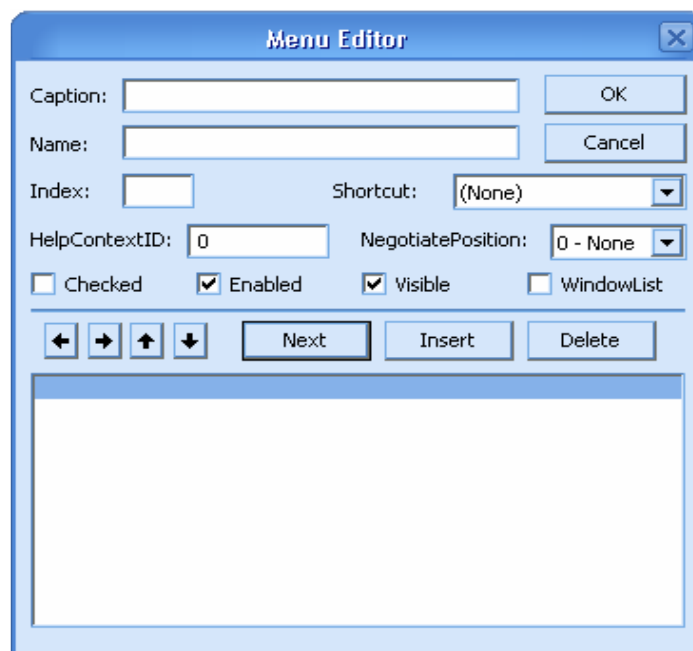


Figura 5.1. Menú Editor de Visual Basic 6.0

- 5.3.1 Descripción de los elementos del Editor de Menús

En la **figura 5.1** se muestra la ventana del *Editor de Menús* que posee todas las herramientas necesarias para la creación de estos. A continuación se describen cada uno de los elementos del *Editor de Menús*.

- **Caption (Título):** En esta *caja de texto* se escribe el nombre del menú o elemento de un menú o submenú. En esta *caja de texto*, el carácter (& **ampersand** tiene un significado especial, ya que indica que el carácter o la letra a la derecha de él será el acceso directo a ese menú o elemento de un menú o submenú. Por ejemplo, si un menú denominado **Archivo** tiene el carácter (& **ampersand** posicionado a la izquierda, es decir, delante de la letra **A** significa que sólo basta con pulsar las combinaciones de las teclas **Alt + A** para tener acceso a ese menú, elemento de un menú o submenú.
- **Name (Nombre):** En esta *caja de texto* se especifica el nombre del menú, elemento de un menú o submenú, que se utiliza para referenciar a ese menú en el *editor de códigos*.
- **Index (Índice):** La *caja de texto Índice* hace referencia a la posibilidad de crear *arrays* de menús.
- **ShortCut (Acceso directo):** Permite asignar acceso directo a los elementos de cada menú. En esta lista se muestran una serie de combinaciones, el cual el usuario puede seleccionar la más adecuada para ese elemento de menú.



- *Checked (Verificación)*: Permite agregar un cotejo de verificación (✓) a un elemento de un menú.
- *Enabled (Habilitado)*: Indica si el menú o elemento del menú responderá a los eventos del usuario.
- *Visible (Visible)*: Indica si el menú o elemento del menú estará visible o no.

Las demás opciones *HelpContextID*, *NegotiatePosition* y *WindowList* son pocas usadas, por tal razón sean limitado su descripción. Para más información acerca de estas opciones, consulte la ayuda (help) de Visual Basic.

A continuación, describiremos cada uno de los botones del menú editor:



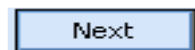
El botón *flecha izquierda* se utiliza para eliminar cuatro puntos suspensivos a la izquierda, que indican si ese elemento es un comando de un menú o submenú.



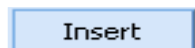
El botón *flecha derecha* agrega cuatro puntos suspensivos (...) a la izquierda de un elemento de un menú o submenú. Si este botón se pulsa dos veces se agregan ocho puntos suspensivos (.....) a la izquierda de un elemento, indicando que es un elemento de un submenú. Puede pulsar este botón las veces que se necesario siempre y cuando sepa establecer el orden jerárquico entre los elementos del menú.



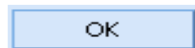
Los botones *flecha arriba* y *flecha abajo* se utilizan para desplazarse entre los menús, elementos de menú o submenú.



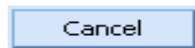
El botón *Next (siguiente)* se utiliza para agregar un menú, un elemento de menú o submenú. Al pulsar este botón sobre un elemento ya agregado se inserta otra nueva línea en el editor de menú con el mismo formato de la línea interior, es decir, si el elemento sobre el cual se pulsa este botón es un elemento de un menú, entonces la línea que se agrega también será un elemento de menú para ese mismo menú.



El botón *insertar* se utiliza para insertar un elemento o ítem en la posición de un elemento seleccionado quedando este debajo del nuevo elemento.



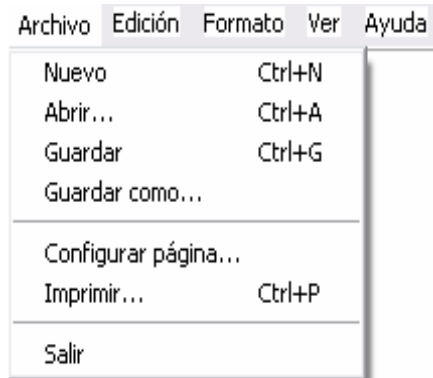
El botón *OK (Aceptar)* guarda todas las modificaciones echas en el *Editor de Menús*.



El botón *Cancel (cancelar)* omite cualquier modificación echa en el *Editor de Menús* y al mismo tiempo cierra la ventana del editor.

5.3.2 Creación de menús en Visual Basic 6.0

Como dijimos anteriormente, para crear un menú utilizaremos el Menú Editor localizado en la barra de herramientas del EID de Visual Basic y representado por la siguiente imagen . Crear un menú es un proceso práctico y por tal razón vamos a crear el menú que se muestra en la siguiente página:



Creación del menú Archivo

Pasos a seguir:

- 1.- Abra un nuevo proyecto desde el menú **File**.
- 2.- Haga clic en el icono del Editor de Menús .
- 3.- En la *caja de texto* **Caption (título)** escriba **&Archivo** y en la *caja de texto* **Name (nombre)** escriba **menuArchivo**.
- 4.- Haga clic en el botón **Next (siguiente)**.
- 5.- Haga clic en el botón *flecha derecha* (→) para agregar cuatro puntos suspensivos (...).
- 6.- Haga clic en la caja de texto **Caption (título)** y escriba **&Nuevo** y en la caja de texto **Name (nombre)** escriba **elementoNuevoMenuArchivo**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + N**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 7.- En la *caja de texto* **Caption (título)** escriba **&Abrir...** y en la *caja de texto* **Name (nombre)** escriba **elementoAbrirMenuArchivo**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + A**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 8.- En la *caja de texto* **Caption (título)** escriba **&Guardar** y en la *caja de texto* **Name (nombre)** escriba **elementoGuardarMenuArchivo**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + G**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 9.- En la *caja de texto* **Caption (título)** escriba **&Guardar como...** y en la *caja de texto* **Name (nombre)** escriba **elementoGuardarComoMenuArchivo**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 10.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta "-" y en la *caja de texto* **Name (nombre)** escriba **Linea1MenuArchivo**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 11.- En la *caja de texto* **Caption (título)** escriba **&Configurar página...** y en la *caja de texto* **Name (nombre)** escriba **elementoConfigurarPaginaMenuArchivo**, y a continuación, haga clic en el botón **Next (Siguiente)**.



12.- En la *caja de texto* **Caption (título)** escriba **&Imprimir...** y en la *caja de texto* **Name (nombre)** escriba **elementoImprimirMenuArchivo**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + P**, y a continuación, haga clic en el botón **Next (Siguiente)**.

13.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta "-" y en la *caja de texto* **Name (nombre)** escriba **Linea2MenuArchivo**, y a continuación, haga clic en el botón **Next (Siguiente)**.

14.- En la *caja de texto* **Caption (título)** escriba **Sa&lr** y en la *caja de texto* **Name (nombre)** escriba **elementoSalirMenuArchivo**, y a continuación, haga clic en el botón **Next (Siguiente)**.

Creación del menú Edición

Archivo	Edición	Formato	Ver	Ayuda
	Deshacer	Ctrl+Z		
	Cortar	Ctrl+X		
	Copiar	Ctrl+C		
	Pegar	Ctrl+V		
	Eliminar	Supr		
	Buscar...	Ctrl+B		
	Buscar siguiente	F3		
	Reemplazar...	Ctrl+R		
	Ir a...	Ctrl+T		
	Seleccionar todo	Ctrl+E		
	Hora y fecha	F5		

Pasos a seguir:

1.-Haga clic en el botón *flecha izquierda* (←) para eliminar los cuatro puntos suspensivos a la izquierda.

2.- En la *caja de texto* **Caption (título)** escriba **&Edición** y en la *caja de texto* **Name (nombre)** escriba **MenuEdicion**, y a continuación, haga clic en el botón **Next (Siguiente)**.

3.-Haga clic en el botón *flecha derecha* (→) para agregar los cuatro puntos suspensivos a la izquierda.

4.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta "-" y en la *caja de texto* **Name (nombre)** escriba **Linea1MenuEdicion**, y a continuación, haga clic en el botón **Next (Siguiente)**.

5.- En la *caja de texto* **Caption (título)** escriba **&Deshacer** y en la *caja de texto* **Name (nombre)** escriba **elementoDeshacerMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + Z**, y a continuación, haga clic en el botón **Next (Siguiente)**.

6.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta "-" y en la *caja de texto* **Name (nombre)** escriba **Linea2MenuEdicion**, y a continuación, haga clic en el botón **Next (Siguiente)**.



Capítulo 5

7.- En la *caja de texto* **Caption (título)** escriba **&Cortar** y en la *caja de texto* **Name (nombre)** escriba **elementoCortarMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + X**, y a continuación, haga clic en el botón **Next (Siguiente)**.

8.- En la *caja de texto* **Caption (título)** escriba **C&opiar** y en la *caja de texto* **Name (nombre)** escriba **elementoCopiarMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + C**, y a continuación, haga clic en el botón **Next (Siguiente)**.

9.- En la *caja de texto* **Caption (título)** escriba **&Pegar** y en la *caja de texto* **Name (nombre)** escriba **elementoPegarMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + V**, y a continuación, haga clic en el botón **Next (Siguiente)**.

10.- En la *caja de texto* **Caption (título)** escriba **&Eliminar** y en la *caja de texto* **Name (nombre)** escriba **elementoEliminarMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Del** ó **Supr**, y a continuación, haga clic en el botón **Next (Siguiente)**.

11.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta “-” y en la *caja de texto* **Name (nombre)** escriba **Linea3MenuEdicion**, y a continuación, haga clic en el botón **Next (Siguiente)**.

12.- En la *caja de texto* **Caption (título)** escriba **&Buscar...** y en la *caja de texto* **Name (nombre)** escriba **elementoBuscarMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + B**, y a continuación, haga clic en el botón **Next (Siguiente)**.

13.- En la *caja de texto* **Caption (título)** escriba **B&uscar siguiente** y en la *caja de texto* **Name (nombre)** escriba **elementoBuscarSiguienteMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **F3**, y a continuación, haga clic en el botón **Next (Siguiente)**.

14.- En la *caja de texto* **Caption (título)** escriba **Re&emplazar** y en la *caja de texto* **Name (nombre)** escriba **elementoReemplazarMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + R**, y a continuación, haga clic en el botón **Next (Siguiente)**.

15.- En la *caja de texto* **Caption (título)** escriba **&lr a...** y en la *caja de texto* **Name (nombre)** escriba **elementolrASiguienteMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + T**, y a continuación, haga clic en el botón **Next (Siguiente)**.

16.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta “-” y en la *caja de texto* **Name (nombre)** escriba **Linea4MenuEdicion**, y a continuación, haga clic en el botón **Next (Siguiente)**.

17.- En la *caja de texto* **Caption (título)** escriba **Seleccionar &todo** y en la *caja de texto* **Name (nombre)** escriba **elementoSeleccionarTodoMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **Ctrl + E**, y a continuación, haga clic en el botón **Next (Siguiente)**.

18.- En la *caja de texto* **Caption (título)** escriba **&Hora y fecha** y en la *caja de texto* **Name (nombre)** escriba **elementoHoraFechaMenuEdición**. En la *caja combinada* de **Shortcut** seleccione **F5**, y a continuación, haga clic en el botón **Next (Siguiente)**.

Creación del menú Formato

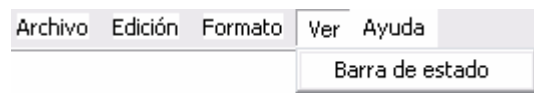




Pasos a seguir:

- 1.-Haga clic en el botón *flecha izquierda* (←) para eliminar los cuatro puntos suspensivos a la izquierda.
- 2.- En la *caja de texto* **Caption (título)** escriba **&Formato** y en la *caja de texto* **Name (nombre)** escriba **MenuFormato**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 3.-Haga clic en el botón *flecha derecha* (→) para agregar los cuatro puntos suspensivos a la izquierda.
- 4.- En la *caja de texto* **Caption (título)** escriba **Ajus&te de línea** y en la *caja de texto* **Name (nombre)** escriba **ElementoAjusteDeLineaMenuFormato**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 5.- En la *caja de texto* **Caption (título)** escriba **&Fuente** y en la *caja de texto* **Name (nombre)** escriba **ElementoFuenteMenuFormato**, y a continuación, haga clic en el botón **Next (Siguiente)**.

Creación del menú Ver



Pasos a seguir:

- 1.-Haga clic en el botón *flecha izquierda* (←) para eliminar los cuatro puntos suspensivos a la izquierda.
- 2.- En la *caja de texto* **Caption (título)** escriba **&Ver** y en la *caja de texto* **Name (nombre)** escriba **MenuVer**, y a continuación, haga clic en el botón **Next (Siguiente)**.
- 3.-Haga clic en el botón *flecha derecha* (→) para agregar los cuatro puntos suspensivos a la izquierda.
- 4.- En la *caja de texto* **Caption (título)** escriba **Barra &de estado** y en la *caja de texto* **Name (nombre)** escriba **ElementoBarraEstadoMenuVer**, y a continuación, haga clic en el botón **Next (Siguiente)**.

Creación del menú Ayuda



Pasos a seguir:

- 1.-Haga clic en el botón *flecha izquierda* (←) para eliminar los cuatro puntos suspensivos a la izquierda.
- 2.- En la *caja de texto* **Caption (título)** escriba **Ay&uda** y en la *caja de texto* **Name (nombre)** escriba **MenuAyuda**, y a continuación, haga clic en el botón **Next (Siguiente)**.



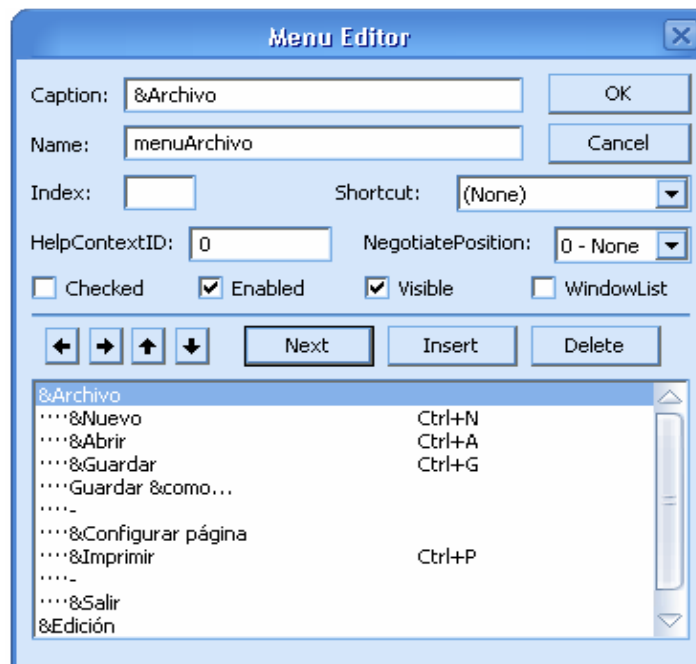
3.- Haga clic en el botón *flecha derecha* (→) para agregar los cuatro puntos suspensivos a la izquierda.

4.- En la *caja de texto* **Caption (título)** escriba **&Temas de Ayuda** y en la *caja de texto* **Name (nombre)** escriba **ElementoTemasDeAyudaMenuAyuda**, y a continuación, haga clic en el botón **Next (Siguiente)**.

5.- En la *caja de texto* **Caption (título)** escriba un guión o el símbolo de resta "-" y en la *caja de texto* **Name (nombre)** escriba **Linea1MenuAyuda**, y a continuación, haga clic en el botón **Next (Siguiente)**.

6.- En la *caja de texto* **Caption (título)** escriba **&Acerca del Bloc de notas** y en la *caja de texto* **Name (nombre)** escriba **ElementoAcercaMenuAyuda**, y a continuación, haga clic en el botón **Next (Siguiente)**.

7.- Haga clic en el botón **OK** para finalizar.



NOTA: En caso de algún error, verifique si ha escrito correctamente los nombres (**Name**) de los menús, elementos de los menús y submenús propuestos. También verifique si ha establecido el orden jerárquico adecuado entre los ítems o elementos de menús y submenús. Si el problema persiste consulte con su maestro, o bien, envíe un E-Mail a { [HYPERLINK "mailto:twinsmaster02@hotmail.com"](mailto:twinsmaster02@hotmail.com) } explicando el problema.

Después de haber realizados todos estos pasos, corra la aplicación pulsando la tecla **F5** y desplácese por todos los menús ya creado.

▪ Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio1-5** para el formulario y **Ejercicio1-5** para el proyecto.



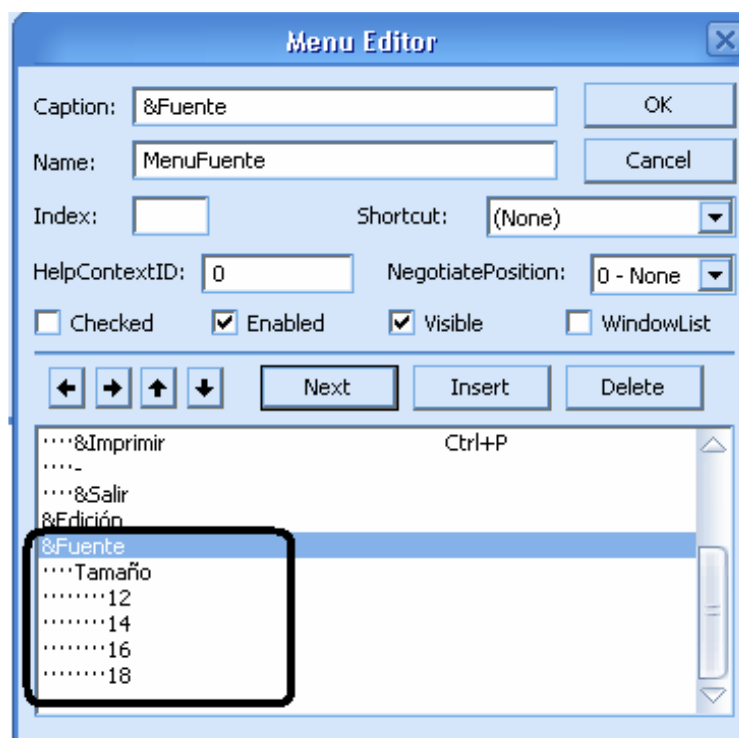
- 5.3.3 Creación de submenús

Los submenús representan a los menús que se encuentran dentro de un menú, es decir, dentro de un elemento de un menú o un elemento de un submenú. Un submenú está representado por medio de una flecha a la derecha (►). A continuación se muestra un submenú típico en un menú:



Se puede observar en la que el menú **Fuente** contiene un *elemento* o *ítem* con el título **Tamaño** y este elemento está precedido por una *pequeña flecha* (►), que indica que existe un submenú dentro de ese elemento.

La creación de submenús consiste en establecer un orden jerárquico de menor a mayor, dónde el elemento que contendrá el submenú debe tener cuatro puntos suspensivos menos a la izquierda, como se muestra a continuación:





El recuadro resaltado es la zona donde se especifica el menú **Fuente** con sus elementos. Se puede observar que el elemento **Tamaño** posee cuatro *puntos suspensivos (...)* menos que los *elementos* 12, 14, 16, 18. Esto indica que estos elementos pertenecen al elemento **Tamaño** que esta dentro del menú **Fuente**.

5.4 EVENTOS SOBRE LOS MENÚS

Es lógico, que **Visual Basic 6.0** permita agregar líneas de código a cada uno de los elementos de los menús y submenús que se agreguen a una aplicación. Esto lo hace mediante el *evento general* de los menús, el *evento Click*.

Para agregar código a algún elemento de un menú o submenú, sólo tiene que hacer clic sobre el menú en modo de diseño y luego hacer otro clic sobre el elemento al cual quiere agregar el código.

El *evento Click* debe parecerse a esto:

Private Sub ElementoX_Click ()

Sentencias...

End Sub

Podrá activar o desactivar una opción de algún menú mediante la propiedad **Enabled**. Por ejemplo, si al cargar una aplicación usted desea que algunas opciones de los menús aparezcan deshabilitada, entonces, dentro del evento **Load** del formulario deberá escribir el nombre de la opción del menú con la propiedad **Enabled** establecida a **False**:

Private Sub Form_Load()

miOpcion1.Enabled = False 'Desactiva una opción que tiene el nombre MiOpcion1.
MiOpcion2.Enabled = False 'Desactiva una opción que tiene el nombre MIOpcion2.

End If

Podrá activar cualquier opción desactivada desde cualquier procedimiento, estableciendo el valor **True** en la propiedad **Enabled** de dicha opción.

5.5 EJERCICIOS PROPUESTOS

- 1.- Ejecute la aplicación *Microsoft Paint* y diseñe los menús y las opciones de cada menú que contiene el programa. Para ejecutar el *Paint*, haga clic en el menú *Inicio → Programas → Accesorios → Paint*.
- 2.- Ejecute la aplicación *Microsoft WordPad* y diseñe los menús y las opciones de cada menú que contiene el programa. Para ejecutar el *WordPad*, haga clic en el menú *Inicio → Programas → Accesorios → WordPad*.
- 3.- Ejecute la aplicación *Microsoft Word* y diseñe los menús y las opciones de cada menú que contiene el programa.



CONTENIDO

6.1 Los formularios.

- 6.1.1 Concepto de formulario.
- 6.1.2 Propiedades de los formularios.
- 6.1.3 Eventos de los formularios.
- 6.1.4 Trabajando con múltiples formularios.
 - 6.1.4.1 Descargar un formulario.
 - 6.1.4.2 Cargar un formulario.
 - 6.1.4.3 Interactuar con controles de distintos formularios.

6.2 Controles comunes de ventanas.

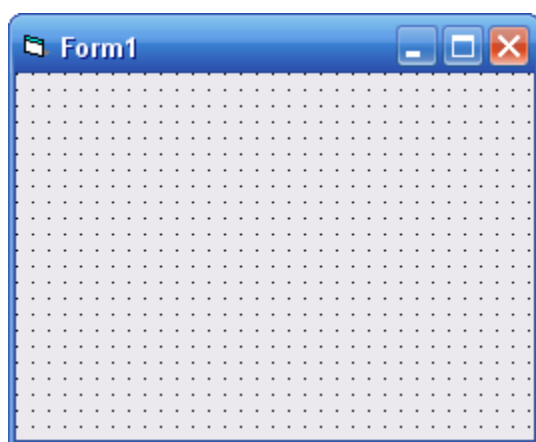
- 6.2.1 El control ImageList (Lista de Imágenes).
 - 6.2.1.1 Configurando el control ImageList.
 - 6.2.1.2 Extraer las imágenes de un ImageList.
 - 6.2.1.3 Ejercicio práctico.
- 6.2.2 Control TreeView.
 - 6.2.2.1 Configurar el control TreeView en tiempo de diseño.
 - 6.2.2.2 Propiedades más importantes del control TreeView.
 - 6.2.2.3 Cómo añadir objetos Node (Nodos).
 - 6.2.2.4 Cómo leer el texto del Nodo seleccionado.
 - 6.2.2.5 Eliminar uno o todos los elementos del control TreeView.
 - 6.2.2.6 Ejercicios práctico.
- 6.2.3 Control ListView.
 - 6.2.3.1 Configurar el control ListView en tiempo de diseño.
 - 6.2.3.2 Cómo añadir encabezados en modo de diseño.
 - 6.2.3.3 Cómo añadir encabezados en modo de ejecución.
 - 6.2.3.4 Cómo añadir objetos ListViewItem (Filas).
 - 6.2.3.5 Leer los datos de una o varias columnas.
 - 6.2.3.6 Ordenar el contenido de las columnas.
 - 6.2.3.7 Búsqueda de elementos.
 - 6.2.3.8 Ejercicios práctico.
- 6.2.4 Control ToolBar.
 - 6.2.4.1 Configurar el control ToolBar en tiempo de diseño.
 - 6.2.4.2 Cómo reaccionar ante las acciones del usuario.
 - 6.2.4.3 Crear menús dentro de los botones (ButtonMenu).
- 6.2.5 Control TabStrip.
 - 6.2.5.1 Configurando el control TabStrip en tiempo de diseño.
 - 6.2.5.2 Agregar objetos Tab.
 - 6.2.5.3 Cómo determinar la ficha pulsada por el usuario.
 - 6.2.5.4 Preparación de los contenedores.
- 6.2.6 Ejercicios propuestos.



6.1 LOS FORMULARIOS

Hasta ahora hemos creado aplicaciones haciendo uso del contenedor principal de una aplicación que son los *formularios*, pero no nos hemos detenido a analizar la forma en como están contruidos, las propiedades, los métodos y los objetos que estos poseen. En el presente capítulo estudiaremos trataremos detalladamente cada una de sus propiedades, eventos y métodos. También analizaremos un gran número de controles comunes que utilizaremos frecuentemente sobre los formularios, exceptuando los ya vistos en los capítulos anteriores de este libro.

- 6.1.1 Concepto de formulario



Un **formulario** es un contenedor donde se colocan todos los controles que componen una aplicación. En el paradigma de la programación orientada a objetos, un formulario esta compuesto por un **modulo de clase** y un **diseñador**. Los **diseñadores** son módulos integrados en el entorno de Visual Basic que permiten a los programadores diseñar visualmente ciertas características de los objetos que se instanciarán en tiempo de ejecución.

El diseñador de formulario le permite definir el aspecto de su formulario en tiempo de diseño colocando los controles hijos en su superficie y definiendo sus propiedades. Es obvio que estamos hablando de una parte del EID (Entorno Integrado de Desarrollo) de Visual Basic. Cuando se ejecuta la aplicación, el ejecutable de Visual Basic convierte estas piezas de información en un conjunto de llamadas a las funciones API de Windows que crearán la ventana principal y, posteriormente, todos sus controles hijos.

En conclusión, un formulario es considerado como un objeto especial compuesto por una interfaz de usuario y están compuestos por un gran conjunto de propiedades, métodos y eventos al igual que los objetos normales.

- 6.1.2 Propiedades de los formularios

Los formularios poseen las propiedades comunes de los controles ya vistos anteriormente, y podrá acceder a cada una de ellas de igual forma como lo hacia con un objeto normal. Antes de comenzar a definir las propiedades más importantes de un formulario abra un nuevo proyecto.

Propiedad	Descripción
Name	Al igual que en los controles vistos anteriormente, se utiliza para asignarle el nombre al <i>objeto</i> . Este nombre va a permitir identificar un formulario de los demás formularios que contenga una aplicación.
BackColor	Devuelve o establece el color de fondo del formulario. Podrá especificar un color de la paleta de colores disponible en modo de diseño, o bien, asignar un color desde código especificando una constante de color o un color expresado en forma hexadecimal. La propiedad BackColor en un formulario tiene el mismo efecto que en los controles ya estudiados en este libro.



Propiedad	Descripción
BorderStyle	Devuelve o establece el estilo de borde del formulario. Puede tomar los valores 0-None (sin borde), 1-FixedSingle(Simple fijo), 2-Sizable(Tamaño ajustable), 3-Fixed Dialog (Dialogo fijo), 4-Fixed ToolWindow (Ventana fija) y 5-Sizable ToolWindow (Ventana de tamaño ajustable). Para que tenga claro el efecto que causa cada uno de estos valores, asígnelo uno por uno en la propiedad BorderStyle del formulario a medida que se van definiendo cada uno de ellos: 0-None: Elimina los bordes del formulario, como consecuencia, no podrá modificar su tamaño y no verá la barra de título de la ventana. Este valor es muy utilizado en las aplicaciones para mostrar una pequeña ventana inicial con información acerca del programa. Por ejemplo, cuando inicia Microsoft Word aparece una ventana con el nombre del programa, el nombre de la compañía, la versión, y algunas imágenes decorativas, como el logo del programa o la compañía. Aunque no aparezcan los bordes de la ventana, la barra de título y los botones de control Minimizar, Maximizar y Cerrar, sigue siendo de todos modos un formulario. 1-FixedSingle: Define un borde simple en el formulario. Incluye la barra de título y el botón de Cerrar. No podrá modificar el tamaño del formulario. 2-Sizable: Es el valor por defecto de esta propiedad. Incluye la barra de título, el botón de Minimizar, el botón de Maximizar y el botón de Cerrar. Puede modificar el tamaño del formulario por los bordes. 3-Fixed Dialog: Similar al valor 1-FixedSingle. Puede incluir una barra de título, el botón de Cerrar, pero no los botones de Maximizar ni Minimizar. No permite modificar el tamaño del formulario. 4-Fixed ToolWindow: Ventana fija. Muestra una ventana a la que no le puede cambiar el tamaño con un botón Cerrar y el texto de la barra de título aparece con un tamaño de fuente reducido. 5-Sizable ToolWindow: Similar al anterior. Se diferencia del valor 4-Fixed ToolWindow en que permite modificar el tamaño del formulario.
Caption	Devuelve o establece el texto que aparece en la barra de título del formulario. NOTA: El valor asignado en la propiedad Caption de un formulario no podrá ser apreciado, si esta establecido el valor 0-None en la propiedad BorderStyle de dicho formulario.
ControlBox	Muestra o oculta los botones de control Minimizar, Maximizar y Cerrar. Puede tomar los valores True o False. Cuando es True (valor por defecto) se muestran los tres botones de control, cuando es False se ocultan. NOTA: Cuando el valor de la propiedad ControlBox es True, podrá mostrar los botones de control Minimizar, Maximizar y Cerrar siempre y cuando el valor establecido en la propiedad BorderStyle permita mostrar la barra de título de la ventana.
Icon	Establece un archivo de imagen con extensión (.ico) o (.cur) en la barra de título de la ventana. La primera representa los archivos de icono y la segunda los archivos de tipo cursores. Existen mucho de estos en la carpeta Window\Cursores. Esta imagen aparece a la izquierda del texto especificado en



la propiedad **Caption** del formulario. Puede observar en la barra de título de Visual Basic aparece el icono { EMBED PBrush }, esta imagen identifica el programa Visual Basic de los demás programas creados para Windows. Estos iconos están registrados y tienen derecho de autor. Muchos de estos iconos están disponibles en Internet de forma gratuita y no tendrá ningún problema en agregarlo a sus aplicaciones.

COMENTARIO: Es recomendable que usted mismo diseñe iconos propios para sus aplicaciones, esto le da un valor significativo a los programas. Existen cientos de programas en Internet que permite crearlos. Podrá dibujarlos fácilmente con una aplicación destinada para tal fin, de igual forma como usted diseña una simple imagen en **Paint**. Si no esta muy ligado al diseño gráfico o no quiere perder tiempo en esos detalles, puede contratar un diseñador gráfico para que diseñe su icono.

Ejercicio:

- Seleccione la propiedad **Icon** del formulario.
- Haga clic en el botón con los tres puntos { EMBED PBrush }. Aparecerá una ventana que le permitirá buscar el icono que desea poner en el programa:

{ EMBED PBrush }

- Haga clic sobre la caja combinada para desplegarla y, a continuación, seleccione el disco local (C:), o más bien su unidad principal de disco duro.
- Busque la carpeta **WINDOWS** y haga doble clic sobre esta. Cuando este dentro de la carpeta **WINDOWS**, busque la carpeta **Cursores** y haga doble clic sobre esta.
- Seleccione cualquiera de los iconos que parecen en la carpeta y haga clic en el botón **Abrir**.
- Ahora observe en la barra de titulo el icono que usted seleccionó.

KeyPreview

Devuelve o establece un valor que determina si los eventos de teclado de los formularios se invocan antes que los eventos de teclado de los controles. Los eventos de teclado son **KeyDown**, **KeyUp** y **KeyPress**. Esta propiedad puede tomar los valores **True** o **False** (Valor por defecto). Cuando esta propiedad esta establecida a **True** el código escrito en los eventos **KeyDown**, **KeyUp** y **KeyPress** del formulario se ejecutará primero que el código escrito en los eventos del teclado de los controles colocados sobre este. Cuando esta a **False** el código escrito en los eventos del teclado del formulario no ocurren.



Ejercicio:

- Busque la propiedad **KeyPress** y establezca el valor **True**.
- Haga doble clic sobre el formulario.
- Seleccione el evento **KeyDown** del formulario y escriba el siguiente bloque de código:

'Muestra un mensaje cuando se pulsa la tecla F1.

```
If KeyCode = vbKeyF1 Then
    MsgBox ("Usted presiono la tecla F1.")
End If
```

'Sale del programa si se pulsa la tecla S.

```
If KeyCode = vbKeyS Then
    End
End If
```

- Corra la aplicación.
- Pulse la tecla F1. Podrá observar que aparece un cuadro de mensaje diciendo que se ha pulsado la tecla F1. Cierre el cuadro de mensaje. Si pulsa la tecla S saldrá automáticamente de la aplicación.
- Detenga la aplicación.

MaxButton	Determina si el botón de Maximizar del formulario estará desactivado o activado. Puede tomar los valores True (valor por defecto) o False. COMENTARIO: Algunos programadores deshabilitan este botón para evitar que los usuarios maximicen alguna ventana de su aplicación.
MDIChild	Determina si un formulario es mostrado como un formulario hijo. Los formularios hijos se utilizan en aplicaciones que requieren de una <i>interfaz de múltiples documentos</i> (MDI). Este tipo de interfaz no es tratada en este libro, debido a que la mayoría de las aplicaciones son diseñadas utilizando una <i>interfaz de documento único</i> (SDI). Para mayor información acerca de estos tipos de interfaces consulte la ayuda digital MSDN Library de Visual Basic.
MinButton	Determina si el botón de Minimizar del formulario estará desactivado o activado. Puede tomar los valores True (valor por defecto) o False.
Mouselcon	Algunas aplicaciones establecen un icono personalizado para el puntero del Mouse que se mostrará sobre toda la aplicación. Mediante la propiedad Mouselcon podrá seleccionar un archivo de extensión (.ico) o (.cur) que se mostrará como el puntero del Mouse sobre todo el formulario y los controles colocados en este. Para que el icono especificado en esta propiedad sea mostrado en la aplicación, debe especificar el valor 99-Custom en la propiedad MousePointer del formulario.

Ejercicio:

- Seleccione la propiedad **MousePointer** del formulario y, a continuación, establezca el valor **99-Custom**.



- Seleccione la propiedad **Mouselcon** y, a continuación, haga doble clic sobre los tres puntos (...) suspensivos.
- Busque la carpeta WINDOWS en su disco principal. Dentro de la carpeta WINDOWS localice la carpeta Cursores.
- Seleccione el icono llamado **harrow** { EMBED PBrush }, y haga clic sobre el botón Abrir.
- Corra la aplicación.
- Podrá observar que el puntero predeterminado de su sistema no se muestra sobre el formulario, sino, el icono seleccionado en la propiedad **Mouselcon**.
- Detenga la aplicación.

Moveable	Determina si el formulario se podrá mover sobre la pantalla. Puede tomar los valores True (se puede mover) o False (no se puede mover).
Picture	Especifica una imagen que se muestra en el fondo del formulario. Este archivo de imagen puede ser del tipo BMP, JPG, GIF, ICO, CUR, etc. NOTA: El archivo de imagen se muestra del mismo tamaño en que esta se encuentra diseñada. Si la imagen que tiene colocada sobre el formulario no lo cubre a totalidad, tendrá que utilizar un programa de edición grafica como el Paint para alterar el tamaño original de la imagen y adaptarlo al tamaño deseado. COMENTARIO: Si desea colocar una imagen de fondo sobre todo el formulario, recomiendo utilizar el control Image. Esta permite alterar el tamaño original de la imagen al tamaño deseado. Tendrá que tener en cuenta que el valor True este establecido en la propiedad Stretch del control Image.

Ejercicio:

- Seleccione la propiedad Picture del formulario y, a continuación, haga doble clic sobre los tres puntos (...) suspensivos.
- Busque la carpeta **IMAGENES** que se distribuye con este libro. Dentro de esta carpeta se encuentra una carpeta llamada **FONDOS**. Entre a esta carpeta y seleccione la imagen llamada **fondo15**.
- Haga clic sobre el botón Abrir.
- Corra la aplicación para que pueda ver claramente la imagen, es decir, si la maya de puntos.
- Observe que la imagen seleccionada se establece como fondo sobre todo el formulario. **NOTA:** Si modifica el tamaño del formulario podrá ver la imagen completamente.
- Detenga la aplicación.

ShowInTaskbar	Determina si la aplicación se muestra en la barra de tareas de Windows cuando esta se esta ejecutando o cuando es minimizada. Puede tomar los valores True (se muestra en la barra de tarea) o False (No se muestra en la barra de tareas). El valor por defecto de esta propiedad es True . Por eso, cuando usted corre la aplicación se muestra el nombre del programa y el icono seleccionado en la propiedad Icon del formulario sobre la barra de tarea de Windows.
----------------------	--



StartPosition	Determina la posición inicial del formulario cuando este es mostrado en la pantalla. Puede tomar los siguientes valores: ▪ 0-Manual: Al establecer este valor en la propiedad StartPosition la posición del formulario en la pantalla dependerá de los valores establecidos en las propiedades Top y Left. ▪ 1-CenterOwner: El formulario aparece en el centro de la ventana a la que pertenece. ▪ 2-CenterScreen: El formulario se muestra en el centro de la pantalla. ▪ 3-Windows Default: Este es el valor por defecto. El formulario se muestra en la esquina superior izquierda de la pantalla. Establezca cada uno de estos valores en la propiedad StartPosition del formulario. A medida que valla estableciendo los valores corra la aplicación. De esta forma podrá observar las diferencias entre cada uno de ellos. NOTA: El valor 1-CenterOwner tendrá el mismo efecto que el valor 2-CenterScreen si el formulario no es mostrado de forma modal. El valor 1-CenterOwner tendrá efecto, solo si el formulario mostrado es cargado sobre otro formulario en forma modal. Las distintas formas modales son tratadas en este capítulo más adelante.
WindowState	Indica el estado del formulario cuando este es mostrado en la pantalla. Puede tomar los siguientes valores: ▪ 0-Normal: El formulario aparece en la pantalla de forma normal. ▪ 1-Minimized: El formulario aparece minimizado en la barra de tarea de Windows. ▪ 2-Maximized: El formulario aparece maximizado, es decir, cubre todo el tamaño de la pantalla.

- 6.1.3 Eventos de los formularios

Un formulario posee los eventos estándar de los controles (**Click**, **DbClick**, **MouseDown**, **MouseMove**, **MouseUp**, **KeyUp**, **KeyDown**, **KeyPress**, **GotFocus**, **LostFocus**). Además de estos, los formularios poseen eventos que son muy propios de ellos que corresponde al ciclo de vida de un formulario.

Initialize	Este es el primer evento del ciclo de vida de cualquier formulario. Este evento ocurre tan pronto como haga referencia desde código al nombre del formulario, incluso antes de que Visual Basic cree la ventana y los controles colocados en su superficie. Podrá escribir código en este evento para iniciar correctamente las variables del formulario.
Load	Este es el segundo evento que ocurre antes de que un formulario se muestre en la pantalla. Este evento ocurre cuando Visual Basic crea el formulario y los controles hijos, pero aun no son visibles. Podrá hacer referencia a las propiedades de los objetos colocados sobre el formulario desde este evento, incluso, la mayoría de los programadores inician sus variables desde este evento. Algunos métodos de los controles no pueden ser ejecutados para objetos que son invisibles. Por ejemplo, el método SetFocus no lo podrá utilizar desde el evento Load.



Ejercicio:

En el siguiente ejercicio mostramos como programar el evento Load para hacer referencias a los controles colocados sobre un formulario.

- Abra un nuevo proyecto.
- Coloque un control **ListBox** sobre el formulario.
- Dentro del evento Load del formulario escriba:

'Agrega una lista de países al ListBox.

```
List1.AddItem "República Dominicana"
List1.AddItem "Puerto Rico"
List1.AddItem "Perú"
List1.AddItem "Estado Unidos"
List1.AddItem "Argentina"
List1.AddItem "España"
List1.AddItem "Venezuela"
```

- Corra la aplicación.
- Podrá observar, que la lista de países se agrega al **ListBox** antes de que la aplicación sea mostrada en la pantalla.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio1-6** para el formulario y **Ejercicio1-6** para el proyecto.

Resize

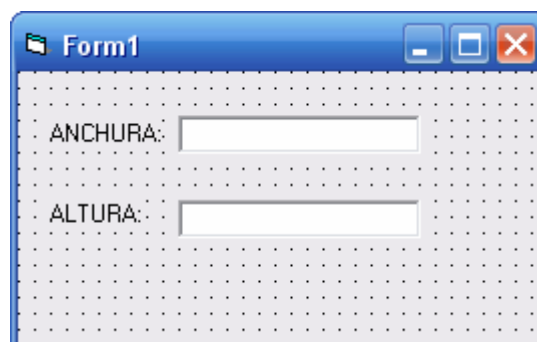
Ocurre un instante antes de mostrarse el formulario. Cuando ocurre este evento el formulario ya esta listo para ser mostrado y los controles sobre su superficie. Podrá utilizar este evento para posicionar los controles contenidos en el formulario con el fin de que llenen el espacio disponible con una buena disposición.

El evento **Resize** también ocurre cuando modifica manualmente o mediante código el tamaño del formulario.

Ejercicio:

El siguiente ejercicio muestra como utilizar el evento **Resize** para mostrar el tamaño de un formulario cuando el usuario modifique su tamaño.

- Abra un nuevo proyecto.
- Coloque dos etiquetas y dos cajas de texto, tal y como se muestra en la siguiente figura:





- Haga doble clic sobre el formulario y de la lista de eventos seleccione el evento **Resize**.
- Dentro del evento **Resize** escriba:

'Coloca la anchura del formulario en la primera caja de texto.
Text1.Text = Form1.Width & " Píxeles."
'Coloca la altura del formulario en la segunda caja de texto.
Text2.Text = Form1.Height & " Píxeles."
- Corra la aplicación.
- Observe que la altura y la anchura del formulario se escriben en las cajas de texto. Esto es porque antes de mostrarse el formulario en la pantalla ocurre el evento **Resize**.
- Modifique el tamaño del formulario por los bordes izquierdo e inferior. Podrá observar, que a medida que módica el tamaño del formulario la caja de texto muestra el tamaño actual y la altura actual del formulario. Esto es, porque cada vez que modificamos el tamaño de un formulario ocurre un evento **Resize**.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio2-6** para el formulario y **Ejercicio2-6** para el proyecto.

Activate	Este evento ocurre inmediatamente termina el evento Resize . Este evento ocurre también cuando el formulario recibe el enfoque. El evento Activate es útil cuando necesite actualizar el contenido del formulario con los datos que se hayan podido modificar en otro formulario.
Paint	Ocurre cuando el formulario se esta dibujando en la pantalla o cuando el formulario una parte del mismo es cubierta por otra ventana. También ocurre cuando otra ventana o formulario se mueve sobre él.
Deactivate	Ocurre cuando el formulario pierde el enfoque, es decir, cuando se muestra otro formulario sobre el formulario inicial. Podrá utilizar este evento para identificar si un formulario esta perdiendo el enfoque.
QueryUnload	Este evento ocurre cuando el formulario se esta cerrando. Este evento posee un <i>argumento</i> llamado UnloadMode . Este <i>argumento</i> permite identificar la fuente por la que se esta cerrando el formulario.

Ejercicio:

El siguiente ejercicio muestra una aplicación que informa al usuario la causa por la que se esta cerrando la ventana.

- Abra un nuevo proyecto.
- Haga doble clic sobre el formulario y seleccione el evento **QueryUnload**.
- Dentro de este evento escriba:

```

If UnloadMode = vbFormControlMenu Then
MsgBox ("Usted esta cerrando el formulario por el botón de Cerrar de la ventana.")
End If

If UnloadMode = vbFormCode Then
MsgBox ("Usted esta cerrando la ventana utilizando código de Visual Basic.")
End If
            
```



```

If UnloadMode = vbAppWindows Then
MsgBox ("La sección de Windows se esta cerrando y a causado que se cierre esta ventana.")
End If

If UnloadMode = vbAppTaskManager Then
MsgBox ("El administrador de tareas esta cerrando la ventana.")
End If

If UnloadMode = vbFormMDIForm Then
MsgBox ("El formulario padre MDI esta cerrando la ventana.")
End If

If UnloadMode = vbFormOwner Then
MsgBox ("El formulario propietario se esta cerrando.")
End If
    
```

- Agregue un botón de comando sobre el formulario. Dentro del evento **Click** del botón de comando escriba:

```

Unload Form1      'Cierra el formulario.
    
```

- Corra la aplicación.
- Haga clic sobre el botón de comando. Podrá observar un mensaje diciendo que el formulario se esta cerrando desde código. Esto es, porque el argumento **UnloadMode** devuelve el valor **vbFormCode** cuando el formulario se esta cerrando desde código.
- Corra nuevamente la aplicación. Cierra la aplicación desde el botón **Cerrar** de la ventana. Podrá observar ahora otro mensaje diciendo que la ventana se esta cerrando desde el botón **Cerrar** de la ventana. Esto es, porque el argumento **UnloadMode** devuelve el valor **vbFormControlMenu** cuando el usuario cierra la ventana desde el botón de **Cerrar**.

Sí observa el código escrito anteriormente, notará que existen muchas causas por las cuales un formulario o ventana esta siendo cerrada. Cada situación es leída desde el argumento **UnloadMode** utilizando las estructuras de control selectiva, tal y como vimos en el código escrito en el evento **QueryUnload** del formulario.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio3-6** para el formulario y **Ejercicio3-6** para el proyecto.

Unload	Este evento ocurre antes de que Visual Basic haga los últimos ajustes para cerrar el formulario. Es la última oportunidad que el programador tiene para cancelar que se cierre el formulario. Este evento posee un argumento llamado Cancel que permite evitar que se cierre el formulario, o bien, notificar al formulario que se va a cerrar el formulario. La diferencia que existe entre el evento QueryUnload y el evento Cancel, es que el primero solo permite identificar la fuente por la que se cierra el formulario. Sin embargo, el argumento Cancel aunque no permite saber la fuente por la que se esta cerrando el formulario, permite cancelar el cierre del formulario.
Ejercicio: ▪ Abra un nuevo proyecto. ▪ Haga doble clic sobre el formulario y, a continuación, seleccione el evento Unload.	



- Dentro del evento **Unload** escriba:

```
'Almacena la respuesta del usuario.
respuesta = MsgBox("¿Está seguro que desea salir?", vbYesNo)

'Si el usuario responde que esta seguro se concluye cerrando la aplicación.
If respuesta = vbYes Then
    Cancel = False 'Confirma el cierre de la aplicación.
Else
    'Si el usuario responde que No, se cancela el cierre de la aplicación.
    Cancel = True 'Cancela el cierre de la aplicación.
End If
```

- Corra la aplicación.
- Cierre la aplicación desde el botón cerrar de la ventana. Aparecerá un cuadro de mensaje solicitando la confirmación del cierre de la aplicación. Si usted responde "Sí" la aplicación se cerrará, y si contesta que "No" el cierre abortará.

Cuando asignamos el valor **False** en el argumento **Cancel** del evento **Unload**, el formulario continua con el proceso de cierre, pero si asigna el valor **True** cancelará el cierre del formulario.

- 6.1.4 Trabajando con múltiples formularios

La mayoría de las aplicaciones integran múltiples formularios dentro de una misma interfaz de usuario. Cuando en una aplicación ejecutamos una opción de un menú, comúnmente se muestra una nueva ventana dentro de la misma aplicación. Esta es la única opción que tenemos para mostrar al usuario el resto de toda la aplicación, o más bien, todas las opciones disponibles en esta.

Para agregar un nuevo formulario en una aplicación hacemos clic en la opción **Project (Proyecto)** de la barra de menús de Visual Basic, y luego, hacemos clic sobre la opción **Add Form (Agregar Formulario)**. El formulario se agrega en el *explorador de proyecto* con el nombre de **Form** seguido de un valor que representa el número de ese formulario dentro de la aplicación.

Después de haber agregado un nuevo formulario en la aplicación, el siguiente paso es mostrarlo utilizando el método **Show** de los formularios. Para mostrar el formulario deberá escribir el nombre de este, seguido del método **Show**, la forma en que desea mostrarlo y el formulario propietario.

Formato:

NombreFormulario.Show [Modo], [FormularioPropietario]

Donde:

<i>NombreFormulario</i>	Es un identificador valido que indica el nombre del formulario dentro de la aplicación.
<i>Modo</i>	Opcional. Un entero que determina si el formulario es modal o no modal. Si <i>estilo</i> es 0, el formulario es no modal ; si <i>estilo</i> es 1, el formulario es modal . Cuando un formulario es no modal podrá trabajar con el <i>formulario propietario</i> y el formulario que se muestra al mismo tiempo. Si es modal , tendrá que cerrar el formulario modal para luego trabajar con el <i>formulario propietario</i> .



Para que tenga idea de lo que es un formulario **no modal**, ejecute Microsoft Word y, a continuación, haga clic en el menú **Edición** y luego en **Buscar...**

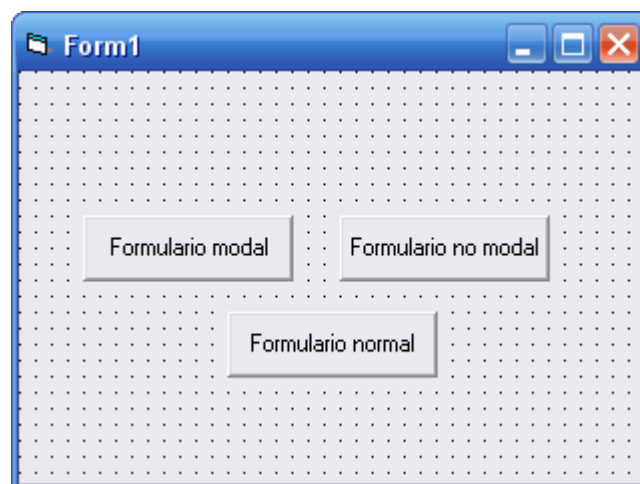
Haga clic sobre la página y escriba algo. Podrá observar que la ventana de búsqueda se queda abierta, permitiendo trabajar con la ventana principal (*ventana propietaria*) y la ventana de búsqueda.

Por otro lado, un formulario **modal** no permite la interacción con la *ventana propietaria* y la ventana **modal**. Para ver esto, haga clic en el menú **Formato** de Microsoft Word y ejecute la opción **Fuente**. Ahora, trate de escribir algo. Podrá observar, que la *ventana propietaria* se inhabilita hasta que se cierre la ventana **modal**.

FormularioPropietario: Es un identificador valido que indica el nombre del *formulario propietario*, es decir, el nombre de la ventana sobre la que se muestra el formulario.

Ejercicio:

- Abra un nuevo proyecto.
- Inserte tres botones de comando, tal y como se muestra en la siguiente figura:



- En la propiedad **Caption** del primer botón escriba *Formulario modal*, en la propiedad **Caption** del segundo botón de comando escriba *Formulario no modal* y, en la propiedad **Caption** del tercer botón de comando escriba *Formulario normal*.

- Agregue un nuevo formulario desde el menú **Project/Add Form**.

- Dentro del evento **Click** del primer botón de comando escriba:

```
'Muestra en forma modal en segundo formulario.  
Form2.Show 1, Form1 'Es lo mismo escribir: Form2.Show 1, Me
```

- Dentro del evento **Click** del segundo botón de comando escriba:

```
'Muestra en forma no modal en segundo formulario.  
Form2.Show 0, Form1 'Es lo mismo escribir: Form2.Show 0, Me
```



- Dentro del evento **Click** del tercer botón de comando escriba:

```
'Muestra de forma normal en segundo formulario.  
Form2.Show
```

- Corra la aplicación.
- Haga clic sobre el primer botón de comando. Ahora, trate de hacer clic sobre el primer formulario. Podrá observar que no puede interactuar con el formulario propietario hasta que cierre el formulario modal.
- Cierre el formulario modal.
- Haga clic sobre el segundo botón de comando. Ahora, haga clic sobre el primer formulario. Podrá observar que puede interactuar con el formulario propietario aun estando el formulario modal sobre este.
- Cierre el formulario modal.
- Haga clic sobre el tercer botón de comando. Ahora, haga clic sobre el primer formulario y luego sobre el segundo. Podrá observar, que hará tiene la posibilidad de decidir con el formulario que desea interactuar. En este caso ambos formularios son propietarios.
- Detenga la aplicación y guarde el ejercicio con los nombres **Form1Ejercicio4-6** para el primer formulario, **Form2Ejercicio4-6** para el segundo formulario y **Ejercicio4-6** para el proyecto.

- 6.1.4.1 Descargar un formulario

Descargar un formulario consiste en descargar de la memoria un formulario y todas las variables cargados en este. Para cerrar o descargar un formulario utilice la sentencia **Unload** seguido del nombre del formulario.

Ejemplo:

```
Unload Form1      'Descarga un formulario llamado Form1.  
Unload FormCliente 'Descarga un formulario llamado FormCliente.  
Unload Me         'Descarga el formulario actual.
```

Si no desea descargar el formulario y solo quiere hacerlo invisible ante la vista del usuario, ya sea para utilizar los valores asignados en algunos controles de este o para otro fin, entonces la mejor opción es establecer el valor **False** en la propiedad **Visible** del formulario. De esta manera el formulario estará invisible, pero podrá acceder a los controles colocados sobre él.

Ejemplo:

```
Form1.Visible = False      'Oculta un formulario llamado Form1.  
FormCliente.Visible = False 'Oculta un formulario llamado FormCliente.  
Me.Visible = False         'Oculta el formulario actual.
```

- 6.1.4.2 Cargar un formulario

En muchas ocasiones queremos ejecutar un formulario sin que el usuario pueda apreciar su apariencia física, es decir, solo cargarlo pero no mostrarlo en la pantalla. Para esto utilizamos la sentencia **Load** seguido del formulario que desea cargar. Ejemplo: **Load FormCliente**.

Cuando cargamos un formulario podemos utilizar todos sus controles desde otro formulario y modificar cualquiera de sus propiedades.



- 6.1.4.3 Interactuar con controles de distintos formularios

Una de las características más potentes de Visual Basic es que permite interactuar desde un formulario con los controles colocados sobre otro formulario. Para tal fin, el usuario deberá especificar el nombre del formulario que contiene el control, seguido del nombre del control y la propiedad, evento o método que ejecutar.

Ejemplo:

'Asigna la cadena Visual Basic en una caja de texto llamada Text1 del formulario Form1.

Form1.Text1.Text = "Visual Basic"

'Borra el contenido de una caja de texto llamada txtImporte del formulario FormCliente.

FormCliente.txtImporte.Text = ""

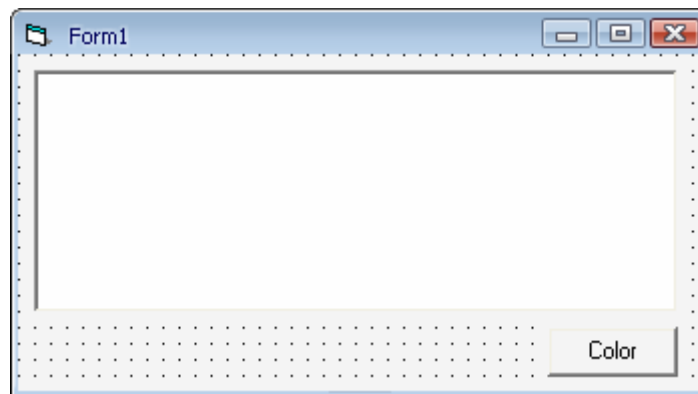
'Oculta un cuadro de imagen llamado Picture1 del formulario FormCliente.

FormCliente.Picture1.Visible = False

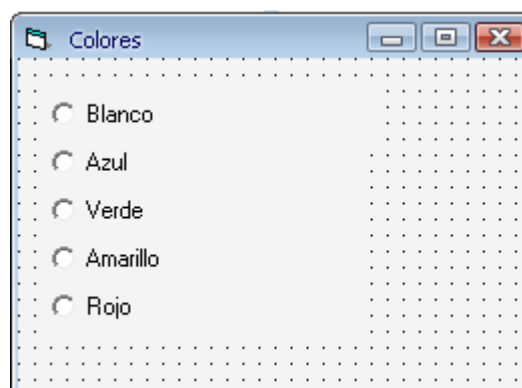
Ejercicio:

El siguiente ejercicio muestra como interactuar con los controles de distintos formulario con una aplicación que permite cambiar el color de una caja de texto desde otro formulario.

- Abra un nuevo proyecto.
- Inserte una caja de texto sobre el formulario y debajo un botón de comando:



- Agregue en nuevo formulario. Sobre este formulario coloca cinco botones de opción uno debajo del otro, tal y como se muestra en la siguiente figura:





- Establezca los siguientes valores los siguientes valores en las propiedades de los controles colocados en el primero formulario:

Control	Propiedad	Valor
Form1	Name	formPrincipal
Text1	Name Text	txtContenido
Command1	Name Caption	cmdColores Color

- Establezca los siguientes valores los siguientes valores en las propiedades de los controles colocados en el segundo formulario:

Control	Propiedad	Valor
Form2	Name	formColores
Option1	Name Caption	optBlanco Blanco
Option2	Name Caption	optAzul Azul
Option3	Name Caption	optVerde Verde
Option4	Name Caption	optAmarillo Amarillo
Option5	Name Caption	optRojo Rojo

- Dentro del evento **Click** del botón de comando escriba:

```
formColores.Show 1, Me 'Muestra el formulario de los colores en forma modal.
```

- Dentro del evento **Click** del primer OptionButton escriba:

```
'Asigna el color blanco al fondo de la caja contenido del formulario principal.  
formPrincipal.txtContenido.BackColor = vbWhite
```

- Dentro del evento **Click** del segundo OptionButton escriba:

```
'Asigna el color azul al fondo de la caja contenido del formulario principal.  
formPrincipal.txtContenido.BackColor = vbBlue
```

- Dentro del evento **Click** del tercer OptionButton escriba:

```
'Asigna el color verde al fondo de la caja contenido del formulario principal.  
formPrincipal.txtContenido.BackColor = vbGreen
```

- Dentro del evento **Click** del cuarto OptionButton escriba:



'Asigna el color amarillo al fondo de la caja contenido del formulario principal.
formPrincipal.txtContenido.BackColor = vbYellow

- Dentro del evento **Click** del quinto OptionButton escriba:

'Asigna el color rojo al fondo de la caja contenido del formulario principal.
formPrincipal.txtContenido.BackColor = vbRed

- Corra la aplicación.
- Haga clic en el botón *Color*. Aparecerá la ventana de colores.
- Seleccione cada uno de los colores y podrá observar como estos se le aplican a la caja de texto que se encuentra en el formulario principal.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio5-6** para el formulario, y **Ejercicio5-6** para el proyecto.

Explicación:

Esta aplicación consta de dos formularios uno llamado *formPrincipal* y el otro llamado *formColores*. En el primer formulario tenemos una caja de texto y un botón de comando que permite mostrar el segundo formulario. El segundo formulario esta compuesto por cinco botones de opción que son utilizados para cambiar el color de fondo de la caja de texto del primer formulario.

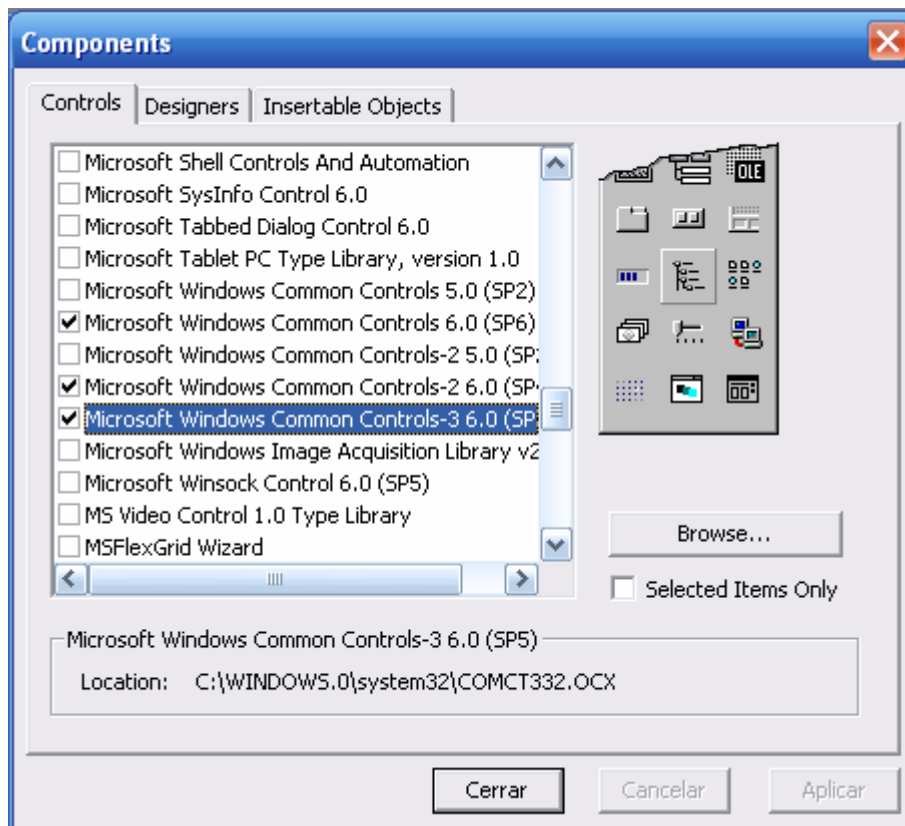
El objetivo principal del ejercicio anterior, es mostrar la forma en que el programador puede acceder a un objeto colocado en otro formulario. Si observa dentro del código escrito en el evento **Click** de cada botón de opción, podrá observar primero el nombre del formulario que contiene el objeto, seguido del nombre del objeto y la propiedad que será afectada. De esta manera, el usuario podrá acceder a cualquier control de otro formulario.

6.2 CONTROLES COMUNES DE VENTANAS

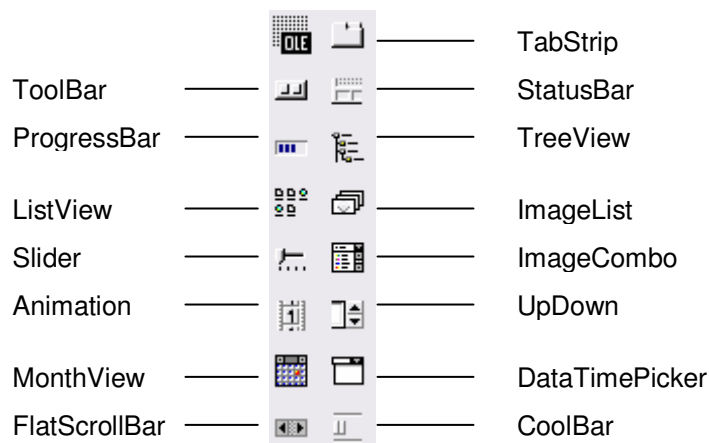
Además de los controles vistos en el **Capítulo 3**, Visual Basic proporciona un gran conjunto de controles que comúnmente vemos en la mayoría de las aplicaciones de Windows. Estos controles son bastante utilizados por los usuarios debido a estos se sienten familiarizados con ellos, además, estos controles contribuyen a crear el aspecto típico de Windows más que cualquier otro grupo de controles. Este conjunto de controles comunes esta compuesto por los controles TabString, ToolBar, StatusBar, ProgressBar, TreeView, ListView, ImageList, Slider, ImageCombo, Animation, UpDown, MonthView, DateTimePicker, FlatScrollBar y CoolBar.

Estos controles no están disponibles desde un principio en la *barra de controles* de Visual Basic. Estos se encuentran en los archivos MsComCtl2.ocx y ComCtl232.ocx localizados en la carpeta de Windows\System32. Para agregar estos controles y poder comenzar a utilizarlo solo tendrá que hacer clic en el menú **Project** y ejecutar la opción **Components...**, o bien, presionar la combinación de tecla Ctrl+T. En la ventana Componentes localice active los siguientes controles:

- ☐ Microsoft Windows Common Controls 6.0 (VB6).
- ☐ Microsoft Windows Common Controls-2 6.0 (VB4).
- ☐ Microsoft Windows Common Controls-3 6.0 (VB5).



Nuestra barra de controles tendrá en la parte inferior los nuevos controles agregados:



- 6.2.1 El control ImageList (Lista de Imágenes)

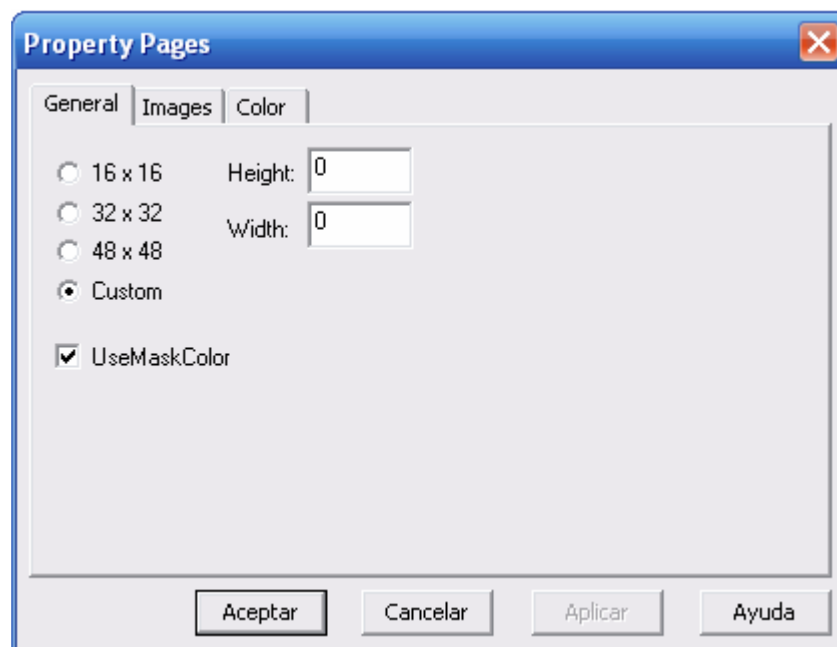
Este es el primer control que debe conocer antes de cualquier otro control, debido a que la mayoría de los demás controles lo utilizan como contenedor de imágenes. Este control permite almacenar las imágenes e iconos empleados por otros controles, tales como TreeView, ListView, TabStrip e ImageCombo. El control **ImageList** es invisible en tiempo de ejecución y para mostrar las imágenes que contiene deberá introducirlo en un formulario, un PictureBox o un control Image o asociarlo con otro control.



Utilizar este control como un almacén para las imágenes que utilizarán otros controles tiene muchas ventajas. Si no utiliza este control tendría que cargar las imágenes desde su disco duro utilizando una función **LoadPicture** que ralentizaría la ejecución y aumentaría el número de archivos que tendría que distribuir con su programa. Además, si el usuario borra uno de estos archivos de imagen accidentalmente, entonces, su aplicación daría un error en tipo de ejecución. Es mucho más sencillo y más eficiente utilizar un control **ImageList** y, posteriormente, hacer referencia a los mismos desde otros controles o desde el código fuente.

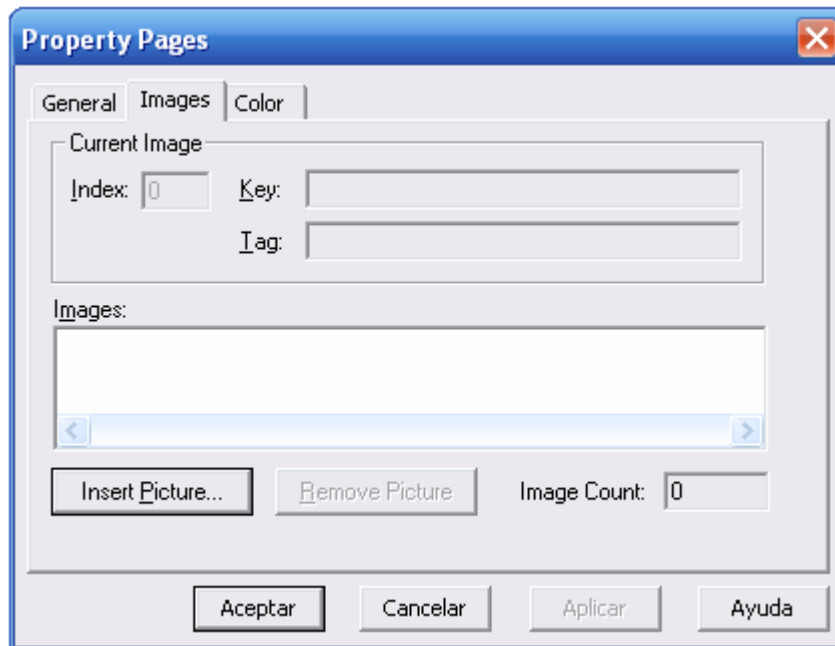
- 6.2.1.1 Configurando el control ImageList

El primer paso después de haber insertado el control **ImageList** en el formulario es configurarlo. Para esto deberá acceder a la ventana propiedades del mismo desde la ventana propiedades **Custom (Personalizado)**, o bien, haciendo clic derecho sobre el control y seleccionando la opción **Properties (Propiedades)**. Aparecerá la ventana de propiedades:



En la ficha general debemos seleccionar el tamaño que deberán tener las imágenes que cargaremos. Las medidas disponibles son 16 x 16, 32 x 32, 48 x 48, y la opción **Custom (Personalizado)** que le permitirá agregar manualmente el tamaño en píxeles de la imagen. Para barra de herramientas utilizamos comúnmente los tamaños 16 x 16 o 32 x 32. Cuando seleccione el tamaño de las imágenes no podrá cambiarlo una vez ha agregado imágenes a la lista de imágenes. Deberá tener claro el tamaño que tendrán las imágenes que cargaremos antes de agregarlas. Como dije anteriormente si va a cargar las imágenes en barra de herramientas, en menús o en botones de comando, recomiendo que las imágenes no excedan los tamaños 16 x 16 o 32 x 32.

Una vez configurado el tamaño de las imágenes, el siguiente paso es agregar las imágenes que cargaremos en los controles. Para esto, debemos hacer clic en la ficha **Imágenes** de la ventana propiedades. En esta ventana veremos tres cajas de texto, una lista de imágenes y dos botones de comando. En la primera caja de texto **Index (Índice)** se agrega automáticamente un número en orden ascendente que será útil para cargar la imagen desde código. Este valor usted no tiene que modificarlo aunque puede hacerlo, solo debe saberlo cuando vaya a cargar una imagen. Para saber este valor una vez agregadas las imágenes solo debe seleccionar la imagen de la lista de imágenes y observar la caja de texto. En la segunda caja de texto **Key (Clave)** escribirá una cadena de texto para identificar cada una de las imágenes que ha agregado.



No se preocupe por escribir en la tercera caja de texto, esta no es a menudo utilizada. Para agregar las imágenes que utilizará, debe hacer clic en el botón **(Insert Picture...)** (*Insertar Imagen*), y luego buscar la imagen que desea cargar en su disco duro. Si desea eliminar una imagen, solo debe seleccionarla y hacer clic en el botón **Remove Picture** (*Eliminar Imagen*).

Podrá agregar imágenes en modo de ejecución utilizando el método **Add** del objeto **ImageList**. Este método tiene el siguiente formato:

`Add [Index], [Clave], [Image]`

El parámetro *Index* lo puede omitir para agregar la imagen al final de la colección. Este es un ejemplo de cómo agregar una imagen desde código a un control **ImageList**.

`Dim Imagen As ListImage`

`Set Imagen = ImageList1.ListImages.Add(, "Guardar", LoadPicture("c:\guardar.bmp"))`

Para eliminarla en tiempo de ejecución solo deberá utilizar el método **Remove** seguido del índice (Index) o la clave (Key):

'Eliminar el que agregamos anteriormente.

`ImageList1.ListImages.Remove Guardar`

Si desea eliminarlas todas en una única operación puede utilizar el método **Clear**:

`ImageList1.ListImages.Clear`

Podrá guardar en un **ImageList** la imagen cargada en un **PictureBox**:

`ImageList1.ListImages.Add , , Picture1.Picture`



- 6.2.1.2 Extraer las imágenes de un ImageList

Para extraer las imágenes y colocarlas en otros controles como un PictureBox o Image deberá indicar el número del índice (*Index*) o la clave (*Key*) de dicha imagen:

Carga en el PictureBox la imagen que tiene el valor 1 como Index.

```
Pictuer1.Picture = ImageList1.ListImages(1).Picture
```

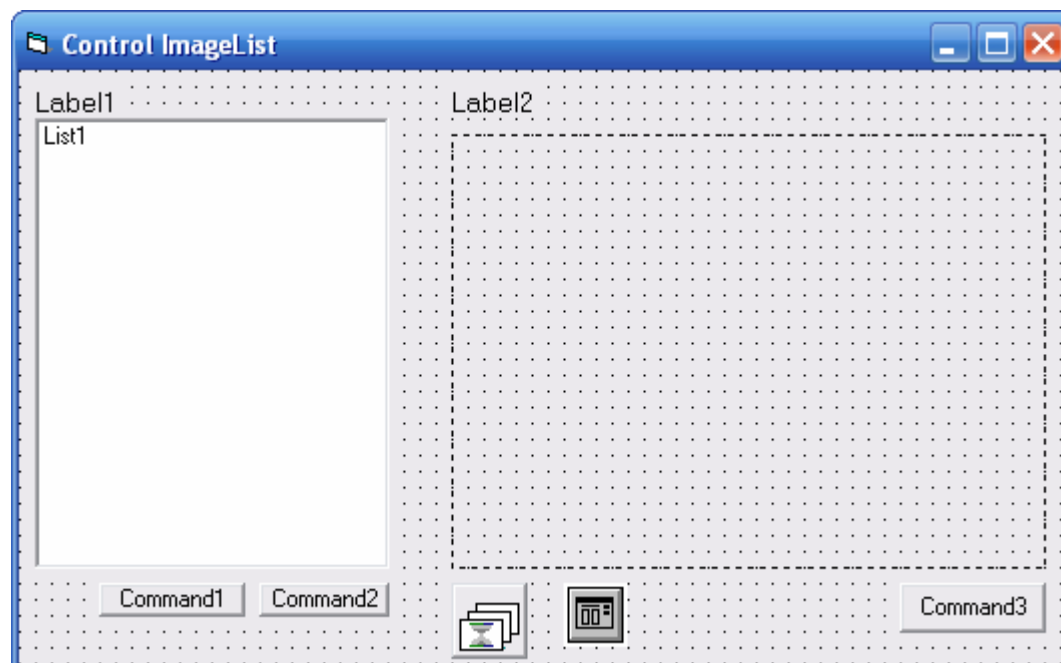
Este otro código haría lo mismo si la clave de la imagen es "Guardar":

```
Pictuer1.Picture = ImageList1.ListImages("Guardar").Picture
```

- 6.2.1.3 Ejercicio práctico

En el siguiente ejercicio se muestra una aplicación que permite cargar en un control **ImageList** las imágenes que usted tenga en su disco duro. Luego puede posteriormente visualizarlas una por una seleccionándolas de una lista. También podrá eliminar las imágenes cargadas en el control **ImageList**.

- Abra un nuevo proyecto.
- Precione la combinación de tecla Ctrl+T. En la ventana componente active los controles *Microsoft Common Dialog Control 6.0 (SP3)* y *Microsoft Windows Common Controls 6.0 (SP6)*.
- Inserte en el formulario un control **CommonDialog** de la barra de controles.
- Inserte en el formulario un control **ImageList** .
- Inserte dos etiquetas, una ListBox, un control Image y dos botones de comando:



- En la propiedad **Caption** de la primera etiqueta escriba "*Lista de imágenes:*". En la propiedad **Caption** de la segunda etiqueta escriba "*Imagen mostrada:*".



- En la propiedad **BorderStyle** del control **Image** seleccione *1 – Fixed Single* y en la propiedad **Stretch** el valor *True*.
- En la propiedad **Caption** del primer botón de comando escriba *&Agregar*, en el segundo botón de comando *&Borrar* y en el tercer botón de comando *&Salir*.
- Haga doble clic en cualquier parte del formulario y en el evento **Load** escriba:

'Configuramos el tamaño de las imágenes que será cargadas.

```
ImageList1.ImageHeight = 100
ImageList1.ImageWidth = 100
```

- En el evento **Click** del control **ListBox** escriba:

On Error Resume Next 'En caso de cualquier error.

'Mostramos en el PictureBox la imagen que seleccionamos en el ListBox.

```
Image1.Picture = ImageList1.ListImages(List1.Text).Picture
```

- En el evento **Click** del botón *Agregar*:

'Establecer **CancelError** a **True**.

```
CommonDialog1.CancelError = True
```

On Error GoTo NoSeleccion

'Establecer los indicadores.

```
CommonDialog1.Flags = cdIOFNHideReadOnly
```

'Establecer los filtros.

```
CommonDialog1.Filter = "Archivos de imagenes (*.JPG, *.BMP)|*.jpg|*.bmp|"
```

'Especificar el filtro predeterminado.

```
CommonDialog1.FilterIndex = 2
```

'Presentar el cuadro de diálogo *Abrir*.

```
CommonDialog1.ShowOpen
```

'Agrega al listbox el nombre del archivo seleccionado.

```
List1.AddItem CommonDialog1.FileName
```

'Lo agregamos al control **ImageList**.

```
ImageList1.ListImages.Add , CommonDialog1.FileName, _
    LoadPicture(CommonDialog1.FileName)
```

Exit Sub

NoSeleccion:

'El usuario ha hecho clic en el botón *Cancelar*.

Exit Sub

- En el evento **Click** del botón *Eliminar*:

'Verificamos que halla algún elemento seleccionado.

If List1.Text = "" Then

```
MsgBox ("Debe seleccionar la imagen que desea eliminar.")
```

Else

```
ImageList1.ListImages.Remove (List1.Text) 'Eliminamos la imagen del control ImageList.
```

'Eliminamos la imagen de la lista.

```
List1.RemoveItem (List1.ListIndex)
```

End If



- En el evento **Click** del botón *Salir* escriba:

```
'Salimos de la aplicación.  
End
```

- Corra la aplicación.

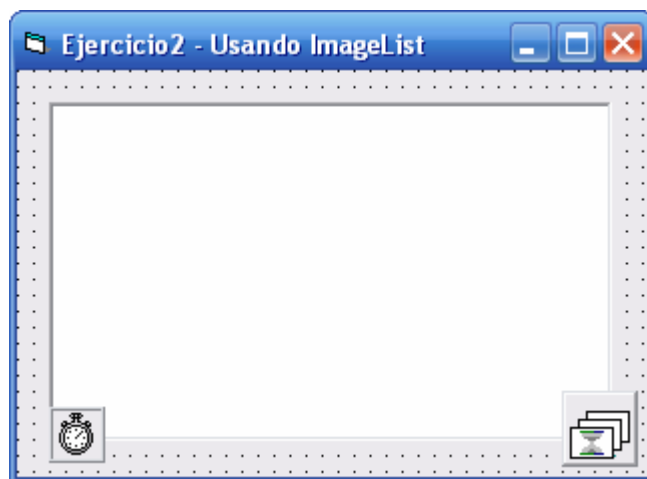
Para agregar imágenes a la lista solo debe hacer clic en el botón *Agregar* y buscarlas en su disco duro. Luego para verlas haga clic sobre la imagen que desea de la lista. Para eliminar una imagen de la lista selecciónela y, a continuación, haga clic en el botón *Borrar*.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio6-6** para el formulario, y **Ejercicio6-6** para el proyecto.

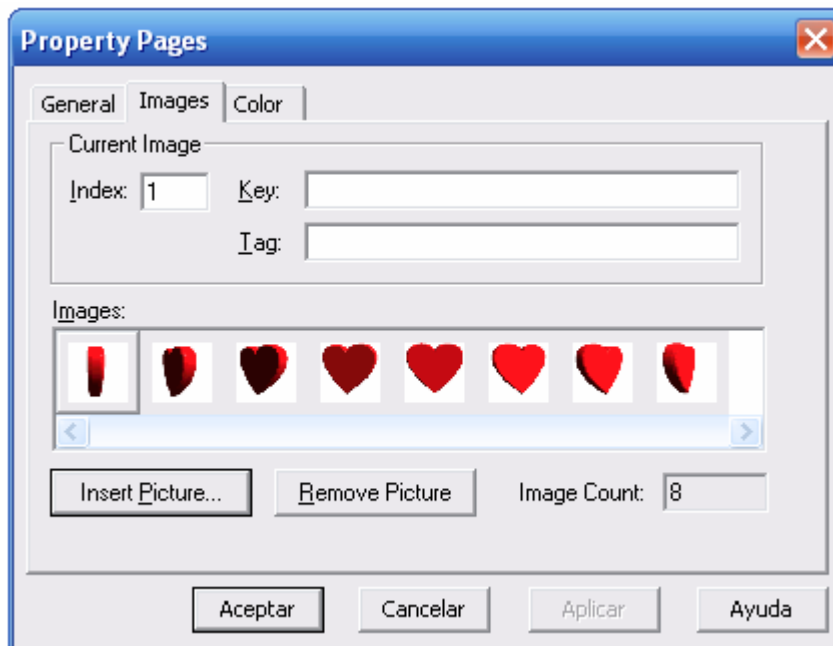
Ejercicio práctico 2:

En el siguiente ejercicio mostraremos como crear una pequeña animación utilizando varias imágenes cargadas de nuestro disco duro. Las imágenes la he preparado exclusivamente para este ejercicio y se encuentran en la carpeta **IMAGENES** que se distribuye con este libro.

- Abra un nuevo proyecto.
- Agregue el componente *Microsoft Windows Common Controls 6.0 (SP6)*.
- Inserte un control *ImageList* y un control *Timer* en formulario.
- Inserte un control *PictureBox* en el formulario. En la propiedad **BackColor** del *PictureBox* establezca el color blanco.



- Haga clic derecho sobre el control *ImageList* y, a continuación, haga clic en la opción *Propiedades* (Properties).
- Seleccione la ficha *Imágenes* (Images) de la ventana propiedades. Haga clic en el botón *Insertar Imagen* (Insert Picture).
- Busque la carpeta **IMÁGENES** que se distribuye con este libro y cargue las imágenes *Corazon1Cap6*, *Corazon2Cap6*, *Corazon3Cap6*, *Corazon4Cap6*, *Corazon5Cap6*, *Corazon6Cap6*, *Corazon7Cap6*, *Corazon8Cap6*. Agregue las imágenes en este mismo orden.



- Después de haber cargado las ocho imágenes haga clic en el botón *Aceptar*.
- Seleccione el control Timer y en la propiedad **Interval** escriba el valor *100*.
- En el evento **Click** del control Timer escriba:

'Declaramos la variable que almacenara el índice de cada imagen del control ImageList.
Static Indicelmagen

'Incrementamos el índice.
Indicelmagen = Indicelmagen + 1

'Cargamos en el PictureBox la imagen correspondiente al índice incrementado.
Picture1.Picture = ImageList1.ListImages(Indicelmagen).Picture

'Cuando el índice de la imagen sea igual al último (8), empezamos nuevamente.
'Esto hace que el ciclo repetitivo vuelva a iniciar.

If Indicelmagen = 8 **Then**
 Indicelmagen = 0
End If

- Corra la aplicación.

Podrá ver como las 8 imágenes combinadas crea un corazón en 3D en movimiento. Esto es posible, porque cada transición de la imagen esta cargada en el control ImageList y mediante el índice de cada una de esas imágenes la hemos cargado en el PictureBox.

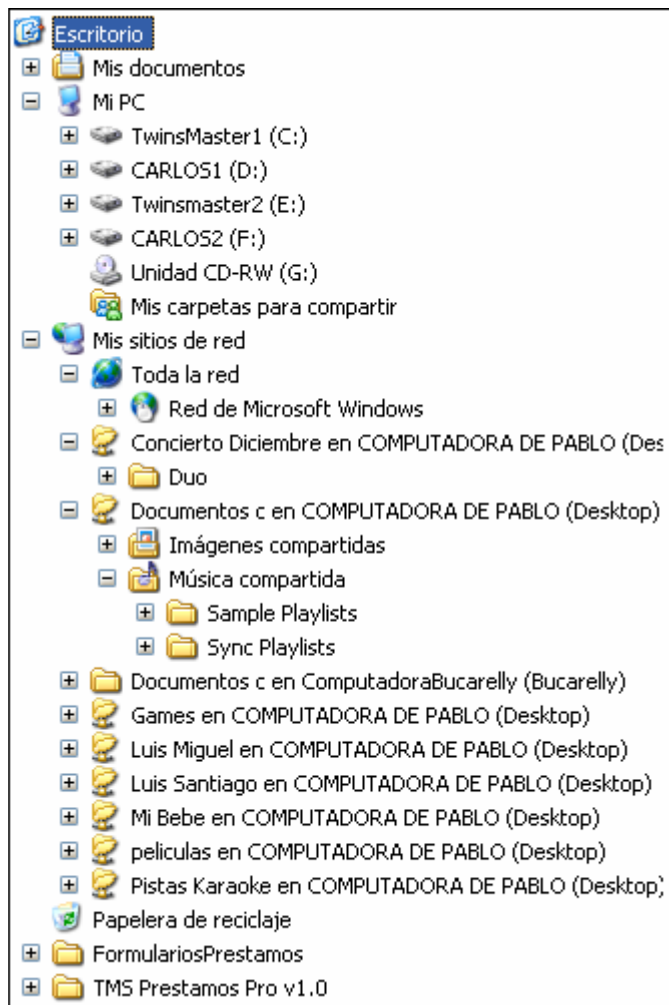
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio7-6** para el formulario, y **Ejercicio7-6** para el proyecto.



- 6.2.2 Control TreeView



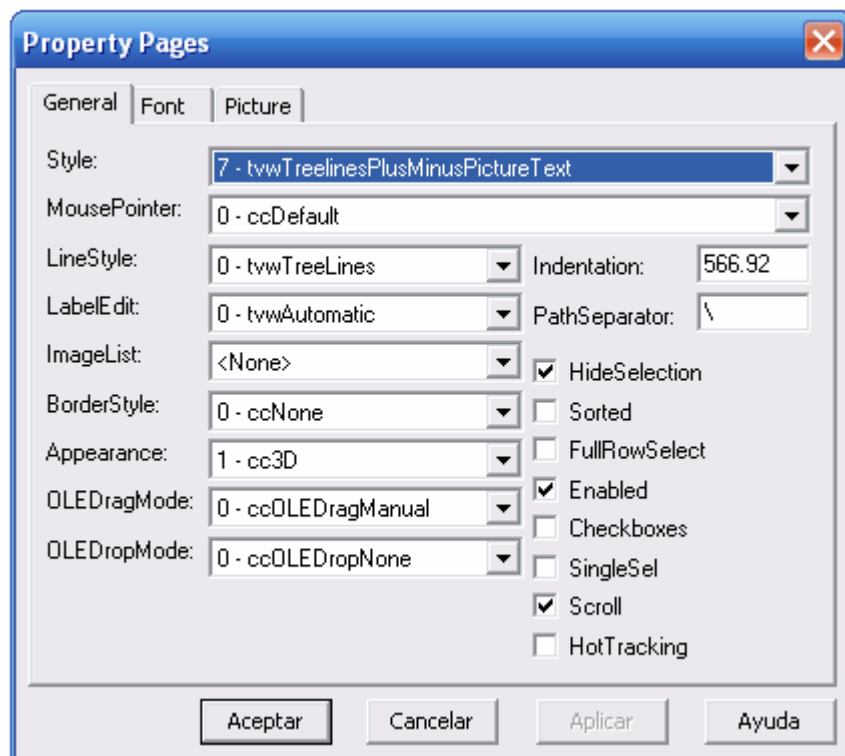
El control **TreeView** de formularios muestra una jerarquía de nodos similar al modo en que se muestran los archivos y las carpetas en el panel izquierdo de la característica Explorador de Windows del sistema operativo Windows. Este está compuesto a su vez por elementos llamados nodos y subnodos (*nodos secundarios*). Con el control TreeView, puede mostrar una jerarquía de nodos a usuarios, del modo en que se muestran los archivos y carpetas en el Explorador de Windows. Cada nodo de la vista de árbol podría contener otros nodos, llamados nodos secundarios. Puede mostrar nodos primarios o nodos que contienen nodos secundarios, como expandidos o contraídos. A continuación, se muestra una imagen con un control TreeView con sus nodos y subnodos:



Cada elemento con el símbolo (+) indica un nodo o subnodo principal que a su vez contiene otros subnodos. Los elementos con el signo (-) indica que un nodo está expandido o ramificado.

- 6.2.2.1 Configurar el control TreeView en tiempo de diseño

Una vez usted haya insertado un control TreeView en el formulario, deberá configurar su aspecto gráfico desde su ventana de diálogo **Propiedades**. Para esto, deberá pulsar el botón derecho del Mouse sobre el control y seleccionar la opción *Propiedades*. Las propiedades que aparecen en esta pantalla también podrán cambiarlas en tiempo de ejecución, pero en contadas ocasiones necesitará modificar el aspecto de un control TreeView después de habérselo presentado al usuario.



La propiedad **Style** afecta a los elementos gráficos que se utilizarán dentro del control. Un control TreeView puede mostrar cuatro elementos gráficos: el texto asociado con cada objeto Nodo, la imagen asociada con cada objeto Nodo, un signo más (+) o menos (-) al lado de cada objeto Nodo y las líneas que van desde cada objeto Nodo a sus objetos hijo. Para la propiedad *Style* podrá seleccionar entre ocho valores posibles, cada uno de los cuales representa una combinación de los cuatro elementos gráficos. En la mayoría de los casos, utilizará el valor predeterminado *7-twwTreelinesPlusMinusPictureText*, que muestra todos los elementos gráficos.

La propiedad **MousePointer** es la propiedad común en la mayoría de los controles de Visual Basic, que afecta el puntero del Mouse sobre un control. No deberá preocuparse por el momento en modificar el valor por defecto de esta propiedad.

La propiedad **LineStyle** especifica si se mostraran o no se mostraran las líneas entre los Nodos del control TreeView. El valor *0-twwTreeLines* (valor por defecto) no muestra líneas entre los objetos Nodos raíces, mientras que el valor *1-twwRootLines* si muestra líneas entre todos los Nodos raíces. La propiedad *Indentation* define la distancia en *twips* que existe entre las líneas punteadas verticales.

La propiedad **LabelEdit** afecta la forma en que los usuarios modifican el texto de cada Nodo en modo de ejecución. El valor *0-twwAutomatic* (valor por defecto), le permitirá al usuario modificar el texto de cada Nodo con el simple hecho de hacer clic sobre este. El valor *1-twwManual*, permitirá modificar el texto de cada Nodo sólo si se llama desde código el método **StartLabelEdit** del control TreeView.

La propiedad **ImageList** muestra todos los controles ImageList insertados en el formulario, donde solo podrá seleccionar uno el cual utilizará el control TreeView para cargar las imágenes que este asignará a los objetos Nodos individuales.

La Propiedad **HideSelection** determina si el objeto Nodo seleccionado continuará estando resaltado cuando el control TreeView pierda el foco.



La propiedad **Sorted** define si los Nodos en el control se ordenarán alfabéticamente de manera automática. Esta propiedad solo afecta a los nodos raíces, no tiene ningún efecto sobre los objetos Nodos hijos a niveles inferiores. Si desea ordenar todas las ramas del árbol, deberá definir como **True** la propiedad **Sorted** de todos los Nodos individuales.

La propiedad **FullRowSelect** determina la forma en que se seleccionan los Nodos del control TreeView. Cuando esta propiedad es **True**, hará que se seleccione un Nodo del control si el usuario pulsa en cualquier lugar de su fila. Si es **False** (valor predeterminado), sólo se puede seleccionar un determinado elemento cuando se pulse con el Mouse sobre él o sobre su símbolo más (+) o menos (-).

La propiedad **Checkboxes** determina si al lado de cada Nodo aparecerá una casilla de verificación. Esta propiedad puede tomar los valores **True** (con casillas) y **False** (valor predeterminado, sin casillas).

La propiedad **SingleSel** determina el modo en que se expanden los Nodos del control. Cuando el **False** (valor por defecto) el usuario tendrá que realizar una doble pulsación sobre los Nodos para expandir o contraer su contenido o pulsar sobre el signo más o menos si estos se encuentran presentes. Pero si define esta propiedad como **True**, podrá expandir y contraer las ramas con una sola pulsación, es decir, tan pronto como las seleccione. Además, cuando expanda un Nodo, el elemento que se expandió previamente se contraerá automáticamente.

La propiedad **Scroll** determina si el control TreeView mostrará una barra horizontal y vertical cuando sea necesario. El valor predeterminado es **True**, aunque puede establecerlo a **False** (sin barras).

Por último, la propiedad **HotTracking** le permitirá crear una interfaz de usuario de aspecto Web. Si define esta propiedad como **True**, el puntero del Mouse se transformará en una mano cuando el Mouse sobrepase el objeto Nodo y el control TreeView subrayará la propiedad **Text** del Nodo.

- 6.2.2.2 Propiedades más importantes del control TreeView

Las propiedades principales del control TreeView son **Nodes** y **SelectedNode**. La propiedad **Nodes** contiene la lista de nodos del nivel superior de la vista de árbol. La propiedad **SelectedNode** establece el nodo actualmente seleccionado. Otra propiedad importante es **ImageIndex** que establece la imagen predeterminada para los nodos en la vista de árbol.

- 6.2.2.3 Como añadir objetos Node (Nodos)

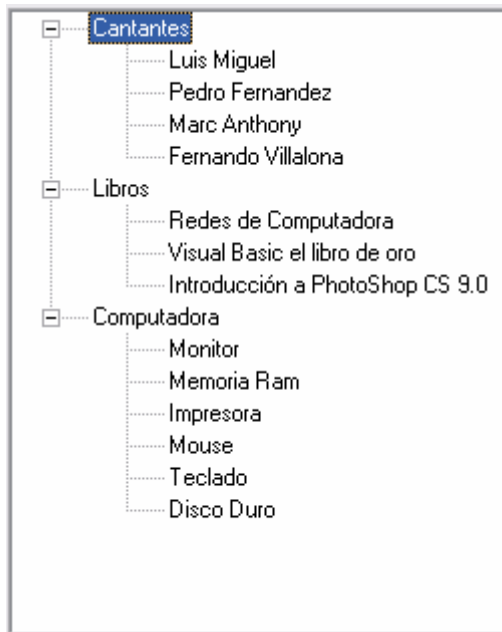
Una de las principales desventajas del control TreeView es que no podrá añadir elementos en tiempo de diseño como lo hacía con los controles ListBox y ComboBox. Sólo podrá añadir objetos Node en tiempo de ejecución mediante código con el método **Add**. La sintaxis del método **Add** es la siguiente:

Add ([Relativo], [Relación], [Clave], [Texto], [Imagen], [ImagenSeleccionada]) As Node

Los atributos **Relativo** es la clave del Nodo donde se insertará el nuevo elemento, si se omite se asume que el elemento insertado es un Nodo raíz. **Relación** indican el lugar en el que se insertará el Nodo, puede tomar los valores **0-tvwFirst** (el Nodo se convierte en el primer elemento del nivel de su nodo relacionado); **1-tvwLast** (El nuevo Nodo se convierte en el último pariente del Nodo relativo); **2-tvwNext** (valor predeterminado, el nuevo Nodo se añade inmediatamente después del Nodo relativo); **3-tvwPrevious** (El Nodo se inserta inmediatamente antes del Nodo relativo, al mismo nivel dentro de la jerarquía) o **4-tvwChild** (El Nodo se convierte en un hijo del Nodo relativo). **Clave** es su clave de cadena en la colección Nodes. **Texto** es el rótulo que aparecerá en el control e **Imagen** es el índice o la clave de cadena del control ImageList que acompaña a la imagen y que aparecerá al lado del Nodo. **ImagenSeleccionada** es el índice o la clave de la imagen que se

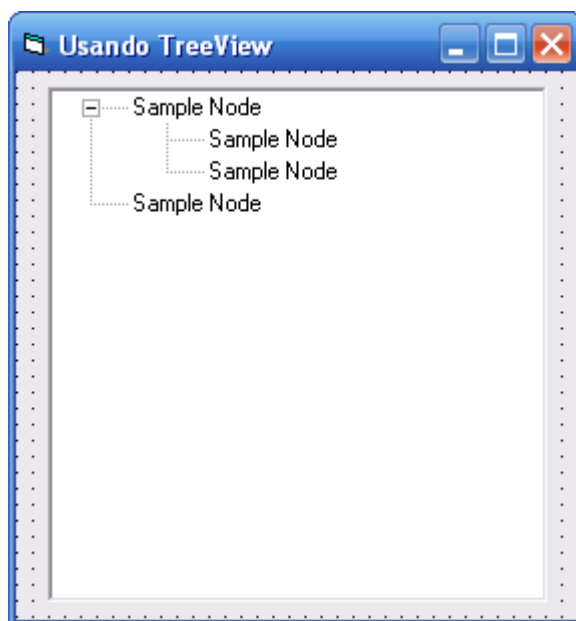


utilizará cuando se seleccione el Nodo. A continuación, veremos como crear un árbol compuesto por varios Nodos raíz y varios nodos hijos. Este ejercicio pretende enseñarle a crear un árbol similar al que se muestra en la siguiente imagen:



Para crear el árbol anterior siga los siguientes pasos:

- Abra un nuevo proyecto.
- Haga clic derecho en la barra de controles y seleccione *Components (Componentes)* del menú que aparece. En la ventana componentes active el control *Microsoft Windows Common Controls 6.0 (SP6)*.
- Dibuje un control TreeView  en el formulario. Tal y como se muestra en la siguiente imagen:





- Haga clic derecho sobre el control `TreeView` y seleccione la opción *Properties (Propiedades)* del menú contextual que aparece. En esta ventana busque la propiedad **LineStyle** y establezca el valor **1-tvwRootLines** y, a continuación, haga clic en el botón *Aceptar*. Esto es para que los Nodos tengan el símbolo de más (+) y menos (-).
- Haga doble clic en cualquier parte del formulario y en el evento **Load** escriba el siguiente bloque de código:

```
Dim NodosPrincipales As Node
Dim NodosHijos As Node

'Agregamos los Nodos Principales.
Set NodosPrincipales = TreeView1.Nodes.Add(, , "cantantes", "Cantantes")
Set NodosPrincipales = TreeView1.Nodes.Add(, , "libros", "Libros")
Set NodosPrincipales = TreeView1.Nodes.Add(, , "computadora", "Computadora")

'Agregamos los Nodos hijos de cada Nodo Principal.
'Nodos hijos para el Nodo principal de Cantantes.
Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "luismiguel", _
"Luis Miguel")

Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "pedrofernandez", _
"Pedro Fernandez")

Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "marcanthony", _
"Marc Anthony")

Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "fernandovillalona", _
"Fernando Villalona")

'Nodos hijos para el Nodo principal de Libros.
Set NodosPrincipales = TreeView1.Nodes.Add("libros", tvwChild, "redesdecomputadora", _
"Redes de Computadora")

Set NodosPrincipales = TreeView1.Nodes.Add("libros", tvwChild, "visualbasic", _
"Visual Basic el libro de oro")

Set NodosPrincipales = TreeView1.Nodes.Add("libros", tvwChild, "photoshop", _
"Introducción a PhotoShop CS 9.0")

'Nodos hijo para el Nodo principal de computadora.
Set NodosPrincipales = TreeView1.Nodes.Add("computadora", tvwChild, "monitor", "Monitor")

Set NodosPrincipales = TreeView1.Nodes.Add("computadora", tvwChild, "memoria", _
"Memoria Ram")

Set NodosPrincipales = TreeView1.Nodes.Add("computadora", tvwChild, "impresora", _
"Impresora")

Set NodosPrincipales = TreeView1.Nodes.Add("computadora", tvwChild, "mouse", "Mouse")

Set NodosPrincipales = TreeView1.Nodes.Add("computadora", tvwChild, "teclado", "Teclado")

Set NodosPrincipales = TreeView1.Nodes.Add("computadora", tvwChild, "discoduro", _
"Disco Duro")
```



- Corra la aplicación. Haga clic en los signos de más (+) para expandir el árbol y en los signos de menos (-) para contraer el árbol.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio8-6** para el formulario, y **Ejercicio8-6** para el proyecto.

Explicación del código:

Dim NodosPrincipales As Node

Dim NodosHijos As Node

Declaramos las variables que nos servirán luego para crear los Nodos y para realizar cualquier operación con estos. Por ejemplo, podrá cambiar independientemente el color de fondo de un Nodo principales colocando sobre este el código: `NodosPrincipales.BackColor = vbRed`. Puede definir una variable para cada Nodo principal y realizar operaciones independientes para cada uno. De igual manera podrá cambiar el color del texto de un Nodo hijo con el siguiente código: `NodosHijos.ForeColor = vbBlue`.

```
Set NodosPrincipales = TreeView1.Nodes.Add(, , "cantantes", "Cantantes")
```

```
Set NodosPrincipales = TreeView1.Nodes.Add(, , "libros", "Libros")
```

```
Set NodosPrincipales = TreeView1.Nodes.Add(, , "computadora", "Computadora")
```

Estas tres líneas nos permiten agregar los Nodos principales. Cada Nodo principal tiene asignada una clave que luego servirá para poder agregar Nodos hijos. Por ejemplo, la primera línea de estas tres tiene como clave *cantantes* que es la clave que diferencia a este Nodo de todos los demás. No confunda la clave con el texto que se mostrará al lado del Nodo. Aunque en este ejercicio he escrito lo mismo en ambos parámetros no es lo mismo.

```
Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "luismiguel", _  
"Luis Miguel")
```

```
Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "pedrofernandez", _  
"Pedro Fernandez")
```

```
Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "marcanthony", _  
"Marc Anthony")
```

```
Set NodosPrincipales = TreeView1.Nodes.Add("cantantes", tvwChild, "fernandovillalona", _  
"Fernando Villalona")
```

En este bloque de código hemos agregado los Nodos hijos para el Nodo principal de los cantantes. Podrá observar, que hemos especificado para cada Nodo hijo, en el primer parámetro del método **Add** la clave *cantantes*, esto es, para que los elementos se agreguen dentro de este Nodo. En el segundo parámetro escribimos el valor **tvwChild** que indica que es un Nodo hijo. En el tercer parámetro escribimos una clave para este Nodo hijo que luego puede ser utilizada para agregar Nodos hijos dentro de este. Por último, escribimos el texto que tendrá el Nodo Hijo. Esto mismo hemos hecho para los Nodos hijos de los Nodos principales libros y computadora, con la única diferencia de la clave en el parámetro *Relativo*.

- 6.2.2.4 Cómo leer el texto del Nodo seleccionado

Para leer el texto de un Nodo seleccionado sólo tendrá que consultar la propiedad **Text** de la propiedad **SelectedItem** del control **TreeView**. El siguiente ejemplo muestra en un cuadro de mensaje cualquier elemento seleccionado en el árbol:

```
MsgBox (TreeView1.SelectedItem.Text)
```



Si escribe la línea de código anterior dentro del evento **Click** del control **TreeView**, se mostrará un cuadro de mensaje con el texto del elemento seleccionado cada vez que haga clic sobre uno de los Nodos.

Podrá recorrer todos los elementos del control **TreeView** asignando un valor a la propiedad **Item**:

Dim i As Long

For i = 1 To TreeView1.Nodes.Count

Print TreeView1.Nodes.Item(i)

Next i

Los elementos se mostrarán en el mismo orden que lo ha agregado al control **TreeView**.

- 6.2.2.5 Eliminar uno o todos los elementos del control **TreeView**

Para eliminar uno o varios elementos del control **TreeView** deberá llamar el método **Remove**. Este método requiere como argumento el valor del elemento o Nodo que desea borrar. También podrá especificar como argumento la clave del Nodo que desea eliminar de árbol. El siguiente ejemplo muestra como eliminar un elemento seleccionado:

'Eliminamos el Nodo seleccionado.

TreeView1.Nodes.Remove (TreeView1.SelectedItem.Index)

Puede eliminar un Nodo por su clave de la siguiente manera:

'Eliminamos el Nodo cantantes.

TreeView1.Nodes.Remove("cantantes")

Si desea eliminar todos los elementos del árbol sólo deberá llamar el método **Clear**:

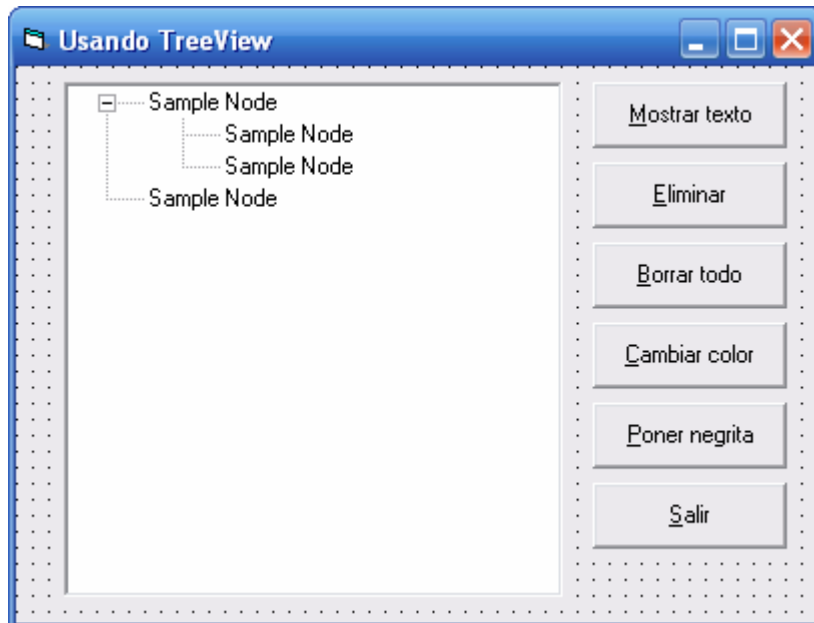
TreeView1.Nodes.Clear **'Elimina todos los elementos del control TreeView.**

- 6.2.2.6 Ejercicios prácticos

- Abra nuestro primer ejercicio del control **TreeView**. A la derecha del control agregue seis botones de comando.

- En la propiedad **Caption** del primer botón de comando escriba *&Mostrar texto*.
- En la propiedad **Caption** del segundo botón de comando escriba *&Eliminar*.
- En la propiedad **Caption** del tercer botón de comando escriba *&Borrar todo*.
- En la propiedad **Caption** del cuarto botón de comando escriba *&Cambiar color*.
- En la propiedad **Caption** del quinto botón de comando escriba *&Poner negrita*.
- En la propiedad **Caption** del sexto botón de comando escriba *&Salir*.

- Haga clic derecho sobre el control **TreeView** y ejecute la opción *Properties (Propiedades)* del menú que aparece. En la propiedad **LabelEdit** establezca el valor **1-tvwManuel** (esto es para que no se pueda cambiar el texto de los Nodos en modo de ejecución). Active la caja de verificación de la propiedad **HotTracking** (Esto es para que el control **TreeView** tenga aspecto de Web). La aplicación deberá tener un aspecto similar al que se muestra en la imagen de la siguiente página...



- Dentro del evento **Click** del primer botón de comando escriba:

'Muestra en un cuadro de mensaje el texto del elemento seleccionado.

```
MsgBox (TreeView1.SelectedItem.Text)
```

- Dentro del evento **Click** del segundo botón de comando escriba:

'Verificamos si la lista tiene elementos.

```
If TreeView1.Nodes.Count > 0 Then
```

'Eliminamos el elemento seleccionado.

```
TreeView1.Nodes.Remove (TreeView1.SelectedItem.Index)
```

```
Else
```

MsgBox ("No hay elementos que borrar.")

```
End If
```

- Dentro del evento **Click** del tercer botón de comando escriba:

'Borramos todos los elementos del TreeView.

```
TreeView1.Nodes.Clear
```

- Dentro del evento **Click** del cuarto botón de comando escriba:

'Cambiamos el color de todos los elementos del TreeView.

```
Dim i As Long
```

```
For i = 1 To TreeView1.Nodes.Count
```

'Ponemos en azul todos los elementos del TreeView.

```
TreeView1.Nodes.Item(i).ForeColor = vbBlue
```

```
Next i
```




- Dentro del evento **Click** del quinto botón de comando escriba:

'Asignando formato de negrita a todos los elementos del TreeView.

Dim i As Long

For i = 1 To TreeView1.Nodes.Count

'Ponemos en negrita todos los elementos del TreeView.

TreeView1.Nodes.Item(i).ForeColor = vbBlue

Next i

- Dentro del evento **Click** del sexto botón de comando escriba:

'Salimos de la aplicación.

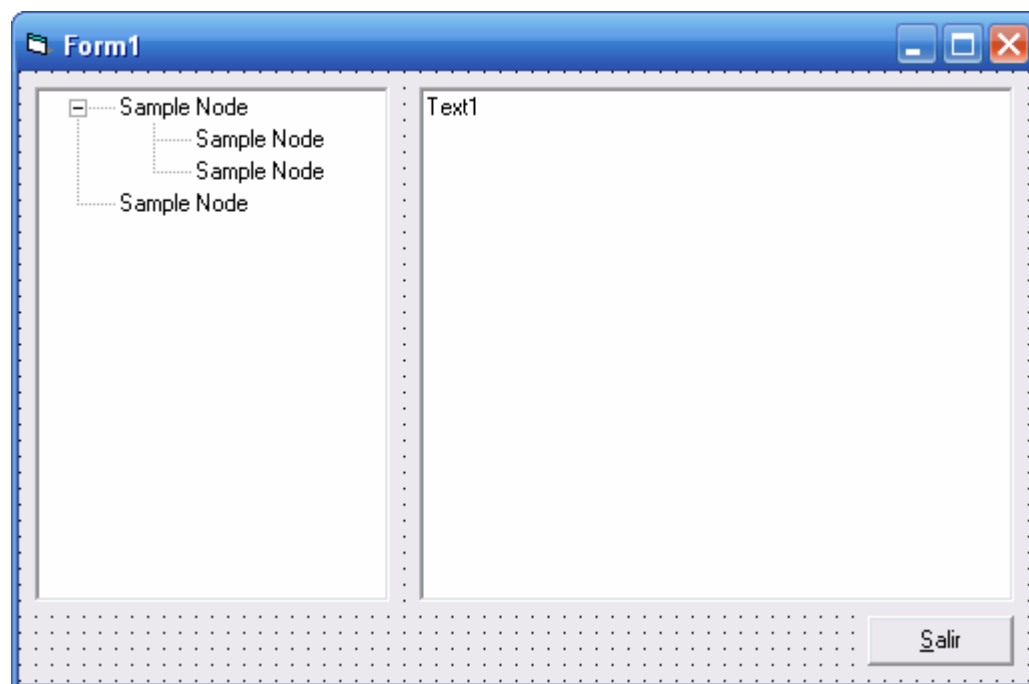
End

- Corra la aplicación. Pruebe cada uno de los botones de comando.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio9-6** para el formulario, y **Ejercicio9-6** para el proyecto.

Ejercicio práctico #2:

En este segundo ejercicio veremos como utilizar un árbol en un control TreeView para aplicar formato a una caja de texto.

- Abra un nuevo proyecto.
- Agregue el componente *Microsoft Windows Common Controls 6.0 (SP6)* a su proyecto.
- Dibuje un control TreeView, una caja de texto y un botón de comando:





- Haga clic derecho sobre el control TreeView y ejecute la opción *Properties (Propiedades)*. En la propiedad **LineStyle** seleccione el valor 1-tvwRootLines. En la propiedad **LabelEdit** seleccione 1-tvwManual y active la casilla **HockTracking**.

- En el evento **Load** del formulario escriba:

```
Dim Efectos As Node
Dim Colores As Node
Dim Tamaños As Node
```

'Definimos el Nodo principal de los efectos.

```
Set Efectos = TreeView1.Nodes.Add(, "efectos", "Efectos")
```

'Elementos del Nodo Efectos.

```
TreeView1.Nodes.Add "efectos", tvwChild, "negrita", "Negrita"
TreeView1.Nodes.Add "efectos", tvwChild, "cursiva", "Cursiva"
TreeView1.Nodes.Add "efectos", tvwChild, "subrayado", "Subrayado"
TreeView1.Nodes.Add "efectos", tvwChild, "tachado", "Tachado"
```

'Definimos el Nodo principal de los Colores.

```
Set Colores = TreeView1.Nodes.Add(, "colores", "Colores")
```

'Elementos del Nodo Colores.

```
TreeView1.Nodes.Add "colores", tvwChild, "azul", "Azul"
TreeView1.Nodes.Add "colores", tvwChild, "verde", "Verde"
TreeView1.Nodes.Add "colores", tvwChild, "rojo", "Rojo"
TreeView1.Nodes.Add "colores", tvwChild, "amarillo", "Amarillo"
TreeView1.Nodes.Add "colores", tvwChild, "cyan", "Cyan"
TreeView1.Nodes.Add "colores", tvwChild, "negro", "Negro"
```

'Definimos el Nodo principal de los tamaño.

```
Set Tamaños = TreeView1.Nodes.Add(, "tamaños", "Tamaños")
```

'Elementos del Nodo Tamaños.

```
TreeView1.Nodes.Add "tamaños", tvwChild, "tm12", "12"
TreeView1.Nodes.Add "tamaños", tvwChild, "tm14", "14"
TreeView1.Nodes.Add "tamaños", tvwChild, "tm16", "16"
TreeView1.Nodes.Add "tamaños", tvwChild, "tm18", "18"
TreeView1.Nodes.Add "tamaños", tvwChild, "tm20", "20"
```

- Dentro del evento **Click** del control TreeView escriba:

'Determinamos el elemento seleccionado y aplicamos los efectos a la caja de texto.

```
Select Case LCase(TreeView1.SelectedItem.Text)
```

```
Case "negrita": Text1.FontBold = True
```

```
Case "cursiva": Text1.FontItalic = True
```

```
Case "subrayado": Text1.FontUnderline = True
```

```
Case "tachado": Text1.FontStrikethru = True
```

```
Case "azul": Text1.ForeColor = vbBlue
```



```
Case "verde": Text1.ForeColor = vbGreen
Case "rojo": Text1.ForeColor = vbRed
Case "amarillo": Text1.ForeColor = vbYellow
Case "cyan": Text1.ForeColor = vbCyan
Case "negro": Text1.ForeColor = vbBlack
Case "12": Text1.FontSize = 12
Case "14": Text1.FontSize = 14
Case "16": Text1.FontSize = 16
Case "18": Text1.FontSize = 18
Case "20": Text1.FontSize = 20
End Select
```

- En el evento **Click** del botón Salir escriba:

```
'Sale de la aplicación.
End
```

- Corra la aplicación.

Haga clic sobre los signos de más para expandir el árbol. Haga clic sobre los efectos, colores y tamaños disponibles en el árbol y verá como estos se aplican a la caja de texto.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio10-6** para el formulario, y **Ejercicio10-6** para el proyecto.

- 6.2.3 Control ListView

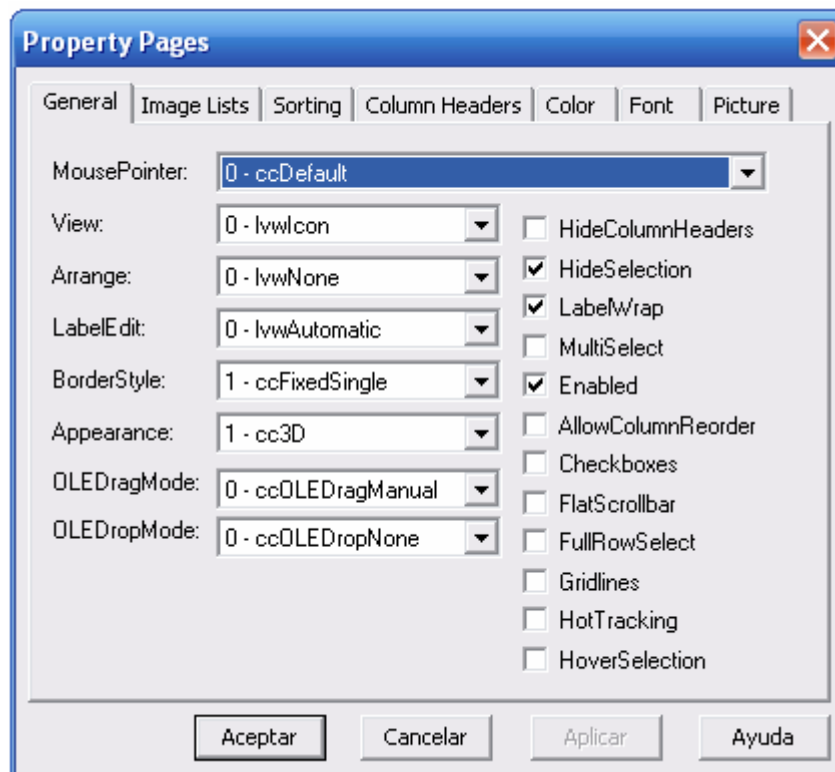
El control **ListView** es aquel que podemos ver a la derecha del control **TreeView** del Explorador de Windows. En el Explorador de Windows este muestra gráficamente el contenido de las carpetas y unidades seleccionadas en el control **TreeView**. En Visual Basic podrá utilizarlo para mostrar elementos gráficos como carpetas, iconos, imágenes, texto, etc. También podrá utilizarlo para mostrar reportes e información de forma organizada.

El control **ListView** cuenta con cuatro modo de visualización: **Icon**, **SmallIcon**, **List** y **Report**. Para ver la diferencia entre cada modo ejecute el Explorador de Windows y pruebe las opciones contenidas en el menú **Ver** (el modo **Report** se corresponde con el mandato **Detalles**).

- 6.2.2.1 Configurar el control ListView en tiempo de diseño

Al igual que el control **TreeView** usted necesitará personalizar el control **ListView** desde su cuadro de diálogo *Propiedades (Properties)*. Para abrir el cuadro de dialogo propiedades ejecute los mismo pasos que realizaba con el control **TreeView**. Aunque podrá utilizar la ventana normal de *Propiedades* para definir la mayor parte de las propiedades del control **ListView**, es preferible utilizar el cuadro de diálogo *Propiedades*. Esta ventana es similar a la del control **TreeView** a diferencia de

algunas propiedades.



La propiedad **View** nos permite seleccionar el tipo de vista del control, esta puede tomar los valores: 0-lvwIcon, 1-lvwSmallIcon, 2-lvwList o 3-lvwReport. Esta propiedad la podrá modificar en tipo de ejecución para ofrecerles a los usuarios la posibilidad de cambiar de tipo de vista, normalmente en un menú Ver. La propiedad **Arrange** le permitirá establecer el tipo de alineación de los iconos, permitiendo seleccionar tres valores posibles: 1-lvwAutoLeft (alineación automática a la izquierda, 2-lvwAutoTop (alineación automática en la parte superior) o 0-lvwNone (valor por defecto, ninguna alineación). Esta propiedad sólo tiene efecto cuando el control se encuentra en el modo de visualización Icon o SmallIcon (Iconos grandes o pequeños).

La propiedad **LabelEdit** determina si el usuario podrá modificar el texto asociado a un elemento del control. Si esta propiedad tiene el valor 0-lvwAutomatic, sólo se podrá iniciar la operación de edición mediante código utilizando el método **StartLabelEdit**. La propiedad **LabelWrap** especifica si las etiquetas de gran tamaño se dividirán en varias líneas de texto cuando la visualización este en modo Icon. La propiedad **HideColumnHeaders** determina si los encabezados de las columnas son visibles en el modo Report (el valor por defecto es False, valor que hace visible los encabezados). La propiedad **MultiSelect** determina si el usuario podrá seleccionar varios elementos (el valor por defecto es False, solo podrá seleccionar un elemento a la vez).

La propiedad **AllowColumnReorder** determina si el usuario podrá reordenar las columnas cuando se encuentre en modo Report. Si define como True la propiedad **Gridlines** el control ListView se visualizará con líneas verticales y horizontales. La propiedad **FlatScrollbar** determina si el control tendrá barras de desplazamiento (cuando es False, valor por defecto, el control visualizará barras de desplazamiento). Las propiedades **Checkboxes**, **FullRowSelect** y **HotTracking** tienen el mismo comportamiento que en el control TreeView. La propiedad **HoverSelección**, si es True, le permitirá seleccionar un elemento con solo colocar el puntero del Mouse sobre él.

En la ficha *Image Lists* podrá apreciar cuadro cajas combinadas que le permitirá asociar tres controles ImageList al control ListView. La primera caja combinada se utilizará para establecer un



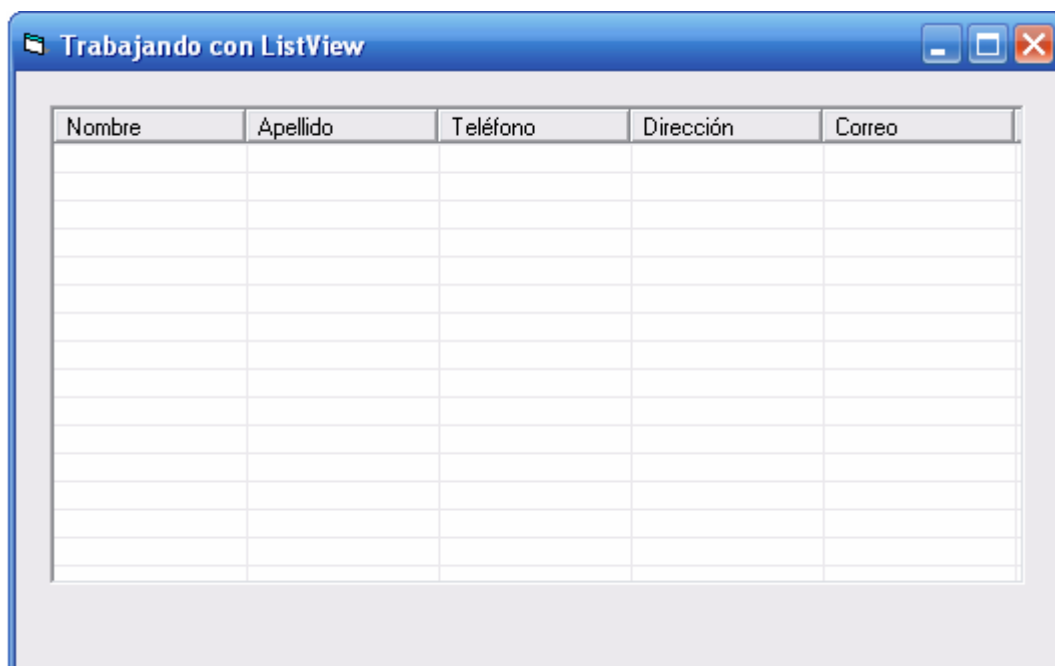
control ImageList para cuando se encuentre en modo Icono, el segundo se empleará cuando el control se encuentre en cualquier otro modo de visualización y el tercero se utilizará para iconos en las columnas.

- 6.2.3.2 Cómo añadir encabezados en modo de diseño

Para añadir encabezados (ColumnHeader) en tiempo de diseño tendrá que utilizar la ficha Encabezados (Column Headers) del cuadro de diálogo propiedades. Sólo tendrá que hacer clic en el botón *Insert Column (Insertar Columna)* y escribir el texto que tendrá el encabezado. Podrá alinear el contenido de cada columna individualmente mediante el atributo *Alignment*. De igual manera podrá especificar manualmente la anchura en *twips* mediante el atributo *Width*. También podrá especificar un valor para las propiedades **Key** y **Tag** y definir el índice del icono que se va a utilizar en ese encabezado. Este índice debe estar definido en el control ImageList asociado a los encabezados. Debe tener en cuenta que sólo en modo Report estarán visibles los encabezados.

Vamos a ver todo esto dicho anteriormente con un pequeño ejercicio práctico:

- Abra un nuevo proyecto.
- Agregue el componente *Microsoft Windows Common Controls 6.0 (SP6)* de la ventana de diálogo *Componentes*.
- Dibuje un control ListView en el formulario. Haga clic derecho sobre el control y del menú que aparece ejecute la opción *Properties (Propiedades)*. En la ventana de diálogo *Propiedades* establezca el valor 3-lvwReport en la propiedad *View* y active la casilla de la propiedad *Gridlines*.
- Haga clic en la ficha *Column Headers* y, a continuación, haga clic en el botón *Insert Column*. En el atributo *Text* escriba *Nombre*. Nuevamente haga clic en el botón *Insert Column* para agregar el encabezado. Escriba ahora a hora en el atributo *Text* el encabezado *Apellido* vuelva a hacer clic en el botón *Insert Column*. Agregue tres encabezados más: *Teléfono*, *Dirección* y *Correo*. Debe de hacer clic en el botón *Insert Column* cada vez que escriba un encabezado (excepto cuando agregue el último). Después de tener los cinco encabezados haga clic en el botón *Aceptar*.
- Corra la aplicación y tendrá el siguiente resultado:





- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio11-6** para el formulario, y **Ejercicio11-6** para el proyecto

- 6.2.3.3 Cómo añadir encabezados en modo de ejecución

Con frecuencia no sabrá en tiempo de diseño la cantidad de columnas o encabezados que deberá mostrar en un control `ListView`. Por ejemplo, puede que desee mostrar el resultado de una consulta al usuario donde usted no conoce de antemano los campos implicados en dicha consulta, para este caso tendrá que agregar las columnas en tiempo de ejecución utilizando el método **Add** mediante la siguiente sintaxis:

Add ([Index], [Key], [Text], [Width], [Alignment], [Icon]) As `ColumnHeader`

Estos son los mismos atributos que aparecen en la ventana de diálogo propiedades en la ficha `Column Headers`. *Index* es la posición dentro de la colección, *Key* (*Clave*) es una clave opcional, *Text* es la cadena de texto que se mostrará en el encabezado y *Width* es la anchura en *Twips* de la columna. El atributo *Alignment* determina el tipo de alineación que tendrán los valores de esa columna; puede ser una de las siguientes constantes: 0-`lvwColumnLeft` (valor por defecto, alineación a la izquierda), 1-`lvwColumnRight` (alineación a la derecha), o 2-`lvwColumnCenter` (alineación al centro). *Icon* es un índice o una clave en el control `ListImage` al que se hará referencia mediante la propiedad `ColumnHeaderIcons`. El siguiente ejemplo muestra como agregar los campos Nombre, Apellido, Teléfono, Dirección y Correo mediante código en tiempo de ejecución:

'Primero borramos cualquier encabezado de columna existente.

```
ListView1.ColumnHeaders.Clear
```

'Agregamos los encabezados.

```
ListView1.ColumnHeaders.Add , , "Nombre", 2000, lvwColumnLeft
ListView1.ColumnHeaders.Add , , "Apellido", 2000, lvwColumnLeft
ListView1.ColumnHeaders.Add , , "Teléfono", 1440, lvwColumnLeft
ListView1.ColumnHeaders.Add , , "Dirección", 2500, lvwColumnLeft
ListView1.ColumnHeaders.Add , , "Correo", 1500, lvwColumnLeft
```

- 6.2.3.4 Cómo añadir objetos `ListItem` (Filas)

Utilizará el mismo método **Add** para agregar objetos `ListItem` al control `ListView`. Este método lo deberá ejecutar desde la colección `ListItems` del control mediante la siguiente sintaxis:

Add ([Índice], [Clave], [Texto], [Icono], [IconoPequeño]) As `ListItem`

Índice (*Index*) es la posición donde se quiere agregar el nuevo elemento (si se omite este parámetro, el elemento se agregará al final de la colección), *Clave* (*Key*) es la clave opcional del elemento insertado dentro de la colección `ListItems`, *Texto* (*Text*) es la cadena de texto que se mostrará en el control, *Icono* (*Icon*) es un índice o una clave en el control `ImageList` apuntado por la propiedad **Icons**, y *SmallIcon* es un índice o una clave en el control `ImageList` apuntado por la propiedad **SmallIcon**.

El siguiente ejemplo muestra como escribir en las cinco columnas de nuestro primer ejemplo:

Dim li As `ListItem`

'Llenamos la primera fila.

```
Set li = ListView1.ListItems.Add( , "Carlos") 'Ponemos el nombre.
li.ListSubItems.Add , , "Rodríguez" 'Ponemos el apellido.
li.ListSubItems.Add , , "809-597-4720" 'Ponemos el teléfono.
```



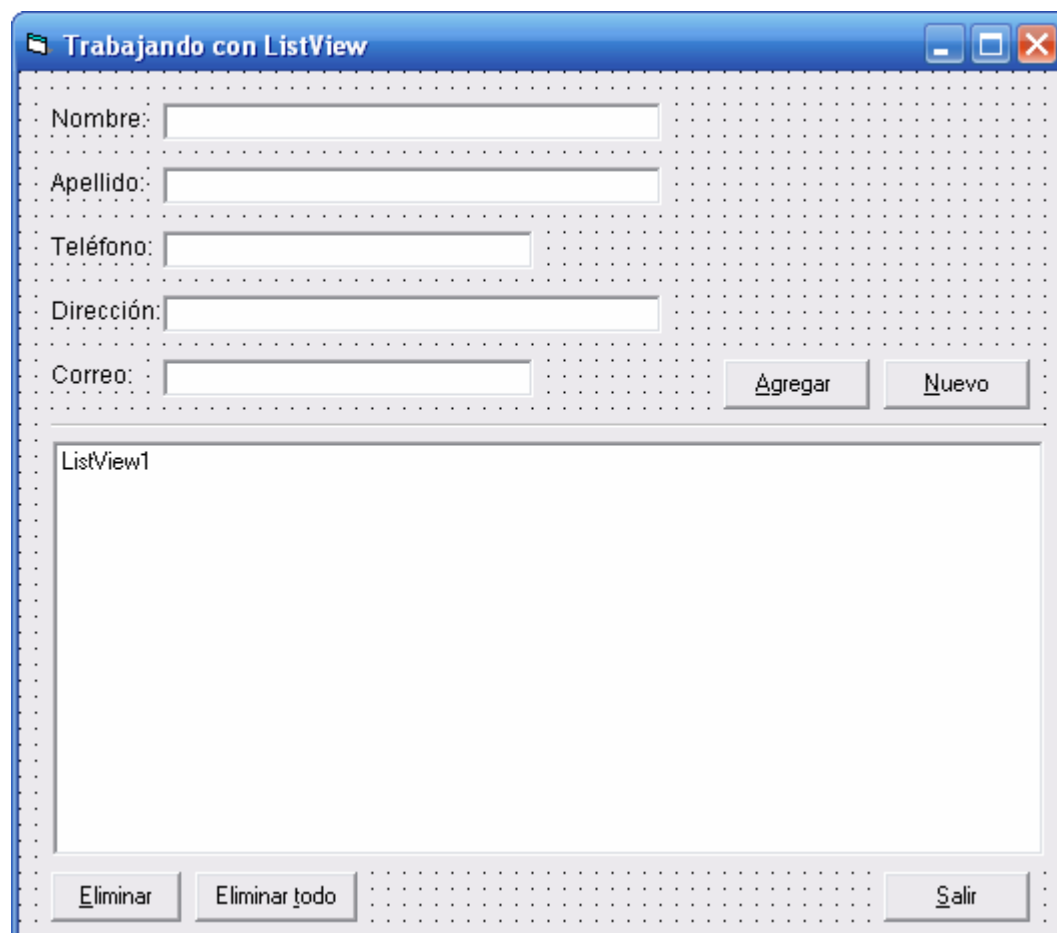
```
li.ListSubItems.Add , , "Calle #5, Lomisa"  
li.ListSubItems.Add , , "twinsmaster@hotmail.com"
```

'Ponemos la dirección.
'Ponemos el correo.

Los valores de la primera columna son agregados mediante el objeto `ListItem` y son agregados con el método **Add**. Las demás columnas pertenecen a la colección `ListSubItems` y son agregadas también con el método **Add**.

A continuación, haremos un ejercicio que le permitirá ver con más claridad la manera de agregar nuevos elementos a un control `ListView`. En este ejercicio mostraremos como cargar los valores de varias cajas de texto en el `ListView`. También verá, cómo eliminar un elemento seleccionado o todos los elementos de la lista.

- Abra un nuevo proyecto.
- Agregue a su proyecto el componente *Microsoft Windows Common Controls 6.0 (SP6)* de la ventana de diálogo *Componentes*.
- Dibuje cinco cajas de texto, cinco botones de comando y un control `ListView` en el formulario, tal y como se ve en la siguiente imagen:



- Haga clic derecho sobre el control `ListView` y del menú que aparece ejecute la opción *Properties* (*Propiedades*). En el cuadro de diálogo *Propiedades* establezca el valor 3-`lvwReport` en la propiedad **View**. En la propiedad **LabelEdit** establezca el valor 1- `lvwManual`. Active las casillas de las propiedades **FullRowSelect** y **Gridlines**. Haga clic en la ficha *Column Headers* de la ventana propiedades y agregue los encabezados: Nombre, Apellido, Teléfono, Dirección y Correo.



- Cuando haya agregado los cinco encabezados, haga clic en el botón *Aceptar* del cuadro de diálogo *Propiedades*.

- En el evento **Click** del botón de comando *Agregar* escriba:

```
'Verificamos que se escriba el nombre, el apellido y la dirección.  
If Len(Trim(Text1.Text)) = 0 Then  
    MsgBox ("Debe escribir el nombre.")  
    Text1.SetFocus  
  
Elseif Len(Trim(Text2.Text)) = 0 Then  
    MsgBox ("Debe escribir el apellido.")  
    Text2.SetFocus  
  
Elseif Len(Trim(Text4.Text)) = 0 Then  
    MsgBox ("Debe escribir la dirección.")  
    Text4.SetFocus  
  
Else  
  
    Dim li As ListItem  
  
    Set li = ListView1.ListItems.Add(, , Text1.Text) 'Ponemos el nombre.  
  
    li.ListSubItems.Add , Text2.Text 'Ponemos el apellido.  
    li.ListSubItems.Add , Text3.Text 'Ponemos el teléfono.  
    li.ListSubItems.Add , Text4.Text 'Ponemos la dirección.  
    li.ListSubItems.Add , Text5.Text 'Ponemos el correo.  
  
End If
```

- En el evento **Click** del botón de comando *Nuevo* escriba:

```
'Limpiamos todas las cajas de texto.  
Text1.Text = ""  
Text2.Text = ""  
Text3.Text = ""  
Text4.Text = ""  
Text5.Text = ""
```

- En el evento **Click** del botón de comando *Eliminar* escriba:

```
'Verificamos si hay elementos en la lista.  
If ListView1.ListItems.Count > 0 Then  
    'Eliminamos la fila seleccionada.  
    ListView1.ListItems.Remove (ListView1.SelectedItem.Index)  
Else  
    MsgBox ("La lista esta vacía.")  
End If
```

- En el evento **Click** del botón de comando *Eliminar Todo* escriba:

```
'Limpiamos la lista.  
ListView1.ListItems.Clear
```




- En el evento **Click** del botón de comando *Salir* escriba:

```
'Salimos de la aplicación.
End
```

- Corra la aplicación. Escriba en las cajas de texto y, a continuación, haga clic en el botón *Agregar*. Repita esto varias veces y verá como se agregan las filas al control **ListView**. Si desea eliminar un elemento, selecciónelo y haga clic en el botón *Eliminar*. Si desea eliminar todos los elementos que ha agregado, entonces, haga clic en el botón *Eliminar todo*.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio12-6** para el formulario, y **Ejercicio12-6** para el proyecto.

- 6.2.3.5 Leer los datos de una o varias columnas

Para recorrer los datos de una o varias columnas usted deberá consultar el objeto **ListItem** y la colección **SubItems** del mismo objeto. Por ejemplo, si queremos leer todos los valores de la primera columna de un **ListView** y cargarlos en una caja de texto (con la propiedad **MultiLine** a **True**):

```
Text1.Text = ""           'Limpiamos la caja de texto.
Dim i As Long
For i = 1 To ListView1.ListItems.Count 'Repetimos hasta la cantidad de elementos del ListView.
    'Cargamos los datos de la primera columna a la caja de texto.
    Text1.Text = Text1.Text & ListView1.ListItems(i).Text & Chr(13) & Chr(10)
Next i
```

Para leer todas las columnas deberá recorrer la colección **SubItems** y **ListItem** respectivamente:

```
Text1.Text = ""           'Limpiamos la caja.
Dim i As Long
Dim columna1, columna2, columna3, columna4, columna5

For i = 1 To ListView1.ListItems.Count 'Repetimos hasta la cantidad de elementos del ListView.

    columna1 = ListView1.ListItems(i).Text           'Columna de los nombres.
    columna2 = ListView1.ListItems(i).SubItems(1)    'Columna de los apellidos.
    columna3 = ListView1.ListItems(i).SubItems(2)    'Columna de los teléfonos.
    columna4 = ListView1.ListItems(i).SubItems(3)    'Columna de las direcciones.
    columna5 = ListView1.ListItems(i).SubItems(4)    'Columna de los correos.

    'Ponemos los datos leído en la caja de texto.
    Text1.Text = Text1.Text & columna1 & " " & columna2 & " " _
        & columna3 & " " & columna4 & " " & columna5

Next i
```

Los datos de la primera columna siempre se almacenaran en el objeto **ListItems**, las demás columnas se almacenan en la colección **SubItems**. Si observa el ejemplo anterior, verá que los valores de la primera columna son obtenidos del objeto **ListItems**. También podrá ver que los valores de las demás columnas son obtenidos de la colección **SubItems**. La primera columna de la colección **SubItems** siempre comenzará desde uno (1) sin importar la cantidad de columna que contenga el control **ListView**. Por ejemplo, si usted tiene en su aplicación un **ListView** con diez columnas, entonces, la colección **SubItems** tendrá nueve elementos comenzando desde uno (1).



- 6.2.3.6 Ordenar el contenido de las columnas

Podrá ordenar el contenido de las columnas de un control ListView utilizando las propiedades **Sorted**, **SortKey** y **SortOrder**. Si quiere ordenar los valores de una columna cuando el usuario haga clic sobre el encabezado, entonces, dentro del evento **ColumnClick** del control ListView escriba:

```
ListView1.SortKey = ColumnHeader.Index - 1 'Determinamos la columna que se va a ordenar.
ListView1.Sorted = True 'Ordenamos el contenido de la columna que se hizo clic.
```

El ejemplo anterior ordena de forma ascendente los valores de las columnas, pero si desea ofrecer la posibilidad al usuario de ordenar tanto de forma ascendente así como descendente tendrá que escribir un código como este:

```
'Con esta línea verificamos si la columna ya esta ordenada.
If ListView1.Sorted = True And ColumnHeader.Index - 1 = ListView1.SortKey Then
    'Si esta ordenada, invertimos la ordenación.
    ListView1.SortOrder = 1 - ListView1.SortOrder
Else
    'Si no estaba ordenada la ordenamos de forma ascendente.
    ListView1.SortOrder = lvwAscending
    ListView1.SortKey = ColumnHeader.Index - 1
End If

ListView1.Sorted = True
```

- 6.2.3.7 Búsqueda de elementos

Podrá buscar con rapidez una cadena de texto en un control ListView utilizando el método **FindItem** mediante la siguiente sintaxis:

FindItem (*Search*, [*Where*], [*Start*], [*Match*]) As ListItem

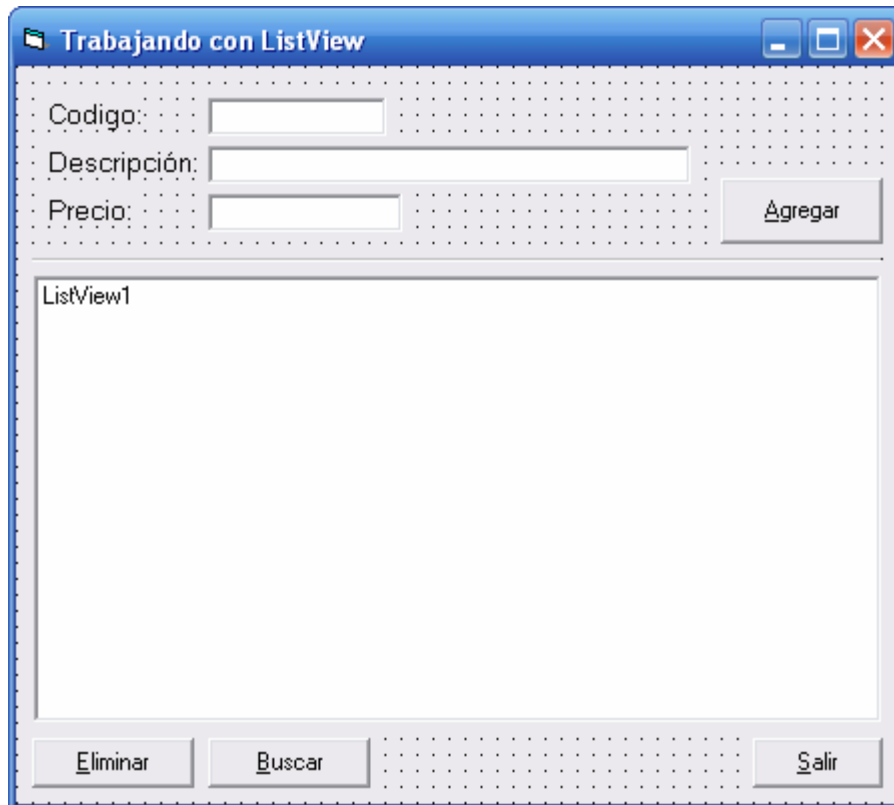
Search (buscar) es la cadena que se va a buscar. *Where* (donde) especifica en que propiedad se buscará la cadena especificada: 0-lvwText para la propiedad **Text** de los objetos ListItem, 1-lvwSubItem para la propiedad **Text** de los objetos ListSubItem o 2-lvwTag para la propiedad **Tag** de los objetos ListItem. *Start* (Inicio) es el índice o la clave del objeto ListItem desde el que comenzará la búsqueda. *Match* establece si se buscará por una palabra completa o por una parte de la cadena. Este parámetro puede tomar los valores 0-lvwWholeWord (Palabra completa) o 1-lvwPartial (parte de una cadena). Sólo se puede utilizar Match cuando el parámetro *Where* es igual a 0-lvwText.

Podrá realizar manualmente búsquedas en una columna individual recorriendo con un bucle el objeto ListItem o la colección SubItems y comparando los valores leídos con la sentencia **If**.

- 6.2.3.8 Ejercicios práctico

En el siguiente ejercicio crearemos una pequeña pantalla de facturación donde podrá agregar productos y eliminarlos. También podrá buscar un elemento de la lista. **Nota:** este ejercicio no contiene bases de datos, es simplemente una simulación de un sistema de facturación para ingresar los productos.

- Abra un nuevo proyecto.
- Agregue a su proyecto el componente *Microsoft Windows Common Controls 6.0 (SP6)* de la ventana de diálogo *Componentes*.
- Dibuje tres etiquetas, tres cajas de texto, un control ListView y cuatro botones de comando.



- Cuando todos los controles estén en el formulario, entonces, modifique las propiedades necesarias para que tengan la apariencia que se muestra en la imagen anterior.
- Haga clic derecho sobre el control ListView y del menú que aparece seleccione la opción Properties (Propiedades). En la ventana de diálogo *Propiedades* establezca el valor 3-lvwReport en la propiedad **View**. En la propiedad **LabelEdit** seleccione 1-lvwManual. Active las casillas de las propiedades **FullRowSelect** y **Gridlines**.
- Haga clic en la ficha *Column Headers* de la ventana de diálogo *Propiedades*. Agregue los siguientes encabezados con las características que se indican:
 - Primer encabezado:
 - Text = Código
 - Alignment = 0-lvwColumnLeft
 - Width = 1000
 - Segundo encabezado:
 - Text = Descripción
 - Alignment = 0-lvwColumnLeft
 - Width = 3500
 - Tercer encabezado:
 - Text = Precio
 - Alignment = 0-lvwColumnLeft
 - Width = 1200
- Una vez echo esto, haga clic en el botón *Aceptar* de la ventana de diálogo *Propiedades*.
- En el evento **Click** del botón de comando *Agregar* escriba:



```
'Verificamos que no se dejen vacías las cajas de texto.
If Len(Trim(Text1.Text)) = 0 Then
    MsgBox ("Debe especificar el código.")
    Text1.SetFocus
Elseif Len(Trim(Text2.Text)) = 0 Then
    MsgBox ("Debe especificar la descripción del producto.")
    Text2.SetFocus
Elseif Len(Trim(Text3.Text)) = 0 Then
    MsgBox ("Debe especificar el precio del producto.")
    Text3.SetFocus

Else

    'Agregamos al control ListView los datos del articulo.
    Dim li As ListItem

    Set li = ListView1.ListItems.Add(, , Text1.Text)
    li.ListSubItems.Add , , Text2.Text
    li.ListSubItems.Add , , Text3.Text

    'Limpiamos las cajas.
    Text1.Text = ""
    Text2.Text = ""
    Text3.Text = ""

End If
```

- En el evento **Click** del botón de comando *Eliminar* escriba:

```
On Error GoTo noSeleccion

'Eliminamos el elemento seleccionado.
ListView1.ListItems.Remove (ListView1.SelectedItem.Index)

Exit Sub
noSeleccion:
MsgBox ("No hay elemento seleccionado.")
```

- En el evento **Click** del botón de comando *Buscar* escriba:

```
Dim cadena As String
cadena = InputBox("Introduzca el texto a buscar:", "Buscar texto")
Dim resultado As ListItem

'Buscamos en la colección SubItems.
Set resultado = ListView1.FindItem(cadena, lvwSubItem, , lvwPartial)

'Verificamos si la búsqueda tubo o no éxito.
If resultado Is Nothing Then
    MsgBox ("No se ha encontrado ninguna coincidencia.")

Else
    resultado.Selected = True      'Seleccionamos el elemento encontrado.
    ListView1.SetFocus             'Hacemos que el control reciba el enfoque.

End If
```



- En el evento **Click** del botón de comando *Salir* escriba:

```
'Salimos de la aplicación.  
End
```

- Corra la aplicación. Ahora, complete las cajas de texto y haga clic en el botón *Agregar*. Podrá observar los datos de los productos se agregan al control *ListView*. Agregue por los menos cinco productos y luego utilice el botón *Buscar* para localizar un producto determinado dentro de la lista.
- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio13-6** para el formulario, y **Ejercicio13-6** para el proyecto.

- 6.2.4 Control *ToolBar*

Mediante el control **ToolBar** (*Barra de Herramienta*) podemos agregar a nuestra aplicación una o más barras de herramientas. Una *barra de herramienta* es aquella que típicamente se coloca debajo de la barra de menús de una aplicación que contiene los mandatos más habituales que el usuario puede ejecutar con una simple pulsación del Mouse. Este control viene incluido también en el componente *Microsoft Windows Common Controls 6.0 (SP6)*.

Una *ToolBar* (barra de herramientas) tiene el siguiente aspecto gráfico:



Esta es una barra típica de un editor de texto que contiene los mandatos: Nuevo, Abrir, Guardar, Imprimir, Vista previa, Buscar, Cortar, Copiar, Pegar, Deshacer e Insertar fecha y hora. Estos mandatos son estándar en la mayoría de las aplicaciones pero no obligatorias, es decir, su aplicación no debe incluir exactamente todos los mandatos de una barra de herramienta estándar, esto dependerá del tipo de aplicación que usted se encuentre diseñando. Usted debe de ser capaz de incluir las opciones más relevantes que usted considere en su aplicación para incluirla en su barra de herramientas.

- 6.2.4.1 Configurar el control *ToolBar* en tiempo de diseño

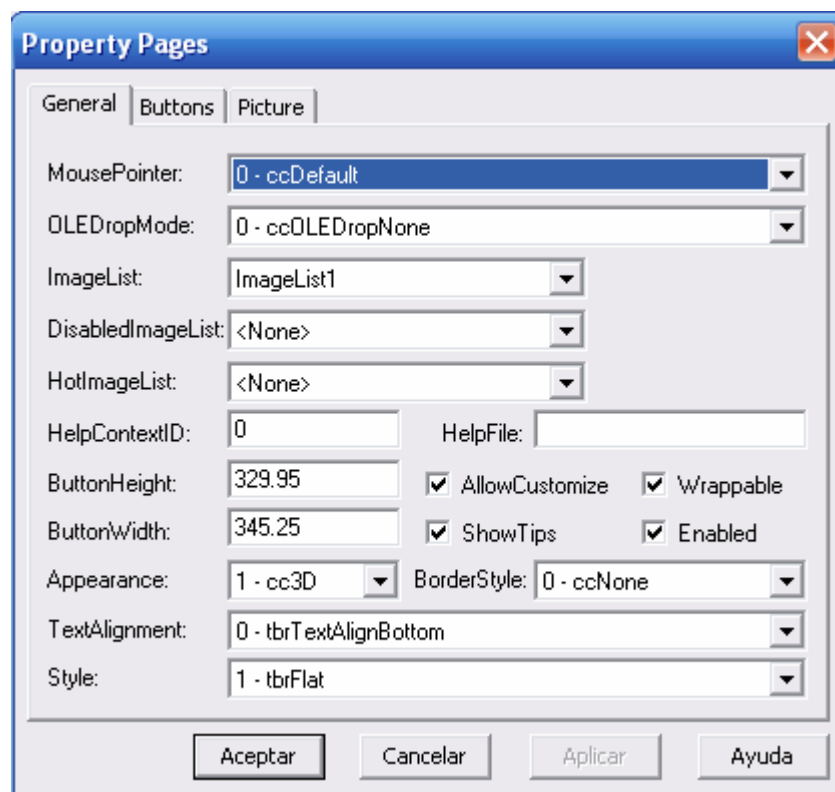
Después de crear una barra de herramientas, podrá configurarla accediendo a sus páginas de propiedades haciendo clic derecho sobre ella y seleccionando la opción *Propiedades*. El cuadro de diálogo *Propiedades* del control *ToolBar* posee tres fichas: La ficha *General* donde se encuentran las propiedades para modificar el aspecto y características en sentido general de la barra de herramientas, como por ejemplo, el ancho y alto de todos los botones de la barra, la alineación del texto de los botones, la apariencia gráfica y el estilo. La ficha *Buttons* (Botones) permite añadir, eliminar, establecer un estilo y otras opciones sólo para los botones de la barra de herramientas. Por último, la ficha *Picture* (Imagen) que permite especificar la apariencia que tendrá el puntero del Mouse cuando el usuario lo coloque sobre los botones.

Vamos a aprender a configurar un control *ToolBar* de forma práctica con un ejercicio:

- Abra un nuevo proyecto.
- Agregue a su proyecto el componente *Microsoft Windows Common Controls 6.0 (SP6)* de la ventana de diálogo *Componentes*.
- Inserte el control *ToolBar*  en el formulario. Este se ubica automáticamente en la parte superior del formulario.



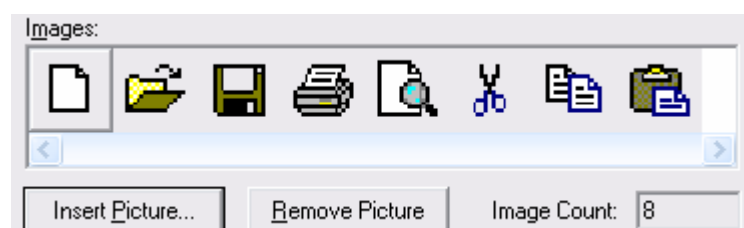
- Haga clic derecho sobre el control ToolBar y del menú que aparece ejecute la opción *Propiedades*. Aparecerá la ventana de diálogo *Propiedades*:



Vamos a explicar las propiedades que usted debe de conocer:

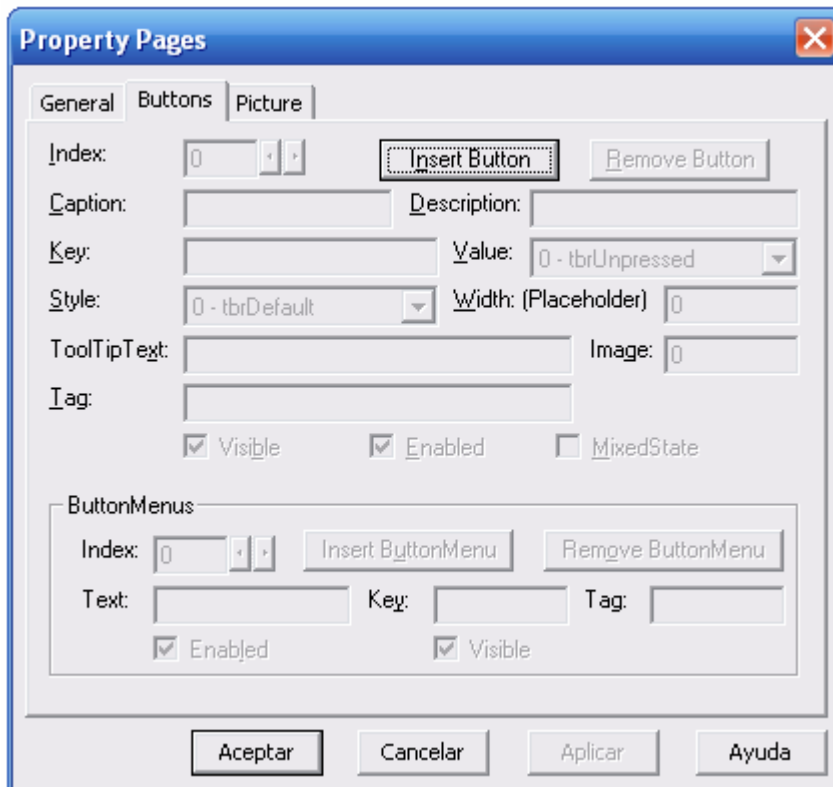
ImageList especifica el control ImageList que usted asociará para leer las imágenes que mostrará en la barra de herramientas. **DisabledImageList** especifica el control ImageList que usted utilizará cuando los botones de la barra de herramientas estén deshabilitados, es decir, cuando su propiedad **Enabled** este False. **HotImageList** especifica el control ImageList que utilizará cuando los botones estén seleccionados. **ButtonHeight** y **ButtonWidth** especifica la altura y la anchura de los botones. **TextAlignment** especifica el tipo de alineación del texto asociado a cada botón. **Style (Estilo)** permite establecer el estilo que tendrán los botones de la barra, puede tener uno de estos dos valores: 1-tbrStandard (las opciones de la barra tendrán el aspecto de un botón de comando) y 2-tbrFlat (las opciones tendrán un aspecto plano, similar a los de Microsoft Word y la mayoría de los programas).

- Después de conocer las propiedades cierre la ventana de diálogo. Ahora, inserte un control ImageList  en el formulario. Haga clic derecho sobre el control ImageList y ejecute la opción *Properties (Propiedades)*. Seleccione la ficha *Images* y agregue las imágenes 001.ico, 002.ico, 003.ico, 004.ico, 005.ico, 007.ico, 008.ico y 009.ico. Estas imágenes se encuentran dentro de la subcarpeta llamada **BarraHerramientas** en la carpeta **Imágenes** que se distribuye con este libro.





- Haga clic en el botón Aceptar del cuadro de diálogo Propiedades del control ImageList. Ahora asociaremos el control ImageList con el control ToolBar. Para esto, haga clic derecho sobre el control ToolBar y ejecute la opción Propiedades. En la propiedad **ImageList** seleccione el control ImageList que usted inserto en el formulario llamado *ImageList1*. En la propiedad **Style** establezca el valor 1-tbrFlat.
- Ahora, agregaremos los botones a la barra de herramientas. Para esto, haga clic en la ficha *Buttons* (Botones) de la ventana de diálogo propiedades. Aparecerá la siguiente ventana:



Antes de continuar con nuestro ejercicio explicare las propiedades más importantes:

Index (Índice) es un valor numérico que se le asigna automáticamente a cada botón de la barra para poder diferenciarlos de los demás dentro de la colección. Este número también es utilizado para determinar desde código cual fue el botón que el usuario ha pulsado. **Caption** (Título) es la cadena de texto que mostrará el botón en la barra de herramientas. Esta propiedad no suele ser muy utilizada. **Key** (Clave) es una cadena de texto que se le asigna al botón para diferenciarlos de los demás botones, es similar a la propiedad *Index* con la única diferencia que es usted quien especifica este valor. **ToolTipText** es la cadena de texto que aparecerá cuando el usuario coloque el puntero del Mouse sobre el botón. Debe escribir un pequeño texto que indique la función de un determinado botón en esta propiedad, y mucho más cuando sólo muestre una imagen o no escriba nada en la propiedad *Caption*. Por último, la propiedad más importante **Image** donde se especifica el número de la imagen correspondiente al control ImageList que utilizará ese botón.

- Para insertar nuestro primer botón en la barra de herramientas, haga clic en el botón de comando *Insert Button* (Insertar Botón). En la propiedad *Key* escriba Nuevo y escriba lo mismo en la propiedad *ToolTipText*. En la propiedad *Image* escriba el valor 1 (este número representa la imagen  **Nuevo** en el control ImageList). Ahora, haga clic en el botón *Aplicar*. Podrá observar como se agregan nuestro primer botón a la barra de herramientas.



▪ Agreguemos los demás botones de la barra de herramientas. Para esto, haga clic nuevamente en el botón *Insert Button*. En la propiedad *Key* escriba *Abrir* y escriba lo mismo en la propiedad *ToolTipText*. En la propiedad *Image* escriba el valor 2. Haga clic en el botón *Aplicar*.

▪ Repita esto mismo para los seis botones restantes:

- Tercer botón:
 - *Key* = Guardar
 - *ToolTipText* = Guardar
 - *Imagen* = 3
- Cuarto botón:
 - *Key* = Imprimir
 - *ToolTipText* = Imprimir
 - *Imagen* = 4
- Quinto botón:
 - *Key* = Vista previa
 - *ToolTipText* = Vista previa
 - *Imagen* = 5
- Sexto botón:
 - *Key* = Cortar
 - *ToolTipText* = Cortar
 - *Imagen* = 6
- Séptimo botón:
 - *Key* = Copiar
 - *ToolTipText* = Copiar
 - *Imagen* = 7
- Octavo botón:
 - *Key* = Pegar
 - *ToolTipText* = Pegar
 - *Imagen* = 3

▪ Después de haber agregado los demás botones, haga clic en el botón *Aceptar*.

▪ Corra la aplicación.

Podrá ver que tiene una barra de herramientas similar a la de la mayoría de las aplicaciones de Windows. Si hace clic sobre cada una de las opciones no pasará nada debido a que no hemos programado el evento **ButtonClick** del control *ToolBar*. En la siguiente sección usted aprenderá a determinar los botones que ha pulsado el usuario sobre la barra y ejecutar acciones según esas pulsaciones.

▪ Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio14-6** para el formulario, y **Ejercicio14-6** para el proyecto.

- 6.2.4.2 Cómo reaccionar ante las acciones del usuario

Cuando el usuario pulse sobre un botón de la barra de herramientas, ocurre un evento o suceso **ButtonClick**. Este suceso tiene un argumento llamado **Button** que contiene la clave del botón que se ha pulsado, lo único que tendrá que hacer es determinar con una sentencia **If** o **Case** el valor de este argumento, ejemplo:

‘Este código lo escribimos dentro del evento **ButtonClick** del control *ToolBar*:

Select Case Button.Key ‘Determina el botón que se pulsa.

Case “Nuevo”

‘Aquí llamamos el evento **Click** de la opción **Nuevo** de la barra de menús.



Case "Abrir"

'Aquí llamamos el evento Click del botón Abrir de la barra de menús.
'Ejemplo: Call optAbrir_Click

Case "Guardar"

'Aquí llamamos el evento Click del botón Guardar de la barra de menús.
'Ejemplo: Call optGuardar_Click

End Select

- 6.2.4.3 Crear menú dentro de los botones (ButtonMenu)

El control ToolBar dispone de la colección **ButtonMenu** que le permitirá agregar un menú desplegable a los botones de la barra de herramientas, tal y como se muestra en la siguiente imagen:



Hacer esto es bastante sencillo, lo podemos hacer tanto en modo de diseño, así como en modo de ejecución. Explicaré como hacer esto en modo de diseño con nuestro primer ejercicio.

- Haga clic derecho sobre la barra de herramientas que diseñamos anteriormente. En el menú que aparece ejecute la opción *Properties (Propiedades)*.
- Haga clic en la ficha *Buttons (Botones)*. Aparecerá nuestro primer botón seleccionado, que en nuestro caso es el botón Nuevo de nuestra barra de herramientas. A este botón le agregaremos un menú con las opciones Documento, Imagen e Informe. Para que este botón pueda tener un menú usted debe establecer el valor 5-tdrDropdown en la propiedad **Style** de dicho botón. Ahora, para agregar las opciones haga clic en el botón de más abajo *Insert ButtonMenu* y escriba Documento en la propiedad *Text*, escriba lo mismo en la propiedad *Key*. Para crear nuestra segunda opción, haga clic nuevamente en el botón *Insert ButtonMenu* y en la propiedad *Text* escriba Imagen igual en la propiedad *Key*. Inserte otro botón de menú y escriba Informe en la propiedad *Text*, lo mismo en la propiedad *Key*. Por último, haga clic en el botón Aceptar.

- Corra la aplicación.

Podrá observar que ahora aparece una flecha a la derecha del botón Nuevo de la barra de herramientas. Si hace clic sobre esa flecha aparecerán las opciones que usted agregó mediante la colección **ButtonMenu**. Para determinar la opción pulsada dentro de un menú desplegable, el control ToolBar dispone del suceso **ButtonMenuClick** y se programa de forma similar al evento **ButtonClick**:

Select Case ButtonMenu.Key

Case "Documento"

'Escribimos aquí el código o llamamos el evento nuevo documento.
'Ejemplo: Call optNuevoDocumento

Case "Imagen"

'Call optNuevaImagen

Case "Informe"

'Call optNuevoInforme

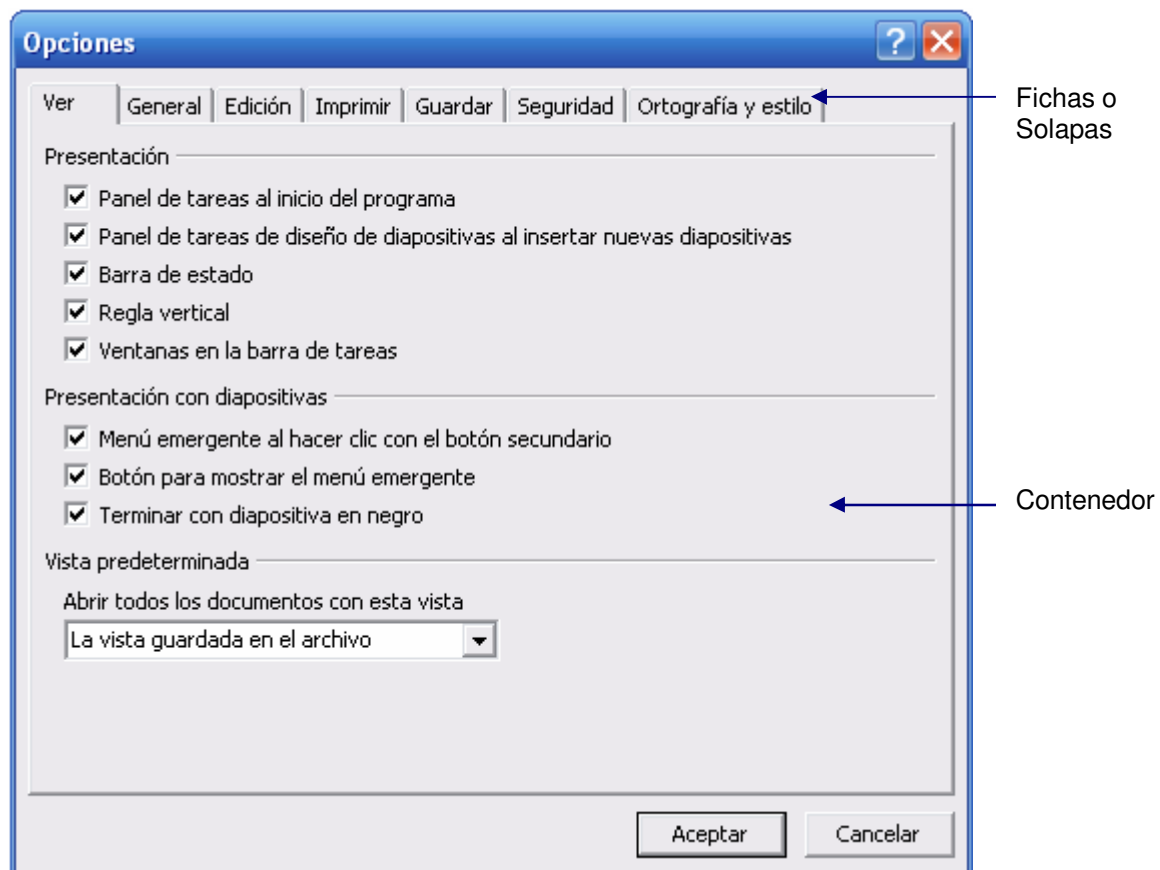
End Select

- 6.2.5 Control TabStrip

Los cuadros de diálogo que contienen varias fichas o solapas de opciones se denominan **TabStrip**. Visual Basic dispone de dos controles para diseñarlos que son: el control común TabStrip y el control SSTab. En este libro describiremos sólo el control TabStrip.

Es importante que conozca una característica de los controles TabStrip que en muchas ocasiones confunde a los programadores. Esto es, que un control TabStrip no es un contenedor, es decir, no contiene secciones donde usted puede colocar los elementos para cada opción, sino, que es un conjunto de fichas o solapas que deben ser asociadas a contenedores como PictureBox y Frame. En este caso, usted debe asociar un contenedor (PictureBox o Frame) a cada opción y visualizarla según la opción que el usuario haya pulsado. Esto es sencillo de hacer, sólo debe poner visible el contenedor correspondiente a la opción pulsada y poner invisible los contenedores de las demás opciones.

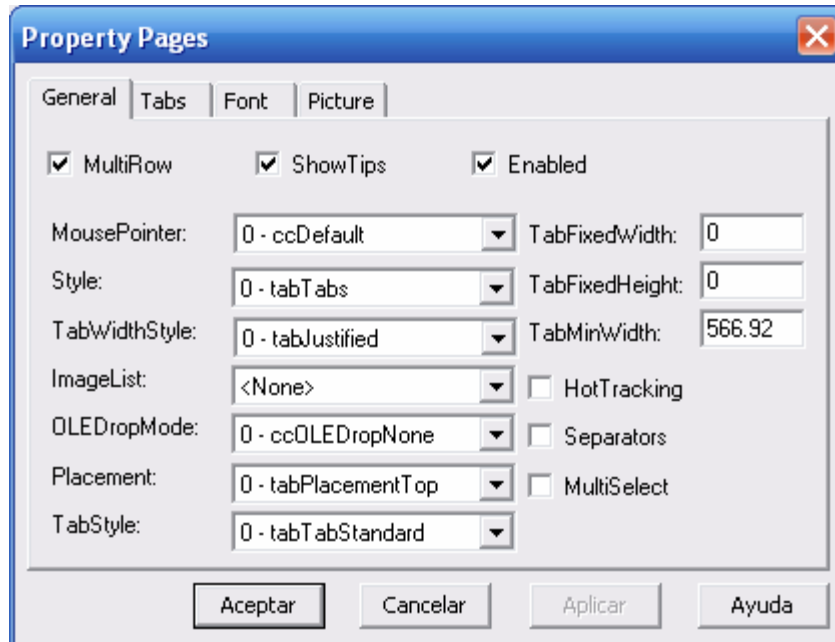
Un objeto TabStrip en una aplicación tiene la siguiente apariencia:



- 6.2.5.1 Configurando el control TabStrip en tiempo de diseño

Para configurar un control TabStrip deberá acceder a sus páginas de propiedades, haciendo clic derecho sobre el control y seleccionando la opción *Properties (Propiedades)*. Podrá definir todas las propiedades generales utilizando la ficha *General* del cuadro de diálogo *Properties (Propiedades)*. En la ficha *Tabs* podrá agregar todas las solapas o fichas que tendrá el control TabStrip. También podrá cambiar el tipo de fuente del texto que estará asociado a cada ficha mediante la solapa *Font (Fuente)*. De igual forma que el control ToolBar, podrá definir la apariencia del puntero del Mouse cuando el usuario lo coloque sobre una de las fichas mediante la solapa *Picture*.

La ficha *General* del cuadro de diálogo *Página de propiedades* del control *TabStrip* contiene las siguientes propiedades:



La primera propiedad que debe definir es la propiedad **Style** que le permitirá modificar el aspecto del control. En la mayoría de los casos lo dejará con su valor predeterminado 0-tabTabs (las fichas se muestran como una colección de páginas etiquetadas), pero también podrá asignarle el valor 1-tabButtons (las fichas se sustituyen por botones y no se muestran los bordes) o 2-tabFlatButtons (las fichas se sustituyen por botones planos y no se muestran los bordes). Si desea separar los botones utilizando separadores deberá definir como True la propiedad **Separators** (esto es válido para los dos últimos estilos).

La propiedad **TabWidthStyle** define el tipo de alineación para las fichas, esta puede tomar los valores 0-tabJustified, 1-tabNonJustified o 2-tabFixed. La propiedad **ImageList** permite especificar el control *ImageList* que asociara a las fichas. La propiedad **Placement** le permite especificar donde quiere que aparezcan las fichas. En la parte superior 0-tabPlacementTop (valor predeterminado), en la parte inferior 1-tabPlacementBottom, a la izquierda 2-tabPlacementLeft o a la derecha 3-tabPlacementRight. La propiedad **TabStyle** afecta al comportamiento en tiempo de ejecución de los controles *TabStrip* que cuenten con varias filas. Cuando asigne a esta propiedad el valor 1-tabTabOpposite, todas las filas que preceden a la actual se mostrarán en el lado opuesto del control.

- 6.2.5.2 Agregar objetos Tab

Una vez haya definido todas las propiedades más importantes podrá crear fichas en la ficha *Tabs* del cuadro de diálogo *Páginas de propiedades*. En esta ficha encontrará propiedades similares a las ya vistas en el control *ToolBar*. La propiedad **Index** (Índice) es un valor numérico que identifica cada ficha dentro de la colección. Mediante la propiedad **Caption** (Título) se especifica el texto que tendrá la ficha. La propiedad **Key** (Clave) es la típica en los controles ya estudiados. **ToolTipText** es el texto que usted quiere que aparezca cuando el usuario coloque el puntero del Mouse sobre las fichas. En la propiedad **Image** especifica el número de la imagen correspondiente al control *ImageList* que utilizará esa ficha. Esta propiedad estará habilitada, sólo si se encuentra un control *ImageList* asociado al control *TabStrip* mediante su propiedad **ImageList**.



Hagamos nuestro primer ejercicio con el control TabStrip. En este ejercicio usted creará un control TabStrip similar al que se muestra en la siguiente imagen:



- Abra un nuevo proyecto.
- Agregue a su proyecto el componente *Microsoft Windows Common Controls 6.0 (SP6)* de la ventana de diálogo *Componentes*.
- Dibuje un control TabStrip en el formulario.
- Haga clic derecho sobre el control TabStrip y ejecute la opción *Properties (Propiedades)*. En la ventana de diálogo *Páginas de propiedades* seleccione la ficha *Tabs (Fichas)*. Para agregar nuestra primera ficha escriba *General* en la propiedad *Caption (Título)* y escriba lo mismo en la propiedad *Key (Clave)*. Para insertar la segunda ficha, haga clic en el botón *Insert Tab (Insertar Ficha)* y escriba *Fuente* en la propiedad *Caption* y en la propiedad *Key*. Agregue las fichas *Estilos*, *Colores*, *Efectos* e *Imágenes* de la misma manera que agregó las dos primeras. Cuando haya terminado, haga clic en el botón *Aceptar*.
- Corra la aplicación.

Haga clic sobre cada una de las fichas. Podrá ver que no aparece nada cuando hacemos clic sobre las fichas. Esto es, porque no hemos creados los contenedores para cada ficha. En la siguiente sección aprenderá a determinar cada una de las fichas pulsadas por el usuario y luego mostrar los contenedores correspondientes a cada una de ellas.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio15-6** para el formulario, y **Ejercicio15-6** para el proyecto.

- 6.2.5.3 Cómo determinar la ficha pulsada por el usuario

Para saber qué ficha ha sido pulsada por el usuario usted debe consultar la propiedad **SelectedItem** del control TabStrip. Esto debe hacerlo en el evento **Click** del control TabStrip:

```
Private Sub TabStrip1_Click( )
```

```
    If TabStrip1.SelectedItem.Index = 1 Then 'Se hizo clic en la primera ficha.
    If TabStrip1.SelectedItem.Index = 2 Then 'Se hizo clic en la segunda ficha.
    If TabStrip1.SelectedItem.Index = 3 Then 'Se hizo clic en la tercera ficha.
```

```
End Sub
```



También podrá determinar la ficha pulsada leyendo la propiedad **Caption** de la ficha seleccionada:

```
Private Sub TabStrip1_Click( )  
    If TabStrip1.SelectedItem.Caption = "General" Then 'Se hizo clic en la ficha general.  
    If TabStrip1.SelectedItem.Caption = "Fuente" Then 'Se hizo clic en la ficha Fuente.  
End Sub
```

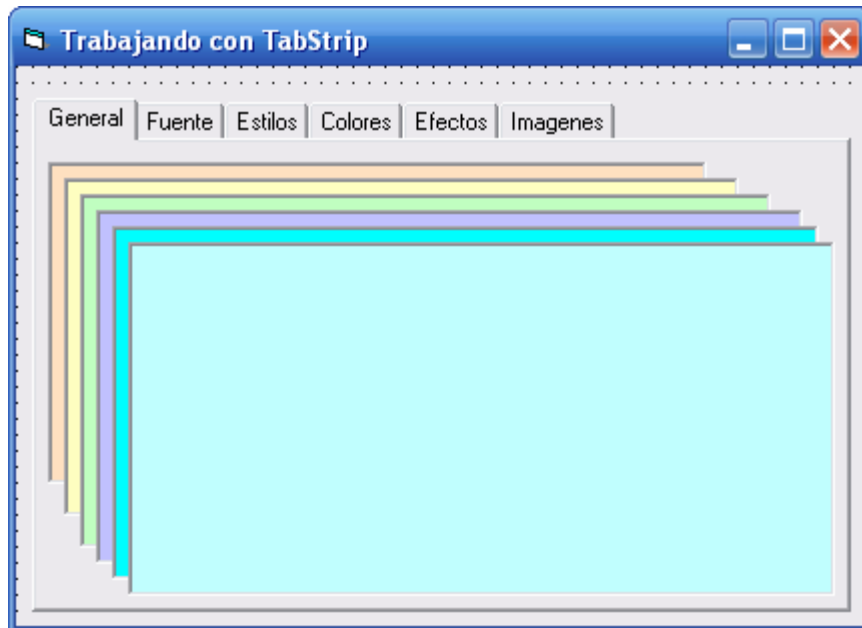
- 6.2.5.4 Preparación de los contenedores

Como el control TabStrip no es un contenedor no podrá introducir en su superficie otros controles hijos. Esta es una de las limitaciones más importantes de este control. Usted deberá crear contenedores (Picture o Frame) dentro del formulario. No es importante el lugar en el que introduzca estos contenedores dentro del formulario porque tendrá que desplazarlos y modificar su tamaño en tiempo de ejecución.

Supongamos que desea mostrar distintas secciones para cada ficha de nuestro ejercicio anterior cada vez que el usuario haga clic sobre una de ellas. Para esto, dibuje en el formulario seis contenedores Picture, teniendo en cuenta no dibujarlos uno dentro del otro, tal y como se muestra en la siguiente imagen:



- Seleccione el primer control PictureBox (el que está en el fondo) y en la propiedad Name escriba General. Seleccione el segundo PictureBox y en la propiedad Name escriba Fuente, en la del tercer PictureBox escriba Estilo, en la del cuarto PictureBox escriba Colores, en la del quinto Efectos y en la del sexto Imagenes.
- Ahora seleccione cada uno de los controles Picture y busque su propiedad **Visible** y póngala a False. Esto es para que estén invisibles cuando la aplicación se encuentre en ejecución.
- Asigne un color diferente a cada Picture en su propiedad BackColor para que note cuando se cambia de un Picture a otro al hacer clic sobre las fichas. Esto lo hago, no porque todos los contenedores deben tener colores distintos (todos deben ser siempre del mismo color), sino, para se de cuenta como cambiamos de un contenedor a otro al hacer clic en cada una de las fichas. Podrá colocar todo tipo de controles dentro de cada Picture.



- Dentro del evento **Load** del formulario escriba:

'Este bloque de código hace que todos los Picture se posicionen adecuadamente, dentro del control TabStrip.
'Este es un código estándar, lo podrá copiar en sus proyectos sin ningún problema.

Dim ctlControl **As** Object

On Error Resume Next

For Each ctlControl **In** Me.Controls 'Recorremos todos los controles del formulario.

'Determinamos si el control leído es un Picture.

If **TypeOf** ctlControl **Is** PictureBox **Then**

'Asignamos el tamaño y la posición adecuada al Picture leído.

ctlControl.Move TabStrip1.ClientLeft, TabStrip1.ClientTop, _
TabStrip1.ClientWidth, TabStrip1.ClientHeight

'Quitamos los bordes del Picture.

ctlControl.BorderStyle = 0

End If

ctlControl.ListIndex = -1
DoEvents

Next ctlControl

'Hacemos que el primer Picture sea el que aparezca cada vez que cargue el formulario.

General.Visible = **True**

- Dentro del evento **Click** del control TabStrip escriba:



```
'Este es un truco para no tener que crear un array de controles.
'El siguiente bloque de código determina la ficha que es pulsada,
'y muestra el PictureBox correspondiente a esa ficha.

Dim ctlControl As Object
On Error Resume Next

For Each ctlControl In Me.Controls 'Recorremos todos los controles del formulario.

    'Verificamos si el control leído es un PictureBox.
    If TypeOf ctlControl Is PictureBox Then

        'Verificamos si el nombre del Picture leído corresponde al Caption de la ficha.
        If ctlControl.Name = TabStrip1.SelectedItem.Caption Then
            'Si es verdadera la comparación anterior ponemos el PictureBox Visible.
            ctlControl.Visible = True

        Else
            'Si es falsa la comparación ponemos el Picture invisible.
            ctlControl.Visible = False

        End If

    End If

    ctlControl.ListIndex = -1
    DoEvents

Next ctlControl
```

- Corra la aplicación.

Haga clic en cada ficha. Podrá ver como cambiamos los contenedores cuando hacemos clic en cada una de las fichas. El código utilizado para hacer esto es un truco que consiste en poner en la propiedad **Name** de cada Picture el mismo título que tienen las fichas en su propiedad **Caption**. Si hace esto para los controles TabStrip que utilice en sus aplicaciones, podrá copiar el bloque de código anterior y obtendrá el mismo resultado sin importar la cantidad de fichas y Picture que agregue a su formulario.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio16-6** para el formulario, y **Ejercicio16-6** para el proyecto.

- 6.2.6 Ejercicios propuestos

- 1.- Crear una aplicación que tenga un control TreeView con varios países. Cada vez que el usuario haga clic sobre uno de los países contenidos en el TreeView se debe mostrar la capital en un MsgBox.
- 2.- Crear una aplicación que mediante un control TreeView cambiemos el color de fondo de un control PictureBox.
- 3.- Crear una aplicación que mediante un control TreeView cambiemos la imagen de un control PictureBox. El control TreeView debe tener diez opciones de imágenes para aplicarla al control PictureBox.
- 4.- Crear un control ListView con tres columnas. La primera y la segunda columna deben tener diez números y en la tercera columna mostrar la suma de las dos primeras.
- 5.- Crear una pantalla que simule un pequeño punto de venta donde se pueda agregar el código del artículo, la descripción, el precio y la cantidad comprada de cada artículo. El programa debe

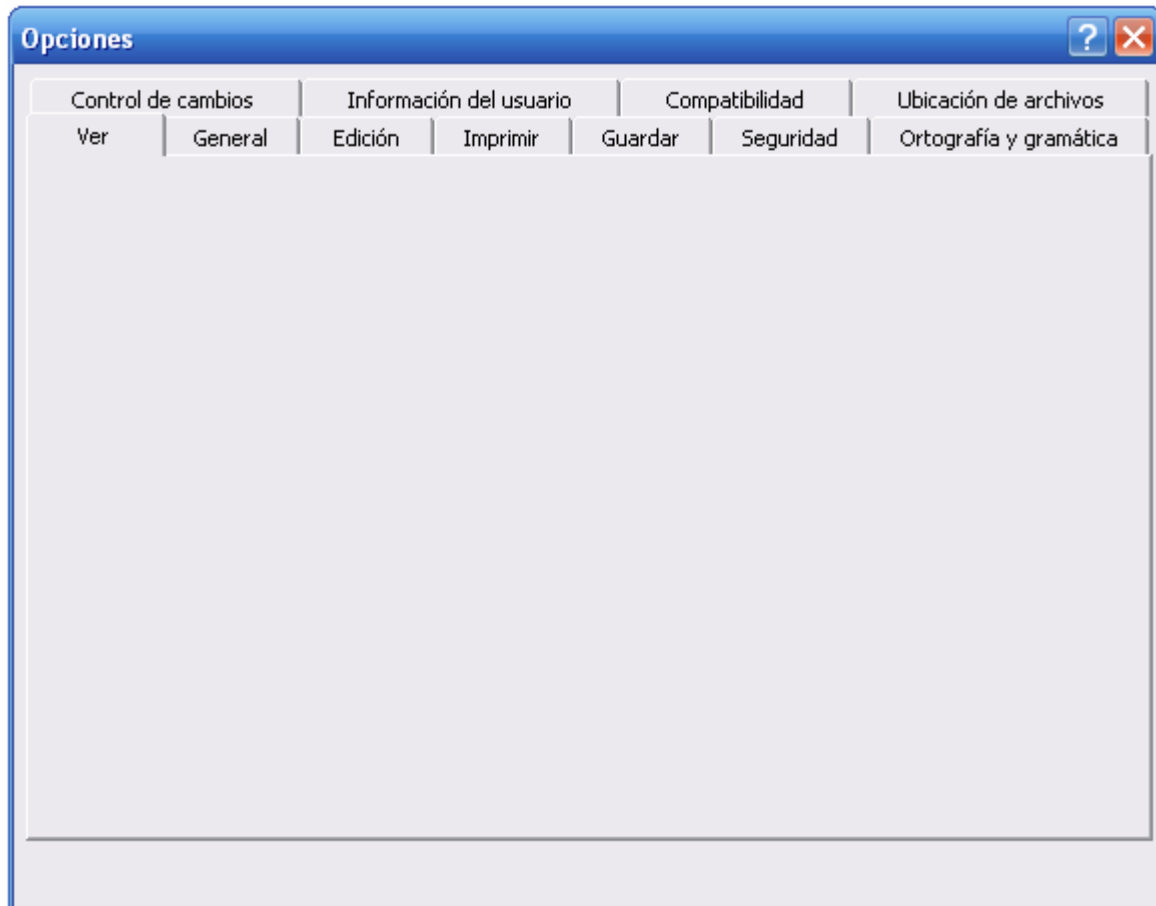


debe sumar la columna de los precios y mostrar en una etiqueta el total a pagar por el cliente.

6.- Utilizando el control ToolBar crear la siguiente barra de herramientas:



7.- Utilizar el control TabStrip para crear la siguiente pantalla:





CONTENIDO

- 7.1 Concepto de Archivo.**
- 7.2 Característica de los Archivos.**
- 7.3 Creación de un archivo aleatorio (sin formato) desde Visual Basic 6.0.**
- 7.4 Almacenamiento de información un archivo de texto.**
- 7.5 Lectura de un archivo aleatorio (sin formato).**
 - 7.5.1 Lectura de un archivo utilizando un bucle.
 - 7.5.2 Creación de un editor sencillo.
- 7.7 Archivos con estructura de base de datos.**
- 7.8 Crear un archivo con estructura de base de datos en Visual Basic 6.0.**
- 7.9 Ejercicios propuestos.**



7.1 CONCEPTO DE ARCHIVO

Un **archivo** o **fichero (file)** es una estructura de datos que reside en memoria secundaria, consistente en un conjunto de informaciones estructuradas en unidades de acceso denominadas registros, todos del mismo tipo y en número indeterminado.

Dentro de un archivo es posible almacenar cualquier tipo de información, ya sea *personalizada* por el usuario ó de modo aleatorio (sin formato). Cuando hablamos de *personalizada* nos referimos a que el usuario tiene la posibilidad de almacenar los datos de acuerdo a sus necesidades, ya sea agregando caracteres especiales, símbolos, etc., para facilitar su lectura.

El termino *modo aleatorio (sin formato)* se refiere al almacenamiento de los datos sin especificaciones especiales, es decir, sin aplicar ninguna alteración a la estructura de los datos. Por ejemplo, el almacenamiento del contenido de una caja de texto en un archivo de texto. Los archivos que poseen cierto tipo de estructuras especiales se les denomina archivos de bases de datos. Este será el caso de estudio del siguiente capítulo.

Un archivo puede estar en cualquier parte de un disco ya sea en una carpeta o en una subcarpeta. Comúnmente vienen representados por un icono y un nombre que permite diferenciarlos de los demás archivos del disco.

7.2 CARACTERÍSTICA DE LOS ARCHIVOS

Un archivo se podría representar gráficamente de la siguiente manera:

NombreArchivo.Extensión

NombreArchivo: Puede ser cualquier cadena de caracteres especificada por el usuario (programador).

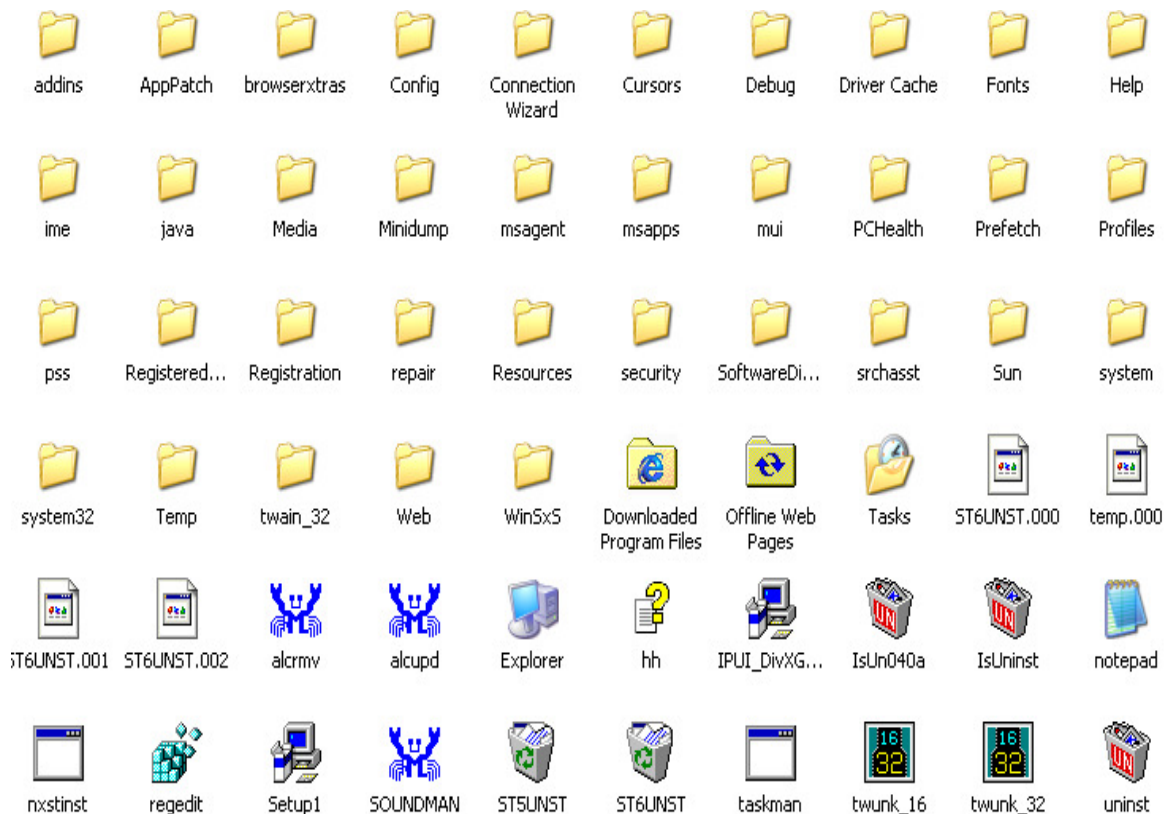
Extensión: Una extensión en un archivo se representa por tres letras que hacen referencia al tipo de archivo. Por ejemplo, la extensión para un archivo de texto sería (.TXT), para un archivo de datos (.DAT), para un archivo de documento (.DOC), para un archivo por lotes (.BAT), para un archivo ejecutable (.EXE), etc.

Un archivo en Windows se representa por un ICONO y debajo de este, el nombre del archivo sin la extensión. Cuando Windows conoce el tipo de archivo entonces muestra un ICONO que hace referencia al tipo de archivo y omite la presentación de la extensión y cuando no conoce el tipo de archivo agrega un ICONO blanco que  representa una aplicación ó archivo desconocido.

Para entender lo explicado anteriormente visualizare el contenido del directorio de Windows en mi disco duro para por observar la característica de los archivos que contiene esta carpeta.

Ejemplo:

Al visualizar mi carpeta de Windows aparecen los siguientes archivos y carpetas:



Si observa detenidamente que cada tipo de archivo posee un tipo de ICONO (imagen) distinto y que los archivos que Windows no conoce se presentan con un ICONO blanco con una ventana .

7.3 CREACIÓN DE UN ARCHIVO ALEATORIO (SIN FORMATO) DESDE VISUAL BASIC 6.0

Para crear un archivo desde Visual Basic disponemos de la sentencia **OPEN** seguida del modo **OUTPUT**. Existen otros modos como **APPEND** e **INPUT** que veremos más adelante.

El formato de esta sentencia es la siguiente:

OPEN Unidad.Carpeta.NombredelArchivo **FOR MODO AS** #numero_archivo_abierto

Donde:

OPEN: Es la sentencia de Visual Basic utilizada para crear un archivo cualquiera.

Unidad.Carpeta.NombredelArchivo: Todo esto representa un archivo con su unidad y su carpeta, es decir, el PATH completo.

Por ejemplo:

- A) Un archivo llamado CARLOS.DOC que se encuentre en el disco "C:" entonces su PATH sería el siguiente:

C:\ CARLOS.DOC

- B) Un archivo llamado JOSE.DOC que se encuentra en el disco "C:" en una carpeta llamada DOCUMENTOS entonces su PATH sería el siguiente:



C:\DOCUMENTOS\JOSE.DOC

- C) Un archivo llamado DATOS.TXT que se encuentre en el disco "C:" en una subcarpeta llamada INFORMACION que se encuentra en la carpeta PROGRAMA entonces su PATH sería el siguiente:

C:\PROGRAMA\INFORMACION\DATOS.TXT

FOR MODO: En este se especifica el tipo de acceso al archivo, es decir, si es para lectura, para escritura o para la creación de un archivo nuevo.

Los modos disponibles en Visual Basic son los siguientes:

- **OUTPUT:** este modo es utilizado para crear un archivo nuevo si este no existe, y si existe entonces lo sustituye por el nuevo archivo.
- **APPEND:** este modo es utilizado para agregar información al final de un archivo existente, si no existe entonces se crea.
- **INPUT:** este modo es utilizado para leer el contenido de un archivo almacenado en un disco.

#numero archivo abierto: En este se especifica un valor que representa el número del archivo abierto en la aplicación. Esto es porque usted puede abrir o crear dos o más archivos y se necesitaría un valor que permita llevar el control de estos archivos abiertos. Por ejemplo, si usted va a crear un archivo llamado CARTA.DOC en el disco "C:" y otro llamado ARCHIVOS.TXT necesariamente usted tendría que usar dos veces la sentencia OPEN e identificar cada una de ellas:

OPEN "C:\CARTA.DOC" FOR OUTPUT AS #1

OPEN "C:\ARCHIVOS.TXT" FOR OUTPUT AS #2

CLOSE #1

CLOSE #2

- La primera línea crea el archivo CARTA.DOC en el disco "C:"
- La segunda línea crea el archivo ARCHIVOS.TXT en el disco "C:"
- La tercera línea cierra el primer archivo abierto, en este caso CARTA.DOC. Cada vez que se abra un archivo con OPEN este debe ser cerrado con la sentencia CLOSE #numeroarchivo.
- La cuarta línea cierra el segundo archivo abierto, en este caso ARCHIVOS.TXT.

7.4 ALMACENAMIENTO DE INFORMACIÓN UN ARCHIVO DE TEXTO

Es lógico que al abrir un archivo para escritura debe de existir una sentencia que permita escribir en el, o más bien, llenar el archivo de datos. Para esto existe la sentencia **PRINT** que permite escribir datos línea por línea en el archivo.

Su formato es:



PRINT #numeroarchivo

Ejemplo:

```
Open "C:\CARTA.DOC" For Output As #1
Print #1, "Querido Padre:"
Print #1, "Le escribo para decirle que lo extraño mucho y que espero verlo pronto."
Print #1, "He hecho todo lo que me dijo y estoy estudiando muchísimo."
Print #1, "Me despido y saludos a todos por allá."
Print #1, "Att: Ernesto Pérez"
Close #1
```

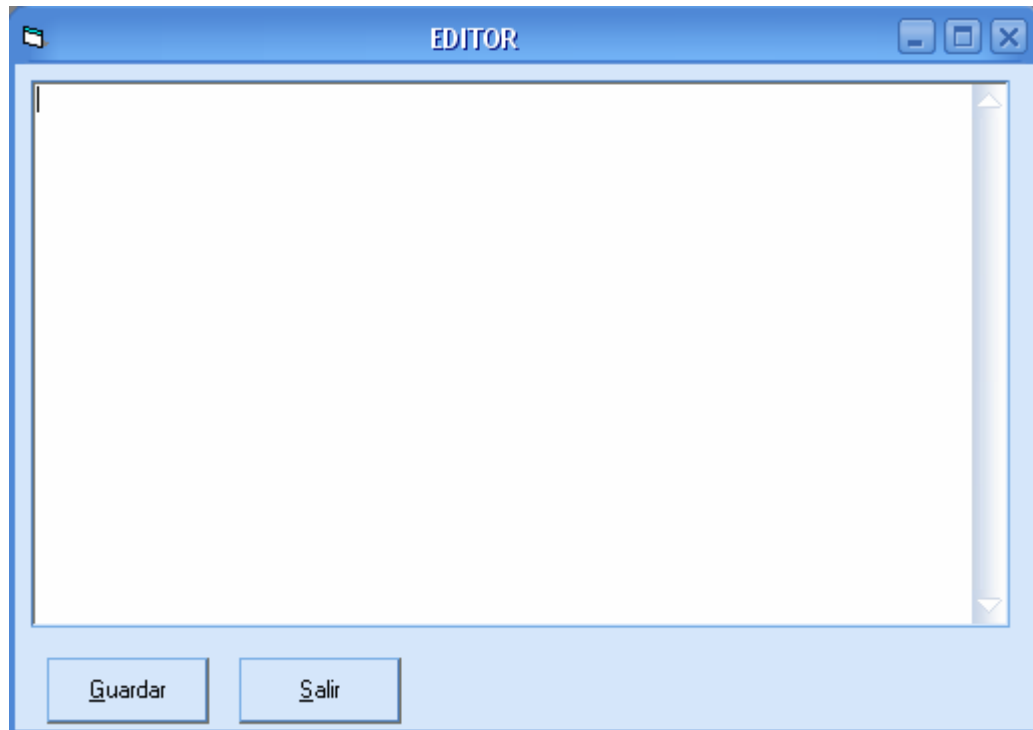
El bloque de código anterior permite crear un archivo llamado CARTA.DOC en el disco "C:" con cinco líneas de información que se especifican con la sentencia **Print**.

Ejercicio:

La siguiente aplicación almacena el contenido de una caja de texto en un archivo.

Para crear esta aplicación vamos a utilizar una caja de texto y un dos botones de comando, uno para guardar los datos de la caja de texto y otro para salir de la aplicación.

- Abra un nuevo proyecto.
- Diseñe la siguiente pantalla:



- Haga clic en la caja de texto y en la ventana propiedades establezca los siguientes valores:
 - a) En la propiedad **MultiLine** establezca **True**.
 - b) En la propiedad **ScrollBars** establezca el valor **2 – Vertical**.



- En el evento **Click** del botón **Guardar** escriba:

```
On Error GoTo ErrorDesconocido
```

```
Open "C:\DATOS.DOC" For Output As #1
```

```
Print #1, Text1.Text
```

```
Close #1
```

```
MsgBox ("Los datos han sido guardado.")
```

```
Exit Sub
```

```
ErrorDesconocido:
```

```
MsgBox ("Ha ocurrido un error al intentar guardar el archivo.")
```

- En el evento **Click** del botón **Salir** escriba:

```
End
```

- Corra la aplicación.
- Después que la aplicación este corriendo escriba algo en la caja de texto.
- Cuando halla escrito algo en la caja, haga clic en el botón guardar, y a continuación, aparecerá una ventana diciendo que “Los datos han sido guardado.”, esto es sino ocurre un error.
- Al hacer clic en el botón guardar, automáticamente se creará un archivo llamado DATOS.DOC en el disco “C:”.

PARA VER SI EL ARCHIVO SE HA CREADO REALMENTE EN EL DISCO “C:” HAGA LO SIGUIENTE:

- A) Haga doble clic en MY PC.
- B) Haga doble clic en Disco C:\
- C) Explore todos los archivos que aparecen en el disco “C:\” y verá que entre todos esos archivos se encuentra el que creamos desde Visual Basic llamado DATOS.DOC.

NOTA: El archivo aparecerá como documento de WORD porque lo hemos creado con la extensión “.DOC” que representan los archivos de WORD.

- Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio1-7** para el formulario, y **Ejercicio1-7** para el proyecto.

Explicación del programa línea por línea:

- La primera línea:

```
On Error GoTo ErrorDesconocido
```

Esta línea de código permite controlar cualquier error que ocurra a partir desde donde se escribe. En este caso la hemos utilizado para verificar si al grabar los datos ocurre algún error.

Traducida al castellano esta línea de código quiere decir lo siguiente:

SI OCURRE UN ERROR SALTA A LA LINEA LLAMADA ErrorDesconocido



- La segunda línea:

`Open "C:\DATOS.DOC" For Output As #1`

Esta línea permite crear el archivo llamado DATOS.DOC en el disco "C:".

Se ha utilizado el modo **Output** porque se sobreentiende que el archivo no existe, pero si este existe, entonces es remplazado con los nuevos datos.

Traducida al castellano esta línea quiere decir:

CREA EL ARCHIVO DATOS.DOC EN EL DISCO "C:" PARA ESCRITURA Y ASIGNE AL ARCHIVO EL NUMERO UNO EN MI APLICACIÓN.

- La tercera línea:

`Print #1, Text1.Text`

Ya se ha explicado que la sentencia **Print** se utiliza para escribir en el archivo. En este caso se ha utilizado para escribir el contenido de la caja de texto en el archivo.

Traducida al castellano esta línea quiere decir:

ESCRIBE EN EL ARCHIVO NUMERO UNO EL CONTENIDO DE LA CAJA DE TEXTO.

- La cuarta línea:

`Close #1`

Esta línea cierra un archivo abierto. Recuerde que cada vez que se abre un archivo ya sea para lectura o escritura este debe ser cerrado.

Traducida al castellano esta línea quiere decir:

CIERRA EL ARCHIVO NUMERO UNO.

- La quinta línea:

`MsgBox ("Los datos han sido guardado.")`

Esta línea muestra una ventana con el mensaje escrito entre los paréntesis y la comilla. En este caso hemos especificado un mensaje que informa al usuario si los datos se han grabado correctamente.

- La sexta línea:

`Exit Sub`

Esta línea de código permite separar un bloque de código con otro bloque de código. Se ha utilizado para separar las últimas dos líneas de código que muestran el mensaje de error en caso de que este ocurra.

Si esta línea de código no se escribe, entonces el mensaje de error aparecería aun los datos se hallan guardado correctamente, esto es, porque no habría nada que separe el código principal que almacena los datos, del código que muestra el error en caso de que ocurra.



Es lógico que el error solamente deba aparecer si ocurre algún error. Y debe de existir algo que separe un bloque de código de otro código y para esto se utiliza **Exit Sub**.

- La séptima línea:

ErrorDesconocido:

Esta línea de código se ha declarado como una etiqueta que va a permitir desde el punto de vista lógico asignarle un nombre a una línea.

Al establecer la etiqueta "ErrorDesconocido:" en una línea, es posible acceder al bloque de código que se encuentra después de la etiqueta utilizando la sentencia **Goto** como se especifico en la primera línea.

Esto es, porque es la única forma de acceder a la codificación que reside después de la sexta línea de código que corresponde a la codificación **Exit Sub**.

- La octava línea:

Msgbox("Ha ocurrido un error al intentar guardar el archivo.")

Muestra un mensaje diciendo que ha ocurrido un error al intentar guardar el archivo en el disco.

7.5 LECTURA DE UN ARCHIVO ALEATORIO (SIN FORMATO)

Es lógico que después de haber creado un archivo debe de existir una forma para leerlo.

Para leer un archivo que haya sido creado anteriormente se debe utilizar el método **linput** en vez de **Output** y utilizar la sentencia **Line Input** que permite leer línea por línea un archivo.

Su formato es:

Line Input #numeroarchivo, Variable

Donde:

#numeroarchivo: Es el número del archivo abierto como ya se había explicado anteriormente.

Variable: Representa una variable tipo cadena (String) o variante (Variant), declarada de antemano para almacenar el contenido de una línea del archivo abierto.

Ejemplo:

Si se quiere leer las primeras ocho líneas de un archivo de texto llamado **Carta.Doc** que se encuentre en el disco duro y luego almacenarla en una caja de texto con la propiedad **MultiLine** establecida a **True**, se haría de la siguiente manera:

Dim Linea1, Linea2, Linea3, Linea4, Linea5, Linea6, Linea7, Linea8 As String

Open "C:\CARTA.DOC" For Input As #1

Line Input #1, Linea1

Line Input #1, Linea2

Line Input #1, Linea3

Line Input #1, Linea4



Line Input #1, Linea6
 Line Input #1, Linea7
 Line Input #1, Linea8
 Close #1

```
Text1.Text = Linea1 & CHR(10) & CHR(13) & Linea2 & CHR(10) & CHR(13) & Linea3 & CHR(10) _
& CHR(13) & Linea4 & CHR(10) & CHR(13) & Linea5 & CHR(10) & CHR(13) & Linea6 & CHR(10) _
& CHR(13) & Linea7 & CHR(10) & CHR(13) & Linea8
```

NOTA: esas rayitas llamadas **UnderScore** deben de ponerse. Se utilizan para seguir una línea de código en Visual Basic cuando esta es demasiado grade. Si no utiliza el **UnderScore** entonces deberá de escribir la línea de código completa pero en una sola línea. Esto no es recomendable porque se haría incomodo leerla cuando sea necesario. Los **UnderScore** se ponen con un espacio en blanco, es decir, no pegado al texto que le queda al lado.

Explicación del programa anterior línea por línea.

- Primera línea:

Dim Linea1, Linea2, Linea3, Linea4, Linea5, Linea6, Linea7, Linea8 As String

Esta línea de código permite declarar ocho variables que luego serán utilizadas para almacenar las ocho primeras líneas del archivo.

Dim es una sentencia utilizada para declarar variables como se puedo notar en la línea de código. **As String** indica el tipo de datos que almacenarán cada variable, en este caso, se declararon como tipo CADENA.

As String significa: "COMO CADENA DE TEXTO O CARACTERES".

- Segunda Línea:

Open "C:\CARTA.DOC" For Input As #1

Esta línea de código permite abrir el archive CARTA.DOC en modo de lectura, por tal razón se utilizo el modo **Input** como se había explicado anteriormente.

Se había dicho que **Input** permite abrir el archivo en modo de lectura, es decir, para leer del archivo y no para escribir en él.

NOTA: el archivo debe de existir en el disco, de lo contrario daría un error al tratar de abrir un archivo que no existe. Con el modo **Output** y **Apeend** no ocurre este problema, porque si el archivo no existe el mismo se crea, pero con **Input** no ocurre lo mismo, sino que el archivo debe de existir o le daría un error.

- 4ta, 5ta, 6ta, 7ma, 8va, 9na, 10ma, 11va Línea:

Line Input #1, Linea1
 Line Input #1, Linea2
 Line Input #1, Linea3
 Line Input #1, Linea4
 Line Input #1, Linea5



Line Input #1, Linea6
Line Input #1, Linea7
Line Input #1, Linea8

Como se puede apreciar cada una de las líneas permite leer una línea de texto del archivo y se almacenan en la variable correspondiente.

- Décima segunda línea (12):

Close #1

Permite cerrar el archivo. Recuerde que cada vez que se abre un archivo este debe de ser cerrado.

- Ultimas líneas:

```
Text1.Text = Linea1 & CHR(10) & CHR(13) & Linea2 & CHR(10) & CHR(13) & Linea3 & CHR(10) _  
& CHR(13) & Linea4 & CHR(10) & CHR(13) & Linea5 & CHR(10) & CHR(13) & Linea6 & CHR(10) _  
& CHR(13) & Linea7 & CHR(10) & CHR(13) & Linea8
```

Permite almacenar el contenido de cada línea en la caja de texto.

El **& Ampersand** se utiliza para unir cada una de la línea.

CHR(10) & CHR(13) es una combinación especial que se utiliza para hacer un salto de línea cada vez que se pone una línea en la caja de texto. Si no se utiliza esta combinación entonces el contenido del archivo aparecería en una sola línea de la caja de texto y esto no es normal.

- 7.5.1 Lectura de un archivo utilizando un bucle

En ocasiones será muy necesario leer el archivo utilizando un bucle que permitirá leer a totalidad el archivo. Esto es porque no se conoce de antemano el total de líneas del archivo y aun conociéndola a veces son demasiadas líneas y sería anormal declarar tantas variables para leer el archivo.

Supóngase un archivo con 1000 mil líneas habría que declarar mil variables para almacenar cada línea del archivo, por tal razón de ahora en adelante solo utilizaremos bucles para la lectura de nuestros archivos de texto.

Si tenemos un archivo en el disco duro llamado CARTA.DOC la forma correcta de leerlo seria de la siguiente manera:

Dim Linea As String

Open "C:\CARTA.DOC" For Input As #1

While Not EOF(1)

Line Input #1, Linea

Text1.Text = Text1.Text & Linea & Chr(13) & Chr(10)

Wend

Close #1

Explicación del código anterior línea por línea:

- Primera línea:



Dim Linea As String

Esta línea declara una variable llamada "Linea" como tipo cadena. En esta variable se almacenará cada una de las líneas del archivo a medida que el bucle se repita.

- Segunda línea:

Open "C:\CARTA.DOC" For Input As #1

Esta línea abre el archivo "CARTA.DOC" ubicado en el disco "C:" para modo de lectura.

- Tercera línea:

While Not EOF(1)

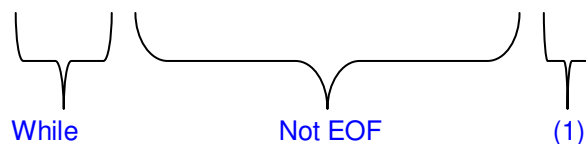
Si usted nunca ha trabajado con bucle esta línea de código le parecerá muy extraña. De todos modos tratare de explicarla.

Un bucle es una sentencia especial que permite repetir una determinada porción de código hasta que se cumpla una determinada condición. Existen dos tipos de bucles en Visual Basic que son muy utilizados el bucle **For** y el bucle **While** que es el utilizado en este caso.

La sentencia **While** permite repetir una porción de código "*Mientras*" no se cumpla una condición, es decir, hasta que la condición sea falsa.

El significado de la línea de código anterior es el siguiente:

Mientras No es el fin del archivo numero (Uno) Repite



While: significa en español "*Mientras*".

Not: significa en español "NO"

EOF: (End Of File) significa en español "Fin Del Archivo".

(1): Es el número asignado cada vez que usted abre un archivo. Recuerde este número lo especifica usted y puede ser cualquier número entero.

- Cuarta línea:

Line Input #1, Linea

Esta línea permite leer cada línea del archivo cada vez que se repite el bucle. La variable "Linea" cambia su valor cada vez que el bucle se repite. Cuando el bucle inicia en esta variable se almacena la primera línea del archivo, cuando el bucle se repite otra vez o sea a la segunda vez, entonces en esta variable se almacena la segunda línea del archivo y así sucesivamente hasta el fin del archivo.



- Quinta línea:

Text1.Text = Text1.Text & Linea & CHR(13) & CHR(10)

Esta línea de código almacena en una caja de texto el contenido de cada línea del archivo. Se especifica dos veces "Text1.Text = Text1.Text" para que se mantenga el contenido actual de la caja de texto.

El significado de la línea anterior es el siguiente:

CONTENIDO DE LA CAJA UNO = CONTENIDO DE LA CAJA UNO Y EL CONTENIDO
Text1.Text Text1.Text &

DE LA VARIABLE LINEA Y SALTA A LA OTRA LINEA DE LA CAJA DE TEXTO.
Linea & CHR(13) & CHR(10)

- Sexta línea:

Wend

Esta línea de código le pertenece al bucle y es el que indica el fin del bucle. Lo que se encuentre entre While y Wend es lo que se va a repetir. En conclusión cuando un bucle se inicia se debe de finalizar o debe de haber algo que indique hasta donde se va a repetir una porción de código. Para esto existe la sentencia Wend.

- Séptima línea:

Close #1

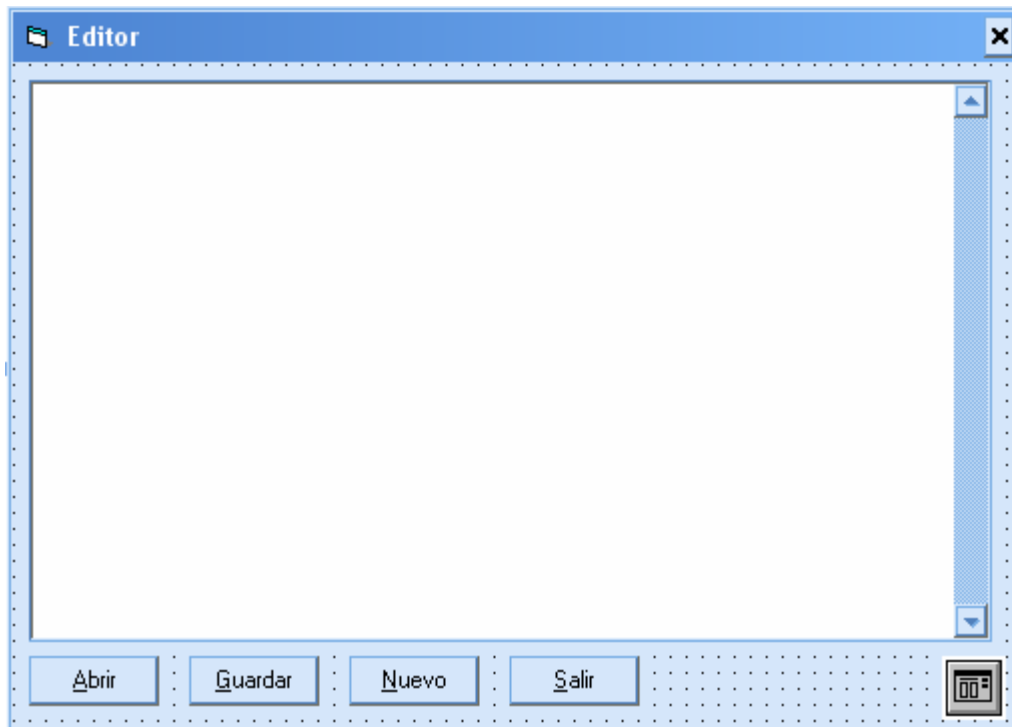
Cierra el archivo abierto que tiene el número uno.

- 7.5.2 Creación de un editor sencillo

Vamos a crear una aplicación que permite guardar y abrir un documento escrito en una caja de texto. Utilizaremos el control *Microsoft Common Dialog* para guardar y abrir los archivos.

Para crear nuestra aplicación siga los siguientes pasos:

- Inicie un nuevo proyecto en Visual Basic.
- Agregue el control *Microsoft Common Dialog* a la barra de controles. Para esto haga clic derecho en la barra de controles y de la ventana de Componentes seleccione *Microsoft Common Dialog Control 6.0 (SP3)* y, a continuación, haga clic en el botón *Aceptar*.
- Inserte el control Common Dialog  en cualquier parte del Formulario.
- Dibuje una caja de texto y cuatro botones de comando en la parte inferior de la caja. Tal y como se muestra en la imagen de la siguiente página...



- Establezca los siguientes valores en las propiedades de los controles utilizados:

Control	Propiedad	Valor
Form1	Caption	Editor
	BorderStyle	1- Fixed Single
	Height	5460
	Width	7635
Text1	Text	(Vació)
	MultiLine	True
	ScrollBars	2- Vertical
	Height	4215
	Width	7215
	Top	120
	Left	120
Command1	Caption	&Abrir
Command2	Caption	&Guardar
Command3	Caption	&Nuevo
Command4	Caption	&Salir

- En el evento **Click** del botón *Abrir* escriba el siguiente bloque de código:

```
Text1.Text = ""
On Error GoTo NoSeleccionoArchivo:
CommonDialog1.Filter = "Archivos de texto (TXT)" & Chr(32) & Chr(124) & "*.txt"
CommonDialog1.ShowOpen
```



```
Dim Linea As String
Open CommonDialog1.FileName For Input As #1
    While Not EOF(1)
        Line Input #1, Linea
        Text1.Text = Text1.Text & Linea & Chr(13) & Chr(10)
    Wend
Close #1
Exit Sub
NoSeleccionoArchivo:
```

Explicación del código anterior:

- Primera línea:

Text1.Text = ""

Permite limpiar la caja de texto. Esto es en caso de que anteriormente se encontrará algún archivo abierto. Es normal que si se va a mostrar el contenido de algún archivo el lugar donde se mostrará debe de estar limpio.

- Segunda línea:

On Error GoTo NoSeleccionoArchivo

Esta línea se ha declarado en caso de que ocurra un error al intentar abrir el archivo.

Esto es muy importante porque en ocasiones pueden ocurrir errores desconocidos y estos errores deben ser controlados desde la aplicación. Esta línea la he declarado principalmente porque siempre que no se selecciona algún archivo en el cuadro de dialogo abrir ocurrirá un error, es decir, cuando se haga clic en el botón Cancelar del cuadro de dialogo Abrir. No intente averiguar porque este error ocurre, simplemente especifique siempre esta línea de código cuando intente abrir un archivo.

- Tercera línea:

CommonDialog1.Filter = "Archivos de texto (TXT)" & Chr(32) & Chr(124) & "*.txt"

Esta línea es muy importante. Permite especificar el tipo de archivo que permitirá visualizar el cuadro de dialogo abrir.

La propiedad Filter te permite especificar el tipo de archivo que el cuadro de dialogo podrá visualizar a la ahora que este sea activado.

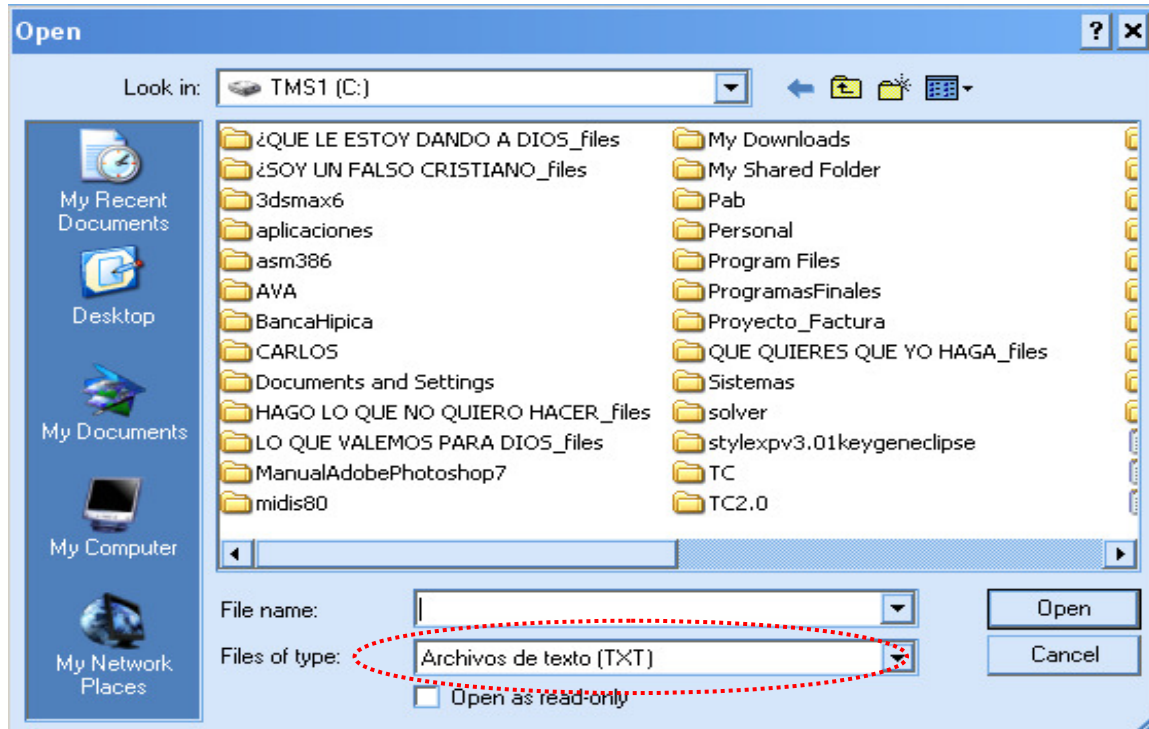
Estructura de la línea anterior:

CommonDialog1.Filter = "Archivos de texto (TXT)" & Chr(32) & Chr(124) & "*.txt"

Entre comillas se especifica el texto que aparecerá en <i>Tipo de archivo</i> del cuadro de diálogo Abrir.	Esta combinación permite obtener el carácter .	Extensión de los archivos que se mostrarán.
--	---	---



En este caso se ha especificado que el cuadro de dialogo Abrir solo visualice los tipos de archivos de texto. Si corre la aplicación y hace clic en el botón Abrir aparecerá la pantalla de Abrir con la siguiente configuración:



Puede observar que en la caja *Files of type* (Tipo de archivos) aparece el texto que especificamos en la propiedad Filter de control Common Dialog. Se pueden especificar más tipos de archivos pero esto lo veremos más adelante.

- Cuarta línea:

```
CommonDialog1.ShowOpen
```

Esta línea de código no tiene mucha novedad simplemente muestra el cuadro de dialogo abrir. El método ShowOpen significar “Mostrar Abrir”, es decir, “Mostrar el cuadro Abrir”.

Existen otros métodos como: ShowPrinter, ShowColor, ShowSave, ShowFont, ShowHelp, etc.

Estos otros métodos los veremos más adelante.

- Quinta línea:

[Dim Linea As String](#)

Esta línea de código declara una variable llamada “Linea” donde se almacenará cada línea de datos del archivo abierto.

- Sexta línea:

[Open](#) CommonDialog1.FileName [For Input As #1](#)



La única novedad que vemos en esta línea de código es "CommonDialog1.FileName". Antes especificábamos el archivo directamente pero en esta ocasión dejamos que la ventana de diálogo Abrir tome el control del nombre del archivo.

El archivo que se seleccione en la ventana de dialogo Abrir, este será el archivo que se muestre en el Editor.

La propiedad FileName almacena el PATH o la ruta completa del archivo seleccionado.

- Ultimas líneas:

```
While Not EOF(1)
    Line Input #1, Linea
    Text1.Text = Text1.Text & Linea & Chr(13) & Chr(10)
Wend
Close #1

Exit Sub
NoSeleccionoArchivo:
```

Estas ultimas líneas las hemos visto ya anteriormente que se utilizan para la lectura de un archivo completo utilizando un bucle. Más arriba ya habíamos trabajado con esto. La sentencia de bloqueo **Exit Sub** también la vimos anteriormente.

- En el evento **Click** del botón *Guardar* escriba:

```
On Error GoTo NoSeleccionoArchivo:
CommonDialog1.Filter = "Archivos de texto (TXT)" & Chr(32) & Chr(124) & "*.txt"

CommonDialog1.ShowSave

Open CommonDialog1.FileName For Output As #1
    Print #1, Text1.Text
Close #1

    MsgBox ("Archivo Guardado.")

Exit Sub
NoSeleccionoArchivo:
```

Explicación del código anterior:

Lo único nuevo que vemos en este bloque de código es el método ShowSave que permite mostrar el cuadro de dialogo Guardar.

En esta ocasión utilizamos este método para guardar el archivo en el disco duro. ShowSave significa "Mostrar Guardar", es decir, "Mostrar el cuadro de diálogo Guardar".

Lo demás lo hemos visto anteriormente. Por ejemplo, utilizamos en esta ocasión el método Output para guardar y especificamos CommonDialog1.FileName en vez de especificar un nombre fijo o un nombre constante. Ya explique eso anteriormente.

- En el evento **Click** del botón de comando *Nuevo* escriba:



```
Text1.Text = ""
Text1.SetFocus
```

Explicación del código anterior:

375

- Primera línea:

La primera línea quita cualquier texto que se halla mostrado o escrito en la caja de texto.

- Segunda línea:

Hace que la caja de texto reciba el enfoque, es decir, que el cursor se posiciona en la caja.

▪ En el evento **Click** del botón de comando *Salir* escriba:

```
'Sale de la aplicación.
End
```

▪ Corra la aplicación y verá que tiene un pequeño Editor. Puede perfeccionarlo agregando negrita, cursiva, subrayado, tipo de letra, justificaciones de texto, etc.

▪ Detenga la aplicación y guarde el ejercicio con los nombres **FormEjercicio2-7** para el formulario, y **Ejercicio2-7** para el proyecto.

7.7 ARCHIVOS CON ESTRUCTURA DE BASE DE DATOS

Una base de datos es un conjunto de datos organizados y relacionados lógicamente entre sí. Una base de datos se podría considerar como el almacenamiento organizado de los datos proporcionados por el programa.

Los archivos con estructura de bases de datos sirven para almacenar de forma personalizada datos de un programa que luego lo podrá utilizar para ejecutar operaciones internas en su aplicación, por ejemplo, podrá utilizar un archivo con estructura de base de datos para almacenar la configuración de su programa con el objetivo de que esta quede almacenada cada vez que cierre el programa y posteriormente leer este archivo cuando la aplicación vuelva a ser ejecutada. No pretenda almacenar en un archivo de texto datos importantes de una empresa, para esto cree bases de datos con sistemas SGBD profesionales como SQL, ORACLE, ACCESS, etc. En el siguiente capítulo tratamos este asunto.

Una base de datos esta compuesta de filas y columnas que físicamente representa una tabla. Cada columna representa un **Campo** en la base de datos y cada Fila un **Registro**. Un campo se define como la unidad de información más pequeña de la base de datos que tiene significado y un **Registro** se define como la colección de todos los campos de una base de datos. Toda lo que se escribe en dicha base de datos se denomina **Datos** y juegan un papel muy importante en la base de datos.

La estructura de una base de datos es la siguiente:

Campos				Registros
Nombre	Apellido	Teléfono	Dirección	
Carlos	Rodríguez	809-699-5858	Carretera Mella KM 8 ½ (Mandinga)	
Pablo	Bucarely	809-585-6325	Urb. Ralma. Calle #5	
Nelson	Pérez	809-755-2352	Urb. Los Mina. Res. Catanga	



La base de datos anterior se podría considerar como una estructura que representa una agenda telefónica donde los datos a almacenar son: Nombre, Apellido, Teléfono y Dirección.

Todos estos datos son de un mismo tipo y deben ser almacenados en un mismo archivo siguiendo una estructura organizada tal y como se muestra en la tabla anterior. Por ejemplo, en este caso el primer registro "Carlos Rodríguez 809-699-5858 Carretera Mella KM 8 ½ (Mandinga)" sería la primera línea del archivo de la base de datos, el segundo registro sería la segunda línea del archivo, el tercer registro sería la tercera línea del archivo y así sucesivamente.

Si almacenamos estos datos en un archivo deberíamos de almacenarlo de la siguiente forma:

```
Carlos;Rodríguez;809-699-5858;Carretera Mella KM 8 ½ (Mandinga)
Pablo;Bucarelly;809-585-6325;Urb. Palma. Calle #5
Nelson;Pérez;809-755-2352;Urb. Los Mina. Res. Catanga
```

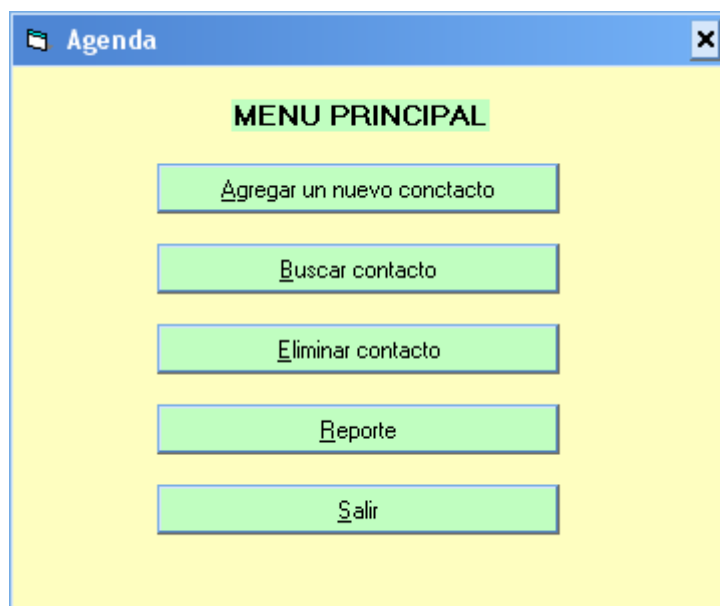
Se ha utilizado un punto y coma (;) para separar cada uno de los campos de la base de datos. Cuando almacenemos los registros en una base de datos deberemos agregar al final de cada campo un punto y coma (;) que nos va a permitir el final de cada campo de la base de datos. Se puede utilizar otro carácter pero el más utilizado siempre ha sido el punto y coma (;).

7.8 CREAR UN ARCHIVO CON ESTRUCTURA DE BASE DE DATOS EN VISUAL BASIC 6.0

Crear una base de datos en Visual Basic no implica utilizar sentencias especiales para el almacenamiento de los datos, sino, que implica utilizar otros métodos y algoritmos para almacenar y leer los datos.

Vamos a utilizar las mismas sentencias Open, Line Input, Print, Output, Input y Append ya vistas anteriormente. Ahora crearemos una pequeña agenda telefónica que le permitirá aprender a crear archivos con estructura de base de datos.

- Cree una carpeta llamada **Agenda** en el disco duro para almacenar nuestro proyecto y nuestra base de datos.
- Abra un nuevo proyecto y dibuje el siguiente entorno:





- El primer botón servirá para agregar un nuevo contacto o una nueva persona.
 - El segundo botón permitirá buscar algún contacto, ya sea por nombre o por el número de teléfono.
 - El tercer botón será utilizado para eliminar un contacto de la base de datos.
 - El cuarto botón se utilizará para obtener una lista de todos los contactos que han sido agregado a la base de datos.
 - El quinto botón será utilizado para salir de la aplicación.
- Establezca los siguientes valores a los controles del formulario anterior para que quede como se muestra en la imagen anterior:

Control	Propiedad	Valor
Form1	Caption	Agenda
	BorderStyle	1- Fixed Single
	Height	4545
	Width	5430
	StartPosition	1 – CenterOwner
	BackColor	{ EMBED PBrush }
Label1	Caption	MENU PRINCIPAL
	AutoSize	True
	Left	1635
	Top	240
	Font	Tamaño 10 y Negrita.
	BackColor	{ EMBED PBrush }
Command1	Caption	&Agregar un nuevo contacto
	Height	375
	Width	3015
	Left	1080
	Top	720
	Style	1- Graphical
Command2	BackColor	{ EMBED PBrush }
	Caption	&Buscar contacto
	Height	375
	Width	3015
	Left	1080
	Top	1320
Command3	Style	1- Graphical
	BackColor	{ EMBED PBrush }
	Caption	&Eliminar contacto
	Height	375
	Width	3015
	Left	1080
Command4	Top	1920
	Style	1- Graphical
	BackColor	{ EMBED PBrush }
	Caption	&Reporte
	Height	375
	Width	3015
	Left	1080
	Top	2520
	Style	1- Graphical
	BackColor	{ EMBED PBrush }

Ing. Carlos Manuel Rodríguez Bucarely



Command4	Caption Height Width Left Top Style BackColor	&Reporte 375 3015 1080 2520 1- Graphical El primer verde.
Command5	Caption Height Width Left Top Style BackColor	&Salir 375 3015 1080 3120 1- Graphical El primer verde.

- Ahora vamos a agregar un nuevo formulario (Form2) para la primera opción del menú principal.
- Haga clic en el menú **Project** y luego haga clic en la opción **Add Form**. En el cuadro de dialogo que aparece haga clic en el botón Open (abrir).
- Ahora aparecerá un formulario en blanco. En este formulario pondremos todo lo necesario para agregar un nuevo contacto en la base de datos.
- Ahora dibuje el siguiente entorno en el nuevo formulario (Form2):

- El primer botón será utilizado para guardar los datos del nuevo contacto.
 - El segundo botón será utilizado para limpiar las cajas de texto y permitir agregar un nuevo contacto.
 - El tercer botón cerrará la ventana de Agregar nuevo contacto.
- Establezca los siguientes valores en los controles del nuevo formulario:



CONTROL	PROPIEDAD	VALOR
Form2	Caption	Agregar nuevo contacto
	BackColor	El primer amarillo.
	BorderStyle	1 – Fixed Single
	ShowInTaskBar	False
	Height	4005
	Width	5445
	StartPosition	1 - CenterOwner
Label1	AutoSize	True
	Caption	INTRODUZCA LOS DATOS DEL NUEVO CONTACTO
	BackColor	El primer verde.
	Left	120
	Top	240
	Font	Tamaño 8 y Negrita.
Label2	AutoSize	True
	Caption	Nombre
	BackStyle	0 – Transparent
	Font	Tamaño 8 y Negrita.
	Left	120
	Top	720
Label3	AutoSize	True
	Caption	Apellido
	BackStyle	0 – Transparent
	Font	Tamaño 8 y Negrita.
	Left	120
	Top	1200
Label4	AutoSize	True
	Caption	Teléfono
	BackStyle	0 – Transparent
	Font	Tamaño 8 y Negrita.
	Left	120
	Top	1680
Label5	AutoSize	True
	Caption	Dirección
	BackStyle	0 – Transparent
	Font	Tamaño 8 y Negrita.
	Left	120
	Top	2160
Text1	Text	(Vacío)
	Appearance	0 – Flat
	Height	285
	Width	3615
	Left	1080
	Top	720
Text2	Text	(Vacío)
	Appearance	0 – Flat
	Height	285
	Width	3615
	Left	1080
	Top	1200



Text3	Text	(Vació)
	Appearance	0 – Flat
	Height	285
	Width	2055
	Left	1080
	Top	1680
Text4	Text	(Vació)
	Appearance	0 – Flat
	Height	285
	Width	3975
	Left	1080
	Top	2160
Command1	Caption	&Guardar
	Height	375
	Width	855
	Left	240
	Top	3000
Command2	Caption	&Nuevo
	Height	375
	Width	855
	Left	1320
	Top	3000
Command3	Caption	&Cancelar
	Height	375
	Width	855
	Left	4200
	Top	3000

- Active o seleccione el primer formulario (Form1) de la ventana de Proyectos (Project) haciendo doble clic sobre el mismo.
- Ahora haga clic en el primer botón “Agregar un nuevo contacto” y escriba en el evento **Click** la siguiente línea de código:

```
Form2.Show 1, Me
```

La línea de código anterior permite mostrar el segundo formulario. El método Show permite mostrar un formulario en la pantalla. El “1, Me” es un método utilizado para que el formulario que se muestra sea el único activo en la pantalla, es decir, hasta que este no sea cerrado no se podrá activar ninguna otra opción del menú principal.

- Active el segundo formulario (Form2) de la ventana de Proyecto (Project) haciendo doble clic sobre el mismo.
- Ahora vamos a codificar el botón de Guardar:
- En el evento **Click** del botón *Guardar* escriba lo siguiente:

```
Open "C:\AGENDA\Agenda.dat" For Append As #1
```

```
Print #1, Text1.Text & ";" & Text2.Text & ";" & Text3.Text & ";" & Text4.Text
```

```
Close #1
```

```
Text1.Text = ""
```

```
Text2.Text = ""
```



```
Text3.Text = ""  
Text4.Text = ""  
  
Text1.SetFocus  
MsgBox ("El nuevo contacto ha sido agregado.")
```

Explicación del código anterior:

- Primera línea:

`Open "C:\AGENDA\Agenda.dat" For Append As #1`

En esta línea de código la única novedad es el método **Append** que permite agregar información al final del archivo y si este no existe entonces se crea. No utilizamos el método **Output** por la razón de que este método crea nuevamente el archivo y elimina los datos que tenía anteriormente para sustituirlo con los nuevos datos.

- Segunda línea:

`Print #1, Text1.Text & ";" & Text2.Text & ";" & Text3.Text & ";" & Text4.Text`

Esta línea permite escribir en una línea del archivo el contenido de cada unas de las cajas de texto separado con un punto y coma (;), como habíamos explicado anteriormente.

- Últimas líneas:

```
Close #1  
  
Text1.Text = ""  
Text2.Text = ""  
Text3.Text = ""  
Text4.Text = ""  
  
Text1.SetFocus  
MsgBox ("El nuevo contacto ha sido agregado.")
```

En estas últimas líneas no hay nada nuevo. Se cierra el archivo, se limpian cada unas de las cajas de texto, se hace que el cursor se posicione en la primera caja de texto y se muestra un mensaje.

- Corra la aplicación y, a continuación, haga clic en el botón *Agregar un nuevo contacto*.

Debe de aparecer el segundo formulario (Form2). Bien agregue un contacto y haga clic en el botón Guardar. Inmediatamente este registro se agrega en la base de datos. Más adelante veremos como leer estos registros.

- Detenga la aplicación.
- Haga doble clic en el botón *Nuevo* del segundo formulario (Form2) y escriba lo siguiente:

```
Text1.Text = ""  
Text2.Text = ""  
Text3.Text = ""  
Text4.Text = ""  
  
Text1.SetFocus
```



Explicación del código anterior:

Ninguna novedad. Limpia todas las cajas de texto y hace que el cursor se posicione en la primera caja.

- En el evento **Click** del botón *Cancelar* escriba:

```
Unload Me
```

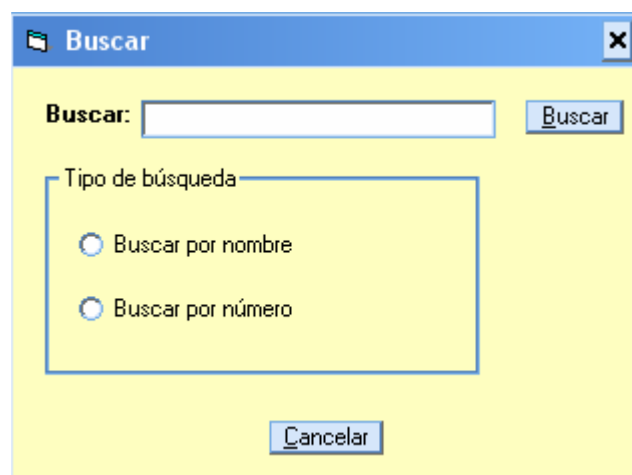
Explicación del código anterior:

```
Unload Me
```

Esta línea de código permite cerrar el formulario y no la aplicación completa.

El significado de esta línea al castellano es: "DESCARGADME".

- Active el primer formulario desde la ventana de Proyectos (Project) haciendo doble clic sobre el mismo.
- Ahora vamos a crear un nuevo formulario que será el formulario de la segunda opción del menú principal. En este formulario agregaremos todo lo necesario para buscar un contacto, ya sea por número telefónico o por nombre.
- Agregue un nuevo formulario haciendo clic en el menú Project y a continuación, en Add Form. En el cuadro de dialogo que aparece haga clic en el botón Open.
- Aparecerá un nuevo formulario (Form3).
- Dibuje un entorno similar al que se muestra en la imagen siguiente en un tercer formulario (Form3). Hágalo tal y como se ve, no daré los valores de los controles como lo había echo anteriormente.



- En el evento **Click** del botón de comando *Buscar* escriba el siguiente bloque de código:

```
If Len(Trim(Text1.Text)) = 0 Then
    MsgBox ("Debe escribir algo en la caja de texto.")
    Text1.SetFocus
Elseif Option1.Value = False And Option2.Value = False Then
```




```

    MsgBox ("Debe seleccionar el tipo de búsqueda.")
Else
On Error GoTo NOarchivo:
Dim Linea As String
Dim Campo1, Campo2, Campo3, Campo4 As String
Dim Posicion1, Posicion2, Posicion3 As Integer
Dim Encontro As Integer
Encontro = 0

Open "C:\AGENDA\Agenda.dat" For Input As #1
While Not EOF(1)
    Line Input #1, Linea
    Posicion1 = InStr(1, Linea, ";", vbTextCompare)
    Posicion2 = InStr(Posicion1 + 1, Linea, ";", vbTextCompare)
    Posicion3 = InStr(Posicion2 + 1, Linea, ";", vbTextCompare)
    Campo1 = Mid(Linea, 1, Posicion1 - 1)
    Campo2 = Mid(Linea, Posicion1 + 1, Posicion2 - 1 - Posicion1)
    Campo3 = Mid(Linea, Posicion2 + 1, Posicion3 - 1 - Posicion2)
    Campo4 = Mid(Linea, Posicion3 + 1, Len(Linea))

    If UCase(Campo1) = UCase(Text1.Text) And Option1.Value = True Then
        MsgBox ("Nombre: " & Campo1 & " Apellido: " & _
            Campo2 & " Teléfono: " & Campo3 & " Dirección: " & Campo4)
        Encontro = 1
    End If

    If Campo3 = Text1.Text And Option2.Value = True Then
        MsgBox ("Nombre: " & Campo1 & " Apellido: " & _
            Campo2 & " Teléfono: " & Campo3 & " Dirección: " & Campo4)
        Encontro = 1
    End If

Wend

If Encontro = 0 Then
    MsgBox ("No se ha encontrado el contacto especificado.")
End If

Close #1

End If

Exit Sub

NOarchivo:
MsgBox ("La base de datos no existe.")

```

Explicación del bloque de código anterior:

- Las primeras tres líneas:



```
If Len(Trim(Text1.Text)) = 0 Then
    MsgBox ("Debe escribir algo en la caja de texto.")
    Text1.SetFocus
```

Este bloque de código permite verificar si la caja de texto se deja vacía, en caso de que se deje vacía entonces se muestra un mensaje y el cursor se envía a la caja de texto.

La sentencia **Len** permite obtener la longitud de una cadena de caracteres, en este caso la longitud de la caja de texto. La sentencia **Trim** permite eliminar los espacios en blanco que posiblemente se escriban al final de la caja de texto.

La traducción del código anterior sería la siguiente:

SI LA LONGITUD DE LA CAJA DE TEXTO ES IGUAL A CERO ENTONCES
MUESTRA EL MENSAJE ("Debe escribir algo en la caja de texto")
HAS QUE LA CAJA DE TEXTO RESIVA EL ENFOQUE

- Línea 4, 5 y 6:

```
Elseif Option1.Value = False And Option2.Value = False Then
    MsgBox ("Debe seleccionar el tipo de búsqueda.")
Else
```

Este bloque de código permite verificar si se selecciona una de las opciones del tipo de búsqueda. La propiedad **Value** permite verificar si el botón de opción esta seleccionado o no. Cando tiene el valor *False* indica que el botón no esta seleccionado.

La traducción del bloque anterior es la siguiente:

SI PRIMERA OPCION = no seleccionada Y SEGUNDA OPCION = no seleccionada ENTONCES
MUESTRA EL MENSAJE ("Debe seleccionar el tipo de búsqueda.")

DE LO CONTRARIO

El operador lógico And permite enlazar dos expresiones. Si ambas expresiones se cumplen, entonces, se ejecutan las líneas de códigos de más abajo.

La cláusula Else (De lo contrario) se ha utilizado para tomar una decisión en caso de que no se cumpla la condición, es decir, en caso de que se seleccione alguna de las opciones. Si una da las opciones se selecciona, entonces, toda la codificación que se encuentra debajo de la cláusula Else se ejecutará.

- Línea 7:

On Error GoTo NOarchivo:

Esta línea de código activa el detector de errores. Si ocurre algún error en la apertura del archivo, entonces, se ejecuta la codificación que se encuentra debajo en la etiqueta NOarchivo.

- Líneas 8, 9, 10, 11 y 12:

```
Dim Linea As String
Dim Campo1, Campo2, Campo3, Campo4 As String
Dim Posicion1, Posicion2, Posicion3 As Integer
```



```
Dim Encontro As Integer
Encontro = 0
```

En esta sección se declararon todas las variables necesarias para proceder con la codificación.

La primera variable Linea se utilizará para almacenar cada unas de las líneas de texto del documento. Las variables Campo1, Campo2, Campo3, Campo4 son variables de tipo cadena donde se almacenarán temporalmente el contenido de cada campo de la base de datos. Las variables Posicion1, Posicion2, Posicion3 son variables de tipo entero, donde se almacenarán las posiciones de cada uno de los punto y coma (;) que utilizamos para separar los campos. Tres posiciones porque utilizamos solamente tres punto y coma.

La variable Encontro como tipo entero, donde se almacenará un valor que indicará si se encontró o no la búsqueda realizada. Si la variable tiene el valor cero entonces no se encontró ningún elemento y si tiene el valor uno entonces se encontró el elemento.

Encontro = 0 se ha especificado para darle un valor inicial a la variable.

- Líneas 13, 14, 15:

```
Open "C:\AGENDA\Agenda.dat" For Input As #1
```

```
While Not EOF(1)
```

```
Line Input #1, Linea
```

Nada que no haya visto. Primero se abre el archivo para modo de lectura, segundo se inicia el bucle y por ultimo se lee cada línea del archivo y se almacena en la variable Linea.

- Líneas 16, 17, 18:

```
Posicion1 = InStr(1, Linea, ";", vbTextCompare)
```

```
Posicion2 = InStr(Posicion1 + 1, Linea, ";", vbTextCompare)
```

```
Posicion3 = InStr(Posicion2 + 1, Linea, ";", vbTextCompare)
```

Posiblemente aquí encuentre muchas cosas extrañas pero trataré de explicarlo con algunos ejemplos.

Primeramente empezaré definiendo la función de la sentencia **InStr**. Esta sentencia permite obtener la posición de un carácter especificado en la cadena de caracteres, es decir, te devuelve el número de la posición donde se encuentra ese carácter en la cadena de texto.

Ejemplo:

Carlos;Manuel;809-589-5858;Urb. Ralma, Calle #4

Puede observar que utilicé tres punto y coma (;) para separar los tres campos (nombre, apellido, teléfono y dirección). Esas posiciones de esos tres punto y coma lo almacenaré en cada una de las variables ya vistas anteriormente.

Por ejemplo, la posición del primer punto y coma (;) lo almacenaré en la variable Posicion1, al posición del segundo punto y coma (;) lo almacenaré en la variable Posicion2 y la posición del tercer punto y coma (;) lo almacenaré en la variable Posicion3.



En el ejemplo anterior las posiciones que se almacenarían en las variables son las siguientes:

Posicion1 = 7
Posicion2 = 14
Posicion3 = 27

Esto es por lo siguiente:

C a r l o s ; M a n u e l ; 8 0 9 - 5 8 9 - 5 8 5 8 ; U r b . R a l m a , C a l l e # 4
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37

Cada carácter tiene una posición en la cadena. Estas posiciones que hemos leído serán de mucha utilidad a la hora de leer los campos de la base de datos. Eso lo veremos a continuación.

El formato de la sentencia **InStr** es el siguiente:

InStr(Valor_Inicial, Cadena_Principal, Cadena_a_Buscar, Tipo_de_búsqueda)

Donde:

Valor_Inicial: Es un número de una posición de algún carácter de la cadena de texto principal. Por ejemplo, si el valor inicial es 1 entonces la búsqueda comienza desde el principio de la cadena, es decir, desde el primer carácter.

Cadena_Principal: Representa la cadena de caracteres en donde se realizará la búsqueda.

Cadena_a_Buscar: Representa la cadena a buscar.

Tipo_de_búsqueda: Es una constante que representa el tipo de búsqueda. Se ha especificado la constante **vbTextCompare** que indica que la búsqueda que se realizará es de tipo texto.

a) Posicion1 = InStr(1, Linea, ";", vbTextCompare)

Esta línea busca el primer punto y coma que aparece en el registro. La búsqueda se realiza a partir del primer carácter, por eso se especifica el número 1.

b) Posicion2 = InStr(Posicion1 + 1, Linea, ";", vbTextCompare)

Esta línea busca el segundo punto y coma que aparece en el registro. La búsqueda se realiza a partir de la posición del primer punto y coma, por eso se especifica **Posicion1 + 1**.

c) Posicion3 = InStr(Posicion2 + 1, Linea, ";", vbTextCompare)

Esta línea busca el tercer punto y coma que aparece en el registro. La búsqueda se realiza a partir de la posición del segundo punto y coma, por eso se especifica **Posicion2 + 1**.

- Líneas 19, 20, 21 y 22:

Campo1 = Mid(Linea, 1, Posicion1 - 1)

Campo2 = Mid(Linea, Posicion1 + 1, Posicion2 - 1 - Posicion1)

Campo3 = Mid(Linea, Posicion2 + 1, Posicion3 - 1 - Posicion2)

Campo4 = Mid(Linea, Posicion3 + 1, Len(Linea))



Estas cuatro líneas de código permiten leer cada uno de los campos de la base de datos.

Antes de explicar cada línea de código, vamos a definir la sentencia **Mid**.

La sentencia **Mid** permite copiar una cadena de caracteres de otra cadena de caracteres, llamada cadena principal. En esta función se debe especificar la cadena de donde se hará la copia que es la cadena principal. Se debe especificar también la posición en la cadena desde donde se quiere iniciar la copia y por ultimo, se debe especificar la cantidad de caracteres a copiar.

Su formato es:

Mid (cadena_principal, posición_inicial, cantidad_de_caracteres_a_copiar)

a) Campo1 = Mid(Linea, 1, Posicion1 - 1)

Esta línea de código lee el campo "Nombre". La variable *Linea* es la variable que tiene el registro completo, o más bien, la cadena principal. El número uno, indica que la copia iniciará desde el primer carácter de la cadena principal. **Posicion1 - 1** indica la cantidad de caracteres a copiar, es decir, se copiarán todos los caracteres hasta el primer punto y coma (;). Recuerde que la variable **Posicion1** tiene la posición del primer punto y coma. El menos uno -1 se especifico para que en la copia no se incluya el punto y coma.

b) Campo2 = Mid(Linea, Posicion1 + 1, Posicion2 - 1 - Posicion1)

Esta línea de código lee el campo "Apellido". **Posicion1 + 1** indica que la copia iniciará después del primer punto y coma (;), es decir, a partir de la posición del primer punto y coma (;). **Posicion2 - 1 - Posicion1** indica la cantidad de caracteres a copiar. Se lo explicaré gráficamente:

C a r l o s ; M a n u e l ; 8 0 9 - 5 8 9 - 5 8 5 8 ; U r b . R a l m a , C a l l e # 4
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37

Campo2 = Mid(Linea, 8, 14 - 1 - 7)

El 8 es por Posicion1 + 1 que es la posición del primer punto y coma (;).
 El 14 es por Posicion2 que es la posición del segundo punto y coma (;).
 El -1 es para que en la copia no se incluya el primer punto y coma (;).

-Posicion1 es la posición del primer punto y coma que se le resta a la segunda posición menos uno, esto le dará la cantidad de caracteres del segundo campo. $14 - 1 - 7 = 6$ que es la longitud del campo Apellido.

c) Campo3 = Mid(Linea, Posicion2 + 1, Posicion3 - 1 - Posicion2)

Esta línea lee el campo "Teléfono". Se aplica lo mismo que al campo anterior pero iniciando desde la posición2.

d) Campo4 = Mid(Linea, Posicion3 + 1, Len(Linea))

Esta línea lee el campo "Dirección". Se aplica lo mismo que el campo anterior pero iniciando desde la posición3. Len(Linea) es lo único diferente y se especifica porque no existe una ultima posición que indique la longitud del último campo. Siempre tendrá que utilizar esta sentencia para leer el último campo, pero todo lo demás es igual.



- Línea 23, 24, 25, 26 y 27:

```
If UCase(Campo1) = UCase(Text1.Text) And Option1.Value = True Then
    MsgBox ("Nombre: " & Campo1 & " Apellido: " & _
        Campo2 & " Teléfono: " & Campo3 & " Dirección: " & Campo4)
    Encontro = 1
End If
```

Este bloque de código permite verificar si el contenido del campo nombre es igual al contenido de la caja de texto para luego mostrar el registro completo en una caja de mensaje.

La sentencia **Ucase** permite convertir un texto en mayúscula. Esta sentencia se utilizó para que no exista problema de mayúscula y minúscula, es decir, si el usuario escribe el nombre en mayúscula y en la base de datos esta en minúscula entonces habrá un problema, porque no sería el mismo texto aunque dijera lo mismo, por esa razón se ha convertido el valor del campo y el valor de la caja de texto en mayúscula.

La traducción del bloque anterior es la siguiente:

SI el campo nombre en mayúscula = al contenido de la caja de texto en mayúscula **Y** la primera opción esta seleccionada (la opción búsqueda por nombre) **Entonces**

Muestra (El nombre, el apellido, el teléfono y la dirección)
Asigna el valor uno a la variable Encontro para saber que se encontró el registro

Finaliza el SI

- Líneas 28, 29, 30, 31 y 32:

```
If Campo3 = Text1.Text And Option2.Value = True Then

MsgBox ("Nombre: " & Campo1 & " Apellido: " & _
Campo2 & " Teléfono: " & Campo3 & " Dirección: " & Campo4)
Encontro = 1

End If
```

Lo mismo que el bloque de código anterior. La única diferencia es que la comparación se hace con el Campo3, es decir, con el campo teléfono. Esto es en caso de que se seleccione la segunda opción que es buscar por teléfono.

La traducción del bloque anterior es:

SI el campo nombre en mayúscula = al contenido de la caja de texto en mayúscula **Y** la primera opción esta seleccionada (la opción búsqueda por nombre) **Entonces**

Muestra (El nombre, el apellido, el teléfono y la dirección)
Asigna el valor uno a la variable Encontro para saber que se encontró el registro

Finaliza el SI

- Líneas 28, 29, 30, 31 y 32:

```
If Campo3 = Text1.Text And Option2.Value = True Then
MsgBox ("Nombre: " & Campo1 & " Apellido: " & _
Campo2 & " Teléfono: " & Campo3 & " Dirección: " & Campo4)
```



Encontro = 1

End If

Lo mismo que el bloque de código anterior. La única diferencia es que la comparación se hace con el Campo3, es decir, con el campo teléfono. Esto es en caso de que se seleccione la segunda opción que es buscar por teléfono.

La traducción del bloque anterior es:

SI el campo teléfono = al contenido de la caja de texto **Y** la segunda opción esta seleccionada (la opción búsqueda por nombre) **Entonces**

Muestra (El nombre, el apellido, el teléfono y la dirección)

Asigna el valor uno a la variable Encontro para saber que se encontró el registro

Finaliza el SI

En este bloque no se utilizó la sentencia **Ucase** por la razón de que los números no se escriben ni en mayúscula ni en minúscula.

- Línea 33:

Wend

Esta sentencia ya la he explicado anteriormente y se utiliza para indicar el final del bucle, o bien, hasta donde se repetirá el bucle.

- Líneas 34, 35 y 36:

If Encontro = 0 Then

MsgBox ("No se ha encontrado el contacto especificado.")

End If

Este bloque de código permite mostrar un mensaje en caso de que no se encuentre el registro especificado. Cuando el valor de la variable **Encontro** es igual a cero entonces no se encontró ningún registro y si el valor es uno entonces se encontró el registro.

- Últimas líneas:

Close #1

End If

Exit Sub

NOarchivo:

MsgBox ("La base de datos no existe.")

Close #1

Close #2

▪ En el evento **Click** del botón *Cancelar* escriba:

'Descarga el formulario.

Unload Me



- Corra la aplicación. Haga clic en el botón *Agregar un nuevo contacto* y, a continuación, agregue un nuevo contacto.
- Cierre la ventana de *Agregar un nuevo contacto*, y a continuación, haga clic en el botón *Buscar contacto*.
- Escriba el nombre del contacto y seleccione la opción *Buscar por nombre*, y a continuación, haga clic en el botón *Buscar*.

Ahora vamos a darle función a la tercera opción del menú principal (Eliminar contacto):

Agregue un nuevo formulario y dibuje el siguiente entorno:

- En el evento **Click** del botón *Buscar* escriba:

```
If Len(Trim(Text1.Text)) = 0 Then
    MsgBox ("Debe escribir algo en la caja de texto.")
    Text1.SetFocus
Elseif Option1.Value = False And Option2.Value = False Then
    MsgBox ("Debe seleccionar el tipo de búsqueda.")
Else
```

On Error GoTo NOarchivo:

```
Dim Linea As String
Dim I As Integer
Dim Campo1, Campo2, Campo3, Campo4 As String
Dim Posicion1, Posicion2, Posicion3 As Integer
Dim Encontro As Integer
Encontro = 0

Dim posicionRegistro As Integer
Dim LineasdelArchivo(1000000) As String
Dim NumeroLinea As Integer
Dim registroBorrar As Integer
posicionRegistro = 0
NumeroLinea = 0
```




```

Open "C:\AGENDA\agenda.dat" For Input As #1
    While Not EOF(1)
        NumeroLinea = NumeroLinea + 1
        Line Input #1, LineasdelArchivo(NumeroLinea)
    Wend
Close #1

Open "C:\AGENDA\Agenda.dat" For Input As #1
    While Not EOF(1)
        posicionRegistro = posicionRegistro + 1

        Line Input #1, Linea

        Posicion1 = InStr(1, Linea, ";", vbTextCompare)
        Posicion2 = InStr(Posicion1 + 1, Linea, ";", vbTextCompare)
        Posicion3 = InStr(Posicion2 + 1, Linea, ";", vbTextCompare)

        Campo1 = Mid(Linea, 1, Posicion1 - 1)
        Campo2 = Mid(Linea, Posicion1 + 1, Posicion2 - 1 - Posicion1)
        Campo3 = Mid(Linea, Posicion2 + 1, Posicion3 - 1 - Posicion2)
        Campo4 = Mid(Linea, Posicion3 + 1, Len(Linea))

        Dim Respuesta

        If UCase(Campo1) = UCase(Text1.Text) And Option1.Value = True Then

            registroBorrar = posicionRegistro

            Respuesta = MsgBox("¿Desea eliminar el registro " & "" & _
                Campo1 & Chr(32) & Campo2 & " ?", vbYesNo)

            If Respuesta = vbYes Then

                'SE MODIFICA EL ARREGLO
                '-----

                For I = 1 To NumeroLinea
                    If I = registroBorrar Then
                        LineasdelArchivo(I) = ""
                    End If
                Next I
            End If
            Encontro = Encontro + 1
        End If

        If UCase(Campo3) = UCase(Text1.Text) And Option2.Value = True Then

            Respuesta = MsgBox("¿Desea eliminar el registro " & "" & _
                Campo1 & Chr(32) & Campo2 & " ?", vbYesNo)

```



```

If Respuesta = vbYes Then
    'SE MODIFICA EL ARREGLO
    '-----
    For I = 1 To NumeroLinea
        If I = registroBorrar Then
            LineasdelArchivo(I) = ""
        End If
    Next I

    End If

    Encontro = Encontro + 1
End If
Wend
Close #1

If Encontro = 0 Then
    MsgBox ("No se ha encontrado el contacto especificado.")
End If

If Encontro > 0 And Respuesta = vbYes Then
    Open "C:\AGENDA\TemporalArchivo.Dat" For Output As #2
    For I = 1 To NumeroLinea

        If Len(Trim(LineasdelArchivo(I))) > 0 Then
            Print #2, LineasdelArchivo(I)
        End If
    Next I
    Close #2

    'SE ELIMINA LA BASE DE DATOS ORIGINAL
    '-----
    Kill "C:\AGENDA\agenda.dat"

    'SE LE PONE EL NOMBRE ORIGINAL A LA BASE DE DATOS TEMPORAL
    '-----

    Name "C:\AGENDA\TemporalArchivo.Dat" As "C:\AGENDA\agenda.dat"
    MsgBox (Encontro & " registro eliminados.")

    Text1.Text = ""

End If
End If
Exit Sub
NOarchivo:
MsgBox ("La base de datos no existe.")
Close #1
Close #2

```



Explicación del bloque de código anterior:

1.- Líneas desde la 1 hasta la 12:

Es la misma codificación que se escribió en el botón buscar del formulario *Buscar contacto*.

2.- Líneas 13, 14, 15, 16, 17 y 18:

```
Dim posicionRegistro As Integer
Dim LineasdelArchivo(1000000) As String
Dim NumeroLinea As Integer
Dim registroBorrar As Integer
```

```
posicionRegistro = 0
NumeroLinea = 0
```

En la línea 13 se declara una variable llamada *posicionRegistro* de tipo entero. En esta variable se almacenará el registro actual que se está leyendo de la base de datos, para saber cuál es el número del registro que se está leyendo.

En la línea 14 se declara un arreglo llamado *LineasdelArchivo* de tipo cadena. Este arreglo permitirá almacenar un millón de registros de la base de datos en caso de que existan. Este arreglo es muy importante, ya que va a permitir almacenar temporalmente cada registro de la base de datos.

En la línea 15 se declara una variable llamada *NumeroLinea* de tipo entero. En esta variable se almacenará el total de registros de la base de datos, es decir, el número de líneas que tiene la base de datos.

En la línea 16 se declara una variable llamada *RegistroBorrar* de tipo entero. En esta variable se almacenará el número de la línea que se va a borrar, es decir, el número del registro que se va a borrar de la base de datos.

Las otras dos líneas establecen un valor inicial en las variables *posicionRegistro* y *NumeroLinea*.

3.- Líneas 19, 20, 21, 22, 23 y 24:

```
Open "C:\AGENDA\agenda.dat" For Input As #1
    While Not EOF(1)
        NumeroLinea = NumeroLinea + 1
        Line Input #1, LineasdelArchivo(NumeroLinea)
    Wend
Close #1
```

Este bloque de código permite almacenar en el arreglo cada uno de los registros de la base de datos. Lo único nuevo es el bloque de código *LineasdelArchivo(NumeroLinea)* que explicaré a continuación:

La variable *NumeroLinea* se incrementará cada vez que se repite el bucle. Gracias a esto es posible almacenar en posiciones diferentes del arreglo cada registro leído de la base de datos.

Cada vez que se repite el bucle sucede lo siguiente:

Line Input #1, LineasdelArchivo(1)	Cuando inicia el bucle
Line Input #1, LineasdelArchivo(2)	La segunda repetición
Line Input #1, LineasdelArchivo(3)	La tercera repetición
Line Input #1, LineasdelArchivo(4)	La cuarta repetición

..... Esto se repite hasta el fin del archivo.



- Líneas desde la 25 a la 35:

```
Open "C:\AGENDA\Agenda.dat" For Input As #1
While Not EOF(1)

    posicionRegistro = posicionRegistro + 1

    Line Input #1, Linea

    Posicion1 = InStr(1, Linea, ";", vbTextCompare)
    Posicion2 = InStr(Posicion1 + 1, Linea, ";", vbTextCompare)
    Posicion3 = InStr(Posicion2 + 1, Linea, ";", vbTextCompare)

    Campo1 = Mid(Linea, 1, Posicion1 - 1)
    Campo2 = Mid(Linea, Posicion1 + 1, Posicion2 - 1 - Posicion1)
    Campo3 = Mid(Linea, Posicion2 + 1, Posicion3 - 1 - Posicion2)
    Campo4 = Mid(Linea, Posicion3 + 1, Len(Linea))
```

Esta codificación se explica en el botón *Buscar* de la ventana *Buscar contacto*.

Se ha agregado una nueva línea de código **posicionRegistro = posicionRegistro + 1**. Esta variable se incrementa cada vez que se lee un registro. En pocas palabras esta variable almacena el número del registro que se lee de la base de datos.

- Líneas desde la 36 a la 51:

```
Dim Respuesta
If UCase(Campo1) = UCase(Text1.Text) And Option1.Value = True Then
    registroBorrar = posicionRegistro

    Respuesta = MsgBox("¿Desea eliminar el registro " & """" & _
        Campo1 & Chr(32) & Campo2 & "" ?" , vbYesNo)

    If Respuesta = vbYes Then
        'SE MODIFICA EL ARREGLO
        '-----
        For I = 1 To NumeroLinea
            If I = registroBorrar Then
                LineasdelArchivo(I) = ""
            End If
        Next I
    End If

    Encontro = Encontro + 1
End If
```



En la primera línea del bloque anterior se declara una variable donde se almacena la respuesta proporcionada por el usuario, cuando se le hace la pregunta "¿Desea borrar el registro?".

Si el usuario hace clic en el botón YES o SI, entonces en la variable se almacena un valor constante que es vbYES. Si hace clic en el botón NO, entonces en la variable se almacena un valor constante que es vbNO.

La segunda línea del código anterior se explicó en el botón *Buscar*.

La línea 38 que es la tercera línea del código anterior, permite almacenar en la variable *RegistroBorrar* el número del registro que se va a borrar, que es el registro actual leído de la base de datos.

```
Respuesta = MsgBox("¿Desea eliminar el registro " & "" & _
                  Campo1 & Chr(32) & Campo2 & " ?", vbYesNo)
```

Lo nuevo en esta línea de código es la variable *Respuesta* delante de la sentencia MsgBox. Esta variable permite almacenar la respuesta proporcionada por el usuario.

If Respuesta = vbYes Then

```
'SE MODIFICA EL ARREGLO
'-----
```

```
For I = 1 To NumeroLinea
    If I = registroBorrar Then
        LineasdelArchivo(I) = ""
    End If
```

```
Next I
```

```
End If
```

```
Encontro = Encontro + 1
```

```
End If
```

En este bloque se comprueba cual fue la respuesta proporcionada por el usuario, si la respuesta es sí, se hace una modificación en el arreglo.

La traducción del bloque anterior es la siguiente:

Si Respuesta = Si Entonces

```
'SE MODIFICA EL ARREGLO
'-----
```

```
Para I = 1 Hasta El número de líneas que tiene el archivo
    Si I = Al registro a Borrar Entonces
        Limpia el elemento correspondiente al valor de I en el arreglo LineasdelArchivo
    Fin Si
Próximo I
```

```
Fin Si
```

Incrementa la variable Encontró, para saber cuantos contactos encontró.

Fin SI



- Líneas desde la 52 hasta la 65

```

If UCase(Campo3) = UCase(Text1.Text) And Option2.Value = True Then

    Respuesta = MsgBox("¿Desea eliminar el registro " & "" & _
        Campo1 & Chr(32) & Campo2 & " ?", vbYesNo)

    If Respuesta = vbYes Then
        'SE MODIFICA EL ARREGLO
        '-----
        For I = 1 To NumeroLinea
            If I = registroBorrar Then
                LineasdelArchivo(I) = ""
            End If
        Next I
    End If

    Encontro = Encontro + 1
End If

```

La misma codificación del bloque de código anterior. La única diferencia es que se hace cuando se selecciona la opción búsqueda por teléfono en vez de búsqueda por nombre.

- Líneas desde la 66 hasta la 70:

Wend

```

Close #1

If Encontro = 0 Then
    MsgBox ("No se ha encontrado el contacto especificado.")
End If

```

Este bloque de código se entiende claramente. Indica primero el fin del bucle, cierra el archivo y verifica si no se encontró el contacto especificado para luego mostrar un mensaje al usuario.

-Líneas desde la 71 hasta la 94:

```

If Encontro > 0 And Respuesta = vbYes Then
    Open "C:\AGENDA\TemporalArchivo.Dat" For Output As #2
    For I = 1 To NumeroLinea
        If Len(Trim(LineasdelArchivo(I))) > 0 Then
            Print #2, LineasdelArchivo(I)
        End If
    Next I
    Close #2

    'SE ELIMINA LA BASE DE DATOS ORIGINAL
    '-----
    Kill "C:\AGENDA\agenda.dat"

    'SE LE PONE EL NOMBRE ORIGINAL A LA BASE DE DATOS TEMPORAL
    '-----
    Name "C:\AGENDA\TemporalArchivo.Dat" As "C:\AGENDA\agenda.dat"

```

Ing. Carlos Manuel Rodríguez Bucarely



```

        MsgBox (Encontro & " registro eliminados.")
        Text1.Text = ""

    End If

End If
Exit Sub
NOarchivo:
MsgBox ("La base de datos no existe.")
Close #1
Close #2

```

La primera línea del bloque anterior permite verificar si se ha encontrado algún elemento para borrar y verifica también si el usuario proporciona la respuesta Si, es decir, si el desea que el registro sea borrado. Es lógico que en la búsqueda se puede encontrar el contacto, pero esto no es suficiente, también se requiere saber si el usuario quiere borrar el contacto de la base de datos. Por tal razón se verifica ambas cosas.

```

'SE CREA UN ARCHIVO TEMPORAL
Open "C:\AGENDA\TemporalArchivo.Dat" For Output As #2
    For I = 1 To NumeroLinea - Encontro
        Print #2, LineasdelArchivo(I)
    Next I
Close #2

```

Este bloque de código crea un archivo temporal con los datos que contiene el arreglo. Es muy importante crear un archivo temporal con los datos de la base de datos original porque no trabajaríamos directamente con la base de datos y se envidarían muchos problemas futuros.

El código se entiende claramente: Se abre el archivo para modo de escritura, se inicia un bucle que se repetirá hasta el total de líneas del archivo menos el total de registros borrados, se almacena cada uno de los elementos del arreglo y se cierra el archivo.

```
Kill "C:\AGENDA\agenda.dat"
```

Esta línea de código elimina la base de datos general, pero esto no importa, porque ya tenemos una copia de la base de datos original.

```
Name "C:\AGENDA\TemporalArchivo.Dat" As "C:\AGENDA\agenda.dat"
```

Esta línea de código cambia el nombre de la base de datos temporal y le pone el nombre de la base de datos original. Esto se debe de entender claramente.

```

MsgBox (Encontro & " registro eliminados.")
Text1.Text = ""

```

La primera muestra el total de registros eliminados y la segunda limpia la caja de texto donde se escribió el nombre o el teléfono del contacto. Las demás líneas las hemos visto anteriormente.

- En el evento **Click** del botón *Cancelar* escriba:

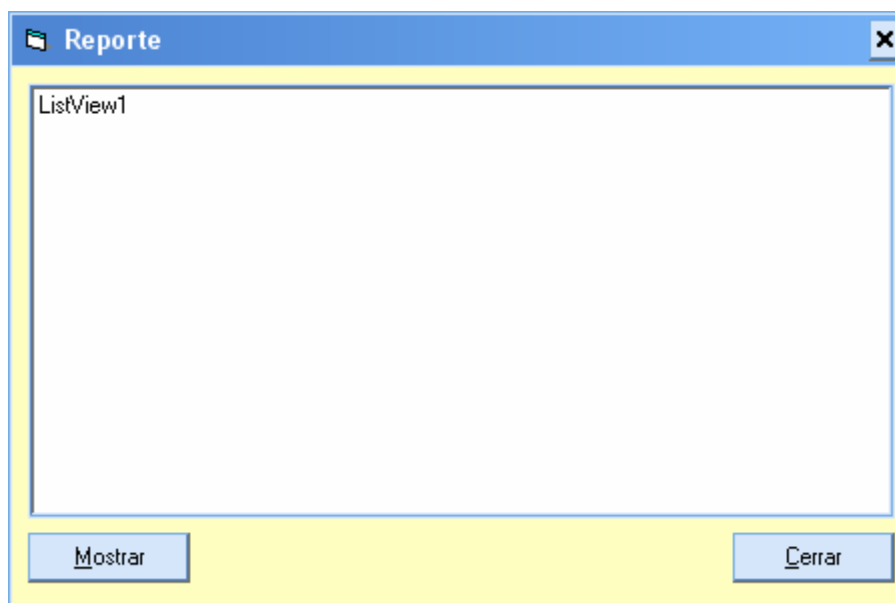
```
Unload ME
```

- En el evento **Click** del tercer botón del menú Principal de nuestra aplicación escriba:

```
Form3.Show 1, Me
```



- Corra la aplicación.
- Haga clic en el botón *Eliminar contacto*, introduzca el nombre del contacto que desea eliminar y a continuación, haga clic en el botón *Buscar*. Si el contacto no existe agréguelo.
- Ahora daremos función a la cuarta opción del menú principal, al botón *Reporte*.
- Agregue un nuevo formulario.
- Antes de proseguir debemos agregar un nuevo control a la barra de controles. Este control se llama **ListView** que estudiamos en el capítulo anterior.
- Haga clic derecho sobre la barra de controles y seleccione la opción *Componentes (Components)*.
- En el cuadro de diálogo *Componente*, busque y active el componente *Microsoft Windows Common Controls* y haga clic en el botón *Aceptar*.
- Dibuje un control **ListView**  en el formulario y dos botones de comando, tal y como se muestra en la siguiente imagen:



- Haga un solo clic sobre el control **ListView** y busque la propiedad **View**. Establezca el valor **3 – lvwReport**. Este valor permitirá mostrar los registros como un reporte.
- En el evento **Load** del formulario escriba:

Dim dato As ListItem

```
ListView1.ColumnHeaders.Add(1) = "Nombre"  
ListView1.ColumnHeaders.Add(2) = "Apellido"  
ListView1.ColumnHeaders.Add(3) = "Telefono"  
ListView1.ColumnHeaders.Add(4) = "Direccion"
```

El bloque de código anterior permite agregar los encabezados al control **ListView**. Los encabezados serán los campos de la base de datos.



- En el evento **Click** del botón *Mostrar* escriba:

On Error GoTo Noarchivo:

```
Dim Linea As String
Dim campo1, campo2, campo3, campo4 As String
Dim Posicion1, Posicion2, Posicion3 As Integer
Dim dato
Dim Encontro As Integer
Encontro = 0

Open "C:\AGENDA\Agenda.dat" For Input As #1
While Not EOF(1)
    Line Input #1, Linea

    If Len(Trim(Linea)) > 0 Then
        Posicion1 = InStr(1, Linea, ";", vbTextCompare)
        Posicion2 = InStr(Posicion1 + 1, Linea, ";", vbTextCompare)
        Posicion3 = InStr(Posicion2 + 1, Linea, ";", vbTextCompare)
        campo1 = Mid(Linea, 1, Posicion1 - 1)
        campo2 = Mid(Linea, Posicion1 + 1, Posicion2 - 1 - Posicion1)
        campo3 = Mid(Linea, Posicion2 + 1, Posicion3 - 1 - Posicion2)
        campo4 = Mid(Linea, Posicion3 + 1, Len(Linea))

        Set dato = ListView1.ListItems.Add(, , campo1)
        dato.SubItems(1) = campo2
        dato.SubItems(2) = campo3
        dato.SubItems(3) = campo4

        Encontro = 1
    End If
Wend
Close #1

If Encontro = 0 Then
    MsgBox ("La base de datos esta vacía")
End If
```

```
Exit Sub
Noarchivo:
MsgBox ("La base de datos no se encuentra.")
```

Nada nuevo en el bloque de código anterior.

- En el evento **Click** del botón *Cerrar* escriba:

```
Unload Me
```

- Corra la aplicación.
- Haga clic en el botón "Reporte". Si hay datos se mostrarán de forma organizada en el control ListView.
- Detenga la aplicación y guárdela en la carpeta "C:\AGENDA".



7.9 EJERCICIOS PROPUESTOS

- 1.- Hacer una aplicación que permita abrir un archivo de texto y mostrarlo en una caja de texto. La aplicación debe permitir seleccionar el archivo de la unidad de disco mediante el cuadro de diálogo Abrir.
- 2.- Hacer una aplicación que permita almacenar en un archivo con estructura de base de datos los campos: Nombre, Apellido, Edad y lugar de nacimiento. El programa debe permitir visualizar los registros agregados en un control ListView. También debe permitir borrar un registro almacenado en el archivo de base de datos.
- 3.- Hacer una aplicación que permita borrar un archivo de una unidad de disco. El nombre de este archivo y la unidad donde este se encuentra debe ser especificado en una caja de texto y mediante un botón de Borrar el programa debe permitir borrarlo. Si el archivo no se encuentra se debe mostrar un mensaje informando al usuario que ese archivo no existe.
- 4.- Hacer una aplicación que permita almacenar en un archivo el texto escrito en una caja de texto. El programa debe permitir al usuario escribir el nombre del archivo y la unidad donde desea guardarlo. Para esto, usted debe utilizar el cuadro de diálogo *Guardar* del control Common Dialog.
- 5.- Hacer una aplicación similar al *Bloc de notas* de Windows. Debe dar funciones a las opciones del primer menú (Archivo) y del tercer menú (Fuente). Utilice para esto el control Common Dialog.



CONTENIDO

8.1 Introducción a los conceptos de base de datos.

- 8.1.1 Concepto de base de datos.
- 8.1.2 Elementos de una base de datos.
- 8.1.3 Estructura de una base de datos.
- 8.1.4 Sistema de Gestión de Base de datos (SGBD).
- 8.1.5 Administrador de base de datos (ABD).
- 8.1.6 Diseñador de base de datos (DBD).

8.2 Creación de una base de datos.

8.3 Conectar una base de datos con Visual Basic 6.0.

- 8.3.1 El Modelo de datos ODBC (Open Database Connectivity).
- 8.3.2 El modelo de objetos DAO (Data Access Object).
- 8.3.3 El modelo de objetos RDO (Remote Data Object).
- 8.3.4 El modelo de objetos ODBCDirect.
- 8.3.5 El modelo de objetos OLE DB.
- 8.3.6 El modelo de objetos ADO (ActiveX Data Object).

8.4 Preparando la conexión a la base de datos.

- 8.4.1 Objeto Recordset.
 - 8.4.1.1 Propiedades del objeto Recordset.
 - 8.4.1.2 Manejo de cursores.
 - 8.4.1.3 Control de concurrencia.
 - 8.4.1.4 Lectura de los campos de un Recordset.
 - 8.4.1.5 Moverse por los registros de un Recordset.
 - 8.4.1.6 Modificación de registros en un Recordset.
 - 8.4.1.7 Eliminar el registro activo del Recordset.
 - 8.4.1.8 Inserción de registros en el Recordset.
 - 8.4.1.8 Establecer y leer la posición de un registro en el Recordset.
 - 8.4.1.9 Ordenación de los registros de un Recordset.
 - 8.4.1.10 Búsqueda de registros.
 - 8.4.1.11 Verificar el estado del Recordset.
- 8.4.2 Eventos del objeto Recordset.
 - 8.4.2.1 Sucesos de recuperación de datos.
 - 8.4.2.2 Sucesos de navegación.
 - 8.4.2.3 Sucesos de modificación de datos.
- 8.4.3 Generación de reportes.

8.5 Ejercicios propuestos.



8.1 INTRODUCCIÓN A LOS CONCEPTOS DE BASE DE DATOS

Cuando hablamos de base de datos nos estamos refiriendo a un lugar donde se almacena cualquier tipo de información que de alguna forma están relacionadas y organizadas. Para que entienda esto, imaginase cualquier empresa, como por ejemplo, un banco o una universidad. Ambas son empresas con propósitos muy diferentes pero tienen una necesidad en común y es almacenar y recuperar la información recolectada. Por un lado un banco requiere almacenar los datos de sus clientes, las transacciones realizadas por los clientes, los datos de sus empleados, etc. Por otro lado, una universidad tiene la necesidad de almacenar los datos de sus estudiantes, las calificaciones, las facturas de pagos, etc. Esto nos da a entender que existe un lugar donde toda empresa almacena y recupera la información. Este lugar recibe el nombre de base de datos.

Una aplicación profesional debe ser capaz de *obtener información* (permitir que el usuario introduzca algún tipo de datos), *procesar información* (transformar la información introducida), *almacenar información* (mantener la información en un dispositivo de almacenamiento), *leer información almacenada* (permitir que el usuario consulte los datos almacenados) y *mostrar los resultados* (presentar por pantalla u otro dispositivo de salida la información requerida). Estas características son las que un sistema de información computarizado debe ser capaz de realizar para que sea realmente profesional y potente. Pero para lograr que una aplicación cumpla con esos requerimientos es necesario utilizar sistemas de bases de datos y no archivos convencionales como hemos estado viendo hasta ahora.

La información en una base de datos esta organizada y relacionada, esto facilita en gran manera las operaciones de consultas y almacenamiento. Además, una base de datos no depende de la aplicación, sino que utilizaremos *sistemas de gestión de base de datos* (SGBD) para crear y modificar la estructura de la base de datos.

Si usted no tiene conocimientos amplio de la importancia y beneficios de una base de datos, no se preocupe porque en este capítulo hablaremos lo necesario acerca de esta para que este más o menos relacionados con los conceptos básicos concernientes a estas.

- 8.1.1 Definición de base de datos

Una base de datos consiste en un conjunto de datos relacionados y organizados lógicamente en un dispositivo de almacenamiento (archivero, disco duro, cintas magnéticas, discos ópticos, etc). En nuestro caso estamos hablando de bases de datos computarizadas donde el principal dispositivo de almacenamiento será el disco duro.

- 8.1.2 Elementos de una base de datos

Una base de datos esta compuesta por los siguientes elementos: dato, campo, registro, tabla y archivo. Un **dato** es la parte esencial de la información, es decir, la información que llega a la base de datos. Los datos pueden ser de distintos tipos como: *texto, numérico, alfanumérico, lógico, memo, etc.*

Un **campo** es la unidad más pequeña de datos. Estos los representan las columnas de las tablas.

Registro es un conjunto de campos o atributos relacionados entre sí. Representan las filas de las tablas.

Tabla: Es el lugar donde la base de datos organiza la información. Esta es compuesta por filas(registros) y columnas (campos).

Archivo: es un conjunto de registros relacionados.



- 8.1.3 Estructura de una base de datos

Explicamos anteriormente que una base de datos esta compuesta por datos, campos, registros, tabla y archivo. Todos estos elementos se encuentran unidos lógicamente en un elemento grafico llamado tabla, tal y como se muestra en la siguiente figura:

Base De datos: VENTAS

Tabla: CLIENTES

Nombre	Apellido	Dirección	Teléfono	Celular
José	Pérez	Calle Terminal #5	809-598-1252	809-598-5631
Pedro	Rodríguez	Urb. Alma Rosa #2	809-563-5985	809-563-5485
Carlos	Bucarelly	Av. Charles de Gaulle	809-596-5485	829-458-5695

Los campos de la tabla están representados por las columnas, que en este caso son cinco **Nombre, Apellido, Dirección, Teléfono y Celular**.

Los registros están compuestos por el nombre, apellido, dirección, teléfono y celular de cada persona. La tabla anterior esta compuesta por tres registros.

Los datos son la mínima información que contiene cada campo, por ejemplo, José es un dato, Pérez es otro dato, etc. La combinación de todos los datos o campos de una fila de la tabla se le llama *registro*.

El archivo esta representado por el nombre de la base de datos, que en este caso es VENTAS. Este archivo contiene una tabla cuyo nombre es CLIENTES. La tabla que hemos puesto de ejemplo contiene información personal acerca de los clientes de una empresa.

- 8.1.4 Sistema de Gestión de Base de datos (SGBD)

Un **Sistema Gestor de base de datos (SGBD)** es un conjunto de programas que permiten crear y mantener una Base de datos, asegurando su integridad, confidencialidad y seguridad. Este sistema debe permitir lo siguiente:

- Definir una base de datos: especificar tipos, estructuras y restricciones de datos.
- Construir la base de datos: guardar los datos en algún medio controlado por el mismo SGBD
- Manipular la base de datos: realizar consultas, actualizarla, generar informes.

Entre los principales SGBD tenemos: Microsoft Access, SQL Server, Oracle y MySQL. En este libro utilizaremos Microsoft Access para crear nuestras bases de datos, debido a que es uno de lo más sencillo y el más económico. Es importante que tenga un poco de conocimiento en el manejo de Access para que tenga una mejor visión de las aplicaciones que se realicen en este capítulo.

- 8.1.5 Administrador de base de datos (ABD)

Es la persona o equipo de personas profesionales responsables del control y manejo del sistema de base de datos, generalmente tiene(n) experiencia en DBMS, diseño de bases de datos, sistemas operativos, comunicación de datos, hardware y programación. Entre las principales responsabilidades que desempeña un ABD tenemos: **repuperabilidad** (capacidad para recuperar datos perdidos en la base de datos, **integridad** (verificar ó ayudar a la verificación en la integridad de datos), **seguridad** (definir y/o implementar controles de acceso a los datos), **disponibilidad** (los usuarios autorizados tengan acceso a los datos cuando lo necesiten para atender a las necesidades del negocio) y **desempeño** (asegurarse del máximo desempeño incluso con las limitaciones).



- 8.1.6 Diseñador de base de datos (DBD)

Es la persona encargada del diseño de la base de datos. El diseñador de bases de datos se encarga de identificar los datos que se almacenarán en la base de datos y elegir las estructuras apropiadas para la misma. Esta tarea suele realizarse antes de que se implemente y se llene de datos la base de datos, aunque muchas veces un diseñador debe trabajar sobre la misma cuando ya esta en funcionamiento.

El/los diseñadote(s) de base de datos se encargan de determinar los requerimientos de los usuarios que usarán la base de datos. A partir de estos requerimientos, diseñarán y crearán la base de datos.

8.2 CREACIÓN DE UNA BASE DE DATOS

Crear una base de datos es realmente un proceso creativo. Se requiere de una buena planeación antes de crear la base de datos. Si en un principio no se identifican realmente los datos que se almacenarán en la base de datos podrían enfrentarse a muchos problemas en el futuro después de haber creado la aplicación. Uno de estos inconvenientes es, que la base de datos no esta almacenando realmente los datos requeridos por la empresa y al momento de generar una consulta no existe la forma de obtenerla, debido a que usted no previó el almacenamiento de un dato importante. Por ejemplo, si usted diseño una aplicación para una tienda que permite registrar todos los productos que llegan al almacén, entre los cuales el sistema solitita al usuario los siguientes datos del artículo: código del artículo, descripción, precio unitario y cantidad entrante. Después que la empresa a utilizado el sistema por alrededor de dos meses el gerente de almacén requiere un reporte de todos los artículos de la tienda por proveedor. Como su sistema no solicito del usuario el nombre del proveedor al momento de registrarlo no es posible obtener una consulta de este tipo. Por tal razón, su sistema no seria realmente eficiente.

Antes de diseñar una base de datos tenga bien en cuenta que todos los datos requeridos se están almacenando en la base de datos. Para diseñar una base de datos realmente consistente siga los siguientes pasos:

1.- Seleccione adecuadamente el SGBD que utilizará para crear la base de datos. El SGBD dependerá del tipo de aplicación. Una empresa pequeña como un colegio, un instituto, un almacén, una distribuidora, entre otras, no requieren de una base de datos tan potente. Podrá utilizar un SGBD como Microsoft Access para almacenar los datos de dicha empresa. Además, de que el costo de licencia para este sistema es muy bajo. Para empresas que requieren almacenar enormes cantidades de datos tendrá que utilizar SGBD más potentes como **SQL Server** y **Oracle**. Ejemplos de estas empresas serían un banco o una universidad.

2.- Identifique cuidadosamente los datos que serán almacenados en la base de datos. Antes de diseñar una base de datos debe tener bien claro cuales son los campos requeridos para cada tabla para satisfacer las necesidades del sistema.

3.- Haga un boceto del diseño de la base de datos. Es recomendable escribir en hoja de papel la estructura que tendrá la base de datos, esto es los *tipos de datos*, *campos*, *relaciones*, *restricciones*, etc.

4.- Utilice el SGBD crear la base de datos. Después de haber hecho el boceto de su base de datos, utilice el SGBD seleccionado para crear la base de datos.

5.- Identifique la fuente de alimentación de la base de datos. Después de haber diseñado la base de datos es necesario introducir, modificar y actualizar los datos en ella mediante algún medio, esto puede ser desde el propio SGBD o desde una aplicación externa.



8.3 CONECTAR UNA BASE DE DATOS CON VISUAL BASIC 6.0

Existen varias formas para conectar una base de datos a una aplicación de Visual Basic, entre las cuales cabe mencionar la utilización de tecnologías **ODBC**, **DAO**, **RDO**, **ODBCDirect**, **OLE DB** y **ADO**. Aunque en este libro me centraré en la tecnología **ADO (ActiveX Data Object)** explicaré brevemente cada una de las interfaces de conexión a bases de datos mencionadas anteriormente, debido a que cada una de estas están íntimamente relacionadas.

- 8.3.1 El modelo de datos ODBC (Open Database Connectivity)

ODBC son las siglas de *Conectividad de bases de datos abiertas (Open Database Connectivity)* y es un conjunto de funciones que le permitirá conectarse a una base de datos local o remota. Es una tecnología que permite acceder a distintas bases de datos en diferentes formatos como Microsoft Visual FoxPro, Microsoft Access, Microsoft SQL Server, dBASE, Oracle y archivos de texto separados por comas. La máquina sobre la que se ejecuta la aplicación se conecta en un DLL denominado **ODBC Driver Manger** (*Gestor del controlador ODBC*) que, a su vez es el encargado de mandar y recibir los datos a un controlador ODBC específico para la base de datos particular que desee utilizar. Existen ciento de controladores ODBC para las distintas bases de datos actuales del mercado, incluso para bases de datos descontinuadas.

El objetivo de ODBC es proporcionar una interfaz común para todas las bases de datos existentes. Teóricamente, podrá preparar una aplicación que utilice ODBC para hablar con una base de datos de Access y, posteriormente, adaptar el programa para una base de datos SQL Server cambiando simplemente el controlador ODBC e introduciendo unas pocas instrucciones en el código fuente.

ODBC tiene muchas ventajas en comparación con otros métodos de acceso a bases de datos, sin embargo, utilizar ODBC no resulta muy sencillo, especialmente para los programadores de Visual Basic. Trabajar con ODBC implicar manejar los conceptos API de Windows que son bastante complejos y si comete un error se suele interrumpir la ejecución de la aplicación con un error fatal. Por este motivo, son pocos los programadores de Visual Basic que escriben aplicaciones que llamen directamente a las funciones ODBC. Increíblemente, la mayoría de las otras técnicas de acceso a datos disponibles para Visual Basic pueden utilizar controladores ODBC como capas intermedias por lo que, en ocasiones, podrá potenciar las otras técnicas con llamadas directas a los API, principalmente con aquellas basadas en RDO. Por desgracia, no podrá mezclar código ODBC API con ADO aun que este utiliza internamente un controlador ODBC.

- 8.3.2 El modelo de objetos DAO (Data Access Object)

DAO (Objeto de acceso a datos) es una técnica de acceso a base de datos de Microsoft Access basado en el motor **Microsoft Jet** que es el que propulsa a Access. Los diseñadores pueden diseñar una base de datos MDB utilizando Access y, posteriormente, utilizar DAO desde una aplicación de Visual Basic para abrir la base de datos, agregar y recuperar registros y gestionar transacciones. Aunque DAO se diseñó pensando en Access, este no limita la conexión a otras bases de datos para la que exista un controlador ODBC.

Una de las desventajas de DAO es que aunque no utilice bases de datos de Access tendrá que cargar completamente el motor Microsoft Jet y distribuirlo en sus aplicaciones. Otra importante desventaja es que, DAO no cuenta con muchas de las funciones que podría utilizar si trabaja directamente con funciones ODBC API. Por ejemplo, no podrá realizar consultas asíncronas o conexiones utilizando DAO, ni tampoco podrá trabajar con conjuntos múltiples de resultados.

DAO fue una de las herramientas de acceso a datos para los primeros programadores de Visual Basic 3. Actualmente este método de acceso no es utilizado por los programadores de Visual Basic 6.0, debido a que su sucesor ADO es mucha más potente que DAO y es el objeto de estudio y desarrollo de la Microsoft para acceso a datos.



- 8.3.3 El modelo de objetos RDO (Remote Data Object)

RDO (*Objetos de Datos Remotos*) es el primer intento que realizó Microsoft para combinar la sencillez del DAO con la potencia de la programación directa del ODBC API. RDO es un modelo de objetos vagamente diseñado a partir de DAO, pero deshecha el Jet Engine y el DAO DLL y trabaja directamente con los controladores ODBC subyacentes. Las aplicaciones basadas en RDO cargan sólo un pequeño DLL en lugar del motor Jet que consumía gran cantidad de recursos. Lo más importante es que RDO fue específicamente diseñado para trabajar con los orígenes ODBC, por lo que cuenta con funciones que no pueden ser utilizadas desde DAO. Sin embargo, RDO es una tecnología de 32 bits, por lo que no podrá utilizarla en las versiones 2 y 3 de Visual Basic. RDO fue implementado por primera vez en la versión 4 de Visual Basic y, posteriormente fue mejorado en la versión 5.

- 8.3.4 El modelo de objetos ODBCDirect

Además del RDO mejorado que incluía Visual Basic 5, este incluía otra tecnología de acceso a datos, denominada **ODBCDirect**, que permitía a los programadores emplear RDO utilizando una sintaxis DAO. ODBCDirect fue concebido como una técnica de transición que ayudaría a los programadores en Visual Basic a transformar sus aplicaciones DAO/Jet a arquitecturas cliente/servidor de mayor potencia. ODBCDirect no es una tecnología propiamente dicha. Es únicamente un conjunto de trucos que puede utilizar para ahorrarse tiempo durante la conversión de aplicaciones.

- 8.3.5 El modelo de objetos OLE DB

OLE DB es una tecnología de acceso a datos de bajo nivel con la que Microsoft pretende sustituir a ODBC como el medio principal de conexión con bases de datos. Aunque OLE DB es una nueva tecnología, podrá encontrar proveedores de OLE DB para las bases de datos más populares, y otras serán comercializadas rápidamente. A pesar de sus aparentes similitudes, las tecnologías ODBC y OLE DB son profundamente distintas. En primer lugar, OLE DB esta basada en COM, que es una arquitectura suficientemente robusta diseñada para mover grandes cantidades de datos por la red. En segundo lugar, OLE DB pretende realizar la tarea de conectar a cualquier fuente de datos, no sólo las bases de datos relacionales e ISAM (modo de acceso secuencial indexado), sino, que forma parte de la estrategia denominada Acceso universal de datos (UDA) de Microsoft, que le permitirá leer y procesar los datos allá donde se encuentren, sin necesidad de convertirlos en primer lugar y de tener que importarlos a una base de datos más tradicional. Utilizando los proveedores OLE DB, podrá procesar los datos contenidos en los mensajes de correo electrónico, páginas HTML, hojas de cálculo y documentos de texto, entre otras fuentes de datos.

- 8.3.6 El modelo de objetos ADO (ActiveX Data Object)

ADO es una interfaz de alto nivel con OLE DB. Al igual que los API de ODBC, OLE DB es una interfaz de bajo nivel a la que no se puede acceder con facilidad utilizando lenguajes de alto nivel como Visual Basic. ADO está construido sobre un OLE DB para proporcionar funciones que no se encuentran disponibles directamente en OLE DB o que exigirían profundos conocimientos y habilidades de codificación a los programadores.

El modelo de objetos ADO es considerablemente menos complicada que los modelos DAO y RDO. ADO contiene mucho menos objetos y colecciones que DAO y RDO, pero los elementos que contiene son, con frecuencia, más complejos que sus equivalentes DAO y RDO porque dispone muchos más métodos y propiedades.

Las principales ventajas de ADO son su facilidad de uso, su gran velocidad, su escasa utilización de memoria y el poco espacio que ocupa en disco. ADO proporciona un acceso a los datos constante y de alto rendimiento para crear un cliente de base de datos para el usuario o un objeto empresarial del nivel medio con una aplicación, una herramienta, un lenguaje o un explorador.



8.4 PREPARANDO LA CONEXIÓN A LA BASE DE DATOS

Al principio de este capítulo mencione que la principal herramienta tecnológica que utilizaríamos en este libro para conectar a una base de datos sería ADO. Esto es, porque considero que es la herramienta más completa y potente de la que dispone Visual Basic para conectar a un origen de datos.

El modelo de datos ADO dispone de un objeto llamado **CONNECTION** que permite establecer una conexión a un origen de datos que puede ser una base de datos, un origen ODBC o cualquier otro origen que disponga un proveedor OLE. El objeto **Connection** le permitirá especificar todos los parámetros necesarios antes de abrir una base de datos.

Cómo establecer la conexión

Para establecer la conexión el objeto **Connection** dispone del método **Open**, seguida de los parámetros necesarios para la conexión.

Su sintaxis es:

Open [*ConnectionString*], [*UserID*], [*Password*], [*Options*]

El primer argumento *ConnectionString* representa una serie de parámetros necesarios para establecer la conexión a la base de datos. Estos parámetros son por ejemplo, el proveedor ODBC, el nombre de la base de datos, el nombre de usuario, la contraseña y el nombre del servidor.

Ejemplo 1:

```
Dim cn As New ADODB.Connection 'Linea necesaria para establecer la conexión.  
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\clientes.mdb"
```

El código del ejemplo anterior abre una base de datos llamada clientes.mdb que se encuentra en el disco local C, mediante el proveedor ODBC **Microsoft.Jet.OLEDB.4.0** que permite conectar a bases de datos de Access 2000 y Access 97 sin ningún problema.

Ejemplo 2:

```
Dim cn As New ADODB.Connection  
cn.Open "Provider=SQLOLEDB;" & "Server=ServerNT;" & "User ID=jose;Password=abc;" _  
& "Data Source=empleados"
```

El código del ejemplo anterior abre una base de datos llamada empleados que se encuentra en un servidor denominado "ServerNT". También especificamos el nombre y la contraseña de usuario.

Los argumentos *UserID* y *Password* son opcionales. Estos no deben especificarse si lo escribe directamente en el argumento *ConnectionString*, tal y como lo vemos en el segundo ejemplo. El argumento opcional *Options* determina si este método debe volver después o antes de que la conexión se establezca. Este argumento puede tomar los valores **-1-adConnectUnspecified** (*Valor por defecto*), que abre una *conexión síncrona* con la base de datos y el valor **16-adAsyncConnect** abre una *conexión asíncrona* con la base de datos. Una *conexión síncrona* indica que Visual Basic no ejecutará la instrucción que sigue al método **Open** hasta que se establezca la conexión, en consecuencia la aplicación no responderá a los eventos del usuario. Por otro lado, una *conexión asíncrona*, permite que el usuario trabaje con el programa aun la conexión no se halla establecido. Y lo mejor de todo, permite informar el estado en que se encuentra la conexión. Este tipo de conexión será el caso de estudio en otra sección de este mismo capítulo, mientras tanto, trabajaremos con *conexiones síncrona*.



Estos son algunos de los argumentos que podrá utilizar en el atributo *ConnectionString* al momento de abrir una conexión.

Argumento	Descripción
Data Source	El nombre del servidor SQL o el nombre de la base de datos MDB a la que se desee conectar. Cuando se conecte a un origen ODBC, este argumento puede ser también el nombre de un origen de datos (DSN).
DSN	Nombre del origen ODBC registrado en la máquina actual; este argumento puede sustituir al argumento Data Source.
FileName	Archivo que contiene información sobre la conexión; este argumento puede ser un archivo ODBC DSN o un archivo Microsoft Data Link (UDL).
Initial Catalog	Nombre de la base de datos predeterminada sobre la conexión. Cuando se conecte a un origen de datos ODBC, también podrá utilizar el argumento Database.
Password	Contraseña del usuario. Cuando se conecte a un origen ODBC, podrá utilizar el argumento PWD. No tendrá que pasar su ID y contraseña de usuario si se está conectando a un servidor SQL y utiliza seguridad integrada.
Persist Security Info	True si ADO almacena el ID y la contraseña de usuario en el vínculo de datos.
Provider	Nombre de un proveedor OLE; el valor predeterminado es MSDASQL, el proveedor para orígenes ODBC.
User ID	Nombre del usuario. Cuando se conecte a un origen ODBC, podrá utilizar en su lugar el argumento UID.

Hasta ahora usted está en la capacidad de realizar una conexión a una base de datos y nada más. Necesita ahora conocer la forma en que puede obtener los datos que se encuentran en la base de datos y mostrarlos en cajas de texto, en un DataGrid o en otro control en la que se puede mostrar datos. En la siguiente sección veremos la forma de hacer esto y al finalizarla crearemos nuestra primera aplicación de base de datos.

- 8.4.1 Objeto Recordset

Además del objeto **Connection**, ADO posee un objeto denominado **Recordset**, que contiene todos los datos que se leerá de una base de datos o que enviará a la misma. Un Recordset puede incluir varias filas y columnas de datos. Cada fila es un registro y cada columna es un campo del registro. Sólo podrá acceder simultáneamente a una fila, la denominada fila actual o registro actual. Podrá examinar un Recordset modificando el registro actual.

- 8.4.1.1 Propiedades del objeto Recordset

Posiblemente la propiedad más importante del objeto Recordset es la propiedad **Source** que contiene el nombre de la tabla, el nombre del procedimiento almacenado o el texto de una consulta SQL utilizada para llenar el Recordset.

Para cargar un Recordset usted debe realizar tres pasos que son realmente necesarios, el primero,



es crear el objeto Recordset, lo segundo es, indicar la fuente de datos que alimentará al objeto Recordset, tercero, abrir el Recordset con el método **Open** y llenar el Recordset con un texto de una consulta SQL.

A continuación, se muestran algunos ejemplos de cómo abrir una base de datos y un Recordset.

Ejemplo 1:

```
Dim cn As New ADODB.Connection      'Creamos el objeto Connection.  
Dim rs As New ADODB.Recordset        'Creamos el objeto Recordset.
```

'Abrimos la base de datos "ventas.mdb".

```
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\ventas.mdb"
```

```
rs.Source = "clientes"              'Especificamos la fuente de datos. En este caso la tabla "clientes".
```

```
rs.Open "select * from clientes", cn 'Abrimos el Recordset y lo llenamos con la consulta SQL.
```

En este ejemplo creamos el objeto Connection y luego creamos el objeto Recordset. Luego abrimos la base de datos utilizando el método **Open** del objeto Connection, tal y como vimos anteriormente. En la cuarta línea de código especificamos el origen de los datos, que en este caso es la tabla "clientes" de la base de datos "ventas.mdb". Por último, utilizamos el método **Open** del objeto Recordset para abrirlo. El texto *"select * from clientes"* es una consulta SQL que permite seleccionar todos los registros de la tabla. En este caso el Recordset se llenará con todos los registros de la tabla clientes.

Ejemplo 2: (Otra forma de abrir el Recordset)

```
Dim cn As New ADODB.Connection      'Creamos el objeto Connection.  
Dim rs As New ADODB.Recordset        'Creamos el objeto Recordset.
```

'Abrimos la base de datos "ventas.mdb".

```
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\ventas.mdb"
```

```
rs.Source = "clientes"              'Especificamos la fuente de datos. En este caso la tabla "clientes".
```

```
rs.ActiveConnection = cn            'Activa la conexión asociándolo a la conexión existente.
```

```
rs.Open "select * from clientes"    'Abrimos el Recordset y lo llenamos con la consulta SQL.
```

En este ejemplo se omite el parámetro de conexión en el método **Open** por la propiedad **ActiveConnection** del objeto Recordset.

Ejemplo 3: (Otra forma es abrir la base de datos de forma implícita)

```
Dim rs As New ADODB.Recordset        'Creamos el objeto Recordset.
```

'Activamos y especificamos los parámetros de la conexión en el objeto Recordset.

```
rs.ActiveConnection = "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\ventas.mdb"
```

```
rs.Source = "clientes"              'Especificamos la fuente de datos. En este caso la tabla "clientes".
```

```
rs.Open "select * from clientes"    'Abrimos el Recordset y lo llenamos con la consulta SQL.
```



- 8.4.1.2 Manejo de cursores

Un **cursor** es un conjunto de registros que constituyen el resultado de una consulta. Los cursores son necesarios definirlos para indicar el comportamiento y la funcionalidad disponibles de los registros en el Recordset. Para definir el tipo de cursor debe indicar donde se debe almacenar los cursores, en la estación de trabajo (cliente) o en el servidor. También debe especificar el tipo del cursor y la opción de bloqueo.

La propiedad **CursorLocation** del objeto Recordset permite definir el lugar donde se almacenarán los cursores. Esta propiedad puede tomar los valores **2-adUseServer** (*valor por defecto*) o **3-adUseClient**. El valor 2-adUseServer es el predeterminado para cuando este trabajando con el Proveedor OLE DB para controladores ODBC y para SQL Server, es un cursor de *sólo avance* creado en el servidor. Este tipo de cursor del lado del servidor es el más eficaz debido a que ofrecen un mejor rendimiento y más tipos de cursores. Los cursores del lado del cliente son frecuentemente utilizados cuando tiene un control DataGrid u otro control complejo que esté unido al Recordset.

Al definir un cursor, también deberá especificar el tipo de cursor con la propiedad **CursorType** que comunica el tipo de cursor que debería crearse en el servidor o en el cliente. Los distintos tipos que se puede crear son:

- { HYPERLINK "http://www.uysssoft.com/Docs/CursoresADO/" \ "El cursor ForwardOnly:" }.
- { HYPERLINK "http://www.uysssoft.com/Docs/CursoresADO/" \ "El cursor Static:" }.
- { HYPERLINK "http://www.uysssoft.com/Docs/CursoresADO/" \ "El cursor KeySet:" }.
- { HYPERLINK "http://www.uysssoft.com/Docs/CursoresADO/" \ "El cursor Dynamic:" }.

El cursor ForwardOnly (Sólo hacia delante):

Un cursor **ForwardOnly** (*Sólo avance*) esta disponible solo del lado del servidor. Es el cursor más simple que existe. Sólo permite desplazamiento hacia delante y al abandonar un registro éste deja de estar disponible, esto hace que sea el cursor más rápido e ideal para operaciones donde debemos abrir un conjunto de registros e ir haciendo una lectura secuencial hasta llegar al final. Un ejemplo típico es llenar una lista de opciones con los valores de una tabla.

Al abrir un cursor de este tipo se leen un numero de registros y se pasan a la caché (en función del valor de la propiedad CacheSize), y a medida que se va avanzando por el cursor, ADO recupera el conjunto siguiente de registros. Este tipo de cursor es realmente eficiente si define la propiedad **LockType** a *adReadOnly* y **CacheSize** igual a 1. Este tipo de cursor se define con el valor **0-adOpenForwardOnly** en la propiedad **CursorType** del Recordset.

El cursor Static (Estático):

Un **cursor estático** es muy similar a un cursor sólo hacia delante, con la salvedad de que admite desplazamiento en todas las direcciones (MoveFirst, MovePrevious, MoveNext, MoveLast y Move), el propio motor de ADO recuperará en la caché los registros que necesite al desplazarnos por el mismo.

El comportamiento de este cursor no muestra los cambios realizados por otros usuarios a los datos de los registros del cursor, de la misma forma que no muestra los registros añadidos ni eliminados, es por ello que se denomina estático. Este es el único cursor posible del lado del cliente y no tendrá que definirlo si pasa el valor *2-adUseClient* a la propiedad **CursorLocation**. Aunque estos cursores son menos eficaces que los cursores de *sólo avance* y aumentan la carga de trabajo del sistema en el que residen, su rendimiento es razonable y suelen ser una buena opción, especialmente cuando el Recordset no incluye demasiados registros. Los cursores estáticos suelen ser la mejor opción para recuperar datos de un procedimiento almacenado. Utilice este tipo de cursores cuando la



estación de trabajo cliente cuenta con suficiente memoria. Este tipo de cursor crea asignando el valor *3-adOpenStatic* en la propiedad **CursorType** del objeto Recordset.

El cursor Keyset (Conjunto de clave):

Los **cursores Keyset** (*conjunto de clave*) son los cursores más potentes y más complejos de la dispone un programador. Básicamente su comportamiento permite tanto actualizar los datos como ver los cambios que los otros usuarios puedan hacer en los registros del cursor, pero se provocará un error si accede a un registro que otro usuario haya borrado. Lo que no permite ver son registros añadidos por otros usuarios a la base de datos. El cursor Keyset (*conjunto de clave*) sólo está disponible como cursor del lado del servidor. Este tipo de cursor se crea asignando el valor *1-adOpenKeyset* en la propiedad **CursorType** del objeto Recordset.

El cursor { HYPERLINK "http://www.uysssoft.com/Docs/CursoresADO/" \I "El cursor Dynamic:" }:

El cursor dinámico es muy parecido a cursor de conjunto de claves. La diferencia está en que cada vez que el cliente solicita otro conjunto de registros el conjunto de claves se vuelve a crear antes de devolver estos registros. Esto provoca que si abrimos un conjunto de registros dinámico y contamos el número de registros, luego nos desplazamos secuencialmente hasta el final y volvemos a contar el número de registros, no tiene por que ser el mismo ya que otros usuarios pueden haber insertado registros.

No resulta sorprendente que estos cursores sean los más exigentes desde el punto de vista del rendimiento y del tráfico en la LAN porque, cada vez que se desplaza a otro registro, se necesita realizar un viaje hasta el servidor para recuperar los valores actuales. Este tipo de cursor sólo se encuentra disponible del lado del servidor.

Prueba de rendimiento

A continuación, se muestra una prueba de rendimiento encontrada en la Web donde se puede apreciar la potencia de cada tipo de cursor:

La prueba se hizo en una maquina con las siguientes características:

- Pentium 4 a 2,8 GHz con 512 Mb RAM
- Windows 2000 PRO + SP 4
- SQL Server 2000 + SP 3
- VB 6.0 + SP 5
- MDAC 2.7

La consulta fue la siguiente:

SELECT * FROM Historico (selecciona todos los registros de la tabla Historico de 22 columnas con clave principal INT IDENTITY, con 567.430 registros).

Al abrir el RecordSet y leerlo hasta el final utilizando cada cursor obtuvimos los siguientes datos:

Tipo de cursor	Duración de la consulta (en segundos)
ForwardOnly	6,02
Static	65,48
KeySet	90,13
Dynamic	89,84
Static (extremo cliente)	39,00

Ing. Carlos Manuel Rodríguez Bucarellly



- 8.4.1.3 Control de concurrencia

El control de concurrencia consiste en un conjunto de estrategias de bloqueo que utilizan los programadores para evitar que varios usuarios que acceden simultáneamente a una base de datos modifiquen al mismo tiempo un determinado registro. Los bloqueos son extremadamente necesarios para no crear bases de datos inconsistentes. Dependiendo de la forma en que desarrolle sus aplicaciones, un bloqueo puede disminuir de forma significativa el rendimiento de su aplicación, y puede, incluso, provocar errores fatales si no genera una buena estrategia de resolución de bloqueo. Cuando se bloquea un registro que está siendo modificado ningún usuario podrá acceder a dicho registro.

Mediante la propiedad **LockType** (*tipo de bloqueo*) del objeto RecordSet usted podrá indicar el tipo de bloqueo que se va a utilizar sobre los datos de la base de datos. Esta propiedad puede tomar los siguientes valores: **1-adLockReadOnly**, **2-adLockPessimistic**, **3-adLockOptimistic** y **4-adLockBatchOptimistic**.

El valor **1-adLockReadOnly** (*bloqueo de solo lectura*) es el predeterminado por ADO, que crea arreglos no actualizables. Es la opción más eficaz porque no impone sobre los datos un bloqueo de escritura. El valor **2-adLockPessimistic** (*bloqueo pesimista*), ADO bloquea el registro inmediatamente un usuario lo esté modificando. El registro estará bloqueado hasta que se ponga en marcha el método **Update** del objeto Recordset. Mientras el registro esté bloqueado ningún otro usuario podrá acceder al mismo para escribir en él, lo que reduce severamente la posibilidad de escalar la aplicación. El valor **3-adLockOptimistic** (*bloqueo optimista*) tiene un mejor comportamiento que el bloqueo pesimista, pero requiere que el programador esté más atento. Con el bloqueo optimista, ADO bloquea el registro actual sólo cuando esté siendo actualizado lo que, normalmente, tardará sólo un pequeño intervalo de tiempo. El valor **4-adLockBatchOptimistic** (*bloqueo optimista diferido*) es un modo especial de bloqueo que sólo está disponible para los cursores estáticos del lado del cliente. En los bloqueos optimistas diferidos, descargará todos los datos en la máquina cliente, permitirá que el usuario realice todas las modificaciones necesarias y, posteriormente, enviará todos los cambios en una única operación. Los bloqueos optimistas son la mejor opción si decide trabajar con cursores del lado del cliente debido a que disminuyen el tráfico en la red. El único problema de este tipo de bloqueo es que los usuarios podrán acceder a un mismo registro que está siendo usado, lo que obligará a que usted desarrolle una estrategia para evitar este y otros tipos de conflictos.

- 8.4.1.4 Lectura de los campos de un Recordset

Los nombres de los campos y los valores de cada campo del registro actual se almacenan en la colección **Fields** del objeto Recordset. Para leer uno o varios campos del registro actual deberá pasar un índice numérico o el nombre del campo, tal y como se muestra en el siguiente ejemplo:

Supóngase que tenemos una base de datos llamada agenda y dentro de esta tenemos una tabla llamada contactos. La tabla contactos está compuesta por los campos nombre, apellido, teléfono y dirección. Para leer cada uno de los campos del registro actual del objeto Recordset escribimos un código similar al que se muestra a continuación:

```
Dim cn As New ADODB.Connection      'Creamos el objeto Connection.
Dim rs As New ADODB.Recordset        'Creamos el objeto Recordset.
```

'Abrimos la base de datos "agenda.mdb".

```
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\agenda.mdb"
```

```
rs.Source = "contactos"              'Especificamos la fuente de datos. En este caso la tabla "contactos".
```

```
rs.CursorType = adOpenKeyset         'Definimos el tipo de cursor.
```

```
rs.LockType = adLockOptimistic       'Definimos el tipo de bloqueo.
```

```
rs.Open "select * from contactos", cn 'Abrimos el Recordset y lo llenamos con una consulta SQL.
```




'Agregamos los campos del registro actual al control ListBox.

```
List1.AddItem rs.Fields(0) & " " & rs.Fields(1) & " " & rs.Fields(2) & " " & rs.Fields(3)
```

En el ejemplo anterior agregamos el primer registro del Recordset al control ListBox, debido a que es el primer registro seleccionado cuando cargamos por primera vez el Recordset. Los valores dentro de los paréntesis representan los campos o los valores de cada campo de la tabla. En la colección Fields el orden de los campos comienzan por cero. En nuestro caso, el valor 0 representa el primer campo, es decir, el nombre, el valor 2 el apellido, y así sucesivamente.

Otra forma de leer los valores de cada campo es especificando el nombre del campo en vez de un número, por ejemplo:

```
List1.AddItem rs.Fields("nombre") & " " & rs.Fields("apellido") & " " & rs.Fields("telefono") & " " & rs.Fields("direccion")
```

Los valores de cada campo del registro seleccionado se almacena en la propiedad **Value** de la colección Fields, aunque este no es necesario especificarlo debido a que es el valor predeterminado por ADO. Por ejemplo, el código explícito para leer los valores de cada campo sería:

```
List1.AddItem rs.Fields("nombre").Value & " " & rs.Fields("apellido").Value & " " & rs.Fields("telefono").Value & " " & rs.Fields("direccion").Value
```

La colección Fields también permite leer el nombre de cada campo del registro actual. Esto es posible mediante la propiedad **Name**. Por ejemplo, si se quiere agregar al ListBox el nombre del campo y su valor al lado hacemos lo siguiente:

```
List1.AddItem rs.Fields(0).Name & "=" & rs.Fields(0).Value & " " & rs.Fields(1).Name & "=" & rs.Fields(1).Value & " " & rs.Fields(2).Name & "=" & rs.Fields(2).Value & " " & rs.Fields(3).Name & "=" & rs.Fields(3).Value
```

Si no conoce el número de campos que contiene el registro puede utilizar la propiedad Count para leer cada uno de los campos, ejemplo:

Dim i As Long

For i = 0 To rs.Fields.Count - 1

```
List1.AddItem rs.Fields(i).Name & "=" & rs.Fields(i).Value
```

Next i

Si queremos leer todos los registros hasta el final del Recordset tendremos que consultar la propiedad **EOF** (*fin del archivo*) y asociarla a un bucle para repetir hasta que se lean todos los registros, ejemplo:

rs.MoveFirst 'Nos posicionamos en el primer registro del Recordset.

Do Until rs.EOF 'Repite hasta que se lea todo el Recordset.

```
List1.AddItem rs.Fields(0) & " " & rs.Fields(1) & " " & rs.Fields(2) & " " & rs.Fields(3)
```

```
rs.MoveNext      'Nos movemos al siguiente registro.
```

Loop

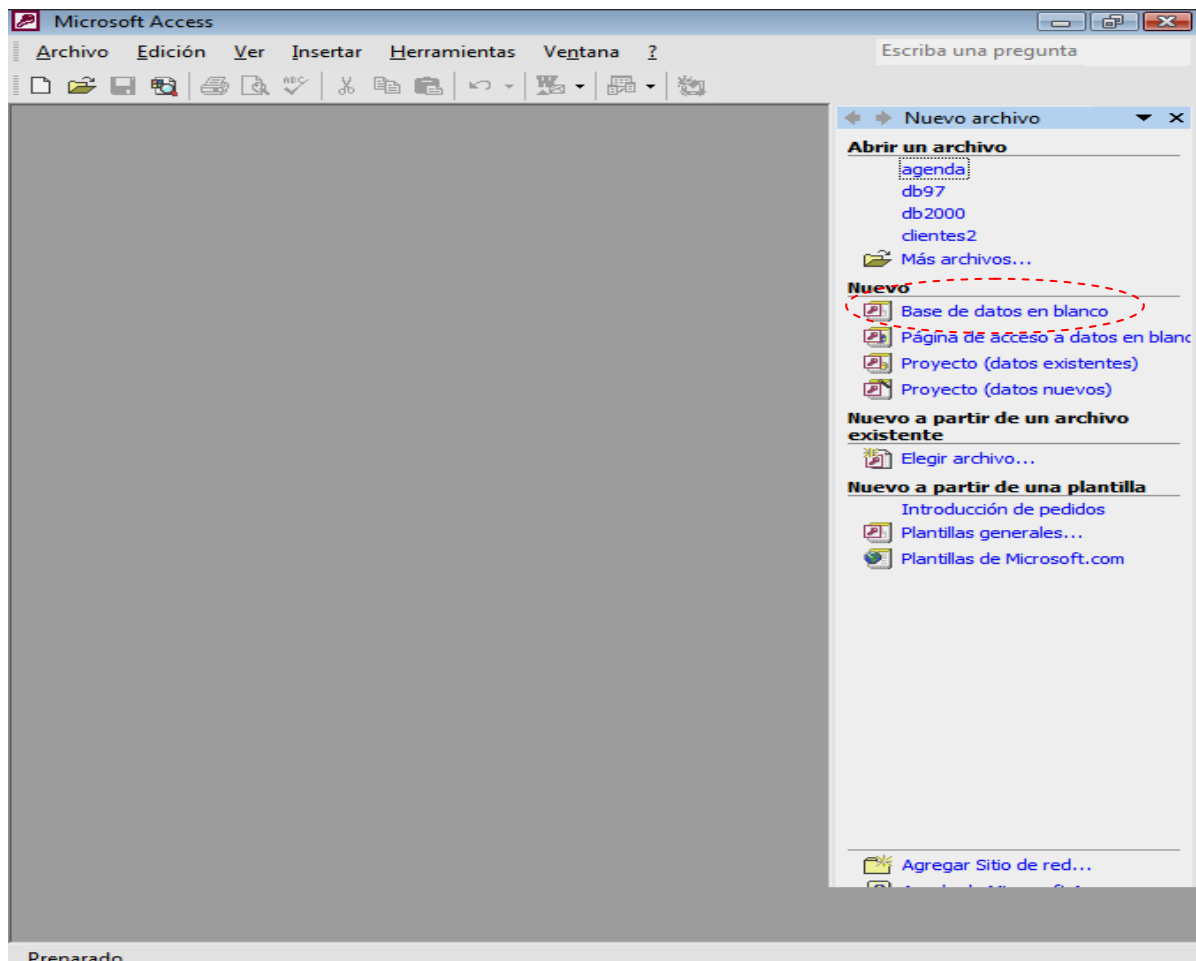


Capítulo 8

Después de haber visto las distintas formas de cómo abrir una base de datos y la forma de explorar los registros de un Recordset estamos lista para hacer nuestros primeros ejercicios con ADO.

En nuestro primer ejercicio crearemos una base de datos en MS Access donde almacenaremos el nombre, apellido, teléfono y dirección de nuestros contactos. La base de datos llevará por nombre agenda y la tabla se llamará contactos. Para realizar este ejercicio tendremos que utilizar MS Access aunque supongo que tiene los conocimientos básicos de este SGBD explicaré detalladamente los pasos para crear la base de datos.

- Iniciemos Access haciendo clic en el menú Inicio → Programas → Microsoft Office → Microsoft Access. En algunas maquinas solo tendremos que realizar tres pasos: Inicio → Programas → Microsoft Access.
- Al iniciar Access aparecerá la siguiente pantalla:

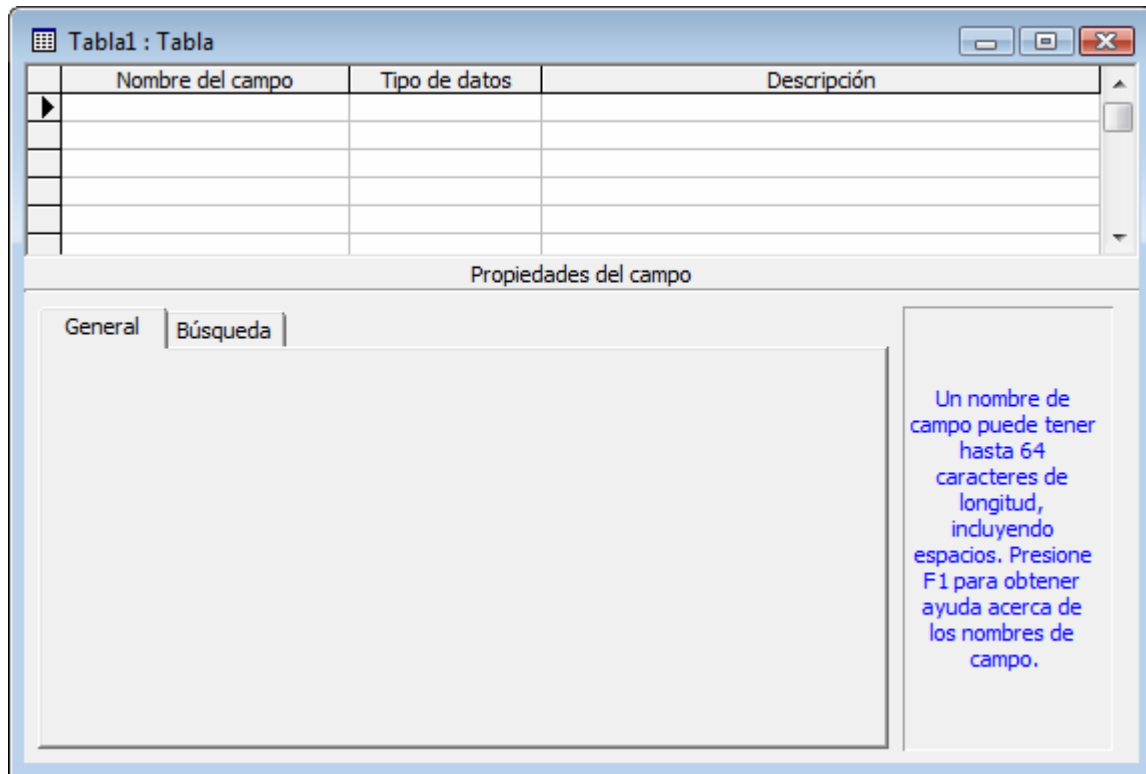


- Haga clic en [Base de datos en blanco](#).
- Aparecerá una ventana solicitando el nombre de la base de datos. El nombre por defecto es bd1, borre ese nombre y escriba **c:\agenda**. Escribimos c:\ para que la base de datos se almacene en el disco local c.
- Aparecerá una ventana con tres opciones: *Crear una tabla en vista de diseño*, *Crear una tabla utilizando el asistente* y *Crear una tabla introduciendo datos*. De estas tres opciones las que nos interesa es la primera. Haga clic sobre la primera opción *Crear una tabla en vista de diseño*.



Capítulo 8

- Aparecerá la ventana que nos permitirá crear la tabla de la base de datos. Esta ventana esta compuesta por tres columnas y la sección de propiedades de los campos. En la primera columna especificaremos el nombre de los campos de la tabla y en la segunda columna el tipo de datos para cada campo. En la columna descripción no especificaremos nada.



En la primera columna escriba:

Nombre del campo
Nombre
Apellido
Telefono
Direccion
Correo

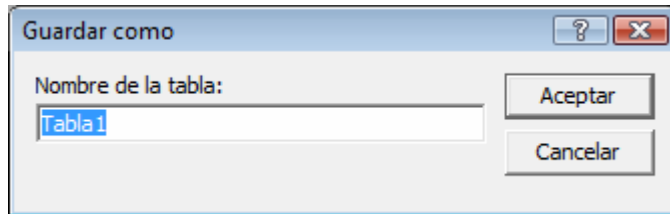
En la segunda columna seleccione los siguientes tipos de datos:

Tipo de datos
Texto
Texto
Texto
Texto
Texto

Algunas versiones de Access agregan automáticamente el tipo de dato **Texto** a los campos agregados a la tabla. Si la versión de Access agrega el tipo de datos **Texto** no tendrá que especificarlo. Si no esta familiarizado con lo que estamos haciendo les recomiendo un curso básico de base de datos. De todos modos les explico: *estamos creando la tabla de la base de datos especificando los campos y el tipo de datos para cada campo.*

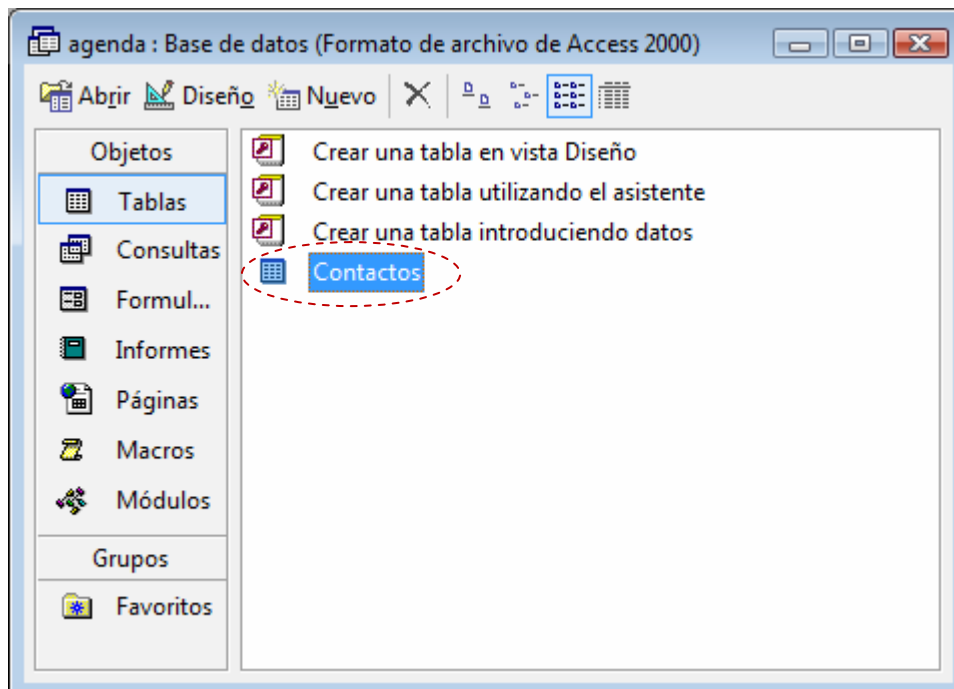


Ahora estamos listos para almacenar la tabla en la base de datos. Haga clic en el botón guardar de la barra de herramientas estándar. Aparecerá un cuadro solicitando el nombre de la tabla:



Borre el texto Tabla1 y escriba **Contactos**. Luego haga clic en el botón *Aceptar*. Access les preguntará si desea crear una clave principal. Conteste que **No**.

La tabla se agregará en la ventana de objetos:



Haga doble clic en la tabla Contactos. Access les mostrará la tabla para que usted agregue registros a la misma. En nuestro caso agregaremos cuatro registros a la tabla para que pueda ver cómo podemos visualizarlos y modificarlos desde Visual Basic. No piense que utilizaremos Access para agregar información a la tabla, esto lo haremos desde nuestra aplicación en Visual Basic. En nuestro caso hemos agregado información (registros) para que observe como se puede navegar por la base de datos desde una aplicación en Visual Basic.

Complete la tabla con los siguientes registros:

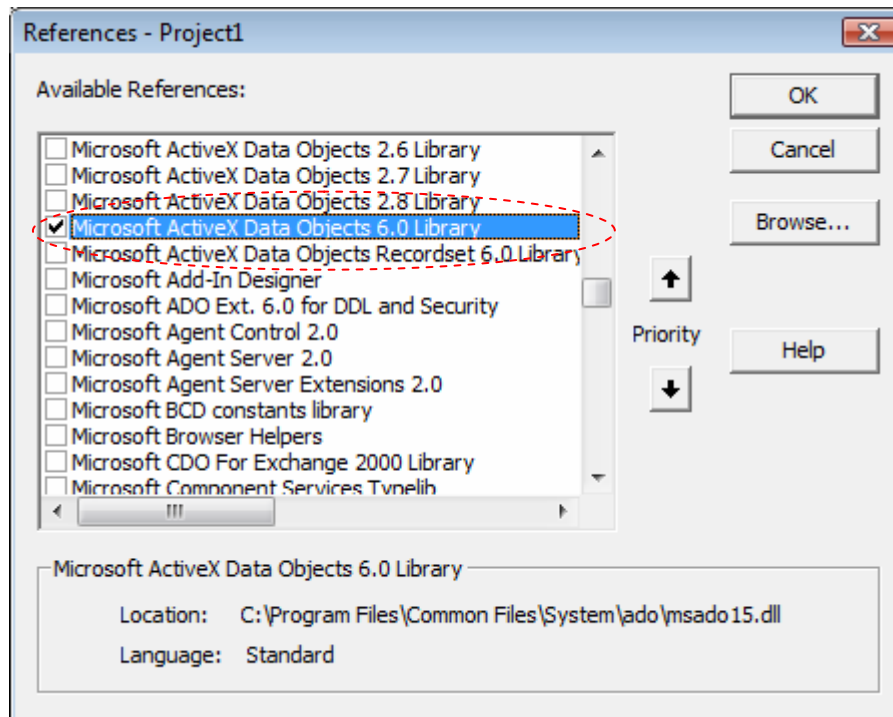
	Nombre	Apellido	Telefono	Direccion	Correo
	Carlos	Bucarelly	809-598-5954	Calle #5, Villa Faro	carlos@hotmail.com
	Pablo	Rodríguez	809-596-5952	Urb. Alma Rosa, Calle #2	pablo@hotmail.com
	Nelson	Rodríguez	809-598-5963	Urb. Capotillo, Villa Faro	nelson@hotmail.com
	José	Peréz	829-526-5236	Calle Terminal Club #1	jose@hotmail.com
*					



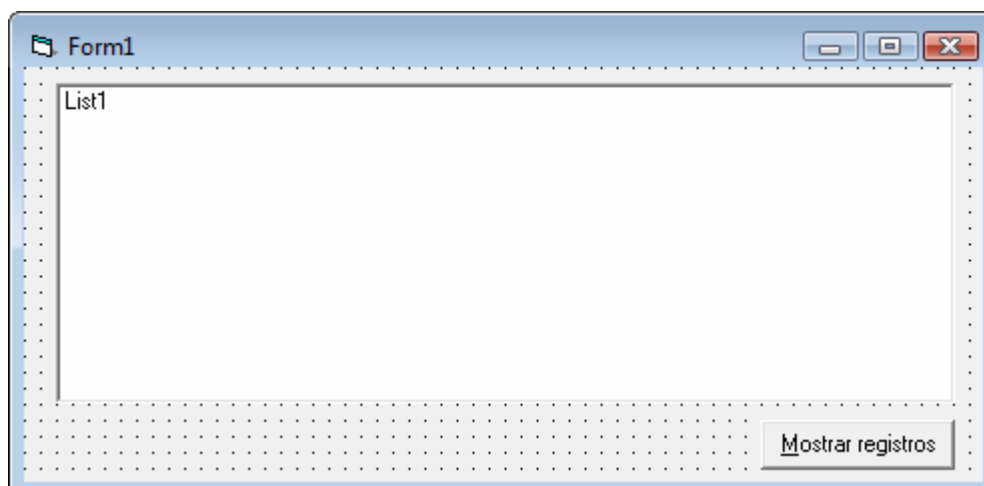
Ahora que hemos agregado información en la tabla, haga clic en el botón Guardar. Los datos serán almacenados en la tabla. Después de esto, cierre el programa Access.

Diseñemos ahora nuestra aplicación en Visual Basic para leer nuestra base de datos.

- Abra un nuevo proyecto.
- Haga clic en el menú **Project (Proyecto)**. Seleccione la opción **References... (Referencias)**. Aparecerá la ventana de referencias:



- Busque la referencia **ActiveX Data Objects 6.0 Library** y selecciónela. A continuación, haga clic en el botón **OK**. Esto que estamos haciendo es necesario para poder utilizar ADO desde nuestra aplicación. Si no tiene la versión **ActiveX Data Objects 6.0 Library** seleccione la versión más actual o la mayor.
- Inserte una ListBox y un botón de comando en el formulario:





- En el evento **Click** del botón *Mostrar registros* escriba:

```
Dim cn As New ADODB.Connection      'Creamos el objeto Connection.
Dim rs As New ADODB.Recordset       'Creamos el objeto Recordset.

'Abrimos la base de datos "agenda.mdb".
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\agenda.mdb"
rs.Source = "contactos"             'Especificamos la fuente de datos. En este caso la tabla "contactos".
rs.CursorType = adOpenKeyset        'Definimos el tipo de cursor.
rs.LockType = adLockOptimistic      'Definimos el tipo de bloqueo.
rs.Open "select * from contactos", cn 'Abrimos el Recordset y lo llenamos con una consulta SQL.

rs.MoveFirst                        'Nos posicionamos en el primer registro del Recordset.
Do Until rs.EOF                    'Repite hasta que se lea todo el Recordset.
    List1.AddItem rs.Fields("nombre") & " " & rs.Fields("apellido") & " " & rs.Fields("telefono") & " " & _
    rs.Fields("direccion") & " " & rs.Fields("correo")

    rs.MoveNext                    'Nos movemos al siguiente registro.
Loop
```

- Corra la aplicación.
- Haga clic en el botón *Mostrar registros*. Si todo esta bien, se mostrarán en la ListBox todos los registros que se agregaron en Access.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio1-8** para el *formulario* y **Ejercicio1-8** para el proyecto.

- 8.4.1.5 Moverse por los registros de un Recordset

Una forma más elegante y profesional para visualizar los registros de un Recordset es moverse por el Recordset cada vez que sea necesario, es decir, avanzar o retroceder uno o varios registros. Para tal fin, el objeto Recordset dispone de los métodos **MoveFirst** (mueve al primer registro), **MovePrevious** (mueve al registro anterior), **MoveNext** (mueve al siguiente registro) y **MoveLast** (mueve al último registro).

Para que pueda entender el correcto funcionamiento de estos métodos crearemos una aplicación que contendrá cuatro botones de comando que permitirán desplazarse por cada uno de los registros del Recordset. Para crear nuestra segunda aplicación haga lo siguiente:

- Abra un nuevo proyecto.
- Haga clic en el menú **Project (Proyecto)** y ejecute la opción **References... (Referencias...)**. En el cuadro que aparece busque la referencia **ActiveX Data Objects 6.0 Library** y selecciónela. A continuación, haga clic en el botón **OK**. Si no tiene la versión **ActiveX Data Objects 6.0 Library** seleccione la versión más actual o la mayor.
- Inserte cuatro etiquetas y al lado de cada etiqueta una caja de texto. También, inserte cinco botones de comando en la parte inferior del formulario. Tal y como se muestra en la imagen de la siguiente página...



- Establezca los valores necesarios para que los controles tengan la misma apariencia que la imagen anterior.
- Haga doble en cualquier parte del formulario y en la sección general (**no en el evento Load**) escriba:

```
Dim cn As New ADODB.Connection      'Creamos el objeto Connection.
Private WithEvents rs As ADODB.Recordset 'Creamos el Recordset con soporte de eventos.
```

- Dentro del evento **Load** del formulario escriba:

```
Set rs = New ADODB.Recordset 'Activamos el Recordset.

'Abrimos la base de datos "agenda.mdb"
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\agenda.mdb"

'Especificamos la fuente de datos. En este caso la tabla "contactos".
rs.Source = "contactos"
rs.CursorType = adOpenKeyset 'Definimos el tipo de cursor.
rs.LockType = adLockOptimistic 'Definimos el tipo de bloqueo.

'Abrimos el Recordset y lo llenamos con una consulta SQL.
rs.Open "select * from contactos", cn

rs.MoveFirst 'Nos movemos al principio del Recordset.

'Cargamos los datos en las cajas de texto.
Text1.Text = rs.Fields("Nombre") 'Ponemos el nombre del registro actual.
Text2.Text = rs.Fields("Apellido") 'Ponemos el apellido del registro actual.
Text3.Text = rs.Fields("Telefono") 'Ponemos el teléfono del registro actual.
Text4.Text = rs.Fields("Direccion") 'Ponemos la dirección del registro actual.
Text5.Text = rs.Fields("Correo") 'Ponemos el correo del registro actual.
```



- Dentro del evento **Click** del botón *Primero* escriba:

```
'Movemos al primer registro del Recordset.
rs.MoveFirst
```

- Dentro del evento **Click** del botón *Anterior* escriba:

```
'Verificamos si no estamos antes del principio del Recordset.
If rs.BOF = False Then
    rs.MovePrevious 'Movemos al registro anterior.
End If
```

- Dentro del evento **Click** del botón *Siguiente* escriba:

```
'Verificamos si no estamos después del último elemento del Recordset.
If rs.EOF = False Then
    rs.MoveNext 'Movemos al siguiente registro.
End If
```

- Dentro del evento **Click** del botón *Último* escriba:

```
'Movemos al último registro del Recordset.
rs.MoveLast
```

- Dentro del evento **Click** del botón *Salir* escriba:

```
rs.close      'Cerramos el Recordset.
cn.close      'Cerramos la conexión.

'Salimos de la aplicación.
End
```

Ahora vamos a escribir el evento que ocurre cada vez que el usuario se mueve por el Recordset. Este es el evento **MoveComplete** (Movimiento completo). Este evento nos permite determinar si el usuario se movió a otro registro del Recordset para luego mostrarlo al usuario, ya sea en cajas de textos, en un *grid* o en un control *Listview*.

- Para crear nuestro evento **MoveComplete** haga doble clic en cualquier parte del formulario para cargar el editor de código. Luego desplácese al final del editor de código y haga clic en la zona blanca para colocar el punto de inserción o indicador y escriba:

```
Private Sub rs_MoveComplete(ByVal adReason As ADODB.EventReasonEnum, ByVal pError
As ADODB.Error, adStatus As ADODB.EventStatusEnum, ByVal pRecordset As
ADODB.Recordset)

'Si estamos antes del principio del Recordset.
If rs.BOF = True Then
    rs.MoveFirst 'Movemos al principio del Recordset.

'Si estamos después del ultimo elemento del Recordset.
Elseif rs.EOF = True Then
    rs.MoveLast 'Movemos al final del Recordset.
```



```

Else
'Si se cambio del registro actual del Recordset a otro.
Text1.Text = rs.Fields("Nombre") 'Ponemos el nombre del registro actual.
Text2.Text = rs.Fields("Apellido") 'Ponemos el apellido del registro actual.
Text3.Text = rs.Fields("Telefono") 'Ponemos el teléfono del registro actual.
Text4.Text = rs.Fields("Direccion") 'Ponemos la dirección del registro actual.
Text5.Text = rs.Fields("Correo") 'Ponemos el correo del registro actual.
End If

End Sub
    
```

- Corra la aplicación. Utilice los botones para desplazarse por los registros de la base de datos.
- Detenga la aplicación y guárdela con los nombres **FormEjercicio2-8** para el *formulario* y **Ejercicio2-8** para el proyecto.

- 8.4.1.6 Modificación de registros en un Recordset

La modificación de un registro consiste en la actualización de uno o más campos del registro que se encuentra cargado en el Recordset. Esto es posible mediante el método **Update** del objeto Recordset que permite la actualización simultanea de varios registro, utilizando la siguiente sintaxis:

Update [Campos], [Valores]

Campos: Este argumento es un tipo Variant que contiene el nombre de un campo, un índice de campo o un array de nombres de campo o índices.

Valores: Este argumento es un tipo Variant que contiene un valor único o un array de valores. En este se deben especificar los nuevos valores que serán establecidos en el argumento *Campos*.

Ambos argumentos son opcionales, pero solo uno de ellos podrá omitir. Si incluye ambos argumentos deberá especificar los mismos números de campos y valores en ambos argumentos. Por ejemplo, si va a modificar dos campos uno llamado *Nombre* y otro *Apellido* deberá especificar los valores que tendrán ambos campos en el argumento *Valores*. El siguiente ejemplo muestra la forma en que usted puede actualizar varios campos utilizando la siguiente sintaxis:

'Actualizar cinco campos en una sola operación.

```
rs.Update Array("Nombre", "Apellido", "Telefono", "Direccion", "Correo"), _
    Array("Carlos", "Rodríguez", "809-598-3696", "Calle #6, Alma Rosa", "carlos@gmail.com")
```

rs.move 0 'Hace que el registro se actualice.

Al utilizar el método Update el registro no se modifica inmediatamente hasta que se desplace a otro registro del Recordset o hasta que se emplee un método como el utilizado en el ejemplo anterior.

ADO dispone del método **CancelUpdate** para cancelar una operación de actualización que se halla realizado sobre un registro y dejarlo como estaba al principio. Si utiliza conjuntamente el método **Update** con el método **CancelUpdate** podrá ofrecer al usuario la posibilidad de confirmar o cancelar las modificaciones realizadas sobre el registro activo:

```

'Si el registro aun no se ha modificado.
If rs.EditMode = adEditInProgress Then
    
```



Dim Respuesta

Respuesta = MsgBox("¿Desea guardar los cambios realizados?", vbYesNo)

```
If Respuesta = vbYes Then
    rs.Update          'Actualizamos el registro.
Else
    rs.CancelUpdate    'Cancelamos la actualización.
End If
```

Practiquemos como actualizar un registro de nuestra base de datos agregando un botón de comando a nuestra aplicación anterior. Escribe en la propiedad **Caption** el texto *Actualizar*. Dentro del evento **Click** del nuevo botón escriba:

```
rs.Update Array("nombre", "apellido", "telefono", "direccion", "correo"), _
    Array(Text1.Text, Text2.Text, Text3.Text, Text4.Text, Text5.Text)

rs.Update 'Actualizamos el registro.

'Verificamos si no ocurrió ningún problema.
If rs.State = 1 Or rs.State = 0 Then
    MsgBox ("El registro se ha actualizado con éxito.")
Else
    MsgBox ("Ha ocurrido un error al actualizar el registro.")
End If
```

▪ Ahora podrá modificar los datos de cada campo y luego actualizarlo haciendo clic sobre el botón Actualizar.

- 8.4.1.7 Eliminar el registro activo del Recordset

Podrá eliminar el registro actual cargado en el Recordset utilizando el método **Delete** mediante la siguiente sintaxis:

```
rs.Delete [AffectRecords]
```

AffectRecords (Registros afectados) es opcional, si este se omite se borrará el registro actual. El argumento *AffectRecords* es utilizando cuando queremos borrar varios registros que resultan de una consulta realizada sobre la base de datos.

Para borrar el registro actual escriba:

```
rs.Delete
rs.MoveNext 'Nos movemos al siguiente registro para no provocar un error.
If rs.EOF Then rs.MoveLast 'Si es el fin del archivo nos movemos al último registro.
```

- 8.4.1.8 Inserción de registros en el Recordset

Para agregar nuevos registros al Recordset utilizamos el método **AddNew**. Su empleo es bastante sencillo, tal y como se muestra en el siguiente ejemplo:

```
rs.AddNew          'Llamamos el método.
rs("nombre") = "Juan" 'Agregamos un nombre en el campo nombre.
rs("apellido") = "Rodríguez" 'Agregamos un apellido en el campo apellido.
rs.Update          'Actualizamos.
```




En el ejemplo anterior hemos agregados los datos directamente para cada campo, pero puede almacenar los datos contenidos en cajas de texto, una variable u otro objeto. El siguiente ejemplo muestra como agregar un nuevo registro con valores obtenidos desde cajas de texto.

```
rs.AddNew
rs("nombre") = txtNombre.Text
rs("apellido") = txtApellido.Text
rs("telefono") = txtTelefono.Text
rs("direccion") = txtDireccion.Text
rs("correo") = txtCorreo.Text
rs.Update
```

Para el ejemplo anterior debe tener en cuenta que las cajas de texto estén etiquetadas en la propiedad **Name** con los nombres especificados. Si agregamos un botón de comando en nuestra aplicación donde las cajas de texto tienen los nombres por defecto el código entonces sería el siguiente:

```
rs.AddNew
rs("nombre") = Text1.Text
rs("apellido") = Text2.Text
rs("telefono") = Text3.Text
rs("direccion") = Text4.Text
rs("correo") = Text5.Text
rs.Update
```

- 8.4.1.8 Establecer y leer la posición de un registro en el Recordset

El objeto Recordset dispone de algunas propiedades que les ayudarán a saber en que lugar del Recordset se encuentra. Este es muy importante cuando quiera activar o desactivar ciertas operaciones o definir marcadores con el propósito de volver rápidamente a un registro que haya visitado.

La primera propiedad a analizar es la propiedad **EOF** (*End of file*), que devuelve un valor verdadero cuando el puntero del registro actual se encuentra situado después del final del Recordset. Esta propiedad es útil cuando recorra todos los registros del **Recordset** utilizando un bucle:

```
'Contar todos los empleados que ganen mas de 15,000 pesos.
rs.MoveFirst 'Nos movemos al principio del Recordset.
Dim Contador As Long

Do Until rs.EOF 'Repetimos hasta el final del Recordset.
    'Verificamos el sueldo del registro actual.
    If rs("sueldo") > 15000 Then Contador = Contador + 1
    rs.MoveNext 'Nos movemos al siguiente registro.
Loop
```

Otro ejemplo:

```
'Contar todos los empleados contratados antes del 1 de enero de 1994.
rs.MoveFirst 'Nos movemos al principio del Recordset.
Dim Contador As Long

Do Until rs.EOF 'Repetimos hasta el final del Recordset.
    'Verificamos la fecha de contrato del registro actual.
    If rs("FechaContrato") < #1/1/1994# Then Contador = Contador + 1
    rs.MoveNext 'Nos movemos al siguiente registro.
Loop
```



El objeto Recordset también dispone de la propiedad **BOF** (*Begin of file*) que es similar a la propiedad **EOF**. Esta devuelve un valor verdadero cuando el puntero del Recordset se encuentra antes del primer registro. Con frecuencia es crucial conocer los valores de la propiedad **EOF** y **BOF**, debido a que cuando una de ellas es **True** (verdadera), la mayoría de los métodos y propiedades devuelven un error porque no existe un registro actual. Por ejemplo, si trata de leer el valor de un campo de algún registro y no verifica primero si una de estas propiedades es **True** se provocará entonces un error en tiempo de ejecución y la aplicación finalizará. Si las dos propiedades, **BOF** y **EOF**, son **True**, entonces el Recordset estará vacío.

Cada registro en el Recordset tiene un valor de tipo *Variant* que usted podrá leer con la propiedad **Bookmark** y luego usarlo para volver rápidamente a dicho registro. Solo debe reasignar el mismo valor a la propiedad **Bookmark**, tal y como se muestra en el siguiente código:

Dim posicion As Variant

posicion = rs.Bookmark

rs.movelast

'Leemos la posición del registro actual.

'Nos desplazamos al último registro. Puede ser a cualquier otro.

rs.Bookmark = posicion

'Vuelve al registro marcado.

- 8.4.1.9 Ordenación de los registros de un Recordset

Podrá ordenar los registros contenidos en un Recordset utilizando la propiedad **Sort**. Para esto, se debe indicar el nombre o los nombres de los campos por el cual se ordenará la columna, por ejemplo:

rs.Sort = "nombre" 'Ordena el Recordset por el campo nombre.

Podemos indicar dos campos para ordenar los registros del Recordset como se muestra en el siguiente ejemplo:

rs.Sort = "nombre, apellido" 'Ordena el Recordset por el campo nombre y apellido.

Los registros se ordenan automáticamente de forma ascendente, pero podrá elegir un orden descendente utilizando el calificador **DESC**:

'Ordena el Recordset empezando con los empleados con mayores sueldos hasta los menores.

rs.Sort = "Sueldo DESC"

- 8.4.1.10 Búsqueda de registros

El objeto Recordset dispone de un método muy sencillo que permite localizar un registro que cumpla un determinado criterio de selección. Este es el método **Find** que tiene la siguiente sintaxis:

Find *CriterioBúsqueda*, [*RegistrosOmitidos*], [*DirecciónBúsqueda*], [*Inicio*]

CriterioBúsqueda es una cadena que contiene la condición de búsqueda, que constará de un nombre de campo seguido de un operador y de un valor. Los operadores que podrá utilizar son = (igual), < (menor que), > (mayor que) y **LIKE** (Patrón de coincidencia). El valor puede ser una cadena encerrada entre comilla, un numero, el valor de una variable, el valor de un objeto o una fecha encerrada entre caracteres #. *RegistrosOmitidos* es un número opcional que indica la cantidad de registros que se van a ignorar antes de comenzar la búsqueda. *DirecciónBúsqueda* indica la dirección en la que se procesará la búsqueda; los valores admitidos son **1-adSearchForward** (*Búsqueda hacia delante, valor predeterminado*) o **1-adSearchBackward** (*Búsqueda hacia atrás*). *Inicio* es un marcador opcional que especifica el registro desde el que se debe comenzar la búsqueda, el valor predeterminado es el registro actual. Podrá omitir todos los parámetros excepto el primero, en este caso la búsqueda iniciaría desde el registro actual.



Capítulo 8

Si la búsqueda tiene éxito, entonces, el registro que cumpla con el criterio establecido se convertirá en el registro activo; si la búsqueda no tiene éxito, el registro actual se convertirá en el último registro del Recordset. Deberá verificar el estado de las propiedades **BOF** y **EOF** antes de leer el valor de los campos del registro cuando realice una búsqueda, de lo contrario provocará un error y abortará la aplicación. Cuando la búsqueda tiene éxito ambas propiedades contienen el valor **False**.

En el siguiente ejercicio verá como utilizar el método **Find** para localizar uno o varios registros en el Recordset.

- Abra un nuevo proyecto. Agregue la referencia a la base de datos en el menú **Project\References...**: *Microsoft ActiveX Data Objects 2.8 Library*. Puede agregar la más reciente.
- Identifique los controles que se muestra en la siguiente imagen y establezca los valores necesarios en las propiedades requeridas.

- Seleccione la caja combinada (ComboBox). En la propiedad **List** escriba los siguientes valores:

- Seleccione la primera caja de texto y en la propiedad **Name** escriba: txtBuscarTexto
- Seleccione la segunda caja de texto y en la propiedad **Name** escriba: txtNombre
- Seleccione la tercera caja de texto y en la propiedad **Name** escriba: txtApellido
- Seleccione la cuarta caja de texto y en la propiedad **Name** escriba: txtTelefono
- Seleccione la quinta caja de texto y en la propiedad **Name** escriba: txtDireccion
- Seleccione la sexta caja de texto y en la propiedad **Name** escriba: txtCorreo



- Dentro del evento **Click** del botón *Buscar* escriba:

```
'Comprobamos que la caja combinada de la búsqueda no se este vacía.
If Len(Trim(Combo1.Text)) = 0 Then
    MsgBox ("Debe especificar el tipo de búsqueda")
    Combo1.SetFocus

'Verificamos si la caja del texto a buscar esta vacía.
Elseif Len(Trim(txtBuscarTexto.Text)) = 0 Then
    MsgBox ("Debe especificar el texto a buscar")
    txtBuscarTexto.SetFocus
Else
    'Verificamos el tipo de búsqueda.
    If LCase(Combo1.Text) = LCase("Apellido") Then
        rs.Find "Apellido = '" & txtBuscarTexto.Text & "'", , , 1 'Buscamos por el Apellido.
    End If

    If LCase(Combo1.Text) = LCase("Nombre") Then
        rs.Find "Nombre = '" & txtBuscarTexto.Text & "'", , , 1 'Buscamos por el Nombre.
    End If

    If LCase(Combo1.Text) = LCase("Telefono") Then
        rs.Find "Telefono = '" & txtBuscarTexto.Text & "'", , , 1 'Buscamos por el Teléfono.
    End If

    'Verificamos si la búsqueda tiene éxito y ponemos los datos en las cajas de texto.
    If rs.BOF = False And rs.EOF = False Then
        txtNombre.Text = rs.Fields("Nombre") 'Ponemos el nombre del registro actual.
        txtApellido.Text = rs.Fields("Apellido") 'Ponemos el apellido del registro actual.
        txtTelefono.Text = rs.Fields("Telefono") 'Ponemos el teléfono del registro actual.
        txtDireccion.Text = rs.Fields("Direccion") 'Ponemos la dirección del registro actual.
        txtCorreo.Text = rs.Fields("Correo") 'Ponemos el correo del registro actual.
    Else
        'Mostramos un mensaje si la búsqueda no tiene éxito.
        MsgBox ("No se ha podido localizar el registro con el parámetro especificado")
    End If
End If
```

- En la sección *General* del formulario escriba:

```
Dim cn As New ADODB.Connection 'Creamos el objeto Connection.
Private WithEvents rs As ADODB.Recordset 'Creamos el Recordset con soporte de eventos.
```

- Escriba dentro del evento **Load** del formulario el siguiente bloque de código:

```
Set rs = New ADODB.Recordset

'Abrimos la base de datos "agenda.mdb".
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\agenda.mdb"
rs.Source = "contactos" 'Especificamos la fuente de datos. En este caso la tabla "contactos".
rs.CursorType = adOpenKeyset 'Definimos el tipo de cursor.
rs.LockType = adLockOptimistic 'Definimos el tipo de bloqueo.
rs.Open "select * from contactos", cn 'Abrimos el Recordset y lo llenamos con una consulta SQL.

'Cargamos los datos en las cajas de texto.
rs.MoveFirst 'Nos movemos al principio del Recordset.
```



- Dentro del evento **Click** del botón *Salir* escriba:

```
'Sale de la aplicación.
End
```

- Corra la aplicación.

Ahora, para probar nuestra aplicación seleccione el tipo de búsqueda y el texto que desea buscar. Debe tener en cuenta que los datos que especifique en la caja de texto deben estar en la base de datos que creamos al principio, de lo contrario la búsqueda no tendrá éxito. Recuerde los datos que contiene la base de datos son los que se muestran a continuación:

	Nombre	Apellido	Telefono	Direccion	Correo
	Carlos	Bucarely	809-598-5954	Calle #5, Villa Faro	carlos@hotmail.com
	Pablo	Rodríguez	809-596-5952	Urb. Alma Rosa, Calle #2	pablo@hotmail.com
	Nelson	Rodríguez	809-598-5963	Urb. Capotillo, Villa Faro	nelson@hotmail.com
	José	Peréz	829-526-5236	Calle Terminal Club #1	jose@hotmail.com
*					

Si en el tipo de búsqueda usted especificar Nombre, entonces, en la caja del texto a buscar podrá escribir uno de los nombres de la primera columna. De igual manera podrá hacer con el Apellido y el Teléfono.

Si existen más de un registro con el mismo parámetro de búsqueda especificado, solo el primero de ellos será el que se active. Esto es, debido a que hemos especificado el valor 1 (*busca siempre desde el principio*) en el argumento *Inicio* del método **Find**. Si desea que la búsqueda continúe a partir del registro actual cada vez que hagamos clic en el botón *Buscar*, entonces, deberá omitir el valor del argumento *Inicio* y especificar el valor 1 en el argumento *RegistrosOmitidos*, ejemplo:

```
rs.Find "Apellido = " & txtBuscarTexto.Text & "", 1
```

```
rs.Find "Nombre = " & txtBuscarTexto.Text & "", 1
```

```
rs.Find "Telefono = " & txtBuscarTexto.Text & "", 1
```

- Detenga la aplicación y guárdela con los nombres **FormEjercicio3-8** para el *formulario* y **Ejercicio3-8** para el proyecto.

Podrá utilizar un bucle para recorrer todos los registros que cumplan un determinado criterio, tal y como se muestra en el siguiente ejemplo:

```
'Buscar todos los empleados que hayan sido contratados después del 1 de enero de 2003.
```

```
rs.MoveFirst 'Nos colocamos al principio del Recordset.
rs.Find "FechaContrato > #1/1/2003#" 'Hacemos nuestra primera búsqueda.
```

```
Do Until rs.EOF
```

```
    'Si encontramos los registros los vamos agregando en una ListBox.
    List1.AddItem rs("Nombre"), rs("Apellido"), rs("FechaContrato")
```

```
    'Buscar el siguiente registro que cumpla el criterio de búsqueda pero ignoramos el actual.
    rs.Find "FechaContrato > #1/1/2003#"
Loop
```



Para realizar búsquedas más complejas puede utilizar el operador **LIKE**. Este operador acepta dos símbolos comodines: el asterisco (*) equivale a ninguno o más caracteres y el subrayado (_) equivale exactamente a un único carácter. A continuación algunos ejemplos:

rs.Find "Nombre LIKE 'C*'" 'Buscar registros cuyo nombre comience con la letra C.
rs.Find "Nombre LIKE '*A*'" 'Buscar registros donde la segunda letra de los nombres sea la A.

- 8.4.1.11 Verificar el estado del Recordset

En muchas ocasiones necesitará verificar el estado del Recordset para mostrar los resultados a los usuarios de alguna operación, o bien, para verificar si alguna operación tuvo o no éxito. Para consultar el estado del Recordset verifique la propiedad **Status** (utilizando una sentencia **If** o **Case**). En esta propiedad puede tener uno de los siguientes valores:

Constante	Valor	Descripción
adRecOK	0	Se ha actualizado el registro con éxito.
adRecNew	1	El registro es nuevo.
adRecModified	2	Se ha modificado el registro.
adRecDeleted	4	Se ha borrado el registro.
adRecUnmodified	8	El registro no se ha modificado.
adRecInvalid	&H10	No se ha guardado el registro porque su marcador no es válido.
adRecMultipleChanges	&H40	No se ha guardado el registro porque afecta a varios registros.
adRecPendingChanges	&H80	No se ha modificado el registro porque hace referencia a una inserción pendiente.
adRecCanceled	&H100	No se ha guardado el registro porque se ha cancelado la operación.
adRecCantRelease	&H400	No se ha guardado el registro porque está bloqueado.
adRecConcurrencyViolation	&H800	No se ha guardado el registro porque se estaba usando una concurrencia optimista.
adRecIntegrityViolation	&H1000	No se ha guardado el registro porque violaría las restricciones de integridad.
adRecMaxChangesExceeded	&H2000	No se ha guardado el registro porque existen demasiados cambios pendientes.
adRecObjectOpen	&H4000	No se ha guardado el registro porque existe un conflicto con un objeto de almacenamiento que se encuentra abierto.
adRecOutOfMemory	&H8000	No se ha guardado el registro porque se ha producido un error del tipo fuera de memoria.
adRecPermissionDenied	&H10000	No se ha guardado el registro porque el usuario no cuenta con los permisos necesarios.
adRecSchemaViolation	&H20000	No se ha guardado el registro porque no cumple con la estructura de la base de datos.
adRecDBDeleted	&H40000	El registro ha sido borrado de la base de datos.



La propiedad **Status** solo proporciona valores que muestran el estado de las operaciones con los registros del Recordset, sin embargo, el Recordset dispone de la propiedad **State** que muestra el estado general del Recordset antes o después de haber sido abierto. Estos son los valores para la propiedad **State**:

Valor	Descripción
0-adStateClosed	El Recordset está cerrado.
1-adStateOpen	El Recordset está abierto.
2-adStateConnecting	El Recordset está conectado.
4-adStateExecuting	El Recordset está ejecutando un mandato.
5-adStateFetching	Se está extrayendo las filas del Recordset.

- 8.4.2 Eventos del objeto Recordset

El objeto Recordset de ADO tiene un total de 11 eventos o sucesos. Estos sucesos le permitirán tener un control total de lo que está sucediendo internamente. Al codificar estos eventos, podrá potenciar las consultas asíncronas, atrapar el momento en el que se modifica un campo o un registro, verificar si se ha movido de un registro a otro e, incluso, agregar datos cuando el usuario alcance el final del Recordset. Para poder apreciar mejor los sucesos del Recordset lo he clasificado en tres grandes grupos: *Sucesos de recuperación de datos*, *Sucesos de navegación* y *Sucesos de modificación de datos*.

- 8.4.2.1 Sucesos de recuperación de datos

Estos sucesos ocurren durante una operación asíncrona cuando se están recuperando los datos. El suceso **FetchProgress** ocurre de forma constante durante una operación asíncrona de gran longitud. Podrá utilizarlo para mostrar al usuario una barra de progreso que indique el porcentaje de los registros que han sido recuperados. Este suceso tiene la siguiente estructura:

```
Private Sub rs_FetchProgress (ByVal Progress As Long, ByVal MaxProgress As Long, _
    adStatus As ADODB.EventStatusEnum, ByVal pRecordset As ADODB.Recordset)
```

'Aquí escriba su código...

```
End Sub
```

El parámetro *Progress* contiene el número de registros que se han extraído hasta el momento. *MaxProgress* es el número total de registros que se esperan recuperar. *adStatus* es el parámetro de estado. *pRecordset* es una referencia al objeto Recordset que ha provocado el suceso (este parámetro no tendrá que utilizarlo porque ya cuenta con una referencia al Recordset).

Cuando la operación asíncrona concluya, se pondrá en marcha el suceso **FetchComplete**. Este suceso tiene la siguiente estructura:

```
Private Sub rs_FetchComplete (ByVal pError As ADODB.error, _
    adStatus As ADODB.EventStatusEnum, ByVal pRecordset As ADODB.Recordset)
```

'Aquí escriba su código.

```
End Sub
```

Utilice el parámetro *pError* para consultar si ha ocurrido algún error durante la transacción.



- 8.4.2.2 Sucesos de navegación

Estos sucesos ocurren cuando se desplace de un registro ha otro o cuando modifique el registro activo. El suceso **WillMove** ocurre antes que ocurra una acción que provoque la modificación del registro activo. Este suceso tiene la siguiente estructura:

```
Private Sub rs_WillMove (ByVal adReason As ADODB.EventReasonEnum, _
    adStatus As ADODB.EventStatusEnum, ByVal pRecordset As ADODB.Recordset)
```

End Sub

El parámetro *adReason* proporciona el motivo por el que se ha puesto en marcha este suceso. Puede ser cualquiera de los valores que se muestran en la siguiente tabla:

Valor	Constante	Valor	Constante
1	adRsnAddNew	9	adRsnClose
2	adRsnDelete	10	adRsnMove
3	adRsnUpdate	11	adRsnFirstChange
4	adRsnUndoUpdate	12	adRsnMoveFirst
5	adRsnUndoAddNew	13	adRsnMoveNext
6	adRsnUndoDelete	14	adRsnMovePrevious
7	adRsnRequery	15	adRsnMoveLast
8	adRsnResynch		

Después de ocurrir el suceso **WillMove** le sigue el suceso **MoveComplete**. Este ocurre cuando la transacción ha sido completada o si ha sido cancelada. Este suceso tiene la siguiente estructura:

```
Private Sub rs_MoveComplete (ByVal adReason As ADODB.EventReasonEnum, _
    ByVal pError As ADODB.error, adStatus As ADODB.EventStatusEnum, _
    ByVal pRecordset As ADODB.Recordset)
```

End Sub

Los parámetros *adReason* y *adStatus* tienen el mismo significado en ambos sucesos. Si el parámetro *adStatus* devuelve el valor *adStatusErrorOccurred*, el objeto *pError* contendrá información sobre el error ocurrido, de lo contrario *pError* es *Nothing*.

Otro suceso de navegación que debe conocer es el suceso **EndOfRecordset**. Este suceso ocurre cuando el puntero de registro activo se coloca después del último registro del Recordset. Este suceso puede ocurrir como resultado de un método **MoveNext** al colocarse sobre un registro que no existe. Este suceso tiene la siguiente estructura:

```
Private Sub rs_EndOfRecordset (fMoreData As Boolean, adStatus As ADODB.EventStatusEnum, _
    ByVal pRecordset As ADODB.Recordset)
```

End Sub

Si desea aprovechar este evento, podrá agregar nuevos registros cuando este suceda. Para esto tendrá que ejecutar el método **AddNew** y rellenar con datos la colección *Fields* y, a continuación, definir el parámetro *fMoreData* como *True* para permitir que ADO sepa que ha añadido nuevos registros.



- 8.4.2.3 Sucesos de modificación de datos

Estos eventos ocurren cuando se modifica el valor de uno o varios campos de un registro o cuando se modifica uno o varios registros del Recordset. Cuando ejecute uno o más campos de un Recordset, se pondrá en marcha el suceso **WillChangeField**. Este suceso tiene la siguiente estructura:

```
Private Sub rs_WillChangeField(ByVal cFields As Long, ByVal Fields As Variant, _  
adStatus As ADODB.EventStatusEnum, ByVal pRecordset As ADODB.Recordset)
```

End Sub

cFields es el número de campos que se van a modificar. *Fields* es un array de tipo Variant que contiene uno o más objetos *Field* con cambios pendientes. Podrá cancelar la operación de actualizar los registros estableciendo el valor *adStatusCancel* en el parámetro *adStatus*.

Cuando la operación de actualización haya concluido, ADO pondrá en marcha el suceso **FieldChangeComplete**. Este le permitirá investigar cualquier error que haya ocurrido durante la operación. Este suceso tiene los mismos parámetros que el evento anterior más el parámetro *pError*.

```
Private Sub rs_FieldChangeComplete(ByVal cFields As Long, ByVal Fields As Variant, _  
ByVal pError As ADODB.error, adStatus As ADODB.EventStatusEnum, _  
pRecordset As ADODB.Recordset)
```

End Sub

Cuando se haya modificado uno o más registros, ADO ejecuta el suceso **WillChangeRecord**:

```
Private Sub rs_WillChangeRecord(ByVal adReason As ADODB.EventReasonEnum, _  
ByVal cRecords As Long, adStatus As ADODB.EventStatusEnum, _  
ByVal pRecordset As ADODB.Recordset)
```

End Sub

El parámetro *adReason* tiene el mismo resultado que en los eventos de navegación **WillMove** y **MoveComplete**. *cRecords* es el número de registros que se van a modificar. *adStatus* es el parámetro que podrá utilizar para cancelar la operación asignando el valor *adStatusCancel*.

Inmediatamente ADO termine la actualización de los registros se pondrá en marcha el suceso **RecordChangeComplete**:

```
Private Sub rs_RecordChangeComplete(ByVal adReason As ADODB.EventReasonEnum, _  
ByVal cRecords As Long, ByVal pError As ADODB.error, adStatus As ADODB.EventStatusEnum, _  
ByVal pRecordset As ADODB.Recordset)
```

End Sub

Todos los parámetros del suceso anterior tienen el mismo significado que en el caso del suceso **WillChangeRecord**.

Cuando ejecute una operación que vaya a modificar el contenido de un Recordset como un todo (tal como los métodos *Open*, *Requerí* y *Resync*) se pondrá en marcha un suceso **WillChangeRecordset** y cuando haya completado la operación un suceso **RecordsetChangeComplete**. La estructura de estos sucesos se muestra en la siguiente página...



```
Private Sub rs_WillChangeRecordset(ByVal adReason As ADODB.EventReasonEnum, _  
    adStatus As ADODB.EventStatusEnum, ByVal pRecordset As ADODB.Recordset)
```

```
End Sub
```

El significado de los parámetros es similar a los vistos en los sucesos anteriores.

```
Private Sub rs_RecordsetChangeComplete(ByVal adReason As ADODB.EventReasonEnum, _  
    ByVal pError As ADODB.error, adStatus As ADODB.EventStatusEnum, _  
    ByVal pRecordset As ADODB.Recordset)
```

```
End Sub
```

- 8.4.3 Generación de reportes

La *generación de reporte* es una de las funciones más relevantes que puede tener un sistema que manipule bases de datos. Esto consiste en presentar al usuario un conjunto de información de forma organizada, ya sea por resultado de una consulta personalizada o una consulta ya definida internamente en el programa. Ejemplos de consultas podrían ser: mostrar al usuario todas las ventas realizadas en la empresa en una fecha determinada, mostrar todos los empleados mayores de treinta años, mostrar todos los empleados con sueldos mayores de cinco mil pesos, etc.

Lo primero que se debe tener en cuenta antes de generar un reporte es el lugar donde se va a mostrar dicho reporte, esto puede ser en cualquier control que contenga rejillas como un Grid o un ListView. Después que los datos están cargados en un Recordset es muy sencillo mostrarlos en un Grid o un ListView. El siguiente ejercicio le enseñara a mostrar todos los registros que contiene nuestra tabla agenda en un control ListView.

- Abra un nuevo proyecto.
- Haga clic derecho en la barra de controles y seleccione la opción *Components*. En la ventana de Componentes localice el control *Microsoft Windows Common Controls 6.0 (SP6)* y actívelo. Luego, haga clic en el botón *Aceptar*.
- Haga clic en el menú **Project (Proyecto)** y ejecute la opción **References (Referencias)**. En la ventana Referencia localice el elemento *Microsoft ActiveX Data Objects 2.0 Library* o una superior. Para cerrar la ventana haga clic en el botón *Aceptar (OK)*.
- Inserte en el formulario un control ListView y cuatro botones de comando.
- Seleccione el control ListView y busque la propiedad View en la *Ventana de propiedades*. Establezca el valor 3-lvwReport en esta propiedad.
- En la propiedad Caption del primer botón de comando escriba: Reporte 1.
- En la propiedad Caption del segundo botón de comando escriba: Reporte 2.
- En la propiedad Caption del tercer botón de comando escriba: Reporte 3.
- En la propiedad Caption del cuarto botón de comando escriba: &Salir.



- En el evento **Load** del formulario escriba:

```
ListView1.GridLines = True      'Hacemos que aparezcan las líneas del Grid.
```

```
'Agregamos los campos al ListView.
```

```
ListView1.ColumnHeaders.Add , , "Nombre", 1600
```

```
ListView1.ColumnHeaders.Add , , "Apellido", 1600
```

```
ListView1.ColumnHeaders.Add , , "Teléfono", 1400
```

```
ListView1.ColumnHeaders.Add , , "Dirección", 3000
```

```
ListView1.ColumnHeaders.Add , , "Correo", 1400
```

```
'Activamos los Recordset.
```

```
Set rs = New ADODB.Recordset
```

```
Set rs2 = New ADODB.Recordset
```

```
Set rs3 = New ADODB.Recordset
```

```
'Abrimos la base de datos "agenda.mdb".
```

```
cn.Open "Provider=Microsoft.Jet.OLEDB.4.0;" & "Data Source=c:\agenda.mdb"
```

```
rs.Source = "contactos"      'Este Recordset es para el primer reporte.
```

```
rs.CursorType = adOpenKeyset
```

```
rs.LockType = adLockOptimistic
```

```
rs2.Source = "contactos"
```

```
'Este Recordset es para el segundo reporte.
```

```
rs2.CursorType = adOpenKeyset
```

```
rs2.LockType = adLockOptimistic
```

```
rs3.Source = "contactos"
```

```
'Este Recordset es para el tercer reporte.
```

```
rs3.CursorType = adOpenKeyset
```

```
rs3.LockType = adLockOptimistic
```



- En el evento **Click** del botón *Reporte 1* escriba:

```
'Limpiamos el ListView en caso de que este tenga información.
ListView1.ListItems.Clear

'En esta primera consulta leemos todos los registros de la base de datos.
If rs.State = 0 Then
    rs.Open "select * from contactos", cn
End If

'Agregamos los datos al ListView
rs.MoveFirst      'Nos movemos al primer registro.

Dim li As ListItem
While rs.EOF = False
    'Ponemos los datos de los campos leídos en el ListView.
    Set li = ListView1.ListItems.Add(, , rs("nombre"))
    li.ListSubItems.Add , , rs("apellido")
    li.ListSubItems.Add , , rs("telefono")
    li.ListSubItems.Add , , rs("direccion")
    li.ListSubItems.Add , , rs("correo")
    rs.MoveNext
    'Pasamos al siguiente registro.
Wend
```

- En el evento **Click** del botón *Reporte 2* escriba:

```
'Limpiamos el ListView en caso de que este tenga información.
ListView1.ListItems.Clear

'En consulta leemos todos los campos nombre y apellido de la base de datos.
If rs2.State = 0 Then
    rs2.Open "select nombre, apellido from contactos", cn
End If

'Agregamos los datos al ListView.
rs2.MoveFirst      'Nos movemos al primer registro.
Dim li As ListItem
While rs2.EOF = False
    'Ponemos los datos de los campos leídos en el ListView.
    Set li = ListView1.ListItems.Add(, , rs2("nombre"))
    li.ListSubItems.Add , , rs2("apellido")
    rs2.MoveNext
    'Pasamos al siguiente registro.
Wend
```

- En el evento **Click** del botón *Reporte 3* escriba:

```
'Limpiamos el ListView en caso de que este tenga información.
ListView1.ListItems.Clear

'En esta consulta leemos los nombre de los contactos que se apelliden Bucarely.
If rs3.State = 0 Then
    rs3.Open "select * from contactos where apellido = 'Bucarely'", cn
End If

'Agregamos los datos al ListView
rs3.MoveFirst      'Nos movemos al primer registro.

Dim li As ListItem
```



```
While rs3.EOF = False
```

```
    'Ponemos los datos de los campos leídos en el ListView.
```

```
    Set li = ListView1.ListItems.Add(, , rs3("nombre"))
```

```
    li.ListSubItems.Add , , rs3("apellido")
```

```
    rs3.MoveNext          'Pasamos al siguiente registro.
```

```
Wend
```

- En el evento **Click** del botón *Salir* escriba:

```
'Sale de la aplicación.
```

```
End
```

- Corra la aplicación.

Haga clic sobre cada uno de los botones de Reporte. Podrá observar que cada botón de comando extrae de la base de datos solo la información que hemos establecido en cada una de las consultas. En el primer botón de reporte especificamos una consulta que lee todos los registros de la base de datos, en el segundo botón de reporte especificamos una consulta que extrae únicamente los nombres y apellidos de la base de datos omitiendo los demás campos (dirección, teléfono y correo). En el último botón de reporte especificamos una consulta personalizada que extrae todos los nombres de las personas que tengan por apellido Bucarely.

- Detenga la aplicación y guárdela con los nombres **FormEjercicio3-9** para el *formulario* y **Ejercicio3-9** para el proyecto.

8.5 EJERCICIOS PROPUESTOS

- 1.- Hacer una aplicación que permite registrar en una base de datos los datos de los empleados de una empresa. Los campos son: Nombre, Apellido, Edad, Cargo, Sueldo, Dirección, Teléfono y Fecha de contrato. El programa debe permitir al usuario obtener distintos tipos de reporte, como por ejemplo: los empleados que ganen más de 10,000 pesos, los empleados mayores de 50 años de edad, etc.
- 2.- Hacer una aplicación que permita gestionar un almacén de un supermercado.
- 3.- Hacer una aplicación que permita gestionar un sistema de inscripción de una universidad o un colegio.
- 4.- Crear un pequeño punto de venta. Esto es un sistema que permita vender artículos, imprimir facturas y generar reportes de ventas.
- 5.- Crear una pequeña aplicación que permita gestionar el almacén de una farmacia.



CONTENIDO

- 9.1 Abrir la ventana de quitar o agregar programas de Windows.
- 9.2 Obtener el directorio desde donde estemos ejecutando nuestro programa.
- 9.3 Verificar si un archivo existe o no.
- 9.4 Capturar la pantalla entera o la ventana activa.
- 9.5 Desplegar la lista de un ComboBox automáticamente.
- 9.6 Cambiar el fondo de Windows.
- 9.7 Comprobar si el sistema posee una tarjeta de sonido.
- 9.8 Apagar el equipo, reiniciar Windows y reiniciar el sistema.
- 9.9 Situar un ScrollBar horizontal en una ListBox.
- 9.10 Centrar una ventana.
- 9.11 Obtener el directorio de Windows y el directorio de Sistema.
- 9.12 Crear un efecto Shade al estilo de los sistemas de instalación.
- 9.13 Hacer sonar un fichero WAV o una secuencia MIDI.
- 9.14 Ocultar y mostrar el cursor.



En el presente apartado he decido colocar una pequeña colección de trucos recogidos de la red. Estos códigos diseñados por aficionados a la programación visual le serán de mucha utilidad en sus aplicaciones, y lo mejor de todo es, que están disponibles gratuitamente para ser usados cada vez que los necesite.

9.1 Abrir la ventana de quitar o agregar programas de Windows

Este código le permitirá abrir la ventana *Agregar o quitar programas* de Windows. En un botón de comando escriba:

```
Dim X
X = Shell("Rundll32.exe shell32.dll,Control_RunDLL appwiz.cpl @0")
```

9.2 Obtener el directorio desde donde estemos ejecutando nuestro programa

Este código es muy utilizado para leer el directorio o path desde donde se esta ejecutando la aplicación:

```
Dim Directorio As String      'Esta variable almacenará el directorio actual.
ChDir App.Path
ChDrive App.Path
Directorio = App.Path

If Len(Directorio) > 3 Then
    Directorio = Directorio & "\"
End If
```

9.3 Verificar si un archivo existe o no

Este código permite verificar si existe o no un archivo en una unidad de disco:

```
'Escriba este código en cualquier parte que quiera utilizarlo.
On Error GoTo Fallo

x = GetAttr("C:\Autoexec.bat") 'Aquí se escribe el fichero.
MsgBox ("El fichero existe.")

Exit Sub
Fallo:
MsgBox ("El fichero no existe.")
```

9.4 Capturar la pantalla entera o la ventana activa

Este código le permitirá copiar al portapales de Windows la imagen de la pantalla completa o la ventana activa.

- En la sección general del editor de código escriba:

```
Private Declare Sub keybd_event Lib "user32" (ByVal bVk As Byte, _
ByVal bScan As Byte, ByVal dwFlags As Long, _
ByVal dwExtraInfo As Long)
```

- En un botón de comando escriba:

```
'Captura la ventana activa.
keybd_event 44, 0, 0&, 0&
```



- En otro botón de comando escriba:

```
'Captura toda la pantalla completa.  
keybd_event 44, 1, 0&, 0&
```

9.5 Desplegar la lista de un ComboBox automáticamente

El siguiente código le permitirá desplegar la lista de un ComboBox al hacer clic sobre un botón de comando:

- En la sección general del editor de código escriba:

```
Private Declare Function SendMessageLong Lib "user32" Alias _  
"SendMessageA" (ByVal hwnd As Long, ByVal wParam As Long, _  
ByVal lParam As Long, ByVal IParam As Long) As Long
```

- En el evento **Load** del formulario escriba:

```
'Agregamos elementos al ComboBox.  
Combo1.Clear  
Combo1.AddItem "Objeto 1"  
Combo1.AddItem "Objeto 2"  
Combo1.AddItem "Objeto 3"  
Combo1.AddItem "Objeto 4"  
Combo1.AddItem "Objeto 5"  
Combo1.AddItem "Objeto 6"  
Combo1.AddItem "Objeto 7"  
Combo1.Text = "Objeto 1"
```

- En un botón de comando escriba:

```
'Hacemos que la lista se despliegue.  
Dim Resp As Long  
Resp = SendMessageLong(Combo1.hwnd, &H14F, True, 0)
```

9.6 Cambiar el fondo de Windows

Podrá cambiar el fondo del escritorio de Windows mediante el siguiente código:

- En la sección general escriba:

```
Private Declare Function SystemParametersInfo Lib "user32" Alias _  
"SystemParametersInfoA" (ByVal uAction As Long, ByVal uParam As _  
Long, ByVal lpvParam As Any, ByVal fuWinIni As Long) As Long
```

- En un botón de comando:

```
Dim fallo As Integer  
fallo = SystemParametersInfo(20, 0, "C:\WINDOWS\FONDO.BMP", 0)
```

9.7 Comprobar si el sistema posee una tarjeta de sonido

El siguiente código le permitirá verificar si una computadora tiene o no instalada una tarjeta de sonido:



- En la sección general escriba:

```
Private Declare Function waveOutGetNumDevs Lib "winmm.dll" () As Long
```

- En un botón de comando escriba:

```
Dim inf As Integer
inf = waveOutGetNumDevs()

If inf > 0 Then
    MsgBox "Tarjeta de sonido soportada.", vbInformation, "Informacion: Tarjeta de sonido"
Else
    MsgBox "Tarjeta de sonido no soportada.", vbInformation, "Informacion: Tarjeta de sonido"
End If
```

9.8 Apagar el equipo, reiniciar Windows y reiniciar el sistema

Con el siguiente código podrá apagar y reiniciar el sistema. También podrá cerrar la sesión en Windows.

- En la sección general escriba:

```
Private Declare Function ExitWindowsEx Lib "user32" (ByVal _
    uFlags&, ByVal dwReserved&)
```

- En un botón de comando escriba:

```
Dim i As Integer
i = ExitWindowsEx(1, 0&) 'Apaga el equipo.
```

- En un segundo botón de comando escriba:

```
Dim i As Integer
i = ExitWindowsEx(0, 0&) 'Reinicia Windows con nuevo usuario.
```

- En un tercer botón de comando escriba:

```
Dim i As Integer
i = ExitWindowsEx(2, 0&) 'Reinicia el Sistema.
```

9.9 Situar un ScrollBar horizontal en una ListBox

Los ListBox no poseen una barra de desplazamiento horizontal, podrá agregar una mediante el siguiente código:

- En la sección general escriba:

```
Private Declare Function ExitWindowsEx Lib "user32" (ByVal _
    uFlags&, ByVal dwReserved&)
```

- En el evento **Load** del formulario escriba:

```
Dim x As Integer, i As Integer
For i = 1 To 20
    List1.AddItem "El número final de la selección es el " & i
Next i
x = SendMessage(List1.hwnd, &H194, 200, ByVal 0&)
```



9.10 Centrar una ventana

- En el evento **Load** del formulario escriba:

```
Move (Screen.Width - Width) \ 2, (Screen.Height - Height) \ 2
```

9.11 Obtener el directorio de Windows y el directorio de Sistema

- En la sección general escriba:

```
Declare Function GetSystemDirectory Lib "kernel32" Alias "GetSystemDirectoryA" _  
(ByVal lpBuffer As String, ByVal nSize As Long) As Long  
Declare Function GetWindowsDirectory Lib "kernel32" Alias _  
"GetWindowsDirectoryA" (ByVal lpBuffer As String, ByVal nSize As Long) As Long
```

- Inserte dos etiquetas y un botón de comando en el formulario.

- En el evento **Click** del botón de comando escriba:

```
Dim Car As String * 128  
Dim Longitud, Es As Integer  
Dim Camino As String  
  
Longitud = 128  
  
Es = GetWindowsDirectory(Car, Longitud)  
Camino = RTrim$(LCase$(Left$(Car, Es)))  
Label1.Caption = Camino  
  
Es = GetSystemDirectory(Car, Longitud)  
Camino = RTrim$(LCase$(Left$(Car, Es)))  
Label2.Caption = Camino
```

9.12 Crear un efecto Shade al estilo de los sistemas de instalación

- En el evento **Load** del formulario escriba:

```
Dim i As Long  
Dim y As Long  
Form1.Cls  
Form1.AutoRedraw = True  
Form1.DrawStyle = 6  
Form1.DrawMode = 13  
Form1.DrawWidth = 2  
Form1.ScaleMode = 3  
Form1.ScaleHeight = (256 * 2)  
  
For i = 0 To 255  
    Form1.Line (0, y)-(Form1.Width, y + 2), RGB(0, 0, i), BF  
    y = y + 2  
Next i
```



9.13 Hacer sonar un fichero WAV o una secuencia MIDI

- En un módulo escriba:

```
Declare Function mciExecute Lib "winmm.dll" (ByVal lpstrCommand As String) As Long
```

- En un botón de comando escriba:

```
Dim Sonido  
Sonido = mciExecute("Play c:\windows\ringin.wav")
```

9.14 Ocultar y mostrar el cursor

- En un módulo escriba:

```
Declare Function ShowCursor Lib "user32" (ByVal bShow As Long) As Long
```

- Agregue dos botones de comando al formulario. En el primer botón escriba:

```
Dim result  
result = ShowCursor(False)
```

- En el segundo botón de comando escriba:

```
Dim result  
result = ShowCursor(True)
```



TABLA DE VALORES ASCII

Códigos ASCII normales (códigos 0 - 127)

000	(nul)	016	► (dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001	☉ (soh)	017	◄ (dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002	⊙ (stx)	018	↑ (dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003	♥ (etx)	019	!! (dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004	♦ (eot)	020	¶ (dc4)	036	\$	052	4	068	D	084	T	100	d	116	t
005	♣ (enq)	021	§ (nak)	037	%	053	5	069	E	085	U	101	e	117	u
006	♠ (ack)	022	— (syn)	038	&	054	6	070	F	086	V	102	f	118	v
007	• (bel)	023	‡ (etb)	039	'	055	7	071	G	087	W	103	g	119	w
008	▣ (bs)	024	↑ (can)	040	(	056	8	072	H	088	X	104	h	120	x
009	(tab)	025	↓ (em)	041	)	057	9	073	I	089	Y	105	i	121	y
010	(lf)	026	(eof)	042	*	058	:	074	J	090	Z	106	j	122	z
011	♂ (vt)	027	← (esc)	043	+	059	;	075	K	091	[	107	k	123	{
012	♀ (np)	028	L (fs)	044	,	060	<	076	L	092	\	108	l	124	
013	(cr)	029	↔ (gs)	045	-	061	=	077	M	093	]	109	m	125	}
014	♫ (so)	030	▲ (rs)	046	.	062	>	078	N	094	^	110	n	126	~
015	✱ (si)	031	▼ (us)	047	/	063	?	079	O	095	_	111	o	127	△

Códigos ASCII extendidos (códigos 128 - 255)

128	Ç	143	Å	158	×	172	¼	186		200	ℒ	214	Í	228	ō	242	̄
129	ù	144	É	159	f	173	;	187	]]	201	ℓ	215	Î	229	õ	243	̅
130	é	145	æ	160	á	174	«	188	]]	202	ℓ	216	Ï	230	μ	244	¶
131	â	146	Æ	161	í	175	»	189	ç	203	ℓ	217	Ĵ	231	þ	245	§
132	ä	147	ô	162	ó	176	⋮	190	¥	204	ℓ	218	ℓ	232	þ	246	÷
133	à	148	ö	163	ú	177	⋮	191	ℓ	205	=	219	ℓ	233	ú	247	˙
134	â	149	ò	164	ñ	178	⋮	192	ℓ	206	ℓ	220	ℓ	234	Û	248	°
135	ç	150	û	165	Ñ	179	⋮	193	ℓ	207	ℓ	221	ℓ	235	Û	249	˝
136	ê	151	ù	166	ª	180	⋮	194	ℓ	208	ð	222	ℓ	236	ý	250	•
137	ë	152	ÿ	167	º	181	À	195	ℓ	209	Ð	223	ℓ	237	Ý	251	¹
138	è	153	Ö	168	¿	182	Â	196	—	210	Ê	224	Ó	238	-	252	³
139	ï	154	Û	169	®	183	Ä	197	†	211	Ë	225	ß	239	´	253	²
140	î	155	ø	170	¬	184	©	198	ã	212	È	226	Ô	240		254	■
141	ì	156	£	171	½	185	¶	199	Ã	213	Ì	227	Ò	241	±	255	



CONTROLES NO TRABAJADOS

- **Shape:** Es un control gráfico que se muestra como un rectángulo, un cuadrado, una elipse, un círculo, un rectángulo redondeado o un cuadrado redondeado.
- **RichTextBox:** Es un control que permite al usuario escribir y modificar texto al tiempo que proporciona características de formato más avanzadas que el control **TextBox** convencional.
- **DataGrid (Control):** Muestra y permite la manipulación de datos de una serie de filas y columnas que corresponden a registros y campos de un objeto **Recordset**.
- **MSFlexGrid:** El control Microsoft FlexGrid (**MSFlexGrid**) muestra datos de tablas y efectúa operaciones en ellos. Proporciona una flexibilidad completa para ordenar, combinar y aplicar formato a tablas que contienen cadenas e imágenes. Cuando se enlaza a un control **Data**, el control **MSFlexGrid** muestra datos de sólo lectura.
- **Winsock:** El control **Winsock**, invisible para el usuario, proporciona un acceso sencillo a los servicios de red TCP y UDP. Pueden usarlo los programadores de Microsoft Access, Visual Basic, Visual C++ o Visual FoxPro. Para escribir aplicaciones de servidor o de cliente no necesita comprender los detalles de TCP ni llamar a las API de **Winsock** de nivel inferior. Si establece las propiedades y llama a los métodos del control, podrá conectar fácilmente con un equipo remoto e intercambiar datos en las dos direcciones.
- **Control Calendar de Microsoft:** Es un control que permite introducir en una aplicación un calendario personalizado.
- **Control Animation:** Puede reproducir archivos AVI para que pueda añadir animaciones sencillas a su programa. Este control es compatible con archivos AVI que no contengan sonido y que no se encuentren en formato comprimido o que hayan sido comprimidos utilizando la tecnología Run-Length Encoding (RLE).
- **Control UpDown:** El control UpDown ofrece una forma sencilla pero eficaz de crear esos botones de incremento/decremento que muchas aplicaciones para Windows muestran a la derecha de los campos numéricos y que permiten al usuario incrementar o decrementar el valor contenido en el campo sin más que pulsar con el Mouse sobre el control.
- **Control FlatsScrollBar:** Es un sustituto de los controles **HScrollBar** y **VScrollBar**. Este posee distintos tipos de estilos gráficos que podrá cambiar en tiempo de diseño mediante su propiedad **Appearance**.
- **Control Monthview y DateTimePicker:** El primero es un control estilo calendario y el segundo es un cuadro de texto en el que podrá introducir fechas y horas. Los dos están estrechamente relacionados, ya que el control **DateTimePicker** utiliza un control **MonthView** cuando el usuario despliega un calendario para seleccionar una fecha.



► MSDN Library Visual Studio 6.0a ◀

► **Aprenda Visual Basic 6.0 (Como si estuviera en primero)** ◀
Javier García de Jalón * José Ignacio Rodríguez * Alfonso Brazález

► **Capítulo 7 (Los archivos). *Documento Electrónico*** ◀
Carlos Castillo Peralta

Nombre de archivo: libro de oro vb corregido
Directorio: D:
Plantilla: C:\Documents and Settings\Astro\Datos de
programa\Microsoft\Plantillas\Normal.dot
Título:
Asunto:
Autor: Twinsmaster
Palabras clave:
Comentarios:
Fecha de creación: 06/07/2008 12:52:00
Cambio número: 26
Guardado el: 06/07/2008 22:32:00
Guardado por: Astro
Tiempo de edición: 79 minutos
Impreso el: 06/07/2008 22:39:00
Última impresión completa
Número de páginas: 455
Número de palabras: 4.159 (aprox.)
Número de caracteres: 22.878 (aprox.)